

低延时分布式开发平台使用指南

1. 文档说明

LDP是专门为高性能应用设计的消息传输总线和应用开发平台，用户可在LDP平台上方便快速地进行多种方式的应用开发。为了方便用户更好地了解和使用LDP平台，特编写此文档。

- V1.0.0.0 2020-06-29 创建初始版本

1.1. 预期读者

本文档主要适用于以下人员：

- 工程实施人员
- 维护人员
- 交付人员

1.2. 手册概况

本手册各章节的内容如下：

一、关于手册	本章主要介绍本手册的概况
二、平台概述	本章主要介绍分布式低延时开发平台，运行环境要求，以及安装、配置说明
三、应用场景	本章主要介绍分布式低延时开发平台的应用场景
四、插件说明	本章主要介绍分布式低延时开发平台的插件的配置及使用说明
五、运维管控平台	本章主要介绍运维管控平台的安装、部署和监控
六、管理功能	本章主要是介绍各个插件的管理功能说明
七、常见问题	本章主要介绍分布式低延时开发平台常见问题和解决方法

1.3. 释义和缩略语

缩写	英文全称	释义
LDP	Low latency Distributed development Platform	低延时分布式开发平台
RCM	Reliable and Consistent Multicast	可靠一致组播

2. 平台概述

本章主要介绍分布式低延时开发平台，运行环境要求，以及安装、配置说明。

2.1. LDP 简介

LDP是专门为高性能应用设计的消息传输总线和应用开发平台：

- 在消息传输上拥有低延时、高吞吐、大容量等特点。
- 作为消息总线拥有分布式、高可靠、易扩展等特点。
- 支持多种应用开发方式：插件开发、业务组件开发、基于消息总线开发等。
- LDP平台对外提供基于主题和分区的信息传输机制，方便用户理解和使用。
- 支持主备和多活应用场景。

2.2. 软件安装

2.2.1. 运行环境

环境项	推荐
操作系统	64位Linux(RedHat Enterprise Linux 7.2以上)
内存	128G
硬盘	500G
CPU	3.0G以上主频
网卡	万兆网卡
交换机	1Gb及以上且支持组播的以太网交换机，支持IGMP V2及以上

2.2.2. 程序安装

安装包获取

LDP平台安装包可从恒生效能平台下的技术平台制品中通过搜索LDP获取

下载地址：<https://dev.hundsun.com/frameV2/cicd/artifactOriginRDC>

安装包所含文件说明

LDP平台完整发布包结构如下：

-----workspace /	运行目录，包括示例配置文件、启动停止脚本
-----start.sh	启动脚本
-----stop.sh	停止脚本
-----ldp_ha_local.json	本地配置下的主备模式示例配置
-----ldp_aa_local.json	本地配置下的双活模式示例配置
-----ldp_ha.json	zk集中配置下的主备模式示例配置
-----ldp_aa.json	zk集中配置下的双活模式示例配置
-----mount_nfs.sh	nfs挂载示例脚本
-----umount_nfs.sh	nfs卸载示例脚本
-----bin/	可执行程序目录
-----plugins/	插件目录
-----components /	公用库目录
-----xst.local /	示例数据组件目录，xst表示示例应用名
-----modules /	示例功能组件目录
-----messages/	平台日志信息描述目录
-----ldp.zh_CN	错误信息描述文件
-----message.level	错误号等级描述文件，支持用户自主命名，并加载当前目录下所有的.level结尾的文件。注：需要保证不同文件中相同错误号的级别应该一致，否则存在相互覆盖情况。
-----bizmessage/	业务日志信息描述目录
-----ldp.zh_CN	错误信息描述文件
-----message.level	错误号等级描述文件，支持用户自主命名，并加载当前目录下所有的.level结尾的文件。注：需要保证不同文件中相同错误号的级别应该一致，否则存在相互覆盖情况。
-----hsl dp	主程序
-----dev/	开发包目录，包括插件以及组件开发头文件和示例代码

安装步骤

LDP平台目前只支持linux.x64平台，操作系统版本要求Redhat 7.2以上。通过直接拷贝安装包，并按照规定设置系统环境变量，即可完成安装。具体安装步骤如下：

1. 拷贝安装文件至用户指定目录

将LDP平台安装包中的workspace和bin两个文件夹拷贝到指定目录下，安装完成后的目录结构如下：

-----BaseDir /	选择的安装根目录
-----workspace /	运行目录，包括配置文件、启动停止脚本
-----log /	日志目录，运行以后自动生成
-----bin/	可执行程序目录

2. 设置系统环境变量

设置环境变量

LDP平台运行依赖HSLDP_HOME系统环境变量。需要将该环境变量指向到\$BaseDir/bin路径下：

export HSLDP_HOME=\$BaseDir/bin

将LDP主程序目录加入PATH系统环境变量中：

export PATH=\$HSLDP_HOME:\$PATH

- 若使用对应的启动脚本启动，脚本内已经包含上述环境变量设置语句，无需手动设置。

设置系统预留随机端口范围

随机预留端口：Linux系统默认会预留随机端口范围，为了防止启动应用配置中的端口被随机生成的端口给占用，导致应用无法启动。所以一般用户设置的端口要在系统预留随机生成端口范围外。Linux系统默认生成的预留随机端口范围（32768,65535）

获取临时预留随机端口的范围：sysctl net.ipv4.ip_local_port_range 或 cat /proc/sys/net/ipv4/ip_local_port_range

临时修改系统预留随机端口范围（必须为root用户）：echo 1024 65535 > /proc/sys/net/ipv4/ip_local_port_range

- 范围左值必须大于0，右值必须小于65536，左值必须小于右值，否则执行失败。
- 临时修改后的预留随机端口范围再系统重启后恢复默认设置。

永久修改系统预留随机端口范围（必须为root用户，系统重启以后不变）：

- 步骤1、在/etc/sysctl.conf文件中添加：net.ipv4.ip_local_port_range = 50000 65535
- 步骤2、生效内核参数：sysctl -p /etc/sysctl.conf

3. 根据当前需要定制配置文件

LDP平台的部署模式支持主备模式和双活模式。配置文件的读取模式支持本地配置模式和zk集中配置模式。不同模式对应的配置文件如下所示：

模式	配置文件
本地配置模式	主备模式
	ldp_ha_local.json
zk集中配置模式	双活模式
	ldp_aa_local.json
	主备模式
	ldp_ha.json
	双活模式
	ldp_aa.json

选择模式后，需要将对应的配置文件名修改为ldp.json.

- 如果使用本地配置模式，对应的json配置文件需放置于当前工作路径下，即\$BaseDir/workspace/。
- 如果使用zk集中配置模式，需要在zk上创建/ldp/xxx/config/yyy节点，值对应json配置文件的内容，其中xxx为产品名，yyy可为任意的节点名。

对于主备模式，预先需要先搭建nfs环境，可以参考《NFS安装》章节。
平台选择配置文件以及zk节点的规则参照《配置说明》章节。

4. 启动应用

对于zk集中配置模式，将start.sh脚本中的-s参数值修改为zk的地址，格式为ip:port，如果zk为集群，格式为ip1:port1,ip2:port2。然后运行对应的脚本启动应用。

5. 停止应用

运行stop.sh脚本停止应用

2.3. 安装示例

本章以本地配置下的主备模式LDP平台安装部署步骤作为示例。

1. 复制安装包

复制安装包至指定目录:

```
[ldp@localhost ldp]$ ls
bin  dev  workspace
```

2. 修改启动脚本

当前使用本地配置模式，因此删除start.sh脚本中的 -s 参数。

```
#!/bin/bash

cwd=`pwd`
export HSLDP_HOME=$cwd/./bin
export PATH=$HSLDP_HOME:$PATH
hsl dp startdaemon ldp xst#0
```

注1：默认情况下有可能\$BaseDir/bin/目录下的hsl dp文件无执行权限，需要附加执行权限。

```
[ldp@localhost ldp]$ cd bin/
[ldp@localhost bin]$ ll
total 3184
drwxrwxr-x 2 ldp ldp 4096 Aug 21 13:26 bizmessage
drwxrwxr-x 2 ldp ldp 4096 Aug 21 13:26 components
-rw-rw-r-- 1 ldp ldp 3236342 Aug 21 13:26 hsl dp
drwxrwxr-x 2 ldp ldp 4096 Aug 21 13:26 message
drwxrwxr-x 2 ldp ldp 4096 Aug 21 13:26 plugins
drwxrwxr-x 3 ldp ldp 4096 Aug 21 13:26 xst.local
```

```
[ldp@localhost bin]$ chmod 775 hsl dp
```

注2：如果要修改应用名，即示例中的xst，例如要修改为ex_xst，需要一并修改：

- 对应配置文件中的应用名"xst#0"，改为"ex_xst#0"
- 目录\$BaseDir/bin/下的目录xst.local，改为ex_xst.local
- 启动脚本start.sh与停止脚本stop.sh中的应用名xst#0，改为ex_xst#0

3. 启动应用

执行启动脚本。

```
[ldp@localhost workspace]$ sh start.sh
hsl dp V1.0.0.4 (Jul 31 2020)
[ldp@localhost workspace]$ CXstDataImpl::Init()
*****GetBizProcInfo begin *****
ConcurrentMode = 1
ThredCount = 2
BizDataCount = 1
ArbStatus = BIZ_ACTIVE
HaMode = ActiveStandby
IsLocalMode = true
(uQueueAlias = 1) [0,0]
(uQueueAlias = 2) [1,1]
*****GetBizProcInfo end *****
BizConfig BizName = xst
xstfunc Init()
CXstDataImpl::Start()
CXstDataImpl::ThreadInit() 0
CXstDataImpl::ThreadInit() 1
实例 ldp-ex_xst#0 启动完成。执行“hsl dp stop ldp ex_xst#0”停止
```

启动完成。

4. 停止应用

执行stop.sh脚本。

```
[ldp@localhost workspace]$ sh stop.sh
hsl dp V1.0.0.4 (Jul 31 2020)
停止信号已发送进程 11905
[ldp@localhost workspace]$ CXstDataImpl::Stop()
CXstDataImpl::ThreadExit() 0
CXstDataImpl::ThreadExit() 1
实例 ldp-ex_xst#0 停止完成
CXstDataImpl::Exit()
xstfunc Exit()
```

停止完成。

2.4. 系统配置

2.4.1. 相关概念

应用

基于hsl dp开发，包含公共库、平台插件、自定义插件等模块的程序。

应用编号

集群部署时，一般为集群成员编号；非集群时，一般为0。

插件

基于hslbp应用框架开发的功能模块，可以被应用框架加载、初始化、启动、停止等。

实例

多个插件的Lib配置项为同一个动态库时，每个插件称为一个实例，多个实例需要有不同的编号，称为实例编号，实例编号配置标签为Instance。

引用

引用提供一个应用实例的蓝本，这样基于引用得到的应用实例只需基于引用进行局部调整，就可得到预期的配置内容。

派生

派生基于指定的引用，且指定了需要调整的内容。这样综合引用与派生就可自动构建出最终的应用实例配置。

2.4.2. 模板引用

配置文件格式

配置文件为json格式，以下以一个具体的示例，说明模板引用功能：

```
{
  "REFS":{
    "front":{
      {
        "Components": {
          "Libs": [],
          "GlobalLibs": [
            "tcp"
          ]
        },
        "Plugins": [
          {
            "Lib": "ldp_front",
            "Instance": 0,
            "Timer": 1,
            "Wan": {
              "Port": 23150
            },
            "Lan": {
              "Port": 23151
            },
            "Routes": [
              {
                "Functions": "*",
                "Backend": 1
              }
            ],
            "Backends": [
              {
                "Id": 1,
                "Type": "AS-TCP",
                "Sites": [
                  {
                    "Host": "192.168.86.20",
                    "Port": 18111
                  }
                ]
              }
            ]
          }
        ],
        {
          "Lib": "ldp_mproxy",
          "Instance": 0,
          "Timer": 0,
          "Network": {
            "ListenPorts": "9010"
          },
          "Performance": {
            "EnableKeepAlive": true,
            "KeepAliveTimeoutMilli": 5000
          }
        },
        {
          "Lib": "ldp_logproxy",
          "Instance": 0,
          "Timer": 0,
          "BizLogLevel": 1
        }
      ]
    },
    ...
  },
  "Derived":{
    "front":{
      "front#0":{
        "ldp_front.Wan.Port":23160,
        "ldp_front.Backends.[0].Sites": [
          {
            "Host": "192.168.86.102",
            "Port": 8000
          }
        ]
      },
      "front#1": {
        "ldp_front.Backends.[0].Sites.[1]":
          {
            "Host": "192.168.86.101",
            "Port": 80001
          },
        "ldp_logproxy.LogLevel":1
      }
    },
    ...
  },
  "Applications": {
    ...
  }
}
```

配置文件最外层包含三个标签REFS, Derived, Applications。

REFS用于定义引用实例集合, 每个引用实例由引用名, 引用内容构成。引用内容将作为后续构建应用实例的蓝本。

上述示例中, 我们定义了一个名字为front的引用实例, 其内容可作为后续构建前置类型应用实例的蓝本。

Derived用于定义派生实例集合, 每个派生实例需要先确定要派生的引用实例。

每个派生实例由实例名, 派生内容构成。实例名将作为自动构建的应用实例的名字, 派生内容指明了在引用实例基础上的调整信息。

上述示例中, 我们定义了两个名字为front#0, front#1的派生实例。

对front#0, 它基于名为front的引用实例。

front#0的内容分别为引用实例中两个字段指定了新的值。

综合front#0, front的内容, 我们会自动构建一个应用实例front#0。front#0及其内容将自动插入到Applications下。

对front#1, 它基于名为front的引用实例。

front#1的内容分别为引用实例中两个父对象各自新增了一个新成员。

综合front#1, front的内容, 我们会自动构建一个应用实例front#1。front#1及其内容将自动插入到Applications下。

派生实例中替换内容

派生实例支持对引用实例的字段进行替换, 以便设置符合自己需要的新值。

以上述例子中front#0的ldp_front.Backends.[0].Sites为例说明替换语法。

- 1.通过.分隔的多个键来定位到被替换值的对象。
- 2.首个键必须是引用实例的Plugins数组中某个插件的名字。插件的名字放在插件对象的Lib字段里。
- 3.父对象是对象类型, 需要通过其子成员名字来定位子成员。如通过Backends来定位ldp_front插件对象名字为Backends的子成员。
- 4.父对象是数组类型, 需要通过[nIndex]指定子成员索引来定位子成员。如通过[0]来定位Backends的数组值对象中首个子成员。
- 5.设置的新值需要和老的值具备一致的类型。
 - 5.1.老的值是true或false类型, 新的值也得是true或false类型。
 - 5.2.老的值是数值类型, 新的值也得是数值类型。
 - 5.3.老的值是字符串, 新的值也得是字符串。
 - 5.4.老的值是数组, 新的值也得是数组。
 - 5.5.老的值是对象, 新的值也得是对象。

派生实例中新增内容

派生实例支持在引用实例基础上新增字段, 以便新增符合自己需要的新成员。

以上述例子中front#1的ldp_front.Backends.[0].Sites.[1]为例说明为数组新增子成员语法。

- 1.通过.分隔的多个键来定位新的子成员的父对象。
- 2.首个键必须是引用实例的Plugins数组中某个插件的名字。插件的名字放在插件对象的Lib字段里。
- 3.父对象是对象类型, 需要通过其子成员名字来定位子成员。如通过Backends来定位ldp_front插件对象名字为Backends的子成员。
- 4.父对象是数组类型, 需要通过[nIndex]指定子成员索引来定位子成员。如通过[0]来定位Backends的数组值对象中首个子成员。
- 5.为数组新增子成员时, 必须将新的子成员索引设置为数组最后元素下一元素的索引。如上述Sites代表的数组在引用实例中只有一个成员。故新成员的索引需是1。

以上述例子中front#1的"ldp_logproxy.LogLevel":1为例说明为对象新增子成员语法。

1~4和为数组新增子成员相同。

5.为对象新增子成员时, 必须同时提供新的子成员的名字, 值。如上述为ldp_logproxy代表的对象新增了名字为LogLevel值为1的新的子成员。

2.4.3. 应用配置

配置文件格式

hsl dp配置文件为json格式, 示例如下:

```
{
  "Applications": {
    "xst#0": {
      "TimerUnit" : 1000,
      "EnableRdtsc" : true,
      "Components": {
        "Libs": [],
        "GlobalLibs": [
          "ARM平台dbclient插件若使用则需添加unysql"
          "unysql"
          ...
        ]
      },
      "Thread": [
        {
          "Cpus": [0, 1, 2]
        },
        {
          "Name": "bizproc_worker",
          "Cpus": [3, 4, 5]
        }
      ],
      "Plugins": [
        {
          "Lib": "ldp_bizproc",
          "Instance": 0,
          "Timer": 0,
          ...
        },
        ...
      ],
      "Rcm": {
        ...
      }
    }
  }
}
```

配置文件最外层包含两个标签Applications和Rcm，Applications标签存放应用相关的配置，可以包含多个应用，示例中包含一个应用xst#0；Rcm标签存放主题和上下文等信息，由所有应用共用。

应用配置包含TimerUnit、Components、Thread和Plugins四个标签：

- TimerUnit为框架定时器的精度，单位为毫秒，小于零表示不启用定时器，等于零或不配置，则为默认值1000毫秒。
- Components存放公共库配置，又包含Libs和GlobalLibs两个标签，二者区别是GlobalLibs加载的库可以被其他库依赖，而Libs一般为独立库。此处支持配置简单库名或完整库名（如：tcp或libtcp.so），也支持配置带相对路径的完整库名（如：../abc/libabc.so），基准路径固定为bin/components。
- Thread为独占核的配置，独占核即该核心只能被某一个线程使用。配置分为默认独占核配置与特定线程组的独占核配置，用户可以根据线程组名绑定指定的核，如果线程组数量超过配置的核数，超出的线程则会使用默认的独占核，如果默认独占核也被用完了，则不绑。
- Plugins标签内是个数组，存放各个插件的配置，插件的配置由平台配置项和私有配置项组成，平台配置项包含Lib、Instance和Timer，说明见示例中注释说明，插件的私有配置项参考插件说明。
- EnableRdtsc是否启用rdtsc，在启用Rdtsc的情况下，可以优化Add操作的打点性能。如果通过Addex打点接口传入单调时间，可能与rdtsc的时间有冲突（timelog中展示的时间不准确），强烈建议调用Add接口。特别的，该选项需要在高性能环境下启用（不开节能、不开降频、不开超频）

ZK集中配置和本地配置

平台支持从ZK或本地读取配置文件。如果启动命令包含-l选项，则从本地读取指定文件名的配置文件，不会从ZK上读取配置文件。如果启动命令包含-s选项但是不包含-l选项，则从ZK读取配置文件。其他情况则从本地读取配置文件。

- 当从ZK读取配置时，配置文件在ZK上的路径格式为/ldp/xxx/config/yyy，其中xxx为产品名，yyy为任意文件名，/ldp/xxx/config路径下可以有一个或多个文件，当只有一个文件时，须为一个完整有效的json格式配置文件，当有多个文件时，hsl dp会将多个文件拼接到一起，并在最外层添加一对大括号，形成最终的配置文件。
- 当从本地读取配置文件，配置文件名为xxx.json，其中xxx为产品名。

配置更新

当从ZK读取配置时，hsl dp支持动态更新配置，可动态更新的配置项参考各插件说明。

2.5. 启动应用命令行

LDP平台通过主程序hsl dp进行启动，用法：

```
hsl dp 命令 产品名 应用名[#编号] -s 配置服务器地址 -A zk鉴权信息 -c 共享核列表 -f 快照文件名 -a [--args 业务参数列表] -l 本地配置文件名 -z
```

对应的启动脚本内即使用上述命令行启动应用。

平台命令行参数详细含义如下：

参数		说明
命令	start	启动服务器
	startdaemon	启动服务器守护进程
	stop	停止服务器
配置服务器地址	ip:port	目前支持ZooKeeper，不指定则使用本地配置
共享核列表	cpu0,cpu1,cpu2	配置共享核列表，平台内所有共享类型的线程都会绑定在共享列表指定的核中

参数	说明
产品名	长度不超过32
应用名	长度不超过32
编号	不小于0，可以省略，缺省是0
业务参数列表	以 - 或者 -- 开头的为选项(option)，选项后面紧跟着对应选项的参数(arguments)，其中选项不可省略，选项后面的参数可视情况省略

通过Onload启动ldp时默认会加速进程所有socket（也包括zk客户端socket），若希望禁用默认加速可以使用'-a'参数。我们建议使用的Onload配置为latency，禁止使用latency-best，否则可能导致ldp进程崩溃。zk鉴权信息格式参考ldp_zkhandler插件配置UserAuth的描述。

--args 后面跟业务参数列表，用以向业务透传命令行参数；例如：`--args -d /home/someDir --f fileName` 平台会将 --args 后面跟随的选项和参数按照两个键值对(`-d:/home/someDir, --f:fileName`)存储起来，用户可以通过ICmdOptions提供的接口获取对应的参数。
业务参数列表说明如下：

选项类型	说明
-xxx	业务参数选项，可通过 -xxx 获取其配置的参数，其中 xxx 可以为任意字符
--xxx	业务参数选项，可通过 --xxx 获取其配置的参数，其中 xxx 可以为任意字符

2.6. 其它命令行工具

2.6.1. cleanhsl dp

此脚本及脚本中调用的rm_zknode、read_ldp_config、同hsl dp程序一起默认安装在bin目录中。使用前先把bin加入用户PATH环境变量并使之生效。

用法：

```
cleanhsl dp 产品名 应用名[#编号] -s 配置服务器地址 -r redo备份目录 -a zk鉴权信息 -R -c 日志保留天数 -q 主备重演请求备份目录 -Q 主备重演应答备份目录 -C -t -d todb错误信息文件备份目录 -l 本地配置文件名 -L -x 排除清理的选项
```

命令行参数与hsl dp类似, 含义如下：

参数		说明
ZooKeeper服务器	-s	如果为ZooKeeper，则执行命令为-s
keeper服务器	-e	如果为keeper，则执行命令为-e
配置服务器地址	ip:port	目前支持ZooKeeper和keeper,不指定则使用本地配置
redo备份目录	绝对路径,或相对当前路径	不指定则使用配置的redo目录加上'_bak', 如: kdpredo_bak
zk鉴权信息		同应用中kdp_zkhandler的UserAuth配置
产品名		长度不超过32
应用名		长度不超过32
编号		不小于0，可以省略，缺省是0
-R		只清理重演模式上下文的发送端持久化数据
-c		清理脚本日志保留天数，日志名称YYMMDD-cleanhsl:dp.cleanlog
主备重演请求备份目录	-q 绝对路径,或相对当前路径	不指定则使用配置的主备重演目录加上'_bak', 如: kdp_bizproc_reqlog_bak
主备重演应答备份目录	-Q 绝对路径,或相对当前路径	不指定则使用配置的主备重演目录加上'_bak', 如: kdp_bizproc_rsplog_bak
-C		检查模式，用于检查清理脚本是否完全清理成功
-t		只清理 todb 插件的事务号，用于 todb 重启后重头开始落库
todb错误信息文件备份目录	绝对路径,或相对当前路径	不指定则使用配置的redo目录加上'_todb_bak', 如: kdpredo_todb_bak
本地配置文件名	-l	配置此参数后，将不使用服务器上的配置文件，而使用指定文件名的本地配置文件进行清理
-E		清理异常消息标识类型，默认：2（清理持久化文件和共享内存），其他支持的类型：1（仅清理共享内存），0（都不清理）
-L		只清理本地持久化信息，不清理服务器上的信息
排除清理的选项	-x	配置的选项将会被排除清理，本地持久化信息和服务器上的其他信息将会被强制清理。现在仅支持todb_trans_no，配置后将不会清理服务器上的todb的transno信息

参数	说明
调用者应判断脚本返回值：	

- 0 成功, 其它失败
- 目录/zk节点不存在的情况, 不当作错误, 脚本会打印提示信息, 并继续执行其它清理动作
- zk连接失败, 目录/文件/zk节点删除失败等情况, 视为错误, 脚本退出不继续执行

脚本输出在stdout,stderr, 需要时可手动重定向到文件。

RCM上下文RecordReplayStartPoint为0，即为重演模式上下文，一般为（交易）核心的高可用上下文。

清理的内容:

1. 清除zk上arb信息, todb信息，核心与toddb本地通信文件
2. 在指定的redo备份目录中创建yyyymmdd-hhmmss目录，其中时间为cleanhsldp执行时间，把redo文件移到此目录
3. 清除rcm相关的共享内存、本地缓存、持久化信息
4. 清除partkeeper生成的持久化文件(夹)

脚本日志格式：

1. 日志信息格式：MMDD HHMMSS.纳秒 PID 日志等级 日志内容(行号)
2. 脚本启动会在日志输出执行脚本的路径和启动参数
3. 脚本执行成功会在终端和日志输出“cleanhsldp execute successfully”
4. 部分特殊日志未追加行号

备注：

· 当给 cleanhsldp 传入 -C 、 -R 和 -t 参数中的一个或多个时，只有最后一个会生效。

2.6.2. ldpEnvPrepare

此脚本为环境内存检测脚本，通过不同的输入参数可以查看当前环境的内存是否存在隐患，也可以修改不同资源类型

执行 sh ldpEnvPrepare.sh --help 可以获取该脚本的具体使用方法。根据当前系统的语言环境输出对应的帮助手册。export LANG=zh_CN.UTF-8 即可切换为中文，反之为英文。

检测规则：满足swap>=shm>=物理内存

执行sh ldpEnvPrepare.sh，会出现当前环境是否合规。比较时都是取正整数，进行比较。如：7.9G=7G

用法:

sh ldpEnvPrepare.sh -t 资源类型 -o 命令选项	
-t(--type)资源类型	说明
shm	操作共享内存
swap	操作Swap交换分区
-o(--options)命令选项	说明
mode=on(off)	供swap使用, on为增加Swap交换内存; off为删除Swap交换分区。不填默认为on。
size=50	为修改的内存大小, =后面必须为整数, 单位为G。（增加swap大小时, 每创建一个文件就存在4k误差。所以可能free -h有误差。可以使用swapon -s 查看）
file=/home/a	供swap使用, 为增加或者能够删除swap内存的文件。（若不配置, 默认当前路径下名为'swapfile'的文件）

示例:

1. sh ldpEnvPrepare.sh -t shm -o size=50 修改当前共享内存为50G
2. sh ldpEnvPrepare.sh -t swap -o size=10,file=a,mode=on 当前swap分区增加10G,并挂载在当前路径下的a文件
3. sh ldpEnvPrepare.sh -t swap -o file=a,mode=off 删除当前swap分区,并卸载当前路径下的a文件

注:

1. 修改资源类型必须为root用户
2. 如果用户在增加swap分区过程中，kill了当前脚本的进程，此时a文件已经扩容了部分大小。接着如果执行swap增加分区命令对a文件进行操作，增加的分区大小小于当前a文件大小，会导致报错。用户需要手动删除文件，重新执行脚本

2.6.3. ldpconfig

此脚本从配置上简化用户的部署流程，提升部署效率。

非拆分模式，用户执行ldpconfig成功后，会默认创建zk节点/ldp/xxx/config/config，并将节点值修改为本地的配置文件内容，其中xxx为产品名。

拆分模式，用户执行成功ldpconfig成功后，会为配置文件中REFS，Derived，Applications，Rcm，McBiz部分分别创建符合其层次的节点结构。

执行ldpconfig前，需先放置本地配置到程序执行目录，确保其能正常读取，否则执行将报错。例如产品名为ldp，则应将ldp.json文件放置在ldpconfig执行目录。

用法:

```
ldpconfig 产品名 -s 配置服务器地址 -a zk鉴权信息 -d -m 运行模式
```

命令行参数与hsl dp类似, 含义如下:

参数		说明
配置服务器地址	ipport	目前支持ZooKeeper，不指定则使用本地配置
zk鉴权信息		同应用中ldp_zkhandler的UserAuth配置
产品名		长度不超过32
-d		是否生成hsl dp运行后的配置，文件名：产品名_details
运行模式		设置为0时，使用非拆分模式上传。设置为非0，使用拆分模式上传。默认0。

注:

1. 若本地配置文件json格式错误，则上传失败。
2. 若之前使用某个产品名启动过hsl dp进程，则上传失败。
3. 非拆分模式上传下，若节点/ldp/xxx/config/含多个子配置节点，则上传失败。
4. 非拆分模式上传下，若先前已创建/ldp/xxx/config/yyy节点，不创建默认配置节点名config，则会复用yyy节点，并修改其节点值，yyy为用户自定义配置节点名。
5. ldpconfig依赖于ldp_mproxy、ldp_front、ldp_offerproc、ldp_fens、ldp_todb、ldp_bizproc的身份注册功能，发布包要求LDP1.0.V202203.00.000及以上。
6. 使用拆分模式上传时，若/ldp/xxx/config/含之前非拆分模式上传内容，应手动清理后再上传。
7. 若/ldp/xxx/config/节点内容含字段ConfigFlags且值为split，则必然采用拆分模式上传。其他情况，存在-m以-m指定的为准。否则，按非拆分模式上传。

2.6.4. ldpPing

此脚本及脚本中调用的ldpPing可执行程序, 同hsl dp程序一起默认安装在bin目录中。是以0号ldp内部功能号作为测试功能号的测试工具，使用的核心版本至少为1.0.7.7及以上（其他版本不兼容）。发布包中提供了ldpPing.sh来执行ldpPing程序。

用法: ldpPing -h 192.168.86.101:1234 --threadcorebind 4 --tps 1000 --testcase bizproc_base 等

命令行参数含义如下:

参数		说明
help	-h or --help	如果输入的参数中存在-h或者--help，告知ldpPing的用法和参数意义
服务端地址	192.168.86.101:1234	测试工具主动去连接的服务端地址ipport，除了help，必须填写在第一位。必须填写
测试类型	--testcase bizproc_base	目前只支持到核心（bizproc_base），默认为bizproc_base
TPS	--tps 10000	每秒发送消息个数。默认1000
一共发送的消息包个数	--packcount 1000000	默认10000
消息包大小	--packsize 120	包大小必须在[72,4096]区间，默认128
绑核	--threadcorebind 3	测试工具tcp通信线程绑核。没填则随机绑定
时延打点	--opents false	是否开启时延分析。默认false，不开启
展示细节时延数据	--showdetails false	是否记录所有收发消息的时间戳。默认false，不开启
指定工作线程处理	--locatethreadno 0	默认0，配置错误，请求消息会定位失败

为了测出最好的性能，绑核配置方式一定要正确，可以浏览一下知识平台上Light LDP平台团队的技术总结文档中的《LDP平台绑核流程》。

程序流程及打印:

```
lshijw@host-86-101 client$ sh ldpPing.sh
onload: ERROR: EF_TCP_SYNRECV_MAX=262140 and EF_MAX_ENDPOINTS=8192 are inconsistent.
onload: EF_TCP_SYNRECV_MAX is set to 262140 based on /proc/sys/net/ipv4/tcp_max_syn_backlog value and assuming up to 4 listening sockets in the Onload stack
onload: Too few endpoints requested: -4 syn-receive states consume one endpoint.
2022-12-09 09:22:42 testcase[bizproc_base], serveraddr[192.168.86.101:8565], tps[1000], packcount[10000], packsize[128], showdetails[false], opents[false] (main.cpp:165:main)
onldpPing[717]: Using OpenOnload 201811-01 Copyright 2006-2019 Solarflare Communications, 2002-2005 Level 5 Networks [11.0-p717]
2022-12-09 09:22:44 Onconnected (bizproc_testcase.cpp:25:Onconnected)
2022-12-09 09:22:44 Init Success (bizproc_testcase.cpp:20:Init)
2022-12-09 09:22:44 Start Warming (bizproc_testcase.cpp:168:PreRun)
2022-12-09 09:22:45 PreRun Success (bizproc_testcase.cpp:76:PreRun)
2022-12-09 09:22:45 Start Running (bizproc_testcase.cpp:82:Run)
2022-12-09 09:22:55 Run Success (bizproc_testcase.cpp:91:Run)
testcase      packsize      tps      min      max      average      middle      99file      stddev      errorcount
bizproc_base  128      1000    18363    2625640    48703      47583      77386      25769.37      0
2022-12-09 09:22:55 Finish Success (bizproc_testcase.cpp:117:Finish)
```

上图可知，每次执行程序：①.程序会先连上服务端，打印OnConnected。②.进程初始化，打印Init Success。③.开始热身，发送500笔和输入PackSize一样的消息包，打印Start Warming。④.热身结束，打印PreRun Success。⑤.开始正式发送消息包，打印Start Running。⑥.发送结束，打印Run Success。⑦.等待结束，完成整理计算平均值等操作，Wait for Finish。⑧.结果打印。结束

程序输出：

每次执行结束，都会生成一份bizproc_base_tps[--tps]_size[--packsize]_result.csv的文件，tps和size后面的值为输入的tps大小和packsize大小，重复一样的执行会覆盖生成的文件。如果showdetails为true，则会同时生成一份例如bizproc_base_tps[--tps]_size[--packsize]_detail.csv的文件，其中的数据为从第0个消息包到packcount个的消息包，客户端发送出去，到收到应答的所有时间戳。

2.6.5. ldp_operation

此脚本放置于workspace下，脚本需要调用的ldpmgrtool,放置在bin下。该脚本提供了启动，暂停，检查流程。ldp_mproxy的版本至少为1.0.7.4及以上

用法: sh ldp_operation.sh runmode productname appname [OPTION]...

命令行参数含义如下:

参数		说明
help	-h or --help	如果输入的参数中存在-h或者--help，告知ldp_operation的用法和参数意义
脚本执行模式	runmode	脚本执行模式有start, startdaemon, stop, check。必须填写
产品名	productname	产品名。必须填写
应用名	appname	应用名，需要填写应用实例，如xst#0。必须填写
以下是可选参数 [OPTION]...		
-s 192.168.86.102.2181		zk模式启动
-e 192.168.86.102.23781		keeper模式启动
-t 5		超时时间设置，单位秒(start, startdaemon或者check模式，会去check一次进程是否可用，如果不可用，每隔1秒去check一次直到超过超时时间或者成功才会停止。不配置默认为10秒)

该脚本可以作为标准的启动，停止脚本和检查脚本。启动或者检查时会调用ldpmgrtool，获取管理代理的GetAppsStatus中各个app告警和最终状态，打印输出于界面。

2.6.6. ldpmdb_tools

此工具用于解析LDPMDB在运行过程中产生的中间文件，放置在发布包的bin目录下，方便用户排查内存数据库问题。

ldpmdb_tools目前功能包含：

- 解析redo文件：
 - 允许用户只将指定的某张表或者某几张表的redo数据（包括增删改数据，用户自定义数据，序列号数据，清表数据）转换到csv文件；
 - 允许用户指定从哪个事务号开始转换redo数据到csv文件；
 - 可以选择依赖datamagr或依赖共享内存来解析redo文件。
- 解析共享内存文件：
 - 解析共享内存表数据文件；
 - 解析共享内存索引数据文件；
 - 清理共享内存数据库异常状态。
- 解析undo文件
- 根据togw业务错误日志解析redo文件：
 - 支持通过工具分析、查看回库失败对应的事务详细信息，方便用户排查问题

注意： 该工具依赖于内存数据库(libuftmdb.so)及rcm独立库(librcm.so)，用户需要额外设置环境变量LD_LIBRARY_PATH库路径，默认为发布包/bin/components及/bin/linux.x64/lib/目录下。

本工具支持两种使用模式，分别是使用配置文件运行和使用命令选项运行。

查看使用说明： ./ldpmdb_tools -h

查看示例配置： ./ldpmdb_tools -e

使用配置文件运行时，用法为： ./ldpmdb_tools test.xml

test.xml示例配置如下：

```
<!-- 以下是mode=0的示例配置 -->
<tool mode="0" note="mode: 0=解析redo, 依赖datamgr">
  <params file_out="out.csv" note="指定解析后的输出文件名">
    <args redo_path="/home/test/ldp_SecuUFT3.0/Sources/workspace/record/wsl dp_mdb#0_ldp_remoter_0" note="指定redo文件所在目录" trans_no="0" trans_filter="123,456-500,789" fragment mode="0"/>
    <datamgr datamgr_path="/home/test/ldp_SecuUFT3.0/Sources/bin" note="指定datamgr库所在目录">
      <component dll="s_ldpmdb_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
      <component dll="s_ldpmdb_demo_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
    </datamgr>
    <cut_off_condition note="只解析指定表的redo记录, 可选">
      <filter_field table="entrust" note="解析branch表的redo, 可选"/>
      <filter_field table="account" note="解析account表的redo, 可选"/>
    </cut_off_condition>
  </params>
</tool>
<!-- 以下是mode=1的示例配置 -->
<tool mode="1" note="mode: 1=解析共享内存">
  <params>
    <args database_name="test"/>
    <datamgr datamgr_path="/home/test/ldp_SecuUFT3.0/Sources/bin" note="指定datamgr库所在目录">
      <component dll="s_ldpmdb_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
      <component dll="s_ldpmdb_demo_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
    </datamgr>
    </params>
    <shmparams>
      <tasks command="0" note="0=查看共享内存表信息, 1=解析共享内存表, 2=查看共享内存索引, 3=解析共享内存索引, 4=查看共享内存数据库, 5=清理共享内存数据库">
        <task table="branch" index="idx_branch" note="任务0: 解析branch表索引idx_branch"/>
        <task table="branch" refer_index="idx_ref_branch" note="任务1: 解析branch表关联索引idx_ref_branch"/>
      </tasks>
    </shmparams>
  </tool>
<!-- 以下是mode=2的示例配置 -->
<tool mode="2" note="mode: 2=解析undo">
  <params>
    <args database_name="test" />
    <datamgr datamgr_path="/home/test/ldp_SecuUFT3.0/Sources/bin" note="指定datamgr库所在目录">
      <component dll="s_ldpmdb_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
      <component dll="s_ldpmdb_demo_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
    </datamgr>
    </params>
    <undoparams>
      <args undo_path="/dev/shm" undo_file="uftmdb.undo.test.0.0" out_file="1.txt" note="指定解析后的输出文件名"/>
      <filters note="只解析指定表的undo记录, 可选">
        <filter table="branch" note="解析branch表的undo, 可选"/>
        <filter table="contract" note="解析contract表的undo, 可选"/>
      </filters>
    </undoparams>
  </tool>
<!-- 以下是mode=3的示例配置 -->
<tool mode="3" note="mode: 3=解析redo, 依赖共享内存">
  <params file_out="out.csv" note="指定解析后的输出文件名">
    <args database_name="test" redo_path="/home/test/ldp_SecuUFT3.0/Sources/workspace/record/wsl dp_mdb#0_ldp_remoter_0" note="指定redo文件所在目录" trans_no="0" />
    <cut_off_condition note="只解析指定表的redo记录, 可选">
      <filter_field table="entrust" note="解析branch表的redo, 可选"/>
      <filter_field table="account" note="解析account表的redo, 可选"/>
    </cut_off_condition>
  </params>
</tool>
<!-- 以下是mode=4的示例配置 -->
<tool mode="4" note="mode: 4=根据togw业务错误日志解析redo">
  <params file_out="out.csv" note="指定解析后的输出文件名">
    <args redo_path="/home/test/ldp_SecuUFT3.0/Sources/workspace/record/wsl dp_mdb#0_ldp_remoter_0" note="指定redo文件所在目录" trans_no="0" bizerr_log_dir="/home/test/ldp_SecuUFT3.0/Sources/workspace_todb/log/" file_pattern="20241017-hsl dp-wsl dp-togw#0.log"/>
    <datamgr datamgr_path="/home/test/ldp_SecuUFT3.0/Sources/bin" note="指定datamgr库所在目录">
      <component dll="s_ldpmdb_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
      <component dll="s_ldpmdb_demo_datamgr" note="指定datamgr库, 不配置dll时默认查找datamgr_path下的所有datamgr库, 可选"/>
    </datamgr>
    <cut_off_condition note="只解析指定表的redo记录, 可选">
      <filter_field table="entrust" note="解析branch表的redo, 可选"/>
      <filter_field table="account" note="解析account表的redo, 可选"/>
    </cut_off_condition>
  </params>
</tool>
```

配置含义如下:

配置项		说明
mode	解析工具模式	0=解析redo, 依赖datamgr; 1=解析共享内存; 2=解析undo; 3=解析redo, 依赖共享内存; 4=根据togw业务错误日志解析redo
file_out	指定解析后的输出文件名	默认为./out.csv
database_name	核心配置的内存数据库名称	
redo_path	指定的redo文件目录	rcm持久化文件目录
trans_no	指定起始事务号解析redo	默认值: 0, 可选
trans_filter	指定事务号范围解析redo	支持指定单个事务号、多个不连续的事务号以及指定事务号范围进行解析redo数据(事务号范围≥0)
datamgr_path	指定datamgr库所在目录	在目录下递归查找dll配置的所有datamgr
dll	datamgr库名	不配置此项时默认查找datamgr_path下的所有datamgr
table	指定解析redo的表名	不配置此项时默认解析所有表的redo

command	指定操作指令	0=查看共享内存表信息；1=解析共享内存表数据；2=查看共享内存索引信息；3=解析共享内存索引数据；4=查看共享内存数据库信息；5=清理共享内存数据库状态；
table	指定要解析的表名称	command=0/1时，all表示解析所有表；command=2/3时，不支持all参数
index	指定要解析的表索引名称	
refer_index	指定要解析的关联索引名称	
undo_path	指定undo文件所在目录	
undo_file	指定undo文件名	
out_file	指定输出文件名	
table	指定解析undo的表名	不配置此项时默认解析所有表的undo
bizerr_log_dir	指定togw业务错误日志所在目录	
file_pattern	指定togw业务错误日志名	支持模糊匹配
fragment_mode	指定redo分片合并模式	1表示开启redo分片合并，其他表示关闭redo分片合并，默认关闭

解析redo文件

- 依赖datamgr模式(mode=0):

- 使用配置文件运行:

```
用法: ./ldpmdb_tools test.xml
```

- 使用命令行运行:

```
用法: ./ldpmdb_tools -m 0 -o out.csv -p
/home/test/ldp_SecuUFT3.0/Sources/workspace/record/wsl dp_mdb#0_ldp_remoter_0 -L
/home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -t entrust -n 0 -N "123,456-500,789"
```

- 依赖共享内存模式(mode=3):

- 使用配置文件运行:

```
用法: ./ldpmdb_tools test.xml
```

- 使用命令行运行:

```
用法: ./ldpmdb_tools -m 3 -o out.csv -d test -p
/home/test/ldp_SecuUFT3.0/Sources/workspace/record/wsl dp_mdb#0_ldp_remoter_0 -t entrust -n 0
```

- 根据togw业务错误日志解析redo模式(mode=4):

- 使用配置文件运行:

```
用法: ./ldpmdb_tools test.xml
```

- 使用命令行运行:

```
用法: ./ldpmdb_tools -m 4 -o out.csv -p
/home/test/ldp_SecuUFT3.0/Sources/workspace/record/wsl dp_mdb#0_ldp_remoter_0 -L
/home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -t entrust -n 0 -b
/home/test/ldp_SecuUFT3.0/Sources/workspace_todb/log/ -x "20241017-hsldp-wsl dp-togw#0.log"
```

解析redo文件，命令选项说明:

命令行参数	说明
-m	指定mode，依赖datamgr模式时固定取值为0，依赖共享内存模式时固定取值为3，等价于配置文件的tool→mode
-o	指定输出的csv文件，默认输出到out.csv文件，等价于配置文件的params→file_out
-d	核心配置的内存数据库名称，等价于配置文件的args→database_name
-p	指定redo文件所在目录，等价于配置文件的 args→redo_path
-L	指定datamgr库所在目录，在目录下递归查找dll配置的所有datamgr，等价于配置文件的 datamgr→datamgr_path
-l	datamgr库名，可以配置多项，不配置此项时默认查找datamgr_path下的所有datamgr，等价于配置文件的 datamgr→component→dll
-t	指定解析redo的表名，不配置则解析所有表的redo，等价于配置文件的cut_off_condition→filter_field→table
-n	指定起始事务号解析redo，等价于配置文件的args→trans_no

-N	指定事务号范围解析redo，等价于配置文件的args→trans_filter,当-N和-n一起配置时,N参数优先级更高
-b	指定togw业务错误日志所在目录，等价于配置文件的args→bizerr_log_dir
-x	指定togw业务错误日志名，等价于配置文件的args→file_pattern
-M	指定redo分片合并模式，等价于配置文件的args→fragment_mode

解析redo文件时，并非所有参数都是必要的，其中：

- mode=0时：
 - 必选：解析工具模式(-m)，redo文件所在目录(-p)，datamgr所在目录(-L)。
 - 可选：输出文件名(-o)，datamgr库名(-l)，解析redo的起始事务号(-n)，解析redo的事务号范围(-N)，解析redo的表名(-t)，redo分片合并模式(-M)。
- mode=3时：
 - 必选：解析工具模式(-m)，内存数据库名称(-d)，redo文件所在目录(-p)。
 - 可选：输出文件名(-o)，解析redo的起始事务号(-n)，解析redo的事务号范围(-N)，解析redo的表名(-t)，redo分片合并模式(-M)。
- mode=4时：
 - 必选：解析工具模式(-m)，redo文件所在目录(-p)，datamgr所在目录(-L)，togw业务错误日志所在目录(-b)，togw业务错误日志名(-x)。
 - 可选：输出文件名(-o)，datamgr库名(-l)，解析redo的起始事务号(-n)，解析redo的事务号范围(-N)，解析redo的表名(-t)，redo分片合并模式(-M)。

解析redo文件的结果说明：

1、形如：0,0,0,UserDefine,5,0,93,[define_data] 含义为：事务号,redo类型,传输号,用户自定义,自定义类型,自定义状态,自定义数据长度,自定义数据
2、形如：1,0,1,UserDefine(Init),Begin/End,[define_data] 含义为：事务号,redo类型,传输号,用户自定义(初始化),初始化开始/初始化完成,自定义数据
3、形如：3,0,3,UserDefine(DynamicSeq),CreateSeq,[SerialNumberName],0,1000000,1,10000 含义为：事务号,redo类型,传输号,用户自定义(动态序列号),创建序列号,序列号名称,序号,初始值,每次递增数,切换步长
4、形如：3,0,3,UserDefine(DynamicSeq),DeleteSeq,[SerialNumberName],0 含义为：事务号,redo类型,传输号,用户自定义(动态序列号),删除序列号,序列号名称,序号
5、形如：3,0,3,Sequence(Y/S),0,10000,1,100,10086,10000 含义为：事务号,redo类型,传输号,操作(重置序列号/同步序列),序号,初始值,每次递增数,同步间隔,序列当前值,最近一次同步的序列号
6、形如：3,0,3,2408,1204-100038.474705,entrust,0,Create,branch_no,0,fund_account,HS600570,..... 含义为：事务号,redo类型,传输号,事务大小,日期-时间,表名,记录号,操作,(字段名,字段值),(字段名,字段值).....
7、形如：3,0,3,2408,1204-100038.474705,entrust,Clear 含义为：事务号,redo类型,传输号,事务大小,日期-时间,表名,清表
8、形如：2,0,2,1214-205436.331426,UpdateRefRecord,99,brach,0,1,0,contract 含义为：事务号,redo类型,传输号,日期-时间,操作,记录号,表名,(redo模式: 0表示全量模式,其他表示增量模式),关联记录的数量,(关联记录的记录号,关联记录的表名).....
9、形如：2,0,2,2408,1214-205436.331426,brach,0,UpdateRecordKey,0,0,branch_no,0,..... 含义为：事务号,redo类型,传输号,事务大小,日期-时间,表名,记录号,操作,(redo模式: 0表示全量模式,其他表示增量模式),(关联记录的数量: 当更新关联记录时表示关联记录的数量,其他情况为0),(字段名,字段值).....
10、形如：1,0,1,1216-144102.336877,InsertIntoIndex,100,branch,-1,branch,0 含义为：事务号,redo类型,传输号,时间,操作,记录号,表名,(关联索引所属的记录号: 若不是关联索引则本字段不使用),索引所属的表名,索引所属表中本索引的下标
11、形如：2,0,2,SetMdbPrivilege,1,1,1,{"UserInfo":[{"UserName":"User1","RoleCount":1,"Roles":[0]}],"RoleInfo":[{"RoleName":"Role1","RolePrivilege":[{"Category":2312,"Privilege":11}]}]} 含义为：事务号,redo类型,传输号,操作,权限信息的版本号,权限信息中用户的个数,权限信息中角色的个数,存所有用户的具体信息
12、形如：2,0,2,1,1,PartitionNo=100 含义为：事务号,redo类型,传输号,记录号,表号,分区号

根据togw业务错误日志解析redo文件的结果说明：

1、形如：1,0,1,0,101,无效的调用,0,UserDefine,5,0,93,[define_data] 含义为：事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,用户自定义,自定义类型,自定义状态,自定义数据长度,自定义数据
2、形如：1,0,1,0,102,无效的输入参数,1,UserDefine(Init),Begin/End,[define_data] 含义为：事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,用户自定义(初始化),初始化开始/初始化完成,自定义数据
3、形如：1,0,1,0,101,无效的调用,3,UserDefine(DynamicSeq),CreateSeq,[SerialNumberName],0,1000000,1,10000 含义为：事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,用户自定义(动态序列号),创建序列号,序列号名称,序号,初始值,每次递增数,切换步长

4、形如: 1,0,1,0,101,无效的调用,3,UserDefine(DynamicSeq),DeleteSeq,[SerialNumberName],0 含义为: 事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,用户自定义(动态序列号),删除序列号,序列号名称,序号
5、形如: 1,0,1,0,101,无效的调用,3,Sequence(Y/S),0,10000,1,100,10086,10000 含义为: 事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,操作(重置序列号/同步序列),序号,初始值,每次递增数,同步间隔,序列当前值,最近一次同步的序列号
6、形如: 1,0,1,0,101,无效的调用,3,2408,1204-100038.474705,entrust,0,Create,branch_no,0,fund_account,HS600570,..... 含义为: 事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,事务大小,日期-时间,表名,记录号,操作,(字段名,字段值),(字段名,字段值).....
7、形如: 1,0,1,0,101,无效的调用,3,2408,1204-100038.474705,entrust,Clear 含义为: 事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,事务大小,日期-时间,表名,清表
8、形如: 1,0,1,0,101,无效的调用,2,1214-205436.331426,UpdateRefRecord,99,brach,0,1,0,contract 含义为: 事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,日期-时间,操作,记录号,表名,(redo模式: 0表示全量模式,其他表示增量模式),关联记录的数量,(关联记录的记录号,关联记录的表名).....
9、形如: 1,0,1,0,101,无效的调用,2,2408,1214-205436.331426,brach,0,UpdateRecordKey,0,0,branch_no,0,..... 含义为: 事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,事务大小,日期-时间,表名,记录号,操作,(redo模式: 0表示全量模式,其他表示增量模式),(关联记录的数量:当更新关联记录时表示关联记录的数量,其他情况为0),(字段名,字段值).....
10、形如: 1,0,1,0,101,无效的调用,1,1216-144102.336877,InsertIntoIndex,100,branch,-1,branch,0 含义为: 事务号,redo类型,线程号,集群号,错误号,错误信息,传输号,时间,操作,记录号,表名,(关联索引所属的记录号:若不是关联索引则本字段不使用),索引所属的表名,索引所属表中本索引的下标
11、形如: 1,0,1,0,101,无效的调用,2,SetMdbPrivilege,1,1,1,{"UserInfo":[{"UserName":"User1","RoleCount":1,"Roles":[{"RoleName":"Role1","RolePrivilege":[{"Category":2312,"Privilege":11}]}]}] 含义为: 事务号,redo类型,线程号,集群号,错误号,传输号,操作,权限信息的版本号,权限信息中用户的个数,权限信息中角色的个数,所有用户的具体信息

解析共享内存文件

使用配置文件运行

```
用法: ./ldpmdb_tools test.xml
```

使用命令行运行:

```
用法: ./ldpmdb_tools -m 1 -c 0 -d test -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -t all
```

解析共享内存数据文件, 命令选项说明:

命令行参数	说明
-m	指定mode, 解析共享内存时固定取值为1, 等价于配置文件的 tool→mode
-d	核心配置的内存数据库名称, 等价于配置文件的args→database_name
-L	指定datamgr库所在目录, 在目录下递归查找dll配置的所有datamgr, 等价于配置文件的 datamgr→datamgr_path
-l	datamgr库名, 可以配置多项, 不配置此项时默认查找datamgr_path下的所有datamgr, 等价于配置文件的 datamgr→component→dll
-c	操作指令, 等价于配置中: shnparams→tasks→command. 0=查看共享内存表信息; 1=解析共享内存表数据; 2=查看共享内存索引信息; 3=解析共享内存索引数据; 4=查看共享内存数据库信息; 5=清理共享内存数据库状态;
-t	指定要解析的表名称. command=0/1时, all表示解析所有表; command=2/3时, 不支持all参数; 等价于配置文件的shnparams→tasks→task→table
-i	指定要解析的表级索引名称. 等价于配置文件的shnparams→tasks→task→index
-r	指定要解析的关联索引名称. 等价于配置文件的shnparams→tasks→task→refer_index

解析共享内存数据文件时, 并非所有参数都是必要的, 其中:

- command=0/1: 需要解析工具模式(-m)、内存数据库名称(-d)、datamgr所在目录(-L)、datamgr库名(-l)、操作指令(-c)、数据表(-t)参数
- command=2/3: 需要解析工具模式(-m)、内存数据库名称(-d)、datamgr所在目录(-L)、datamgr库名(-l)、操作指令(-c)、数据表(-t)、索引名称(-i)或关联索引名称(-r)参数
- command=4/5: 需要解析工具模式(-m)、内存数据库名称(-d)、datamgr所在目录(-L)、datamgr库名(-l)、操作指令(-c)参数

生成文件说明:

- command=0: 生成内存表信息文件(/out/ \$表名/ table/\$表名.json)
- command=1: 生成内存表数据文件(/out/ \$表名/ table/\$表名.csv)
 - 数据格式: 记录前缀,@_@,记录的值
- command=2: 生成内存索引信息文件(/out/\$表名/index/\$表名.json)
- command=3: 生成内存索引数据文件(/out/\$表名/index/\$索引名称.csv)
 - 数据格式: 索引节点状态,@_@,索引到的记录的值

- command=4：生成内存数据库信息文件(/out/\$数据库名.json)
- command=5：清理内存数据库，生成文件包含：
 - 调用内部清理函数 CleanWorkProcess 的执行结果： /out/clean_work_process_YYYYMMDDHHMMSS.csv
 - 解锁的记录信息： /out/\$表名/table/unlock_YYYYMMDDHHMMSS.csv
 - 解锁的索引信息： /out/\$表名/index/all_unlock_YYYYMMDDHHMMSS.csv

命令行使用示例：

- 1、查看表信息： /ldpmdb_tools -m 1 -d demo -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -c 0 -t branch
- 2、遍历表记录： /ldpmdb_tools -m 1 -d demo -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -c 1 -t branch
- 3、查看索引信息： /ldpmdb_tools -m 1 -d demo -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -c 2 -t branch
- 4、遍历表级索引： /ldpmdb_tools -m 1 -d demo -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -c 3 -t branch -i idx_branch
- 5、遍历关联索引： /ldpmdb_tools -m 1 -d demo -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -c 3 -t branch -r idx_branch_refer
- 6、查看数据库信息： /ldpmdb_tools -m 1 -d demo -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -c 4
- 7、清理数据库状态： /ldpmdb_tools -m 1 -d demo -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -c 5

解析undo文件

使用配置文件运行：

```
用法： ./ldpmdb_tools test.xml
```

使用命令行运行：

```
用法： ./ldpmdb_tools -m 2 -d test -L /home/test/ldp_SecuUFT3.0/Sources/bin -l "s_ldpmdb_datamgr,s_ldpmdb_demo_datamgr" -p /dev/shm -u uftmdb.undo.test.0.1 -o out.txt
```

解析undo文件，命令选项说明：

命令行参数	说明
-m	指定mode，解析共享内存时固定取值为2，等价于配置文件的 tool→mode
-d	核心配置的内存数据库名称，等价于配置文件的args→database_name
-L	指定datamgr库所在目录，在目录下递归查找dll配置的所有datamgr，等价于配置文件的 datamgr→datamgr_path
-l	datamgr库名，可以配置多项，不配置此项时默认查找datamgr_path下的所有datamgr，等价于配置文件的 datamgr→component→dll
-p	待解析的undo文件路径，等价于配置文件的undoparams→args→undo_path
-u	待解析的undo文件名称，等价于配置文件的undoparams→args→undo_file
-o	指定输出的文件名，等价于配置文件的undoparams→args→out_file
-t	指定解析undo的表名，(eg: -t "branch,contract")不配置则解析所有表的undo，等价于配置文件的 undoparams→filters→filter

注意：解析 undo 必须依赖数据库共享内存的管理文件【/dev/shm/uftmdb.admin.demo】

注意事项

- 1、命令行和配置文件同时设置一个参数值时，命令行参数优先于配置文件。
- 2、依赖datamgr解析redo、解析共享内存数据时都需要依赖datamgr库。其中，database和datamgr库必须相对应，否则可能导致工具执行异常。
- 3、解析共享内存、redo文件时，需要依赖对应的共享内存文件、datamgr.so。
- 4、根据togw业务错误日志解析redo文件时，只会展示与错误日志有关的事务详细信息。如果在错误日志中有多条相同事务号的错误信息时，以相同事务号中最新的错误信息为准。
- 5、使用此工具前，请退出所有有关ldpmdb的进程，否则可能导致工具执行异常。

2.6.7. ldpRedoParser

此工具(ldpRedoParser)放置在发布包的/bin目录下，其配置文件(config.json)放置在发布包的/workspace目录下。此工具用于解析Redo文件，方便用户排查redo相关的问题。

注意：该工具依赖t2sdk独立库(libt2sdk.so)，用户需要在Cres的发布包中下载该so文件，将其放到LDP发布包/bin目录下，并额外设置环境变量LD_LIBRARY_PATH库路径为LDP发布包/bin目录。

使用方法:

```
./ldpRedoParser
```

config.json示例配置如下:

```
{
  "/*": "redo文件路径配置",
  "redoFilePath": "./ldpredo/current",
  "/*": "消息结构配置文件/文件夹",
  "structCfg": ".",
  "/*": "输出端配置, 0: 输出到文件、1: 输出到标准输出、2: 输出到标准错误",
  "outputDest": 0,
  "/*": "解析输出文件名配置, 仅在outputDest = 0时生效",
  "outputFileName": "./redo_result/Test",
  "/*": "解析输出文件单独是否存储在以时间命名的文件夹中",
  "timeFolder": false,
  "/*": "解析输出文件, 单个文件大小的上限, 仅在outputDest = 0时生效, 单位: MB",
  "fileMaxSize": 100,
  "/*": "解析输出格式, 0: JSON、1: CSV",
  "outputFormat": 1,
  "/*": "JSON格式化开关, false: 关闭、true: 打开, 仅在outputFormat = 0时生效",
  "jsonPretty": true,
  "/*": "写文件句柄打开模式, 复用(适用功能号偏多的情况)、1: 缓存(适用提高写入效率的情况), 仅在outFileMerge = false时生效",
  "fdUseMode": 1,
  "/*": "数据缓存时间间隔, 未到缓存容量上限前, 每隔一定时间写线程fflush一次磁盘IO, 单位: 秒(关闭缓存时填0)",
  "fflushInterval": 1,
  "/*": "数据缓存容量大小, 达到上限时, 写线程fflush一次磁盘IO, 单位: Bytes, 仅在fflushInterval > 0时有效。注: 1 MB=1048576 B",
  "cacheSize": 1048576,
  "/*": "展示redo头信息开关, false: 关闭、true: 打开",
  "showRedoHead": true,
  "/*": "过滤事务类型, 0: 分析全部事务, 1: 仅分析重放事务, 2: 仅分析回库事务, 3: 分析即重放又回库事务",
  "redoFlagOpt": 0,
  "/*": "原char类型数据, 是否按ASCII码进行显示: false: 字符形式, true: Ascii形式, 仅在outputFormat = 1时生效",
  "showCharToAscii": false,
  "/*": "解析输出文件, 是否合并输出到单个文件中, false: 按照功能号拆分文件, true: 合并输出到单个文件",
  "outFileMerge": false,
  "/*": "支持通过配置文件设置日志级别, 数字和日志级别对应关系如下: 0 - debug, 1 - info, 2 - warn, 3 - error, 4 - imp, 5 - fatal",
  "logLevel": 2
}
```

支持设置工具输出的日志级别, 有两种方式: 命令行参数或配置项, 如果配置项和命令行参数都没配置则默认info级别, 如果配置项和命令行参数设置了其中一个, 则立即生效, 如果都设置最终以配置项为准, 命令行参数使用方法如下:

```
ldpRedoParser 支持通过命令行参数设置日志级别, 使用示例: 'ldpRedoParser --log-level info'
'ldpRedoParser --log_level error' 'ldpRedoParser -l error'
```

即支持三种参数字符串(作用是一样), 日志级别大小写不敏感, 即: ldpRedoParser --log-level INFO与ldpRedoParser --log-level Info 都可以, 最终会根据设置的日志级别过滤日志, 没达到日志级别的日志不会输出. 参数设置日志级别和ldp log日志级别对应关系如下

日志级别	ldp 日志级别
debug	LEVEL_DEBUG
info	LEVEL_INFO
warn	LEVEL_WARN
error	LEVEL_ERROR
inp	LEVEL_IMP
fatal	LEVEL_FATAL

配置含义如下:

配置项		说明
redoFilePath	redo文件路径配置	
structCfg	消息结构配置文件/文件夹	
outputDest	输出端配置	0: 输出到文件、1: 输出到标准输出、2: 输出到标准错误
outputFileName	解析输出文件名配置	仅在outputDest = 0时生效
timeFolder	解析输出文件单独是否存储在以时间命名的文件夹中	
fileMaxSize	解析输出文件, 单个文件大小的上限	仅在outputDest = 0时生效, 单位: MB
outputFormat	解析输出格式	0: JSON、1: CSV
jsonPretty	JSON格式化开关	false: 关闭、true: 打开, 仅在outputFormat = 0时生效
fdUseMode	写文件句柄打开模式	0: 复用(适用功能号偏多的情况)、1: 缓存(适用提高写入效率的情况), 仅在outFileMerge = false时生效
fflushInterval	数据缓存时间间隔	未到缓存容量上限前, 每隔一定时间写线程fflush一次磁盘IO, 单位: 秒(关闭缓存时填0)

cacheSize	数据缓存容量大小	达到缓存容量上限时，写线程flush一次磁盘IO，单位：Bytes，仅在flushInterval>0时有效。注：1 MB=1048576 B
showRedoHead	展示redo头信息开关	false: 关闭、true: 打开
redoFlagOpt	过滤事务类型	0: 分析全部事务，1: 仅分析重放事务，2: 仅分析回库事务，3: 分析即重放又回库事务
showCharToAscii	原char类型数据，是否按ASCII码进行显示	false: 字符形式，true: Ascii形式，仅在outputFormat = 1时生效
outFileMerge	解析输出文件，是否合并输出到单个文件中	false: 按照功能号拆分文件，true: 合并输出到单个文件

FAQ

Q1: 运行界面中出现./ldpRedoParser: error while loading shared libraries: libt2sdk.so: cannot open shared object file: No such file or directory

A1: 这个问题是因为/bin目录下没有libt2sdk.so或者没有设置LD_LIBRARY_PATH到bin目录下，如果bin目录下没有libt2sdk.so，请在Cres发布包中下载该so文件并将其放到LDP发布包/bin目录下，并再次尝试，如果依旧出现该问题，则需要额外设置环境变量LD_LIBRARY_PATH库路径为LDP发布包/bin目录

Q2: 运行界面中出现Redo Init failed, errno:1206, error:参数异常 JsonObj LoadFromFile error

A2: 这个问题是因为config.json被重命名或者缺少config.json文件，请将文件名改回为config.json或从发布包的/bin目录下获取该文件

Q3: 解析后输出的文件中出现FunctionID is not exist in cfg file

A3: 这个问题是因为没有配置structCfg或structCfg配置错误，请将该配置项配置到struct.cfg文件所在路径

2.6.8. ldpMsgParser

此工具(ldpMsgParser)放置在发布包的/bin目录下，其配置文件(ParserConfig.json)放置在发布包的/workspace目录下。此工具用于解析rcm持久化文件，支持根据消息请求、回复的功能码过滤。

注意：该工具依赖libt2sdk.so、librcm.so，用户需要在Cres的发布包中下载libt2sdk.so文件，将其放到LDP发布包/bin目录下，并额外设置环境变量LD_LIBRARY_PATH库路径为LDP发布包/bin目录。接收端和发送端解析有差异性，所以在配置上需要注意。其他疑问点可以参照ldpRedoParser使用中遇到的问题。

使用方法:

```
./ldpMsgParser
```

ParserConfig.json示例配置如下:

```
{
  "/*": "文件类型: 0: rcm接收端持久化; 1: rcm发送端持久化;",
  "FileType": 0,
  "/*": "请求白名单, 黑白都配, 只有黑名单生效",
  "RequestWhiteFunctionID": "101;5",
  "/*": "回复白名单, 黑白都配, 只有黑名单生效",
  "ResponseWhiteFunctionID": "5;101",
  "/*": "请求黑名单, 黑白都配, 只有黑名单生效",
  "RequestBlackFunctionID": "",
  "/*": "回复黑名单, 黑白都配, 只有黑名单生效",
  "ResponseBlackFunctionID": "",
  "/*": "接收端文件路径配置, 当FileType 0时生效",
  "FilePath": "/home/lizong/LDP1.0.V202302.00.000/workspace/ldp_lz_front#0_ldp_front_0_backend_Singleton",
  "/*": "发送端文件路径配置, 当FileType 1时生效",
  "RecordDir": "/home/lizong/LDP1.0.V202302.00.000/workspace/",
  "/*": "上下文前缀, 当FileType 1时生效",
  "Prefix": "ldp_lz",
  "/*": "上下文, 当FileType 1时生效",
  "Context": "front#0_ldp_front_0_backend_Singleton",
  "/*": "topic 名称, 当FileType 1时生效",
  "Topic": "trade",
  "/*": "分区号, 当FileType 1时生效",
  "PartitionNo": 1,
  "/*": "消息结构配置文件/文件夹",
  "structCfg": "",
  "/*": "输出端配置, 0: 输出到文件; 1: 输出到标准输出; 2: 输出到标准错误",
  "outputDest": 0,
  "/*": "解析输出文件名配置, 仅在outputDest = 0时生效",
  "outputFileName": "./result/Test",
  "/*": "解析输出文件单独是否存储在以时间命名的文件夹中",
  "timeFolder": false,
  "/*": "解析输出文件, 单个文件大小的上限, 仅在outputDest = 0时生效, 单位: MB",
  "fileMaxSize": 100,
  "/*": "解析输出格式, 0: JSON; 1: CSV",
  "outputFormat": 1,
  "/*": "JSON格式化开关, false: 关闭; true: 打开, 仅在outputFormat = 0时生效",
  "jsonPretty": true,
  "/*": "写文件句柄打开模式, 0: 复用 (适用功能号偏多的情况); 1: 缓存 (适用提高写入效率的情况), 仅在outFileMerge = false时生效",
  "fdUseMode": 1,
  "/*": "数据缓存时间间隔, 未到缓存容量上限前, 每隔一定时间写线程fflush一次磁盘IO, 单位: 秒 (关闭缓存时填0)",
  "fflushInterval": 1,
  "/*": "数据缓存容量大小, 达到上限时, 写线程fflush一次磁盘IO, 单位: Bytes, 仅在fflushInterval > 0时有效。注: 1 MB=1048576 B",
  "cacheSize": 1048576,
  "/*": "原char类型数据, 是否按ASCII码进行显示: false: 字符形式, true: Ascii形式, 仅在outputFormat = 1时生效",
  "showCharToAscii": false,
  "/*": "解析输出文件, 是否合并输出到单个文件中, false: 按照功能号拆分文件, true: 合并输出到单个文件",
  "outFileMerge": false
}
```

配置含义如下:

配置项	说明
-----	----

FileType	文件类型	0: rcm接收端持久化; 1: rcm发送端持久化;
RequestWhiteFunctionID	请求白名单	使用;隔开, 如果请求黑名单存在, 那么请求白名单失效
ResponseWhiteFunctionID	回复白名单	使用;隔开, 如果回复黑名单存在, 那么回复白名单失效
RequestBlackFunctionID	请求黑名单	使用;隔开, 如果请求黑名单存在, 那么请求白名单失效
ResponseBlackFunctionID	回复黑名单	使用;隔开, 如果回复黑名单存在, 那么回复白名单失效
FilePath	rcm接收端持久化文件路径配置, 当FileType 0时生效	解析的是rcm_rcm**的文件
RecordDir	rcm发送端持久化文件路径配置, 当FileType 1时生效	解析的是rcm_rms**的文件
Prefix	上下文前缀(产品名), 当FileType 1时生效	启动脚本中可以查到
Context	上下文名, 当FileType 1时生效	zk配置文件中可以查看
Topic	topic 名称, 当FileType 1时生效	zk配置文件中可以查看
PartitionNo	分区号, 当FileType 1时生效	zk配置文件中可以查看
structCfg	消息结构配置文件/文件夹	
outputDest	输出端配置	0: 输出到文件、1: 输出到标准输出、2: 输出到标准错误
outputFileName	解析输出文件名配置	仅在outputDest = 0时生效
timeFolder	解析输出文件单独是否存储在以时间命名的文件夹中	
fileMaxSize	解析输出文件, 单个文件大小的上限	仅在outputDest = 0时生效, 单位: MB
outputFormat	解析输出格式	0: JSON、1: CSV
jsonPretty	JSON格式化开关	false: 关闭、true: 打开, 仅在outputFormat = 0时生效
fdUseMode	写文件句柄打开模式	0: 复用(适用功能号偏多的情况)、1: 缓存(适用提高写入效率的情况), 仅在outFileMerge = false时生效
flushInterval	数据缓存时间间隔	未到缓存容量上限前, 每隔一定时间写线程flush一次磁盘IO, 单位: 秒(关闭缓存时填0)
cacheSize	数据缓存容量大小	达到缓存容量上限时, 写线程flush一次磁盘IO, 单位: Bytes, 仅在flushInterval > 0时有效。注: 1 MB=1048576 B
showRedoHead	展示redo头信息开关	false: 关闭、true: 打开
redoFlagOpt	过滤事务类型	0: 分析全部事务, 1: 仅分析重放事务, 2: 仅分析回库事务, 3: 分析即重放又回库事务
showCharToAscii	原char类型数据, 是否按ASCII码进行显示	false: 字符形式, true: Ascii形式, 仅在outputFormat = 1时生效
outFileMerge	解析输出文件, 是否合并输出到单个文件中	false: 按照功能号拆分文件, true: 合并输出到单个文件

2.6.9. multicast_test

此工具(multicast_test)放置在发布包的/bin目录下

multicase_test工具主要用于测试下列问题: 两台机器之间的组播网络是否能够正常通讯; 是否存在组播连接建立3分钟后自动断开; 组播会话数超过一定个数后, 无法建立新的连接;

文件说明

```
multicast_test linux端可执行程序
multicast_test.exe windows端可执行程序
    附加依赖配置文件:
        无
    附加依赖库文件:
        无
```

使用说明

- 1.同时拷贝multicast_test放到待测试连通性的两台服务器上, 将multicast_test工具放在任意目录下皆可运行, 他不依赖除系统库外的其他库
- 2.linux端启动前先对multicast_test工具赋予可执行权限, 然后执行该程序, windows端直接双击multicast_test.exe程序即可, 不需要

命令行参数

- 3.启动后会要求选择作为发送端或者是接收端，在一台机器上输入0选择发送端，另外一台机器上输入1选择接收端
- 4.输入后会提示上次测试留下的配置信息，如下图。第一次启动输入n，重新配置组播地址、网卡和端口等信息。

```
slightrm test tool V3.0.1.0
-----
请选择 [0:发送端 1:接收端]:0
-----
组播地址: 235.0.0.10
端口: 29110
网卡1: 192.168.86.0
网卡2: 192.168.86.0
发送编号: 1
-----
是否使用上述组播地址, 请确认 (y/n):
```

- 5.按程序提示的要求依次输入组播地址，端口和测试用的网卡1和网卡2。接收端和发送端配置的组播地址和端口需要一样。(组播地址范围224.0.0.0~239.255.255.255) 注：网卡1和网卡2既可以配网段也可以配具体的IP地址，网卡1和网卡2用于双网卡测试，如果不存在双网卡，或者不使用双网卡时，网卡2可以配置任意无效的地址
- 6.发送端启动后总是每秒发送一条组播报文，接收端在建立组播通信后屏幕上会打印connect，并且提示收到了发送端发过来的组播包。
- 7.测试结束后,linux端直接Ctrl+C键关闭程序即可， windows直接关闭程序窗口即可
- 8.程序退出后，会在发送端生成一个名为rm_send.cfg的文件，保存了上次配置的发送端信息，下次测试或者测试过程中异常退出后，在 是否使用上述组播地址，请确认(y/n): 界面直接输入y可以直接复用之前的配置，而不需要再重新配置；同样在接收端也会生成一个名为rm_recv.cfg的文件，保存了上次配置的接收端信息，用于重启时直接复用上次的配置。

输入输出重定向

以下使用方法windows和linux同理，均适用。下面说明使用windows演示

- 1.准备一个给发送端的输入配置文件in_0.txt如下图：

```
in_0.txt - 记事本
文件(F)  编辑(E)  格式(O)  查看(V)  帮助(H)
0
n
239.0.0.9
30000
192.168.28.0
192.168.28.0
0
      此处需要有一个换行符
```

每一行分别代表了程序启动过程中要求输入的配置 第一行0表示发送端，第二行n表示不使用上一次启动留下的配置。第三行表示组播地址，第四行表示组播端口 第五行为网卡1的IP网段或者IP地址，第六行为网卡2的IP网段或者IP地址 第七行为发送端编号

- 2.再准备一个给接收端的输入配置文件in_1.txt如下：

```
in_1.txt - 记事本
文件(F)  编辑(E)  格式(O)  查看(V)  帮助(H)
1
n
239.0.0.9
30000
192.168.28.0
192.168.28.0
      此处需要有一个换行符
```

每一行的含义与发送端的输入配置相同

- 3.再使用如下命令启动

```
发送端
multicast_test.exe <in_0.txt >>out_0.txt
接收端
multicast_test.exe <in_1.txt >>out_1.txt
```

- 4.查看输入文件 out_0.txt

```
out_0.txt - 记事本
文件(F)  编辑(E)  格式(O)  查看(V)  帮助(H)

-----
slightrm test tool V3.0.1.0
-----
请选择[0:发送端 1:接收端]:
-----

组播地址: 235.0.0.10
端口: 29110
网卡1: 192.168.28.0
网卡2: 192.168.28.0
发送编号: 1
-----

是否使用上述组播地址, 请确认(y/n):请输入组播地址:请输入组播端口:请输入网卡1:请输入网卡2:请输入发送编号:
-----作为组播发送端-----

组播地址: 235.0.0.10
端口: 29110
网卡1: 192.168.28.0
网卡2: 192.168.28.0
发送编号: 1
-----

send data [send0]
send data [send1]
send data [send2]
send data [send3]
send data [send4]
send data [send5]
send data [send6]
send data [send7]
send data [send8]
send data [send9]
send data [send10]
send data [send11]
```

2.6.10. 机房信息修改工具

此脚本及脚本中调用的ldpZoneTool的可执行文件。用户可以执行当前工具，修改zk上的机房信息，同时可以同步给对应的节点。

示例: ldpZoneTool 产品名 -s zk服务端地址 -A zk鉴权信息 -f 配置文件 -c 4

命令行参数含义如下:

参数		说明
help	--help	如果输入的参数中存在-h或者—help，告知ldpZoneTool的用法和参数意义
产品名	productname	告知当前需要修改的产品名。必须填写
zk服务端地址	-s 192.168.86.101:1234	当前工具修改zk的服务端地址。必须填写
zk鉴权信息	-A UserAuth	同应用中ldp_zkhandler的UserAuth配置
超时时间	-t 20	当前工具修改zk,等待在线节点将自己的节点机房名改为正确。默认30秒，整型，单位秒
配置文件	-f 文件名	使用时，工具会去打开配置的json文件，并读取数据上传给zk。默认为：ZoneInfo.json
清理节点	--clean	清理ZoneInfo下的所有子节点，子节点下所有节点必须不在线才能完成清理。如果—clean后跟了紧跟force字段则无论是否有在线节点，都会清空
清理节点超时时间	-c 4	对于整个机房下，所有节点的连接超时时间。单位秒。默认3秒，如果3秒内，连接无法成功，则认为所有的节点都不在线，可以清理

配置文件说明:

```
{
  "ZoneInfo":
  [
    {
      "ZoneName": "Zone1",
      "Type": 1
    },
    {
      "ZoneName": "Zone2",
      "Type": 2
    },
    {
      "ZoneName": "Zone3",
      "Type": 2
    },
    {
      "ZoneName": "Zone4",
      "Type": 2
    },
    {
      "ZoneName": "Zone5",
      "Type": 3
    }
  ]
}
```

- 1. "ZoneName"是当前机房名。"Type"是指当前机房类型。1为主机房，2为同城机房，3为异地机房
- 2. 同一个json中不能存在相同的"ZoneName"。

3. 同一个json中"Type", 必须要有Type为1的机房, 且只能有一个。其他类型可以有多个

运行说明:

1. 执行脚本后, 生成的zk路径为:

---ldp/	zk根目录
---productname	产品名路径
---ZoneInfo	机房信息路径 (由当前工具创建)
---Zone1	Zone1机房: zk内容为 "Type":1 (由当前工具创建)
---xst#0	xst#0节点名: 节点内容为 "Type": 1, "MproxyIpAddr": ["192.168.86.102"],
"MproxyPort":12345 (由框架创建)	
---running	xst#0节点running节点: 是否在线判断。如果不存在running, 则去连接mproxy的ip端口去判断是否在线
(由框架创建)	
---front#0	front#0节点名: 节点内容为 "Type": 1, "MproxyIpAddr": ["192.168.86.102"],
"MproxyPort":12345 (由框架创建)	
---Zone2	Zone2机房: 节点内容为 "Type":2 (由当前工具创建)
---Zone3	Zone3机房: 节点内容为 "Type":2 (由当前工具创建)
---Zone4	Zone4机房: 节点内容为 "Type":2 (由当前工具创建)
---Zone5	Zone5机房: 节点内容为 "Type":3 (由当前工具创建)

1. 执行脚本, 修改zk上的机房信息时。如果将原机房类型从主机房降为同城或者异地, 或者将原机房从同城降为异地, 则先确定当前机房下是否存在有节点在线, 如果在线, 则无法修改。(机房动态下调等级, 需要当前机房下无在线节点)
2. 执行脚本, 修改zk上的机房信息时。如果将原机房类型从异地升级为同城或者主机, 或者将原机房从同城升级为主机房, 可以直接修改机房类型(机房动态升级, 无需考虑当前机房有无在线节点)
3. 当修改机房成功后, 会等在线的节点把 xst#0内的Type节点改为和Zone1的Type一致, 才算正常执行结束。如果超时时间不配置, 则默认30秒
4. 执行清理时, 如果ZoneInfo节点下存在运行的子节点, 则删除失败, 一个都不会删除。否则删除全部ZoneInfo节点下的子节点
5. 如果修改机房名, 需要执行清理后, 才能上传最新的配置

2.6.11. locate规则操作工具

此脚本及脚本中调用的ldpLocateTool的可执行文件。用户可以执行当前工具, 上传对应zk上的locate规则, 下载对应zk上locate规则

示例: ldpLocateTool -s zk服务端地址 -p /cate/zkRules -l load.json -f update.json -r NodenoStatus

命令行参数含义如下:

参数		说明
help	--help	如果输入的参数中存在-h或者--help, 告知ldpLocateTool的用法和参数意义
zk服务端地址	-s 192.168.86.101:1234	当前工具修改zk的服务端地址。必须填写
zk上locate规则路径	-p /cate/zkRules	-p为zk上的规则绝对路径, 必须填写。如果zk上没有, 则会创建。
更新文件	-f 文件名	如果存在-f, 则工具会扫描文件, 将文件中的规则上传到zk, 如果文件中规则不对, 则上传失败
下载文件	-l 文件名	如果不存在-f, 只存在-l, 则是load模式, 会将当前zk下的规则路径保存到对应的文件中
规则名	-r 规则名	如果不存在-r, 则全量规则替换, 如果增加-r则指定规则替换, 且需要保证更新文件存在该规则

2.6.12. ldp集群工具ldpClusterTool

工具(ldpClusterTool)放置在发布包的/bin目录下

此工具适用于 Rcm插件 开启是否仅主中心活跃模式(OnlyMainZoneActive)的模式时使用。

此模式下只有主机房会产生主机(ACTIVE)与备机(INACTIVE), 其他机房均为SYNC同步状态。详细说明见Rcm插件的OnlyMainZoneActive配置说明

工具还支持回库在冷备模式的灾备切换时, 与核心事务号比较后更新回库进度, 解决机房切换后回库与核心序号不一致的问题。

ldpClusterTool工具支持的操作指令有:

set: 设置切换集群的主机房, 主机只会在主机房中产生

check_steady: 检查集群是否达到稳态(至少存在一主ACTIVE和一备机INACTIVE)

check_all_ready: 检查集群中所有成员是否就绪(只有一个节点且为SINGLE或者有一个ACTIVE其余均为INACTIVE)

update_todb: 与核心事务号比较后更新(如果备机房核心集群的最小事务号小于回库的最小事务号, 则将其更新为新的回库最小事务号)

文件说明

ldpClusterTool_Solution.json: 当前工具输出错误码对应的处理措施配置文件, 若没有当前文件则会输出错误码"ErrorNo": 1, 对应措施"Solution": "检查配置文件ldpClusterTool_Solution.json 是否存在"
ldpClusterTool linux端可执行程序 附加依赖配置文件: ldpClusterTool_Solution.json // 当前 ldpClusterTool_Solution.json 配置文件应在 ldpClusterTool 可执行文件同名目录下 附加依赖库文件: 无

使用说明

1.set的使用说明

将某一个机房设置为集群的主机房, 主机只会在主机房中产生。

使用前提条件:

- 1.原主机房中的所有节点全部已经关闭。

- 2.目标机房跟原主机的机房不是同一个机房
- 3.zk上只有目标机房Zone的节点。
- 4.所有节点必须配有机房信息

示例:

```
ldpClusterTool set [product_name] [cluster_name_list] [main_zone] -s [Ip:Port] [-A zk鉴权信息]
```

参数顺序不能调换

product_name 要设置的目标产品名

cluster_name_list 要设置的的集群名列表，可以多个，多个之间使用英文逗号隔开

main_zone 设置为集群主机房的机房名称

-s ip:port zk的服务器地址，可以多个，多个之间使用英文逗号隔开

-A zk鉴权信息，同应用中ldp_zkhandler的UserAuth配置

执行后会在屏幕上打印如下内容

```
{
  "ErrorNo": 0,
  "ErrorInfo": "",
  "Solution": ""
}
```

如果失败则ErrorNo不为0，并且ErrorInfo中会有失败的信息。

2.check_steady的使用说明

检查集群中是否至少存在一个主机(ACTIVE)和一个备机(INACTIVE)，达到稳态。

使用前提条件：无

示例:

```
ldpClusterTool check_steady [product_name] [cluster_name_list] -s [Ip:Port] -t [timeout] [-A zk鉴权信息]
```

参数顺序不能调换

product_name 要检查的目标产品名

cluster_name_list 要检查的集群名列表，可以多个，多个之间使用英文逗号隔开

-s ip:port zk的服务器地址，可以多个，多个之间使用英文逗号隔开

-t timeout 表示检查的超时时间，最长检查等待的时间，还有集群没有达到稳态则返回结果，如果所有集群都稳态了则会立即返回结果

-A zk鉴权信息，同应用中ldp_zkhandler的UserAuth配置

执行后会在屏幕上打印如下内容

```
{
  // "ReadyNodes"就绪的节点列表，当IsAllClusterSteady为false并且程序没有执行出错的话，此处才会有内容，具体内部结构参考
  check_all_ready的使用方法
  "ReadyNodes": [],
  // "ErrorNo"表示执行是否有出错，0表示执行成功，其他表示出错
  "ErrorNo": 0,
  // "IsAllClusterSteady"当要检查的所有集群名都是稳态了，则返回true，只要任意一个集群没有达到稳态，则为false，同时在ReadyNodes里展示
  已经就绪的节点。
  "IsAllClusterSteady": true,
  // "ErrorInfo"如果ErrorNo不为0，则此处会输出错误信息
  "ErrorInfo": "",
  // "Solution"输出当前ErrorNo对应的Solution，如果当前ErrorNo没有对应的Solution，则会输出“未获取到当前错误对应的建议操作”
  "Solution": ""
}
```

3.check_all_ready的使用说明

检查集群中是否所有节点都已经准备就绪。

满足就绪条件：

- 1.集群中有1个单机(SINGLE)节点，且没有任何其他节点。
- 2.集群中只有一个主机(ACTIVE),并且其他节点均为(INACTIVE)

使用前提条件：无

示例:

```
ldpClusterTool check_all_ready [product_name] [cluster_name_list] -s [Ip:Port] -t [timeout] [-A zk鉴权信息]
```

参数顺序不能调换

product_name 要检查的目标产品名

cluster_name_list 要检查的集群名列表，可以多个，多个之间使用英文逗号隔开

-s ip:port zk的服务器地址，可以多个，多个之间使用英文逗号隔开

-t timeout 表示检查的超时时间，最长检查等待的时间，还有某个集群的某个节点没有就绪则返回结果，如果所有集群的所有节点都就绪了则会立即返回结果

-A zk鉴权信息，同应用中ldp_zkhandler的UserAuth配置

执行后会在屏幕上打印如下内容

```
{
  // "ReadyNodes"当本工具没有执行出错的时候，此字段内才会有内容
  "ReadyNodes": [
    {
      // "Cluster"集群名
      "Cluster": "xst",
      // "Nodes"集群下的已经就绪的节点，如果没有就绪的节点不会出现在此处
      "Nodes": [
        "xst#2",
        "xst#1"
      ]
    } // 如果入参集群有多个，则此处有多个。
  ],
  // "ErrorNo"表示执行是否有出错，0表示执行成功，其他表示出错
  "ErrorNo": 0,
  // "ErrorInfo"如果ErrorNo不为0，则此处会输出错误信息
  "ErrorInfo": "",
  // "Solution"输出当前ErrorNo对应的Solution，如果当前ErrorNo没有对应的Solution，则会输出“未获取到当前错误对应的建议操作”
  "Solution": ""
}
```

4.update_todb的使用说明

机房切换，更新回库最小事务号时使用，目前包含两种场景：同城机房切换、异地机房切换。

同城机房切换：与核心事务号比较后更新，如果备机房核心集群的最小事务号小于回库的最小事务号，则将其更新为新的回库最小事务号。

使用前提条件：

- 1.原主机房中的所有节点全部已经关闭
- 2.备机房对应的核心集群中至少有一个节点在运行
- 3.须在同城机房ZK从观察者集群切换为主集群后执行，因为观察者无法执行写操作
- 4.须在开启备机房所有核心集群选举前执行，保证命令的重复执行是幂等的

异地机房切换：运维获取异地机房各核心集群的最小事务号后，通过-c参数输入，该入参设置的集群最小事务号将被指定为对应的回库最小事务号。

使用前提条件：

- 1.原主机房中的所有节点全部已经关闭
- 2.备机房对应的核心集群中至少有一个节点在运行
- 3.异地机房全部核心都就绪
- 4.切换机房信息前执行

指令用法示例：

```
ldpClusterTool update_todb product_name todb_cluster -s [Ip:Port] -c [ClusterName:TransNo] [-A zk鉴权信息]
```

指令说明：

- 1.除选项参数外，其余参数顺序不能调换

update_todb 操作指令

product_name 目标产品名

todb_cluster 回库的集群名，只支持填一个

-s Ip:Port zk的服务器地址，可以多个，多个之间使用英文逗号隔开

-c ClusterName:TransNo 指定核心集群名和对应的最小事务号，多个之间用英文逗号分隔。例如：mdb:500,xst:800

-A zk鉴权信息，同应用中ldp_zkhandler的UserAuth配置

2.-c参数说明

如果是同城机房切换，不能设置-c参数

如果是异地机房切换，必须设置-c参数

如果设置了-c参数，工具将只根据该入参作为核心集群的最小事务号并且只处理入参中指定的核心集群，否则工具将自行查阅回库各旁路对应的核心集群最小事务号。

执行后会在标准输出(stdout)打印如下内容

```
{
  "ErrorNo": 0, // 0表示成功，否则表示错误号，一般表示命令行入参校验或内部资源初始化失败等
  "ErrorInfo": "", // 错误信息
  "Solution": "", // 针对错误的操作建议
  "IsAllSuccess": false, // 是否收取的全部核心的todb事务号更新成功（无需更新的也视为成功）
  "SuccessClusters": [ // 成功更新事务号的对应核心集群
    {
      "SourceCluster": "mdb", // 发送回库数据的集群
      "MinTransNo": 100 // 最新的事务号
    },
    {
      "SourceCluster": "xst",
      "MinTransNo": 100
    }
  ],
  "FailClusters": [ // 列出执行失败的对应核心集群，遇到执行失败的集群后，会继续尝试执行下一个集群
    {
      "Bypass": "redo_ws_1", // 发送旁路的主题分区信息
      "SourceCluster": "mdb2", // 发送回库数据的集群
      "ErrorNo": 1, // 错误号
      "ErrorInfo": "connect timeout", // 错误信息
    }
  ]
}
```

- 1.如果是整体性报错，只显示"ErrorNo"、"ErrorInfo"和"Solution"字段，不是每次返回都会包含所有字段

2.回库更新多个核心集群时，一个核心集群执行失败不影响下一个集群执行

3.如果"IsAllSuccess"为true表示所有核心集群都完成了事务号比较后更新，否则观察"FailClusters"中的失败节点和具体的错误信息

4.同城机房切换，在开启备机房所有核心集群选举前，多次执行命令是幂等的

5.如果-c参数中指定的核心不存在，将被视为处理失败

2.6.13. rcm配置修改工具rcmcf_change_tool

工具放置在发布包的/bin目录下，用于修改应用启动过程中生成的动态 RCM 配置， 目前仅能支持修改如下 RCM 配置信息

配置	配置项说明
RCM 上下文 Ip	需加入组播的IP地址（网卡），可以配置网段
RCM 上下文 SyncIp	集群同步网卡地址；用于接收集群同步的IP地址（网卡），可以配置网段
RCM 上下文 SyncAddr	集群同步地址；用于发送集群同步信息的地址，可以配置组播地址或多个单播地址，地址间以逗号分隔。单播模式时，通常配置为每个集群成员的IP。
RCM 上下文 Bypass/Ip	旁路发送网卡地址；旁路发送使用的本地IP地址（网卡），可以配置网段。允许和通常收发使用不同网卡。默认值：不配置时自动使用通常消息收发的IP。
RCM 主题的 Addr	通讯地址；可以配置为组播地址、单个ip地址、多个ip地址。多个地址以逗号分隔，若第一个地址为单播地址，则后面存在的组播地址无效。通常为目的地地址。

工具的行为说明

1. 从配置服务器上获取当前的配置
2. 获取并解析 RCM 配置
3. 根据节点名（或应用实例名）查找需要修改的 RCM 上下文配置，
4. 校验 RCM 上下文Ip、SyncIp、Bypass/Ip配置项是否相同，如果不相同，认为不是动态生成的 RCM 上下文，不进行修改
5. 校验 RCM 上下文Ip配置项与传入的节点地址是否相同，如果相同，则不执行配置修改
6. 如果配置需要修改，则将Ip、SyncIp、Bypass/Ip配置项修改为节点地址，同时如果SyncAddr为单播地址，则将其其中与原Ip相同的部分替换为节点地址
7. 对 RCM 上下文中的RxTopics的Addr进行修改
8. 如果配置发生变化，则将修改后的配置重新上传至配置服务器
9. 工具同时会将原始配置和修改后的配置保存在本地，文件名分别为YYMMDD_HHMMSS_hsl dp_old.json 和YYMMDD_HHMMSS_hsl dp_new.json

工具用法

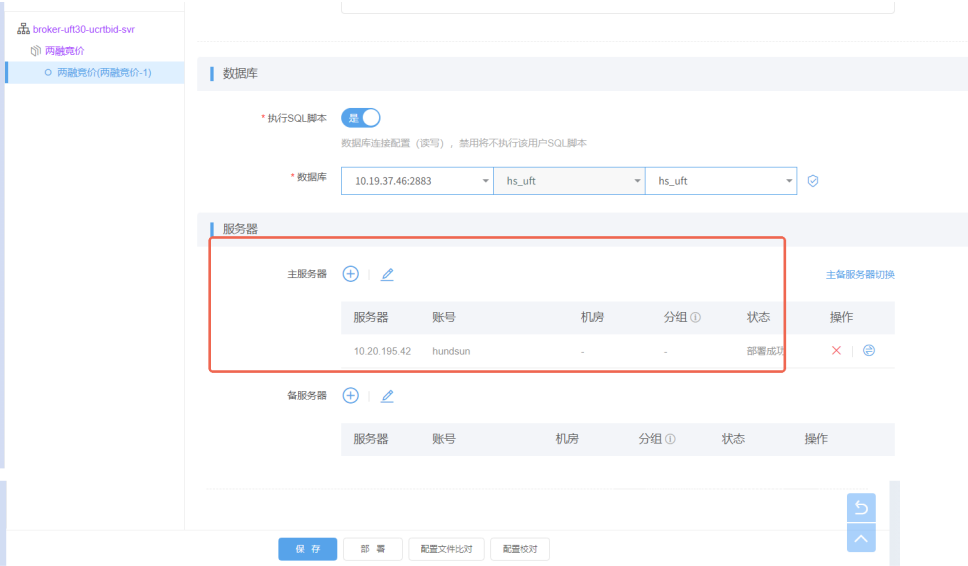
```
rcmcf g_change_tool 产品名 应用名#编号 节点部署地址 配置服务器地址 -h -L
```

命令行参数含义如下:

参数		说明
产品名		长度不超过32
节点名	应用名#编号	应用名长度不超过32，编号不小于0
节点地址		节点部署的机器地址
配置服务器地址	ip:port	目前支持ZooKeeper，不指定则使用本地配置
-L		是否启动宽松模式，宽松模式下，工具修改配置时不会校验节点的运行状态和主备身份状态
-h		显示帮助信息

工具使用场景和限制说明

目前主要使用在应用实例(或节点)更换机器重新部署的场景



该使用场景有如下特点

1. RCM 动态配置不会删除

- 2. 应用实例名（或节点名）不会发生改变，上一次运行生成的动态 RCM 配置信息在配置服务器上有记录
- 3. 应用实例运行的机器IP发生变化
- 4. 可以同时更改多个应用实例的部署机器
- 5. 点击部署按钮后，应用实例会被自动运行起来
- 6. 应用实例更换机器部署后再次启动，实际上表现为清流重启

使用限制说明如下

- 1. 目前对于 RCM 多进程上下文主备场景下，如果主节点清流重启，各节点带流重启，会出现协商失败的问题。因此操作上需要确保更换机器的应用实例不是集群的主节点
- 2. 为了减小修改配置对产品下其他应用的影响，确保在产品下所有应用都处于停止状态下使用

see 发布包制作说明

需要在应用的部署包对应的afterinstall脚本中写入如下命令

```
# 根据传入的产品名、节点名、节点IP地址、zk地址进行RCM动态配置修改
rcmcfp_change_tool ${product_name} ${t_zone_prefix}${sys_name} ${name_suffix} ${biz_sharding_no} ${biz_server_no}
${primary_ip_address} ${svc_zookeeper_connect_string} -L
```

由于通过see部署会自动将应用实例启动起来，因此需要指定在宽松模式(指定-L参数)下使用该工具，且部署成功执行一次应用的重启动作

2.6.14. ldp_coreparsertool

此工具(ldp_coreparsertool)放置在发布包的/bin目录下。此工具只解析核心节点中，因业务代码崩溃导致的coredump文件，方便用户排查问题。

注意：该工具依赖t2sdk独立库(libt2sdk.so)，用户需要在Cres的发布包中下载该so文件，将其放到LDP发布包/bin目录下，并额外设置环境变量LD_LIBRARY_PATH库路径为LDP发布包/bin目录。

使用方法:

```
ldp_coreparsertool -c 核心转储文件名完整路径 -p 指定解析结果文件夹路径 -f 显示的字段 -l 日志等级 -d cfg文件路径或文件名
#eg: ldp_coreparsertool -c "/home/uft30/ucrtbid52/workspace/core.60427" -p "./coreparser_result" -f
"fund_account,Name" -l 0 -d "struct_demo.cfg"
```

配置含义如下:

命令行参数	含义	说明
-c	核心转储文件名完整路径	必填，coredump文件的完整路径
-p	指定解析结果文件夹路径	选填，日志和解析结果将会保存在该文件夹中，默认为./coreparser_result
-f	需要展示的字段	选填，支持多个，以英文逗号分隔，不配置时全展示
-l	日志等级	选填，设置日志等级，0=debug, 1=INFO, 2=WARN, 3=ERROR, 4=IMP, 5=FATAL，默认为1
-d	指定cfg文件路径	可选，指定消息结构的配置文件（.cfg）。若提供该参数，程序将依据配置文件中定义的功能号及其对应的请求结构来解析codump文件。不配置此配置项时默认按照ldp-pack来解析codump文件。

执行后会在标准输出(stdout)打印精简版解析结果，按照ldp-pack来解析codump文件只展示-f参数指定的消息体字段:

```
{
  "///": "core dump文件名",
  "FileName": "/home/uft30/ucrtbid52/workspace/core.87869",
  "///": "解析结果",
  "ParserResult": "Success",
  "///": "线程信息",
  "ThreadInfos": [
    {
      "///": "线程id",
      "ThreadId": 1,
      "///": "是否是崩溃线程",
      "IsCrashThread": true,
      "///": "功能号",
      "FunctionNo": 910011,
      "Body": [
        {
          "fund_account": "999"
          "Name": "777"
        }
      ]
    }
  ]
}
```

配了-d按照struct请求解析codump文件时不支持-f参数指定消息体字段

```
{
  "核心dump文件名": "core dump文件名",
  "FileName": "/home/uft30/ucrtbid52/workspace/core.150509",
  "核心dump文件创建时间, 年月日 hh:mm:ss": "2025-07-28 15:21:06",
  "解析结果",
  "ParserResult": "Success",
  "线程信息",
  "ThreadInfos": [
    {
      "线程id",
      "ThreadId": 1,
      "是否是崩溃线程",
      "IsCrashThread": true,
      "功能号",
      "FunctionNo": 5,
      "Body": {
        "TAG_REQ_ACCOUNT_REGISTER": [
          {
            "szFundAccount": "usr00000",
            "szPassword": "pwd00000",
            "szZero": "",
            "TotalFee": "100.000000",
            "ReportFee": "200.000000",
            "structData": [
              {
                "strData": "optstruct"
              }
            ],
            "OptionNestingLdpmsgName": {
              "OptionLdpMsg": [
                {
                  "iData1": "1",
                  "strData2": "OpldpMsgData"
                }
              ]
            }
          ],
          "DataSet1": [
            {
              "menoy": "3000.00000000",
              "strInfo": "datasetinfo"
            }
          ]
        }
      ]
    }
  ]
}
```

1. 解析后的完整版结果将会默认保存在工作路径下的coreparser_result文件夹中。
2. 完整版解析结果的文件命名规则为`<core文件名>-<解析时间>-parser-result.json`，例如`core.83614-20250728-191256-ParserResult.json`。
3. 完整版解析结果内容如下(ldp-pack的codump文件):

```
{
  "核心dump文件名",
  "FileName": "/home/uft30/ucrtbid52/workspace/core.87869",
  "核心dump文件创建时间, 年月日 hh:mm:ss",
  "CreateName": "2025-07-28 15:21:06",
  "解析时间, 年月日 hh:mm:ss.SSSSSus",
  "ParserTime": "2025-07-28 15:56:26.232573",
  "启动命令行",
  "StartCommandLine": "hsl dp startdaemon uft30 mdb#0 -s 192.168.74.41:4245",
  "解析结果",
  "ParserResult": "Success",
  "ThreadInfos": [
    {
      "线程id",
      "ThreadId": 1,
      "是否是崩溃线程",
      "IsCrashThread": true,
      "功能号",
      "FunctionNo": 910011,
      "消息类型",
      "MsgType": "Request",
      "消息长度",
      "MsgLen": 110,
      "去主去备标识",
      "Dest": "ToMaster",
      "扩展头长度",
      "ExtHeadLength": 0,
      "节点号",
      "NodeNo": 0,
      "固定字段长度",
      "FixedSize": 78,
      "系统号",
      "SystemNo": 0,
      "令牌标识",
      "Token": 0,
      "连接号",
      "ConnectionID": 1207959552,
      "用户自定义",
      "UserDefined": 0,
      "核心工作线程号",
      "BizThreadNo": 0,
      "Body": [
        {
          "fund_account": "999",
          "branch_no": "888",
          "Name": "777"
        }
      ]
    }
  ]
}
```

完整版解析结果内容如下(struct的codump文件):

```
{
  "fileName": "core_dump文件名",
  "FileName": "/home/uft30/ucrtbid52/workspace/core.150509",
  "createName": "2025-12-11 09:59:09",
  "ParserTime": "2025-12-11 09:59:34.480592",
  "StartCommandLine": "hsl dp startdaemon uft30 mdb#0 -s 192.168.86.102:3333",
  "ParserResult": "Success",
  "ThreadInfos": [
    {
      "ThreadId": 1,
      "IsCrashThread": true,
      "FunctionNo": 5,
      "MsgType": "Request",
      "MsgLen": 110,
      "Dest": "ToMaster",
      "ExtHeadLength": 0,
      "NodeNo": 0,
      "FixedSize": 584,
      "SystemNo": 0,
      "Token": 0,
      "ConnectionID": 1073807360,
      "UserDefined": 0,
      "BizThreadNo": 0,
      "Body": {
        "TAG_REQ_ACCOUNT_REGISTER": [
          {
            "szFundAccount": "usr00000",
            "szPassword": "pwd00000",
            "szZero": "",
            "TotalFee": "100.000000",
            "ReportFee": "200.000000",
            "structData": [
              {
                "strData": "optstruct"
              }
            ],
            "OptionNestingLdpmsgName": {
              "OptionLdpMsg": [
                {
                  "iData1": "1",
                  "strData2": "OptLdpMsgData"
                }
              ]
            }
          }
        ],
        "DataSet1": [
          {
            "menoy": "3000.00000000",
            "strInfo": "datasetlinfo"
          }
        ]
      }
    }
  ]
}
```

2.6.15. ldp_bizproc_tool

该工具用于解析bizproc节点在运行过程中产生的中间文件，方便后续排查问题。

用法:

```
ldp_bizproc_tool 产品名 应用名[#编号] -t 执行的工作类型 -f 核心配置文件名 -l 日志等级 -p 结果输出方式 -o 结果输出文件名 -w 核心节点的工作路径
```

示例:

```
ldp_bizproc_tool ldptest xst#0 -f ldptest.json -t excep -l 1 -p 1 -o ldp_bizproc_tool_out.json -w ../workspace
```

注意：该工具依赖于ldp_os库(libldp_os.so)，用户需要额外设置环境变量LD_LIBRARY_PATH库路径，默认为发布包/bin/plugins目录下。-t=excep模式下，依赖核心节点机器的/dev/shm目录下的核心写入的共享内存文件和核心节点工作路径下的*_Exception异常信息记录文件。

命令行参数含义如下:

参数	说明
产品名	长度不超过32
应用名	长度不超过32
编号	不小于0，可省略，缺省为0
-t	执行的工作类型 可选，当前仅支持'excep'，表示解析跳过消息信息，缺省为'excep'

参数	核心配置文件名	必填, 支持绝对路径或相对路径 说明
-l	0~5	可选, 日志等级: 0=debug, 1=INFO, 2=WARN, 3=ERROR, 4=IMP, 5=FATAL. 缺省为1
-p	0/1	可选, 结果输出方式: 0=输出到控制台, 1=输出到文件, 缺省为0
-o	结果输出文件名	可选, 当-p为1时生效, 缺省为`ldp_bizproc_tool_out.json`
-w	核心节点的工作路径	必传。核心节点的工作路径, 支持完整路径或相对路径。

输出说明:

1. 工具日志固定生成在执行路径下的`./log`目录。
2. -t=excep模式下, 输出结果为json格式, 可能包含以下字段:

字段名	含义
Type	异常消息类型
Count	Details数组长度
TotalLength	Details下所有Length的总和
TxContextId	发送端所属的上下文id
TransportId	发送端主题id
TxMsgNo	发送端消息编号
Identify	标识 (仅部分类型可能输出)
Length	字段长度

-t=excep模式下, 输出结果举例:

```
{
  "ExceptionMsgs": [
    {
      "Type": "RcmMsgSqn",
      "Count": 3,
      "TotalLength": 29,
      "Details": [
        {
          "TxContextId": 1,
          "Length": 2
        },
        {
          "TransportId": 3,
          "Length": 2
        },
        {
          "TxMsgNo": 200,
          "Length": 8
        }
      ]
    }
  ]
}
```

2.6.16. parse_skipped_msg.sh

该脚本用于一键解析核心在EnableSafeMode模式下的重放过程中跳过的消息。脚本会先调用ldp_bizproc_tool生成跳过消息的过滤条件文件, 再调用ldpMsgParser并传入过滤条件文件, 只输出命中的消息。

依赖:

1. ldp_bizproc_tool可执行文件 (发布包/bin目录), 版本大于等于V1.2501.2.0。
2. ldpMsgParser可执行文件 (发布包/bin目录), 版本大于等于V1.2501.2.0。
3. ldpMsgParser配置文件 (如ParserConfig.json, 通常放在发布包/workspace目录)。配置方法见《2.6.8.ldpMsgParser》章节。
4. ldp_bizproc_tool依赖libldp_os.so, 需设置环境变量LD_LIBRARY_PATH库路径包含发布包/bin/plugins目录。
5. ldpMsgParser依赖libt2sdk.so、librcm.so, 需在Cres发布包中下载libt2sdk.so放到LDP发布包/bin目录, 并设置LD_LIBRARY_PATH库路径包含发布包/bin目录和/bin/ibase/lib目录。
6. 依赖核心节点机器的/dev/shm目录下的核心写入的共享内存文件和核心节点工作路径下的*_Exception异常信息记录文件。

用法:

```
sh parse_skipped_msg.sh 产品名 应用名[#编号] 核心节点的工作路径 BizProc核心配置文件 MsgParser配置文件
```

示例:

```
sh parse_skipped_msg.sh ldptest xst#0 ../workspace ./conf/ldptest.json ../workspace/ParserConfig.json
```

说明:

1. 脚本内部固定使用`./ldp\_bizproc\_tool\_out.json`作为ldp_bizproc_tool的输出结果文件, 并将该文件作为ldpMsgParser的过滤条件输入。
2. 当脚本入参不完整时, 会输出Usage并退出。

3. 目前仅支持解析设置过唯一标识或者来自于高可靠上下文的消息，有关其他限制建议查看《异常消息跳号说明》章节。

注意：

目前执行该脚本之前需要保证崩溃节点所在机器不重启，且崩溃的核心重启，否则会因为rcm的持久化没有完成而导致解析出来的消息出现遗漏。

2.7. 线程资源

2.7.1. 线程资源分类

LDP平台内部线程资源分为两类：

- 独占类：此类线程要求运行时单独占用一个核，不允许和其他线程共享核。此类线程对性能的要求较高，属于关键线程。
- 共享类：此类线程可以允许和多个共享类线程一起使用一个核，此类线程对性能的要求不高，属于非关键线程。

2.7.2. LDP平台线程资源管理

LDP平台支持线程绑核与线程命名。

对于共享类线程的管理，通过配置启动命令“-c”选项，设置共享类线程绑核，详细参数请参考《启动应用命令行》章节内容。共享类线程将按照共享核列表轮流绑核，即使共享类线程数量超过共享核列表数量，共享线程也是按照共享核列表轮流绑核。

对于独占类线程的管理，通过修改配置文件，设置独占类线程组以及默认线程的绑核。详细参数请参考《系统配置》章节内容。独占类线程按照独占配置绑核，如果该线程组的独占核用完，则使用默认独占核，如果默认独占核也用完，则不绑核。

2.7.3. LDP平台绑核逻辑梳理

共享核

- 线程注册为Share生效，通过启动命令hsldp “-c”选项设置
 - 有hsldp -c，则跟随hsldp -c列表中的某个核
 - 没有hsldp -c，则不绑核（即跟随父线程的核），例如核心工作线程创建报盘的工作线程，则报盘工作线程跟着核心工作线程的核

独占核

- 线程注册为Exclusive生效，配置独占核：Thread
 - 如果配置了Thread，若线程名命中，则取对应列表中的核独占，对应列表核不够时，则不绑核；如果未命中，则取默认列表中的核独占，默认列表核不够时，不绑核
 - 如果未配置Thread，则不绑核

独占核不够时跟随主线程

- 线程注册为ExclusiveWithMain生效，配置独占核：Thread
 - 如果配置了Thread，若线程名命中，则取对应列表中的核独占，对应列表独占核不够时，则找默认的独占核列表，若没有找到默认的独占核列表或者默认独占核不够时，跟随主线程的核
 - 如果未配置Thread，则不绑核

独占核不够时轮流绑独占核

- 线程注册为ExclusiveWithShare生效，配置独占核：Thread
 - 如果配置了Thread，若线程名命中，则共享这一组核，轮询绑核
 - 如果未配置Thread，则不绑核

主线程的核

- 启动命令有hsldp -c，则跟随hsldp -c中的某个核
- 启动命令无hsldp -c，则系统默认或者taskset -c

核心绑核

- 独占核（PollingLoop非0时生效）
 - 如果核心插件配置了绑核（ThreadBindCpu）,未配置Thread,则以ThreadBindCpu核为主
 - 如果核心插件未配置绑核（ThreadBindCpu）,配置了Thread,则以Thread核为主
 - 如果核心插件配置了绑核（ThreaBindCpu）,也配置了Thread，则以Thread核为主
- 共享核（PollingLoop为0时生效）
 - 启动命令有hsldp -c，则跟随hsldp -c中的某个核，ThreadBindCpu失效
 - 启动命令无hsldp -c,如果ThreadBindCpu配置了，则以ThreadBindCpu核为准，未配置跟随主线程的核

2.7.4. 平台线程汇总

hsldp

线程名称	属性	说明	备注
hsldp	共享	hsldp主线程	

ldp_bizproc

线程名称	属性	说明	备注
bizproc_worker	独占/共享	核心工作线程	当PollingLoops大于0，为独占；等于0为共享
bizproc_pipe	独占/共享	核心流水线线程	当PollingLoops大于0，为独占；等于0为

			共享
bizproc_multi	独占/共享	核心多线程框架	当PollingLoops大于0，为独占；等于0为共享
bizproc_timer	共享	核心定时线程	
bizproc_copy	共享	核心redo拷贝线程	
bizproc_replay	共享	核心redo重放线程	
GlobalQueue_Run	共享	核心可靠队列处理线程	

lib_tcp

线程名称	属性	说明	备注
由使用者决定，默认tcprun，一般为：模块_tcprun	由使用者决定，默认独占	收发合并线程	
由使用者决定，默认tcpsend，一般为：模块_tcpsend	由使用者决定，默认独占	发送线程	
由使用者决定，默认tcprecv，一般为：模块_tcprecv	由使用者决定，默认独占	接收线程	
由使用者决定，默认tcpdma，一般为：模块_tcpdma	由使用者决定，默认独占	rdma接收线程	
由使用者决定，默认tcpcheck，一般为：模块_tcpcheck	由使用者决定，默认共享	检测线程	

ldp_front

线程名称	属性	说明	备注
front_fctime	共享	前置流控时间线程	

ldp_clean

线程名称	属性	说明	备注
clean_run	共享	清理流程管理线程	

ldp_logproxy

线程名称	属性	说明	备注
logproxy_run	共享	日志队列处理线程	

ldp_mproxy

线程名称	属性	说明	备注
mproxy_worker	共享	管理代理工作线程	
mproxy_master	共享	管理代理master线程	

ldp_todb

线程名称	属性	说明	备注
todb_syncredo	共享	回库同步redo线程	
todb_timer	共享	回库定时线程	
todb_send	共享	回库异步发送线程	

arbserver

线程名称	属性	说明	备注
arb_elect	共享	选举线程	
arb_check	共享	主机状态检测线程	
arb_watch	共享	监控线程	

ldp_offerproc

线程名称	属性	说明	备注
offerproc_worker	独占/共享	报盘工作线程	可能在进程运行中动态创建和销毁，线程拥有者为ldp_offerproc_instance_offerindex, instance

			为实例编号，offerindex为报盘序号， 如：ldp_offerproc_0_10
offerproc_timer	共享	报盘定时线程	

ldp_zkhandler

线程名称	属性	说明	备注
zk_io	共享	ZK库网络通信读写线程	
zk_completion	共享	ZK库事件回调处理线程	

ldp_cres_agent

线程名称	属性	说明	备注
cres_main	共享	cres主线程	

ldp_front_proxy

线程名称	属性	说明	备注
fproxy_Timer	共享	前置代理定时线程	

ldp_rcm

线程名称	属性	说明	备注
rcm_tx_send	共享	发送端异步发送线程	
rcm_tx_repair	共享	发送端补缺线程	
rcm_tx_record	共享	发送端持久化线程	
rcm_bp_tx_send	共享	旁路发送端发送线程	
rcm_bp_tx_recv	共享	旁路发送端补缺线程	
rcm_rx_record	共享	接收端持久化线程	
rcm_rx_sqn_sync	共享	接收端集群同步发送线程	
rcm_rx_recv	独占	接收端接收线程	
rcm_arb_elect	共享	仲裁线程	
rcm_ctx_check	共享	多进程上下文检测线程	
rcm_ctx_metric	共享	多进程上下文度量线程	
rcm_sync_recv	独占	多进程上下文接收线程	
rcm_sync_record	共享	多进程上下文持久化线程	
rcm_sync_send	独占	多进程上下文异步发送线程	
rcm_sync_repair	共享	多进程上下文补缺线程	
unix_domain_server	共享	Unix域服务端工作线程	
unix_domain_client	共享	Unix域客户端工作线程	

2.8. 平台配置项加密

2.8.1. 密文格式说明

加密后的配置字符串格式为 SE1(base64)

2.8.2. 加密流程说明

首先将明文通过sm4进行加密，然后将加密的密文通过base64再转换成可见字符串，然后使用 SE1() 包裹，得到最终的密文

目前加密是通过see进行加密，平台不提供加密工具

2.8.3. 解密流程说明

平台自动识别配置项是否被 SE1() 包裹，如果是则按照加密流程解密后再使用配置项，否则认为是明文配置项直接使用

目前只支持了zk插件的Auth配置项和 bizproc插件中mdbdata的数据库PassWord配置项

2.9. 平台机房信息设置

2.9.1. 启动命令和环境变量

1.当需要以机房模式启动时，启动命令后需要增加 -z命令

如： hsldp start xst xst#0 -s 192.168.86.102:2181 -z

2.必须要增加环境变量，如果-z运行，但是没有配置环境变量，则启动失败

如： export HSLDP_ZONE=Zone1

2.9.2. 运行模式

1. 启动时，当zk上，路径/ldp/productname/ZoneInfo/Zone1 不存在时，当前进程会一直等待，直到ldpZoneTool将当前路径创建，进

程才会继续往下执行。

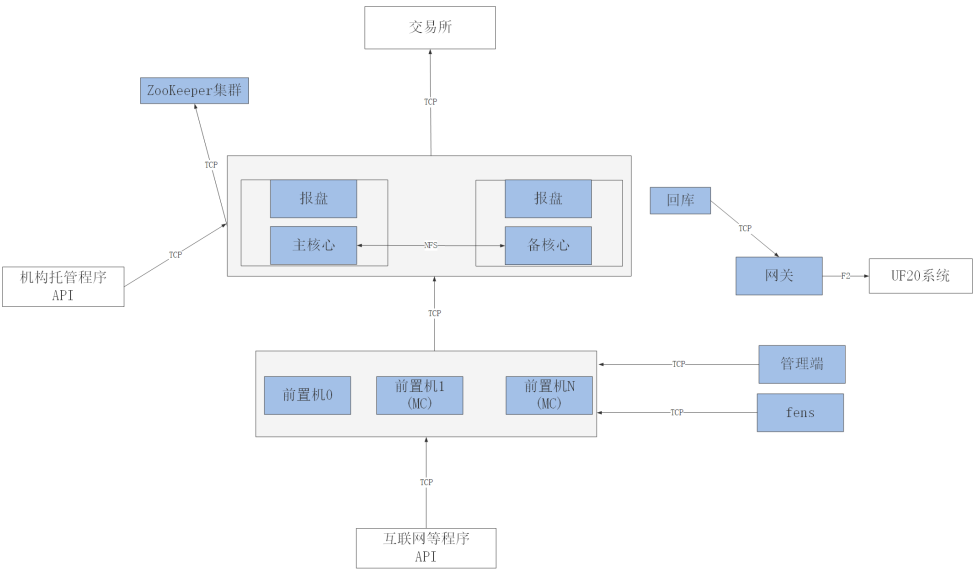
2. 启动时，当zk上，路径/ldp/productname/ZoneInfo/Zone1 存在，则直接会在当前路径下创建应用名节点，将类型，管理功能ip和端口写入该节点中，watch /ldp/productname/ZoneInfo/Zone1路径，并创建临时running节点。
3. 运行时，如果zk上的/ldp/productname/ZoneInfo/Zone1 内容修改，如果修改正确，则更新当前应用节点类型，并通知各个插件。如果内容有问题，则不通知。

3. 应用场景

本章主要介绍分布式低延时开发平台的应用场景

3.1. 主备

主备模式下的交易处理流程如下图所示：



机构托管程序API或者互联网程序API先连接fens，获取下一个接入点地址。机构托管程序API返回的是主核心的地址，互联网程序API返回的是前置机的地址。

互联网程序API后续流程：

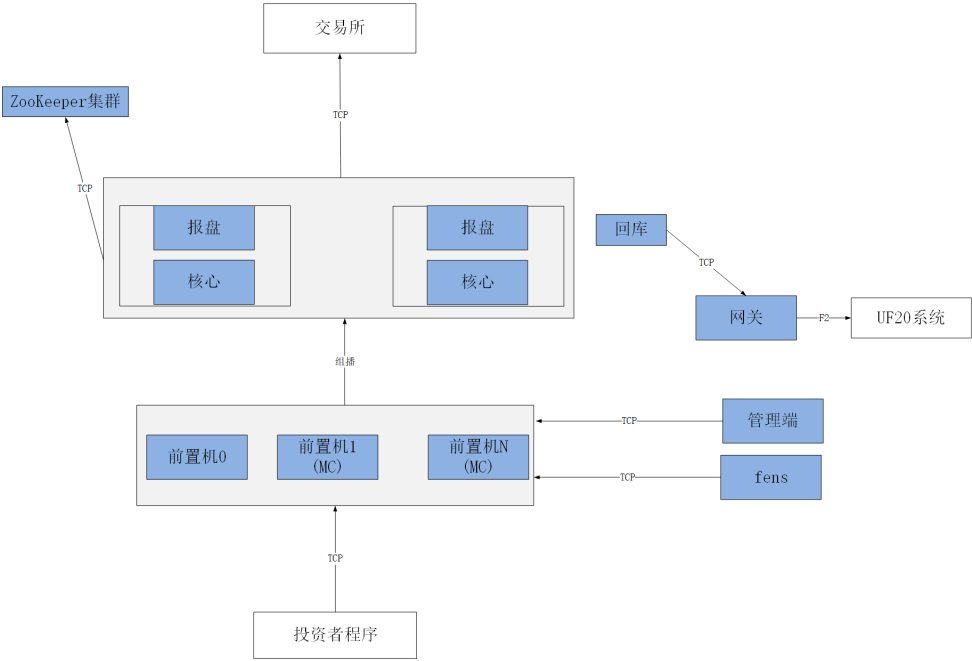
1. 互联网程序API发送交易请求给前置机
2. 前置机把交易请求给主核心
3. 主核心处理后应答给前置机
4. 前置机应答给互联网程序API
5. 主核心委托申报给报盘
6. 报盘申报交易所
7. 交易所返回委托回报、成交、交易所状态等给报盘
8. 报盘把交易所申报确认、委托回报、交易所状态信息给主核心

机构托管程序API后续流程：

1. 机构托管程序API发送交易请求给主核心
2. 主核心处理后应答给机构托管程序API
3. 主核心委托申报给报盘
4. 报盘申报交易所
5. 交易所返回委托回报、成交、交易所状态等给报盘
6. 报盘把交易所申报确认、委托回报、交易所状态信息给主核心

3.2. 多活

多活模式下的交易处理流程如下图所示：



1. 投资者程序连接fens获取接入点地址
2. 投资者程序发送交易请求给前置机
3. 前置机把交易请求组播给核心
4. 核心处理后应答给前置机
5. 前置机应答给投资者程序
6. 核心委托申报给报盘
7. 报盘申报交易所
8. 交易所返回委托回报、成交、交易所状态等给报盘
9. 报盘把交易所申报确认、委托回报、交易所状态信息给核心

4. 插件说明

本章主要介绍分布式低延时开发平台的插件的配置及使用说明

4.1. fens插件 ldp_fens

ldp_fens是一个LDP插件，它的功能是提供站点查询，支持按组，分区，类型对后端节点进行管理。提供多个查询监听接口，返回与查询匹配的后端站点集合。返回多个前置站点时，会对前置站点按负载进行排序后返回。对核心站点，集群定位下，只返回主核心节点。

插件名	支持平台	插件标识	说明
libldp_fens.so	linux.x64	ldp_fens	fens

4.1.1. 配置说明

fens插件的配置格式

以下是fens插件配置的一个抽象示例，后续对其具体展开解释

```

"Plugins":
[
  {
    // 全局部分
    "Lib": "ldp_fens",
    "Instance": 0,
    "Timer":1,
    "AccountPartitionMode": false,
    "EnableLocate": true,
    "AccInfoQueryInSec":100,
    "CoreNotConnectTimeoutInSec": 110,
    "CoreAccInfoCollectTimeoutInSec": 110,
    "QueryLoadIntervalSec":1,
    "PrivateAddress":
    [
      {
        "begin":"10.0.0.0",
        "end":"10.255.255.255"
      },
      ...
    ],
    "Tcp":
    {
      "MaxSendBuffSizeBytes": 0,
      "MinRecvBuffSizeBytes": 256,
      "HeartbeatIntervalSec": 3600,
      "HeartbeatTimes": 3,
      "ReconnectIntervalSec": 10
    },

    // 后台服务站点, server数组中一个元素是一个后端站点
    "BackendSites":
    {
      "Server":
      [
        {
          "Group": "A",
          "AccountPartition":
          [
            0
          ],
          "Type": "front",
          "SiteName": "A_0_F_0",
          "OperatorType": 1,
          "GeographicLocation" : 1,
          "Sites":
          [
            {
              "Host": "192.168.86.102",
              "Port": 44000,
              "bWan": true
            },
            {
              "Host": "192.168.86.102",
              "Port": 44001,
              "bWan": false
            }
          ],
          "AccountCategories":
          [
            "Category_1",
            "Category_2"
          ],
        },
        {
          "Group": "A",
          "AccountPartition": 0,
          "Type": "core",
          "SiteName": "A_0_C_1",
          "GlobalRdmaIp123": "192.168.867.102",
          "EnableQuerySiteType": true,
          "EnableQuerySystemNo": true,
          "EnableQueryNodeNo": true,
          "EnableQueryAccPart": true,
          "IsPrivate": false,
          "ProductName": "normal",
          "ClusterName": "Cluster_0",
          "Sites":
          [
            {
              "Host": "192.168.86.102",
              "Port": 18006
            }
          ],
          "AccountCategory": "Category_1",
          "AccountCategoryPriority": 1
        },
        ...
      ]
    },

    // 前台服务接口, server数组中一个元素是一个前端站点
    "FrontendSites":
    {
      "Server":
      [
        {
          "Type" : "front",
          "Host" : "192.168.86.102",
          "Port" : 33330,
          "Groups":
          [
            "A"
          ]
        },
        ...
      ]
    }
  }
]

```

全局配置内容及其解释参考以下

```
// 插件名称
"Lib": "ldp_fens",
"Instance": 0,
// 必须配置且值为1
"Timer": 1,
// 可以配置，也可以不配
// 因为fens对外服务需要tcp连接，fens与后端站点也需要tcp连接
// 所以可以通过此配置，来显示设置tcp连接相关属性
// 以下为所有可以在此单元内配置的字段
"Tcp":
{
    // 设置网络库发送与接收线程个数
    "SendRecvThreadCount": 1,
    // 设置网络库接收线程个数
    "RecvThreadCount": 1,
    // 设置网络库发送线程个数
    "SendThreadCount": 1,
    // 设置Rdma接收线程个数
    "RdmaRecvThreadCount": 1,
    // 设置线程与cpu绑定
    "ThreadBindCpu": [1,2,3],
},

// 设置账户分区是否开启
// 开启下，对后台服务站点划分类别是既有组，又有分区。
// 不开启下，对后台服务站点仅按组划分类别
// 开启下，fens启动时需要先向所有核心类型后台服务站点请求得到每个站点
// 账户-分区信息集合
// 才能开启对外服务，会造成启动延时。
"AccountPartitionMode": false,
// 是否开启节点定位模式，账号分区模式和节点定位模式不允许同时开启
"EnableLocate": true,
// 动态更新间隔，单位s
"AccInfoQueryInSec": 100,
// 核心未连接超时，单位s
"CoreNotConnectTimeoutInSec": 110,
// 核心信息收集超时，单位s
"CoreAccInfoCollectTimeoutInSec": 110,
// 对前置类型后台服务站点，按此字段指定的间隔不断查询
// 当前连到其的所有内网客户端数量，外网客户端数量
// 必须配置为大于0
"QueryLoadIntervalSec": 10,
// 显示配置私有地址
// 在begin--end范围内fens客户端被认为属于私有地址范围
// 属于私有地址范围的客户端属于内网客户端，不在此范围的客户端属于外网客户端
// 不配置时，
// 采用
// 10.0.0.0-10.255.255.255
// 172.16.0.0-172.31.255.555
// 192.168.0.0-192.168.255.255
// 作为私有地址范围
"PrivateAddress":
[
    {
        "begin": "10.0.0.0",
        "end": "10.255.255.255"
    },
    ...
],
],
```

fens插件的后端服务站点配置

后端服务站点配置内容及其字段参考以下

```

"BackendSites":
{
    // 可以配置，也可以不配
    // 因为fens与后端站点需要tcp连接
    // 所以可以通过此配置，来显示设置tcp连接相关属性
    // 以下为所有可以在此单元内配置的字段
    "Tcp":
    {
        // 最大发送缓存
        "MaxSendBuffSizeBytes": 0,
        // 最小接收缓存
        "MinRecvBuffSizeBytes": 256,
        // 心跳时间间隔
        "HeartbeatIntervalSec": 30,
        // 心跳超时次数
        "HeartbeatTimes": 3,
        // 重连间隔
        "ReconnectIntervalSec": 10,
        // 是否启用快速ACK
        "EnableQuickAck": true,
        // 是否启用Nagle
        "EnableNagle": false
    },
    // 仅仅在全局配置开启账户分区后，才需要配置以下对象
    // 开启账户分区后，通过组-分区两个属性来进行后端站点定位
    // 对每个组可以通过DefAccountPart来配置该组的默认分区号
    "DefAccountPart":
    [
        {
            // 指定一个组
            "Group": "Default",
            // 指定该组的默认分区。
            "DefAccountPartIndex": 0
        },
        ...
    ],
},
"Server":
[
    // 一个前置类型后端服务站点的配置
    {
        // 站点类型：可以为front, core, frontproxy
        "Type": "front",
        // 站点所在的组，长度控制在31字符或之内
        "Group": "A",
        // 仅仅在全局配置中开启了账户分区才需要配置分区信息。
    }
]
}

```

```

// 站点所在时分区。
// 前置类型站点分区以数组对象指定，可属于多个分区。
"AccountPartition":
[
    0,
    1
],

// 用于定位该后端站点的服务接口
// 只有定位正确，fens才能作为客户端连接到该后端接口
// 向其发送请求，获取后端站点的服务
// 此部分需要结合后端站点对外接口实际情况来配置
"Sites":
[
    {
        "Host": "192.168.86.102",
        "Port": 44000,
        "bWan": true
    },
    {
        "Host": "192.168.86.102",
        "Port": 44001,
        "bWan": false
    }
],

// 站点名称，长度控制在31字符或之内
"SiteName": "A_0_1_F_0",
// 供应商类型
"OperatorType": 1,
// 地理定位
"GeographicLocation": 1,
"/": "站点所属账户类别",
"AccountCategories":
[
    "Category_0",
    "Category_1"
]
},
// 一个核心类型后端站点的配置
{
    // 参考之前说明
    "Type": "core",
    // 参考之前说明
    "Group": "A",
    // 仅仅在全局配置中开启了账户分区才需要配置分区信息。
    // 站点所在的分区。
    // 核心类型站点分区只能指定一个。
    "AccountPartition": 0,

    // 核心类型后端站点的定位可以通过
    // 配置ProductName, ClusterName以集群来定位
    // 也可以通过配置Sites对象来定位
    // 这里为通过集群方式来定位
    // 不允许两个核心类型后端站点以集群方式定位时
    // ProductName, ClusterName均相等
    // 长度控制在31字符或之内
    "ProductName": "partition",
    // 长度控制在31字符或之内
    "ClusterName": "Cluster_0",
    // 站点名称，长度控制在31字符或之内
    "SiteName": "A_0_C_2",
    // 使能查询账户站点类型开关，默认为false
    "EnableQuerySiteType": true,
    // 使能查询账户系统号开关，默认为true
    "EnableQuerySystemNo": true,
    // 使能查询账户节点号开关，默认为true
    "EnableQueryNodeNo": true,
    // 使能查询账户账户分区开关，默认为true
    "EnableQueryAccPart": true,
    // 是否为私有站点，默认为false
    "IsPrivate": false,
    "/": "zk服务端地址",
    "ZkServerAddr": "192.168.87.137:2888",
    "/": "站点所属账户类别",
    "AccountCategory": "Category_0",
    // 账户优先级
    "AccountCategoryPriority": 2
},
{
    // 参考之前说明
    "Type": "core",
    // 参考之前说明
    "Group": "A",
    // 参考之前说明
    "AccountPartition": 0,

    "SiteName": "A_0_C_3",
    // 核心类型后端站点的定位可以通过
    // 配置ProductName, ClusterName以集群来定位
    // 也可以通过配置Sites对象来定位
    // 这里为通过Sites对象来定位
    "Sites":
    [
        {
            "Host": "192.168.86.102",
            "Port": 8111
        },
        ...
    ],
    "/": "站点所属账户类别",
    "AccountCategory": "Category_1",
    // 账户优先级
    "AccountCategoryPriority": 1
},
// 一个前置代理类型后端站点的配置
{
    // 站点类型
    "Type": "frontproxy",
    // 分区
    "AccountPartition": [
        1,2
    ],
    // 站点列表
    "Sites": [
        {
            "Host": "192.168.87.137"

```

```
        "Port": 13142
    },
    {
        "Host": "192.168.87.137",
        "Port": 13143
    }
]
...
},
],
```

fens插件的前端服务接口配置

前端服务接口配置内容及其字段参考以下

```
// 前台服务接口
"FrontendSites":
{
    // 可以配置，也可以不配
    // 因为Fens作为服务端通过前台服务接口对外提供服务，依赖tcp连接。
    // 所以可以通过此配置，来显示设置tcp连接相关属性
    // 以下为所有可以在此单元内配置的字段
    "Tcp":
    {
        // 最大发送缓存
        "MaxSendBuffSizeBytes": 0,
        // 最小接收缓存
        "MinRecvBuffSizeBytes": 256,
        // 心跳时间间隔
        "HeartbeatIntervalSec": 30,
        // 心跳超时次数
        "HeartbeatTimes": 3,
        // 最大连接数
        "MaxConnectCount": 10000,
        // 连接起始索引
        "ConnectStartIndex": 0,
        // 是否启用快速ACK
        "EnableQuickAck": true,
        // 是否启用Nagle
        "EnableNagle": false,
        // 白名单
        "AddressWhiteList" : "97.64.25.*;...;97.65.27.12;",
        // 黑名单
        "AddressBlackList" : "98.64.??.32;...;98.65.27.12;"
    },
    "Server":
    [
        {
            // 每个Fens的外部接口
            // 一方面通过指定Host,Port给出自身定位信息，以便客户端可以据此向该外部接口发起连接，请求服务
            // 一方面通过指定Type和Groups对Fens的后端服务站点集合
            // 按类型，按组进行一个筛选。
            // 筛选的结果是，只有指定类型，指定组的后端服务站点会被该Fens外部接口作为结果反馈给请求客户
            "Type" : "front",
            "Groups":
            [
                "A"
            ],
            "Host" : "192.168.86.102",
            "Port" : 33330
        },
        ...
    ]
},
```

4.1.2. 配置项说明

名称	类型	说明	备注
Timer	整型	定时事件发生频率	单位秒, 填1, 必须配置
Tcp	对象	配置tcp的工厂对象的属性	
SendRecvThreadCount	整型	设置网络库工作线程个数	默认为1，该值大于0时，工厂对象的每个工作线程同时负责套接字上数据发送和接收
RecvThreadCount	整型	设置网络库接收线程个数	默认为1，SendRecvThreadCount设置为0时，该值有效
SendThreadCount	整型	设置网络库发送线程个数	默认为1，SendRecvThreadCount设置为0时，该值有效
ThreadBindCpu	整型数组	线程与cpu绑定	设置线程和cpu的绑定
AccountPartitionMode	布尔型	账户分区模式开关	默认为false。开启后会向核心后端收集账户信息。查询时，依据账户确定分区，类别。再结合查询客户端所连监听端口的组与类别来确定可返回查询站点。
EnableLocate	布尔型	节点定位开关	默认为false。开启后，必须配置节点定位插件。通过节点定位插件获取节点号，再根据节点号、站点类型、组信息、站点的账户分区信息，筛选符合要求的站点并返回。API请求时，若使用AccountPartition构造请求报文，则默认使用"Default"规则进行匹配；若要支持传入规则信息，则必须使用NodeLocateReq构造请求报文。此外，业务核心需实现功能号406来返回查询核心NodeNo和SystemNo信息
AccInfoQueryInSec	整型	动态更新间隔	默认为-1。单位s。账户分区开启下，动态更新间

			隔大于0时，每次收集完账户信息。间隔AccInfoQueryInSec后，再次进行一次账户信息收集。
CoreNotConnectTimeoutInSec	整型	核心未连接超时	默认为-1。账户分区开启下，向核心收集账户信息时，若核心无法连接，经过CoreNotConnectTimeoutInSec仍然无法连接时，停止向其收集信息。为-1时，持续等待该核心收集完成。
CoreAccInfoCollectTimeoutInSec	整型	核心账户信息收集超时	默认为-1。账户分区开启下，向核心收集账户信息时，若核心无法及时返回所有账户信息，经过CoreAccInfoCollectTimeoutInSec仍然无法完成该核心所有账户信息收集时，停止向其收集信息，已经收集部分也丢弃。为-1时，持续等待该核心收集完成。
QueryLoadIntervalSec	整型	前置负载查询间隔	默认为10。向前置后端发送负载查询间隔。
PrivateAddress	ip区间数组	私有地址范围	配置私有地址范围。ip落在此范围的客户端被认定为内网客户端。
BackendSites	对象	fens后端站点	
Top	对象	与后端站点相连的客户端的属性	
MaxSendBufSizeBytes	整型	最大发送缓存	默认为0，表示不限制
MinRecvBufSizeBytes	整型	最小接收缓存	默认为256
HeartbeatIntervalSec	整型	心跳间隔	默认为30
HeartbeatTimes	整型	心跳超时次数	默认为3
ReconnectIntervalSec	整型	重连间隔	默认为10
EnableQuickAck	布尔	是否启用快速ACK	默认为true
EnableNagle	布尔	是否启用Nagle	默认为false
DefAccountPart	对象数组	组及其默认分区	账户分区开启下有效。用于为一个或多个组配置该组的默认分区号。
Group	字符串	组名	组与组相互隔离
DefAccountPartIndex	整型	默认分区号	设置关联组下默认分区号
Server	对象数组	数组内每个对象为一个fens后端站点	
Type	字符串	站点类别	fens支持三类站点：前置，核心，前置代理。对应配置分别为"front","core","frontproxy"
AccountPartition	整型或整型数组	站点所属分区	前置站点，前置代理站点可以属于一个或多个分区，核心站点只能属于一个分区。
Sites	对象数组，每个数组元素表示一个站点实体	站点实体集合	
Host	字符串	站点地址	支持配置ip地址或域名
Port	整型	站点监听端口	站点监听端口
bWan	布尔	站点内外网属性	对前置后端有效。前置后端一般含两个监听实体，一个面向内网客户，一个面向外网客户。
SiteName	字符串	站点名	站点名信息
EnableQuerySiteType	布尔	使能查询账户站点类型开关	
EnableQuerySystemNo	布尔	使能查询账户系统号开关	默认为true
EnableQueryNodeNo	布尔	使能查询账户节点号开关	默认为true
EnableQueryAccPart	布尔	使能查询账户账户分区开关	默认为true
IsPrivate	布尔	是否为私有站点	true: 此站点为私有站点，该站点信息不能被返回，false: 此站点不是私有站点，该站点信息可以被返回，默认为false
Enable	布尔	站点使能开关	
OperatorType	整型	供应商类型	供应商类型
GeographicLocation	整型	地理定位	地理定位
AccountCategories	字符串数组	站点所属账户类别	站点所属账户类别，只有核心站点账户类别"AccountCategory"配置项的值在"AccountCategories"中，此前置才会返回给客户

			端。不配置默认包含Default，配置了以配置为准。
ProductName	字符串	产品名	针对采用集群定位的核心后端有效。
ClusterName	字符串	集群名	针对采用集群定位的核心后端有效。
ZkServerAddr	字符串	zk服务端地址，可以配置多个，以逗号分隔	配置此项后，fens将会从配置的zk地址获取该核心站点的主备身份，否则将会从fens所使用的zk地址获取该核心站点的主备身份，该zk地址不支持鉴权，仅针对采用集群定位的核心后端有效。
AccountCategory	字符串	站点所属账户类别	站点所属账户类别 默认值是Default
AccountCategoryPriority	整型	站点所属类别优先级	站点所属类别优先级，范围[0,255]
FrontendSites	对象	fens对外服务信息	
Tcp	对象	fens对象服务时被动客户端的属性信息	
MaxSendBuffSizeBytes	整型	最大发送缓存	默认为0，表示不限制
MinRecvBuffSizeBytes	整型	最小接收缓存	默认为256
HeartbeatIntervalSec	整型	心跳间隔	默认为30
HeartbeatTimes	整型	心跳超时次数	默认为3
MaxConnectCount	整型	最大连接数目	默认为10000
ConnectStartIndex	整型	连接起始索引	默认为0
EnableQuickAck	布尔	启用快速ACK	默认为true
EnableNagle	布尔	启用Nagle	默认为false
AddressWhiteList	字符串	IP白名单	支持模糊匹配,*号必须放最开头或最结尾,?号位置随意,每个ip必须以;号结尾,包括最后一个 最长支持1024个字符
AddressBlackList	字符串	IP黑名单	支持模糊匹配,*号必须放最开头或最结尾,?号位置随意,每个ip必须以;号结尾,包括最后一个 最长支持1024个字符
Server	对象数组	每个数组对象描述一个fens对外服务接口	
Type	字符串	可提供服务后端站点类别	前置为"front",核心为"core",前置代理为"frontproxy",不配置时，表示同时支持前置，核心，前置代理。
Groups	字符串数组	可提供服务组	组与组间相互独立。
Host	字符串	对外接口的ip	支持三种配置方式。1.不配置，此时采用本机任何可用ip。2.配置为一个具体本机ip。3.配置为一个本机ip的网段。
Port	整型	监听端口	对外接口的监听端口

4.1.3. fens运行模式

fens 主要有三种运行模式。账户分区模式下，fens主要依靠账户分区号进行后端站点的定位。节点定位模式下，fens依靠定位插件获得的节点号来进行后端站点的定位。账户分区和节点定位都不开启的模式下，fens主要根据客户端连接的前端站点的类型来进行后端站点的定位。以下依次介绍这三种模式。

账户分区模式

该模式适合账户规模小，并且由业务自己确定分区规则的场景。当配置项AccountPartitionMode=true且EnableLocate=false时，认为开启账户分区模式。该模式下，fens会定时向后端核心收集账户信息。当客户端发起站点查询请求时，fens根据从核心收集到的账户信息、后端站点的配置信息、前端站点的配置信息综合匹配出符合条件的站点，并将其站点信息返回给客户端。

该模式的示例配置如下：

```
{
  "Lib": "ldp_fens",
  "Instance": 0,
  "Timer": 1,
  "//": "开启账户分区模式",
  "AccountPartitionMode": true,
  "//": "关闭节点定位模式",
  "EnableLocate": false,
  "BackendSites": {
    "Server": [
      {
        "//": "一个前置类型的后端站点",
        "Type": "front",
        "SiteName": "102front",
        "OperatorType": 1,
        "GeographicLocation": 1,
        "Group": "Default",
        "AccountPartition": [
          2
        ],
        "Sites": [
          {
            "Host": "192.168.86.102",
            "Port": 10011,
            "bWan": true
          },
          {
            "Host": "192.168.86.102",
            "Port": 10012,
            "bWan": false
          }
        ]
      },
      {
        "//": "一个核心类型的后端站点",
        "Type": "core",
        "SiteName": "CoreSet",
        "AccountPartition": 2,
        "Group": "Default",
        "//": "查询账户站点类型开关，开启后将会保存查询到的账户站点类型",
        "EnableQuerySiteType": true,
        "ProductName": "ldp_test",
        "ClusterName": "xst"
      }
    ]
  },
  "FrontendSites": {
    "Server": [
      {
        "Type": "front",
        "Host": "192.168.86.102",
        "Port": 12001,
        "Groups": [
          "Default"
        ]
      },
      {
        "Host": "192.168.86.102",
        "Port": 12002,
        "Groups": [
          "Default"
        ]
      },
      {
        "Type": "core",
        "Host": "192.168.86.102",
        "Port": 12003,
        "Groups": [
          "Default"
        ]
      }
    ]
  }
}
```

账户分区模式下，fens定位站点的逻辑如下：

- 1、解析客户端发来的站点查询请求，获得账户Account。
- 2、用账户Account与从后端核心收集的账户信息进行匹配，匹配出该账户对应的完整账户信息。
- 3、通过完整账户信息中的账户类型Type和账户分区AccPart匹配符合的后端站点
- 4、通过客户端连接的前端站点的IP端口，获得前端站点的组Group和前端站点类型Type，将Group和Type与第三步筛选出来的后端站点进行匹配，匹配出符合条件的后端站点。
- 5、在匹配出的站点中，继续筛选出站点类型为前置或前置代理，且与其相连的核心在线的站点。
- 6、继续筛选出站点类型为核心，在线且为主节点的站点。
- 7、当客户端从外网接入时，继续筛选出所有核心站点和前置及前置代理的外网站点，当客户端从内网接入时，继续筛选出置及前置代理的内网站点。
- 8、根据站点负载从小到大排序前置和前置代理站点信息，按照前置、核心、前置代理的顺序打包返回筛选后的站点信息。

节点定位模式

该模式适合账户规模大的场景，支持多种账户分区规则（枚举、范围、模糊匹配）。当配置项AccountPartitionMode=false且EnableLocate=true时，认为开启节点定位模式。该模式下，fens会主动定时向后端核心收集系统号和节点号信息。当客户端发起站点查询请求时，fens根据从核心收集到的系统号和节点号信息、后端站点的配置信息、前端站点的配置信息综合匹配出符合条件的站点，并将其站点信息返回给客户端。节点定位模式下，应用需要额外配置节点定位插件，插件说明见节点定位插件ldp_locate一节。

该模式的示例配置如下：


```

{
  "Lib": "ldp_fens",
  "Instance": 0,
  "Timer": 1,
  "QueryLoadIntervalSec": 1,
  "/*": "关闭账户分区模式",
  "AccountPartitionMode": false,
  "/*": "开启节点定位模式",
  "EnableLocate": true,
  "BackendSites": {
    "Server": [
      {
        "Type": "front",
        "SiteName": "102front",
        "OperatorType": 1,
        "GeographicLocation": 1,
        "Group": "Default",
        "AccountPartition": [
          2
        ],
        "Sites": [
          {
            "Host": "192.168.86.102",
            "Port": 10011,
            "bWan": true
          },
          {
            "Host": "192.168.86.102",
            "Port": 10012,
            "bWan": false
          }
        ]
      },
      {
        "Type": "core",
        "SiteName": "ARCS_Core",
        "AccountPartition": 2,
        "Group": "Default",
        "ProductName": "ldp_fens",
        "ClusterName": "arcs",
        "/*": "风控核心开启站点查询",
        "EnableQuerySiteType": true
      },
      {
        "Type": "core",
        "SiteName": "SECU_Core",
        "AccountPartition": 1,
        "Group": "Default",
        "ProductName": "ldp_fens",
        "ClusterName": "secu",
        "/*": "交易核心关闭站点查询",
        "EnableQuerySiteType": false
      }
    ]
  },
  "FrontendSites": {
    "Server": [
      {
        "Type": "front",
        "Host": "192.168.86.102",
        "Port": 12001,
        "Groups": [
          "Default"
        ]
      },
      {
        "Host": "192.168.86.102",
        "Port": 12002,
        "Groups": [
          "Default"
        ]
      },
      {
        "Type": "core",
        "Host": "192.168.86.102",
        "Port": 12003,
        "Groups": [
          "Default"
        ]
      }
    ]
  }
}

```

节点定位模式下，fens定位站点的逻辑如下：

- 1、解析客户端发来的站点查询请求，获得账户Account。
- 2、使用账户Account，通过节点定位插件，获得账户对应的节点号NodeNo。
- 3、通过节点号NodeNo，从收集的系統号节点号信息和后端站点配置中，匹配符合的后端站点。
- 4、在开启查询站点类型开关，即EnableQuerySiteType=true时，还会额外获得账户对应的账户类型Type。
- 5、通过客户端连接的前端站点的IP端口，获得前端站点的组Group和前端站点类型Type，将Group和Type与第三步筛选出来的后端站点和第四步获得的账户类型Type（如果有）进行匹配，匹配出符合条件的后端站点。
- 6、在匹配出的站点中，继续筛选出站点类型为前置或前置代理，且与其相连的核心在线的站点。
- 7、继续筛选出站点类型为核心，在线且为主节点的站点。
- 8、当客户端从外网接入时，继续筛选出所有核心节点和前置及前置代理的外网站点，当客户端从内网接入时，继续筛选出前置及前置代理的内网站点。
- 9、根据站点负载从小到大排序前置和前置代理站点信息，按照前置、核心、前置代理的顺序打包返回筛选后的站点信息。

账户分区模式和节点定位模式都不开启

该模式适合不需要关心分区规则的场景，当配置项AccountPartitionMode=false且EnableLocate=false时，认为账户分区模式和节点定位模式都不开启。该模式下，fens不会主动向后端核心收集账户信息。当客户端发起站点查询请求时，fens根据后端站点的配置信息、前端站点的配置信息综合匹配出符合条件的站点，并将其站点信息返回给客户端。

该模式的示例配置如下：

```
{
  "Lib": "ldp_fens",
  "Instance": 0,
  "Timer": 1,
  "QueryLoadIntervalSec": 1,
  "//": "关闭账户分区模式",
  "AccountPartitionMode": false,
  "//": "关闭节点定位模式",
  "EnableLocate": false,
  "//": "后端节点",
  "BackendSites": {
    "Server": [
      {
        "Type": "front",
        "SiteName": "102front",
        "OperatorType": 1,
        "GeographicLocation": 1,
        "Group": "Default",
        "Sites": [
          {
            "Host": "192.168.86.102",
            "Port": 33573,
            "bWan": true
          },
          {
            "Host": "192.168.86.102",
            "Port": 43573,
            "bWan": false
          }
        ]
      },
      {
        "Type": "front",
        "SiteName": "101front",
        "OperatorType": 1,
        "GeographicLocation": 1,
        "Group": "Default",
        "Sites": [
          {
            "Host": "192.168.86.101",
            "Port": 10011,
            "bWan": true
          },
          {
            "Host": "192.168.86.101",
            "Port": 10012,
            "bWan": false
          }
        ]
      }
    ],
    {
      "Type": "core",
      "SiteName": "CoreSet",
      "Group": "Default",
      "//": "产品名",
      "ProductName": "ldp_test",
      "//": "集群名",
      "ClusterName": "xst"
    }
  ],
  "//": "fens对外服务信息",
  "FrontendSites": {
    "Server": [
      {
        "Type": "front",
        "Host": "192.168.86.102",
        "Port": 12001,
        "Groups": [
          "Default"
        ]
      },
      {
        "Host": "192.168.86.102",
        "Port": 12002,
        "Groups": [
          "Default"
        ]
      },
      {
        "Type": "core",
        "Host": "192.168.86.102",
        "Port": 12003,
        "Groups": [
          "Default"
        ]
      }
    ]
  }
}
```

账户分区模式和节点定位模式都不开启的情况下，fens定位站点的逻辑如下：

- 1、解析客户端发来的站点查询请求，获得账户Account。
- 2、在开启查询站点类型开关，即EnableQuerySiteType=true时，还会额外获得账户对应的账户类型Type。
- 3、通过客户端连接的前端站点的IP端口，获得前端站点的组Group和前端站点类型Type。
- 4、使用第二步获得的账户类型Type（如果有）、第三步获得的前端站点的组Group和前端站点类型Type匹配出符合条件的后端站点。

5、在匹配出的站点中，继续筛选出站点类型为前置或前置代理，且与其相连的核心在线的站点。

6、继续筛选出站点类型为核心，在线且为主节点的站点。

7、当客户端从外网接入时，继续筛选出所有核心站点和前置及前置代理的外网站点。当客户端从内网接入时，继续筛选出置及前置代理的内网站点。

8、根据站点负载从小到大排序前置和前置代理站点信息，按照前置、核心、前置代理的顺序打包返回筛选后的站点信息。

4.1.4. 依赖说明

- 依赖于libtcp.so，即需要配置公共库：tcp，例如：

```
"Components": {
  "Libs": [],
  "GlobalLibs": [
    "tcp"
  ]
}
```

4.2. 前置机插件 ldp_front

ldp_front是一个LDP插件，它的功能是连接客户端与后台服务系统,在需要时做通信协议转换和路由;并提供一种扩展机制,可以让用户开发业务模块,处理一些特别的请求,比如登录,退出,消息推送。

插件名	支持平台	插件标识	说明
libldp_front.so	linux.x64	ldp_front	前置机

4.2.1. 示例配置

```
{
  "Lib": "ldp_front",
  "Instance": 0,
  "Timer": 0,
  "Tcp": {
    "Tcp": {
      "RdmaRecvThreadCount": 1,
      "SendRecvThreadCount": 5,
      "RecvThreadCount": 5,
      "SendThreadCount": 5,
      "ThreadBindCpu": [1, 2, 3],
      "EnableAsynSend": true
    },
    "Wan": {
      "Port": 13572,
      "Address": "127.0.0.1",
      "RdmaIp": "3.3.3.101",
      "AddressWhiteList": "97.64.25.*;97.64.??;97.65.27.12;",
      "AddressBlackList": "98.64.25.*;98.64.??;98.65.27.12;",
      "MaxConnectCount": 50000,
      "MaxSendBuffSizeBytes": 0,
      "MinRecvBuffSizeBytes": 256,
      "HeartbeatIntervalSec": 30,
      "HeartbeatTimes": 3,
      "Onload": 1
    },
    "Lan": {
      "Port": 23572,
      "Address": "127.0.0.1",
      "MaxConnectCount": 10000,
      "MaxSendBuffSizeBytes": 0,
      "MinRecvBuffSizeBytes": 256,
      "HeartbeatIntervalSec": 30,
      "HeartbeatTimes": 3,
      "Onload": 1
    },
    "TimeStampBlackFuncList": "11;11?;11??",
    "TimeStampWhiteFuncList": "*",
    "Routes": [
      { "Functions": "*", "Systems": "1", "Nodes": "1", "Backend": 1 },
      { "Functions": "*", "Systems": "5", "Nodes": "250", "Backend": 5 },
      { "Functions": "11;111;1111", "Systems": "0;2", "Nodes": "65535", "Backend": 1 },
      { "Functions": "33??", "Systems": "0", "Nodes": "65", "Backend": 2 },
      { "Functions": "9000011", "Systems": "11;12", "Nodes": "*", "Backend": 11 },
      { "Functions": "*", "Systems": "13", "Backend": 13 }
    ],
    "FrontBizLibs": {
      "ReqLibs": ["libhello.so", "libustbiz.so"],
      "RspLibs": ["libustbiz.so", "libdummy.so"],
      "ErrLib": "libustbiz.so"
    },
    "FlowControl": {
      "Functions": [
        { "Functions": "11?", "Rate": 500000, "Scale": 100, "ControlType": "Throw" },
        { "Functions": "22?", "Rate": 500000, "Scale": 100, "ControlType": "Throw" },
        { "Functions": "*", "Rate": 250000, "Scale": 100, "ControlType": "Throw" }
      ],
      "FunctionsPerConnection": [
        { "Functions": "22?", "Rate": 10000, "Scale": 100, "ControlType": "Throw" },
        { "Functions": "3311;3322", "Rate": 10000, "Scale": 100, "ControlType": "Throw" },
        { "Functions": "*", "Rate": 100000, "Scale": 100, "ControlType": "Throw" }
      ],
      "Connection": [
        { "Rate": 7000, "Scale": 100, "ControlType": "Throw" }
      ]
    },
    "TcpInitiator": {
      "Onload": 1
    }
  }
}
```

```

    },
    "Backends": [
        {
            "Name": "ha-core1",
            "Id": 1,
            "Type": "AS-TCP",
            "Arb": false,
            "ProductName": "frontx",
            "ClusterName": "ldptest",
            "EnableSession": true,
            "EnableQueryFirst": false,
            "QueryFirstMode": 1,
            "Tcp": {
                "EnableQuickAck": true,
                "EnableNagle": false,
                "RdmaIp": "3.3.3.101",
                "MaxSendBuffSizeBytes": 0,
                "MinRecvBuffSizeBytes": 256,
                "HeartbeatIntervalSec": 30,
                "HeartbeatTimes": 3,
                "ReconnectIntervalSec": 30,
                "Onload": 1
            },
            "Sites": [
                {
                    "Host": "127.0.0.1",
                    "Port": 14551
                },
                {
                    "Host": "127.0.0.1",
                    "Port": 14561
                }
            ]
        },
        {
            "Name": "ha-core15",
            "Id": 15,
            "Type": "AS-TCP",
            "Sites": [
                {
                    "Host": "127.0.0.1",
                    "Port": 19896
                },
                {
                    "Host": "127.0.0.1",
                    "Port": 19806
                }
            ]
        },
        {
            "Name": "ha-core2",
            "Id": 2,
            "Type": "AS-TCP",
            "Sites": [
                {
                    "Host": "127.0.0.1",
                    "Port": 14552
                },
                {
                    "Host": "127.0.0.1",
                    "Port": 14562
                }
            ]
        },
        {
            "Name": "gateway1",
            "Id": 3,
            "Type": "GW",
            "Sites": [
                {
                    "Host": "127.0.0.1",
                    "Port": 14553
                },
                {
                    "Host": "127.0.0.1",
                    "Port": 14563
                }
            ]
        },
        {
            "Name": "gateway2",
            "Id": 4,
            "Type": "GW",
            "MsgType": "UST",
            "Sites": [
                {
                    "Host": "127.0.0.1", "Port": 14554
                },
                {
                    "Host": "127.0.0.1", "Port": 14564
                }
            ]
        },
        {
            "Name": "aa-trade",
            "Id": 11,
            "Type": "AA",
            "TxTopic": "trade",
            "TxPartitions": [
                1
            ],
            "RcmContext": "front_trade_0"
        },
        {
            "Name": "aa-query",
            "Id": 12,
            "Type": "AA",

```

```
        "TxTopic": "query",
        "TxPartitions": [
            1
        ],
        "RxPartition": 1,
        "RcmContext": "front_trade_0"
    },
    {
        "Id": 13,
        "Type": "AS-RCM",
        "TxTopic": "trade",
        "Arb": true,
        "ProductName": "as-rcm",
        "ClusterName": "xst"
    }
]
{
    "//1": "若配置PLUGIN类型，必须要配置PluginName所指向的插件，本例中PluginName为ldp_bizproc，所以必须配置ldp_bizproc，并且Instance需一致，否则启动报错",
    "Name": "PluginBackend",
    "Id": 5,
    "//2": "插件交互类型，用于插件直接交互，即不走网络的方式交互",
    "Type": "PLUGIN",
    "//3": "插件名",
    "PluginName": "ldp_bizproc",
    "//4": "实例号，此处必须和Lib所指向的插件的实例标号一致",
    "Instance": 0
},
{
    "Id": 6,
    "Type": "RCM",
    "TxTopic": "query",
    "ForwardMode": "Token"
},
{
    "ID": 7,
    "Type": "AS-TCP",
    "Arb": true,
    "ProductName": "test",
    "ClustetName": "ldptest",
    "ZkServerAddr": "192.168.86.102:6782,123.324.235.12:1234,123.54.23.1:234"
}
],
"//": "如果使用rcm代理，则需要在Components数组中加入rcm_proxy。以下前置使用rcm代理需要的配置（backend中需要有类型为AA的对象）"
"RspTopic": {
    "Name": "result",
    "PartitionNo": 1
},
"LocalNetworkNumber": "192.168.86.0",
"Subscribe": [
    {
        "Name": "push",
        "PartitionNos": [
            1
        ]
    }
],
"BackendContext": {
    "IsRecord": true,
    "TTL": 8
},
"EnableReplyErrRsp": true,
"FlowControlVersion": 2,
"LdpMsgAutoUnCompress": true,
"BackendContextCount": 2,
"BackendContexts": [
    {
        "BindCoreId": 9,
        "SendBindCoreId": 10,
        "RepairBindCoreId": 11
    },
    {
        "BindCoreId": 12,
        "SendBindCoreId": 13,
        "RepairBindCoreId": 14
    }
]
}
```

4.2.2. 配置项说明

名称	类型	说明	备注
Timer	整型	定时事件发生频率	单位秒, 非必须。目前前置无法响应框架的OnTimer线程。前置有自己的OnTimer线程。
Tcp	对象	影响发送接收的tcp参数	
RdmaRecvThreadCount	整型	rdma接收线程个数	如果需要使用rdma, 默认1
SendRecvThreadCount	整型	设置网络库工作线程个数	默认为1, 该值大于0时, 工厂对象的每个工作线程同时负责套接字上数据发送和接收
RecvThreadCount	整型	设置网络库接收线程个数	默认为1, SendRecvThreadCount设置为0时, 该值有效
SendThreadCount	整型	设置网络库发送线程个数	默认为1, SendRecvThreadCount设置为0时, 该值有效
ThreadBindCpu	整形数组	每个线程绑定cpu的序号	默认值: -1, 表示不绑定
EnableAsyncSend	布尔型	tcp通讯库的异步发送模式	默认值: false, 表示不开启异步发送模式
Wan	对象	外网服务属性	
Address	字符串	绑定IP地址	不配置表示通配地址
Port	整型	端口	

Onload	整型	是否开启Onload加速	在onload启动模式时，默认值：1，即开启加速。 有效值：0或1
RdmaIp	字符串	rdma网卡IP地址	如果不使用rdma, 则删除此项配置
AddressWhiteList	字符串	IP白名单	支持模糊匹配,*号必须放最开头或最结尾,?号位置随意,每个ip必须以;号结尾,包括最后一个 最长支持1024个字符
AddressBlackList	字符串	IP黑名单	支持模糊匹配,*号必须放最开头或最结尾,?号位置随意,每个ip必须以;号结尾,包括最后一个 最长支持1024个字符
MaxConnectCount	整型	最大连接数	默认65535,对于Wan口，上限为65535；对于Lan口，上限为65535。
MaxSendBuffSizeBytes	整型	发送缓存	单位字节,默认0, 不限制
MinRecvBuffSizeBytes	整型	接收缓存	单位字节,默认256
HeartbeatIntervalSec	整型	心跳时间间隔	单位秒,默认30秒
HeartbeatTimes	整型	心跳超时次数	默认3,3次超时后断开连接
Lan	对象	内网服务属性	支持的属性及默认值与上面Wan一致
TimeStampBlackFuncList	字符串	不支持时延记录的功能号集合	英文分号分隔,如"11;30?;319",?号表示一个数字,只能出现在最后 默认是空。如果白名单和黑名单同时存在,则以黑名单为准。都不存在,则都不记录时延。
TimeStampWhiteFuncList	字符串	支持时延记录的功能号集合	英文分号分隔,如"11;30?;319",?号表示一个数字,只能出现在最后 默认是空。如果白名单和黑名单同时存在,则以黑名单为准。都不存在,则都不记录时延。
EnableDynamicRoutes	布尔型	开启动态修改路由功能	默认值： false ,表示不开启动态修改变态路由
Routes	数组	功能号路由表	每个数组下参数Functions, Systems, Nodes为与的关系,必须满足三个条件下,才会正确的路由
Functions	字符串	功能号集合	英文分号分隔,如"11;30?;319",支持"*",支持模糊匹配,?号表示一个数字,只能出现在最后。若不配置,则代表支持所有功能号
Systems	字符串	系统号集合	英文分号分隔,如"11;30;255",支持"*",不支持"?".若不配置,则代表支持所有系统号。范围是[0,255]。
Nodes	字符串	节点号集合	英文分号分隔,如"11;30;65535",支持"*",不支持"?".范围是[0,65535]。若配置Nodes:""或Nodes:"",前置启动加载配置时会报错,前置启动失败。Nodes字段可查看本文档4.2.3节选
Backend	整型	目标backendid	必须存在相应的 Backends.Id
FrontBizLibs	对象	业务组件配置	组件存放路径： bin /应用名.local。同名的组件只会加载一次,使用的同一个实例。详细的使用说明与开发流程请参考《低延时分布式开发平台_前置业务组件开发使用指南》
ReqLibs	数组	业务请求处理组件	前置收到所有请求类型的消息后,都会按顺序调用所有的业务请求处理组件。最多支持添加8个,超出不加载
RspLibs	数组	业务应答处理组件	前置收到所有应答类型的消息后,都会按顺序调用所有的业务应答处理组件。最多支持添加8个,超出不加载
ErrLib	字符串	业务错误处理组件	前置在消息处理失败后,都会调用错误处理组件。最多支持添加1个
FlowControl	对象	流量控制	在 zk 上修改流控配置支持动态生效,本地模式不支持动态生效功能
Functions	数组	全局功能号流量控制	全局功能号流控最多支持配置128个
Functions	字符串	被流控的具体功能号	必填,支持模糊匹配,和路由表中功能号的设置规则一样,在此不详述。由于流控匹配只要匹配上了就跳出匹配逻辑,所以*号功能号必须放在最后
Rate	整型	流控速度	必填,单位是秒,即每秒最多可通过的包数量,值必须大于 Scale ,无上限
Scale	整型	缩放比例	必填,取值范围1-1000,将时间尺度细化,例配置 Scale 为100, Rate 为1000,则评分到每0.01秒内最多可通过10个包
ControlType	字符串	流控类型	选填,目前支持Disconnect(断开), Throw(丢弃),默认

			为Throw(丢弃),即当速度超过流控速度后, 如果配置了Disconnect, 则断开连接, 如果配Throw, 则丢弃掉这个包
FunctionsPerConnection	数组	每个连接上功能号的流控	最多支持配置128个, 配置项和全局功能号流控完全一致, 在此不详述
Connection	数组	每个连接的流控	目前仅支持配置一个, 且对所有连接生效
TcpInitiator	对象	TcpInitiator的, SectionName为"Initiator"的配置透传对象	对象内部字段需平铺
Onload	整型	是否开启Onload加速	在onload启动模式时, 默认值: 1, 即开启加速。 有效值: 0或1
Backends	数组		对应核心或网关, 最大支持65534
Name	字符串	核心名称	目前没有使用
Id	整型	编号	[1, 65534]
Type	字符串	核心类型	GW, 即网关 AA, 即多活 AS-TCP, 即TCP主备 AS-RCM 即RCM主备 PLUGIN, 插件交互 RCM, 对接查询核心
MsgType	字符串	消息类型	目前该项配置仅对网关有效, 默认LDP LDP, 即对接新网关 UST, 即对接老网关
Arb	布尔型	是否使用仲裁	当Type为AS-TCP或AS-RCM时有效 默认值:false. 当为true时, 启动时应指定zk, 否则报错退出 当为false时, 必须有Sites配置, 否则报错退出
ClusterName	字符串	集群名	当Type为AS-TCP或AS-RCM, 同时设置仲裁模式时有效, 建议前置与核心部署在同一个应用的话, 配置为应用名, 如果前置与核心没有部署在同一个核心, 则配置为核心的应用名, 默认值:应用名
ProductName	字符串	产品名	当Type为AS-TCP或AS-RCM, 同时设置仲裁模式时有效, 默认值:启动时命令行指定的产品名
ZkServerAddr	字符串	zk服务端地址	只有当Type为AS-TCP且Arb为true时才会生效。支持配置为其他zk服务端地址, 前置能正常收发集群消息和收到最新的集群身份信息。注意: 无论产品名是否一致, 无论是否在同一zk, 集群名都需要不一致
EnableSession	布尔型	使能会话模式	默认值:false. 当为true时, 开启会话模式 当为false时, 关闭会话模式
EnableQueryFirst	布尔型	是否开启优先去备模式	默认值:false. 当为true时, 开启优先去备模式 当为false时, 关闭优先去备模式
QueryFirstMode	整型	优先去备模式	默认值0. 目前只支持拥有多进程上下文的服务端。Backend类型, 非使用Rcm代理启动类型, 网关类型, 插件类型不支持 当且仅当EnableSession, 和EnableQueryFirst都为true时有效 0为不区分机房模式(负载均衡发送给备机), 1为主机房模式(负载均衡发送给和主机机房一致的备机), 2为本地机房模式(负载均衡发送给和本地机房一致的备机), 3为只去主机房模式(负载均衡发送给和主机机房一致的备机, 如果备机不存在, 直接发给主机) 所有模式, 当备机全部失效后, 消息只能发往主机
TxTopic	字符串	Rcm发送主题	当Type为AA或AS-RCM时有效, 在RcmContext中必须存在
TxPartitions	数组	Rcm发送分区	当Type为AA或AS-RCM时有效, 如[1, 3], 或[1]
RxPartition	整型	Rcm接收分区	当Type为AA或AS-RCM时有效, 默认为应用编号+1
RcmContext	字符串	Rcm Context	当Type为AA或AS-RCM时有效, 默认 AppName_PluginName_PluginInstance 比如 front#0_kdp_front_0
Tcp	对象	Tcp参数配置	当Type为AS-TCP, 或GW

Onload	整型	是否开启Onload加速	在onload启动模式时，默认值：1，即开启加速。 有效值：0或1
RdmaIp	字符串	rdma网卡IP地址	如果不使用rdma,则删除此项配置
SendThreadCount	整型	发送线程个数	默认1
EnableQuickAck	布尔型	快速确认	默认true, 启用
MaxSendBuffSizeBytes	整型	发送缓存	单位字节, 默认0, 不限制
MinRecvBuffSizeBytes	整型	接收缓存	单位字节, 默认256
HeartbeatIntervalSec	整型	心跳时间间隔	单位秒, 默认30秒
HeartbeatTimes	整型	心跳超时次数	默认3, 3次超时后断开连接
ReconnectIntervalSec	整型	重连间隔	单位秒, 默认30秒
Sites	数组		当Type为AS-TCP且Arb配置为true, 或GW时有效, 如果Sites里配置多个ip和端口, 对于GW来说, 数 据发送采用轮询的方式依次发送, 对AS-TCP来说 则是由客户端的请求来决定。 注：最多配置8个Site
Host	字符串	核心主机IP或者域名	
Port	整型	核心侦听端口	
ForwardMode	字符串	转发类型	当Type为RCM时有效 当为Token时：根据请求中的Token作一致性Hash, 获取查询核心的发送分区号 当为NodeNo时：根据请求中的NodeNo作为查询 核心的发送分区号 当为LoadBalance时，会负载均衡发送给各个符合 要求的核心中去处理，支持不区分机房和本地机 房模式。目前只支持多分片场景。 默认为NodeNo
RspTopic	对象	应答主题	
Name	字符串	应答主题	
PartitionNo	字符串	引用模版	非必填，不填则使用其中一个上下文Id作为分区号
LocalNetworkNumber	字符串	本机ip或者网段	
Subscribe	数组	前置接收主推的主题	
Name	字符串	接收主推的主题	
PartitionNos	整型数组	分区号	可选。接收主推主题的分区，不填则接收所有该 主题的分区
BackendContext	对象	上下文参数	非必填，填写后前置上下文中的参数透传给前置 上下文中，以当前配置下的参数为主
EnableReplyErrRsp	布尔型	开启主动回错误应答	默认值： false, 表示不开启主动回错误应答。配置 了ErrLib时，错误处理优先由ErrLib回应答；当没 有配置ErrLib时，配置true则错误处理由front主动回 应答，配置false则错误处理不回复应答
EnableReplyErrRsp	布尔型	开启主动回错误应答	默认值： false, 表示不开启主动回错误应答。配置 了ErrLib时，错误处理优先由ErrLib回应答；当没 有配置ErrLib时，配置true则错误处理由front主动回 应答，配置false则错误处理不回复应答
FlowControlVersion	整型	流控版本配置	默认值： 1, 表示使用老的流控逻辑，但是它必须 配置FlowControl的相关配置才会真正生效，这是兼 容以前的逻辑。当时为2时，会调用专门的流控插 件接口。
LdpMsgAutoUnCompress	布尔型	是否开启消息自动解压	默认值： true, 默认情况下，所有经过前置的消息 前置都会进行解压（如果消息压缩的情况下）， 业务可以设置为false，由业务组件自行调用 ILdpMsgReader::UnCompress接口解压
BackendContextCount	整型	Backend实例数	默认值： 1, 一个Backend实例
BackendContexts	数组	上下文参数	非必填，会原样透传给rcm上下文
BindCoreId	整型	绑定CPU的CoreId	CPU核心编号，只有接收端的接收线程绑定。默 认值：-1，不绑定。
SendBindCoreId	整型	发送端异步发送线程绑定 CPU的CoreId	CPU核心编号，用于发送端的异步发送线程绑 定。默认值：-1，不绑定。
RepairBindCoreId	整型	发送端补缺线程绑定CPU的 CoreId	CPU核心编号，用于发送端补缺线程的绑定。默 认值：-1，不绑定。要求版本rcm 1.1.15.6+

4.2.3. 路由

所有经过前置的请求，都会去查找路由表（消息类型为应答时，不会查找路由表），定位消息发送的目的地。
路由配置示例如图所示：

```
"Routes": [
  {
    "Functions": "*", "Systems": "1", "Nodes": "1", "Backend": 1},
  {
    "Functions": "*", "Systems": "5", "Nodes": "250", "Backend": 5},
  {
    "Functions": "11;111;1111", "Systems": "0;2", "Nodes": "65535", "Backend": 1},
  {
    "Functions": "33??", "Systems": "0", "Nodes": "65", "Backend": 2},
  {
    "Functions": "900011", "Systems": "11;12", "Nodes": "*", "Backend": 11},
  {
    "Functions": "*", "Systems": "13", "Backend": 13}
],
```

路由匹配规则如下：

- 1. 路由的查找匹配是由节点号,功能号以及系统号共同决定的，当这三个字段都匹配上了，这一项路由才算命中。
- 2. 路由项从上往下匹配，先匹配节点号，再匹配功能号，接着匹配系统号，一旦匹配成功，则跳出匹配，不会再往下继续匹配，所以尽量将常用的路由项放在前面。
- 3. 功能号只支持数字，英文*，英文?，系统号只支持数字，英文*，范围为[0,255]，节点号只支持数字，英文*，范围为[0,65535]。
- 4. 英文*号只能单独存在，不能与其他组合，例"12*"不支持，"12?*"不支持，"*"支持。
- 5. 问号必须连续且在最后，一个问号代表一位，即问号后面不能再有数字，例"11???2"不支持，"??111?"不支持，"1111?"支持。

针对Nodes的匹配规则特殊说明：

- 1. 如果是Ust兼容消息，则只能匹配不包含Nodes配置项的路由表条目。（即使Nodes配置为"*"，也认为是不匹配）（如果所有路由条目都没有包含Nodes配置项，那么无论如何，都会路由失败）
- 2. 如果是Ldp消息，遇到没有Nodes的路由表条目，则认为该条目的Nodes项符合，继续匹配其他字段

4.2.4. 依赖说明

- 前置机不管是多活还是主备模式都依赖于libtcp.so，即需要配置公共库：tcp，例如：

```
"Components": {
  "Libs": [],
  "GlobalLibs": [
    "tcp"
  ]
}
```

- 前置机对接多活模式核心时，依赖于可靠一致组播（ldp_rcm）插件，即在多活模式（AA）下需要配置ldp_rcm可靠一致组播（ldp_rcm）插件。

4.2.5. 前置自动创建RCM配置

开启条件

- Components配置中加入rcm_proxy组件
- ldp_rcm插件增加ConfigUpdate配置项，值为true
- 增加应答主题配置，即RspTopic配置项
- 增加LocalNetworkNumber配置项，用于指定组播通讯的网卡

使用限制

- 对接双活核心时，核心也必须启用自动创建RCM配置功能

4.2.6. 查询请求跨机房转发

开启条件

由于需要跨机房，所以之前非多进程上下文的服务端无法存在一主多备模式，机房概念也就没意义。所以查询请求跨机房转发是以服务端为多进程为例子展开

由于支持跨机房转发，所以新增环境变量机房名，设置当前进程的机房信息：

```
export HSLDP_ZONE=Zone1
```

* 如果上述环境变量没有设置，或者为""，则认为当前进程没有设置机房信息。（客户端和服务端都需要）

rcm机房配置如下例子：

```
{
  "Zones": [
    {
      "Name": "Zone1",
      "RPO": 0
    },
    {
      "Name": "Zone2",
      "RPO": 0
    }
  ]
}
```

服务端HSLDP_ZONE环境变量状态	服务端rcm中“Zone”配置存在	所服务端rcm中“Zone”配置不存在
未配置环境变量	正常启动	正常启动
配置了环境变量，为Zone3	无法启动	无法启动
配置了环境变量，为Zone1或者Zone2	正常启动	无法启动

如果需要更换环境变量，则需要删除rcm自动生成的上下文配置后，重启才会生效

客户端Backend类型中，目前不使用rcm代理的rcm类型，GW类型，插件类型都不支持

Backend中EnableSession和EnableQueryFirst必须为true，新增QueryFirstMode（可以看配置注释）

配置项说明

下列为无机房模式意思为环境变量无法获取到HSLDP_ZONE机房信息，有机房模式为能获取到机房信息。

下列为理想化状态，所有服务端都为有机房或者无机房。

客户端类型	所有服务端（无机房模式）	所有服务端（有机房模式）
tcp集群模式（无机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效
tcp集群模式（有机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：相当于无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：有效 3、配置为不区分机房或者不配置：有效
tcp非集群模式（无机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效
tcp非集群模式（有机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：相当于无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：有效 3、配置为不区分机房或者不配置：有效
老rcm仲裁backend（无机房模式）	无效（没有一主多备概念）	无效（没有一主多备概念）
老rcm仲裁backend（有机房模式）	无效（没有一主多备概念）	无效（没有一主多备概念）
老rcm非仲裁backend（无机房模式）	无效（没有一主多备概念）	无效（没有一主多备概念）
老rcm非仲裁backend（有机房模式）	无效（没有一主多备概念）	无效（没有一主多备概念）
rcm代理非多分片单机backend（无机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：无效 3、配置为不区分机房或者不配置：有效
rcm代理非多分片单机backend（有机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：相当于无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：有效 3、配置为不区分机房或者不配置：有效
rcm代理非多分片仲裁backend（无机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：无效 3、配置为不区分机房或者不配置：有效
rcm代理非多分片仲裁backend（有机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：相当于无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：有效 3、配置为不区分机房或者不配置：有效
rcm代理多分片backend（无机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：无效 3、配置为不区分机房或者不配置：有效
rcm代理多分片backend（有机房模式）	1、配置为主机房优先：无效，为不区分机房模式 2、配置为本机房优先：相当于无效，为不区分机房模式 3、配置为不区分机房或者不配置：有效	1、配置为主机房优先：有效 2、配置为本机房优先：有效 3、配置为不区分机房或者不配置：有效

4.2.7. 配置项BackendContextCount和BackendContexts绑定规则

配置优先级说明

- 优先级1: BackendContexts[Index] - 如果索引存在, 使用数组元素配置
- 优先级2: BackendContext - 如果数组不存在或索引越界, 使用对象配置 (可能存在共享相同CPU核心的情况)

4.2.8. 前置消息转换组件front_transfer

- 前置消息转换组件是LDP平台为方便客户在前置机上进行LDP消息的转换而提供的LDP前置业务组件, 放置在发布包的bin/front.local目录下, 其功能为转换LdpStruct消息到LdpPack消息和转换LdpPack消息到LdpStruct消息。
- 注意: 该组件依赖libldptransfer.so, 用户使用时需要额外设置环境变量LD_LIBRARY_PATH库路径, 默认为发布包/bin/linux.x64/lib/目录下。

前置消息转换组件示例配置

```
{
  {
    "Lib": "ldp_front",
    "Instance": 0,
    "Timer": 1,
    "/*": "从这里开始为前置消息转换组件配置",
    "FrontBizLibs" : {
      "ReqLibs": ["libldp_front_transfer.so"],
      "RspLibs": ["libldp_front_transfer.so"],
      "FrontTransfer": {
        "PackVersion": 3,
        "LdpStruct2LdpPack": "12345",
        "LdpPack2LdpStruct": "34567",
        "TransferConfigPath": "struct.cfg"
      }
    }
  }
}
```

配置项说明

名称	类型	说明	备注
FrontBizLibs	对象	业务组件配置	组件存放路径: bin/应用名.local。同名的组件只会加载一次,使用的同一个实例
ReqLibs	数组	业务请求处理组件	指定业务请求处理组件, 此处固定配置为libldp_front_transfer.so
RspLibs	数组	业务应答处理组件	指定业务应答处理组件, 此处固定配置为libldp_front_transfer.so
FrontTransfer	对象	前置消息转换组件的配置	
PackVersion	整型	Pack包类型	目前支持配置2(PackV2)或3(PackV3), 默认为3
LdpStruct2LdpPack	字符串	LdpStruct消息转LdpPack消息功能号集合	英文分号分隔,如"11;30?;319",?号表示一个数字,只能出现在最后。只要有"*"号, 则代表支持所有的功能号。不配置则不使用
LdpPack2LdpStruct	字符串	LdpPack消息转LdpStruct消息功能号集合	英文分号分隔,如"11;30?;319",?号表示一个数字,只能出现在最后。只要有"*"号, 则代表支持所有的功能号。不配置则不使用
TransferConfigPath	字符串	转换所需的配置文件或路径	文件后缀要求命名为".cfg", 可以配置文件夹路径, 会读取该路径下所有cfg文件,配置文件格式参照《低延时分布式开发平台_转换组件使用指南》中的配置文件说明一节。

4.2.9. 前置错误转换组件front_error_transfer

- 前置错误转换组件是LDP平台为方便客户自定义返回客户端报错信息的错误号和错误信息, 支持平台报错和业务报错, 放置在发布包的bin/front.local目录下, 其功能为自定义经过前置的错误的错误号和错误信息转换。
- 注意: 该组件依赖libldptransfer.so, 用户使用时需要额外设置环境变量LD_LIBRARY_PATH库路径, 默认为发布包/bin/linux.x64/lib/目录下。

前置错误转换组件示例配置

```
{
  {
    "Lib": "ldp_front",
    "Instance": 0,
    "Timer": 1,
    "/*": "从这里开始为前置消息转换组件配置",
    "FrontBizLibs" : {
      "RspLibs": ["libldp_front_error_transfer.so"],
      "ErrLib": "libldp_front_error_transfer.so",
      "FrontErrorTransfer": {
        "/*": "自定义错误配置文件",
        "ErrorTransferConfigPath": "errormap.ini"
      }
    }
  }
}
```

配置项说明

名称	类型	说明	备注
FrontBizLibs	对象	业务组件配置	组件存放路径: bin/应用名.local。同名的组件只会加载一次,使用的同一个实例
RspLibs	数组	业务应答处理组件	指定业务应答处理组件, 此处固定配置为libldp_front_error_transfer.so

FrontErrorTransfer	对象	前置错误转换组件的配置	
ErrorTransferConfigPath	字符串	转换所需的配置文件或路径	文件后缀要求命名为".ini", 可以配置文件夹路径。

错误转换配置文件说明

```
[range]
1000,1999=1000|系统繁忙,请稍后重试
2000,2999={}|业务处理异常,请联系客服
5000,5999=1000|{}
[enum]
3001={} |前置服务暂时不可用,请稍后重试
3002=3001|参数格式不正确
5001=1001|{}
5002=1001|{}

```

范围配置示例: 起始错误号,结束错误号=映射错误号|映射错误信息
枚举配置示例: 原始错误号=映射错误号|映射错误信息
{ }表示维持原来错误号或者错误信息不转换
枚举的优先级比范围高
错误号的范围为: -2^31 到 2^31 - 1

4.3. 业务框架插件 ldp_bizproc

ldp_bizproc是一个业务框架插件，加载业务程序的so，接收外部请求，根据业务请求的功能号调用业务处理接口，业务进行具体的业务逻辑，并保障业务的高可用和高性能。
ldp_bizproc支持主备和多活两种高可用架构。可选择其中一种高可用架构进行部署。

插件名	支持平台	插件标识	说明
libldp_bizproc.so	linux.x64	ldp_bizproc	业务框架插件

4.3.1. 示例配置

```
{
  "Lib": "ldp_bizproc",
  "Instance": 0,
  "Timer": 0,
  "ConcurrentMode": 1,
  "EnableBizStatusCtrl": true,
  "WatchList": [
    {
      "ProductName": "mdbtest",
      "AppName": "arb",
      "AppNo": 0
    }
  ],
  "ThreadInitTimeoutSecs": 5,
  "TimeLogAlias": "xxxxxxx",
  "EnableSafeMode": false,
  "ForceWriteBinLog": false,
  "EnableTraceRcmRedo": true,
  "EnableFilterReq": true,
  "DefaultFilterShmCount": 10,
  "FilterShmBlockSizeK": 512,
  "Threads": [
    {
      "Count": 1,
      "ThreadBindCpu": [
        1,
        2,
        3
      ],
      "Priority": [
        0,
        1
      ],
      "QueueAlias": 1,
      "PollingLoops": 10000000,
      "SleepTimeUS": 2,
      "MaxQueueSize": 32768,
      "DisableBizChannel": true,
      "MultithreadFrame": {
        "Count": 4,
        "ThreadBindCpu": [
          7,
          8,
          -1,
          -1
        ],
        "PollingLoops": 100,
        "SleepTimeUS": 2,
        "InitQueueSize": 2048,
        "SharedMode": false
      }
    },
    {
      "Count": 1,
      "ThreadBindCpu": [
        4,
        5,
        6
      ],
      "QueueAlias": 2,
      "PollingLoops": 0,
      "SleepTimeUS": 5,
      "MaxQueueSize": 32768
    }
  ],
  "FunctionThreadAffinity": [
    {
      "Functions": "800?;567??",
      "QueueAlias": 1,
      "LoadStrategy": 1
    }
  ]
}
```

```

        "Functions": "*",
        "QueueAlias": 2,
    }
},
"PipelineThreads": [
    {
        "Count": 1,
        "ThreadBindCpu": [
            19
        ],
        "QueueAlias": 1,
        "PollingLoops": 0,
        "SleepTimeUS": 20,
        "MaxQueueSize": 32768
    },
    {
        "Count": 1,
        "ThreadBindCpu": [
            19
        ],
        "QueueAlias": 2,
        "PollingLoops": 0,
        "SleepTimeUS": 20,
        "MaxQueueSize": 32768,
        "MultithreadFrame": {
            "Count": 4,
            "ThreadBindCpu": [
                9,
                10,
                -1,
                -1
            ],
            "PollingLoops": 100,
            "SleepTimeUS": 2,
            "InitQueueSize": 2048
        }
    }
},
"TimerThread": {
    "ThreadBindCpu": -1,
    "TimerMilli": 1000,
},
"AppStatusCtrlWhiteFuncList": "123;456;34;1??;6553?;677??;676756",
"BizTimeStampFuncList" : "123;456;34;1??;6553?;677??;676756",
"FuncBlackList": "123;456;34;1??;6553?;677??;676756",
"TimeStampRuleWhiteFuncList" : "123;456;34;1??;6553?;677??;676756",
"TimeStampRuleBlackFuncList": "123;456;34;1??;6553?;677??;676756",
"IgnUnknownSystemModule":true,
"BizDataLibs": {
    "xstdat1": [
        "dataaddin1",
        "dataaddin2"
    ],
    "xstdat2": []
},
"libxstdat.so"
},
"BizFuncLibs": [
    "xstfunc"
],
"LdpErrorRspWithReqMsg" : false,
"HaMode": 0,
"ShardNo" : 1,
"ActiveStandby": {
    "LocalNetworkNumber": "",
    "Tcp": {
        "RdmaRecvThreadCount": 1,
        "RdmaIp": "3.3.3.102",
        "Address": "",
        "Port": 8001,
        "SocketReuseAddress": true,
        "HeartbeatIntervalSec": 20,
        "HeartbeatTimes": 3,
        "Performance": {
            "Onload": 1,
            "ThreadCount": 1,
            "ThreadBindCpu": [
                7,
                -1,
                9
            ],
            "MergeSendRecvThread": true,
            "EnableAsyncSend": false,
            "EnableDirectRecv": false,
            "SendWaitTimeMilli": 0,
            "MaxSendBuffSizeBytes": 0,
            "MinRecvBuffSizeBytes": 512,
            "MaxConnectionCount": 10000,
            "EnableQuickAck": true,
            "EnableNagle": true
        }
    }
},
"RedoRead": {
    "ReadPath": "./ldpredo_master"
},
"EnableMaxContinuousMsgNo": true
},
"EnableRedoSupplement": false,
"ReplyCompressThreshold": 1000,
"PushCompressThreshold": 1000,
"RedoWrite": {
    "WritePath": "./ldpredo",
    "MaxFileSize": 1024,
    "WriteFlag": true,
    "PriorityGetTransNo": false,
    "CompressThreshold": 1000,
    "AsyncRedo": {
        "Count" : 1,
        "PollingLoops" : 0,
        "SleepTimeUS" : 0,
        "MaxQueueSize" : -1,
        "ThreadBindCpu" : [0]
    }
},
"IsReplyMode": false,
"RedoWrite": {

```

```

    "ReqLogWrite": {
      "WriteFlag": true,
      "WritePath": "./ldp_bizproc_reqlog",
      "MaxFileSizeMB": 100
    },
    "RspLogWrite": {
      "WriteFlag": true,
      "WritePath": "./ldp_bizproc_rsplog",
      "MaxFileSizeMB": 100
    },
    "LocateFunction": 100,
    "Replay": {
      "EnableReproducible": true,
      "LocateFunction": 100,
      "Persistent": {
        "WritePath": "replaylog",
        "MaxFileSizeMB": 100
      }
    },
    "TopicMapping": [
      {
        "ReqTopic": "trade",
        "RspTopic": "result"
      }
    ],
    "//": "路由, Backend配置合并",
    "Rpc": {
      "Routes": [
        {
          "Functions": "*",
          "Systems": "*",
          "Topic": "trade",
          "Partitions": [
            1
          ]
        }
      ]
    },
    "//": "路由, Backend配置分离",
    "Rpc": {
      "Routes": [
        {
          "Functions": "*",
          "Systems": "*",
          "Nodes": "*",
          "Backend": 1
        }
      ],
      "Backends": [
        {
          "Id": 1,
          "Type": "AS-TCP",
          "Arb": true,
          "ClusterName": "xst"
        }
      ],
      "EnableBackendAppStatusCtrl": false,
      "EnableBackendBizStatusCtrl": false
    },
    "PushService": [
      {
        "Name": "REPORT",
        "Type": "AA",
        "Topic": "report"
      },
      {
        "Name": "PUSH",
        "Type": "AA",
        "Topic": "push"
      }
    ],
    "CleanInfo": {
      "GroupIndex": 2,
      "MustExist": true
    },
    "//": "Docker到宿主机准换关系",
    "DockerAdressConvert": [
      {
        "DockerIp": "192.168.86.101",
        "HostIp": "10.20.39.146",
        "DockerPort": 13882,
        "HostPort": 65708
      },
      {
        "DockerIp": "192.168.86.102",
        "HostIp": "10.20.39.188",
        "DockerPort": 12882,
        "HostPort": 23908
      }
    ],
    "//": "自动创建RCM相关配置",
    "ProtocolMode": "UDP-MULTICAST",
    "LocalNetworkNumber": "192.168.86.0",
    "HighReliable": {
      "REF": "Default",
      "RxTopics": [
        {
          "UseRcmProxy": false,
          "Name": "trade",
          "PartitionNos": [1]
        },
        {
          "Name": "return",
          "PartitionNos": [1]
        }
      ]
    },
    "//": "配置为对象, 一个Singleton通道",
    "Singleton": {
      "TxTopics": [
        {
          "AliaName": "MyReport_1",
          "Name": "report",
          "PartitionNos": [1, 2, 3]
        }
      ],
      "RxTopics": [

```

```
        {
            "UseRcmProxy": false,
            "Name": "return",
            "REF": "Default",
            "PartitionNos": [
                1
            ]
        },
        {
            "Name": "return1",
            "EnableMultiShard": true
        }
    ]
},
"/": "配置为数组，多个Singleton通道",
"Singleton": [
    {
        "ZonesAutoStart": "Zone3;Zone2",
        "TxTopics": [
            {
                "AliaName": "MyReport_1",
                "Name": "report",
                "PartitionNos": [1,2,3]
            }
        ],
        "RxTopics": [
            {
                "Name": "return",
                "REF": "Default",
                "PartitionNos": [
                    1
                ]
            }
        ]
    }
],
"Tier": {
    "REF": "Tier",
    "TierName": "MultiTest",
    "IsConsumerCreate": true,
    "RxTopics": [
        {
            "Name": "trade",
            "PartitionNos": [
                1
            ]
        }
    ],
    "TxTopics": [
        {
            "Name": "push",
            "PartitionNos": [
                1
            ]
        },
        {
            "Name": "report"
        }
    ]
},
"TimerZonesRelayRoleStart": "Zone1;Zone2",
"FilterPartFunctions": "800?;567??",
"Sync": true,
"RoleChangeStateTimeoutMs":2000
}
}
```

4.3.2. 配置项说明

名称	类型	说明	备注
ConcurrentMode	整型	并发模式	默认值：0，表示单线程，1表示多线程。多线程模式下，Threads.Count之和才能配置大于1
EnableBizStatusCtrl	布尔型	是否启用业务状态控制	默认值：false，不开启
WatchList	数组	守护的观察列表，如果观察列表中，所有人都退出（存在异常退出），则本进程将主动退出	若要使用守护功能，需要额外配置守护插件，且本插件的WatchList应该包含在守护插件的WatchList中，否则watch失效（不会校验有效性）
ProductName	字符串	产品名	默认值：本产品名
AppName	字符串	应用名	默认值：本应用名
AppNo	整型	应用编号	默认值：本应用编号
ThreadInitTimeoutSecs	整型	业务线程初始化超时时间，单位秒	默认值：5秒，最小值为1秒
TimeLogAlias	字符串	设置时延日志服务别名	默认值：业务框架插件名
EnableSafeMode	布尔型	是否启用安全模式	默认值：false。如果为false，则和原来一致。如果为true，则支持异常消息跳号。可以查看《异常消息跳号说明》章节
ForceWriteBinLog	布尔型	是否强制写binlog文件	默认值：false。如果为true，则一定会记录请求到binlog中，否则根据写入级别来定
EnableTraceRcmRedo	布尔型	是否链路跟踪备机主备同步	默认值：true。如果为false，则不会记录409功能号
EnableFilterReq	布尔型	是否开启核心去重功能	默认值：false。如果为true，则会根据前置的请

			求序号，做重复的消息过滤
DefaultFilterShmCount	整型	提前预热创建的去重共享内存块数	默认值：10, EnableFilterReq才有效，可配置0，根据前置去重队列数配置，过多会浪费
FilterShmBlockSizeK	整型	去重队列分块存储，单块的大小，单位为K字节	默认值：512，即512K字节。常规情况下，一个去重队列会使用1-2块
AppStatusCtrlWhiteFuncList	字符串	允许备机处于ToInactive时接入的功能号列表，只对主备模式生效	英文分号分隔, 如"11;30?;319", ?号表示一个数字, 只能出现在最后 默认是空。不支持*号
BizTimeStampFuncList	字符串	支持时延记录的功能号集合	英文分号分隔, 如"11;30?;319", ?号表示一个数字, 只能出现在最后 默认是空。只要有"*"号，则代表支持所有的功能号
FuncBlackList	字符串	需要忽略的功能号集合（黑名单）	英文分号分隔, 如"11;30?;319", ?号表示一个数字, 只能出现在最后。不配置则不使用。不允许存在"*"或者"0", 否则会启动失败
TimeStampRuleWhiteFuncList	字符串	时延规则触发功能号白名单，支持? 和 *	英文分号分隔, 如"11;30?;319", ?号表示一个数字, 只能出现在最后 默认是空。只要有"*"号，则代表支持所有的功能号
TimeStampRuleBlackFuncList	字符串	时延规则触发功能号黑名单，支持?	英文分号分隔, 如"11;30?;319", ?号表示一个数字, 只能出现在最后。不配置则不使用。不允许存在"*"或者"0", 否则会启动失败。白名单和黑名单都存在的时候，只有黑名单生效
IgnUnknownSystemModule	布尔型	判断如果对应数据组件没加载，是否跳过加载功能组件。完成核心的正常启动	默认值：false, 如果对应数据组件未加载，有多余功能组件，则加载启动失败
BizDataLibs	对象	可配置多个业务数据扩展的数据组件名	每个成员表示一个数据组件动态库名
xstdata1	字符串数组	数据组件名	每个数组成员表示一个业务数据扩展组件动态库名
BizDataLibs	字符串数组	数据组件名	每个成员表示一个数据组件动态库名，不支持配置业务数据扩展组件
BizFuncLibs	字符串数组	数据组件名	每个成员表示一个功能组件动态库名
Threads	对象数组	工作线程相关	
Count	整型	线程个数	本组工作线程的个数，默认值：1
ThreadBindCpu	整形数组	本组线程组的每个线程绑定cpu的序号	本组线程组的每个线程绑定cpu的序号，默认值：-1，表示不绑定 例如：配置[1, -1, 3]，线程个数配置为4，则表示该线程组中线程编号0绑定CPU序号1，线程编号2绑定CPU序号3，线程编号1和4不绑定CPU
Priority	整形数组	每个队列的优先级	默认为1个队列，值必须大于等于0，重复值被忽略。值越大优先级越低，优先级高的队列被调度到的几率大于等于优先级低的队列，具体差异取决于调度算法。 目前使用完全公平调度算法，该算法相邻优先级之间调度次数比约为1.25。允许的优先级最大值为39。
QueueAlias	整型	队列别名	默认值：0，不同的线程组该值不能相等
PollingLoops	整型	工作线程轮询次数	（1）配置大于0时，工作线程会轮询队列，连续轮询PollingLoops次以后，仍然没有请求，则工作线程睡眠SleepTimeUS微秒后再轮询队列。 （2）配置等于0时，工作线程被事件通知去从队列获取请求处理
SleepTimeUS	整型	睡眠时间	默认值：0，表示不睡眠，即一直轮询，单位微秒。PollingLoops大于0时有效。
DisableBizChannel	布偶型	是否关闭当前线程组调用私有接收发送接口	默认值：false，表示当前线程组默认开启。为true则表示当前线程组不需要调用默认私有接口
MaxQueueSize	整型	工作线程队列积压请求个数上限	默认值：32768，表示一个工作线程的队列最大积压的请求个数
MultithreadFrame	对象	多线程框架相关	
Count	整型	线程个数	本组工作线程的个数，默认值：4
ThreadBindCpu	整形数组	本组多线程框架的每个线程绑定cpu的序号	本组多线程框架的每个线程绑定cpu的序号，默认值：-1，表示不绑定 例如：配置[1, -1, 3]，线程个数配置为4，则表示该线程组中线程编号0绑定CPU序号1，线程编号2绑定CPU序号3，线程编号1和4不绑定CPU

PollingLoops	整型	工作线程轮询次数，默认值：100	（1）配置大于0时，工作线程会轮询队列，连续轮询PollingLoops次以后，仍然没有请求，则工作线程睡眠SleepTimeUS微秒后再轮询队列。 （2）配置等于0时，工作线程被事件通知去从队列获取请求处理
SleepTimeUS	整型	睡眠时间	默认值：2，表示不睡眠，即一直轮询，单位微秒。PollingLoops大于0时有效。
InitQueueSize	整型	初始化队列个数	默认值：2048,初始化队列的个数。
SharedMode	布尔型	是否开启共享模式	默认值：false，表示不开启，为true时开启。
FunctionThreadAffinity	对象数组	功能号线程亲和性相关	可选
Functions	字符串	功能号	默认为*,表示所有功能号
QueueAlias	整型	队列别名	和工作线程组中的队列别名绑定
LoadStrategy	整型	调度策略	线程调度策略，0-轮流 1-负载均衡，其他值则忽略此项亲和性设置，默认0
PipelineThreads	对象数组	流水线线程相关	
Count	整型	线程个数	本组工作线程的个数，默认值：1
ThreadBindCpu	整形数组	本组线程组的每个线程绑定cpu的序号	本组线程组的每个线程绑定cpu的序号，默认值：-1，表示不绑定 例如：配置[1,-1,3]，线程个数配置为4，则表示该线程组中线程编号0绑定CPU序号1，线程编号2绑定CPU序号3，线程编号1和4不绑定CPU
QueueAlias	整型	队列别名	默认值：0，不同的线程组该值不能相等
PollingLoops	整型	工作线程轮询次数	（1）配置大于0时，工作线程会轮询队列，连续轮询PollingLoops次以后，仍然没有请求，则工作线程睡眠SleepTimeUS微秒后再轮询队列。 （2）配置等于0时，工作线程被事件通知去从队列获取请求处理
SleepTimeUS	整型	睡眠时间	默认值：0，表示不睡眠，即一直轮询，单位微秒。PollingLoops大于0时有效。
MaxQueueSize	整型	工作线程队列积压请求个数上限	默认值：32768，表示一个工作线程的队列最大积压的请求个数
MultithreadFrame	对象	多线程框架相关	
Count	整型	线程个数	本组工作线程的个数，默认值：4
ThreadBindCpu	整形数组	本组多线程框架的每个线程绑定cpu的序号	本组多线程框架的每个线程绑定cpu的序号，默认值：-1，表示不绑定 例如：配置[1,-1,3]，线程个数配置为4，则表示该线程组中线程编号0绑定CPU序号1，线程编号2绑定CPU序号3，线程编号1和4不绑定CPU
PollingLoops	整型	工作线程轮询次数，默认值：100	（1）配置大于0时，工作线程会轮询队列，连续轮询PollingLoops次以后，仍然没有请求，则工作线程睡眠SleepTimeUS微秒后再轮询队列。 （2）配置等于0时，工作线程被事件通知去从队列获取请求处理
SleepTimeUS	整型	睡眠时间	默认值：2，表示不睡眠，即一直轮询，单位微秒。PollingLoops大于0时有效。
InitQueueSize	整型	初始化队列个数	默认值：2048,初始化队列的个数。
TimerThread	对象	定时线程相关	TimerMill配置大于0时，才会开启定时线程
TimerMilli	整型	定时器间隔	定时调用数据组件OnTimer接口的时钟周期，默认值：0，此时定时线程不开启。单位毫秒
ThreadBindCpu	整型	线程绑定cpu的序号	默认值：-1，表示不绑定
LdpErrorRspWithReqMsg	布尔型	平台错误应答是否包含请求消息	默认值：false，默认不发送请求消息
HaMode	整型	高可用模式	默认值：0，0表示主备模式，1表示多活模式，2表示单机模式（核心不读取和使用该模式），3表示分区路由模式
ShardNo	整型	分片号	默认值：0，配置了不为0，则可以在业务组件中通过GetShardNo函数获取
ActiveStandby	对象	主备模式配置	主备模式下才有效
LocalNetworkNumber	字符串	本地网卡的网络号	主备仲裁模式下使用，如果是本地模式的话，可以不用配置，例如：192.168.86.0
Tcp	对象	TCP相关	

RdmaRecvThreadCount	整型	rdma接收线程个数	如果需要使用rdma, 默认1
RdmaIp	字符串	rdma网卡IP地址	如果不使用rdma, 则删除此项配置
Address	字符串	侦听IP地址	侦听IP地址, 不配置表示通配地址
Port	整型	侦听端口	侦听端口, 默认值: 8001
SocketReuseAddress	布尔型	是否端口复用	默认值: true
HeartbeatIntervalSec	整型	心跳时间间隔	默认值: 20, 单位: 秒
HeartbeatTimes	整型	心跳超时次数	默认值: 3
Performance	对象		
Onload	整型	是否开启Onload加速	在onload启动模式时, 默认值: 1, 即开启加速。有效值: 0或1
ThreadCount	整型	通讯线程个数	默认值: 1
ThreadBindCpu	整型数组	通讯线程绑定CPU的序号	默认值: -1, 表示不绑定 例如: 配置[1, -1, 3], 通讯线程个数配置为4, 则表示该通讯线程中线程编号0绑定CPU序号1, 线程编号2绑定CPU序号3, 线程编号1和4不绑定CPU
MergeSendRecvThread	布尔型	是否合并收发线程	默认值: true
EnableAsyncSend	布尔型	是否异步发送消息	默认值: false
EnableDirectRecv	布尔型	是否采用直接接收模式	默认值: false
SendWaitTimeMilli	整型	发送等待时间	默认值: 0, 单位: 毫秒
MaxSendBuffSizeBytes	整型	最大发送缓存	默认值: 0, 表示不限制, 单位: 字节
MinRecvBuffSizeBytes	整型	最小接收缓存	默认值: 512, 单位: 字节
MaxConnectionCount	整型	最大连接数	默认值: 65535
EnableQuickAck	布尔型	是否启用快速ack	默认值: true
EnableNagle	布尔型	是否启用Nagle	默认值: false
RedoRead	对象	读redo相关	
ReadPath	字符串	读redo的目录路径	默认值: /ldpredo_master
EnableMaxContinuousMsgNo	布尔型	是否启用设置到HighReliable的消息序号为最大连续值, 为true, 则使用最大连续值; 为false, 则使用历史最大值	默认值: true
EnableRedoSupplement	布尔型	是否启用Redo跳号补充, 为true, 则跳号补充; 为false, 则跳号截断	默认值: false
ReplyCompressThreshold	整型	应答压缩阈值, 应答消息内容长度超过时开启压缩	默认值: 0, 表示不压缩, 单位字节
PushCompressThreshold	整型	主推压缩阈值, 主推消息内容长度超过时开启压缩	默认值: 0, 表示不压缩, 单位字节
RedoWrite	对象	写redo相关	
WritePath	字符串	写redo的目录路径	默认值: /ldpredo
MaxFileSize	整型	一个redo文件最大大小	默认值: 1024, 单位: 兆
WriteFlag	布尔型	写redo 配置开关	默认值: true, 即默认写redo
IsReplyMode	布尔型	重演模式开关	默认值: false, 即默认不开启重演模式。此项配合重演工具使用, 在重演时才开启
ReqLogWrite	对象	请求落地相关, 不配置即不开启此功能	
WritePath	字符串	请求落地的目录路径	默认值: /ldp_bziproc_reqlog
MaxFileSize	整型	一个请求落地文件最大大小	默认值: 100, 单位: 兆
WriteFlag	布尔型	请求落地配置开关	默认值: false, 即默认不进行请求落地
RspLogWrite	对象	应答落地相关, 不配置即不开启此功能	
WritePath	字符串	应答落地的目录路径	默认值: /ldp_bziproc_rsplog
MaxFileSize	整型	一个应答落地文件最大大小	默认值: 100, 单位: 兆
WriteFlag	布尔型	应答落地配置开关	默认值: false, 即默认不进行应答落地

LocateFunction	整型	二次定位执行的功能号，为true时定位函数返回 LDP_BIZPROC_REDIRECTION 错误码表示需要执行再次定位，否则视为定位失败，-1表示不可重定位	默认值：-1
Replay	对象	重演相关配置	
EnableReproducible	布尔型	是否可重演，为true时Replay配置下的内容才会生效，否则查看配置ReqLogWrite和配置RspLogWrite	默认值：false
LocateFunction	整型	过时的配置，请使用外层的LocateFunction	默认值：-1
Persistent	对象	重演文件落地相关	
WritePath	字符串	重演落地的目录路径	默认值：replaylog
MaxFileSizeMB	整型	一个重演落地文件最大大小	默认值：100，单位：兆
PriorityGetTransNo	布尔型	优先获取Redo事务号	默认值：false，为true时，优先获取Redo事务号
CompressThreshold	整型	压缩阈值，消息内容长度超过时开启压缩	默认值：0，表示不压缩，单位字节
AsyncRedo	对象	异步写redo相关，不配置即不开启此功能	
Count	整型	线程个数	异步写redo线程的个数，默认值：0，即不开启此功能，建议和核心工作线程数保持一致
PollingLoops	整型	异步写redo线程轮询次数，默认值：0	（1）配置大于0时，异步写redo线程会轮询队列，连续轮询PollingLoops次以后，仍然没有请求，则异步写redo线程睡眠SleepTimeUS微秒后再轮询队列。 （2）配置等于0时，异步写redo线程被事件通知去从队列获取请求处理
SleepTimeUS	整型	睡眠时间	默认值：0，单位微秒。PollingLoops大于0时有效。
MaxQueueSize	整型	异步写redo线程队列积压请求个数上限	默认值：-1，表示无限
ThreadBindCpu	整型数组	异步写redo线程绑定CPU的序号	默认值：-1，表示不绑定 例如：配置[1, -1, 3]，异步写redo线程个数配置为4，则表示该异步写redo线程中线程编号0绑定CPU序号1，线程编号2绑定CPU序号3，线程编号1和4不绑定CPU
TopicMapping	对象	请求应答主题映射关系	
ReqTopic	字符串	请求主题名	
RspTopic	字符串	应答主题名	
Rpc	对象	RPC相关	
Routes	数组	RPC路由	
Functions	字符串	功能号	默认为*,表示所有功能号
Systems	字符串	系统号	默认为*,表示所有系统号
Topic	字符串	发送主题名	
Partitions	数组	发送分区列表	默认为空，表示该主题的所有分区
Nodes	字符串	节点号集合	英文分号分隔, 如”11;30;65535”, 支持”*”,不支持”?”。范围是[0,65535]
Backend	整型	目标backendid	必须存在相应的 Backends.Id
Id	整型	编号	[1, 65534]
Type	字符串	核心类型	GW, 即网关 AA, 即多活 AS-TCP, 即TCP主备 AS-RCM 即RCM主备 PLUGIN, 插件交互 RCM, 对接查询核心
Arb	布尔型	是否使用仲裁	当Type为AS-TCP或AS-RCM时有效 默认值false. 当为true时, 启动时应指定zk, 否则报错退出 当为false时, 必须有Sites配置, 否则报错退出
ClusterName	字符串	集群名	当Type为AS-TCP或AS-RCM, 同时设置仲裁模

EnableBackendAppStatusCtrl	布尔	是否开启接出端应用状态控制	式时有效，配置为应用名 可选，默认false，若为true，则客户端会通过remoter心跳包获取服务端的应用状态，状态为AppStatus::Ready后，才会将消息发送出去（生效于Backend，状态同步存在延迟）
EnableBackendBizStatusCtrl	布尔	是否开启接出端业务状态控制	可选，默认false（生效于Backend，状态同步存在延迟）
PushService	数组	主推服务	
Name	字符串	服务名	所有主推服务名必须唯一，应用通过服务名获取对应的服务对象
Type	字符串	服务类型	AA, 通过RCM主推 PLUGIN, 通过插件主推
Topic	字符串	发送主题	当Type为AA时有效
Arb	布尔型	是否使用仲裁	当Type为AA时有效。配置为true时，发送的分区号会根据主备核心接收的分区号自动识别，此时必须配置zk
ClusterName	字符串	集群名	当Type为AA，同时启用仲裁模式时，配置为核心的应用名，默认值为本节点的应用名（表示核心和前置部署在同一个应用节点上）
ProductName	字符串	产品名	当Type为AA，同时启用仲裁模式时，配置为核心的产品名，默认值为本节点的产品名（表示核心和前置属于同一个产品）
PluginName	字符串	插件标识	当Type为PLUGIN时有效
Instance	整型	插件实例编号	当Type为PLUGIN时有效
CleanInfo	对象	清理相关配置	
GroupIndex	整型	所在的清理组序号，清理按组进行	默认0，最大31
MustExist	布尔	是否必须存在标志	默认true，为true表示该节点在清理过程中必须存在，如果不存在或者掉线，则清理失败；如果为false则表示节点非必须存在，如果不存在或掉线，会忽略他
DockerAdressConvert	数组	Docker到宿主机转换关系	
DockerIp	字符串	Docker主机Ip	
HostIp	字符串	Host主机Ip	
DockerPort	整型	Docker内部端口	
HostPort	整型	Host外部端口	
ProtocolMode	字符串	协议类型	支持Udp组播：UDP-MULTICAST;Tcp单播：TCP-UNICAST;Udp单播：UDP-UNICAST;默认Udp组播
LocalNetworkNumber	字符串	本机ip或者网段	
HighReliable	对象	高可靠上下文	
REF	字符串	引用模版	必填，默认引用"Default"模版，必须保证对应模版存在
RxTopics	数组	接收主题	
Name	字符串	主题名	必填
PartitionNos	整形数组	分区号	必填，用于标识能接收哪些分区
Singleton	对象 或 数组	非集群通道	支持多个非集群通道配置，最多8个。对象和数组同时存在时，靠前的配置生效
REF	字符串	引用模版	必填，默认引用"Default"模版，必须保证对应模版存在
UseRcmProxy	字符串	当前主题是否对接纯rcm	默认为true，如果对接纯rcm则必须为false
ZonesAutoStart	字符串	当前进程机房在ZonesAutoStart中存在，则无论什么身份下，当前上下文会优先启动。	以,隔开，支持多机房，字符串比较
TxTopics	数组	发送主题	
AliaName	字符串	别名	可选，如果填写，则必须保证全局唯一
Name	字符串	主题名	必填
PartitionNos	整形数组	分区号	可选，用于标识能接收哪些分区，若不填，则

			必须配置REF
REF	字符串	引用模版	可选，若不填，则必须配置PartitionNos
RxTopics	数组	接收主题	
EnableMultiShard	布尔型	是否启用多分片模式	可选，默认不启用
Tier	对象	集群通道	主备模式下，使用该通道，将启用多进程上下文功能，但要保证主备核心都用同一个TierName
REF	字符串	引用模版	必填，默认引用"Default"模版，必须保证对应模版存在
TierName	字符串	集群名称	非必填
IsConsumerCreate	布尔型	是否创建消费者	创建了消费者，就意味着会接收主机同步过来的redo，默认为true
TxTopics	数组	发送主题	
Name	字符串	主题名	必填
PartitionNos	整型数组	分区号	可选，用于标识可以发送哪些分区，不填则可以发往所有分区
RxTopics	数组	接收主题	
Name	字符串	主题名	必填
PartitionNos	整型数组	分区号	可选，用于标识能接收哪些分区，若不填，则必须配置REF
REF	字符串	引用模版	可选，若不填，则必须配置PartitionNos
TimerZonesRelayRoleStart	字符串	定时器依赖身份启动的机房列表	在开启仅主机房活跃模式下，列表中的机房即使不是主机房，也会启动定时任务。可配置多个，配置多个时以英文分号分隔，例如Zone1;Zone2
FilterPartFunctions	字符串	特定分区要丢弃的请求功能号列表	功能号列表，分区模式下如果请求带255分区，且该节点该分区不可写，那么该请求被丢弃，格式同其它功能号配置
Sync	布尔型	是否启用同步模式	是否开启同步模式，开启时支持写Redo和同步Redo，支持落地持久化文件等相关功能，关闭则不支持上述功能（在DB模式下关闭），默认为true开启。需要注意在关闭的情况下，由于DB模式不启动多进程上下文，请勿配置ldp_arb插件，ldp_copy插件等依赖多进程上下文启动的插件，配置相关插件会导致服务启动失败。具体使用请参考DB模式开发手册。
RoleChangeStateTimeoutMs	整型	身份切换超时时间	身份切换超时时间配置，默认为2000。

4.3.3. 依赖说明

- 业务框架不管是多活还是主备模式都依赖于libtcp.so，即业务框架插件需要配置公共库：tcp，例如：

```
"Components": {
  "Libs": [],
  "GlobalLibs": [
    "tcp"
  ]
}
```

- 业务框架在多活模式下，依赖于可靠一致组播（ldp_rcm）插件，即在多活模式下需要配置ldp_rcm可靠一致组播（ldp_rcm）插件。
- 业务框架在主备模式下，主备同步同步依赖于NFS，NFS的安装可以参考《[NFS安装]》。

4.3.4. 数据组件配置项说明

- 扩展的配置顺序决定了数据对象扩展创建、初始化（包含Init和ThreadInit）的顺序。
- 当新老格式"BizDataLibs"配置项同时存在时，以配置位置在前的"BizDataLibs"配置项为准，忽略位置靠后的"BizDataLibs"配置项。
- 框架依然支持解析以前业务数据组件配置格式，业务框架会通过判断"BizDataLibs"配置项的类型自动判断配置格式，进而保证向前兼容。

4.3.5. MDB数据组件配置项说明

- MDB数据组件是LDP平台为方便用户使用开发工具以及MDB内存数据库而提供的组件。MDB数据组件启动时会加载AresStudio开发工具生成的业务功能组件，同时借助于MDB内存数据库来存放业务数据。AresStudio的开发指南参见《低延时分布式开发平台_开发工具使用指南.pdf》。
- MDB数据组件配置如下:其中,BizDataDir指定数据组件目录，默认是\$AppName.local; BizDataLibs指定MDB数据组件名称,固定配置为libuftmdb_data.so

```

"BizDataDir": "mdb.local",
"BizDataLibs": [
    "libuftmdb_data.so"
]

```

MDB数据组件示例配置

```

{
    "Lib": "ldp_bizproc",
    "Instance": 0,
    "Timer": 0,
    "ConcurrentMode": 1,

    "//": "从这里开始为MDB数据组件配置",
    "BizDataConfig": {
        "PackerMaxCount": 256,
        "ProcessThreadCount": 2,
        "DefaultBatchSize": 1,
        "SqlCacheSizeLimit": 1000,
        "UpdateIndexWaitReadTimeoutMilli": 10,
        "RedoFragmentEnabled": 1,
        "RedoFragmentSize": 10240,
        "SeqSkipMilliSecondWhenAllocError": 1000,
        "RpcSequenceAfterFuncNo": 61234,
        "RpcSequenceAllocFuncNo": 61235,
        "ExportTableToFilePath": "/home/exportfile",
        "ExportTableToFileThreadCount": 8,
        "BizLibs": [
            "pub",
            "secu"
        ],
        "ToActiveFuncList": "800014;800005",
        "ToActiveFuncSyncList": "800024;800025",
        "ToActiveFuncSyncTimeoutSec": 100,
        "MaxSequenceInCrement": 10000,
        "SnapshotDir": "./",
        "SnapshotInterval": 60,
        "SnapshotMoment": "21:40:15:30",
        "SnapFileMaxCount": 3,
        "MdbGenSnapDir": "./snap",
        "SnapCommitSqnNum": 1,
        "SnapCommitMs": 1000,
        "VarcharNullReturnEmpty": 1,
        "Transaction": {
            "TransactionTimeOutMilli": 5
        },
        "Function": {
            "SqlFunction": 508902,
            "SqlFunctions": [508902, 508903],
            "TimerFunctions": [
                {
                    "Function": 800000,
                    "InterVal": 5,
                    "EnableTrace": true
                },
                {
                    "Function": 800003,
                    "InterVal": 10,
                    "EnableTrace": false,
                    "StartTime": "11:41:00",
                    "StopTime": "11:41:59"
                }
            ]
        },
        "DataBase": "ldptest",
        "UserName": "test",
        "PassWord": "test123",
        "//": "MDB分区权限校验级别,仅网格模式下有效。0:不开启;1:开启校验但不报错,仅记录日志;2:开启校验并报错;其它值:非法,等效于不开启",
        "CheckRecordPartition": 0,
        "//": "是否开启自动写事务,0-不开启,1-开启,其它值-非法,等效于不开启",
        "AutoBeginWriteTrans": 0,
        "//": "db_driver模块的DbClient",
        "DbClient": {
            "ConnectTimeOutSec": 60,
            "DatabaseConnectionMode": 0,
            "Libs": [
                {
                    "//": "连接oracle数据库",
                    "Name": "libuftmdb_dbclient_oracle19.so",
                    "DbType": "oracle"
                },
                {
                    "//": "连接mysql数据库",
                    "Name": "libuftmdb_dbclient_mysql.so",
                    "DbType": "mysql"
                },
                {
                    "//": "连接ob_oracle数据库",
                    "Name": "libuftmdb_dbclient_ob_oracle.so",
                    "DbType": "ob_oracle"
                }
            ],
            "Dss": [
                {
                    "//": "连接oracle数据库参数配置",
                    "LogicName": "usersvr",
                    "DbType": "oracle",
                    "Server": "MY_ORACLE102",
                    "UserName": "system",
                    "PassWord": "system",
                    "Port": "1521",
                    "ReadTimeOutSec": 60,
                    "WriteTimeOutSec": 60
                },
                {
                    "//": "连接mysql数据库参数配置",
                    "LogicName": "usersvr",
                    "DbType": "mysql",
                    "Server": "192.168.86.30",
                    "UserName": "root",
                    "PassWord": "Uftmysql@123",
                    "DataBase": "ldp_mdb",
                    "Port": "3306",
                    "MyCnfName": "./my.cnf",
                    "ReadTimeOutSec": 60,
                    "WriteTimeOutSec": 60
                },
                {
                    "//": "连接ob_oracle数据库参数配置",
                    "LogicName": "usersvr",
                    "DbType": "ob_oracle",
                    "Server": "192.168.86.30",
                    "UserName": "root",
                    "PassWord": "Uftmysql@123",
                    "DataBase": "ldp_mdb",
                    "Port": "3306",
                    "MyCnfName": "./my.cnf",
                    "ReadTimeOutSec": 60,
                    "WriteTimeOutSec": 60
                }
            ]
        }
    }
}

```

```
        "uriType": "op_oracle",
        "Server": "obproxy.ops.openlabs.hundsun.com:3306", "//": "注意：这里的端口必须配置",
        "Port": "3306",
        "UserName": "hs_irs@UF300_38#obtest",
        "PassWord": "hundsun",
        "DataBase": "HS_IRS",
        "ReadTimeOutSec": 60,
        "WriteTimeOutSec": 60,
        "OblbConfigFile": "oblb.cfg"
    }
}

},
"/": "报错信息是否转换成[] []格式，format接口错误信息默认不转换",
"EnableFormatConvert": true,
"/": "对外错误应答信息转换相关配置",
"ExtErrorNoConvert": {
    "/": "转换文件,格式必须为xml",
    "XmlFileName": "externerror.xml",
},
"ShmUseProtect": 1,
"SqlHistoryThreshold": 100000,
"LongTimeTransactionThreshold": 500000,
"EnableForceSqlAuth": false,
"RoundMode": 1,
"/": "是否忽略因子模块加载报错，默认true",
"IgnoreRsLoadError": true,
"/": "表示需要Base64处理的字段",
"LoadFromFileBase64Fields": [
    {
        "/": "用于配置需要进行Base64处理的字段，其中字段以,分割并且TableName不能重复配置",
        "TableName": "table1",
        "FieldName": "field1,field2"
    }
],
"/": "表示文件下场的目标路径，路径长度最大为259，若配置为相对路径，则是相对于workspace/./ExportCsvFiles，配置的时候不用额外写/符号，若使用文件下场功能，则必须配置",
"ExportToFilePath": "xxx/xxx/xxx",
"/": "表示是否开启文件和内存数据库的字段类型校验，0表示关闭，1表示开启，默认为0",
"LoadTableFromFileCheckFieldType": 0,
"/": "数据组件运行模式，不配置时默认为MDB模式， 0：MDB模式，1：DB模式",
"LoadMode": 0,
"/": "数据源访问策略",
"DataSourceAccessPolicy": [
    {
        "/": "线程组的队列别名，对应bizproc->Threads下的QueueAlias",
        "QueueAlias": [1],
        "/": "数据源名称，对应DbClient->Dss中的数据源名称",
        "DataSources": ["obtest", "usersvr"]
    },
    {
        "QueueAlias": [2],
        "DataSources": ["goldensvr"]
    },
],
}
}
```

配置项说明

名称	类型	说明
BizDataConfig	对象	MDB数据组件配置
BizLibs	字符串数组	指定业务组件目录，存放开发工具生成的组件，包括datamgr、逻辑/原子服务功能组件、因子组件。该路径为相对路径，相对于HSLDP_HOME目录出发（即bin目录），可以通过在开发工具中的UFT对象和模块属性中配置so目录，从而将业务模块编译的so生成到配置的目录下。 BizLibs配置的路径结构如下： pub libxx_datamgr.so risk --因子组件目录 modules --功能组件目录
PackerMaxCount	整型	Packer池最大数量，默认值：256
ToActiveFuncList	字符串	切主功能号列表，多个功能号按分号隔开，按配置顺序依次执行，若失败则提前终止，后续的功能号不执行，功能号如果重复不会去重
ToActiveFuncSyncList	字符串	切主功能号列表，多个功能号按分号隔开，按配置顺序依次执行，若失败则提前终止，后续的功能号不执行，同步等待执行完成后才执行 ToActiveFuncList中的功能号，功能号如果重复不会去重
ToActiveFuncSyncTimeoutSec	整型	ToActiveFuncSyncList中的功能号，同步等待执行完成的时间，单位：秒，默认值：(uint64_t)-1
MaxSequenceIncement	整型	序列号最大增量，默认值：10000
ProcessThreadCount	整型	流水线并发数据加载处理线程个数，默认值：2
DefaultBatchSize	整型	默认批量遍历大小，默认值：1,有效范围[1,256]
SqlCacheSizeLimit	整型	每个事务控制的Sql缓存条数上限，默认值1000,有效范围[0, intmax]

UpdateIndexWaitReadTimeoutMilli	整型	更新索引操作，如被更新的记录被其他事务读取，超时等待时间（毫秒），默认值10，有效范围[0, 10000]
RedoFragmentEnabled	整型	是否开启Redo分片功能：0-不开启，1-开启，默认0（主备机必须配置一致）
RedoFragmentSize	整型	单个Redo分片的大小，单位KB，范围[1KB,1GB]，默认10MB, 10 * 1024 = 10240
SeqSkipMilliSecondWhenAllocError	整型	序列号远端预取错误时，跳过该序列号接下去的N毫秒取号段请求，默认值1000，有效范围[0, 100000]
RpcSequenceAfterFuncNo	整型	序列号Rpc申请号段后，调用的功能号，默认值-1，类型为int64
RpcSequenceAllocFuncNo	整型	序列号Rpc申请号段功能号，默认值-1，类型为int64
ExportTableToFilePath	字符串	导出文件的路径，需要注意的是最后一级的文件夹不需要加/，默认为工作目录下/ufmldb_data_dir/ufmldb_data_dir/export_files
ExportTableToFileThreadCount	整形	导出文件的线程数量，默认为4，允许配置的范围为[1, 16]
SnapshotDir	字符串	数据组件生产快照目录, 默认值：“/”
SnapshotInterval	整型	数据组件生产快照间隔，配置代表具备生成快照功能，值乘以TimerThread中的TimerMilli为一个周期
SnapshotMoment	字符串	数据组件生产快照时刻，多个时刻按分号隔开，时刻精确到时分，例如1530
SnapFileMaxCount	整型	数据组件预留最大的快照数量，默认值：3
MdbGenSnapDir	字符串	内存数据库生成快照目录，默认值：“/snap”
SnapCommitSqnNum	整型	内存数据库快照事务提交数，默认值：1
SnapCommitMs	整型	内存数据库快照事务间隔，默认值：1000
VarcharNullReturnEmpty	整型	设置变长字段存储着null，读取时是否转化为"",合法值:0（默认）- 变长字段存储值null读取时给出null，1 - 变长字段存储值null读取时给出"",其它值-非法,等效于不开启
Transaction	对象	内存数据库事务配置
TransactionTimeOutMilli	整型	不同事务插入、删除、修改同一条记录时的等待时间，小于0为默认值，默认是5毫秒，配置成0表示不等待 单位：毫秒
Function	对象	SQL功能号配置
SqlFunction	整型	sql请求功能号，默认为508902
SqlFunctions	整型数组	sql请求功能号，最多支持配置8个，不能重复
TimerFunctions	对象数组	定时任务功能配置
Function	整型	定时任务功能号
InterVal	整型	定时任务时间间隔，值为定时器间隔 TimerMilli的倍数,例：TimerMilli为1000，InterVal为2，表示每隔2s调用一次配置功能号
EnableTrace	布尔型	是否开启全链路打点，true表示开启全链路打点，false表示关闭全链路打点，默认false
StartTime	字符串	一天中开始执行的时间（当前时间>=StartTime才执行，不保证该时刻一定会执行），格式HH:MM:SS，默认值"00:00:00"，必须和StopTime同时存在或同时省略，且小于等于StopTime
StopTime	字符串	一天中结束执行的时间（当前时间<=StopTime才执行），格式HH:MM:SS，默认值"23:59:59"，必须和StartTime同时存在或同时省略，且大于等于StartTime
DataBase	字符串	内存数据库名，不配置默认为：产品名_节点名
UserName	字符串	ufmldb内置系统用户名,默认可以不配置，如果配置不允许为空，长度范围在128以内(不包括128)，不允许带有中文
PassWord	字符串	ufmldb内置系统用户名对应的密码,默认可以不配

		置，如果配置不允许为空，长度范围在128以内(不包括128)，不允许带有中文
CheckRecordPartition	整数	MDB分区权限校验级别,仅网格模式下有效。0:不开启;1:开启校验但不报错,仅记录日志;2:开启校验并报错;其它值:非法,等效于不开启
AutoBeginWriteTrans	整数	是否开启自动写事务,0-不开启,1-开启,其它值-非法,等效于不开启
DbClient	对象	数据库相关配置
DatabaseConnectionMode	整型	数据库连接模式，0-长连接，1-短连接。默认值为1：短连接
ConnectTimeOutSec	整型	数据库连接超时时间,单位秒,注意：建议同时配置ConnectTimeOutSec、ReadTimeOutSec、WriteTimeOutSec超时时间，否则在DB异常场景，会出现卡住等异常
Dss	对象	数据库配置
LogicName	字符串	数据源逻辑名
DbType	字符串	数据库类型： <code>mysql</code> 或者 <code>oracle</code> ， <code>ob_oracle</code>
Server	字符串	数据库服务名或ip地址
UserName	字符串	用户名
PassWord	字符串	用户密码：密文形式保存，通过ca工具生成密文支持加密，搜索“平台配置项加密”查看
Port	字符串	端口
MyCnfName	字符串	<code>mysql</code> 驱动创建连接时，会读取工作目录下的MyCnfName配置的cnf配置文件，设置连接参数。目前主要配置连接的字符集。对于mysql驱动必须增加相对路径或者绝对路径。
ReadTimeOutSec	整型	数据库读取超时时间,单位秒，默认为0,注意：建议同时配置ConnectTimeOutSec、ReadTimeOutSec、WriteTimeOutSec超时时间，否则在DB异常场景，会出现卡住等异常
WriteTimeOutSec	整型	数据库写入超时时间,单位秒，默认为0,注意：建议同时配置ConnectTimeOutSec、ReadTimeOutSec、WriteTimeOutSec超时时间，否则在DB异常场景，会出现卡住等异常
ObIbConfigFile	字符串	obproxy负载均衡的配置文件，默认为空。 ob_oracle支持该功能的客户端驱动版本建议在2.2.8以上，为ob工程师推荐的版本。目前对于ob228的库都必须配置负载均衡文件。
Libs	对象数组	数据库驱动库的so配置
Lib	字符串	需要加载的驱动库配置
Name	字符串	数据库驱动库的so名，例如：so的名称为libufitmlb_dbclient_oracle19.so，则配置ufitmlb_dbclient_oracle19
DbType	字符串	数据库类型： <code>mysql</code> 或者 <code>oracle</code> 或者 <code>ob_oracle</code> 或者 <code>mysql_oracle</code> （dss标签配置同时配置了mysql和oracle）
EnableFormatConvert	布尔型	报错信息是否转换成JSON格式，format接口错误信息默认不转换。 <code>true</code> ：开启转换， <code>false</code> ：关闭转换。
ExtErrorNoConvert	对象	对外错误应答信息转换相关配置
XmlFileName	字符串	转换文件，格式必须为xml
ShmUseProtect	整型	内存数据库共享内存使用保护，当开启功能时，可保证已做保护的共享内存访问的可靠性，不会出现访问非法内存崩溃的问题，但是对性能会有一定影响。0-关闭共享内存使用保护，1-开启共享内存使用保护。默认值为1
SqlHistoryThreshold	整型	记录慢SQL阈值，单位微秒，默认值100000
LongTimeTransactionThreshold	整型	慢事务的时间阈值，需要大于等于0，单位微秒，默认值500000
EnableForceSqlAuth	布尔型	是否开启强制Sql运维功能权限校验，开启后执行Sql运维功能时将会强制校验用户名和密码，默认false
RoundMode	整型	设置SQL向带精度实数字段存入数据时如何取

		整。0表示不调整，1表示四舍五入，默认值1
IgnoreRsLoadError	布尔型	是否忽略因子模块加载报错，默认true，false表示不忽略，不忽略时核心会启动失败
ParallelReplay	json对象	该json对象表示redo并发回放的配置段，如果整个配置段都不存在，表示不使用并发回放
Mode	整数	并发回放模式。0-不使用（默认值），1-普通同步式并发回放。默认值0。无该配置项或者配置值非法时使用默认值
ThreadCount	整数	并发回放线程数。默认4。有效值1到8。无该配置项或者配置值非法时使用默认值
SleepTimeUS	整数	轮询PollingLoops次后，若无redo回放，则休眠SleepTimeUS微秒。默认0.有效值：非负整数。0表示不休眠，忙等。无该配置项或者配置值非法时使用默认值
PollingLoops	整数	轮询redo任务次数。默认值1000.有效值：正整数。无该配置项或者配置值非法时使用默认值
ThreadBindCpu	json数组	并发回放线程的绑核配置。默认值：空数组，不绑核。单个配置值必须为非负整数，否则忽略该值。不检查配置的值是否为真正存在的cpu号，绑核失败时也不会提醒。如果配置的cpu数量比线程数量少，比如cpu配置了2个，线程配置了4个，那么第3、4个线程就不绑核了。
LoadFromFileBase64Fields	json数组	用于配置需要进行Base64处理的字段
ExportToFilePath	字符串	表示进行文件导出时，目标文件夹的路径，可以配置绝对路径和相对路径，若配置相对路径，则相对于workspace/./ExportCsvFiles/，示例为"test/dir"，传入相对路径时前后不需要带"/"，使用文件上下场的情况下必须配置此配置项，需要注意若配置"~/xxx"，则会被认为是相对路径，而不会把~符号展开为当前用户的家路径
LoadTableFromFileCheckFieldType	整数	表示是否需要校验文件中字段类型和mdb中字段类型是否一致，默认为0，不进行校验
LoadMode	整型	数据组件运行模式，0-MDB模式，1-DB模式。默认值为0：MDB模式。当配置为1时，必须配合核心的Sync配置为false，表示不进行redo同步。
DataSourceAccessPolicy	对象数组	数据源访问策略（白名单机制）：在长连接模式下，用于指定哪些线程组（通过 QueueAlias 标识）被允许连接哪些数据源。核心启动时仅连接明确授权的数据源；未在策略中配置的线程组默认禁止建立任何数据库连接。实际连接采用懒连接方式（即功能调用时才建立）。若该策略未配置，则默认所有线程组可访问全部数据源（向前兼容）。

环境变量配置

- 核心启动加载业务功能组件时，需要将配置的功能组件目录（bin/xxx/bin/xxx/modules:bin/xxx/risk）设置环境变量。其中，xxx是指BizLibs下配置的功能组件模块名。例如：`export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HSLDP_HOME/components:$HSLDP_HOME/pub:$HSLDP_HOME/pub/modules:$HSLDP_HOME/pub/risk:$HS`

对外错误应答信息转换相关配置说明

- 配置了ExtErrorNoConvert之后，数据组件将在workspace下寻找[ExtErrorNoConvert→XmlFileName]配置的文件（默认为externerror.xml）进行解析，并将解析后error_source（错误来源，error_sort为1时，此项代表LS功能号）、error_no（内部错误号）、extern_code（外部错误号）之间的映射关系保存在数据组件中。解析转换文件失败时，数据组件将会报错返回。
- 数据组件将只解析同时包含非空error_sort、error_source、error_no、extern_code字段且error_sort="1"的ExternError。

并发回放特别说明

- SleepTimeUS这个值为0时，忙等任务，性能会好，但也会时占满cpu。
- PollingLoops和SleepTimeUS的配置是性能与资源占用的权衡，可以参考bizproc的任务队列的配置方法
- 如果ThreadCount配置为8，则有8个并发线程，每个线程独占一个TC事务控制器。
- 如果仅仅是为了使得并发回放性能最好，那么需要配置绑核，并且绑核的配置和bizproc里处理409功能号的工作线程绑定在同一个NUMA节点的CPU下
- 并发回放线程是与处理409功能号的那个TC事务控制器强绑定的。也就是说，如果处理409功能号的线程不是固定的，那么就会使用不同的事务控制器，也会创建多份并发回放的执行体。所以强烈建议处理409功能使用同一个线程，或固定使用几个线程，不要随意切换，不然会极大的浪费资源。
- 配置示例

```
{
  "BizDataConfig": {
    "EnableForceSqlAuth": false,
    "SqlHistoryThreshold": 0,
    "SnapCommitSqnNum": 10000000,
    "SnapCommitMs": 3600000,
    "///<": "并发回放配置段",
    "ParallelReplay": {
      "Mode": 1,
      "ThreadCount": 4,
      "ThreadBindCpu": [80, 81, 82, 83, 84, 85, 86, 87]
    }
  }
}
```

4.3.6. 功能组件配置项说明

- 通过配置"BizFuncLibs"字段，来确定业务框架加载的业务功能组件库。
- 若配置了"BizFuncLibs"字段，则至少需要配置一个有效的功能组件（非空字符串）
- 若未配置"BizFuncLibs"字段，则默认加载modules路径下的所有动态库

4.3.7. 业务框架自动创建RCM配置

开启条件

- Components配置中加入rcm_proxy组件
- ldp_rcm插件增加ConfigUpdate配置项，值为true
- 核心插件增加非高可用或者高可用通道配置
- 核心插件增加LocalNetworkNumber配置项，用于指定组播通讯的网卡

使用限制

- 目前只有双活模式下才支持自动创建RCM配置
- 前置需要配套升级并启用自动创建配置功能
- 组播网关暂不支持对接该模式的业务框架

4.3.8. 业务框架请求可靠说明

- 新增一个全局可靠队列，缓存指定的可靠请求，保证其不被丢弃
- 新增一个线程用于处理全局可靠队列缓存的请求
- 双活模式下所有请求都可靠，不被丢弃
- 主备模式下，通过功能组件的GetFunctions中的FUNCTIONS表设置RELIABLE标识，保证请求可靠不被丢弃
- 设置方式参考《低延时分布式开发平台_业务组件开发和使用指南》的专项说明

4.3.9. 业务框架请求落地说明

- 支持主备模式下通过配置ReqLogWrite配置项，开启请求落地
- 开启请求落地后，主核心会在指定路径下生成落地文件，备机不会进行请求落地
- 请求落地需要开启核心定时器用于清理需要关闭的落地文件，即业务框架的 Timer 配置项
- 每次启动后，会在指定的路径下生成一个带时间信息的文件夹，用于本次启动的请求落地

4.3.10. 业务框架记录池内存表查看功能说明

- 对于使用了 记录池内存表 的业务组件，在对应的 app 配置中增加 ldp_table_analyse 插件，即可在轻量管控平台查看 记录池内存表 的数据
- ldp_table_analyse 插件使用说明查看本指南 ldp_table_analyse 插件的使用说明
- 轻量管控平台内存表查看功能参考《低延时分布式开发平台_轻量管控平台业务组件开发使用指南》
- 通过管控平台发起的请求会在对应业务框架的 0 号线程被处理，因此推荐将 0 号线程作为管理线程使用
- 目前仅支持 记录池内存表 的查看，不支持 记录池内存表 的修改

4.3.11. mdb场景使用限制

- 较为安全的线程配置要求：工作线程数 + 流水线线程数 + 总多线程框架线程数 + 流水线并发数据加载线程数整体建议小于124个
 - 工作线程数：核心配置中Threads中每个线程组的Count总和
 - 流水线线程数：核心配置中PipelineThreads中每个线程组的Count总和
 - 总多线程框架线程数：Threads和PipelineThreads每个线程组的Count乘以对应MultithreadFrame.Count的总和
 - 流水线并发数据加载线程数：每个表加载的sql分段总和

4.3.12. 异地机房说明

- 异地机房核心配置Bypass通道，接收主题指定为主机房RCM多进程上下文中旁路发送主题
- 异地机房RCM多进程上下文中指定旁路发送主题，必须和主机房RCM多进程的旁路发送主题不同，否则会形成环路
- 异地机房主核心从Bypass通道收到消息后使用RCM多进程上下文同步给备核心，切换为主机房后关闭Bypass通道，同步不控rpo，若主机重启，消息可能丢失
- 异地机房备核心不接收Bypass通道的消息，只接收RCM多进程上下文的消息

4.3.13. 异常消息跳号说明

- 当且仅当EnableSafeMode为ture时生效。
- 配置生效后，核心会根据消息的唯一标识，来进行跳号。
- 目前唯一标识只支持由用户自定义（可以查看《低延时分布式开发平台_业务组件开发和使用指南》）或者消息来自于高可靠上

下文。

- 如果收到用户自定义消息或者高可靠消息时，会比较当前消息是否是异常消息，是则跳过。
- 当收到上述消息后，核心在执行消息前，会将消息塞入共享内存，并设置状态。如果成功后，则将共享内存消息状态设置为成功；若失败，则再次重新拉起核心，会将共享内存的异常消息更新到持久化文件中。并在切主过程中，将消息同步给备机。
- 跳号场景1：高可靠消息导致核心（主机）挂了，核心B切主，核心B切完主机后，跟前置序号协商完，消息还是会导致核心B崩溃，需要人工再次拉起当前核心B，这时候核心B能跳过。再次拉起核心A，这时核心B会将崩溃信息同步给核心A，后续核心A收到相同的请求，不会再次崩溃。
- 跳号场景2：自LDP1.0.V202501.02.000平台版本后，redo异常消息能跳过处理。核心A同步redo消息给核心B（备机），导致核心B崩溃，人工手动拉起核心B，核心B能跳过这一笔redo消息，继续工作。
- 如果多个线程同时执行时挂了，重启以后，可能会跳过不超过线程个数的消息
- 在当前模式下，如果对程序执行了非hslstp终止信号的信号（如kill -9），不是因为遇到异常消息而退出，在程序退出时，还在线程中运行的消息，统统会被标记为异常消息，后续还是以安全模式拉起，会将这些消息跳号处理。（这种场景下，清理工具支持清理当前共享内存的异常消息）

4.3.14. ldp_bizproc插件使用限制

- 在自定义写 redo 的场景下，如果 ldp_bizproc 节点所在机器发生磁盘写满故障，核心会停止写入 redo 文件；即使磁盘空间恢复，redo 写入也不会自动恢复。出现该情况时，建议先清理磁盘空间，再重启核心进程。

4.3.15. NFS安装

安装说明

业务框架主备模式下，主备同步依赖于NFS服务搭建的网络文件共享环境，主备之间的NFS文件挂载需要在系统部署的时候事先挂载好。rpcbind服务辅助NFS服务实现固定端口号以及一些监控功能。

- 业务框架单机或者本地部署的情况下，可以不需要安装NFS和挂载业务框架机器的目录。

安装步骤

1、先确认系统环境是否已经安装NFS和rpcbind，如果已经安装则无需重新安装。执行下图所示的命令，若如下所示说明当前环境已经安装了nfs和rpcbind软件。

```
[root@ldp-1 ~]# rpm -qa|grep nfs
nfs-utils-1.2.3-39.el6.x86_64
nfs-utils-lib-1.1.5-6.el6.x86_64
[root@ldp-1 ~]# rpm -qa|grep rpcbind
rpcbind-0.2.0-11.el6.x86_64
```

2、使用安装包安装NFS和rpcbind

获取安装包nfs-utils-???x86_64.rpm和rpcbind-???x86_64.rpm，安装时用户需先安装rpcbind然后再安装nfs。

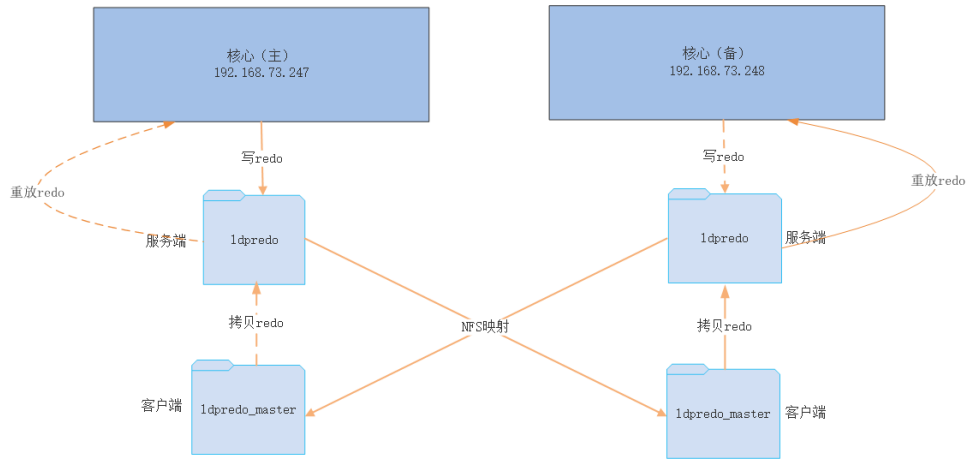
```
[root@ldp-1 ~]# rpm -ivh rpcbind-???x86_64.rpm
[root@ldp-1 ~]# rpm -ivh nfs-utils-???x86_64.rpm
```

- 安装包从系统安装盘获取。

NFS配置说明

两个框架的环境示例配置如下：

IP	工作目录	用户目录	挂载目录	映射目录
192.168.73.247	workspace	/home/ldp	workspace/ldpredo workspace/	ldpredo_master
192.168.73.248	workspace	/home/ldp	workspace/ldpredo workspace/	ldpredo_master



示意图说明：247环境上的ldpredo为原始文件夹，248环境上的ldpredo_master文件夹挂载到247上的ldpredo文件夹下，可以通过ldpredo_master文件夹直接访问247环境上的ldpredo文件夹内的数据；同理，247环境的ldpredo_master文件夹挂载到248环境的

ldpredo文件夹。

NFS挂载说明

1、设置挂载脚本和取消挂载脚本的执行权限，脚本需要放在工作目录下。

挂载脚本	mount_nfs.sh
取消挂载脚本	umount_nfs.sh
设置执行权限	chmod +x mount_nfs.sh umount_nfs.sh

2、检查目录的权限是否正确。

服务端必须保证挂载的目录权限对其他用户开放读取权限，例如要共享/home/ldp/workspace/ldpredo，则需要保证/home、/home/ldp、/home/ldp/workspace、/home/ldp/workspace/ldpredo四个文件夹都对外开放，推荐都使用755权限。

```
[root@ldp-1 ~]# ls -ld /home
drwxr-xr-x. 12 root      root      4096 May 11 19:49 home
[root@ldp-1 ~]# ls -ld /home/ldp
drwxr-xr-x. 11 ldp      ldp      4096 Jul  9 20:43 ldp
[root@ldp-1 ~]# ls -ld /home/ldp/workspace
drwxr-xr-x. 11 ldp      ldp      4096 Jul  6 16:54 workspace
[root@ldp-1 ~]# ls -ld /home/ldp/workspace/ldpredo
drwxrwxrwx.  2 ldp      ldp      12288 Jul 27 00:01 ldp-1
```

3、挂载服务端目录，具体的参数说明参见附录。

挂载前需要在主机和备机的工作目录下创建ldpredo和ldpredo_master文件夹。

192.168.73.247：使用root用户在工作目录下执行提供的挂载脚本，命令如下：

```
[root@ldp-1 worksapce]# sh mount_nfs.sh -s ldp@192.168.73.248:/home/ldp/workspace/ldpredo_master mount to
ldp@192.168.73.247:/home/ldp/workspace/ldpredo
```

192.168.73.248：使用root用户在工作目录下执行提供的挂载脚本，命令如下：

```
[root@ldp-2 worksapce]#sh mount_nfs.sh -s ldp@192.168.73.247:/home/ldp/workspace/ldpredo_master mount to
ldp@192.168.73.248:/home/ldp/workspace/ldpredo
```

4、挂载客户端目录

192.168.73.247：使用root用户在工作目录下执行提供的挂载脚本，命令如下：

```
[root@ldp-1 worksapce]# sh mount_nfs.sh -c ldp@192.168.73.247:/home/ldp/workspace/ldpredo_master mount to
ldp@192.168.73.248:/home/ldp/workspace/ldpredo
```

192.168.73.248：使用root用户在工作目录下执行提供的挂载脚本，命令如下：

```
[root@ldp-2 worksapce]# sh mount_nfs.sh -c ldp@192.168.73.248:/home/ldp/workspace/ldpredo_master mount to
ldp@192.168.73.247:/home/ldp/workspace/ldpredo
```

1. 如果挂载失败，不论是什么错误，必须先执行umount_nfs.sh脚本，且参数必须与执行mount_nfs.sh脚本的参数完全一致。

2. 必须先执行服务端挂载指令才能执行客户端挂载指令，否则将导致客户端挂载失败。

3. 在填写客户端和服务端目录时，需要保证目录尾部没有斜杠，例如/home/ldp/workspace/ldpredo/尾部的/是不允许的。

NFS挂载验证

1、测试服务端已经启动服务，并且开放对应的目录。

这里以192.168.73.247为例：

```
[root@ldp-1 ~]# /etc/init.d/rpcbind status
rpcbind (pid 21661) is running...
[root@ldp-1 ~]# /etc/init.d/nfs status
rpc.svcgssd is stopped
rpc.mountd (pid 21762) is running...
nfsd (pid 21777 21776 21775 21774 21773 21772 21771 21770) is running...
rpc.rquotad (pid 21758) is running...
[root@ldp-1 ~]# showmount -e 192.168.73.247
Export list for 192.168.73.247:
/home/ldp/workspace/ldpredo 192.168.73.248
```

2、测试客户端挂载成功

这里以192.168.73.247为例：

```
/home/ldp/workspace/ldpredo_master from 192.168.73.248:/home/ldp/workspace/ldpredo
Flags:
rw,relatime,vers=4,rsz=524288,wsz=524288,namlen=255,soft,proto=tcp,port=0,timeo=1,retrans=2,sec=sys,clientaddr=192.1
< >
```

3、测试客户端成功读取挂载的网络文件数据

这里以192.168.73.247为例：

在192.168.73.248的/home/ldp/workspace/ldpredo文件夹内创建文件test.txt，并输入一段数据。

```
[root@ldp-2 ~]# vi /home/ldp/workspace/ldpredo/test.txt
123456789
```

在192.168.73.247查看/home/ldp/workspace/ldpredo_master 文件夹是否有新的文件产生，且能正确读取数据。

```
[root@ldp-1 ~]# cat /home/ldp/workspace/ldpredo_master/test.txt
123456789
```

4.4. 可靠一致组播插件 ldp_rcm

ldp_rcm是一个LDP插件，可靠一致性组播，在基于组播的基础之上，保证消息的可靠性和在分布式系统中的一致性，通过简单的API，提供低延迟，高吞吐，高可靠，强一致性的分布式消息总线，并基于主题的发布和订阅，隐藏底层通信实现，同步实现等细节。

插件名	支持平台	插件标识	说明
libldp_rcmso	linux.x64	ldp_rcm	可靠组播

4.4.1. RCM插件示例配置

```
{
  "Lib": "ldp_rcm",          // 插件动态库名，不含扩展名
  "Instance": 0,             // 插件实例编号，不配置时，默认为0
  "Timer": 0,                // 定时器间隔，0表示不启用
  "ConfigUpdate": true,      // 是否启动动态配置，默认为false
  "PrettyUploadConfig": true, // 上传动态配置时是否格式化，默认为true
  "ConfigUpdateDelayMilli": 0, // 配置更新的处理时延，单位：毫秒，默认为0
}
```

4.4.2. RCM插件配置项说明

名称	类型	说明	备注
ConfigUpdate	布尔型	是否启用动态配置更新	不配置默认为false，该配置项与IRcmFactory的SetParam接口开启动态配置相同，如果又调用了接口，则以最后一次接口调用为准
PrettyUploadConfig	布尔型	上传动态配置时是否格式化	默认为true，如果不格式化能减小上传的动态配置的大小，该配置项与IRcmFactory的SetParam接口设置"PrettyUploadConfig"相同，如果又调用了接口，则以最后一次接口调用为准 启用动态配置时配合使用 版本要求：1.1.15.14+
ConfigUpdateDelayMilli	无符号整型	配置更新的处理时延	默认为0，单位：毫秒。当发生配置更新事件时，在（当前时间+ConfigUpdateDelayMilli）时将这段时间内全部的更新事件聚合处理 该配置项与IRcmFactory的SetParam接口设置"ConfigUpdateDelayMilli"相同，如果又调用了接口，则以最后一次接口调用为准 启用动态配置时配合使用 版本要求：1.1.15.14+

4.4.3. RCM可靠组播示例配置

```
{
  "Rcm": {
    "Transports": [
      {
        "Name": "Default",
        "PartitionNo": 1,
        "UseSharedMemory": false,
        "HeartbeatIntervalMilli": 1000,
        "HeartbeatTimeoutMilli": 3000,
        "AckIntervalMilli": 1000,
        "AckTimeoutMilli": 3000,
        "Port": 0,
      },
      {
        "Id": 1,
        "Topic": "MiniTopic",
        "Port": 5001,
        "Addr": "239.0.1.1"
      }
    ],
    "Contexts": [
      {
        "Name": "Default",
        "Ip": "192.168.86.0",
        "IsSingleton": true,
        "McLoop": 1,
        "TTL": 8,
        "MTU": 1500,
        "SendWindowSize": 256,
        "MaxMemoryAllowedMBytes": 256,
        "SocketBufferSizeKBytes": 256,
        "RecordDir": ".",
        "BindCoreId": -1,
        "IsRecord": false,
        "IsTotalOrder": false,
        "RepairPort": 40000,
        "StartTimeoutSecs": 5,
        "MaxCachedMsgCount": 10240,
        "RepairPortStart": 30001,
      }
    ]
  }
}
```

```

        "RepairPortEnd": 40000
    },
    {
        "REF": "Default",
        "Name": "MiniContext",
        "Id": 1,
        "TxTopics": [
            {
                "Name": "MiniTopic"
            }
        ],
        "RxTopics": [
            {
                "Name": "MiniTopic"
            }
        ]
    },
    {
        "REF": "Default",
        "Name": "Tier",
        "IsSingleton": false,
        "SyncIp": "192.168.86.0",
        "SyncAddr": "239.4.1.1",
        "SyncRepairPortStart": 40001,
        "SyncRepairPortEnd": 50000,
        "SyncSendWindowSize": 256,
        "SyncSocketBufferSizeKBytes": 256,
        "UseSameMachine": false,
        "SyncMcLoop": 1,
        "SyncTTL": 8,
        "IsObserver": false,
        "SyncHeartbeatIntervalMilli": 1000,
        "SyncHeartbeatTimeoutMilli": 3000,
        "SyncAckIntervalMilli": 1000,
        "SyncAckTimeoutMilli": 3000,
        "SyncMTU": 1500,
        "SyncRepairPort": 0,
        "SyncMaxDiff": 10000,
        "SyncPort": 5001,
        "AckCount": 1,
        "IsTotalOrder": true
    },
    {
        "REF": "Tier",
        "Name": "HaNode1",
        "Id": 2,
        "TierName": "HaTest",
        "RxTopics": [
            {
                "Name": "MiniTopic",
                "Partitions": [1]
            }
        ]
    },
    {
        "REF": "Tier",
        "Name": "HaNode2",
        "Id": 3,
        "TierName": "HaTest",
        "RxTopics": [
            {
                "Name": "MiniTopic",
                "Partitions": [1]
            }
        ]
    }
],
"Zones": [
    {
        "Name": "Zone_1",
        "RPO": 0,
        "SyncAckPoint": "after_consume"
    },
    {
        "Name": "Zone_2",
        "RPO": 0,
        "SyncAckPoint": "after_consume"
    },
    {
        "Name": "Zone_3",
        "RPO": 0,
        "SyncAckPoint": "after_recv"
    }
],
"InterZones": [
    {
        "Zones": ["Zone_1", "Zone_2"],
        "SyncAckPoint": "after_recv"
    },
    {
        "Zones": [{"Zone_1", "Zone_2"}, {"Zone_3"}],
        "SyncAckPoint": "after_recv"
    }
]
}
}
}

```

4.4.4. RCM可靠组播配置项说明

RCM配置以JSON格式组织，各部分均包含于对象"Rcm"中。

RCM目前包含以下6部分配置："Transports","Contexts","MaterialRange","LdpAdmin","Zones","InterZones"。

在JSON结构上，都是对象"Rcm"的子对象。

- "Transports"和"Contexts"为必需配置，含义分别是通信使用的传输通道（Transports）和参与通信的各成员（Contexts）的配置。
- "MaterialRange"为非必需配置，启用动态配置更新功能时必需，含义是自动分配Id、组播地址等资源时，允许生成的范围。
- "LdpAdmin"为非必需配置，可通过此配置，将RCM接入轻量管控平台（LDP ADMIN）。
- "Zones"为非必需配置，含义是同中心节点间消息同步时的配置和中心自身的配置，目前用于多进程上下文消息同步场景。
- "InterZones"为非必需配置，含义是跨中心节点间消息同步时的配置，目前用于多进程上下文消息同步场景。

Transports: 主题分区配置

"Transports"数组的每个成员表示一个Transport（主题分区，可认为是一个传输通道），或者一个Transport的配置模板。

配置项	类型	含义	备注
Name	字符串	模板名称	此段配置作为模板时，其他配置可通过"REF"指定此模板名，继承此模板。
REF	字符串	引用的模板名	如果不配置，默认引用模板名为Default的模板；若Default模板不存在，则不引用。
Id	整型	Transport编号	有效范围[1, 65535]，作为模板时不需配置，默认值：0。Id为0时认为本配置是一份模板。 一个Transport编号（Id）唯一对应一个主题分区（Topic.PartitionNo）。
Topic	字符串	主题名	最大允许长度为127字节，不应包含操作系统规定的文件名非法字符。
PartitionNo	整型	分区编号	有效范围[1, 65535]，默认值：1。表示是此主题的第几个分区。
Addr	字符串	通讯地址	可以配置为组播地址、单个ip地址、多个ip地址。多个地址以逗号分隔，若第一个地址为单播地址，则后面存在的组播地址无效。 通常为目的地地址。 支持配置一个或多个域名（版本要求:ldp_rcm Version V1.2501.0.0+）
Port	整型	通讯端口	通常为目的地端口。
UseSharedMemory	布尔型	是否启用共享内存通讯通道	默认为false。
HeartbeatIntervalMilli	整型	心跳间隔	发送端空闲时定时发送心跳，单位：毫秒。默认值：1000。
HeartbeatTimeoutMilli	整型	心跳超时时间	接收端一定时间没有收到发送端心跳，则认为该发送端不正常，单位：毫秒。默认为3000毫秒。
AckIntervalMilli	整型	确认间隔	接收端定时回复确认，单位：毫秒。默认值：1000。
AckTimeoutMilli	整型	确认超时时间	发送端一定时间没有收到接收端确认，则认为接收端不正常，单位：毫秒。默认值：3000。
AsynchronousRms	整型	是否采用异步发送	数据发送接口的行为。0：采用同步发送；1：写入内存就返回，采用异步发送。默认值：0。
MandatoryAckers	整型	发送端等待的主ACK个数	默认值：-1，表示发送端需要收到全部主ACK才认为消息已被确认；其他正整数表示发送端收到此配置值的主ACK，即认为消息已经被确认。
RepairDataMode	整型	补缺模式	发送端发送补缺数据的方式。0：使用组播发送，1：使用单播发送。默认值：0。
ReqTopic[已弃用]	字符串	应答对应主题	不为空串时表示当前主题为应答主题，配置项值表示对应的请求主题名，应答主题的分区信息会动态创建，不需要显式配置。
IsBypass	布尔型	是否为旁路主题	true：此主题用作旁路主题；false：此主题为通常消息主题。默认值：false。
UserDefined	字符串	用户自定义配置项	用户可填写自定义内容，长度没有限制，但如果中间有"0"会被截断。默认内容为空。
NakFlowControlCount	整型	补缺流控数量	与NakFlowControlTime一起配合使用，在Time指定的时间内最多能补Count条不重复的消息，超过Count条的消息会被流控无法补缺。
NakFlowControlTime	整型	补缺流控数量	与NakFlowControlCount一起配合使用，RepairPastData是组播模式下有效，在Time指定的时间内最多能补Count条不重复的消息，在Time时间内重复补缺会被过滤。
IsReceiverInitiate	布尔型	是否为接收端主动	是否为接收端主动连接发送端模式，默认为false。设置为true时，主题配置中的IP地址为发送端地址。版本要求:1.1.8.13+

Contexts: 上下文配置

"Contexts"数组的每个成员表示一个Context（上下文，可认为是参与通信的一个成员，包含接收消息、发送消息等功能），或者一个Context的配置模板。

配置项	类型	含义	备注
REF	字符串	引用的模板名	如果不配置，默认引用模板名为Default的模板；若Default模板不存在，则不引用。

Id	整型	Context编号	有效范围是[1, 65535]，默认为0。Id为0则认为本配置是一份模板。
Name	字符串	上下文名称或模板名称	最大允许长度为127字节，不应包含操作系统规定的文件名非法字符。 此段配置作为模板时，其他配置可通过"REF"指定此模板名，继承此模板。
Ip	字符串	网卡地址	需加入组播的IP地址（网卡），可以配置网段。
TransMode	字符串	传输模式	目前支持： "udp_mc": udp组播； "udp": udp单播； "tcp": tcp单播； "efvi_mc": efvi udp组播； "efvi_udp": efvi udp单播； "rdma_mc": rdma组播模式。 默认值："udp_mc"。
IsSingleton	布尔型	是否为单机模式	true：单机模式；false：集群模式。默认值：true。
IsTotalOrder	布尔型	接收消息是否排序	默认值：true，表示排序。
IsRecord	布尔型	是否持久化	默认值：true。 开启持久化时，会在系统共享内存目录下(/dev/shm)生成共享内存文件，在持久化目录下生成持久化文件。 接收端开启持久化功能，需要以开启排序模式为前提（IsTotalOrder配置）。
IsTxRecord	布尔型	上下文中的发送端是否持久化	默认值：true。 上下文的发送端开启持久化，会在系统共享内存目录下(/dev/shm)生成共享内存文件，在持久化目录下生成持久化文件。 IsTxRecord与IsRecord互斥，二者不能同时配置 rcm version 1.1.7.9 新增
IsRxRecord	布尔型	上下文中的接收端是否持久化	默认值：true。 上下文的接收端开启持久化，会在系统共享内存目录下(/dev/shm)生成共享内存文件，在持久化目录下生成持久化文件。 接收端开启持久化功能，需要以开启排序模式为前提（IsTotalOrder配置）。 IsRxRecord与IsRecord互斥，二者不能同时配置 rcm version 1.1.7.9 新增
RecordDir	字符串	持久化目录	支持绝对路径和相对路径，相对路径的基点是工作目录。默认值：工作目录。
IsHandshake	整型	接收端是否序号协商	0：接收端不序号协商，1：接收端需要序号协商。 默认值：0。
StartTimeoutSecs	整型	发送端清流启动序号协商超时时间	单位秒。配置为0时，发送端不进行序号协商。默认值：3。
TxTopics	数组	上下文发送主题信息	每个成员表示发送的主题信息。
Name	字符串	主题名	发送主题的名称。
Partitions	数组	分区号	表示发送主题的分区编号，不配置时表示允许向该主题的全部分区发送。
RxTopics	数组	上下文接收主题信息	每个成员表示接收该主题的分区信息。
Name	字符串	主题名	接收主题的名称。
Partitions	数组	分区号	表示接收主题的分区号，不配置表示接收该主题的全部分区。
Bypass	对象	旁路发送配置	旁路发送端相关配置。上下文内要存在排队模式的接收端，旁路发送端才会起作用。
Topic	字符串	旁路主题	旁路发送的主题，和PartitionNo共同确定一个旁路的Transport。
PartitionNo	整型	旁路主题的分区号	默认值：1。该旁路主题的分区号。
Ip	字符串	旁路发送网卡地址	旁路发送使用的本地IP地址（网卡），可以配置网段。允许和通常收发使用不同网卡。默认值：不配置时自动使用通常消息收发的IP。
TransMode	字符串	旁路发送传输模式	支持的模式见通常消息收发的TransMode说明。默认值：不配置时自动使用通常消息收发的"TransMode"。
SendWindowSize	整型	发送窗口大小	上下文内发送端的发送窗口大小，当发送进度与收到ACK确认进度的差值超过设置值，将暂停发送。默认值：64。

IsCheckMsg	布尔型	是否开启防止窜包及产品名校验	是否开启防止组播窜包及产品名校验，同时开启或关闭二者的校验，默认值： false 。 可通过配置IsCheckProductName独立控制是否校验产品名。
IsCheckProductName	整型	是否单独开启产品名校验	0 ：关闭校验； 1 ：开启校验。默认值：和IsCheskMsg的行为保持一致。
BindCoreId	整型	绑定CPU的CoreId	CPU核心编号，只有接收端的接收线程绑定。默认值： -1 ，不绑定。
McLoop	整型	是否开启本地回环	用于控制跟发送端处于同一机器的接收端是否能够收到组播数据。 0 ：不接收本机组播； 1 ：允许接收本机组播。默认值： 1 。
TTL	整型	存活时间	用于控制数据包在网络上的存活时间（转发跳数），默认值： 8 。
MTU	整型	最大传输单元	单位：字节。默认值： 1500 。注意： rdma 模式下MTU默认为 1476 ，在该模式下修改MTU，不会改变其默认值 1476 。
RecvTimeoutMilli	整型	接收超时时间	底层接收线程调用接收的超时时间，单位：毫秒。默认值： 500 。
EnableQuickAck	整型	是否开启TCP的快速ACK	0 ：使用系统默认值； 1 ：开启。默认值： 1 。
RecordReplayStartPoint	整型	重放持久化的数据的起始序号	接收端重新启动时，从此序号的消息（包含）开始重放。默认值：不开启重启时重放功能。
MaxMemoryAllowedMBytes	整型	上下文使用内存上限	上下文内所有发送端数据缓存的总大小，单位 MB 。默认值： 256 。
SocketBufferSizeKBytes	整型	套接字接收缓存大小	上下文内的接收端套接字的接收缓存大小，单位 KB 。默认值： 0 ，表示使用系统默认值。
RepairPortStart	整型	补缺端口范围下限(包含)	上下文内的发送端会在此范围内选择一个可用的端口作为补缺端口，默认值： 30001
RepairPortEnd	整型	补缺端口范围上限(不包含)	上下文内的发送端会在此范围内选择一个可用的端口作为补缺端口，默认值： 40000
RepairPort	整型	补缺端口	如果上下文只有一个发送端，那么可以通过RepairPort直接指定一个补缺端口，默认值： 0 ，不使用此配置，从补缺端口范围内寻找。注意：如果该端口不可用，会导致发送端创建失败。
AppName	字符串	应用名	最大允许长度为 255 字节，上层可从配置读取接口中获取到此配置。RCM内部不会使用此配置。
RepairPastData	布尔型	接收端序号协商时是否需要补缺历史数据	只在接收端开启序号协商时生效。 true ：清流接收端和带流发送端序号协商后，接收端会补缺发送端缓存的全部历史消息。 false ：接收端序号协商后不需要补缺历史数据，从发送端当前要发送的消息开始接收。默认值： false 。
RepairPastDataWhenReconnected	布尔型	接收端断开重连时是否需要补缺断开期间数据	只在接收端开启序号协商时生效。 true ：会补缺断开期间数据； false ：必须RepairPastData也配置为 false ，不补缺断开期间数据，从最新序号开始接收。默认值： true 。
GenerateIdForRx	布尔型	发送端是否要为接收端生成ID	默认值： false 。注意：发送端Rcm配置项MaterialRange中的ContextId生成范围要被包含于接收端的ContextId生成范围内。
IdGeneratedByTx	布尔型	接收端上下文ID是否由发送端生成	true ：本上下文的ID由发送端分配； false ：本上下文使用配置文件中的ID。默认值： false 。 如果设置为 true ，则不支持RCM工厂的ZK配置模式，也不支持接收端持久化。注意：接收端Rcm配置项MaterialRange中的ContextId生成范围要包含全部发送端能生成上下文ID的范围；目前只支持接收端从一个发送端获取ID
AutoAckWhenNoReceiver	布尔型	无在线接收端时发送端是否自动确认	true ：自动确认； false ：不自动确认，必须等待ACK。默认值： false 。 发送端发送数据时，如果没有接收端发送确认消息，发送窗口将不移动而导致发送卡住，配置为自动确认后，发送端将自动确认，使发送窗口移动。
TransceiverSeparation	布尔型	UM层发送端开启套接字收发分离	true ：开启收发分离； false ：不开启。默认值： false 。该功能只在TransMode为 udp 、 udp_mc 时才有效。如果配置为 true ，且"EnableAccSocket"配置不为 2 （关闭加速），则会同时启用套接字加速。由于Onload默认不开启组播环回，若需要组播发送端和接收端位于同机器通信，应指定其位于相同Onload协议栈。详见配置项"EnableAccSocket"以及"OnloadStackName"的说明。

			注： vma 加速下不支持开启收发分离,若开启会导致发送端与接收端无法通信。 开启时要求上下文补缺端口范围[RepairPortStart, RepairPortEnd)、集群补缺端口范围[SyncRepairPortStart, SyncRepairPortEnd)分别与该上下文接收主题的端口或集群同步端口不能有交集，否则可能会引起通信异常。
EnableShmBusyWait	布尔型	RCM共享内存模式启用忙等	true : 开启忙等； false : 不开启。默认值： true 。
ShmInterruptEventName	字符串	RCM共享内存模式启用非忙等时的事件名称	如果不设置，默认使用系统用户名，最大允许长度为128字节。
NakDetectMicro	整型	接收端补缺检测	接收端发现数据丢包或乱序后开始发起补缺的时间，单位微秒。默认值：1000。最大支持100000，即100毫秒，如果配置0表示不启用该功能。
RepairFlowControlIntervalMill	整型	补缺流控间隔时间	发送端发送补缺数据时流控的时间粒度，单位毫秒。默认值：1。
RepairFlowControlSendLimitKBytes	整型	补缺流控间隔时间内的流量上限	发送端发送补缺数据在单位时间内的流量上限，单位KByte。默认值：10000000，表示不限制。
MaxCacheMsgCount	整型	发送端缓存的消息数	发送端缓存的消息数，即使已经被确认也不会删除。默认值：1。
ShmContext	字符串数组	共享内存通信模式发送端上下文名	开启共享内存通信模式时，对应发送端的上下文名。默认为空。 rcm version 1.1.14.3 以后支持使用UseSharedMemory替换ShmContext
UseSharedMemory	布尔型	控制上下文是否使用共享内存通信	与ShmContext配置二选一，如果配置ShmContext，则只会接收配置的发送端上下文。如果不想配置ShmContext，则需要将本配置UseSharedMemory配置为 true ，打开本配置后，接收端会自动查找配置内的所有共享内存的发送端。默认为 false 。 rcm version 1.1.14.3 以后支持使用UseSharedMemory替换ShmContext
UsePreAllocate	布尔型	是否开启内存预分配	默认值： true
NoReceiverNoSend	布尔型	是否仅在处于连接状态的分区中做负载均衡	true : 仅在在线的分区中做负载均衡； false : 在所有分区中做负载均衡。默认值： false 。
IsSeparateLog	布尔型	是否开启上下文独立日志	默认值： false 。
AsyncProcess	整型	是否异步处理消息	0 : 应用直接在OnMessage中处理消息。 1 : 应用异步处理消息，需要通过SendAck接口通知底层消息处理完成。默认值： 0 。
AllowPresetTopicType	字符串	允许预设置的主题类型	目前支持： "Tx": 发送主题； "Rx": 接收主题； "Bypass": 旁路主题； 默认值：""，不允许预设置主题。 预设了该主题类型，在上下文添加相应主题时，即使该主题还不存在具体的主题分区配置，也不会报错。目前接收主题的预设只限于配置，当要创建实际的rcm接收端前，需要动态创建真实的接收主题分区。
UserDefined	字符串	用户自定义配置项	用户可填写自定义内容，长度没有限制，但如果中间有"0"会被截断。默认内容为空。
RecvBindAddr	布尔型	接收套接字是否绑定组播地址	true : 绑定； false : 不绑定。默认值： false 。 如果为 true 则接收端只能收到组播地址+端口全匹配的报文，如果为 false 则接收端能收到端口匹配的报文。只有当该上下文的所有接收主题的组播地址+端口完全相同时才允许开启该配置，否则会导致收不到部分主题的数据。 注：该配置项只有在udp组播模式下可用，并且暂时只支持Linux、Windows下无效。
TxMaxHistoryMsgCount	无符号整型	发送端最多保留的已被确认的历史消息个数	开启持久化模式(IsRecord: true)，最大可以保留已经被接收端确认的消息的个数，超过保留个数的消息不能再被补缺，使用该配置后会自动清理不能再被补缺的持久化磁盘文件。允许和TxMaxHistoryMsgTimeMill同时配置，同时配置时，二者满足其中一个条件就会删除消息，默认值：保留所有历史消息。 不开启持久化模式(IsRecord: false),此时TxMaxHistoryMsgCount的行为与MaxCacheMsgCount配置项保持一致，此时默认值：1。

			注意事项：TxMaxHistoryMsgCount与MaxCacheMsgCount互斥，二者只能配置其中一个。 1.1.6.12版本增加
TxMaxHistoryMsgTimeMill	无符号整型	发送端已被确认的历史消息最多保留时间	仅在开启持久化模式下有效，启用该配置后发送端的被全部接收端确认的消息最多会保留配置的时间，超过配置时间后则不能再被补缺，并且会自动清理超时的持久化文件。允许和TxMaxHistoryMsgCount同时配置。同时配置时，二者满足其中一个条件就会删除消息。单位：毫秒，默认值：保留所有历史消息。 注意事项： 1.1.6.12版本增加
IsReplay	布尔型	是否开启重演模式	默认值：false，需要以开启持久化为前提（IsRecord配置）。 如果开启重演模式，重放持久化数据的起始序号（RecordReplayStartPoint配置）会被强制设置为0，同时该上下文发送端会关闭序号协商。重演过程中，上次通过本上下文已经发送过的消息不会被重复发送。
FlushOnStop	整型	上下文停止时缓存区数据写入到持久化文件的选项	如果需要支持机器重启后应用不清理持久化文件启动的场景，可开启该配置（需要以开启IsRecord配置为前提），目前支持： 1：上下文停止时，Tx发送端将缓存区数据写入文件； 默认值：0，上下文停止时不需要将缓存区数据写入文件。 这里指的缓存区数据包括待确认消息和运行状态数据，开启该功能会带来的影响：1.停止时持久化缓存消息会耗时；2.机器重启后，应用启动将未确认消息恢复到缓存会耗时。 如果程序异常终止，缓存消息和运行状态没有被完整写入文件，重启机器会丢失消息，重启机器后再次启动应用会失败。
SendBindCoreId	整型	发送端异步发送线程绑定CPU的CoreId	CPU核心编号，用于发送端的异步发送线程绑定。默认值：-1，不绑定。
NakRequestInterval	整型	接收端Nak请求间隔	用于控制丢包场景下大量接收端可能存在的大量Nak请求。默认值：10，单位毫秒。
TrigQuickNakCount	整型	接收端触发快速Nak的乱序包个数	当接收端收到该配置数量的乱序包后会去触发Nak补缺请求。默认值：3，收到3个乱序包后触发Nak补缺请求。
Zone	字符串	所属中心	上下文所属的中心名，默认为空。
ArbPriority	整型	仲裁优先级	集群进行主备切换时候，假如序号进度相同的情况下，数字越高则优先级越大，按优先级从大到小的顺序切主机。
EnableProducerBusyWait	整型	多进程上下文生产者是否忙等RPO	当多进程上下文的生产者向备机同步完一条消息后，需要等待RPO。此配置可以选择等待RPO的方式：1表示轮询忙等模式，2表示事件通知模式
IsMandatoryAcker	布尔型	是否必须向发送方确认收到数据	如果设置为false，表示该接收端不影响发送端发送，具体表现为： 1.该接收端不影响发送端与正常接收端的通信。 2.该接收端不影响发送端的消息缓存释放。 例如它接收慢或处理慢时不会反压发送端发送速度，不会导致发送端消息缓存无法释放。 可靠性说明： 1.如果发送端开启持久化，能够保证该接收端数据接收的可靠性。 2.如果发送端不开启持久化，不保证接收的可靠性，例如在发生网络丢包时，消息可能丢失。 目前只支持配置普通上下文接收端，默认值：true。 注意事项：一个主题分区的接收端中如果只有配置IsMandatoryAcker为false的接收端，发送端行为和不开接收端时一样。 版本要求：1.1.14.4+
EnableRcmProtocolCheck	布尔型	校验收到报文的合法性	默认值true，true表示收到网络报文后，会校验报文是否属于Rcm协议，校验报文是否合法。配置为false，关闭校验。对普通上下文和多进程上下文内的收到的所有报文均会生效。 版本要求：1.1.14.11+
EnableSyncLatencyMetric	布尔型	是否开启主备同步时延度量	开启时，调用相关管理功能接口时可观察到主备同步的时延信息。 默认值：false，表示不开启。

			版本要求：1.1.14.11+
CheckNameConflict	布尔型	是否开启校验上下文冲突	开启时，会在共享内存下生成一个名为“上下文名.files”的文件，用于防止同一台机器中启动了同名的上下文。 默认值： true ，表示开启。 版本要求：1.1.14.12+
TimeMillOfSingleStatisticalPeriod	整型	每个统计周期的时长	单位：毫秒。开启主备同步时延度量时，还会将每个统计周期内的时延数据分别统计分析，用于控制每个统计周期的时间长度 默认值： 1000 版本要求：1.1.15.3+
CountOfSingleStatisticalPeriod	整型	每个统计周期的笔数	开启主备同步时延度量时，还会将每个统计周期内的时延数据分别统计分析，用于控制每个统计周期的笔数 默认值： 1000 版本要求：1.1.15.3+
ShmMemoryFileHole	布尔型	是否开启共享内存文件空洞	true 表示开启文件空洞，使用文件空洞的形式向操作系统申请的共享内存，等到访问的时候才会实际占用共享内存，开启后如果共享内存不足时，可能会导致 bus error 。 false 表示关闭文件空洞，会立即向操作系统申请共享内存，确保操作系统共享内存足够程序才会使用 默认值： true 版本要求：1.1.15.4+
RepairBindCoreId	整型	发送端补缺线程绑定CPU的CoreId	CPU核心编号，用于发送端补缺线程的绑定。默认值：-1，不绑定。要求版本rcm 1.1.15.6+
SendLockMode	整型	发送端三个发送接口保证线程安全的策略	枚举值 0：表示使用自旋锁等待，性能较好，1：表示使用普通锁等待。默认值:0。要求rcm版本 1.1.15.6+
EnableAccSocket	整型	套接字是否启用加速模式	枚举值，合法取值： 0表示不启用加速套接字，使用原生的socket接口创建套接字，该套接字是否会被加速取决于加速库是否捕获了该接口并创建加速套接字； 1表示启用加速套接字，并且按加速模式创建。此时即使全局默认不开启加速，该套接字也会被加速，相当于白名单的机制； 2表示启用加速套接字，并且按非加速模式创建。注意这种模式下，即使全局开启了加速，该套接字也不会被加速，相当于黑名单的机制。 默认值：0。 目前仅支持控制Onload加速的行为 注意，Onload加速默认不开启组播环回。故当使用udp组播进行通信，且发送端、接收端位于相同机器时，若启用加速套接字，并且按加速模式创建，需要使用"OnloadStackName"系列配置，指定发送端与接收端位于同名Onload协议栈，并且设置环境变量EF_MULTICAST_LOOP_OFF=0，才可正常通信。 要求rcm版本 1.1.15.7+
OnloadStackName	字符串	加速套接字所属的Onload协议栈名	默认值：空字符串。不包括"0"结束符，最大支持8字节。仅在"TransMode"为"udp"或"udp_mc"时，使用onload加速，且启用了加速套接字（见"EnableAccSocket"配置说明）的情况下生效。本配置生效时，会覆盖onload的EF_NAME环境变量。要求rcm版本 1.1.15.7+ 且hslp版本 1.0.15.15+
TxOnloadStackName	字符串	发送端使用的加速套接字所属的Onload协议栈名	默认值：空字符串。如果不是空字符串，则发送端创建加速套接字时，本配置会覆盖OnloadStackName，本配置优先生效。其余限制和OnloadStackName相同。要求rcm版本 1.1.15.7+ 且hslp版本 1.0.15.15+
RxOnloadStackName	字符串	接收端使用的加速套接字所属的Onload协议栈名	默认值：空字符串。如果不是空字符串，则接收端（包括集群同步所使用的）创建加速套接字时，本配置会覆盖OnloadStackName，本配置优先生效。其余限制和OnloadStackName相同。要求rcm版本 1.1.15.7+ 且hslp版本 1.0.15.15+
BypassOnloadStackName	字符串	旁路发送使用的加速套接字所属的Onload协议栈名	默认值：空字符串。如果不是空字符串，则旁路发送端创建加速套接字时，本配置会覆盖OnloadStackName，本配置优先生效。其余限制和OnloadStackName相同。要求rcm版本 1.1.15.7+ 且hslp版本 1.0.15.15+
CallBackMsgCountEachRound	整型	单轮回调的最大消息数量	默认值：64。表示最大会连续回调64条消息后才会处理通信接收，改大此配置后，可能会导致连续回调消息数量过多导致接收通信线程被卡住。要求rcm版本 1.1.15.9+

ProducerMemPreallocateCount	整型	预分配的内存块个数	默认值：3。表示每次预分配的内存块个数为3，设置此配置后，按照该配置值决定预分配的内存块个数。实际中该值可配置成大于当前所设计的通过Producer同步的最大线程数的值，即可避免同步申请内存。要求rcm版本 1.1.15.11+
AllowDataRotate	布尔值	是否允许数据轮转的功能	开启数据轮转后，才允许用户通过数据轮转接口清理和备份Rcm上下文持久化文件，同时将这些数据从上下文移除，否则用户调用轮转接口无实际效果。默认值：false。要求rcm版本rcm 1.1.15.19+
以下为高可用集群配置，"IsSingleton":false 时有意义			
TierName	字符串	集群名	该上下文所属的集群名称。默认值：上下文名。
SyncIp	字符串	集群同步网卡地址	用于接收集群同步的IP地址（网卡），可以配置网段
SyncAddr	字符串	集群同步地址	用于发送集群同步信息的地址，可以配置组播地址或多个单播地址，地址间以逗号分隔。单播模式时，通常配置为每个集群成员的IP。
SyncPort	整型	集群同步端口	用于发送集群同步信息的端口。
SyncRepairPortStart	整型	集群补缺端口范围下限(包含)	集群主机在此范围内选择一个可用的端口作为补缺端口，默认值：40001，建议不与[RepairPortStart, RepairPortEnd)有交集
SyncRepairPortEnd	整型	集群补缺端口范围上限(不包含)	集群主机在此范围内选择一个可用的端口作为补缺端口，默认值：50000，建议不与[RepairPortStart, RepairPortEnd)有交集
SyncMcLoop	整型	集群同步是否开启本地回环	0：禁止，本机将收不到自己组播出去的数据；1：表示放开本机接收组播数据。默认值：1。
SyncTTL	整型	集群同步的报文存活时间	默认值：8。
SyncMTU	整型	集群同步网络的最大传输单元	单位：字节。默认值：1500。注意：rdma模式下SyncMTU默认为1476，在该模式下修改SyncMTU，不会改变其默认值1476。
SyncMode	整型	集群数据同步模式	0：状态机复制，集群内仅复制序号；1：消息复制，集群内复制完整消息。默认值：0。目前非多进程上下文仅支持状态机复制，多进程上下文仅支持消息复制。
SyncHeartbeatIntervalMilli	整型	集群同步组播心跳间隔	主节点定时发送心跳，单位毫秒。默认值：1000。
SyncHeartbeatTimeoutMilli	整型	集群同步组播心跳超时时间	从节点一定时间没有收到主节点心跳，则认为该主节点不正常，单位毫秒。默认值：3000。
OutOfSyncTimes	整型	集群同步备节点离开集群超时时间（倍数）	备节点心跳超时后，经过一定时间未重连，认为该备机失去同步，将其移出集群。此配置表示多少倍的集群同步组播心跳间隔时间（SyncHeartbeatIntervalMilli），即 超时时间的绝对值=OutOfSyncTimes*SyncHeartbeatIntervalMilli。默认值：6。支持版本 rcm 1.1.15.1+
SyncAckIntervalMilli	整型	集群同步组播ACK间隔	从节点定时回复确认，单位毫秒。默认值：1000。
SyncAckTimeoutMilli	整型	集群同步组播ACK超时时间	主节点一定时间没有收到从节点确认，则认为从节点不正常，单位毫秒。默认值3000。
SyncSendWindowSize	整型	集群间同步的发送窗口大小	表示未收到任何确认时，主节点最多能够发送的排队序号个数，主要用于集群主节点发送流控。默认值：64。
AckCount	整型	接收集群备节点确认数	-1表示需要全部备节点确认。默认值：1。
AsynchronousSqn	整型	是否开启异步方式的集群sqn同步	0：快速同步sqn；1：异步同步sqn。序号复制模式下，表示是否使用异步的发送线程进行状态机序号复制。消息复制模式下，表示是否使用异步发送线程进行消息复制。默认值：0。注：多进程上下文使用此配置需要rcm version 1.1.15.1+
UseSameMachine	布尔型	是否集群中有节点部署在同一台机器上	默认值：false。目前不支持true。
UseCommitted	整型	是否启用两阶段提交	0：不启用；1：启用，默认值：0。
ArbElectionWaitMemberMilli	整型	集群重新选主时等待未在线成员加入超时时间	集群重新选主时，若发现上一轮集群中还有其他备节点未在线，等待其上线的时间，单位毫秒。若超时后该备机还未上线，则其不参与本轮竞选。默认值：1000。支持版本 rcm 1.1.15.1+

TierRmsLimitCount	整型	ms消息与sqn差距容忍范围	如果ms消息多于已经确认的sqn，且差值超出此配置值，则不回ack，则不再接收ms消息，只做序号同步。默认值：不开启此功能，任意时刻都允许接收ms消息。
Zone	字符串	所属中心	上下文所属的中心名，默认为空。
EnableSyncMulticastAdaption	布尔型	是否启用集群同步组播异常时自动切换单播功能	默认值： true 。仅在 udp_mc 模式下有效，当组播断开但单播正常时，自动将集群同步的网络通信方式从 udp 组播切换为 udp 单播模拟组播。切换后，仅会与当前单播连接中的备机通信。切换为单播模拟组播后，无法感知到新备机加入。支持版本 rcm 1.1.14.11+
SyncRepairDataMode	整型	集群内数据补缺模式	集群主机发送补缺数据的方式。 0 ：使用组播发送， 1 ：使用单播发送。默认值： 0 。目前只适用于消息复制的集群，要求版本 rcm 1.1.15.1+
EnableSyncThreadBusyWait	布尔型	集群异步发送线程忙等	false ：集群内异步发送线程不忙等，使用 wait 事件通知模式，减少CPU占用。 true ：集群内异步发送线程轮询忙等，不会等待事件通知。默认值： false ，要求版本 rcm 1.1.15.6+
SyncBindCoreId	整型	集群异步发送线程绑定CPU的CoreId	CPU核心编号，用于集群异步发送线程的绑定。默认值： -1 ，不绑定。要求 rcm 版本 1.1.15.6+
SyncRepairBindCoreId	整型	集群异步补缺线程绑定CPU的CoreId	CPU核心编号，用于集群异步补缺线程的绑定。生效前提： EnableSyncRepairThread 为 true ，即开启集群的异步处理补缺功能。此功能暂时只支持多进程上下文的主备集群的模式。默认值： -1 ，不绑定。要求版本 rcm 1.1.15.7+
EnableSyncRepairThread	布尔型	是否开启集群异步补缺的功能	true ：集群同步的时候单独开启一个线程，用于处理集群内的Nak补缺请求。 false ：在接收线程同步处理Nak补缺请求。此功能暂时只支持多进程上下文的主备集群的模式。默认值： false 。要求版本 rcm 1.1.15.7+
DefaultMainZone	字符串	默认主中心名	默认值：无，不指定默认主中心。指定的主中心名必须在中心配置 "Zones" 中有定义。当ZK上无历史仲裁信息时启动集群，如果不指定默认主中心，任意节点都允许选主，如果指定了主中心，只有所属中心（ "Contexts" 的配置项 "Zone" ）与默认主中心名一致的节点允许选主。当ZK上有历史仲裁信息时启动集群，行为不受本配置影响。要求版本 rcm 1.1.15.10+
OnlyMainZoneActive	布尔值	是否仅主中心活跃模式	默认值： false 。集群的主节点所属中心为主中心。 true ：只有主中心内的其他备机会被加入集群，不属于主中心的备机不会加入集群。 false ：所有备机都可被加入集群。未配置中心名也意味着不属于主中心。未加入集群的节点身份会保持SYNC，不会影响集群内的其他节点的流控、数据提交、RPO控制。要求版本 rcm 1.1.15.10+

MaterialRange: 物料生成范围

配置项	类型	含义	备注
TransportId	整型	主题Id	MinValue: 最小Id值, MaxValue: 最大Id值, MaxValue要大于等于MinValue, 最大合法范围为 [1, 65535]
ContextId	整型	上下文Id	MinValue: 最小Id值, MaxValue: 最大Id值, MaxValue要大于等于MinValue, 最大合法范围为 [1, 65535]
PartitionNo	整型	主题分区号	MinValue: 最小分区号, MaxValue: 最大分区号, MaxValue要大于等于MinValue, 最大合法范围为 [1, 65535]
MulticastAddr	字符串	主题组播地址	MinValue: 最小地址值, MaxValue: 最大地址值, MaxValue要大于等于MinValue, 最大合法范围为 ["224.0.1.0", "238.255.255.255"], 不同地址范围之间不能相互包含
MulticastPort	整型	主题组播端口	MinValue: 最小端口号, MaxValue: 最大端口号, MaxValue要大于等于MinValue, 最大合法范围为 [1024, 65535], 主题组播端口范围可与集群同步组播端口范围存在重叠, 但不能与单播端口范围有相互包含
SyncAddr	字符串	集群同步组播地址	MinValue: 最小地址值, MaxValue: 最大地址值, MaxValue要大于等于MinValue, 最大合法范围为 ["224.0.1.0", "238.255.255.255"], 不同地址范围之间不能相互包含

SyncPort	整型	集群同步组播端口	MinValue: 最小端口号, MaxValue: 最大端口号, MaxValue要大于等于MinValue, 最大合法范围为[1024, 65535], 集群同步组播端口范围可与主题组播端口范围存在重叠, 但不能与单播端口范围有相互包含
LocalPort	整型	单播端口	MinValue: 最小端口号, MaxValue: 最大端口号, MaxValue要大于等于MinValue, 最大合法范围为[1024, 65535], 单播端口范围与其他端口范围之间不能相互包含

LdpAdmin: 对接轻量管控平台配置

配置项	类型	含义	备注
Addr	字符串	轻量管控平台地址	以tcp连接, 轻量管控平台ip
Port	整型	轻量管控平台端口	
Servers	数组	轻量管控平台地址数组	每个成员表示一个轻量管控平台地址, 当配了此配置Servers, 则原来的Addr和Port不生效
Addr	字符串	轻量管控平台地址	以tcp连接, 轻量管控平台ip
Port	整型	轻量管控平台端口	

Zones: 中心内配置

配置项	类型	含义	备注
Name	字符串	中心名	最大允许长度为127字节, 不应包含操作系统规定的文件名非法字符。 需保证中心名唯一性, 必需字段。
RPO	整型	同中心节点间RPO值	不配置默认RPO无限
SyncAckPoint	字符串	消息同步的确认时刻	"after_recv": 半同步（通信确认） "after_consume": 全同步（业务确认） 默认值: "after_recv"

InterZones: 中心间配置

配置项	类型	含义	备注
Zones	数组	中心间描述	数组, 逗号分隔, 数组元素为中心名, 必须是存在于中心内配置的合法中心名, 必需字段。 数组元素固定为2个, 数组元素可以是字符串(单个中心名), 也可以为集合(子数组, 子数组内不能再包含数组), 但数组元素类型必须一致。 当元素为字符串时, 表示定义"Zones"内左中心与右中心的中心间配置。 当元素为集合时, 表示定义"Zones"内左集合任意一个子元素与右集合任意一个子元素的中心间配置。
RPO	整型	跨中心节点间RPO值	不配置默认RPO无限
SyncAckPoint	字符串	消息同步的确认时刻	"after_recv": 半同步（通信确认） "after_consume": 全同步（业务确认） 默认值: "after_recv"

4.4.5. RCM配置拆分

当配置模式为服务端配置模式时, 配置特性可设置为配置拆分"split"或合并"integrate", 不设置时默认为合并, 也就是老版本的配置方式。

配置特性设置方式参阅命令行工具ldpconfig的使用说明。

配置拆分时的路径节点说明:

```
|-- product
|   |-- config           // Ldp配置目录节点, 节点内容包含: { "ConfigFlags": "split" }
|   |-- Rcm              // Rcm静态配置节点, 存放不会在运行时变更的配置, 如果不开启动态配置, 使用该节点配置内容。
|   |-- rcm_dynamic      // Rcm动态配置目录节点, 开启动态配置时使用
|   |-- config           // Rcm动态配置节点, 开启动态配置时实际读取配置的节点, 存放包括Rcm静态配置节点内容在内的全量Rcm配置内容
|   |-- rcm_material     // Rcm物料目录节点, 开启动态配置时使用
```

需注意, 开启动态配置时, 如果rcm_dynamic/config动态配置节点存在且内容不为空, Rcm静态配置节点内容的修改不会被同步到该节点。

4.5. 管理代理插件 ldp_mproxy

管理代理插件主要用于对外提供WEB服务, 使得外部可以通过HTTP协议来访问LDP平台相关插件管理功能, 可用于节点运行状态监控以及进行一些基本配置。访问管理代理时POST和GET方法都支持, 使用GET方式不支持请求参数嵌套, 推荐使用POST访问。

插件名	支持平台	插件标识	说明
libldp_mproxy.so	linux.x64	ldp_mproxy	管理代理插件

管理代理插件既支持明文通信, 也支持通用的openssl加密通信, 可以使用https方式访问。这里制作的证书需要自签名证书, linux下

简单的示例步骤如下：

```
openssl genrsa -des3 -out server.key 1024
openssl req -new -key server.key -out server.csr
cp server.key server.key.orig
openssl rsa -in server.key.orig -out server.key
openssl x509 -req -days 3650 -in server.csr -signkey server.key -out server.crt
cp server.crt server.pem
cat server.key >> server.pem
```

4.5.1. 接口基本格式

URL设计基本格式如下：

```
/ldp?PluginName=aaa&FuncName=bbb&InstanceNo=ccc&field1=a&field2=b
```

- PluginName 表示插件名
- FuncName表示管理功能名
- InstanceNo表示被访问的插件的实例编号
- field1和field2表示管理功能入参
- URL没有指定PluginName时（/ldp?）返回所有插件的管理功能的超链接列表，可以直接点击访问某个管理功能

管理功能入参还可以在HTTP的POST请求包体中，监控模块会自动合并URL和包体中的参数（如果入参名重复，则以包体中的为准），统一作为函数入参传给被访问的管理功能。

请求包体统一以 json 格式封装，格式如下：

```
{
  "PluginName": "aaa",
  "FuncName": "bbb",
  "InstanceNo": 1,
  "StructFilePath": "struct",
  "Request": {
    "field1": 123,
    "field2": {
      "field3": "123"
    }
  }
}
```

Request是请求的参数列表，其中的第一层会覆盖URL中的同名参数。这意味着本例中，如果URL参数中有field1和field2参数，则会被覆盖。Instance实例号如果不传，则取0值。

管理功能的出参会放到HTTP应答包体中，返回给调用方。具体的应答报文格式由各个插件自行定义，管理代理插件只是负责转发。

各个插件应答中有两个字段是统一的，分别是ErrorNo和ErrorMsg。返回成功时，ErrorMsg字段和ErrorNo可不传。

4.5.2. 示例配置

```
{
  "Lib": "ldp_mproxy",
  "Instance": 0,
  "Timer": 0,
  "StructFilePath": "struct",
  "PersistLevel": 1,
  "PersistDir": "log/mdata",
  "Network": {
    "ListenPorts": "8080",
    "TCPNodelay": "1"
  },
  "SSL": {
    "Version": "0",
    "CertFile": "server.pem",
    "CipherList": "ALL",
    "VerifyDepth": "9",
    "VerifyPeer": "no",
    "CaFile": "ca.pem"
  },
  "Performance": {
    "NumThreads": 50,
    "ListenBackLog": "200",
    "ConnQueue": "20",
    "EnableKeepAlive": false,
    "KeepAliveTimeoutMilli": 5000,
    "RequestTimeoutMilli": 1000
  },
}
```

4.5.3. 配置项说明

名称	类型	说明	备注
Network			
ListenPorts	字符串	侦听端口，支持多个，多个端口之间使用英文逗号连接。如果是SSL连接，端口后面加上字符s	默认值：8080
EnableNagle	布尔型	是否启用Nagle算法	true：启用 false：禁用 默认值：false
SSL			默认不启用
Version	字符串	支持的SSL协议版本，取值范围如下： SSL2 SSL3 TLS1.0	默认值：TLS1.2

		TLS1.1 TLS1.2	
CertFile	字符串	SSL证书	默认无证书
CipherList	字符串	SSL使用的加密套件	默认值: ALL
VerifyDepth	整型	如果启用客户端证书校验, 支持校验的证书链长度。只有在配置了需要校验客户端时才生效。最大值: 9	默认值: 9
VerifyPeer	布尔型	是否校验客户端	默认值: false true: 检验 false: 不校验
CaFile	字符串	根证书	默认无证书
Performance			
NumThreads	整型	处理线程数	默认值: 1
ListenBackLog	整型	Listen队列大小	默认值: 50
ConnQueue	整型	接收到数据等待处理线程处理的连接队列	默认值: 50
EnableKeepAlive	布尔型	是否启用长连接	默认值: false, 表示不启用。已废弃, 固定为false
KeepAliveTimeoutMilli	整型	连接上无数据收发时, 保持连接时间, 在配置EnableKeepAlive为true时才生效。单位毫秒	默认值: 5000
RequestTimeoutMilli	整型	读取一个http请求的超时时间, 超过后连接断开。单位毫秒	默认值: 1000
StructFilePath	字符串	postman、pkcap加载配置文件目录, 如果是相对路径则以workspace为基准的相对路径, 支持绝对路径	默认值: 当前workspace工作目录
PersistLevel	整型	管理功能持久化级别, 0-DEBUG 1-INFO, 负数表示不持久化	默认值: 1
PersistDir	字符串	管理功能持久化目录, mproxy退出时会在此目录下生成一个persistlog结尾的文件, 落地管理功能的当前值	默认值: log/mdata

4.5.4. 监控管理

为便于外部系统直接对LDP应用节点进行监控运维, 管理插件对外提供了监控管理功能, 外部系统可直接从ZK服务器上获取应用节点的管理代理插件的接入信息。
管理代理插件会在ZK服务器相应产品路径下, 创建 register 路径, 然后将当前节点的相应插件身份和节点管理代理接入信息, 一并注册到该路径下, 如下图所示:

The screenshot displays a ZK client interface. On the left, a tree view shows the hierarchy: / > ldp > register > bizproc. The 'register' and 'bizproc' nodes are highlighted with a red box. Below 'bizproc', two nodes are visible: 'xst#0' and 'xst#1'. On the right, the JSON response is shown in the 'RAW' tab. The JSON contains 'ManageIp' as an array of IP addresses and 'ManagePort' as 8090.

```
{  "ManageIp": [    "192.168.86.102",    "10.99.0.11",    "3.3.3.102",    "2.2.2.189",    "172.17.1.1",    "10.244.0.0",    "10.244.0.1",    "1.1.1.102"  ],  "ManagePort": 8090}
```

1. 身份类型目前包含以下:

- fens
- front
- bizproc
- offerproc
- todb

2. 身份节点下挂应用节点的管理代理接入信息，如图中，核心身份下，拥有两个在线的主备核心应用节点，节点内容为应用节点的管理代理插件的接入ManageIp和ManagePort。

3. ManageIp返回的是该应用节点所在设备上的所有网卡的IP地址列表，外部系统需自行判断有效的管理代理接入IP。

4. 应用节点是临时节点，当相应的应用节点在线时存在，下线时临时节点也将删除。

4.6. 日志代理插件 ldp_logproxy

日志代理插件主要用于记录业务日志、时间戳日志、平台日志，日志代码插件对外提供两个对象记录日志，两个对象的SID 为 ILogProxyEx和ILogProxy， ILogProxyEx对应ILogProxyEx对象， ILogProxy对应ILogProxy对象, ILogProxy对象记录日志接口不需要传ldp消息， ILogProxyEx对象记录日志的时候需要传ldp消息。
所记录的日志存放在工作目录的log目录下，并按照日期和大小分割文件，每个日志文件最大保留天数、保留文件总大小和每个分片文件的大小可配置。

插件名	支持平台	插件标识	说明
libldp_logproxy.so	linux.x64	ldp_logproxy	日志代理插件

4.6.1. 示例配置

```
{
  "Lib": "ldp_logproxy",
  "Instance": 0,
  "Timer": 0,
  "LogLevel": 0,
  "BizLogLevel": 0,
  "SyncMode": false,
  "MaxDays":7,
  "MaxFileSize": 10,
  "MaxTotalSize": 100,
  "MergeListSize": 32,
  "MergeTimeoutSec": 5,
  "MergeThreshold": 16,
  "LogThreadBindCpu": -1,
  "MaxQueueSize": 10240,
  "EnableTimeLogTag": true,
  "BizLogVersion": 2,
  "IsMaskMode":true,
  "MaskProcessorConfigPath":"mask.json",
  "TimeStampRules": [
    {
      "Type": "Req;Rsp;Err",
      "/*/*:"列表关键字以英文(;)进行分割",
      "TName": "bizworker-1;bizproc_work_2;",
      "ServiceName": "ldp_bizproc;ldp_front",
      "Functions": "101;102;301;1??",
      "ETimeThold": 500,
      "QcntThold": 10
    },
    {
      "/*/*:"当使用*时，只能独立使用，不能与其他任何符号组合",
      "TName": "**",
      "ServiceName": "**",
      "Functions": "**",
      "ETimeThold": 500,
      "QcntThold": 10
    }
  ],
  "TimeLog":
  {
    "TSPointLevel": 2
  },
  "LogPath": {
    "BinLog": "/home/lijianan/Sources/bin/log/",
    "BizLog": "/home/lijianan/Sources/bin/log/",
    "Log": "../testlog/",
    "MsgLog": "/home/lijianan/Sources/bin/log/",
    "TimeLog": "/home/lijianan/Sources/bin/log/",
    "TraceIDLog": "/home/lijianan/Sources/bin/log/"
  },
  "TimeStampSize":128,
  "TimeInfoPoolBlockCount":4,
  "MaxQueueSize":10240,
  "CopyThreadWaitMs":1,
  "WriteThreadWaitMs":1,
  "MaxBlockCount":4096,
  "BlockSize":1048576,
  "NotifyThreshold":10,
  "QueueDropThreshold":1,
  "CopyTsThreadBindCpu":11,
  "WriteTsThreadBindCpu":10,
  "EnableCompress":true,
  "LockMode": 1,
  "EnableLockStat": false
}
```

4.6.2. 配置项说明

名称	类型	说明	备注
LogLevel	整型	平台日志级别	LEVEL_DEBUG 为0, //debug日志 LEVEL_INFO 为1 //info级别 LEVEL_WARN 为2 //告警日志 LEVEL_ERROR 为3 //错误日志 LEVEL_IMP 为4 //重要事件 LEVEL_FATAL 为5 //致命错误 默认值: LEVEL_INFO
BizLogLevel	整型	业务日志级别	LEVEL_DEBUG 为0, //debug日志 LEVEL_INFO 为1 //info级别 LEVEL_WARN 为2 //告警日志 LEVEL_ERROR 为3 //错误日志

			LEVEL_IMP 为4 //重要事件 LEVEL_FATAL 为5 //致命错误 默认值: LEVEL_INFO
SyncMode	Bool类型	是否是同步写日志模式	False: 异步记录日志 True: 同步记录日志 默认值: false
MaxDays	整型	当前应用日志的保留最大天数, 超过配置天数自动删除	默认值: 0(无上限), 非0情况下, 会触发按照配置天数删除逻辑
MaxFileSize	整型	当前应用日志的单个最大文件大小(MB)	默认值: 100 ,最大值不得超过1024
MaxTotalSize	整型	当前应用日志的最大文件总量(MB), 超过总数删除最早的文件	默认值: 0(无上限), 非0情况下, 配置项大于500 (MB) 才会触发按照总大小删除逻辑
MergeListSize	整型	log和bizlog的合并队列长度	默认值: 32, 配0则不启用日志合并
MergeTimeoutSec	整型	log和bizlog的合并超时时间, 单位秒	默认值: 5, 配0则不启用日志合并
MergeThreshold	整型	log和bizlog的合并阈值	默认值: 16, 配0则不启用日志合并
LogThreadBindCpu	整型	logproxy_run线程绑定cpu的序号	默认值: -1, 表示不绑定
MaxQueueSize	整型	日志队列的大小	默认值: 10240, 小于10240会被设置为10240
EnableTimeLogTag	Bool类型	是否使能时延日志Tag输出	False: 不使能 True: 使能 默认值: false
BizLogVersion	整型	业务日志版本	默认值: 0
IsMaskMode	Bool类型	Binlog脱敏开关	默认值: false
MaskProcessorConfigPath	字符串	脱敏所需的配置文件或路径	文件后缀要求命名为".json"
TimeStampRules	数组	时延日志规则	规则内的对象, 按数组的顺序从上向下一次匹配, 完全匹配上规则内的所有元素即为命中, 命中的记录到时延日志中, 其他则丢弃。
Type	字符串	消息类型	例如: "Req;Rsp;Err" 使用 英文(;) 进行分割, 当使用任意时, 只能独立使用*号不能与其他组合。
TName	字符串	线程名列表	例如: "bizproc_work_1;bizproc_work_2;" 使用 英文(;) 进行分割, 当使用任意时, 只能独立使用*号不能与其他组合。
ServiceName	字符串	插件名称	例如: "kdp_bizproc;kdp_front" 使用 英文(;) 进行分割, 当使用任意时, 只能独立使用*号不能与其他组合。
Functions	字符串	功能号列表	例如: "101;102;301;1??",只能是0~9, ?, *等, 详细参考功能号规范
ETimeThold	无符号整型	执行耗时	默认值 0 单位 ns, 不配或者配 0, 效果一样。匹配规则大于等于。
QcntThold	无符号整型	当前队列的积压数	默认值 0 不配或者配 0, 效果一样。匹配规则大于等于。
TSPointLevel	无符号整型	点位等级	默认值 2 不配或者配 2, 效果一样。仅支持0 1 2 3 的等级配置。匹配规则大于等于。
LogPath	对象	日志文件路径	不配置则使用默认路径, 如果配置则使用配置路径, 路径支持绝对路径和相对路径(相对于workspace的路径)。
TimeStampSize	整型	时延无锁队列的大小	默认为128, 暂时不建议外部修改
TimeInfoPoolBlockCount	整型	时延点位内存池块, 预留大小	默认4, 暂时不建议外部修改
MaxQueueSize	整型	安全队列的最大值	默认值10240, 暂时不建议外部修改
CopyThreadWaitMs	整型	copy线程忙等时间	默认值1ms, 暂时不建议外部修改
WriteThreadWaitMs	整型	写文件线程忙等时间	默认值1ms, 暂时不建议外部修改
MaxBlockCount	整型	内存池的最大块数内存池每块的大小	默认值4096, 暂时不建议外部修改
BlockSize	整型	内存池每块的大小	默认值1048576字节, 暂时不建议外部修改
NotifyThreshold	整型	事件通知阈值	默认值10, 暂时不建议外部修改
QueueDropThreshold	整型	队列丢弃阈值	默认值为1, 暂时不建议外部修改

CopyIsThreadBindCpu	整型	copy线程绑核	默认不绑核
WriteIsThreadBindCpu	整型	写文件线程绑核	默认不绑核
ArchiveThreadBindCpu	整型	归档线程绑核	默认不绑核
EnableCompress	布尔	压缩开关	默认不压缩，压缩前提MaxDays和MaxTotalSize 同时大于0
LockMode	整型	锁模式与队列模式配置：0=有锁+mutex，1=有锁+spinlock，2=无锁队列	0：日志队列和内部对象使用互斥锁；1：日志队列和内部对象使用自旋锁；2：日志队列启用无锁模式。默认值：1
EnableLockStat	Bool类型	是否启用锁耗时统计	True：开启锁持有时间统计（LockCount/HoldTimeNs/MaxHoldTimeNs），用于排查锁等待问题；False：关闭。默认值：false

脱敏配置文件mask.json说明

```
{
  "version": "1.0",
  "enabled": true,
  "default_mask": "*",
  "rules": [
    {
      "path": ["secret_*"],
      "type": "HIDE"
    }
  ]
}
```

名称	类型	说明	备注
version	字符串	配置文件版本号	默认值:1.0
enabled	Bool类型	是否启用脱敏功能	默认值: false
default_mask	字符串	默认待替换成的值	默认值: "*"，字段脱敏之后的长度与原字段实际长度一致
rules	数组	脱敏规则数组	每个规则包含路径表达式和脱敏类型
path	数组	脱敏匹配字段	支持字符串和通配符模糊匹配(phone、phone_*)
type	字符串	脱敏类型	默认值"Hide",完全替换为*

4.6.3. 其他说明

- 脱敏功能需要配置ldp_pack插件,其中脱敏接口依赖t2sdk独立库(libt2sdk.so)，用户需要在Cres的发布包中下载该so文件 (CRES2.0.V202503.00.007往后版本)，将其放到LDP发布包/bin目录下，并额外设置环境变量LD_LIBRARY_PATH库路径为 LDP发布包/bin目录。

4.6.4. 日志类型说明

文件后缀	说明
.log	平台日志
.bizlog	业务日志 .v1.bizlog属于带版本号的业务日志
.msglog	抓包日志
.timelog	时延分析日志
.tracelog	用于结合timelog，做时延分析使用
.binlog	全链路消息日志

4.6.5. 抓包功能

使用说明

- 通过管理功能EnableMsgLog开启/关闭抓包，参数包括插件标识和插件实例编号。通过管理功能GetMsgLogFlag查看插件是否开启抓包。
- 通过管理功能GetMsgLog查看抓包缓存Base64编码的数据，需给定起始范围或查询数量。通过管理功能ClearMsgLog清空抓包缓存。
- 通过管理功能SetFilter设置过滤条件项，可追加过滤条件项，参数包括功能号列表，支持模糊匹配。例如: "Functions": "S12000;S13?01"或者"Functions": "*"
- 通过管理功能GetFilter查询已经设置的过滤条件项。通过DeleteFilter删除过滤条件项，参数为过滤条件项的索引，索引从1开始，如果索引为0表示全部删除

抓包落地文件说明

文件内容：①时间 ②功能号 ③插件名 ④附加说明信息(标识消息的位置) ⑤二进制包体

```
1 1202 155504.224139 5 ldp_front Frontend_OnMessage H...
2 1202 155504.224173 5 ldp_front ASBackend_Push H...
3 1202 155504.224286 5 ldp_front ASBackend_OnMessage B...
4 1202 155504.224307 5 ldp_front Frontend_Push B...
5 1202 155507.112554 102 ldp_front Frontend_OnMessage H...
6 1202 155507.112971 102 ldp_front ASBackend_Push H...
7 1202 155507.113044 102 ldp_front ASBackend_OnMessage ...
8 1202 155507.113056 102 ldp_front Frontend_Push ...
9 1202 155507.113067 303 ldp_front ASBackend_OnMessage C...
10 1202 155510.653008 103 ldp_front Frontend_OnMessage 8...
11 1202 155510.653031 103 ldp_front ASBackend_Push B...
12 1202 155510.655312 103 ldp_front ASBackend_OnMessage ...
13 1202 155510.655523 103 ldp_front Frontend_Push ...
```

附加说明信息

附加信息	说明
Frontend_OnMessage	前置接收到客户端消息
Frontend_Push	前置发送消息给客户端
Frontend_Push	前置发送Tcp消息
ASBackend_OnMessage	前置收到Tcp消息
RcmBackend_Push	前置发送组播消息
RcmReceiver_OnMessage	前置收到组播消息
RcmRecv	核心收到组播消息
RcmSend	核心发送组播消息
TcpRecv	核心收到TCP消息
TcpSend	核心发送TCP 消息

4.6.6. 记录系统日志功能

通过调用日志代理插件的日志接口（仅限于平台日志），选择参数OUTPUT_OSLOG将日志写入操作系统日志。
写入路径：/var/log/messages。

注：写入内容为英文，防止乱码

日志格式：

```
月 日 时分秒 主机名 hsl dp[pid]: 日志信息。
```

例如：

```
Dec 1 09:00:09 host-86-101 hsl dp[25590]: this is a demo
```

示例代码（错误等级要大于等于日志插件当前输出等级，最后一个参数选择OUTPUT_OSLOG）：

```
//插件内使用
int32_t CldpArithPlugin::Init(ILdpHost * lpILdpHost, uint8_t nInstance)
{
    lpILdpHost->GetLogProxy()->Write(1, LEVEL_ERROR, L"this is a demo", OUTPUT_OSLOG);
    return 0;
}

//前置组件通过Init的ldphost获取日志对象
int32_t IFrontBiz::Init(ILdpHost *ldpHost, IFront *front)
{
    ldpHost->GetLogProxy()->Write(1, LEVEL_ERROR, L"this is a demo", OUTPUT_OSLOG);
}

//核心组件
int32_t Func10001(CXstContext *lpContext)
{
    lpContext->m_lpILdpHost->GetLogProxy()->Write(1, LEVEL_ERROR, L"this is a demo", OUTPUT_OSLOG);
}
```

4.6.7. 时延分析功能

时延分析是ldp平台提供的低开销的性能分析功能，用于分析基于ldp开发的程序的内部处理耗时。时延分析功能通过预先埋点的方式，获取请求在各节点中的时间戳，并记录为时延分析日志。通过配套工具就能分析出各节点的耗时。为了区分不同请求，平台提供的sdk会在请求内生成全链路id（业务框架会自动拷贝到应答中），并记录到时延分析日志中。通常情况下，应用服务将一笔请求/应答处理的整个过程记录为一条时延分析日志，每条日志包括多个时间戳，比如sdk的请求发送、前置上行或下行处理等。

插件记录一条时延分析日志的流程如下：

```
// 申请时延分析对象，开始进行时延分析
ITimestamp* lpTimestamp = ILogProxyEx->BeginTimestamp();
// 打点
lpTimestamp->Add("t1");
// 业务逻辑1
lpTimestamp->Add("t2");
// 业务逻辑2
lpTimestamp->Add("t3");
// 业务逻辑3
lpTimestamp->Add("t4");
// 业务逻辑4
// 完成时延分析，记录时延分析日志
ILogProxyEx->EndTimestamp(lpTimestamp);
```

基于前置和业务框架的组件开发时，应用不需要创建时延分析对象，直接使用框架上下文提供的对象打点即可。前置和业务框架的组件开发模式下，应用新增打点的方式如下：

前置业务组件：

前置调用业务组件的IFrontBiz::ProcessRequest和IFrontBiz::ProcessResponse方法，会传入上下文BizContext对象指针。通过BizContext::m_lpTimeStamp来添加时间戳。示例代码如下：

```
int32_t IFrontBiz::ProcessRequest(const BizContext *lpContext)
{
    // 时间戳1
    lpContext->m_lpTimeStamp->Add("t1");
    // 业务逻辑 .....
    // 时间戳2
    lpContext->m_lpTimeStamp->Add("t2");
    return ret;
}
```

在核心业务组件：

核心在调用数据组件ThreadInit函数时，会传入上下文ILdpContext对象指针。数据组件把这个指针保存在业务上下文中，便能够在功能函数中通过业务上下文访问ILdpContext::lpTimeStamp来添加时间戳。示例代码如下：

```
int32_t Func100001(CXstContext *lpContext)
{
    // 时间戳1
    lpContext->m_lpLdpContext->lpTimeStamp->Add("t1");
    // 业务逻辑 .....
    // 时间戳2
    lpContext->m_lpLdpContext->lpTimeStamp->Add("t2");
    //...
}
```

时间戳日志格式：

```
MMDD HHMMSS.Microsecond TraceId FuncID LdpMsgType PluginName Remark1 TimeSamp1 Remark2 TtimeStamp2 ...
```

一条完整的时间戳日志,例如

```
1130 185001.855308 00000000000000000000000000000000 5 Req ldp_front TcpFrontendRecv 1638269401855190704 FrontRecvReq
14167 BizReqStart 153 hello 10331 ustbiz 33873 BizReqEnd 46 ForwardReq 175 LookUpRouteTable 1332 ASPushReq 372
TcpBackendSend 990
```

时间戳日志说明：

字段	说明
MMDD HHMMSS.Microsecond	时间戳日志写入文件的时间 格式:月日 时分秒.微秒
TraceId	全局唯一的标记
FuncID	消息的功能号
LdpMsgType	消息类型 Req 请求 Rsp 正确应答 ErrorRsp 错误应答
PluginName	插件名
Remark1	第一次打点时传入的附加信息
TimeSamp1	第一次打点的绝对时间 纳秒级时间戳（1970-01-01 08:00:00 到当前时间的纳秒数）
Remark2	打点时传入的附加信息
TimeStamp2	相对第一次打点的纳秒时间间隔
...	...
Remarkn	第n次打点传入的附加信息
TimeStampn	第n次相对于n-1次打点的纳秒时间间隔

4.6.8. 日志合并功能

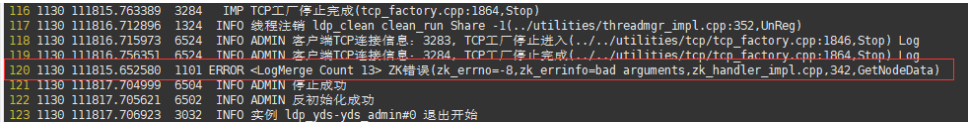
使用说明

通过管理功能SetLogMerge设置是否开启日志合并，默认情况下开启日志合并功能。

日志内容说明

当短时间写入多条相同日志时，会进行日志合并。

对于合并的日志，会增加标记 <LogMerge Count n>，其中n代表合并的日志数量，日志记录时间为最后一条被合并日志的时间



4.6.9. 日志字段解析

平台日志

截图说明：1: 时间 2: 错误号 3:日志级别 4: 进程号 5: 线程号 6: 日志详细信息

列字段	说明
时间	时间戳日志写入文件的时间 格式: 月日 时分秒.微秒
功能号	系统内部函数的编号

插件名	参见《插件说明》章节中各插件的插件标识
附加说明信息	由各自插件自定义点位名称，此处不做具体说明
二进制包体	数据包信息

时延分析日志

规范化：MMDD HHMMSS.MicroSecond traceId FuncID LdpMsgType PluginName [.SpanId Value1 .DepotId Value2 .errno Value3 .CommPattern Value4] Remark1 Timestamp1 ...

```
0704 104504.690105 73447400D080000046701751597344 409 Req ldp_bizproc .SpanId 0.1-108.1-112 .DepotId 1 .QCnt 0 .Tid 106087 .TName bizproc_worker-1 .Type Req
.BTime 62040 .MergeResp 0 .CommPattern 0x22 .errno 0 RecvTime 1751597344690042180 Enqueue 13530 BizStart 35580 BizEnd 12800
```

日志说明：

列字段	说明
MMDD HHMMSS.MicroSecond	时延日志写入文件的时间 格式:月日 时分秒.微秒
TraceId	链路标识
FuncionId	消息的功能号
LdpMsgType	消息类型: Req 请求 Rsp 正确应答 ErrorRsp 错误应答
PluginName	插件名
.FieldName	可选字段, FieldName为字段名称 .SpanId 跨度 .DepotId 跳数 .MergeRsp 是否合并应答日志, 值为1表示合并 .CommPattern 通讯模式: 33表示组播, 32位整数。0-3位为交互方式 1 请求应答 2 消息推送(无需应答) 4~7位为参与者关系1 单播(一对一) 2 组播 4 广播
Field Value	可选字段值
Remark1	打点名(第一个)
TimeSamp1	第一次打点的绝对时间 纳秒级时间戳(1970-01-01 08:00:00 到当前时间的纳秒数)
Remark2	打点名(第二个)
TimeStamp2	相对第一次打点的纳秒时间间隔
...	...
Remarkn	打点名(第n个)
TimeStampn	第n次相对于n-1次打点的纳秒时间间隔

traceid日志

截图说明:

根据timelog日志, 只能看出单个消息的在节点之间、在节点内部流转、耗时情况, 但是一个完整的业务流程, 往往需要多个消息动作来完成, 比如之前提到的一笔委托请求, 需要等待交易所确认、交易所成交两个行为才能认为这笔委托成功了。这三笔消息的TraceID是不一样的, 因此需要有traceidlog文件将这三个链路关联起来。

```
1 AQAAAAAAAAAB3bcuH2hyVAA== 1 3600000213 3600000213 0060000000 2 211101 1 1
9 AQAAAAAAAAAB3bcuH2hyVAA== AQAAAAAAAAAB3bcuH2hyVAA==
0 AgAAAAAAAAA1a1k32xyVAA== 3600000213 2
9 AgAAAAAAAAA1a1k32xyVAA== AQAAAAAAAAAB3bcuH2hyVAA==
0 AwAAAAAAAAADcaZ+y3xyVAA== 3600000213 3
9 AwAAAAAAAAADcaZ+y3xyVAA== AQAAAAAAAAAB3bcuH2hyVAA==
```

日志说明:

委托										
字段名	DataSyncFlag	Request.TraceID	SessionID	OrderRef	OrderID	AccountID	ExchangeID	SeatID	OrderAssStatus	BizSysType
字段含义	MsgType	请求消息中TraceID的Base64编码字符串	会话编号	报单引用	交易所申报编号	资产账户	交易所申报编号	席位编号	报单辅助状态	业务系统类型
字段说明	1: 全量业务字段属性值 0: 缺省业务字段属性值 9: OrderID无法关联消息 时, 启用GTID进行关联		拓展头16bytes Base64编码后的长度计算公式: (Length / 3 + 1) * 4 所以: (16 / 3 + 1) * 4 = 24bytes				"1": SSE: 上海证券交易所 "2": SZSE: 深圳证券交易所 "F1": CZCE: 郑州商品交易所 "F2": DCE: 大连商品交易所 "F3": SHFE: 上海期货交易所 "F4": CFEX: 中国金融期货交易所 "F5": INE: 上海国际能源交易中心股份有限公司		1: 报单已申报: OrderReported 2: 报单已被交易所确认: OrderConfirmed 3: 报单被交易所撮合成交: OrderTraded	1: 证券: LDPSECU 2: 两融: LDPDRCT 3: 期权: LDPDOPT 4: 期货: LDPFUTU
字段示例	1	AQAAAAAAAAAB3bcuH2hyVAA==	1	869987	859988	90010302	2	654321	1	1

2. 确认

字段名	DataSyncFlag	ExchangeConfirmPush.TraceID	OrderID	OrderAssStatus	BizSysType
字段含义	MsgType	请求消息中TraceID的Base64编码字符串	交易所申报编号	报单辅助状态	业务系统类型
字段说明	1: 全量业务字段属性值 0: 缺省业务字段属性值 9: OrderID无法关联消息 时, 启用GTID进行关联		1: 报单已申报: OrderReported 2: 报单已被交易所确认: OrderConfirmed 3: 报单被交易所撮合成交: OrderTraded		1: 证券: LDPSECU 2: 两融: LDPDRCT 3: 期权: LDPDOPT 4: 期货: LDPFUTU
字段示例	0	AgAAAAAAAAAa1k32xyVAA==	859988	2	1

3. 成交

字段名	DataSyncFlag	ExchangeConfirmPush.TraceID	OrderID	OrderAssStatus	BizSysType
字段含义	MsgType	请求消息中TraceID的Base64编码字符串	交易所申报编号	报单辅助状态	业务系统类型
字段说明	1: 全量业务字段属性值 0: 缺省业务字段属性值 9: OrderID无法关联消息 时, 启用GTID进行关联		1: 报单已申报: OrderReported 2: 报单已被交易所确认: OrderConfirmed 3: 报单被交易所撮合成交: OrderTraded		1: 证券: LDPSECU 2: 两融: LDPDRCT 3: 期权: LDPDOPT 4: 期货: LDPFUTU
字段示例	0	AwAAAAAAAAADcaZ+y3xyVAA==	859988	3	1

全链路消息日志

规范化：MMDD HHMMSS.MicroSecond FuncId traceId [.SpanId Value1 .DepotId Value2] MsgLen BinaryData

0704 101920.805000 409 734474000060014758061751595560 .SpanId 0.1-108.1-112 .DepotId 1 .Type Req 5435 ; [REDACTED]

4.7. zk访问插件 ldp_zkhandler

zk访问插件是封装对zookeeper客户端的操作，上层应用不需要关心会话的创建，只需要直接调用操作的接口即可。

插件名	支持平台	插件标识	说明
libldp_zkhandler.so	linux.x64	ldp_zkhandler	zk访问插件

4.7.1. 示例配置

```
{
  "Lib": "ldp_zkhandler",
  "Instance": 0,
  "Timer": 0,
  "UserAuth": "用户名:密码",
  "ZkLockCount": 2
}
```

4.7.2. 配置项说明

名称	类型	说明	备注
UserAuth	字符串	鉴权的用户名和密码以及鉴权方式	字符串允许为空，默认值为空，即表示不需要鉴权。 如果配置，格式必须为用户密码或者 userpasswordscheme，scheme的有效值为auth和digest，前一种格式的scheme为auth。 如果某个集群内，某个应用配置了鉴权，建议该集群内的所有应用的鉴权用户名和密码保持一致。 支持加密，搜索“平台配置项加密”查看 所有ldp节点下面的路径鉴权方式必须一致 鉴权信息发生变化时需要删除已有的节点
ZkLockCount	整型	ZK锁会话池容量	ZK锁句柄是全局资源，上层应用获取ZK锁，使用完后，应归还句柄，默认值：2，最大值：8

zk访问插件支持配置zk的超时时间：在环境变量中配置ZK_TIMEOUTMILLI项。单位：ms。

打开环境变量：vi ~/.bash_profile,添加如下图的环境变量。

```
export ZK_TIMEOUTMITTLI=80
```

默认值 6000ms

4.8. 回库插件 ldp_todb

回库插件用于解析redo文件，实时回库。支持对接新网关和老网关（ust）。

插件名	支持平台	插件标识	说明
libldp_todb.so	linux.x64	ldp_todb	回库插件

4.8.1. 注意事项

必须加载 "ldptcpsdk"：

```
"Components": {
  "Libs": [],
  "GlobalLibs": ["tcp", "ldptcpsdk"]
}
```

4.8.2. 示例配置

```
{
  "Lib": "ldp_todb",
  "Instance": 0,
  "Address": "10.40.3.21:9001,10.40.3.22:9002,www.test.com:9003",
  "IsUstGW": false,
  "RegName": "usttodb",
  "MaxSendKBytesLimit": 10,
  "MaxResendCount": 3,
  "ResendIntervalSec": 8,
  "MaxCacheCount": 10000,
  "ReadRedoInterval": 1000,
  "ProductName": "ldp",
  "AppName": "xst",
  "AppNo": 0,
  "RedoTypes": ["todb", "customized_1"],
  "FuncNoBlackList": "301;10?",
  "GWMasterSlaveOn": true,
  "MultiThreadsMode": true,
  "ThreadsCount": 5,
  "BizErrResendMode": true,
  "MultiThreads": {
    "ErrorNoWhiteList": "3311;3322",
    "ErrRspResendIntervalSec": 8,
    "BizErrMaxResendCount": 2,
    "PlatErrMaxResendCount": 5,
    "BizErrConf": {
      "IsBizErrAutoResend": false,
      "BackupPath": "./todb_bak"
    }
  },
  "CleanInfo": {
    "GroupIndex": 2,
    "MustExist": true
  },
  "StandaloneMode": {
    "Enable": false,
    "RedoPath": "./ldpredo/current/"
  }
}
```

4.8.3. 配置项说明

名称	类型	说明	备注
Address	字符串	网管的ip和端口号，多个地址以逗号","分割，支持域名方式	
IsUstGW	布尔型	true为连接老ust网关，false为连接新网关	默认false
RegName	字符串	连接ust网关时todb的注册名，IsUstGW=true时有效	默认值：“usttodb”
MaxSendKBytesLimit	整型	流控阈值：每秒最大发送字节数，多线程模式下无效	默认不启用
MaxResendCount	整型	最大重传次数	默认值：单线程：3 多线程：一直重传
ResendIntervalSec	整型	超时重传间隔时间（单位：秒）	默认：8
MaxCacheCount	整型	最大缓存记录数，是单线程模式下的缓存上限数，多线程模式下超过这个数之后读取缓存变慢，缓存没有上限	默认值：10000
ReadRedoInterval	整型	轮询redo文件时间间隔	默认值：1000,单位微妙
ProductName	字符串	核心的产品名	默认值：回库节点的产品名；不允许产品不同、应用名相同的多个回库对应同一个核心
AppName	字符串	核心的集群名	默认值：回库节点的集群名
AppNo	整型	核心的应用编号	默认值：回库节点的应用编号
GWMasterSlaveOn	布尔型	网关主备模式开关，true为主备模式，只会给一个网关发数据，false会给全部网关发所有数据	默认值：false
MultiThreadsMode	布尔型	多线程传输模式开关，true为多线程同步发送模式，false为单线程异步发送模式	默认值：false
ThreadsCount	整型	多线程模式下的多线程个数，最大个数为200，当“MultiThreadsMode”=true时	默认值：1

		有效	
BizErrResendMode	布尔型	业务错误应答重传模式开关，当“MultiThreadsMode”=true时有效	默认值：false
CleanInfo	对象	清理相关配置	
GroupIndex	整型	所在的清理组序号，清理按组进行	默认0，最大31
MustExist	布尔	是否必须存在标志	默认true，为true表示该节点在清理过程中必须存在，如果不存在或者掉线，则清理失败；如果为false则表示节点非必须存在，如果不存在或掉线，会忽略他
RedoTypes	数组	回库发送的redo类型	默认"todb"，可配置范围["todb","replay","customized_1","customized_2","customized_3","customized_4"]
FuncNoBlackList	字符串	回库功能号黑名单	英文分号分隔，如"11;30?:319"，?号表示一个数字，只能出现在最后。只要有"*"号，则代表支持所有的功能号。不配置则不使用
ErrorNoWhiteList	数组	错误号白名单	只在“MultiThreadsMode”=true时有效
ErrRspResendIntervalSec	整型	错误应答重传间隔	默认8s
IsBizErrAutoResend	布尔	业务错误是否自动重发，“BizErrResendMode”=true时有效	默认true，true表示业务错误自动重发，false表示业务错误事务号落地，通过调用管理功能ResendBizErr重新回库
BackupPath	字符串	业务错误开启手动重传时，落地文件在清理时的备份目录	默认"/todb_bak"
BizErrMaxResendCount	整型	开启业务错误自动重传后，业务错误重传最大次数	默认一直重传
PlatErrMaxResendCount	整型	开启平台错误自动重传后，平台错误重传最大次数	默认一直重传
StandaloneMode	对象	独立部署模式	
Enable	布尔	是否开启独立部署模式，该模式下可以和核心部署在不同机器上，同时redo文件的跨机器传输需要用户使用rsync等方式自行实现，且必须使用非覆盖且追加写入的方式，不能使用NFS进行redo文件传输	默认false，true表示开启独立部署模式，false表示关闭独立部署模式
RedoPath	字符串	读取redo文件的路径，独立部署模式下todb将改为从此路径下读取redo，开启独立部署模式时必须配置	默认""

4.9. 下场插件 ldp_todb_v2

下场插件用于解析redo文件，实时回库。

插件名	支持平台	插件标识	说明
libldp_todb.so	linux.x64	ldp_todb	下场插件

4.9.1. 注意事项

4.9.2. 示例配置

```
{
  "Lib": "ldp_todb",
  "Instance": 0,
  "TodbOnTimerMs":1000,
  "IsOpenTimeStamp": false,
  "IsOpenTraceByReq": false,
  "ProcessingRedoCount": 10000,
  "MsgPoolInitSize": 256,
  "WaitResendOverTime": 30,
  "MemBlockCount": 4096,
  "PartGroupCount": 4,
  "ClusterName": ["mdb52_1", "cbp63_3"],
  "LinkLatencyMilli": 2000,
  "GlobalMultiThread": [
    {
      "QueueAlias": 1,
      "Count": 4,
      "PollingLoops": 0,
      "SleepTimeUS": 2,
      "InitQueueSize": 2048
    }
  ],
  "..."
}
```

```

"inreads": [
    {
        "Count": 3,
        "QueueAlias": 2,
        "PollingLoops": 0,
        "SleepTimeUS": 5,
        "MaxQueueSize": 10000,
        "TimerUS": 60000000,
        "BigTransMultiThreadQueueAlias": 1,
        "GlobalMultiThreadQueueAlias": 1,
        "MultithreadFrame": {
            "ThreadBindCpu": [
                7,
                8
            ],
            "Count": 2,
            "PollingLoops": 0,
            "SleepTimeUS": 2,
            "InitQueueSize": 2048
        },
        "MemBlockCount": 4096
    }
],
"RedoFrom": {
    "/*": "接收核心的旁路redo，一个核心集群发一个旁路，不能冲突，配置(除了LocalNetworkNumber)保持跟remote一致",
    "Rcm": {
        "LocalNetworkNumber": "192.168.86.0",
        "ProtocolMode": "UDP-MULTICAST",
        "Contexts": [
            {
                "/*": "只提供仲裁功能，跟下面Multi_todb两者留一个，不能两个同时配置",
                "REF": "TODBDEFAULT",
                "TierName": "todb",
                "IsSingleton": false,
                "ArbPriority": 0,
                "RxTopics": [
                    {
                        "REF": "Default",
                        "Name": "todb_donothing",
                        "PartitionNos": [1]
                    }
                ]
            },
            {
                "/*": "多进程上下文，用于todb主备同步，且提供了仲裁，跟上面todb_donothing两者留一个，不能两个同时配置",
                "REF": "TODBDEFAULT",
                "TierName": "todb",
                "IsSingleton": false,
                "ArbPriority": 0,
                "RxTopics": [
                    {
                        "REF": "Multi_todb",
                        "TierName": "todb",
                        "IsMsgSync": true,
                        "IsSingleton": false,
                        "ArbPriority": 0,
                        "RxTopics": [
                            {
                                "Name": "todb_sync",
                                "PartitionNos": [1]
                            }
                        ]
                    }
                ]
            },
            {
                "REF": "Default",
                "RxBypass": [
                    {
                        "Name": "redo",
                        "PartitionNos": [2]
                    }
                ]
            }
        ]
    }
},
"FileQueue": {
    "Enable": true,
    "RecordDir": "./todb_tmp",
    "MaxFileSizeMB": 1024,
    "MaxFileCount": 64
},
"BizErrResend": {
    "Enable": true,
    "MemBlockCount": 4096,
    "QueueAlias": 2,
    "/*": "跟FileQueue区分目录，不要设置相同目录",
    "ResendRecordDir": "./todb_err",
    "MaxFileSizeMB": 999,
    "MaxFileCount": 8
},
"/*": "用于备机切主时的二次确认，配置一个有效端口",
"Frontend": {
    "Tcp": {
        "Acceptors": [
            {
                "Port": 10000
            }
        ]
    }
},
"Routes": [
    {
        "/*": "路由，配置跟remote保持一致",
        {
            "Functions": "*",
            "Backend": 1
        }
    }
],
"Backends": [
    {
        "/*": "Backends配置跟remote保持一致，可配置PLUGIN或者GW两种type",
        {
            "Id": 1,
            "Type": "GW",
            "Name": "ldp_gateway",
            "MsgType": "LDP",
            "Sites": 1
        }
    }
]

```

```
        {
            "Host": "192.168.86.102",
            "Port": 18102
        }
    ]
},
"ResultHandler": {
    "/*": "组件处理失败。比如获取数据连接失败、发送失败等",
    "Failed": {
        "ResendIntervalSec": 3,
        "MaxResendCount": 3
    },
    "Timeout": {
        "ResendIntervalSec": 8,
        "MaxResendCount": 3
    },
    "PlatErrorRsp": {
        "ErrorNo": "-1;1;-61",
        "ResendIntervalSec": 6,
        "MaxResendCount": 0
    },
    "BizErrorRsp": {
        "ResendIntervalSec": 5,
        "MaxResendCount": 0,
        "ErrThreadMaxResendCount": 3,
        "ErrThreadResendIntervalSec": 8
    },
    "MultiAccount": {
        "ResendIntervalSec": 5,
        "MaxResendCount": 10
    }
},
"/*": "回库业务主组件，具体配置见下面小节",
"PrimaryModule": {
},
"/*": "其他过滤组件",
"SecondaryModules": [],
"/*": "事务号定时任务",
"OnTimerTransNoTasks": {
    "/*": "开启事务号定时任务，默认false",
    "Enable": false,
    "/*": "同步最大已回库事务号的请求功能号，仅主机房定时发送",
    "SynFunctionNo": 910010
}
}
```

4.9.3. todb配置项说明

名称	类型	说明	备注
TodbOnTimerMs	整型	定时线程轮询时间	默认1000
IsOpenTimeStamp	布尔	是否开启时延日志	默认值 false
IsOpenTraceByReq	布尔	是否根据请求全链路标志记录时延日志	默认值 false，IsOpenTimeStamp为true时生效
ProcessingRedoCount	整型	在途请求数量，即异步等待应答的请求数量	默认10000
MsgPoolInitSize	整型	时延日志池初始化大小	默认256
WaitResendOverTime	整型	等待重试线程清空的时间（单位：秒）	默认2147483647
GlobalMultiThread	数组	工作线程数组	
Count	整型	线程个数	本组工作线程的个数，默认值：4
ThreadBindCpu	整形数组	本组多线程框架的每个线程绑定cpu的序号	本组多线程框架的每个线程绑定cpu的序号，默认值：-1，表示不绑定 例如：配置[1, -1, 3]，线程个数配置为4，则表示该线程组中线程编号0绑定CPU序号1，线程编号2绑定CPU序号3，线程编号1和3不绑定CPU
PollingLoops	整型	工作线程轮询次数，默认值：100	（1）配置大于0时，工作线程会轮询队列，连续轮询PollingLoops次以后，仍然没有请求，则工作线程睡眠SleepTimeUS微秒后再轮询队列。 （2）配置等于0时，工作线程被事件通知去从队列获取请求处理
SleepTimeUS	整型	睡眠时间	默认值：2，表示不睡眠，即一直轮询，单位微秒。PollingLoops大于0时有效。
InitQueueSize	整型	初始化队列个数	默认值：2048,初始化队列的个数。
Threads	数组	工作线程数组	
Count	整型	线程数量	
QueueAlias	整型	队列别名	

PollingLoops	整型	线程轮询次数	默认值：0
SleepTimeUS	整型	睡眠时间	
MaxQueueSize	整型	最大队列数量	
TimerUS	整型		
BigTransMultiThreadQueueAlias	整型	大事务处理全局因子线程组别名	默认值：1
GlobalMultiThreadQueueAlias	整型	大事务及多账户事务处理全局因子线程组别名	默认值： 1，BigTransMultiThreadQueueAlias 和GlobalMultiThreadQueueAlias同时配置时 GlobalMultiThreadQueueAlias生效
MultithreadFrame	对象	多线程框架相关	
Count	整型	线程个数	本组工作线程的个数，默认值：4
ThreadBindCpu	整形数组	本组多线程框架的每个线程绑定cpu的序号	本组多线程框架的每个线程绑定cpu的序号，默认值：-1，表示不绑定 例如：配置[1, -1, 3]，线程个数配置为4，则表示该线程组中线程编号0绑定CPU序号1，线程编号2绑定CPU序号3，线程编号1和3不绑定CPU
PollingLoops	整型	工作线程轮询次数，默认值：100	（1）配置大于0时，工作线程会轮询队列，连续轮询PollingLoops次以后，仍然没有请求，则工作线程睡眠SleepTimeUS微秒后再轮询队列。 （2）配置等于0时，工作线程被事件通知去从队列获取请求处理
SleepTimeUS	整型	睡眠时间	默认值：2，表示不睡眠，即一直轮询，单位微秒。PollingLoops大于0时有效。
InitQueueSize	整型	初始化队列个数	默认值：2048,初始化队列的个数。
RedoFrom		redo相关配置	
Rcm		从rcm接收redo，配置(除了LocalNetworkNumber)跟remote保持一致，	
Routes		路由，跟remote配置保持一致	
Backends		backends，跟remote配置保持一致	
LocalNetworkNumber	字符串	本机ip或者网段	
FileQueue	对象	文件队列配置对象	
Enable	布尔	是否使用文件队列	默认值：true
RecordDir	字符串	文件队列数据保存路径	默认值：.
MaxFileSizeMB	整型	文件队列单文件大小，单位MB	默认值：1000
MaxFileCount	整型	文件队列文件数量	默认值：16
Frontend	对象	remoter服务端配置对象，目前只配置用来做主备切换的二次确认	
Tcp	对象	remoter的Tcp服务端配置对象，目前只配置用来做主备切换的二次确认	
Acceptors	对象	remoter的Tcp服务端监听配置对象，目前只配置用来做主备切换的二次确认	
Port	整型	监听端口，目前只配置用来做主备切换的二次确认	默认值：参考remoter的默认配置
ResultHandler	对象	错误处理策略配置	
Failed	对象	组件Process错误处理	
ResendIntervalSec	整数	错误重传间隔，单位秒	默认8s
MaxResendCount	整数	重发次数	默认不重发
Timeout	对象	应答超时错误处理	
ResendIntervalSec	整数	错误重传间隔，单位秒	默认8s
MaxResendCount	整数	重发次数	默认不重发

PlatErrorRsp	对象	应答返回平台错误处理	
ResendIntervalSec	整数	错误重传间隔，单位秒	默认8s
MaxResendCount	整数	重发次数	默认不重发
ErrorNo	字符串	应答返回错误时，平台错误号的范围。支持枚举（单个数组）和范围（用区间表示，如 [100,200]，表示100到200这个区间属于平台错误，包含100和200）	配置项举例："1;1;-61",配置中不要出现空格，否则会解析失败，范围区间不做重复校验。不允许同时存在平台错误/业务错误两个规则，只能二选一，未配置的取反向范围，同时存在会报错。都不配置时默认设置平台错误为 "1;1"
BizErrorRsp	对象	应答返回业务错误处理	
ResendIntervalSec	整数	错误重传间隔，单位秒	默认8s
MaxResendCount	整数	重发次数	默认不重发
ErrThreadResendIntervalSec	整数	错误重试线程重传间隔，单位秒	ErrThreadMaxResendCount为-1时，为 BizErrorRsp.ResendIntervalSec，否则为配置值
ErrThreadMaxResendCount	整数	错误重试线程重发次数	默认-1
ErrorNo	字符串	应答返回错误时，业务错误号的范围。支持枚举（单个数组）和范围（用区间表示，如 [100,200]，表示100到200这个区间属于平台错误，包含100和200）	配置项举例："[-1000,-100];666;[1000,20000]",配置中不要出现空格，否则会解析失败，范围区间不做重复校验。不允许同时存在平台错误/业务错误两个规则，只能二选一，未配置的取反向范围，同时存在会报错。都不配置时默认设置平台错误为 "1;1"。
MultiAccount	对象	多账户事务错误处理	
ResendIntervalSec	整数	错误重传间隔，单位秒	默认8s
MaxResendCount	整数	重发次数	默认5，请业务根据具体情况配置
PrimaryModule		主业务组件	只会有一个主业务组件
SecondaryModules	数组	副业务组件	可以由多个副业务组件
BizErrResend	数组		业务错误重试JsonObject
Enable	布尔		是否开启业务错误重试
QueueAlias	整型		重试线程使用哪个线程组
IsMsgSync	布尔		是否启用多进程Rcm上下文，其他配置同remote插件（详细解释请见remote插件）
OnTimerTransNoTasks	对象	事务号定时任务	
Enable	布尔	开启事务号定时任务	默认false，开启后主机房会定时发送同步事务号请求，定时间隔见ToddbOnTimerMs
SynFunctionNo	整型	同步最大已回库事务号的请求功能号	仅主机房定时发送，开启事务号定时任务后必须配置
MemBlockCount	整形	内存池中内存块数量	默认4096, 每块内存块2M，不能小于128块
PartGroupCount	整形	不同类型集群的核心数之和	默认0, 不开网格模式，大于0表示开启网格模式，PartGroupCount的值就是网格模式下的核心的数量，有几个核心就配置成多少,比如有2个分片，一个分片有3个核心节点，一个分片有2个核心节点，那么PartGroupCount = 3 + 2 = 5
ClusterName	字符串数组	[集群名_分片号, 集群名_分片号]	开启网格模式下必须配置, 参考keeper的配置, 兼容老的单字符串配置方式，即配置成"mdb52_1"和 ["mdb52_1"] 是等价的，数组中每个元素都是一个分片
LinkLatencyMilli	整形	togw与kdp_agent之间的rpc时长	默认1000, 单位毫秒, Timeout.ResendIntervalSec - LinkLatencyMilli 为LS → AS 的超时时间

4.9.4. todb插件主备仲裁说明

todb有两种主备仲裁方式：

1. 在RedoFrom的Rcm上下文中增加一个集群上下文，即IsSingleton=false的上下文，并且配置一个唯一的接收上下文，但是不使用
2. 在插件中增加一个FrontEnd对象，并参考示例配置设置一个端口

注: 如果需要同城双机房部署支持切换优先级，需要使用Rcm集群上下文作为主备仲裁对象，并配置ArbPriority设置优先级，数值越大优先级越高

4.9.5. gwredo主组件示例配置

```
"PrimaryModule": {
  "/*": "组件名",
  "Name": "ldp_gwredo",
  "/*": "mdb业务库目录",
  "DataMgrDir": [
    "../bin/pub"
  ],
  "/*": "敏感字段列表，多个字段按；隔开",
  "SensitiveField": "password;xxxx",
  "/*": "物理数据库名",
  "LogicDBName": "ljlusr",
  "/*": "清库的redo是否回库",
  "EnableClear": false,
  "/*": "不回库的mdb表名",
  "TablesDontTodb": [],
  "/*": "必须回库的mdb表名",
  "TablesTodb": [],
  "/*": "用于做线程定位的字段名",
  "FieldLocate": "fund_account",
  "/*": "发送的功能号",
  "FunctionNo": 76,
  "/*": "组件发送模式",
  "BizSyncSend": true,
  "/*": "最大packer限制，单位M，默认12M",
  "MaxPackerSize": 12,
  "/*": "内存池中内存块数量，默认4096，不能小于128块 每块2M",
  "MemBlockCount": 4096,
  "Tables": [
    {
      "/*": "内存表的表名",
      "MdbTableName": "user",
      "/*": "填写核心应用名，当前table允许回库的核心集群",
      "ClusterTodb": ["redo_2"]
    }
  ],
  "/*": "支持按表并发的表",
  "TableLocate": [
    {
      "/*": "内存表名",
      "MdbTableName": "money",
      "/*": "降频回库的最大间隔数",
      "IntervalThresholdCount": 1000
    },
    {
      "MdbTableName": "user_table"
    }
  ],
  "/*": "按表并发的线程组编号",
  "TableLocateAlias": 2,
  "/*": "默认为false，不校验",
  "EnableTableConfigCheck": false,
  "/*": "回库属性配置文件，只有TodbConfigFile配置时，EnableInitDataFilter和EnableTradeDataFilter配置才生效",
  "TodbConfigFile": "todbconfig.json;xxx.json",
  "/*": "是否过滤上场数据，默认为true",
  "EnableInitDataFilter": true,
  "/*": "是否过滤日间数据，默认为false",
  "EnableTradeDataFilter": false,
  "Remote": {
    "/*": "是否过滤上场数据，默认为false",
    "EnableInitDataFilter": false,
    "/*": "是否过滤日间数据，默认为false",
    "EnableTradeDataFilter": false,
    "/*": "清库的redo是否回库",
    "EnableClear": false,
    "/*": "不回库的mdb表名",
    "TablesDontTodb": [],
    "/*": "必须回库的mdb表名",
    "TablesTodb": [],
    "/*": "发送的功能号",
    "FunctionNo": 76,
    "Tables": [
      {
        "/*": "内存表的表名",
        "MdbTableName": "user",
        "/*": "填写核心应用名，当前table允许回库的核心集群",
        "ClusterTodb": ["mdb_test"]
      }
    ]
  },
  "/*": "分片号",
  "ShardNo": 1,
  "/*": "redo合并配置",
  "RedoMerge": {
    "/*": "合并后redo日志落地文件开关，默认关，1为开，其他为关",
    "MergeRedoWriteToFile": 0,
    "/*": "redo合并结果集是否写Debug日志，默认关，1为开，其他为关",
    "MergeRedoWriteDebugLog": 0,
    "/*": "积压redo日志数量，超过该数量后不再接收redo日志，默认5000，配置为0则使用默认值",
    "RedoBacklogMaxCount": 5000,
    "/*": "redo合并后日志总大小，超过后直接回调结果集，默认4096M，单位：M，配置为0则使用默认值",
    "TotalMergeRedoMaxSizeMB": 4096,
    "/*": "单个redo合并器理论内存占用上限，可能会超，默认8GB，单位：GB，配置为0则使用默认值",
    "MaxRedoMergeMemoryGB": 8,
    "/*": "单个redo合并器理论缓存内存占用上限，可能会超，默认2GB，单位：GB，配置为0则使用默认值",
    "MaxRedoMergeBufferMemoryGB": 2,
    "/*": "单个redo最大尺寸，默认12000000，单位字节",
    "RedoMaxSize": 12000000,
    "/*": "redo合并后单个redo日志中单表操作记录数上限，至少512及以上，否则可能存在内存问题",
    "RedoMaxRecordCountPerTable": 512,
    "/*": "redo合并后单个redo日志中单表操作记录数上限，至少512及以上，否则可能存在内存问题"
```

```
"/": "redo业务结束等待时长，默认1秒，单位：毫秒，最大50秒",
"ResultReturnWaitTimeMs": 1000,
"/": "高优先级表名集合，高优先级redo日志项放在最前",
"HighPriorityTables": [
  "TableName1",
  "TableName2"
],
"/": "内存池中内存块数量，默认4096，不能小于128块 每块2M",
"MemBlockCount": 4096
},
"/": "定位模式，只配置0或1，默认为0，0使用“FieldLocate”， 1使用“FieldLocateThreadAffinity”，其他不支持",
"LocateMode": 0,
"/": "用于做线程定位的字段名，支持配置多个定位字段",
"FieldLocateThreadAffinity": [
  {
    "/": "定位字段名",
    "FieldName": "fund_account",
    "/": "线程组别名",
    "QueueAlias": [1,2],
    "/": "不配置时为FieldName",
    "FieldDesc": "资金账户"
  },
  {
    "FieldName": "stock_account",
    "QueueAlias": [3],
    "FieldDesc": "股东账户"
  }
],
"/": "是否按表拆分子线程回库，默认为false,不开启",
"EnableTransTableSplit": true,
"/": "表拆分组，未配置时，按单表维度负载，配置了则按组负载，不包含在组里面的表也是按单表负载，所有子线程都是这个规则",
"TransTableSplitGroup": [
  "entrust;account"
]
}
```

4.9.6. gwredo业务组件配置项说明

名称	类型	说明	备注
Name	字符串	业务组件名	
DataMgrDir	数组	mdb业务库目录	会加载数组里所有目录下的库
SensitiveField	字符串	敏感字段，多个按;隔开	
LogicDBName	字符串	数据库逻辑名	
EnableClear	布尔	清库的redo是否回库	
FieldLocate	字符串	用于做线程定位的表字段名，mdb中的字段名	
TablesDontToddb	数组	不回库的mdb表名	1、配置TablesToddb一定回库 2、配置了TablesDontToddb或者datamgr里设置不回库则不回库 3、第一第二条都不满足的回库
TablesToddb	数组	回库的mdb表名	如上
FunctionNo	整数	下场功能号	默认0
BizSyncSend	布尔	组件发送模式	默认false异步发送，异步发送时不支持开启错误重试功能（对应配置BizErrResend的Enable为true）
MaxPackerSize	整数	最大packer限制，单位M	默认12M
TableLocate	对象	按表并发的相关配置	
MdbTableName	字符串	内存表名	必须配置
IntervalThresholdCount	整数	降频回库的最大间隔数。非0表示降频，通过丢弃部分Update类型的记录来实现降频，但保证进程内最后一笔Update能回库。对于类似差量redo这种，对前面所有redo都存在依赖的情况，将无法保证最终一致性	默认为0，则不降频，每一笔都回库，不降频则能保证最终一致性
TableLocateAlias	整数	用于指定按表并发的线程组	若不配置，则TableLocate所有配置将无效；表并发开启时不支持开启错误重试功能（对应配置BizErrResend的Enable为true）
EnableTableConfigCheck	布尔	是否校验回库内存表和配置表的一致性	默认false，不校验
ToddbConfigFile	字符串	回库配置文件，含表属性配置，存在多个时按‘；’分开	不配置时使用老模式（TablesToddb，TablesDontToddb）标识表回不回库，配置时使用新模式，具体格式见ToddbConfigFile配置简述
EnableInitDataFilter	布尔	是否过滤上场数据	默认true
EnableTradeDataFilter	布尔	是否过滤日间数据	默认false
Remote	json数组	切为异地机房后是否回库的相关配置	如果不填则不支持zone切换

EnableInitDataFilter	布尔	是否过滤上场数据	默认false
EnableTradeDataFilter	布尔	是否过滤日间数据	默认false
EnableClear	布尔	清库的redo是否回库	
TablesDontTodb	数组	不回库的mdb表名	1、配置TablesTodb一定回库 2、配置了TablesDontTodb或者datamgr里设置不回库则不回库 3、第一第二条都不满足的回库
TablesTodb	数组	回库的mdb表名	如上
FunctionNo	整数	下场功能号	默认0
Tables	字符串	内存表的表配置	
MdbTableName	字符串	内存表的表名	
ClusterTodb	字符串	填写核心应用名，当前table允许回库的核心集群	如果不填则认为所有集群都可回库
ShardNo	整数	分片号，配置后将会将分片号设置进LDP消息头中的NodeNo字段；取值范围[1,65535]	默认值0，如果不配置或配置为0代表不会将分片号设置进LDP消息头中的NodeNo字段
RedoMerge	对象	Redo事务合并的相关配置	
MergeRedoWriteToFile	整形	合并后redo日志落地文件开关，默认关，1为开，其他为关	
MergeRedoWriteDebugLog	整形	redo合并结果集是否写Debug日志，默认关，1为开，其他为关	
RedoBacklogMaxCount	整形	积压redo日志数量，超过该数量后不再接收redo日志，默认5000	
TotalMergeRedoMaxSizeMB	整形	redo合并后日志总大小，超过后直接回调结果集，单位：M，默认4096	
RedoMaxSize	整形	单个redo最大尺寸，默认12000000，单位字节	
RedoMaxRecordCountPerTable	整形	单个redo单表最大记录数，默认512，至少512及以上，否则可能存在内存问题	
ResultReturnWaitTimeMs	整形	redo拆分结果集返回等待时长，默认1秒，单位：毫秒，最大值60秒	
HighPriorityTables	数组	高优先级表名集合，高优先级表redo日志项放在最前	
LocateMode	整形	定位模式，只配置0或1，默认为0，0使用“FieldLocate”，1使用“FieldLocateThreadAffinity”，其他不支持	
FieldLocateThreadAffinity	数组	定位字段下线程亲和性配置	规则：1、按json配置顺序优先匹配定位字段 2、FieldName支持多字段，多字段时按';'隔开，字段匹配规则为完全匹配，并且是基于单条记录匹配 3、QueueAlias支持配置多个，多个时在多个组内负载
FieldName	字符串	定位字段名，支持配置多个字段，多个字段时按';'隔开	
QueueAlias	数组	线程组别名	
FieldDesc	字符串	定位字段名描述，默认为FieldName	
EnableTransTableSplit	布尔	是否开启事务级表并发，默认为false	开启事务级表并发时，如果子线程出现不同的报错类型，取重试次数最小的，如果重试次数相等，则取第一个优先匹配，重试间隔按匹配的走
TransTableSplitGroup	字符串数组	事务级表并发组, 多个表时，按；隔开	未配置时，按单表维度负载，配置了则按组负载，不包含在组里面的表也是按单表负载，所有子线程都是这个规则
MemBlockCount	整形	内存池中内存块数量	默认4096, 内存池不能小于128块, 每个内存块2M

4.9.7. mdbredo主组件示例配置

```
"PrimaryModule": {
  "///": "组件名",
  "Name": "ldp_mdbredo",
  "///": "开发工具生成的datamgr库",
  "DataMgrDir": [
    "../bin/pub"
  ]
}
```

```

1,
    "/": "hsdb中的数据库逻辑名",
    "LogicDBName": "usersvr_yangds_todb",
    "/": "清库的redo是否回库，没做",
    "EnableClear": false,
    "/": "日初数据是否过滤，默认为true过滤不回库",
    "EnableInitDate": true,
    "/": "空字符或者空字符串是否需要转换成空格，该配置只对 oracle 数据库有效",
    "ToSpace": false,
    "/": "用于做线程定位的表字段名，mdb中的字段名",
    "FieldLocate": "fund_account",
    "/": "不回库的mdb表名",
    "TablesDontTodb": [],
    "/": "必回库的mdb表名",
    "TablesTodb": [],
    "Tables": [
        {
            "/": "内存表的表名",
            "MdbTableName": "account",
            "/": "物理数据库表名",
            "LogicTableName": "account_1",
            "/": "不回库的mdb字段名",
            "FieldsDontTodb": [],
            "/": "物理数据库和mdb里字段名不一致时的数组",
            "FieldsRename": [
                {
                    "/": "在mdb中的字段名",
                    "MdbFieldName": "fund_account",
                    "/": "在物理数据库中的名字，不填或填空，和mdb的名字保持一致",
                    "LogicFieldName": "fund_account1",
                }
            ],
            "/": "填写核心应用名，当前table允许回库的核心集群",
            "ClusterTodb": ["redo_100"],
            "/": "配置的物理库索引名，不配置时由回库查物理库得到索引名",
            "LogicIndexName": "idx_name"
        }
    ],
    "/": "支持按表并发的表",
    "TableLocate": [
        {
            "/": "内存表名",
            "MdbTableName": "money",
            "/": "降频回库的最大间隔数",
            "IntervalThresholdCount": 1000
        },
        {
            "MdbTableName": "user_table"
        }
    ],
    "/": "按表并发的线程组编号",
    "TableLocateAlias": 2,
    "/": "落库前是否检查持久化信息表",
    "EnableTodbCheck": false,
    "/": "默认为false，不校验",
    "EnableTableConfigCheck": false,
    "/": "回库属性配置文件，只有TodbConfigFile配置时，EnableInitDataFilter和EnableTradeDataFilter配置才生效",
    "TodbConfigFile": "todbconfig.json;xxx.json",
    "/": "是否过滤上场数据，默认为true",
    "EnableInitDataFilter": true,
    "/": "是否过滤日间数据，默认为false",
    "EnableTradeDataFilter": false,
    "Remote": {
        "/": "是否过滤上场数据，默认为false",
        "EnableInitDataFilter": false,
        "/": "是否过滤日间数据，默认为false",
        "EnableTradeDataFilter": false,
        "/": "清库的redo是否回库，没做",
        "EnableClear": false,
        "/": "日初数据是否过滤，默认为true过滤不回库",
        "EnableInitDate": true,
        "/": "空字符或者空字符串是否需要转换成空格，该配置只对 oracle 数据库有效",
        "ToSpace": false,
        "/": "必回库的mdb表名",
        "TablesTodb": [],
        "/": "不回库的mdb表名",
        "TablesDontTodb": [],
        "Tables": [
            {
                "/": "内存表的表名",
                "MdbTableName": "account",
                "/": "物理数据库表名",
                "LogicTableName": "account_1",
                "/": "不回库的mdb字段名",
                "FieldsDontTodb": [],
                "/": "物理数据库和mdb里字段名不一致时的数组",
                "FieldsRename": [
                    {
                        "/": "在mdb中的字段名",
                        "MdbFieldName": "fund_account",
                        "/": "在物理数据库中的名字，不填或填空，和mdb的名字保持一致",
                        "LogicFieldName": "fund_account1",
                    }
                ],
                "/": "填写核心应用名，当前table允许回库的核心集群",
                "ClusterTodb": ["redo_100"],
                "/": "配置的物理库索引名，不配置时由回库查物理库得到索引名",
                "LogicIndexName": "idx_name"
            }
        ]
    },
    "/": "redo合并配置",
    "RedoMerge": {
        "/": "合并后redo日志落地文件开关，默认关，1为开，其他为关",
        "MergeRedoWriteToFile": 0,
        "/": "redo合并结果集是否写Debug日志，默认关，1为开，其他为关",
        "MergeRedoWriteDebugLog": 0,
        "/": "积压Redo日志数量，超过该数量后不再接收redo日志，默认5000，配置为0则使用默认值",
        "RedoBacklogMaxCount": 5000,
        "/": "redo合并后日志总大小，超过后直接回调结果集，默认4096M，单位：M，配置为0则使用默认值",
        "TotalMergeRedoMaxSizeMB": 4096,
        "/": "单个redo合并器理论内存占用上限，可能会超，默认8GB，单位：GB，配置为0则使用默认值",
        "MaxRedoMergeMemoryGB": 8,
        "/": "单个redo合并器理论缓存内存占用上限，可能会超，默认2GB，单位：GB，配置为0则使用默认值",
        "MaxRedoMergeBufferMemoryGB": 2,
        "/": "单个redo最大尺寸，默认12000000，单位字节",
        "RedoMaxSize": 12000000,
    }
}

```

```
        "/*": "redo合并后单个redo日志中单表操作记录数上限，至少512及以上，否则可能存在内存问题",
        "RedoMaxRecordCountPerTable": 512,
        "/*": "redo拆分结果集返回等待时长，默认1秒，单位：毫秒，最大60秒",
        "ResultReturnWaitTimeMs": 1000,
        "/*": "高优先级表名集合，高优先级redo日志项放在最前",
        "HighPriorityTables": [
            "TableName1",
            "TableName2"
        ]
    },
    "/*": "索引字段黑名单，包含黑名单索引字段的索引将会被忽略",
    "IndexFieldsBlackList": ["id","name"],
    "/*": "定位模式，只配置0或1，默认为0，0使用“FieldLocate”，1使用“FieldLocateThreadAffinity”，其他不支持",
    "LocateMode": 0,
    "/*": "用于做线程定位的字段名，支持配置多个定位字段",
    "FieldLocateThreadAffinity": [
        {
            "/*": "定位字段名",
            "FieldName": "fund_account",
            "/*": "线程组别名",
            "QueueAlias": [1,2],
            "/*": "不配置时为FieldName",
            "FieldDesc": "资金账户"
        },
        {
            "FieldName": "stock_account",
            "QueueAlias": [3],
            "FieldDesc": "股东账户"
        }
    ],
    "/*": "是否按表拆分子线程回库，默认为false，不开启",
    "EnableTransTableSplit": true,
    "/*": "表拆分组，未配置时，按单表维度负载，配置了则按组负载，不包含在组里面的表也是按单表负载，所有子线程都是这个规则",
    "TransTableSplitGroup": [
        "entrust;account"
    ],
    "/*": "是否开启批量回库，默认false",
    "EnableBatchSql": false,
    "/*": "批量sql大小，默认50",
    "BatchSqlCount": 50,
    "/*": "批量回库表顺序，不配置时默认按表名排序",
    "BatchTableSequence": [
        "entrust",
        "account"
    ]
}
}
```

4.9.8. mdbredo业务组件配置项说明

名称	类型	说明	备注
Name	字符串	业务组件名	
DataMgrDir	数组	mdb业务库目录	会加载数组里所有目录下的库
LogicDBName	字符串	hsdb中的数据库逻辑名	
TablesTodb	数组	回库的mdb表名	1、配置TablesTodb一定回库 2、配置了TablesDontTodb或者datangr里设置不回库则不回库 3、第一第二条都不满足的回库
TablesDontTodb	数组	不回库的mdb表名	如上
EnableClear	布尔	清库的redo是否回库，没做	
EnableInitDate	布尔	日初数据是否过滤	默认为true过滤不回库
ToSpace	布尔	空字符或者空字符串是否需要转换成空格，该配置只对 oracle 数据库有效	
FieldLocate	字符串	用于做线程定位的表字段名，mdb中的字段名	
Tables	字符串	内存表的表配置	
MdbTableName	字符串	内存表的表名	
LogicTableName	字符串	在物理数据库中的名字，不填或填空，和mdb的名字保持一致	
FieldsRename		mdb和物理数据库字段名不同的数组列表	
MdbFieldName	字符串	在mdb中的字段名	
LogicFieldName	字符串	在物理数据库中的名字，不填或填空，和mdb的名字保持一致	
FieldsDontTodb	数组	不回库的字段	
ClusterTodb	字符串	填写核心应用名，当前table允许回库的核心集群	如果不填则认为所有集群都可回库
LogicIndexName	字符串	配置的物理库索引名	不配置时由回库查物理库得到索引名，如果配置的索引名，物理库中不存在程序报错退出
TableLocate	对象	按表并发的相关配置	
MdbTableName	字符串	内存表表名	必须配置
IntervalThresholdCount	整数	降频回库的最大间隔数。非0表示降频，通过丢	默认为0，则不降频，每一笔都

		弃部分Update类型的记录来实现降频，但保证进程内最后一笔Update能回库。对于类似差量redo这种，对前面所有redo都存在依赖的情况，将无法保证最终一致性	回库，不降频则能保证最终一致性
TableLocateAlias	整数	用于指定按表并发的线程组	若不配置，则 TableLocate 所有配置将无效；表并发开启时不支持开启错误重试功能（对应配置 BizErrResend 的 Enable 为 true）
EnableTodbCheck	布尔	落库前是否检查持久化信息表	默认为 false
EnableTableConfigCheck	布尔	是否校验回库内存表和配置表的一致性	默认 false，不校验
TodbConfigFile	字符串	回库配置文件，含表属性配置，存在多个时按‘；’分开	不配置时使用老模式（TablesTodb，TablesDontTodb）标识表回不回库，配置时使用新模式，具体格式见 TodbConfigFile 配置简述
EnableInitDataFilter	布尔	是否过滤上场数据	默认 true
EnableTradeDataFilter	布尔	是否过滤日间数据	默认 false
Remote	json数组	切为异地机房后是否回库的相关配置	如果不填则不支持 zone 切换
EnableInitDataFilter	布尔	是否过滤上场数据	默认 false
EnableTradeDataFilter	布尔	是否过滤日间数据	默认 false
EnableClear	布尔	清库的 redo 是否回库，没做	
EnableInitDate	布尔	日初数据是否过滤	默认为 true 过滤不回库
ToSpace	布尔	空字符或者空字符串是否需要转换成空格，该配置只对 oracle 数据库有效	
TablesTodb	数组	回库的 mdb 表名	1、配置 TablesTodb 一定回库 2、配置了 TablesDontTodb 或者 datanrg 里设置不回库则不回库 3、第一第二条都不满足的回库
TablesDontTodb	数组	不回库的 mdb 表名	如上
Tables	字符串	内存表的表配置	
MdbTableName	字符串	内存表的表名	
LogicTableName	字符串	在物理数据库中的名字，不填或填空，和 mdb 的名字保持一致	
FieldsDontTodb	数组	不回库的字段	
FieldsRename		mdb 和物理数据库字段名不同的数组列表	
MdbFieldName	字符串	在 mdb 中的字段名	
LogicFieldName	字符串	在物理数据库中的名字，不填或填空，和 mdb 的名字保持一致	
ClusterTodb	字符串	填写核心应用名，当前 table 允许回库的核心集群	如果不填则认为所有集群都可回库
LogicIndexName	字符串	配置的物理库索引名	不配置时由回库查物理库得到索引名，如果配置的索引名，物理库中不存在程序报错退出
RedoMerge	对象	Redo 事务合并的相关配置	
MergeRedoWriteToFile	整形	合并后 redo 日志落地文件开关，默认关，1 为开，其他为关	
MergeRedoWriteDebugLog	整形	redo 合并结果集是否写 Debug 日志，默认关，1 为开，其他为关	
RedoBacklogMaxCount	整形	积压 redo 日志数量，超过该数量后不再接收 redo 日志，默认 5000	
TotalMergeRedoMaxSizeMB%	整形	redo 合并后日志总大小，超过后直接回调结果集，单位：M，默认 4096	
RedoMaxSize	整形	单个 redo 最大尺寸，默认 12000000，单位字节	
RedoMaxRecordCountPerTable	整形	单个 redo 单表最大记录数，默认 512，至少 512 及以上，否则可能存在内存问题	
ResultReturnWaitTimeMs	整形	redo 拆分结果集返回等待时长，默认 1 秒，单位：毫秒，最大值 60 秒	
HighPriorityTables	数组	高优先级表名集合，高优先级表 redo 日志项放在	

		最前	
IndexFieldsBlackList	数组	索引字段黑名单	包含黑名单索引字段的索引将会被忽略
LocateMode	整形	定位模式，只配置0或1，默认为0，0使用“FieldLocate”，1使用“FieldLocateThreadAffinity”，其他不支持	
FieldLocateThreadAffinity	数组	定位字段下线程亲和性配置	规则：1、按json配置顺序优先匹配定位字段 2、FieldName支持多字段，多字段时按';'隔开，字段匹配规则为完全匹配，并且是基于单条记录匹配 3、QueueAlias支持配置多个，多个时在多个组内负载
FieldName	字符串	定位字段名，支持配置多个字段，多个字段时按';'隔开	
QueueAlias	数组	线程组别名	
FieldDesc	字符串	定位字段名描述，默认为FieldName	
EnableTransTableSplit	布尔	是否开启事务级表并发，默认为false	开启事务级表并发时，如果子线程出现不同的报错类型，取重试次数最小的，如果重试次数相等，则取第一个优先匹配，重试间隔按匹配的走。该功能开启时仅为全局表并发线程组可不配置 MultiThreadFrame，其他线程组（如工作线程和重试线程）如果未配置，进程启动报错退出，
TransTableSplitGroup	字符串数组	事务级表并发组,多个表时，按；隔开	未配置时，按单表维度负载，配置了则按组负载，不包含在组里面的表也是按单表负载，所有子线程都是这个规则
EnableBatchSql	布尔	是否开启批量sql执行功能，默认为false	
BatchSqlCount	整形	批量sql大小，默认为50	
BatchTableSequence	数组	批量sql表排序功能，不配置时默认按表名排序，配置时优先按配置中的表名排序，不在配置中的还是按表名排序	

4.9.9. ldp_defaultredo主组件(默认组件)示例配置

```
"PrimaryModule": {
  "Name": "ldp_defaultredo",
  "RedoTypes": ["todb", "replay"],
  "BizSyncSend": true
}
```

4.9.10. ldp_defaultredo业务组件配置项说明

名称	类型	说明	备注
Name	字符串	业务组件名	
RedoTypes	数组	需要回库的redo类型	默认"todb", 可配置范围 ["todb","replay","customized_1","customized_2","customized_3","customized_4"]
BizSyncSend	布尔	组件发送模式	默认false异步发送

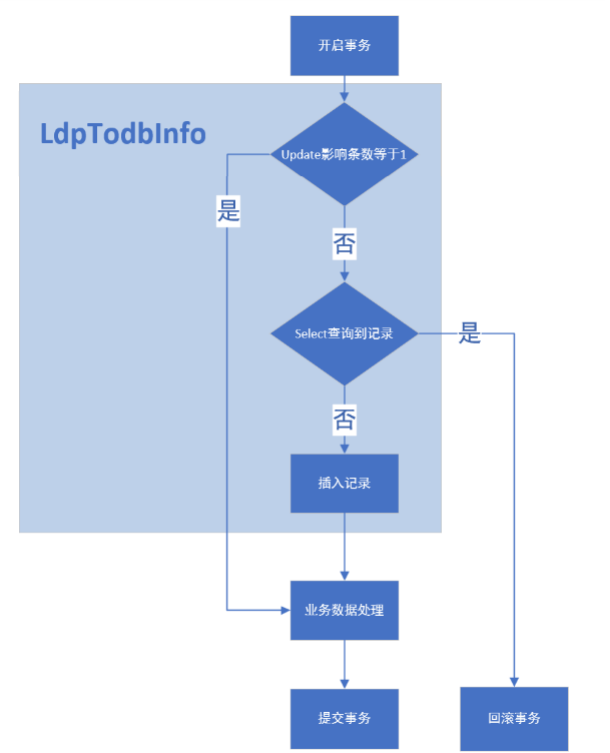
4.9.11. ldp_todb_proc_info表说明

功能背景

回库主备，原主机被孤岛后，备机切主，新主机继续回库；等原主机孤岛恢复后，但还没有退出之前，继续向物理库回了老的数据，导致物理库内数据回退，增加此表解决老数据回库覆盖问题。

表结构

字段名称	字段说明	字段类型	备注
concurrency_info	并发控制标识，表全局唯一索引	字符串，HsChar128	由"ProductName_TodbClusterName_TopicName_PartitionNo_ThreadNo"组成（产品名+回库集群名+主题名+分片号+线程号），如果是错误重试消息还会再拼接定位字段信息，即（产品名+回库集群名+主题名+分片号+线程号+定位字段值），如果是开启了事务级表并发功能，在子线程中的并发标识需要在拼接子线程号
term_str	任期	字符串，HsChar32	由todb仲裁服务提供，用于todb回库时，确认仲裁状态
transaction_str	事务号	字符串，HsChar100，少于20位，左补0	同步模式下，设置为当前线程最后处理的事务号；异步模式下，设置为当前异步在途中最小的事务号
ldp_short_apprname	核心集群名	字符串，HsChar64	同步模式下，设置为当前线程最后处理的事务号；异步模式下，设置为当前异步在途中最小的事务号



使用提示

如果回库对接的核心做了清理，那么ldp_todb_proc_info表也应该清理，否则会出现新事物被过滤导致丢数据的问题

4.9.12. OnTimerTransNoTasks事务号定时任务说明

功能背景

主机房和异地机房的双机房环境，主机房出现故障时，会执行异地机房切换为主机房流程。因为异地机房和主机房是不同的zk集群，所以主机房zk记录的最大已回库事务号没有同步到异地机房，导致异地机房切换为主机房后，最大已回库事务号为0，导致从头开始回库，效率极低。为解决该问题，需要增加事务号定时任务，定时发送各集群的最大已回库事务号到主机房核心，由主机房核心通过机房间同步到异地机房核心。在异地机房执行切主流程前，需要从异地机房核心的内存表中获取最大已回库事务号，并设置到zk。这样就不会从头开始回库，提升回库效率。

同步最大已回库事务号的功能号

用户需要在核心业务代码中实现同步最大已回库事务号的功能号，处理请求，解析出最大已回库事务号并保存到内存表中，并返回应答。请求应答格式均为PackV4。用户需要识别机房前缀，并在保存内存表时去掉该前缀。

输入参数	参数名	参数类型	备注
busi_data	业务数据	HsClob	详细内容格式示例如下

```
{
  "busi_data": [
    {
      "核心集群名",
      "todb_core_cluster_name": "ucrtbid",
      "最大已回库事务号",
      "todb_max_transno": 333,
      "回库节点集群名",
      "todb_cluster_name": "togw"
    },
    {
      "todb_core_cluster_name": "ucrtcbp",
      "todb_max_transno": 666,
      "todb_cluster_name": "todb"
    }
  ]
}
```

输出参数	参数名	参数类型	备注
remark1	备用字段	HsChar16	预留，当前返回空串即可

核心新增ldp_todb_transno_info表

用户需要在接收同步最大已回库事务号的核心上新增ldp_todb_transno_info表，用来保存最大已回库事务号，用于切换异地机房时使用。该表在日初时需要清理、日间需落redo（用于主备同步）、日终不需要下场。

字段名	字段类型	字段含义	备注
todb_core_cluster_name	char[128]	核心集群名	唯一索引
todb_cluster_name	char[128]	回库节点集群名	唯一索引
todb_max_transno	int64	当前最大已回库事务号	

4.9.13. TodbConfigFile配置简述

格式如下：

```
{
  "TableAttributeInfo": [
    {
      "/*": "MDB表名称",
      "MdbTableName": "ucbp_entrust",
      "/*": "默认true， 表示主机房日初数据是否回库",
      "EnableInitDataFilter": true,
      "/*": "默认false， 表示主机房日间数据是否回库",
      "EnableTradeDataFilter": false,
      "/*": "默认false， 表示备机房日初数据是否回库",
      "EnableInitDataFilterOfRemote": false,
      "/*": "默认false， 表示备机房日间数据是否回库",
      "EnableTradeDataFilterOfRemote": false,
      "/*": "回库集群",
      "ClusterTodb": [
        {
          "/*": "回库的核心集群名，支持模糊匹配",
          "ClusterName": "*usesbid*",
        }
      ]
    }
  ]
}
```

4.9.14. togw回库UF2.0超时优化

ldp消息中的55号域需要转为esb消息的15号域,需要配置转换规则@tag(55:15)

4.10. 报盘框架插件 ldp_offerproc

ldp_offerproc是一个报盘框架插件，加载报盘业务程序的so，接收外部请求，根据交易请求的报盘序号定位报盘，调用其处理接口，报盘进行具体的业务逻辑，并保障业务的高可用和高性能。

ldp_offerproc采用主备架构。

插件名	支持平台	插件标识	说明
libldp_offerproc.so	linux.x64	ldp_offerproc	报盘框架插件

4.10.1. 示例配置

```
{
  "/*": "报盘框架库名，这个名字固定，不需要修改",
  "Lib": "ldp_offerproc",
  "/*": "实例编号",
  "Instance": 0,
  "OfferShardNo": 1,
  "ProxyShardNo": 1,
  "ClusterMember": "192.168.86.101,192.168.86.102",
  "TimeStamp": false,
  "ThreadBindType": 0,
  "ArbStatus": true,
  "IsQuery": false,
  "TimeLogAlias": "test",
  "/*": "自动创建RCM相关配置",
  "ProtocolMode": "UDP-MULTICAST",
  "LocalNetworkNumber": "192.168.86.0",
  "NotifyStateIntervalMilli": 1000,
  "OfferGroup": {
    {
    },
    "OfferGroupLibs": [
      "libdemo_offer.so",
    ],
    "PushService": [
      {
        {
          "Name": "PushCore1",
          "Type": "RCM",
          "Topic": "return",
          "PartitionNo": 1
        },
        {
          "Name": "PushOffer",
          "Type": "PLUGIN",
          "PluginName": "ldp_bizproc",
          "Instance": 0
        }
      ]
    },
    "Tier": {
      "REF": "Tier",
      "RxTopics": [
        {
          "REF": "Default",
          "Name": "manager"
        }
      ]
    }
  },
  "Singleton": {
    "REF": "Default",
    "TxTopics": [
      {
        "REF": "Default",
        "Name": "result"
      },
      {
        "REF": "Default",
        "Name": "return"
      }
    ],
    "RxTopics": [
      {
        "REF": "Default",
        "Name": "report"
      }
    ]
  }
}
```

```
    }
  ]
},
"Proxy": {
  "REF": "Default",
  "TxTopics": [
    {
      "REF": "Default",
      "Name": "report"
    }
  ]
},
"TopicMapping": [
  {
    "ReqTopic": "manager",
    "RspTopic": "result"
  },
  {
    "ReqTopic": "trade",
    "RspTopic": "return"
  }
],
"TradeRcm": {
  "Topic": "report",
  "PartitionNo": 1
},
"RspTopic": {
  "Name": "response",
  "REF": "Default"
},
"Rpc": {
  "Routes": [
    {
      "Functions": "454",
      "Systems": "*",
      "Nodes": "*",
      "Backend": 1
    },
    {
      "Functions": "*",
      "Systems": "*",
      "Nodes": "*",
      "Backend": 2
    }
  ],
  "RspTopic": {
    "Name": "rpcresult"
  },
  "Backends": [
    {
      "Id": 1,
      "Type": "AS-RCM",
      "TxTopic": "query",
      "Arb": true,
      "ProductName": "ldp_ws",
      "ClusterName": "bizproc_xst"
    },
    {
      "Id": 2,
      "Type": "AS-RCM",
      "TxTopic": "querytocore",
      "Arb": true,
      "ProductName": "ldp_ws",
      "ClusterName": "bizproc_xst"
    }
  ]
}
}
```

4.10.2. 配置项说明

名称	类型	说明	备注
OfferShardNo	整型	报盘分片序号	报盘插件仅仅充当报盘代理角色时无需配置，否则必须配置，默认值：1
ProxyShardNo	整型	报盘代理分片序号	报盘插件与业务框架部署在一起，则必须配置，否则无需配置，默认值：1
ClusterMember	字符串	Tcp单播集群成员	Rcm采用Tcp模式通讯，且配置中包含Tier、Singleton、Proxy标签时，需配置
TimeStamp	布尔类型	统计时间戳标志	true: 统计时间戳；false: 不统计时间戳，默认值：false
ThreadBindType	整形	工作线程绑核类型	0: 共享核；1: 独占核，默认值：0
ArbStatus	布尔类型	报盘是否有仲裁状态	true: 有仲裁状态，false: 无仲裁状态，默认值:true，例如核心（主备）报盘合并部署时，配置为false，避免核心报盘仲裁状态不一致。
IsQuery	布尔类型	是否查询管理系统	true: 查询，false: 不查询，默认值:false。
TimeLogAlias	字符串	设置时延日志服务别名	默认值：报盘框架插件名
ProtocolMode	字符串	协议类型	支持Udp组播：UDP-MULTICAST;Tcp单播：TCP-UNICAST;Udp单播：UDP-UNICAST;默认Udp组播
LocalNetworkNumber	字符串	本机ip或者网段	
NotifyStateIntervalMilli	整形	通知报盘连接状态的时间间隔	如果未配置或者配置为0，那么不通知

OfferGroup	对象	报盘组配置	报盘业务统一配置
OfferGroupLibs	字符串数组	所有报盘组的库名	
Tier	对象	高可用Rcm信息	
Singleton	对象	非高可用Rcm信息	
Proxy	对象	代理Rcm信息	
REF	字符串	上下文或则主题通道模版	
TxTopics	数组	上下文发送主题信息	每个成员表示发送的主题信息
RxTopics	数组	上下文接收主题信息	每个成员表示接收的主题信息
Name	字符串	主题名	
PushService	数组	主推服务	
Name	字符串	服务名	所有主推服务名必须唯一,应用通过服务名获取对应的服务对象
Type	字符串	服务类型	RCM: 通过RCM主推; PLUGIN: 通过插件主推
Topic	字符串	主题名	Type 为 RCM 时需配置
PartitionNo	整形	分区号	Type 为 RCM 时需配置
PluginName	字符串	插件名	Type 为 PLUGIN 时需配置
Instance	整形	实例编号	Type 为 PLUGIN 时需配置
TopicMapping	数组	发送和接收主题映射	
ReqTopic	字符串	请求主题名	
RspTopic	字符串	应答主题名	
TradeRcm	对象	交易接收组播	若报盘无需接收交易组播,则可省略配置,例如行情报盘
Topic	字符串	交易主题名	
PartitionNo	整形	交易主题分区	此分区为申报主题接收分区,未配置时为 OfferShardNo, 默认值: 1
RspTopic	对象	应答主题	
Name	字符串	应答主题	
REF	字符串	引用模版	可选
Rpc	对象	Rpc相关配置	可选, 详细用法请参考ldprpc组件开发指南

4.10.3. 依赖说明

- 报盘框架通过rcm实现主备架构, 依赖于可靠一致组播 (ldp_rcm) 插件, 即报盘框架需要配置ldp_rcm可靠一致组播 (ldp_rcm) 插件。
- 由于可靠一致组播 (ldp_rcm) 插件依赖于libtcp.so, 所以报盘框架插件需要配置公共库: tcp
- 报盘框架动态生成配置依赖于librem_proxy.so, 所以报盘框架插件需要配置公共库: rcm_proxy
- 若报盘框架需要使用rpc服务, 依赖于libldprpc.so, 所以报盘框架插件需要配置公共库: ldprpc 例如:

```
"Components": {
  "Libs": [],
  "GlobalLibs": [
    "tcp", "rcm_proxy", "ldprpc"
  ]
}
```

4.11. 清理插件 ldp_clean

ldp_clean是一个LDP插件, 它的功能是管理平台清理流程, 方便平台的其他插件执行各自的清理工作, 以满足平台7*24小时运行的要求。
清理流程按组进行, 用户可以通过TCP或者管理功能发起清理请求, 组与组之间的清理过程相互独立。

插件名	支持平台	插件标识	说明
libldp_clean.so	linux.x64	ldp_clean	clean

4.11.1. 示例配置

```
{
  "Lib": "ldp_clean",
  "Instance": 0,
  "Timer": 1,
  "CleanTcpServer": {
    "Addr": "127.0.0.1",
    "Port": 53573
  },
  "TimeoutSec": 90
}
```

4.11.2. 配置项说明

名称	类型	说明	备注
Timer	整型	定时事件发生频率	单位秒
CleanTcpServer	对象	清理模块的tcp服务相关配置	
Addr	字符串	监听服务端Ip地址	必须配置
Port	整型	监听服务端端口	必须配置
TimeoutSec	整型	清理过程各个阶段的超时时间，单位秒	可以不配置，不配置表示无超时时间，永久等待

4.11.3. 依赖说明

- clean插件依赖于libtcp.so，即需要配置公共库：tcp，例如：

```
"Components": {
  "Libs": [],
  "GlobalLibs": [
    "tcp"
  ]
}
```

4.12. 记录池插件 ldp_recordpool

ldp_recordpool是LDP框架下的一个记录内存管理插件，是CUSTable2底层实现，封装了扩容、内存管理等功能。

插件名	支持平台	插件标识	说明
libldp_recordpool.so	linux.x64	ldp_recordpool	记录池

4.12.1. 示例配置

性能配置示例

性能型配置, 内存使用多

```
{
  "Lib": "ldp_recordpool",
  "Instance": 0,
  "Timer": 0,
  "PreAllocNumaNode": [0, 1],
  "/*": "异步扩容线程绑定的cpu号，每个numa节点需配置一个",
  "AsyncRefillThreadBindCpu": [7, 15],
  "/*": "以下配置值不建议修改",
  "AsyncRefillThreadWaitType": 1,
  "EnableHugePages": false,
  "ExpandStep": {
    "TableBlockSizeBits": 12,
    "TableBlockCountBits": 5,
    "TableStep": 64,
    "PoolStep": 2048,
    "PoolMngStep": "2GB",
    "NodePoolStep": "8GB",
    "NodePoolStepFactor": 2
  },
  "RecordTypeThreshold": 32,
  "MngBufSize": "32MB",
  "PoolMinSize": 2048,
  "PoolFactor": 4,
  "EnableTableView": false,
  "DefaultLogLevel": 2,
  "LogCostThresholdNs": 100000,
  "PreFaultOnAsync": false
}
```

节约配置示例

节约型配置, 内存使用少

```
{
  "Lib": "ldp_recordpool",
  "Instance": 0,
  "Timer": 0,
  "PreAllocNumaNode": [],
  "/*": "异步扩容线程绑定的cpu号，每个numa节点需配置一个",
  "AsyncRefillThreadBindCpu": [7, 15],
  "/*": "以下配置值不建议修改",
  "AsyncRefillThreadWaitType": 1,
  "EnableHugePages": false,
  "ExpandStep": {
    "TableBlockSizeBits": 6,
    "TableBlockCountBits": 0,
    "TableStep": 16,
    "PoolStep": 64,
    "PoolMngStep": "4MB",
    "NodePoolStep": "16MB",
    "NodePoolStepFactor": 2
  },
  "RecordTypeThreshold": 32,
  "MngBufSize": "4MB",
  "PoolMinSize": 64,
  "PoolFactor": 2,
  "EnableTableView": true,
  "DefaultLogLevel": 2,
  "LogCostThresholdNs": 100000,
  "PreFaultOnAsync": false
}
```

4.12.2. 常用配置项说明

--	--	--	--

名称	类型	说明	备注
AsyncRefillThreadBindCpu	整数数组	异步扩容线程绑定的cpu	没有默认值，表示不绑定cpu 详细描述请见下文描述
PoolMngStep	字符串	线程级别的内存池扩容步长	默认128MB，建议按实际情况配置 配置形式为 正整数 加上 单位 其中单位可选。 其中数值部分， 必须是2的N次方的值 - 没有单位表示字节数 - 单位配置错误使用默认值 - 配置值要小于NodePoolStep - 不能超过系统可用物理内存大小 支持的单位如下（英文半角大写字母） - PB - TB - GB - MB - KB
NodePoolStep	字符串	NUMA节点级别的内存池扩容步长	默认512MB，建议按实际情况配置。其中数值部分， 必须是2的N次方的值 配置规则参考PoolMngStep，配置值要大于PoolMngStep，但不能超过系统可用物理内存大小 可按照 (工作线程数+1)*PoolMngStep配置。但非强制。
NodePoolStepFactor	整数数字，是2的N次方的值	当配置的NodePoolStep太小，不满足内存分配需求时，会乘以这个数字，使得NodePoolStep变大，以满足分配需要	默认2

- AsyncRefillThreadBindCpu

异步扩容线程绑定的cpu号 实际真正启动的异步扩容线程，会根据工作线程的分布而定，而不是配置了就一定会启动。（比如业务工作线程全部在NUMA节点1，那么只会在NUMA节点1启动一个异步扩容线程，一个NUMA节点最多一个异步扩容线程）

如果不是NUMA架构，此处只需要配置一个cpu的编号即可，比如：希望异步扩容线程绑定到3号cpu上，则配置为：[3]

如果是NUMA架构，此处需要配置的cpu号数目与NUMA节点个数相同 举例：一台有2个NUMA节点的机器，其NUMA节点与cpu号信息如下

NUMA 节点0 CPU	0, 1, 2, 3, 4, 5, 6, 7
NUMA 节点1 CPU	8, 9, 10, 11, 12, 13, 14, 15

则最多启动2个异步扩容线程 当异步线程启动时，如果期望绑定如下 NUMA 节点0 上的异步扩容线程绑定到 3 号cpu NUMA 节点1 上的异步扩容线程绑定到 11 号cpu 则配置为：[3, 11]

4.12.3. 冷门配置项说明

- 可配置，但不太常用的在此列出

在修改此类配置时，请确认已经充分理解配置项的含义

名称	类型	说明	备注
AsyncRefillThreadWaitType	数字	异步扩容线程被唤醒进行工作的通知方式	默认值: 1 1: 使用事件通知方式唤醒异步扩容线程 >= 1000: 异步扩容线程休眠一定时间后轮询，休眠时间单位: 微妙，休眠时长: 配置值/1000后取商的部分，余数丢弃 - 其它值: 异步扩容线程轮询(忙等)
EnableHugePages	布尔型	是否使用大页内存	默认值: false 大页内存请自行搜索资料或参考: https://www.kernel.org/doc/html/latest/admin-guide/mm/hugetlbpage.html
TableStep	数字	UstTable2的扩容步长(记录数)	默认值: 64

			必须是2的N次方的值 当UstTable2中记录用完时，会一次性向“池”要这么多记录
PoolStep	数字	记录池的扩容步长(记录数)	默认值: 2048 必须是2的N次方的值 当“池”中记录数小于该值时，自动再增加这么记录。 比如: “池”中记录数为2047条时(已经小于2048)，自动增加2048条，使“池”内记录数总量到达4095条。
RecordTypeThreshold	数字	切换UstTable2工作模式的结构体(struct)大小阈值	默认值: 32 当UstTable2中存储的struct的 sizeof(struct) < 配置值时，UstTable2按照块大小进行扩容(可以理解为完全等效于CUSTTable)。否则UstTable2按照TableStep扩容。 可以理解为: 当sizeof(struct) < 配置值时, 会切换为更高性能(更费内存)的工作模式
MngBufSize	字符串	管理功能使用的缓冲区大小	默认值: 32MB 该配置值仅影响管理功能，不影响其它部分 配置方式请参看PoolMngStep，该配置值还可以通过管理功能修改，请参考记录池插件的管理功能介绍部分
PoolMinSize	数字	记录池内最小记录个数	默认值: 2048 必须是2的N次方的值 此处“池”的含义为: 以线程为单位，该线程内某个数据结构(struct)
PoolFactor	数字	池扩容步长放大倍数	默认值: 4 当记录池的扩容步长比UstTable2的扩容步长小时，会以该配置值放大记录池的扩容步长。使得满足如下条件: 记录池的扩容步长 >= UstTable2的扩容步长 必须是2的N次方的值
TableBlockSizeBits	数字	表的内部实现中，单块记录数	默认值: 12 单块记录数=2^TableBlockSizeBits
TableBlockCountBits	数字	表的内部实现中，存放块指针数的初始值	默认值: 0 块指针数=2^TableBlockCountBits 默认值0表示在创建表对象时(但未创建记录时)，不预先分配块指针。而在真正创建记录时，才分配块指针。 非0值表示在创建表对象时(但未创建记录时)，立即分配块指针。
EnableTableView	布尔值	是否支持表记录查看功能	默认值: true 该功能的开启会损失一定的性能，如果追求极致性能且不需要此功能的，可以配置为false
PreAllocNumaNode	整数数组	在指定的NUMA节点上进行内存预分配	默认值: [] 该配置项的目的是进一步提升性能，避免在NUMA上首次创建记录时，需要分配内存。但是会额外消耗内存。需要权衡。 无效的配置值会告警（日志关键字discardInvalidNumaNode）并丢弃
DefaultLogLevel	数字	默认的日志级别	默认2 该值为LDP平台日志系统统一的枚举值
LogCostThresholdNs	数字，单位纳秒	记录池内部会有一些性能统计点，当超过这个值时，会打印日志	默认100000 这个值太小的话，会打印太多日志，注意。

PreFaultOnAsync	布尔值	异步扩容线程在申请内存时，要不要做prefault，即对内存写入使得系统分配物理内存	默认false
-----------------	-----	--	---------

4.13. 节点定位插件 ldp_locate

ldp_locate是一个LDP插件，它的功能是根据定位规则为账户查找相应的节点号。其中，规则支持持久化与动态更新，多个定位插件之间的规则也可以相互同步。

插件名	支持平台	插件标识	说明
libldp_locate.so	linux.x64	ldp_locate	节点定位插件

4.13.1. 示例配置

```
{
  "Lib" : "ldp_locate",
  "Instance" : 0,
  "RuleLocal": false,
  "RulesPath" : "/aaaa/bbbbb"
}
```

4.13.2. 配置项说明

名称	类型	说明	备注
RulesPath	字符串	规则路径，单机模式下表示本地配置文件路径，ZK模式下表示ZK节点路径。	可以不配，若不配置，无法持久化规则，也无法同步其他定位插件的规则。若希望多个定位插件之间数据同步，则需要将规则路径配置一致。但是路径不能设置为/ldp/当前产品名。
RuleLocal	布偶型	规则是否在本地。	可以不配置，默认为false。老版本是因为-s带zk地址启动后无法变为本地配置模式，只能是zk模式。增加该配置能-s启动也能是本地配置模式。

4.13.3. 匹配规则

对于单个规则，具体匹配逻辑如下：

```
{
  "Name" : "Default",
  "DefaultNodeNo" : 100,
  "Rules" : [
    {
      "ShardingKeys" : "abcd10000;abcd20000;", "全字段枚举匹配"
      "NodeNo" : 1
    },
    {
      "ShardingKeys" : "200-3000;123-321;", "范围匹配"
      "NodeNo" : 2
    },
    {
      "ShardingKeys" : "1a000;1b001;1c002;1d003;", "全字段枚举匹配"
      "NodeNo" : 3
    },
    {
      "ShardingKeys" : "[2]22;[-3]345;", "截断枚举匹配"
      "NodeNo" : 4
    },
    {
      "ShardingKeys" : "[3]200-3000;[4]123-321;", "截断范围匹配"
      "NodeNo" : 5
    },
    {
      "ShardingKeys" : "[S][3]SAB1ad-Zsdf34;[S][4]ss-zz;", "截断字符串范围匹配"
      "NodeNo" : 6
    },
    {
      "ShardingKeys" : "[S]AB1ad-sdf34;[S]s1s-zz3;", "字符串范围匹配"
      "NodeNo" : 7
    }
  ]
}
```

- 一个规则项内，所有子项规则必须统一，规则匹配优先级：全字段枚举匹配优先级大于所有匹配规则，其他匹配规则优先级相同。全字段枚举匹配 > 截断枚举匹配 = 范围匹配
- "ShardingKeys"支持数字、大写字母、小写字母。
- "NodeNo"取值[1,65534]。
- "ShardingKeys"匹配规则说明：
 - 范围匹配：存在数字范围匹配和字符串范围匹配。差别为，字符串范围匹配多了[S]
 - 整型全匹配。两个数字之间用‘-’连接，如“200-3000”，表示匹配[200,3000]范围内的所有值。
 - 字符串全匹配。每个规则前带有[S]的为字符串匹配规则。1、该规则下，‘-’前后的字符个数应该相同 2、‘-’前的字符串应该比右边的小，大小由strcmp函数比较。3、只有等于或者落在范围中的字符串才算命中。4、截取字符串和之前的规则一致。
 - 范围匹配支持截取匹配：
 - 从左到右截取。在key的开头，配置[n]，控制字符为英文‘[’和英文‘]’，表示从左往右的第n位(从第0位开始数)开始匹配，如[2]34-66，对于8735，就匹配成功，对于8767,就匹配失败，并且[n]应该放在Key的开头。
 - 从右到左截取。在key的开头，配置[-n]，控制字符为英文‘[’和英文‘]’，表示从右往左的第n位开始匹配，如[-3]345-688，对于350897，就匹配成功，对于689976，就匹配失败，并且[-n]应该放在Key的开头。

b. 枚举匹配:

- i. 全匹配。**key**中字符仅包含数字、大写字母、小写字母时，要求入参与**key**完全相同时才算匹配成功。如**key**为123abc，对于123abc则匹配成功，对于123abcd，则匹配失败。
- ii. 全匹配支持截取匹配:
 - A. 从左到右截取。在**key**的开头，配置[n]，控制字符为英文 '[' 和英文 ']'，表示从左往右的第n位(从第0位开始数)开始匹配，如[2]345A，对于98345A，就匹配成功，对于987345A,就匹配失败，并且[n]应该放在Key的开头。
 - B. 从右到左截取。在**key**的开头，配置[-n]，控制字符为英文 '[' 和英文 ']'，表示从右往左的第n位开始匹配，如[-3]34A5，对于34A5897，就匹配成功，对于34A58976，就匹配失败，并且[-n]应该放在Key的开头。

完整的规则如下:

```
{
  "LocateRules": [
    {
      "Name": "Default",
      "DefaultNodeNo": 10,
      "Rules": [
        {
          "ShardingKeys": "abcd10000;abcd20000;",
          "NodeNo": 80
        }
      ]
    },
    {
      "Name": "test1",
      "DefaultNodeNo": 100,
      "Rules": [
        {
          "ShardingKeys": "abcd10000;",
          "NodeNo": 202
        },
        {
          "ShardingKeys": "[S]abcd10000-b13jk0002;[S]AB-a1;[S][2]Ssd9-zsdf",
          "NodeNo": 203
        }
      ]
    },
    {
      "Name": "test3",
      "DefaultNodeNo": 22,
      "ShardingKeys": "account",
      "Value": "passwd",
      "Rules": [
        {
          "/*": "ShardingKeys为账号",
          "account": "abcd10000;",
          "/*": "NodeNo为密码",
          "passwd": 55
        }
      ]
    },
    {
      "Name": "test4",
      "Rules": [
        {
          "/*": "NodeNo为账号",
          "ShardingKeys": "abcd10000;",
          "/*": "第一个扩展值",
          "Key1": "Value1",
          "/*": "第二个扩展值",
          "Key2": "Value2",
          "/*": "第三个扩展值",
          "Key3": "Value3",
          "/*": "NodeNo为23",
          "NodeNo": 23
        }
      ]
    }
  ]
}
```

1. "test3"规则中，用户可以自定义的修改"ShardingKeys"的描述,"NodeNo"的描述用"Value"字段代替，更好的理解规则的内容。如果都不填写，规则还是"ShardingKeys"和"NodeNo"
2. "test4"规则中，在节点定位时，需要匹配到所有Key和Value，才能匹配成功，获取到对应的"NodeNo"。
3. 在"ldp_locate V1.0.14.3"和"gw_locate V1.0.14.3"版本后，Rules下规则支持"/"备注，如"test3"和"test4"
4. 错误示例:

```
{
  "ShardingKeys" : "200-3000;123-321;12345",
  "NodeNo" : 2
}
```

错误，一个规则项中，所有子项类型必须一致，12345属于枚举匹配，而前两个子项属于范围匹配，所以配置错误

```
{
  "ShardingKeys" : "2ab-300;",
  "NodeNo" : 2
}
```

错误，范围匹配时，仅支持正整数，除非增加[S]

```
{
  "ShardingKeys" : "[S]2ab-3000;",
  "NodeNo" : 2
}
```

错误，当字符串范围匹配时，左值和右值的字符串个数应该要一致

使用限制

- 实际的规则加载路径为: **\$RulesPath**/current。其中，子目录current如果不存在，定位插件将自动创建。如果用户希望手动上传

或者修改持久化规则，则应该在 **\$RulesPath/current** 路径中进行操作。

- 关于手动上传规则文件的说明：

1.如果是zk模式，由于zk存在单个节点大小限制，所以节点名必须按照前缀"node_"来命名，比如上传的文件大小超过zk单个文件大小限制，那么文件需要被切分为多个节点，在路径 **\$RulesPath/current** 下，就会有node_0, node_1, node_2.....编号必须从0开始，所有文件的编号连续递增。若规则文件大小未超过单个节点限制，则只需要一个节点node_0即可。
2.如果是本地模式，规则文件就没有分片逻辑，即路径 **\$RulesPath/current** 中只应该存在一个文件，并且文件名没有限制，建议和zk模式一致，也使用名称node_0。

- zk模式时，如果所有节点定位插件都配置了相同的路径，当其中一个定位插件修改规则后，其他所有定位插件也会同步更新规则。
- 本地模式时，节点定位插件无法持久化，无法和其他定位插件同步规则数据。对**ldp_locate**插件来说，框架如果为本地配置模式启动，则**locate**直接切为本地模式，其更新规则接口，无法调用成功。

4.14. cres代理插件 ldp_cres_agent

ldp_cres_agent 是cres代理插件，支持 1. 主动加载hsserver; 2. ldp与cres中插件交互。

插件名	支持平台	插件标识	说明
libldp_cres_agent.so	linux.x64	ldp_cres_agent	cres代理

4.14.1. 示例配置

```
{
  "Lib": "ldp_cres_agent",
  "Instance": 0,
  "Timer": 1,
  "UseCache": true,
  "CresLoad": {
    "StartParam": "-start mainsvr -f gw_agent_ar.xml -p 12345678 -s 0 -t ar"
  },
  "LdpAgent": {
    "CresToLdp": {
      "PluginName": "ldp_front",
      "Instance": 1,
      "TimeoutMill": 3000
    },
    "LdpToCres": {
      "PluginName": "ldp_agent",
      "Instance": 0,
      "TimeoutMill": 3000
    }
  },
  "TraceFlag": true
}
```

4.14.2. 配置项说明

名称	类型	说明	备注
Timer	整型	定时事件发生频率	单位秒，定时器中存在主动清理缓存的操作，建议配置
UseCache	布尔	向上下游发送报文时，是否使用连接缓存	默认true; 当false时，上下游只能各有一个插件，因为无法区分多个插件的连接号
CresLoad	对象	加载相关配置	配置CresLoad，则表示本实例承担了加载hsserver的角色，不配置，则本实例不加载hsserver
StartParam	字符串	Cres启动参数，必填	
LdpAgent	对象	代理相关配置	配置LdpAgent，则表示本实例承担了Ldp在Cres中的代理角色，不配置，则无法通过本实例与Cres交互
CresToLdp	对象	Cres向Ldp发请求的相关配置	不配置，则不支持Cres向Ldp发送请求消息
PhugnName	字符串	Cres的请求转发目的插件	可以不配置，默认是 "ldp_front"
Instance	整型	实例编号	可以不配置，默认是本实例的编号
TimeoutMill	整型	由于本插件内部会缓存请求部分的部分内容，所以引入了缓存超时淘汰时间，单位毫秒	默认永不超时
LdpToCres	对象	Ldp向Cres发请求的相关配置	不配置，则不支持Ldp向Cres发送请求消息
PluginName	字符串	Ldp的请求转发目的插件	可以不配置，默认是 "ldp_agent"
Instance	整型	实例编号	可以不配置，默认是本实例的编号
TimeoutMill	整型	由于本插件内部会缓存请求部分的部分内容，所以引入了缓存超时淘汰时间，单位毫秒	默认永不超时
TraceFlag	布尔	是否打开链路追踪	默认为false

4.14.3. 说明

- 本插件对外暴露hsserver的容器接口实例IF2Core，可以通过插件接口获取（应该向加载了hsserver的实例去获取IF2Core）。

```
// 应该在插件的start阶段获取

// lpHost为hsldp的框架指针，已知加载了hsserver的实例编号为 Instance。
auto lpPlugin = lpHost->GetPlugins()->Find("ldp_cres_agent", Instance);
auto lpIF2Core = (IF2Core *)lpPlugin->GetInterface("hsserver");

// 在插件PreStop阶段以后，不应该再使用
```

4.15. 表分析插件 ldp_table_analyse

ldp_table_analyse是一个LDP插件，它的功能是表分析流程，方便平台的其他插件在使用记录池接口后，可以通过该插件查询记录池中记录的内容。

插件启动时会解析配置文件中指定的so文件的符号表，获取记录池元素的类型信息（目前只支持通过CUstTable2接口创建的记录，且元素必须为类类型，不能为基本类型及其别名）。运行时可以在Ldppadmin管理平台上所有该类型的记录内容，每个数据成员显示为一个列。

目前仅支持如下类型的成员：内置类型，位域，枚举，类型嵌套（类类型成员和非虚继承），数组，模板类；除此之外均不支持（包括但不限于引用、指针、联合等）。插件默认不导出所有模板类的成员，然而so文件中可以提供一个白名单函数指定需要被导出的模板名称，原型为

```
extern "C" const char **GetTemplateWhiteList(void);
```

在此白名单中的模板类型成员会导出到Ldppadmin管理平台。例如以下代码会导出S和S2模板实例化后的类型，如S<int>，S2<float>，但不会导出S3实例化后的类型，如S3<string>。另外需要注意的是，通过SQL语句修改数据时，用户必须确保传入的值在有效范围内，否则行为是未定义的（通常后台会静默截断数据而不给出提示）。

```
template <class T> struct S { T t; };
template <class T> struct S2 { T t; };
template <class T> struct S3 { T t; };
extern "C" const char **GetTemplateWhiteList(void)
{
    // 以数组形式列出所有需要导出的模板名（不带模板参数）
    static const char *templates[] = {
        "S",
        "S2",
        NULL // 以NULL结尾
    };
    return templates;
}
```

插件名	支持平台	插件标识	说明
libldp_table_analyse.so	linux.x64	ldp_table_analyse	table analyse

4.15.1. 示例配置

```
{
  "Lib": "ldp_table_analyse", // 插件动态库名，不含扩展名
  "Instance": 0, // 插件实例编号，不配置时，默认为0
  "Timer": 0, // 定时器间隔，0表示不启用
  // 解析单个目标时可以使用该配置项，解析多目标时需要删除FileName配置项，使用FileNameList配置项
  "FileName" : "../bin/crdt.local/libcrdt_data.so",
  // 列表内容为需要解析的文件名。可以是绝对路径 或 是对workspace的相对路径
  "FileNameList" : [
    "../bin/crdt.local/libcrdt_data.so"
  ],
  // 原始数据默认编码格式为GBK，主要编码格式"GB2312", "UTF-8", 如需其他请自行查阅。
  "InEncodingFormat": "GBK"
}
```

4.15.2. 配置项说明

名称	类型	说明	备注
FileName	字符串	需要解析文件的位置及名称	
FileNameList	字符串数组	元素为需要解析文件的位置及名称	
InEncodingFormat	字符串	字符串字段编码	默认值：GBK
DumpDir	字符串	导出csv目录	默认值：当前workspace

4.16. 前置代理插件 ldp_front_proxy

ldp_front_proxy是一个前置代理插件，提供前置代理功能，支持配置一个或多个前置后端。

插件名	支持平台	插件标识	说明
libldp_front_proxy.so	linux.x64	ldp_front_proxy	前置代理插件

4.16.1. 示例配置

```
{
  "Lib": "ldp_front_proxy",
  "Instance": 0,
  "Timer": 1,
  "Tcp": {
    "SendRecvThreadCount": 2,
    "RecvThreadCount": 1,
    "SendThreadCount": 1,
    "ThreadBindCpu": [
      2,
      3
    ]
  },
  "Mode":1,
  "Routes":[
    {
      "Functions":"10;1000;",
      "Backend":1
    },
    {
      "Functions":"20;2000",
      "Backend":2
    }
  ],
  "Front": {
    "Port": 13142,
    "Address": "192.168.86.102",
    "EnableQuickAck": true,
    "EnableNagle": false,
    "MaxConnectCount": 10000,
    "MaxSendBuffSizeBytes": 0,
    "MinRecvBuffSizeBytes": 256,
    "AddressWhiteList": "97.64.25.*;97.64.??32;97.65.27.12;",
    "AddressBlackList": "98.64.25.*;98.64.??32;98.65.27.12;"
  },
  "Backends": [
    {
      "Tcp": {
        "EnableQuickAck": true,
        "EnableNagle": false,
        "MaxSendBuffSizeBytes": 0,
        "MinRecvBuffSizeBytes": 256,
        "HeartbeatIntervalSec": 30,
        "HeartbeatTimes": 3,
        "ReconnectIntervalSec": 10,
        "ConnectTimeoutSec" : 6
      },
      "Host": "192.168.86.102",
      "Port": 23150,
      "bWan": true
    },
    {
      "Tcp": {
        "EnableQuickAck": true,
        "EnableNagle": false,
        "MaxSendBuffSizeBytes": 0,
        "MinRecvBuffSizeBytes": 256,
        "HeartbeatIntervalSec": 30,
        "HeartbeatTimes": 3,
        "ReconnectIntervalSec": 10,
        "ConnectTimeoutSec" : 6
      },
      "Host": "192.168.86.102",
      "Port": 23151,
      "bWan": false
    }
  ]
},
```

4.16.2. 配置项说明

名称	类型	说明	备注
Timer	整型	定时事件发生频率	单位秒, 填1
Mode	整型	运行模式	默认0。非0时，支持路由转发
Routes	数组	功能号路由表	每个数组表项下参数Functions指定功能号信息，Backend指定后端
Functions	字符串	功能号集合	英文分号分隔，如"11;30?;319"，支持"*"，支持模糊匹配，?号表示一个数字, 只能出现在最后
Backend	整型	后端索引	必须存在相应的后端
Tcp	对象	配置tcp的工厂对象的属性	
SendRecvThreadCount	整型	设置网络库工作线程个数	默认为1，该值大于0时，工厂对象的每个工作线程同时负责套接字上数据发送和接收
RecvThreadCount	整型	设置网络库接收线程个数	默认为1，SendRecvThreadCount设置为0时，该值有效
SendThreadCount	整型	设置网络库发送线程个数	默认为1，SendRecvThreadCount设置为0时，该值有效
ThreadBindCpu	整型数组	线程与cpu绑定	设置线程和cpu的绑定
Backends	对象	后端前置站点	
Tcp	对象	与后端站点相连的客户端的属性	
EnableQuickAck	布尔	是否启用快速ACK	默认为true

EnableNagle	布尔	是否启用Nagle	默认为false
MaxSendBuffSizeBytes	整型	最大发送缓存	默认为0，表示不限制
MinRecvBuffSizeBytes	整型	最小接收缓存	默认为256
HeartbeatIntervalSec	整型	心跳间隔	默认为30
HeartbeatTimes	整型	心跳超时次数	默认为9
ReconnectIntervalSec	整型	重连间隔	默认为10
ConnectTimeoutSec	整型	连接超时时间	默认为6
Host	字符串	站点 <code>ip</code>	站点 <code>ip</code> 地址
Port	整型	站点监听端口	站点监听端口
bWan	布尔	站点内外网属性	前置后端一般含两个监听实体，一个面向内网客户，一个面向外网客户。
Front	对象	对外服务信息	
Address	字符串	对外接口的 <code>ip</code>	
Port	整型	监听端口	对外接口的监听端口
EnableQuickAck	布尔	启用快速ACK	默认为true
EnableNagle	布尔	启用Nagle	默认为false
MaxConnectCount	整型	最大连接数目	默认为10000
MaxSendBuffSizeBytes	整型	最大发送缓存	默认为0，表示不限制
MinRecvBuffSizeBytes	整型	最小接收缓存	默认为256
AddressWhiteList	字符串	IP白名单	支持模糊匹配, *号必须放最开头或最结尾, ?号位置随意, 每个 <code>ip</code> 必须以;号结尾, 包括最后一个 最长支持1024个字符
AddressBlackList	字符串	IP黑名单	支持模糊匹配, *号必须放最开头或最结尾, ?号位置随意, 每个 <code>ip</code> 必须以;号结尾, 包括最后一个 最长支持1024个字符

4.16.3. 注意事项

- 前置代理使用的tcp版本是1.0.6.3或之后，前置后端使用的tcp版本是1.0.6.3之前，客户端使用的sdk版本是1.0.6.3之前时，需要客户端或者前置后端至少一个开启了心跳机制
- Mode非0下使用时，客户端接入成功后，应该延迟一会儿再向前置代理发送请求

4.16.4. 依赖说明

- 前置代理依赖于libtcp.so，即前置代理插件需要配置公共库：tcp，例如：

```
"Components": {
  "Libs": [],
  "GlobalLibs": [
    "tcp"
  ]
}
```

4.17. 守护插件 ldp_daemon

ldp_daemon是一个LDP插件，它的功能是对配置该插件的进程进行监控，监控分为两种形式：守护和观察。守护目的：配置在守护列表中的进程异常时，可以通过异常进程注册的启动命令将异常进程重新启动起来。观察目的：配置在观察列表中的进程异常时，可以对自身进程采取一定的执行策略。

插件名	支持平台	插件标识	说明
libldp_daemon.so	linux.x64	ldp_daemon	daemon

4.17.1. 示例配置

```
{
    "Lib": "ldp_daemon",
    "Timer" : 0,
    "Instance":0,
    //单位毫秒，循环检测，更新间隔
    "TimeIntervalMilli": 100,
    // 执行策略次数
    "ExecutionPolicyCount": 3,
    //配置异常拉起时，需要使用的额外参数。如果onload目录不在环境变量中，需要使用绝对路径
    "StartPrefix":"onload -p latency taskset -c 3",
    //守护列表和监控列表不能配置相同的产品名和应用名
    //守护列表，列表内部不能配置相同的产品名和应用名
    "GuardList":[
    {
        //是否自动运行，配置非0时，本进程启动时会拉起 StartCommand指定的进程。配置 0 时，不启动StartCommand指定的进程，需要手工启动。
        "AutoRun":1,
        //建议使用产品名 应用名配置方式
        //产品名
        "ProductName":"ldpTAS",
        //应用名 当#id未配置时，使用与启动应用相同的id号
        "AppName":"TA#0",
        //不推荐使用，因为配置可执行程序需要指定路径，相对路径或绝对路径
        //当#id未配置时，使用与启动应用相同的id号
        //"StartCommand": "/home/workspace/ldp/bin/hsl dp startdaemon ldpTAS TA#0 -s 192.3.3.3",
    },
    ],
    //观察列表，列表内部不能配置相同的产品名和应用名
    "WatchList":[
    {
        //产品名
        "ProductName":"ldpTAS",
        //应用名 当#id未配置时，使用与启动应用相同的id号
        "AppName":"TA#1",
    },
    ],
}
```

4.17.2. 配置项说明

名称	类型	说明	备注
TimeIntervalMilli	整型	单位毫秒，循环检测，更新间隔。默认值 0，当值小于 100，周期间隔按100ms。	小于0时，更新时间为0
ExecutionPolicyCount	整型	执行策略次数，默认值3	小于0时，按默认执行次数3次
StartPrefix	字符串	配置异常拉起时，需要使用的额外参数，可执行命令需要使用绝对路径	
GuardList	数组	守护列表	
AutoRun	整型	是否自动运行，配置非0时，本进程启动时会拉起 StartCommand指定的进程。配置 0 时，不启动 StartCommand指定的进程，需要手工启动。默认值 0	
ProductName	字符串	产品名	建议使用产品名 应用名配置方式
AppName	字符串	应用名 当#id未配置时，使用与启动应用相同的id号	
StartCommand	字符串	当#id未配置时，使用与启动应用相同的id号	不推荐使用，因为配置可执行程序需要指定路径，相对路径或绝对路径
WatchList	数组	观察列表	
ProductName	字符串	产品名	
AppName	字符串	应用名 当#id未配置时，使用与启动应用相同的id号	

4.17.3. 使用指南与内部原理

- 守护者可以启动时拉起被守护者
- 守护者既可以永久守护，又可以按照次数/时长守护(是指启动时，每隔多久（TimeIntervalMilli）检测一次，当次数（ExecutionPolicyCount）达到后，便不再守护)
- 永久守护时，需要被守护进程也配置守护插件。如果被守护者配置了守护插件，那么就会注册当前进程信息，修改守护者初始化的被守护者状态信息，守护者判断被守护者是否是非正常退出/是否正常启动，来决定是否拉起被守护者。被守护者每次启动又会将共享内存中状态进行初始化，所以总会一直监护并拉起。建议配置：

```
{
    "Lib": "ldp_daemon",
    "Timer": 0,
    "Instance": 0
}
```

- 当被守护者异常退出（崩溃）时，守护者会拉起。如果不是永久守护，没有被拉起，确认是否时间和间隔配置过少
- 正常停止被守护者不会被拉起，因为正常停止会修改共享内存中的标识位，守护者通过标识位去判断，正常停止不应被拉起
- 如果在永久守护的情况下，守护者发生崩溃，那么再次启动守护者依旧会永久守护被守护者。

4.18. 仲裁插件 ldp_arb

ldp_arb是一个LDP插件，在多进程上下文中起着仲裁通知的作用，启动后将仲裁信息同步给核心。当主机的复制插件崩溃，仲裁插件感知后，退出前会充当复制插件角色，把剩余的redo消息同步给备机

插件名	支持平台	插件标识	说明
libldp_arb.so	linux.x64	ldp_arb	arb

4.18.1. 配置说明

arb插件的配置格式

以下是arb插件配置的一个抽象示例，后续对其具体展开解释

```
"Plugins":
[
  {
    // 全局部分
    "Lib": "ldp_arb",
    "Instance": 0,
    "LocalNetworkNumber": "192.168.86.0",
    "REF": "Multi",
    "AppName": "xst",
    "AppNo": 0,
    "MsgSyncTimeOut" : 1000,
    "CopyProcList": [
      {
        "ProductName": "mdb",
        "AppName": "mdb_copy",
        "AppNo": 0
      }
    ],
    "BizProcList": [
      {
        "ProductName": "mdb",
        "AppName": "mdb_arb",
        "AppNo": 0
      }
    ]
  }
]
```

4.18.2. 配置项说明

名称	类型	说明	备注
LocalNetworkNumber	字符串	本地网卡的网络号	主备仲裁模式下使用，如果是本地模式的话，可以不用配置，例如：192.168.86.0
REF	字符串	引用模版	必填，一定要填写对应的多进程上下文的引用模版
AppName	字符串	应用名	如果核心，arb和copy部署统一进程，则不需要填写，默认为应用名。如果不是部署在同一进程，则需要arb，copy和核心的AppName一致
AppNo	整型	应用名实例编号	如果AppName配置了，则需要和copy和核心的AppNo一致。同一集群的，主机和备机，AppNo不同
MsgSyncTimeOut	整型	消息同步超时时间	如果arb正常断开前，需要将剩余redo消息同步给备机的超时时间。超过则直接停止，不在同步。默认60000毫秒。单位毫秒
CopyProcList	对象数组	arb守护的copy列表	守护copy进程时使用，与守护插件配合使用
ProductName	字符串	产品名	守护copy进程的产品名
AppName	字符串	应用名	守护copy进程的应用名
AppNo	整型	实例号	守护copy进程的实例号，和上述 组成 AppName#AppNo
BizProcList	对象数组	arb守护的核心列表	守护biz进程时使用，与守护插件配合使用
ProductName	字符串	产品名	守护biz进程的产品名
AppName	字符串	应用名	守护biz进程的应用名
AppNo	整型	实例号	守护biz进程的实例号，和上述 组成 AppName#AppNo

4.19. 复制插件 ldp_copy

ldp_copy是一个LDP插件，在多进程上下文中起着消息同步的作用，主机通过copy插件将redo消息同步给备机

插件名	支持平台	插件标识	说明
libldp_copy.so	linux.x64	ldp_copy	copy

4.19.1. 配置说明

copy插件的配置格式

以下是copy插件配置的一个抽象示例，后续对其具体展开解释

```
"Plugins":
[
  {
    // 全局部分
    "Lib": "ldp_copy",
    "Instance": 0,
    "LocalNetworkNumber": "192.168.86.0",
    "Tier": {
      "REF": "Multi",
      "AppName": "xst",
      "AppNo": 0
    },
    "MsgSyncTimeOut" : 1000,
    "ArbProcList": [
      {
        "ProductName": "mdb",
        "AppName": "mdb_arb",
        "AppNo": 0
      }
    ],
    "BizProcList": [
      {
        "ProductName": "mdb",
        "AppName": "mdb_copy",
        "AppNo": 0
      }
    ]
  }
]
```

4.19.2. 配置项说明

名称	类型	说明	备注
LocalNetworkNumber	字符串	本地网卡的网络号	主备仲裁模式下使用，如果是本地模式的话，可以不用配置，例如：192.168.86.0
Tier	对象	多进程上下文使用	必须配置
REF	字符串	引用模版	必填，一定要填写对应的多进程上下文的引用模版
AppName	字符串	应用名	如果核心，arb和copy部署统一进程，则不需要填写，默认为应用名。如果不是部署在同一进程，则需要copy，arb和核心的AppName一致
AppNo	整型	应用名实例编号	如果AppName配置了，则需要和arb和核心的AppNo一致。同一集群的，主机和备机，AppNo不同
MsgSyncTimeOut	整型	消息同步超时时间	如果copy正常断开前，需要将剩余redo消息同步给备机的超时时间。超过则直接停止，不在同步。默认60000毫秒。单位毫秒
ArbProcList	对象数组	copy守护的仲裁列表	守护arb进程时使用，与守护插件配合使用
ProductName	字符串	产品名	守护arb进程的产品名
AppName	字符串	应用名	守护arb进程的应用名
AppNo	整型	实例号	守护arb进程的实例号，和上述 组成 AppName#AppNo
BizProcList	对象数组	copy守护的核心列表	守护biz进程时使用，与守护插件配合使用
ProductName	字符串	产品名	守护biz进程的产品名
AppName	字符串	应用名	守护biz进程的应用名
AppNo	整型	实例号	守护biz进程的实例号，和上述 组成 AppName#AppNo

4.20. 通信服务框架插件 ldp_remoter

LDP REMOTER 插件提供平台统一的接入接出通信服务，包含TCP接入接出，RCM接入接出，插件穿透接入接出，RPC服务，主推服务，路由查询服务等，有效解决了上层插件对于通讯实现的不统一，产生大量冗余代码的问题。使通信模块具备更好的功能移植性和可扩展性。

插件名	支持平台	插件标识	说明
libldp_remoter.so	linux.x64	ldp_remoter	通信服务框架插件

4.20.1. REMOTER 插件示例配置

```
{
  "TcpFactory": [
    {
      "Name": "Default",
      "/*": "默认为1, 该值大于0时, 工厂对象的每个工作线程同时负责套接字上数据发送和接收",
      "SendRecvThreadCount": 1,
      "/*": "默认为1, SendRecvThreadCount设置为0时, 该值有效",
      "SendThreadCount": 1,
      "/*": "默认为1, SendRecvThreadCount设置为0时, 该值有效",
      "RecvThreadCount": 1,
      "/*": "是否异步发送消息",
      "EnableAsyncSend": false,
      "/*": "是否采用直接接收模式",
      "EnableDirectRecv": false,
      "/*": "是否启用Nagle",
      "EnableNagle": false,
      "/*": "是否启用快速ack",
      "EnableQuickAck": true,
      "/*": "设置Rdma接收线程个数",
      "RdmaRecvThreadCount": 1,
      "/*": "每个线程绑定cpu的序号",
      "ThreadBindCpu": [
        1
      ],
      "/*": "最大连接数",
      "MaxConnectionCount": 10000
    }
  ],
  "/*": "Remoter插件自有定时线程时间间隔, 单位: 毫秒, 定时最小间隔: 1ms, 默认: 1000ms",
  "TimerIntervalMilli": 1000,
  "/*": "默认关闭, 开启时Remoter插件生成一个MsgReader消息池, RpcClient不再默认生成1024个MsgReader, 按需消息池中获取",
  "UseMsgReaderPool": false,
  "/*": "默认为1024, 开启UseMsgReaderPool后生效。表示消息池中可持有的最大MsgReader数",
  "MaxMsgReaderNum": 8192,
  "/*": "被动tcp监听支持协议。ALL表示支持所有客户端协议, LDP表示只支持LDP协议的客户端, UST表示只支持UST协议客户端",
  "AcceptorProtocolType": "ALL",
  "/*": "使能Rcm代理发送端并发锁",
  "EnableRcmSendLock": true
}
```

4.20.2. 配置项说明

名称	类型	说明	备注
TcpFactory	对象数组	公有TCP工厂配置	与 REMOTER 对象配置中的私有TCP工厂配置对应, 公有TCP工厂能被所有 REMOTER 对象共用
Name	字符串	TCP工厂名	
SendRecvThreadCount	整型	设置网络库工作线程个数	默认为1, 该值大于0时, 工厂对象的每个工作线程同时负责套接字上数据发送和接收
SendThreadCount	整型	设置网络库发送线程个数	默认为1, SendRecvThreadCount设置为0时, 该值有效
RecvThreadCount	整型	设置网络库接收线程个数	默认为1, SendRecvThreadCount设置为0时, 该值有效
EnableAsynSend	布尔型	tcp通讯库的异步发送模式	默认值: false, 表示不开启异步发送模式
EnableDirectRecv	布尔型	是否采用直接接收模式	默认值: false
EnableNagle	布尔型	是否启用Nagle	默认值: false
EnableQuickAck	布尔型	是否启用快速ack	默认值: true
RdmaRecvThreadCount	整型	rdma接收线程个数	如果需要使用rdma, 默认0
ThreadBindCpu	整形数组	每个线程绑定cpu的序号	默认值: -1, 表示不绑定
MaxConnectionCount	整型	TCP工厂的最大连接数	默认值: 10000
TimerIntervalMilli	整型	Remoter插件自有定时线程时间间隔	单位: 毫秒, 定时最小间隔: 1ms, 默认: 1000ms
UseMsgReaderPool	布尔型	开启时Remoter插件生成一个MsgReader消息池, RpcClient不再默认生成1024个MsgReader, 按需消息池中获取, 调用Reset函数后归还。消息池初始化时默认生成1024个MsgReader, 超过后自动申请。无上限, 用于优化解除RpcClient上线后, RpcClient占用内存过多的问题。	默认值: false
MaxMsgReaderNum	整型	开启UseMsgReaderPool后生效, 配置值小于1024时会被置为1024。表示消息池中可持有的最大MsgReader数, 超过该上限的MsgReader仍然可以申请, 但是在Reset归还后相应空间会被释放。	默认值: 8192, 最小值:1024
AcceptorProtocolType	字符串	被动tcp监听支持协议。ALL表示所有客户端协议, LDP表示只支持LDP协议的客户	默认值: ALL

		端，UST表示只支持UST协议客户端。	
EnableRcmSendLock	布尔型	使能Rcm代理发送端并发锁，默认为true	默认值：true

4.20.3. REMOTER 对象示例配置

```
{
  "/*": "Create时是否自动启动",
  "AutoStart": true,
  "/*": "Backend路由项",
  "Routes": [
    {
      "Functions": "5:102",
      "Systems": "1",
      "Nodes": "1",
      "Backend": 1
    },
    {
      "Functions": "*",
      "Systems": "*",
      "Nodes": "*",
      "Backend": 2
    }
  ],
  "/*": "Backend接收应答主题",
  "RspTopic": {
    "Name": "rsp_topic",
    "EnableUnicastMode": false,
    "/*": "不配置接收分区时，使用上下文id作为分区号（不支持引用）",
    "PartitionNo": 1
  },
  "/*": "Backend订阅主题",
  "Subscribe": [
    {
      "Name": "push",
      "/*": "必须显示指定分区号，可配多个",
      "PartitionNos": [
        1
      ]
    }
  ],
  "/*": "Backend构建Singleton上下文使用的引用模板",
  "REF": "Default",
  "/*": "客户端Rcm上下文协议，支持 Udp组播：UDP-MULTICAST；Tcp单播：TCP-UNICAST；Udp单播：UDPUNICAST，默认Udp组播",
  "ProtocolMode": "UDP-MULTICAST",
  "/*": "没有Backend，说明不需要接出服务；有Backend但是Size=0，说明初始化时无接出服务，但可在运行期CreateBackend",
  "TcpInitiator": {
    "Onload": 1
  },
  "Backends": [
    {
      "/*": "Rcm对接主备核心",
      "Name": "EndName",
      "Id": 1,
      "Type": "AS-RCM",
      "TxTopic": "trade",
      "Arb": true,
      "ProductName": "ldp",
      "ClusterName": "xst",
      "/*": "使能会话模式",
      "EnableSession": true,
      "/*": "使能多分片模式",
      "EnableMultiShard": true,
      "/*": "使能查询优先模式",
      "EnableQueryFirst": true,
      "SendByMsgType": true
    },
    {
      "/*": "Rcm对接双活核心",
      "Id": 2,
      "Type": "RCM",
      "TxTopic": "trade"
    },
    {
      "/*": "Rcm对接查询核心，会话保持模式",
      "Id": 3,
      "Type": "RCM",
      "TxTopic": "trade",
      "ForwardMode": "Token"
    },
    {
      "/*": "Tcp对接主备核心，需握手",
      "Id": 4,
      "Type": "AS-TCP",
      "Arb": true,
      "ProductName": "ldp",
      "ClusterName": "xst",
      "/*": "使能会话模式",
      "EnableSession": true,
      "/*": "使能查询优先模式",
      "EnableQueryFirst": true,
      "Tcp": {
        "Onload": 1
      }
    },
    {
      "/*": "Tcp对接核心站点",
      "Id": 5,
      "Type": "AS-TCP",
      "Arb": false,
      "Sites": [
        {
          "Host": "192.168.86.102",
          "Port": 10001
        }
      ],
      "/*": "使能会话模式",
      "EnableSession": true
    },
    {
      "/*": "Tcp客户端",
      "Id": 6,
    }
  ]
}
```

```

        "Type": "TCP",
        "//": "使能会话模式",
        "EnableSession": true,
        "ForwardMode": "LoadBalance",
        "//": "连接模式",
        "ConnectMode": "ALL",
        "//": "自动重连时是否切换地址, 仅ConnectMode为ONE时有效",
        "ReconnectSwitchAddr" : false,
        "Sites": [
            {
                "Host": "192.168.86.102",
                "Port": 10002
            }
        ]
    },
    {
        "//": "对接LDP网关",
        "Id": 7,
        "Type": "GW",
        "MsgType": "LDP",
        "Name": "ldp_gateway",
        "EnableRegName": true,
        "Sites": [
            {
                "Host": "192.168.86.102",
                "Port": 10003
            }
        ]
    },
    {
        "//": "对接UST网关",
        "Id": 8,
        "Type": "GW",
        "MsgType": "UST",
        "Name": "ust_gateway",
        "Sites": [
            {
                "Host": "192.168.86.102",
                "Port": 10004
            }
        ]
    },
    {
        "//": "插件穿透其他插件",
        "Id": 9,
        "Type": "PLUGIN",
        "PluginName": "ldp_bizproc",
        "EnableCache": false,
        "Instance": 0
    },
    {
        {
            "//": "rcm负载均衡模式",
            "Id": 10,
            "Type": "RCM",
            "//": "必须为LoadBalance",
            "ForwardMode": "LoadBalance",
            "//": "默认分片号",
            "DefaultNodeNo": 1,
            "//": "支持不区分机房和本地机房优先, 其他模式启动失败",
            "QueryFirstMode": 0,
            "EnableSession": true,
            "//": "当前模式支持多分片或者单分片模式;当前配置为多分片",
            "EnableMultiShard": true,
            "TxTopic": "query",
            "//": "忽略消息带的分片号, 使用默认分片号",
            "IgnoreNodeNoInMsg": false
        },
        {
            "//": "rcm负载均衡模式",
            "Id": 11,
            "Type": "RCM",
            "//": "必须为LoadBalance",
            "ForwardMode": "LoadBalance",
            "//": "支持不区分机房和本地机房优先, 其他模式启动失败",
            "QueryFirstMode": 0,
            "TxTopic": "push",
            "//": "当前模式支持多分片或者单分片模式;当前配置为单分片",
            "EnableSession": true,
        }
    },
    {
        "//": "放置backend的singleton上下文的透传字段",
        "BackendContext": {
            "IsRecord": true
        },
        {
            "//": "0: 主备、1: 双活、2: 非集群模式",
            "HaMode": 0,
            "//": "服务端使用rcm代理 或 主备模式使用TcpSvr时所需, 支持网段/IP地址配置, 不能填回环地址",
            "LocalNetworkNumber": "192.168.86.0",
            "//": "服务端接入端",
            "Frontend": {
                "Tcp": {
                    "TcpFactory": [
                        {
                            "Name": "Default",
                            "//": "TCP线程名前缀, 默认: remoter",
                            "ThreadNamePrefix": "remoter",
                            "//": "默认为1, 该值大于0时, 工厂对象的每个工作线程同时负责套接字上数据发送和接收",
                            "SendRecvThreadCount": 1,
                            "//": "默认为1, SendRecvThreadCount设置为0时, 该值有效",
                            "SendThreadCount": 1,
                            "//": "默认为1, SendRecvThreadCount设置为0时, 该值有效",
                            "RecvThreadCount": 1,
                            "//": "是否异步发送消息",
                            "EnableAsyncSend": false,
                            "//": "是否采用直接接收模式",
                            "EnableDirectRecv": false,
                            "//": "是否启用Nagle",
                            "EnableNagle": false,
                            "//": "是否启用快速ack",
                            "EnableQuickAck": true,
                            "//": "设置Rdma接收线程个数",
                            "RdmaRecvThreadCount": 0,
                            "//": "每个线程绑定cpu的序号",
                            "ThreadBindCpu": [
                                1
                            ],
                        }
                    ],
                }
            }
        }
    },
    {

```

```

        "":"/: "最大连接数",
        "MaxConnectionCount": 10000
    }
},
"Acceptors": [
    {
        "Name": "remoter",
        "":"/: "指定TCP工厂配置",
        "FactoryName": "Default",
        "":"/: "RDMA本地网卡, 如果不使用rdma, 则删除此项配置",
        "RdmaIp": "",
        "":"/: "侦听地址, 默认表示所有网卡(不支持配置网段)",
        "Address": "",
        "":"/: "侦听端口号",
        "Port": 8001,
        "":"/: "是否端口复用",
        "SocketReuseAddress": true,
        "":"/: "最大连接数",
        "MaxConnectionCount": 10000,
        "":"/: "连接起始索引",
        "ConnectStartIndex": 0,
        "":"/: "连接自定义数据大小",
        "ConnectArgSizeBytes": 8,
        "":"/: "心跳间隔",
        "HeartbeatIntervalSec": 20,
        "":"/: "心跳超时次数",
        "HeartbeatTimes": 3,
        "":"/: "最大发送缓存",
        "MaxSendBuffSizeBytes": 0,
        "":"/: "最小接收缓存",
        "MinRecvBuffSizeBytes": 512,
        "":"/: "初始化发送缓冲区大小",
        "InitialSendBuffSizeBytes": 4194304,
        "":"/: "发送等待时间",
        "SendWaitTimeMilli": 0,
        "":"/: "黑白名单同时配置时, 只有黑名单生效",
        "AddressWhiteList": "",
        "AddressBlackList": "",
        "Onload": 1
    }
]
},
"":"/: "启用Rcm代理",
"Rcm": {
    "IsRcmProxy": true,
    "":"/: "服务端Rcm上下文协议, 支持 Udp组播: UDP-MULTICAST; Tcp单播: TCP-UNICAST; Udp单播: UDPUNICAST, 默认Udp组播",
    "ProtocolMode": "UDP-MULTICAST",
    "ShardNo": 50,
    "Contexts": [
        {
            "":"/: "端名, 非上下文名",
            "Name": "xxx",
            "AppName": "yyy",
            "AppNo": 0,
            "TierName": "mmm",
            "CtxSuffixName": "nnn",
            "IsSingleton": true,
            "IsAutoStart": true,
            "IsMsgSync": false,
            "IsConsumerCreate": false,
            "REF": "Default",
            "StartOrderMsgSqn": 10,
            "TxTopics": [
                {
                    "Name": "trade",
                    "":"/: "如果该发送主题已经在全局存在过了, 就必须配置别名, 别名必须唯一",
                    "AliaName": "zzz",
                    "":"/: "为true时, 接收端断开后, 在途数据自动确认",
                    "AutoAckWhenNoReceiver": false,
                    "":"/: "是否启动的时候直接创建transportid",
                    "AutoCreate": false,
                    "":"/: "若不配分区号, 表示发送全分区, 根据负载均衡发送",
                    "PartitionNos": [
                        1
                    ]
                }
            ],
            "RxTopics": [
                {
                    "Name": "return",
                    "EnableUnicastMode": false,
                    "":"/: "见配置项 ShardNo 的注释",
                    "PartitionNos": [
                        1
                    ]
                },
                {
                    "Name": "result",
                    "":"/: "仅在 IsSingleton = true 时有效, 用于自动生成接收分区, 分区号为上下文编号",
                    "IsPrivate": false,
                    "":"/: "和ShardNo合成接收主题(允许跟IsPrivate一起使用)",
                    "EnableMultiShard": true
                }
            ],
            "TxBypass": [
                {
                    "Name": "bypassA",
                    "PartitionNo": 1,
                    "":"/: "发送旁路主题协议可单独指定, 支持 Udp组播: UDP-MULTICAST; Tcp单播: TCP-UNICAST; Udp单播: UDPUNICAST, 默认Udp组播",
                    "ProtocolMode": "UDP-MULTICAST",
                    "":"/: "是否启动的时候直接创建transportid",
                    "AutoCreate": false,
                    "":"/: "非必填, 可以传入多个ip或网段",
                    "Ip": "192.168.86.0"
                }
            ],
            "RxBypass": [
                {
                    "Name": "bypassB",
                    "REF": "Default",
                    "PartitionNos": [
                        1
                    ]
                }
            ]
        }
    ]
}

```

```
    ],
    },
    "/*": "直接使用Rcm",
    "Rcm": {
        "IsRcmProxy": false,
        "TopicMapping": [
            {
                "ReqTopic": "req_topic",
                "RspTopic": "rsp_topic"
            }
        ],
        "Contexts": [
            {
                "Name": "yyy",
                "IsAutoStart": true,
                "/*": "纯Rcm模式不支持",
                "IsMsgSync": false,
                "/*": "纯Rcm模式不支持",
                "IsConsumerCreate": false
            },
            {
                "Name": "zzz",
                "IsAutoStart": false
            }
        ]
    }
},
"/*": "服务端主推",
"PushService": [
    {
        "Name": "Report",
        "/*": "插件主推",
        "Type": "PLUGIN",
        "PluginName": "ldp_offerproc",
        "Instance": "0"
    },
    {
        "Name": "Push",
        "/*": "Rcm主推",
        "Type": "RCM",
        "Topic": "push",
        "PartitionNo": 1
    }
],
"EnableMsgTypeCtrl": false,
"EnableAppStatusCtrl": false,
"EnableBizStatusCtrl": false,
"EnableBackendAppStatusCtrl": false,
"EnableBackendBizStatusCtrl": false,
"EnableAppStatusCtrlWhenNoWhitelist": true,
"/*": "服务端功能号列表",
"FunctionNoList": {
    "/*": "uint32_t []",
    "FunctionNos": [
        600570,
        600571
    ],
    "/*": "uint64_t []",
    "BizStatus": [
        0,
        1
    ]
}
}
```

4.20.4. 配置项说明

名称	类型	说明	备注
AutoStart	布尔型	Create时是否自动启动remoter	默认true，create及destroy时自动启停remoter；配置为false，则需模块自行调用start及stop来启停remoter 注意：remoter未启动完成时，主备身份、事件（tcp、rcm连接）或接收消息（tcp、rcm消息）等回调就可能触发；若宿主插件在回调中调用remoter相关接口（如ForwardMessage、SendMessage等），可能存在未定义的异常行为，上层需在remoter启动完成之后，才能正常使用remoter的所有接口
Routes	数组	功能号路由表	每个数组下参数Functions，Systems，Nodes为与的关系，必须满足三个条件下，才会正确的路由
Functions	字符串	功能号集合	英文分号分隔，如"11;30;319"，支持"*"，支持模糊匹配，?号表示一个数字，只能出现在最后。若不配置，则代表支持所有功能号
Systems	字符串	系统号集合	英文分号分隔，如"11;30;255"，支持"*"，不支持"?".若不配置，则代表支持所有系统号。范围是[0,255]。
Nodes	字符串	节点号集合	英文分号分隔，如"11;30;65535"，支持"*"，不支持"?".范围是[0,65535]。若配置Nodes:""或Nodes:""，前置启动加载配置时会报错，前置启动失败。Nodes字段可查看本文档4.2.3节选
Backend	整型	目标backendid	必须存在相应的 Backends.Id
RspTopic	对象	应答主题	
Name	字符串	应答主题	

EnableUnicastMode	布尔	该主题是否启用单播模式	可选，默认false（组播模式）
PartitionNo	整形	应答的分区号	非必填，不填则使用上下文Id作为分区号
Subscribe	数组	LDP RPC组件接收主推的主题	
Name	字符串	接收主推的主题	
PartitionNos	整型数组	分区号	可选。接收主推主题的分区，不填则接收所有该主题的分区
REF	字符串	指定引用模板	可选。Backend构建Singleton上下文使用的引用模板，默认模板 Default
ProtocolMode	字符串	协议类型	非必填，支持 Udp组播：UDP-MULTICAST；Tcp单播：TCP-UNICAST；Udp单播：UDP-UNICAST
TcpInitiator	对象	TcpInitiator的，SectionName为"Initiator"的配置透传对象	对象内部字段需平铺
Onload	整型	是否开启Onload加速	在onload启动模式时，默认值：1，即开启加速。有效值：0或1
Backends	数组		对应核心、网关、插件,最大65535
Name	字符串	端名	可选，默认 <Type>-<Id>
Id	整型	编号	[1, 65535]
Type	字符串	对端类型	AS-RCM, 即RCM主备 RCM, 即多活 AS-TCP, 即TCP主备 TCP, 即TCP客户端 GW, 即网关 PLUGIN, 插件交互
Tcp	对象	Tcp参数配置	当Type为AS-TCP, 或GW
Onload	整型	是否开启Onload加速	在onload启动模式时，默认值：1，即开启加速。有效值：0或1
TxTopic	字符串	Rcm发送主题	当Type为AA或AS-RCM时有效
Arb	布尔型	是否使用仲裁	当Type为AS-TCP或AS-RCM时有效 默认值:false. 当为true时, 启动时应指定zk, 否则报错退出 当为false时, 必须有Sites配置,否则报错退出
ProductName	字符串	产品名	当Type为AS-TCP或AS-RCM,同时设置仲裁模式时有效，默认值:启动时命令行指定的产品名
ClusterName	字符串	集群名	当Type为AS-TCP或AS-RCM,同时设置仲裁模式时有效，建议前置与核心部署在同一个应用的话，配置为应用名，如果前置与核心没有部署在同一个核心，则配置为核心的应用名，默认值:应用名
EnableSession	布尔型	是否开启会话保持	当Type为RCM时，可开启会话保持模式，转发保持模式见 ForwardMode 配置项
EnableMultiShard	布尔型	是否开启多分片模式	默认为false，注意开启时客户端和服务端需同步配置,在开启多分片的时候默认会开启会话
EnableQueryFirst	布尔型	是否开启查询优先模式	默认为false
DefaultNodeNo	整型	默认分片号	当多分片模式，如果消息不带分片号，则使用默认分片号进行分片集群获取并发送
IgnoreNodeNoInMsg	布尔型	是否忽略消息中带的分片号	忽略消息带的分片号, 使用默认分片号进行分片集群获取并发送
MsgCacheSize	整型	重发消息缓存队列大小	非必填，默认值： 0, 表示不开启消息缓存机制。非0代表开启请求重发机制。仅在以下配置下生效：Type=AS-RCM并且 EnableQueryFirst=false并且(EnableSession=true或 EnableMultiShard=true)。约束说明：如果缓存消息个数超过该队列大小，按FIFO原则主动丢弃缓存队列中的历史请求，当出现主备切换场景，这些主动丢弃的历史请求将不会重发导致此场景下RPO不为0
CacheTimeout	整型	重发消息老化时间	非必填，默认值： 5000。该配置项仅在Type=AS-RCM模式且启用消息重传机制下生效，用于控制进入到消息缓存队列后的消息如果超过该时间阈值则该消息将被主动丢弃
ResendBlackFuncList	字符串	请求重发功能号黑名单	非必填，默认值： 空。仅在启用消息重传机制下生效,功能号仅支持数字功能号，以';'分隔，

			支持通配符"?"。"%"表示禁用重发缓存功能
ResendWhiteFuncList	字符串	请求重发功能号白名单	非必填，默认值： 空。仅在启用消息重传机制下生效,功能号仅支持数字功能号，以','分隔，支持通配符"?"，不支持通配符"%".如果重发功能号黑名单非空，则白名单机制失效，以黑名单控制为准；如果黑名单和白名单均为空，则所有功能号均适用于重发机制;白名单机制仅在黑名单列表为空时生效。
SendByMsgType	布尔型	Rcm类型的Backend发送时，是否校验消息类型（若为true，只允许发送请求类型的消息）	默认为true
ForwardMode	字符串	转发模式	当Type为RCM，且开启了会话保持模式时有效 当为Token时：根据请求中的Token作一致性Hash，获取查询核心的发送分区号 当为NodeNo时：根据请求中的NodeNo作为查询核心的发送分区号 当为LoadBalance时，会负载均衡发送给各个符合要求的核心中去处理，支持不区分机房和本地机房模式。目前支持多分片和单分片模式。 默认为NodeNo； 当Type是TCP,支持Foreach和LoadBalance模式； 当为Foreach时，会将消息发送给当前backend下所有的site核心 当为LoadBalance时，会轮询发给Site下的核心，直到发送到正确的核心为止(会话开启时，且EnableBackendBizStatusCtrl为true，则会校验业务状态，而默认校验平台状态，只有所有状态对，才会发送) 默认为Foreach；
ConnectMode	字符串	连接模式	默认为ALL ALL: 与Sites中的所有地址建立连接 ONE: 仅会与Sites中一个地址建立连接，其他作为冷备地址。首次连接的地址为Sites中随机一个，后续按照Sites中的顺序切换
ReconnectSwitchAddr	布尔型	自动重连时，是否切换地址后进行连接	默认为false false: 自动重连时，先尝试连接原地址，失败后再切换地址进行连接 true: 自动重连时，直接切换地址进行连接
Sites	数组		当Type为AS-TCP且Arb配置为false,或GW时有效，如果Sites里配置多个ip和端口，对于GW来说，数据发送采用轮询的方式依次发送，对AS-TCP来说则是由客户端的请求（去主去备）来决定。 注：最多配置128个Site
Host	字符串	主机IP	
Port	整型	主机侦听端口	
MsgType	字符串	消息类型	目前该项配置仅对网关有效,默认LDP LDP,即对接新网关 UST,即对接老网关
EnAbleRegName	布尔型	是否向网关注册客户端名字	
PluginName	字符串	插件名	当Type为PLUGIN时有效
EnableCache	布尔型	是否开启对接的插件服务端的消息连接号缓存功能	默认为false
Instance	整型	插件实例号	当Type为PLUGIN时有效
BackendContext	对象	放置backend的singleton上下文的透传字段	
HaMode	整型	高可用模式	默认值： 0，0表示主备模式，1表示多活模式，2表示非集群模式
LocalNetworkNumber	字符串	本地网卡的网络号	服务端使用rcm代理 或 主备模式使用TcpSvr时所需，支持网段/IP地址配置，不能填回环地址
Frontend	数组	服务端接入端	
Tcp	对象	TCP类型服务端	
TcpFactory	对象	私有TCP工厂配置	与 REMOTER 插件配置中的公有TCP工厂配置对应，私有TCP工厂不能与其他 REMOTER 对象共用
Name	字符串	TCP工厂名	

ThreadNamePrefix	字符串	TCP线程名前缀	默认: <code>remoter</code>
SendRecvThreadCount	整型	设置网络库工作线程个数	默认为1, 该值大于0时, 工厂对象的每个工作线程同时负责套接字上数据发送和接收
SendThreadCount	整型	设置网络库发送线程个数	默认为1, <code>SendRecvThreadCount</code> 设置为0时, 该值有效
RecvThreadCount	整型	设置网络库接收线程个数	默认为1, <code>SendRecvThreadCount</code> 设置为0时, 该值有效
EnableAsynSend	布尔型	<code>tcp</code> 通讯库的异步发送模式	默认值: <code>false</code> , 表示不开启异步发送模式
EnableDirectRecv	布尔型	是否采用直接接收模式	默认值: <code>false</code>
EnableNagle	布尔型	是否启用Nagle	默认值: <code>false</code>
EnableQuickAck	布尔型	是否启用快速ack	默认值: <code>true</code>
RdmaRecvThreadCount	整型	<code>rdma</code> 接收线程个数	如果需要使用 <code>rdma</code> , 默认0
ThreadBindCpu	整形数组	每个线程绑定cpu的序号	默认值: -1, 表示不绑定
MaxConnectionCount	整型	TCP工厂的最大连接数	默认值: 10000
Acceptors	数组	TCP服务端配置	最多支持8个
Name	字符串	TCP服务端名	用户自定义, 不能与其他TCP服务端名字重复
FactoryName	字符串	指定TCP工厂配置	可指定私有或公有TCP工厂, 优先在私有TCP工厂中查找 私有TCP工厂找不到, 再从公有TCP工厂中查找 公有TCP工厂找不到, 若指定的TCP工厂名是 <code>Default</code> , 但在 <code>REMOTER</code> 插件中未配置 <code>Default</code> TCP工厂, 底层会自动创建 <code>Default</code> TCP工厂 否则启动报错
RdmaIp	字符串	<code>rdma</code> 网卡IP地址	如果不使用 <code>rdma</code> , 则删除此项配置
Address	字符串	侦听IP地址	侦听IP地址, 不配置表示通配地址
Port	整型	侦听端口	侦听端口, 默认值: 8001
SocketReuseAddress	布尔型	是否端口复用	默认值: <code>true</code>
MaxConnectionCount	整型	<code>Acceptor</code> 的最大连接数	默认值: 10000, 优先级高于 TCP 工厂的最大连接数
ConnectStartIndex	整型	连接起始索引号	
ConnectArgSizeBytes	整型	连接自定义数据大小	
HeartbeatIntervalSec	整型	心跳时间间隔	默认值: 20, 单位: 秒
HeartbeatTimes	整型	心跳超时次数	默认值: 3
MaxSendBuffSizeBytes	整型	最大发送缓存	默认值: 0, 表示不限制, 单位: 字节
MinRecvBuffSizeBytes	整型	最小接收缓存	默认值: 512, 单位: 字节
InitialSendBuffSizeBytes	整型	初始化缓冲区大小	默认值: 4MB, 单位: 字节, 最大支持 16MB, 最小支持128kb, 当配置非法时, 那么使用默认值
SendWaitTimeMilli	整型	发送等待时间	默认值: 0, 单位: 毫秒
AddressWhiteList	字符串	IP白名单	支持模糊匹配, *号必须放最开头或最结尾, ?号位置随意, 每个ip必须以;号结尾, 包括最后一个最长支持1024个字符
AddressBlackList	字符串	IP黑名单	支持模糊匹配, *号必须放最开头或最结尾, ?号位置随意, 每个ip必须以;号结尾, 包括最后一个最长支持1024个字符
Rcm	对象	<code>Rcm</code> 类型服务端	
IsRcmProxy	布尔型	是否使用RCM代理模式	默认为 <code>true</code>
ShardNo	整型	分片号	上下文的 <code>RxTopics</code> 分区生效规则 (数字代表优先级顺序): 1, 配了 <code>PartitionNos</code> , 就用 <code>PartitionNos</code> 中的分区号 2, 没配 <code>PartitionNos</code> 但配了 <code>REF</code> , 就用主题模板中的分区号 3, 没配 <code>PartitionNos</code> & <code>REF</code> 但配了 <code>ShardNo</code> , 就用 <code>ShardNo</code> 指定的分区号 注: 以上三者都没配会报错
Contexts	数组	上下文配置	
Name	字符串	<code>Rcm</code> 服务端名	用户自定义, 不能与其他RCM服务端名字重复

			注：非上下文名，Rcm代理模式下，上下文名自动生成 服务端普通高可用上下文名：<本节点AppName>_<宿主插件名>_<宿主插件实例号>_Tier 服务端非高可用上下文名：<本节点AppName>_<宿主插件名>_<宿主插件实例号>_Singleton 客户端非高可用上下文名：<本节点AppName>_<宿主插件名>_<宿主插件实例号>_backend_Singleton
AppName	字符串	集群应用名	仅在 IsMsgSync = true 时有效，建议配成Arb插件所在节点的集群名（主/备相同） 注：作为多进程上下文的 TierName(集群名)，若宿主插件没有配置AppName，或者AppName为空串，则取本节点的ShortAppName
AppNo	整型	集群应用实例号	仅在 IsMsgSync = true 时有效，建议配成Arb插件所在节点的集群编号（主/备不同） 注：作为多进程上下文的 Name(上下文名，生成格式：<配置的AppName>#<AppNo>_kdp_remoter_0)，若宿主插件没有配置AppNo，则取本节点的AppNo
TierName	字符串	集群名	仅在 IsMsgSync = false 时有效，核心以前有这个配置，向前兼容 注：作为高可用上下文的 TierName(集群名)，若宿主插件没有配置TierName，则生成默认的集群名：<本节点ShortAppName>_<宿主插件名>_<宿主插件实例号>
CtxSuffixName	字符串	上下文名后缀名	仅在 IsMsgSync = false && IsSingleton = true 时有效，非必填项，没有该选项时，自动生成的上下文名无相应后缀
IsSingleton	布尔型	是否高可用上下文	默认为true
IsAutoStart	布尔型	上下文是否自动开启接收消息，若为false时，需通过 ILdpRcmFrontend::Start 接口开启接收消息	默认为true
IsMsgSync	布尔型	是否启用多进程Rcm上下文	默认为false
IsConsumerCreate	布尔型	仅在IsMsgSync=true时有效，是否需要创建Consumer，不创建Consumer的情况下依然可以收到角色改变通知	默认为false
REF	字符串	引用模版	必填，默认引用"Default"模版，必须保证对应模版存在
StartOrderMsgSqn	整型	RCM最大排队序号	仅在IsMsgSync=true时有效，非-1时设置到rcm的多进程上下文
TxTopics	数组	发送主题	
Name	字符串	主题名	必填
AliaName	字符串	别名	可选，如果填写，则必须保证全局唯一
AutoAckWhenNoReceiver	布尔	无接收端时自动确认	可选，默认为false，详细说明见《低延时分布式开发平台_使用指南》中rcm配置说明
PartitionNos	整形数组	分区号	可选，用于标识可以发送哪些分区，不填则可以发往所有分区
AutoCreate	布尔	是否需要启动时创建transportid，能直接启动成功	默认false。如果为true，则启动的时候就会创建transportid。注意，当配置存在AutoCreate为true是，Txtopic必须配置PartitionNos，TxBypass必须配置PartitionNo。
RxTopics	数组	接收主题	可选，用于标识可以发送哪些分区，不填则可以发往所有分区
Name	字符串	主题名	必填
IsPrivate	布尔	是否为私有主题	可选，默认false，用于自动生成接收分区，分区号为上下文编号，目前仅支持非集群通道
EnableMultiShard	布尔	开启服务端多分片模式	可选，默认false，要求：服务端和客户端要同时开启或同时关闭，服务端开启时，需指定配置ShardNo
PartitionNos	整形数组	分区号	可选，用于标识能接收哪些分区，若不填，则必须配置REF

TxBypass	数组	旁路发送主题	目前一个通道仅支持配置一个
Name	字符串	主题名	必填
PartitionNo	整形	分区号	非必填，默认分区1
ProtocolMode	字符串	协议类型	非必填，支持 Udp组播：UDP-MULTICAST；Tcp单播：TCP-UNICAST；Udp单播：UDP-UNICAST
Ip	字符串	本机ip或网段，多个ip或网段之间用逗号分隔，不可以存在其他符号	非必填，可以传入多个ip或网段，rcm代理会根据正确网段匹配本机ip
RxBypass	数组	旁路接收主题	目前仅支持非集群通道
Name	字符串	主题名	必填
REF	字符串	引用模版	可选，若不填，则必须配置PartitionNos
PartitionNos	整形数组	分区号	可选，用于标识能接收哪些分区，若不填，则必须配置REF
TopicMapping	对象	请求应答主题映射关系	仅在IsRcmProxy=false时有效
ReqTopic	字符串	请求主题名	仅在IsRcmProxy=false时有效
RspTopic	字符串	应答主题名	仅在IsRcmProxy=false时有效
PushService	数组	主推服务	
Name	字符串	服务名	所有主推服务名必须唯一,应用通过服务名获取对应的服务对象
Type	字符串	服务类型	AA, 通过RCM主推 PLUGIN, 通过插件主推
PluginName	字符串	插件标识	当Type为PLUGIN时有效
Instance	整型	插件实例编号	当Type为PLUGIN时有效
Topic	字符串	发送主题	当Type为AA时有效
PartitionNo	整形	发送分区号	非必填，仅在RcmProxy模式下有效，默认值：0（分区负载均衡发送） 注：非RcmProxy模式下，主推必须指定发送分区号
EnableMsgTypeCtrl	布尔	是否开启消息类型控制	可选，默认false，仅主备模式有效（开启后，主机过滤去备消息，备机过滤去主消息）
EnableAppStatusCtrl	布尔	是否开启应用状态控制	可选，默认false，若为true，则：Frontend需要上层设置为AppStatus::Ready后，该端的消息才会回调给上层；Backend需要服务端的Frontend设置为AppStatus::Ready，且随心跳包同步到客户端后，该消息才会被发送给服务端
EnableBizStatusCtrl	布尔	是否开启业务状态控制	可选，默认false
EnableBackendAppStatusCtrl	布尔	是否开启接出端应用状态控制	可选，默认false，若为true，则客户端会通过remoter心跳包获取服务端的应用状态，状态为AppStatus::Ready后，才会将消息发送出去（生效于Backend，状态同步存在延迟）
EnableBackendBizStatusCtrl	布尔	是否开启接出端业务状态控制	可选，默认false（生效于Backend，状态同步存在延迟）
FunctionNoList	对象	服务端功能号列表	仅在 EnableBizStatusCtrl = true 时有效，由Frontend提供
FunctionNos	数组	功能号列表	
BizStatus	数组	业务状态列表	要求与 功能号列表一一对应
EnableAppStatusCtrlWhenNoWhiteList	布尔	在未开启白名单的状态下，是否使用应用状态控制	默认为true，当核心开启功能号白名单（"AppStatusCtrlWhiteFuncList"）时，该配置无效。关闭时，通过该开关来决定是否启用应用状态控制。建议在DB将该开关置为false，DB模式无需使用应用状态控制；MDB模式时可以开启该配置，使用应用状态控制。

4.20.5. 开发示例

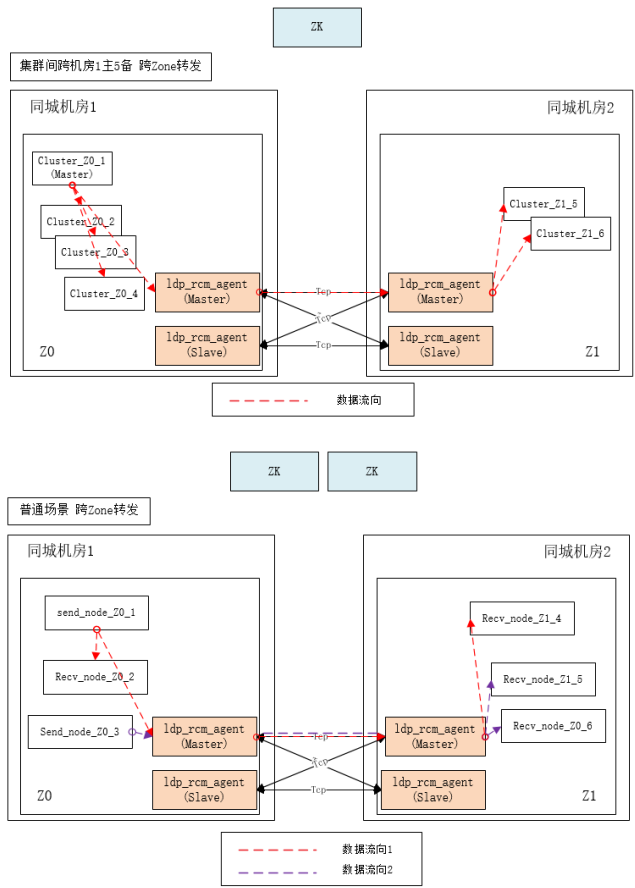
- 详见《低延时分布式开发平台_服务调用框架插件开发指南.pdf》。

4.21. Rcm网络代理插件 ldp_rcm_agent

ldp_rcm_agent是一个透明的rcm组播包转发节点，主要是解决 rcm组播不通的问题。

插件名	支持平台	插件标识	说明

4.21.1. ldp_rcm_agent在系统中的位置及数据流向



4.21.2. ldp_rcm_agent 使用到的配置

1. 各Zone中各Agent服务节点的Ip和端口配置
2. 转发路由表
3. Rcm 配置 (这部分配置数据不属于本插件，而是由本插件和其它使用rcm进行通信的插件共用)

4.21.3. 配置项说明

```

"Lib": "ldp_rcm_agent",
"Instance": 0,
"Timer": 0,
"LocalZone": "Z0",
"/": "多网卡根据下面的mask决定绑定的本地Ip",
"LocalNetworkNumber": "10.20.137.0",
"TcpSockBufferSizeKBytes": 16384,
"UdpSockBufferSizeKBytes": 512,
"Agent": [
    {
        "Zone": "Z1",
        "/": "Z1 中的两个Agent实例Ip与端口",
        "Sites": [
            {
                "Host": "10.20.137.137",
                "Port": 8003
            },
            {
                "Host": "10.20.137.139",
                "Port": 8004
            }
        ]
    },
    {
        "Zone": "Z0",
        "/": "Z0 中的两个Agent实例Ip与端口",
        "Sites": [
            {
                "Host": "10.20.138.138",
                "Port": 8003
            },
            {
                "Host": "10.20.138.140",
                "Port": 8004
            }
        ]
    }
],
"ForwardTable": [
    {
        "/": "Z0, Z3的所有主题、集群间复制消息转发到Z1, Z2的对应主题或集群，主机房和同城机房一般采用此规则即可",
        "From": {
            "Zone": "Z0, Z3"
        },
        "To": {
            "Zone": "Z1, Z2"
        }
    },
    {
        "/": "Z0的Topic1, Topic2, Topic3*主题的所有分区，转发到Z2的Topic1主题的对分区， 主机房与异地机房采用此规则即可",
        "From": {
            "Zone": "Z0",
            "Topic": "Topic1, Topic2, Topic3*"
        },
        "To": {
            "Zone": "Z2"
        }
    },
    {
        "/": "Z0的Topic2主题的1、2分区，转发到Z1的Topic2主题的对分区",
        "From": {
            "Zone": "Z0",
            "Topic": "Topic2",
            "Partitions": [
                1,
                2
            ]
        },
        "To": {
            "Zone": "Z1"
        }
    },
    {
        "/": "Z0的Topic3主题的1分区，转发到Z1的Topic4主题的2分区",
        "From": {
            "Zone": "Z0",
            "Topic": "Topic3",
            "Partitions": [
                1
            ]
        },
        "To": {
            "Zone": "Z1",
            "Topic": "Topic4",
            "Partitions": [
                2
            ]
        }
    }
],
]
}

```

4.21.4. 插件ldp_rcm_agent实例配置参数说明

配置项	类型	含义	备注
LocalZone	字符串	本实例所属的Zone	
LocalNetworkNumber	字符串	ldp_rcm_agent实例节点获取本地用于传输Tcp包的网卡Ip地址掩码	
LocalNetworkNumberUdp	字符串	ldp_rcm_agent实例节点获取本地用于传输udp组播的网卡的Ip地址掩码	如果没有配置,udp组播网卡Ip会默认使用用于Tcp传输的网卡Ip
TcpSockBufferSizeKBytes	整型	tcp套接字缓冲区长度(KBytes)	默认或配置参数值非法, 取16M与系统默认缓冲区长度的两者中较大值,最大长度1G字

			节，如果超出1G也会被设成1G
UdpSockBufferSizeKBytes	整型	udp套接字缓冲区长度(KBytes)	默认或配置参数值非法，取512K与系统默认缓冲区长度的两者中较大值,最大长度1G字节，如果超出1G也会被设成1G
Agent	数组	各Zone中各Agent服务节点的Ip和端口配置	
Zone	字符串	Zone的名字	必须来自于Rcm配置中的Zones里已存在的Zone名
Sites	数组	本Zone包含的多个ldp_rcm_agent节点的Ip和端口列表	
Host	字符串	ldp_rcm_agent实例节点的Ip	
Port	整型	ldp_rcm_agent实例节点的端口	
ForwardTable	数组	路由转发表配置	
From	对象	转发数据源信息	
Zone	字符串	转发源Zone名称，可以是逗号分隔的一个或多个Zone名	必须来自于Rcm配置中的Zones里已存在的Zone名
Topic	字符串	转发源的Topic(逗号分隔的一个或多个Topic名称，Topic名称只可以在名称最后包含*，表示按前缀模糊匹配多个Topic名称)	
Partitions	整型数组	分区号列表	
To	对象	转发目的信息	
Zone	字符串	目的Zone名称，可以是逗号分隔的一个或多个Zone名	必须来自于Rcm配置中的Zones里已存在的Zone名
Topic	字符串	转发源或目的 Topic	
Partitions	整型数组	分区号列表	

4.21.5. 转发路由表支持的配置项类型

转发路由表里主要记录了源Zone [+Topic + Partitions] 到目的 Zone [+Topic + Partitions]的映射转发表里的选项，主要有四种类型：

- 1. Zone1 ⇒ Zone2 : 将Zone1 和Zone2按Topic名字和Partition进行匹配转发，不同的不转发
- 2. Zone1 + Topiclist(例如:Topic1,Topic2,Topic3*) ⇒ Zone2 : 将Zone1里的包含在Topiclist 里的topic 按名字和Partition匹配转发；
- 3. Zone1 + Topic + Partitions ⇒ Zone2 : 将Zone1里的 Topic + Partitions 按Topic名字+分区匹配转发到Zone2
- 4. Zone1 + Topic + Partitions ⇒ Zone2 + Topic+ Partitions : 按Topic + Partitions 匹配转发

以上四种类型1到4，匹配条件由泛化到精确，后面的精确匹配的配置覆盖前面泛化条件的配置；配置时也需要把泛化条件放前面，精确匹配条件放后面；如果目的Zone(Zone2)里找不到对应的Partition,则不转发

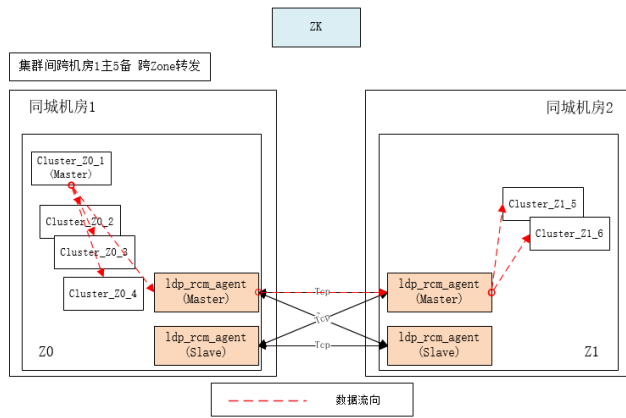
实例启动时，根据LocalNetworkNumber 获取本地绑定地址(可能包含多网卡)；根据LocalZone 获得本节点所属Zone,然后向其它Zone的Agent节点建立Tcp转发连接； 依据 两个Zone的rcm配置 + 转发表，生成常驻内存的转发表缓存，供真正做包转发时查询路由使用。

4.21.6. 其他说明

- 1. 代理集群间复制的组播流量时，不能与集群节点部署在同一台机器，因为集群模式不支持本机网络环回！
- 2. 从源Zone发往目的Zone的流量，同一时刻只存在于一路Tcp连接中，当Tcp连接出现问题才会做连接切换，切换后会一直使用该连接直到下一次切换。
- 3. 源Zone和目的Zone Rcm的上下文(Context)Id的物料范围不能重叠，Rcm 上下文(Context)的名字不能有相同，否则代理将不能正常工作。
- 4. 源Zone和目的Zone组播是互通的时候，使用网络代理将会产生较多无用的转发包，降低整体性能。
- 5. 在部署阶段，为保证网络代理能正常工作，需要手动在各自机房的rcm配置里加上名称为 "Mirror_Default"的上下文模板。
- 6. 运维阶段，如果配置发生过较多修改，建议先停止所有网络代理节点然后再依次启动
- 7. 跨Zone的Tcp数据传输在当前版本未做加密,建议使用专线部署。

4.21.7. 示例配置

示例1： 1主5从 集群间数据跨zone同步， Z0, Z1包含的所有业务节点和ldp_rcm_agent代理节点，共用同一个zk上的同一份配置。



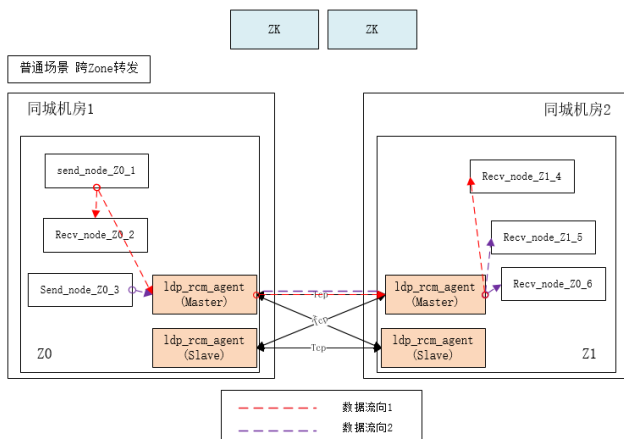
```
{
  "Applications": { },
  "REFS": {
    "rcm_agent": {
      {
        "Components": {
          {
            "Libs": [ ],
            "GlobalLibs": [ ]
          },
          "Plugins": [
            {
              "Lib": "ldp_rcm_agent",
              "Instance": 0,
              "Timer": 0,
              "LocalNetworkNumber": "10.20.137.0",
              "TcpSockBufferSizeKBytes": 16384,
              "UdpSockBufferSizeKBytes": 512,
              "Agent": [
                {
                  "Zone": "Z0",
                  "Sites": [
                    {
                      "Host": "10.20.137.137",
                      "Port": 8001
                    },
                    {
                      "Host": "10.20.137.139",
                      "Port": 8001
                    }
                  ]
                },
                {
                  "Zone": "Z1",
                  "Sites": [
                    {
                      "Host": "10.20.138.138",
                      "Port": 8001
                    },
                    {
                      "Host": "10.20.138.140",
                      "Port": 8001
                    }
                  ]
                }
              ]
            },
            {
              "ForwardTable": [
                {
                  "From": {
                    "Zone": "Z0"
                  },
                  "To": {
                    "Zone": "Z1"
                  }
                }
              ]
            },
            {
              "Lib": "ldp_mproxy",
              "From": "参见 ldp_mproxy 配置"
            },
            {
              "Lib": "ldp_logproxy",
              "From": "参见 ldp_logproxy 配置"
            },
            {
              "Lib": "ldp_rcm",
              "ConfigUpdate": true, "From": "必须设置为true"
            }
          ]
        }
      },
      "Derived": {
        "From": "4个ldp_rcm_agent 节点实例",
        "rcm_agent": {
          "rcm_agent#0": {
            "ldp_rcm_agent.LocalZone": "Z0",
            "ldp_rcm_agent.LocalNetworkNumber": "10.20.137.0"
          },
          "rcm_agent#1": {
            "ldp_rcm_agent.LocalZone": "Z0",
            "ldp_rcm_agent.LocalNetworkNumber": "10.20.137.0"
          },
          "rcm_agent#2": {
            "ldp_rcm_agent.LocalZone": "Z1",
            "ldp_rcm_agent.LocalNetworkNumber": "10.20.138.0"
          },
          "rcm_agent#3": {
            "ldp_rcm_agent.LocalZone": "Z1",
            "ldp_rcm_agent.LocalNetworkNumber": "10.20.138.0"
          }
        }
      }
    }
  }
}
```

```

    }
  },
  "Rcm": {
    "Transports": [
      {
        "Name": "Default",
        "PartitionNo": 1,
        "AsynchronousRms": 1
      }
    ],
    "Contexts": [ "/*": "1主5从6个rcm contexts",
      {
        "REF": "Default",
        "Name": "Replication",
        "IsTotalOrder": true,
        "IsRecord": true,
        "IsSingleton": false,
        "TierName": "cluster_Tier",
        "SyncIp": "10.20.137.0",
        "SyncAddr": "237.0.0.1",
        "SyncPort": 30005,
        "SyncMode": 1
      },
      {
        "REF": "Replication",
        "Name": "cluster_Z0_1",
        "Id": 14,
        "TierName": "Tier1",
        "Zone": "Z0"
      },
      {
        "REF": "Replication",
        "Name": "cluster_Z0_2",
        "Id": 15,
        "TierName": "Tier1",
        "Zone": "Z0"
      },
      {
        "REF": "Replication",
        "Name": "cluster_Z0_3",
        "Id": 16,
        "TierName": "Tier1",
        "Zone": "Z0"
      },
      {
        "REF": "Replication",
        "Name": "cluster_Z0_4",
        "Id": 17,
        "TierName": "Tier1",
        "Zone": "Z0"
      },
      {
        "REF": "Replication",
        "Name": "cluster_Z1_5",
        "Id": 18,
        "TierName": "Tier1",
        "Zone": "Z1"
      },
      {
        "REF": "Replication",
        "Name": "cluster_Z1_6",
        "Id": 19,
        "TierName": "Tier1",
        "Zone": "Z1"
      }
    ],
    "Zones": [
      {
        "Name": "Z0",
        "RPO": 0,
        "SyncAckPoint": "after_consume"
      },
      {
        "Name": "Z1",
        "RPO": 0,
        "SyncAckPoint": "after_consume"
      }
    ],
    "MaterialRange": {
      "/*": "参见rcm插件配置"
    }
  }
}

```

示例2: Topic跨Zone转发, Z0 ⇒ Z1



Z0配置, rcm配置仅包含属于Z0的Transports 和Contexts

```

"Applications": {},
"REFS": {
  "rcm_agent1": {
    "Components": {
      "Libs": [],
      "GlobalLibs": []
    },
    "Plugins": [
      {
        "Lib": "ldp_rcm_agent",
        "Instance": 0,
        "Timer": 0,
        "LocalNetworkNumber": "10.20.137.0",
        "TcpSockBufferSizeBytes": 16384,
        "UdpSockBufferSizeKBytes": 512,
        "Agent": [
          {
            "Zone": "Z0",
            "Sites": [
              {
                "Host": "10.20.137.137",
                "Port": 8001
              },
              {
                "Host": "10.20.137.139",
                "Port": 8001
              }
            ]
          },
          {
            "Zone": "Z1",
            "Sites": [
              {
                "Host": "10.20.138.138",
                "Port": 8001
              },
              {
                "Host": "10.20.138.140",
                "Port": 8001
              }
            ]
          }
        ]
      },
      {
        "ForwardTable": [
          {
            "From": {
              "Zone": "Z0",
              "Topic": "Topic1,Topic2,Topic3*"
            },
            "To": {
              "Zone": "Z1"
            }
          }
        ]
      },
      {
        "Lib": "ldp_mproxy",
        "///<": "参见 ldp_mproxy 配置"
      },
      {
        "Lib": "ldp_logproxy",
        "///<": "参见 ldp_logproxy 配置"
      },
      {
        "Lib": "ldp_rcm",
        "ConfigUpdate": true, "///<": "必须设置为true"
      }
    ]
  }
},
"Derived": {
  "rcm_agent1": {
    "rcm_agent1#0": {
      "ldp_rcm_agent.LocalZone": "Z0",
      "ldp_rcm_agent.LocalNetworkNumber": "10.20.137.0"
    },
    "rcm_agent1#1": {
      "ldp_rcm_agent.LocalZone": "Z0",
      "ldp_rcm_agent.LocalNetworkNumber": "10.20.137.0"
    },
    "rcm_agent1#2": {
      "ldp_rcm_agent.LocalZone": "Z1",
      "ldp_rcm_agent.LocalNetworkNumber": "10.20.138.0"
    },
    "rcm_agent1#3": {
      "ldp_rcm_agent.LocalZone": "Z1",
      "ldp_rcm_agent.LocalNetworkNumber": "10.20.138.0"
    }
  }
},
"Rcm": {
  "Transports": [
    {
      "Name": "Default",
      "PartitionNo": 1,
    },
    {
      "Id": 1,
      "Topic": "Topic1",
      "Addr": "235.0.0.1",
      "Port": 30000
    },
    {
      "Id": 2,
      "Topic": "Topic2",
      "Addr": "235.0.0.2",
      "Port": 30001
    },
    {
      "Id": 3,
      "Topic": "Topic3",
      "Addr": "235.0.0.3",
      "Port": 30002,
    }
  ],
  "Contexts": [

```

```

    {
        "Name": "Default",
        "Ip": "10.20.137.0",
        "IsRecord": false,
        "IsSingleton": true,
        "IsTotalOrder": false,
    },
    {
        "REF": "Default",
        "Name": " send_node_z0_1",
        "Zone": "Z0",
        "Id": 1,
        "TxTopics": [
            {
                "Name": "Topic1"
            }
        ]
    },
    {
        "REF": "Default",
        "Name": " recv_node_z0_2",
        "Zone": "Z0",
        "Id": 2,
        "RxTopics": [
            {
                "Name": "Topic1"
            }
        ]
    },
    {
        "REF": "Default",
        "Name": " send_node_z0_3",
        "Id": 3,
        "TxTopics": [
            {
                "Name": "Topic2"
            }
        ]
    }
],
"Zones": [
    {
        "Name": "Z0",
        "RPO": 0,
    },
    {
        "Name": "Z1",
        "RPO": 0,
    }
],
"MaterialRange": {
    "//": "参见rcm插件配置"
}
}

```

Z1配置， rcm配置仅包含属于Z1的Transports 和Contexts


```

{
  "Applications": {},
  "REFS": {
    "rcm_agent": {
      "Components": {
        "Libs": [],
        "GlobalLibs": []
      },
      "Plugins": [
        {
          "Lib": "ldp_rcm_agent",
          "///<": "参见 ldp_rcm_agent 配置"
        },
        {
          "Lib": "ldp_mproxy",
          "///<": "参见 ldp_mproxy 配置"
        },
        {
          "Lib": "ldp_logproxy",
          "///<": "参见 ldp_logproxy 配置"
        },
        {
          "Lib": "ldp_rcm",
          "ConfigUpdate": true, "///<": "必须设置为true"
        }
      ]
    }
  },
  "Derived": {
    "///<": "与z0配置相同,参见Z0"
  },
  "Rcm": {
    "Transports": [
      {
        "Name": "Default",
        "PartitionNo": 1,
      },
      {
        "Id": 1,
        "Topic": "Topic1",
        "Addr": "235.0.0.1",
        "Port": 30000
      },
      {
        "Id": 2,
        "Topic": "Topic2",
        "Addr": "235.0.0.2",
        "Port": 30001
      },
      {
        "Id": 3,
        "Topic": "Topic3",
        "Addr": "235.0.0.3",
        "Port": 30002,
      }
    ],
    "Contexts": [
      {
        "Name": "Default",
        "Ip": "10.20.138.0",
        "IsRecord": false,
        "IsSingleton": true,
        "IsTotalOrder": false,
      },
      {
        "REF": "Default",
        "Name": " recv_node_z1_4",
        "Zone": "Z1",
        "Id": 1,
        "TxTopics": [
          {
            "Name": "Topic1"
          }
        ]
      },
      {
        "REF": "Default",
        "Name": " recv_node_z1_5",
        "Zone": "Z1",
        "Id": 2,
        "RxTopics": [
          {
            "Name": "Topic2"
          }
        ]
      },
      {
        "REF": "Default",
        "Name": " recv_node_z1_6",
        "Id": 3,
        "TxTopics": [
          {
            "Name": "Topic2"
          }
        ]
      }
    ],
    "Zones": [
      "///<": "与z0配置相同,参见Z0"
    ],
    "MaterialRange": {
      "///<": "参见rcm插件配置"
    }
  }
}

```

4.22. 数据库驱动库 db_driver 模块

db_driver 是 UFTMDB 连接物理数据库的驱动库，主要实现数据上场(数据加载)和数据下场(数据回库)功能，提供数据库连接及数据上场、回库统一接口，数据回库支持多种模式(全表回库、快速回库等)，隐藏不同数据库类型的增删改查 SQL 语句实现细节。

库名	支持平台	插件标识	说明	版本黑白名单
----	------	------	----	--------

库名	支持平台	插件标识	说明	版本黑白名单
libudbdb_client_mysql_oracle10.so		libudbdb_client_mysql_oracle10.so	mysql及oracle10版本数据库驱动库	
libudbdb_client_mysql_oracle11.so	linux.x64	libudbdb_client_mysql_oracle11.so	mysql及oracle11版本数据库驱动库	
libudbdb_client_mysql_oracle12.so	linux.x64	libudbdb_client_mysql_oracle12.so	mysql及oracle12版本数据库驱动库	
libudbdb_client_mysql_oracle19.so	linux.x64	libudbdb_client_mysql_oracle19.so	mysql及oracle19版本数据库驱动库	
libudbdb_client_oracle10.so	linux.x64	libudbdb_client_oracle10.so	oracle10版本数据库驱动库	
libudbdb_client_oracle11.so	linux.x64	libudbdb_client_oracle11.so	oracle11版本数据库驱动库	
libudbdb_client_oracle12.so	linux.x64	libudbdb_client_oracle12.so	oracle12版本数据库驱动库	
libudbdb_client_oracle19.so	linux.x64	libudbdb_client_oracle19.so	oracle19版本数据库驱动库	
libudbdb_client_mysql.so	linux.x64、 aarch64	libudbdb_client_mysql.so	mysql数据库驱动库	白名单: libmysqlclient.so.18(5.5.68~6.1.11); 黑名单: libmysqlclient.so.18(6.1.2)
libudbdb_client_ob_mysql.so	linux.x64、 aarch64	libudbdb_client_ob_mysql.so	ob_mysql数据库驱动库 (obclient20.1.13)	白名单: libmysqlclient.so.18(5.5.68~6.1.11); 黑名单: libmysqlclient.so.18(6.1.2)
libudbdb_client_ob_oracle.so	linux.x64、 aarch64	libudbdb_client_ob_oracle.so	ob_oracle数据库驱动库 (obclient20.1.13)	
libudbdb_client_ob_mysql2_2_8.so	linux.x64、 aarch64	libudbdb_client_ob_mysql2_2_8.so	ob_mysql数据库驱动库(支持obclient2.2.x(x≥8)之后版本)	
libudbdb_client_ob_oracle2_2_8.so	linux.x64、 aarch64	libudbdb_client_ob_oracle2_2_8.so	ob_oracle数据库驱动库(支持obclient2.2.x(x≥8)之后版本)	

4.22.1. db_driver-dbcient 模块配置示例(详情见4.3.5 MDB数据组件配置项说明)

```
"ldp_bizproc.BizDataConfig.DbClient":
{
  "ConnectTimeOutSec": 5,
  "///": "数据库连接超时时间",
  "DatabaseConnectionMode": 0,
  "///": "数据库连接模式，0-长连接-默认，1-短连接",
  "Libs": [
    {
      "Name": "libudbdb_client_ob_oracle2_2_8.so",
      "DbType": "ob_oracle"
    }
  ],
  "Dss": [
    {
      "LogicName": "usersvr",
      "DbType": "ob_oracle",
      "Server": "oboraclehost.com",
      "Port": "3306",
      "UserName": "ldpsecupoc@UF300_38#obtest",
      "PassWord": "hundsun",
      "DataBase": "LDPSECUPOC",
      "ReadTimeOutSec": 60,
      "WriteTimeOutSec": 60,
      "MyCnfName": "my.cnf",
      "ObIbConfigFile": "uft30_ob_ib.cfg"
    }
  ]
}
```

4.22.2. db_driver-dbcient 配置说明详情见4.3.5 MDB数据组件配置项说明

名称	类型	说明	备注
Name	字符串	数据库驱动库的so名称	数据库驱动库的so名，例如：so的名称为libudbdb_client_ob_oracle2_2_8.so，则配置libudbdb_client_ob_oracle2_2_8.so
DbType	字符串	数据库库类型	mysql或者oracle或者ob_oracle或者mysql_oracle

4.22.3. db_driver-dbcient(ob_mysql、ob_oracle)DB驱动库支持ob负载均衡功能配置项示例

```
(DESCRIPTION=
(OBLB = ON)
(OBLB_RETRY_ALL_DOWNS = 20)
(OBLB_RETRY_TIMEOUT = 30000)
(OBLB_GROUP_STRATEGY = ROTATION)
(OBLB_BLACKLIST=
(REMOVE_STRATEGY =
(NAME = timeout)
(TIMEOUT = 5000))
(APPEND_STRATEGY =
(NAME = retryDuration)
(RETRY_TIMES = 1)
(DURATION = 10000)))
(ADDRESS_LIST =
(OBLB_STRATEGY = RANDOM)
(ADDRESS = (PROTOCOL = tcp) (HOST = 10.19.37.208) (PORT = 2883))
(ADDRESS = (PROTOCOL = tcp) (HOST = 10.20.162.100) (PORT = 2883))
(ADDRESS = (PROTOCOL = tcp) (HOST = 10.19.37.46) (PORT = 2883))
(ADDRESS = (PROTOCOL = tcp) (HOST = oboraclehost.com) (PORT = 3306))
(ADDRESS = (PROTOCOL = tcp) (HOST = 100.88.109.130) (PORT = 38884))))
```

4.22.4. db_driver-dbclient(ob_mysql、ob_oracle)DB驱动库支持ob负载均衡功能配置项说明

ob负载均衡功能配置以字符串格式组织，字符串中允许有空格、\n、\t，不允许有其他特殊字符。

名称	类型	说明	取值	备注
DESCRIPTION	字符串	包含连接的描述信息	DESCRIPTION=(),括号"()"内是该连接的相关描述	无
OBLB	字符串	负载均衡总开关	ON:开启 OFF:关闭(默认值)	如果开关为关闭的状态，所有 OBLB 开头的配置信息都无效，并且会选择第一个地址直接连接后续 proxy/observer。
OBLB_RETRY_ALL_DOWNS	字符串	所有HOST尝试连接的总次数	默认值为 120，建议重试次数大于等于配置的地址数	在负载均衡过程中，每次后端尝试连接都会使用一次连接次数，该参数定义连接总次数。如果设置为 10 次，OB会根据负载均衡策略，尝试与后端连接10次（每次地址都是靠负载均衡策略选择），直到次数用完或者连接上后端。
OBLB_RETRY_TIMEOUT	字符串	连接超时总时间	默认值：10000，单位：ms，建议总的超时时间大于（重试次数 * 连接超时时间）	在负载均衡过程中，可能会不断重试，尝试连接上后端 proxy，该配置用于设置总时间，每次重试之前进行检查，如果超时，则报错。
OBLB_GROUP_STRATEGY	字符串	负载均衡组间策略	ROTATION(该值是默认值，同时是当前唯一可选值)	OB负载均衡，允许配置多个proxy组，这个策略能够首先在多个组中选出策略倾向的组，尝试连接。
REMOVE_STRATEGY	字符串	洗白策略	使用 NAME 来标识洗白策略，并带上洗白策略相关的属性。	无
NAME	字符串	洗白和拉黑策略的名称	根据策略选择策略名， TIMEOUT-洗白策略， RetryDuration-拉黑策略， NORMAL-拉黑策略。	无
TIMEOUT	字符串	TIMEOUT 洗白策略下属性，表示洗白的时间	默认50，单位为 ms。	无
APPEND_STRATEGY	字符串	拉黑策略	使用 NAME 来表示拉黑策略，并带上相应拉黑策略的属性。	无
RETRY_TIMES	字符串	RetryDuration 策略下的属性，表示在 DURATION 时间内拉黑需要的失败次数，即在 DURATION 时间内重试多少次会被加入黑名单	数字，默认值为 1。	需要和DURATION配合使用
DURATION	字符串	RetryDuration 策略下的属性，RETRYTIMES后如果时间超过了该字段配置的值，	时长，默认值为 10ms。	需要和 RETRY_TIMES配合使用

		则该HOST将会被加入到黑名单，建议大于等于连接超时时间		
ADDRESS_LIST	字符串	DESCRIPTION 下的配置信息，可以是一组 ADDRESS 配置信息，也可以配置自己独立的 OBLB_STRATEGY。各个组之间按照 OBLB_GROUP_STRATEGY 来进行选择。	ADDRESS_LIST=(), 括号“()”内是该组 ADDRESS 的描述信息。	无
OBLB_STRATEGY	字符串	表示地址组的负载均衡策略	RANDOM: 随机选择地址连接(默认策略) ROTATION: 轮询，按配置顺序尝试 SERVERAFFINITY: 加权随机，配合每个 ADDRESS 的 WEIGHT 属性，达到加权随机效果	无
ADDRESS	字符串	ADDRESS_LIST 下的配置信息，可以配置 Host 相关的信息，包括 <code>hostname</code> 、 <code>port</code> 和权重等。	ADDRESS=(), 括号“()”内是该 ADDRESS 的描述信息。	无
PROTOCOL	字符串	表示协议，继承自 ORACLE TNS_FILE，没有实际意义	tcp或udp	无
HOST	字符串	IP 地址或域名	后端 proxy 或 server 的地址	无
PORT	字符串	端口号	数字	无

本配置文件新增内容路径及文件名由核心OblbConfigFile配置决定，在OblbConfigFile配置的路径中解析相关配置文件内容。连接超时时间通过核心数据组件原有配置ConnectTimeOutSec配置项进行设置，支持负载均衡后，配置范围建议为1-30秒，配置太大如果负载均衡的地址列表中有无法连接的地址，可能会导致需要等待比较长的时间成功连接上数据库。数据库连接满足尝试连接的总次数或者连接超时总时间后，报错退出时，错误信息中会携带对应的报错退出信息：达到重试次数(retry_all_downs is zero)，达到超时退出时间(retry_timeout)。注意，本次给出ob负载均衡配置示例为已自测通过的推荐配置，并非ob负载均衡完整配置。ob支持负载均衡完整配置说明详情见url：<https://www.oceanbase.com/docs/enterprise-obci-cn-1000000000924913>

4.22.5. dbclient my.cnf配置新增配置示例

```
[client]
default-character-set=utf8
init-command="set collation_connection=utf8_general_ci"
init-command="set names utf8"
```

4.22.6. dbclient my.cnf配置新增配置项说明

名称	类型	说明	备注
default-character-set	字符串	设置数据库默认字符集，当前设置为UTF-8	无
init-command	字符串	在 MySQL 中，可以通过设置 <code>init-command</code> 来确保每次连接时都执行特定的命令，如设置字符集或时间区	无

4.23. 重演工具插件 ldp_replay

ldp_replay是一个LDP插件, 重演工具的主要功能是通过记录生产环境中的请求和应答数据，并在测试环境中进行全量模拟，保持时序和线程的一致性，从而实现对软件升级前的仿真验证。

插件名	支持平台	插件标识	说明
libldp_replay.so	linux.x64	ldp_replay	重演工具

4.23.1. 示例配置

```
{
  "Lib": "ldp_replay",
  "Instance": 0,
  "Timer": 0,
  "BaseAppConfig":
  [
    {
      "AppName": "uconvert64",
      "NodeNo":55,
      "SendMsgFilter":
      [
        {
          "FilterFuncId":"1;2;77?";
        }
      ],
      "RspCompare":
      {
        "FunctionIdBlackList": "1;2;77?";,
        "FunctionIdWhiteArray":
        [
          {
            "FunctionIdsList": "789;23?";,
            "FieldsBlackArray": [
              "fund_account",
              "clinet_id"
            ],
            "FieldsWhiteArray":[
              "fund_account",
              "clinet_id"
            ]
          }
        ]
      },
      "SpecialFuncInfo":
      [
        {
          "Type": "OnStage",
          "FunctionId": "960600",
          "TimeOutSec": 60
        },
        {
          "Type": "OnStageReady",
          "FunctionId": "960006",
          "TimeOutSec": 60
        }
      ]
    }
  ],
  "Dss":
  {
    "LogicName": "usersvr",
    "Server": "192.168.86.102",
    "Port": "123",
    "UserName": "admin",
    "PassWord":"admin",
    "DbType": "ob_oracle"
  },
  "PrepareDisabled":false,
  "TableBackup":
  [
    "price",
    "fund_account"
  ],
  "Remoter":
  {
    "AutoStart": false,
    "Routes":
    [
      {
        "Functions": "5;102?";,
        "Systems" : "2",
        "Backend": 1
      }
    ],
    "RspTopic":
    {
      "Name": "replay_rsp",
      "EnableUnicastMode": false
    },
    "Backends" :
    [
      {
        "Id": 1,
        "Type": "AS-RCM",
        "TxTopic": "trade",
        "EnableMutiShard": "true",
        "EnableQueryFisrt": false
      }
    ]
  }
}
```

4.23.2. 配置项说明

名称	类型	说明	备注
Timer	整型	定时事件发生频率	单位秒, 非必须。目前前置无法响应框架的OnTimer线程, 前置有自己的OnTimer线程。
BaseAppConfig	数组	重演工具基础配置	
AppName	字符串	应用名	
NodeNo	整型	分片号	每个应用是不一致的, 不填写启动失败
SendMsgFilter	数组	发送消息过滤配置	不支持动态修改
FilterFuncId	字符串	过滤的功能号集合	英文分号分隔, 如"11;30?;319;"; 支持"*";支持模糊匹配, ?号表示一个数字。若不配置, 则代表所有请求都发送

RspCompare	数组	应答比较过滤配置	不支持动态修改
FunctionIdBlackList	字符串	应答比较过滤的功能号集合	英文分号分隔, 如“11;30;319;”, 支持“*”,支持模糊匹配, ?号表示一个数字。若不配置, 则代表所有应答都比较
FunctionIdWhiteArray	数组	需要对比的功能号数组以及字段信息	当同时存在功能号比对黑白名单时, 先考虑是否存在黑名单中, 如果黑名单中不存在, 则必须比; 再查看比对哪些字段
FunctionIdsList	字符串	需要对比的功能号列表	英文分号分隔, 如“11;30;319;”, 支持“*”,支持模糊匹配, ?号表示一个数字。若不配置, 则代表所有字段都比较
FieldsBlackArray	数组	FunctionIdsList中的功能号应答中不需要比的字段	字段黑名单优先于白名单。黑名单配置了, 白名单就失效了
FieldsWhiteArray	数组	FunctionIdsList中的功能号应答中需要比的字段	字段黑名单优先于白名单。黑名单配置了, 白名单就失效了
SpecialFuncInfo	数组	特殊功能号信息	
Type	字符串	特殊功能号类型。	支持类型: OnStage: 上场 OnStageReady: 设置业务就绪 OffStage: 下场 Serializable: 串行化执行 WaitResponseWhiteList: 应答等待白名单 WaitResponseBlackList: 应答等待黑名单 WaitResponseWhiteList和WaitResponseBlackList不支持同时配置
FunctionId	字符串	功能号	上场、设置业务就绪和下场仅支持功能号精准匹配 串行化执行仅支持配置*, 配置后, 所有消息按序发送 WaitResponseWhiteList和WaitResponseBlackList支持英文分号分隔, 支持*, 支持模糊匹配, ?号表示一个数字, 如“960006;960600;96????;”
TimeOutSec	整型	超时时间	该功能号的同步等待超时时间, 单位秒, 默认值为60秒, Serializable类型时TimeOutSec配置无效
Dss	对象	数据库信息配置	
LogicName	字符串	登录名	
Server	字符串	数据库的服务端地址	
Port	字符串	数据库端口	
UserName	字符串	用户名	
PassWord	字符串	密码	
DbType	字符串	数据库类型	仅支持“ob_oracle”
TableBackup	数组	日初需要持久化的默认表名	日初管理功能配置了TableBackup则默认配置失效
Remoter	对象	路由配置	详细的使用说明参考《低延时分布式开发平台_使用指南》remoter插件配置说明
PrepareDisabled	布尔型	模式开关	用于控制重演工具启动模式, 该开关开启后, 不会进行数据拷贝, 数据恢复等相关逻辑。

4.23.3. 重演工具主流程

- 1、调用日初管理功能PrepareReplay备份日初数据库信息
- 2、调用日终管理功能PrepareReplay备份日终数据库信息和核心持久化数据
- 3、调用StartReplay开始重演
- 4、重演期间可以调用GetReplayProgress查看重演进度
- 5、重演结束调用GetReplayResult查看重演结果

4.24. 统一数据库访问 ldp_dbclient

ldp_dbclient是一个LDP插件, 封装了各个数据库的驱动, 提供了统一的API来执行SQL。类似JDBC、ODBC。

插件名	支持平台	插件标识	说明
libldp_dbclient.so	linux.x64	ldp_dbclient	统一数据库访问插件

4.24.1. 统一SQL使用说明

若要使用SQL转换功能需要配统一SQL

- 通过<https://www.light-pg.com/downloadCate.html> 下载统一SQL动态库。
- 统一SQL的使用说明如下: <https://www.light-pg.com/docs/LTSQL/current/index.html>

ldp_dbclient 使用SQL转换功能的步骤

- 把动态库目录设置(libunisl.so 所在目录)在环境变量LD_LIBRARY_PATH中。
- 把配置文件放到工作目录下，结构为config/unisl.conf， 具体配置请参考<https://www.light-pg.com/docs/LTSQL/current/index.html>

4.24.2. libobclnt.so 下载

通过 <https://en.oceanbase.com/softwarecenter> 下载 so。

4.24.3. ldp_dbclient插件配置示例

```
{
  "Lib": "ldp_dbclient",
  "Instance": 0,
  "//3": "空闲连接清理周期, 单位分钟, 在空闲连接数超过回收阈值 (RecycleConnectionCountThreshold) 时会尝试清理, 默认-1 不清理",
  "ConnFreeTimeMin": 1,
  "Dss": [
    {
      "//4": "数据源名字 必须有 不能空串 该名字作为IDBClient::GetConnection的第一个入参, 无长度限制, 配置错误会有一条错误日志",
      "LogicName": "cbtest",
      "//5": "数据源类型名 必须有 不能空串, 当前有uftmdb/ob_oracle/uftmdb_embed/goldendb, 无长度限制, 配置错误会有一条错误日志",
      "DriverName": "uftmdb",
      "//6": "用户名, 必须有, 不能为空串, 通常是apm上给mdb设置的用户名, 信创DB为数据库的用户名",
      "UserName": "test",
      "//7": "密码, 必须有, 不能为空串, 通常是apm上给mdb设置的密码 通常是密文的base64编码形式, 信创DB为数据库的密码",
      "PassWord": "123456",
      "//8": "服务器IP:端口, 不支持配置多个, 未配置或配置为空串返回错误, 记录错误日志",
      "Server": "192.168.86.102:11191",
      "//9": "uftmdb请求的目的数据库名称, 形式应用名#编号, 未配置或配置为空串返回错误, 记录错误日志; 其他信创DB为数据库名称",
      "DataBase": "mdb#0",
      "//10": "当前数据源是否启用连接池, 默认关闭",
      "EnableConnectionPool": false,
      "//11": "连接池初始连接数, 默认为10, 不能大于MaxConnectionCount, 若大于会报错",
      "InitConnectionCount": 10,
      "//12": "连接池回收连接的连接数阈值, 连接数达到该值开始回收连接, 默认为MaxConnectionCount/2",
      "RecycleConnectionCountThreshold": 50,
      "//13": "连接池连接数量上限, 默认值100",
      "MaxConnectionCount": 100,
      "//14": "连接池已达到最大连接数时, 且无空闲连接时, 从连接池获取连接的等待超时时间, 默认-1一直等待",
      "ConnPoolWaitTimeOutMill": -1,
      "//15": "本数据源最大连接数, 用于非连接池情况,  unlimited连接池连接数, 默认 unlimited",
      "MaxConnections": 100,
      "//16": "连接超时时间 单位毫秒 必须大于1000, 默认由操作系统控制, 对于mdb实际为除以1000后乘以3 在此时间内连接上无任何数据 (包括底层自动的心跳数据) 会断连 不配或者配错时 由底层ldptcpsdk决定",
      "ConnectTimeoutMill": 3000,
      "//17": "查询的超时时间 单位毫秒 必须大于10 表示发起查询请求后 在此时间内必须收到应答数据 不是接收完毕 而是收到 否则断连 不配或者配错时默认1分钟",
      "QueryTimeoutMill": 60000,
      "//18": "数据库读超时时间, 单位秒, 默认0, 取决于服务端配置",
      "ReadTimeoutSec": 0,
      "//19": "数据库写超时时间, 单位秒, 默认0, 取决于服务端配置",
      "WriteTimeoutSec": 0,
      "//20": "统一sql转换源数据库类型, 目前为oracle 或 mysql, 默认oracle",
      "UnislSourceDbType": "oracle",
      "//21": "数据库的配置文件名, 默认为空, 表示没有, 对于mysql可以配置my.cnf",
      "DataBaseConfName": "",
      "//22": "数据库客户端的字符集 (包括发送和接受), 可为gbk, utf8 或空, 默认为空, 表示由驱动确定 (一般根据当前系统设置)",
      "CharsetName": "",
      "//23": "是否启用数据, 压缩客户端和服务端之间发送的所有信息, 默认关闭, 只对某些数据库生效, 开启后能压缩则压缩",
      "Compress": false,
      "//24": "驱动库所在路径, 不配置需要能从环境变量获取",
      "LibPath": "",
      "//25": "驱动的动态库名, 若配置则使用配置的动态库名, 若不配则使用默认的动态库名",
      "LibName": "",
      "//26": "开启自动重连, 默认关闭, 开启后在执行sql前若连接已断开则会自动重连, 只有自动提交模式会被保留, 其他都被重置",
      "EnableAutoReconnect": false,
      "//27": "开启自动提交, 物理db默认非自动提交 (ob_oracle, goldendb), uftmdb只支持自动提交",
      "EnableAutoCommit": false,
      "//28": "调用API接口时, 输入输出的表名, 字段名都为小写, 默认关闭, 开启时需要保证表名, 字段名不会出现大小写混用情况, 都为大写, 或都为小写",
      "LowerCaseNames": false,
      "//29": "心跳间隔, 心跳1秒触发一次, 间隔HeartbeatInterval发送心跳, 默认为0 表示不发心跳 (0, int32_max)",
      "HeartbeatInterval": 0,
      "//x": "数据库驱动配置个性化参数统一放入到args里面配置, 不同数据库类型有不同的配置参数",
      "Args": {
        "//30": "远程mdb独有, 分片号, 不配置默认0, 由请求包目的地的服务器分片号配置决定, 需要确认是各个连接统一还是不同连接数据会传到不同的分片",
        "ShardNo": 0,
        "//31": "SQL功能号可配置, 不配默认使用508902, 配了就用配置值",
        "SqlFunction": 508902,
        "//32": "ldptcpsdk 配置文件, 不配默认ldptcpsdk.ini",
        "DriverConfigFile": "ldptcpsdk.ini",
      }
    },
    {
      "LogicName": "usesbid62#0",
      "DriverName": "uftmdb",
      "UserName": "xxxxxx",
      "PassWord": "xxxxxx",
      "Server": "ip:port",
      "DataBase": "usesbid62#0",
      "Args": {
        "ShardNo": 0,
        "SqlFunction": 508902
      }
    },
    {
      "LogicName": "usesbid62#1",
      "DriverName": "uftmdb_embed"
    },
    {
      "LogicName": "usesbid62#2",
      "DriverName": "ob_oracle",
      "UserName": "xxxxxx",
      "PassWord": "xxxxxx",
      "Server": "ip:port",
      "DataBase": "usesbid62#0"
    },
    {
      "LogicName": "usesbid62#3",
      "DriverName": "ob_oracle",
      "UserName": "xxxxxx",
      "PassWord": "xxxxxx",
      "Server": "ip:port",
      "DataBase": "usesbid62#0"
    }
  ]
}
```

```
        "DriverName": "ob_oracle",
        "UserName": "xxxxx",
        "PassWord": "xxxxx",
        "Server": "ip:port",
        "DataBase": "usesbid62#3",
        "Args":
        {
            "//33": "ob_oracle软负载连接配置文件名称",
            "ObIbConfigFile": "ob_lb.cfg"
        }
    },
    {
        "LogicName": "usesbid62#4",
        "DriverName": "goldendb",
        "UserName": "xxxxx",
        "PassWord": "xxxxx",
        "Server": "ip:port",
        "DataBase": "usesbid62#3",
        "//37": "绑定执行中，字段分配的最大内存，单位为KB，不配置默认为64KB，配置范围为1~65536 KB，即1KB~64M，当前只支持goldendb
数据库"，
        "MaxFieldLengthKB": 64,
        "Args":
        {
            "//34": "是否开启goldendb负载均衡模式连接，不配置默认为false，配置为true表示开启负载均衡连接，该功能使用依赖于goldendb
数据库客户端驱动版本，ZXCLUD-GoldenDB-ALL-Connector-C_V1.0.3及以上",
            "GoldenDBIbMode": false,
            "//35": "在开启负载均衡模式后的连接串信息，开启负载均衡后，必须填写该配置。形式为ip1:port,ip2:port..., 每组ip:port之间
采用英文逗号分割，如127.0.0.1:1234,127.0.0.2:2345",
            "GoldenDBIbIpList": "127.0.0.1:1234,127.0.0.2:2345",
            "//36": "使用0：随机策略；使用1：轮询策略；使用2：主备策略",
            "GoldenDBStrategy": 0
        }
    }
}
}
```

4.24.4. ldp_dbclient插件配置说明

ldp_dbclient 通用配置说明

名称	类型	说明	备注	开始支持版本
ConnFreeTimeMin	整型	空闲连接多久进行清理	在空闲连接数超过回收阈值（RecycleConnectionCountThreshold）时会尝试清理,默认-1，小于1不清理	V1.0.15.7
Dss	数组	配置数据源		V1.0.15.7

数据源通用配置说明

名称	类型	说明	备注	开始支持版本
LogicName	字符串	数据源名字	不能为空，用于IDBClient::GetConnection，配置错误会有一条错误日志	V1.0.15.7
DriverName	字符串	数据源驱动类型名	不能为空，可选值为ufimdb、ob_oracle、ufimdb_embed、goldendb。配置错误会有一条错误日志	V1.0.15.7
UserName	字符串	用户名	通常是apm上给mdb设置的用户名，信创DB为数据库的用户名	V1.0.15.7
PassWord	字符串	密码	通常是apm上给mdb设置的密码 通常是密文的base64编码形式，信创DB为数据库的密码	V1.0.15.7
Server	字符串	服务器连接串	格式由驱动确定。+ob_oracle/goldendb/ufimdb:ipport，未配置或空字符串将返回错误，记录错误日志 +ufimdb_embed:不需要配置	V1.0.15.7
DataBase	字符串	目标数据库名称	ufimdb使用应用名#编号格式，信创DB为数据库名称	V1.0.15.7
EnableConnectionPool	布尔	是否启用连接池	默认关闭	V1.0.15.7
InitConnectionCount	整数	连接池初始连接数	默认值为10，不能大于MaxConnectionCount，若大于会报错	V1.0.15.7
RecycleConnectionCountThreshold	整数	连接池回收连接数的阈值	连接数达到该值开始回收连接，默认值为MaxConnectionCount 的一半，MaxConnectionCount，若大于会报错	V1.0.15.7
MaxConnectionCount	整数	连接池连接数量上限	默认值为100	V1.0.15.7
ConnPoolWaitTimeOutMill	整数	获取连接的等待超时时间	单位毫秒，默认值为-1表示一直等待	V1.2501.0.1
MaxConnections	整数	最大连接数（旧配置）	默认不限。连接池下效果同MaxConnectionCount，若同时设置，必须一致，不然报错。不设置，连接池下使用MaxConnectionCount（默认值或设置值），建议使用MaxConnectionCount。非连接池，使用此配置，限制同时获取的最大连接数。（0，int32_max）	V1.0.15.7
HeartbeatInterval	整数	心跳间隔	心跳1秒触发一次，间隔HeartbeatInterval此发送心跳，默认为0 表示不发心跳	V1.0.15.7

名称	类型	说明	备注 (0~int32_max)	开始支持版本
ConnectTimeoutMill	整数	连接超时时间，单位毫秒	单位毫秒，必须大于1000。对于mdb实际为除以1000后乘以3,在此时间内连接上无任何数据(包括底层自动的心跳数据)会断连，不配或者配错时由底层kltcpsdk决定。对于信创数据库默认由操作系统控制，对 ob_oracle goldendb 实际为除以1000	V1.0.15.7
QueryTimeoutMill	整数	查询超时时间	单位毫秒，必须大于10，表示发起查询请求后 在此时间内必须收到应答数据表示接收完毕，而是收到，否则断连。对于MDB不配或者配错时默认1分钟。对于信创数据库由服务端配置。	V1.0.15.7
ReadTimeoutSec	整数	数据库读超时时间	单位秒，默认值为0，由服务端配置决定，原db_driver的read_timeout	V1.0.15.7
WriteTimeoutSec	整数	数据库写超时时间	单位秒，默认值为0，由服务端配置决定。原db_driver为write_timeout	V1.0.15.7
UnisqlSourceDbType	字符串	SQL转换源数据库类型	默认值为oracle，可为oracle或mysql	V1.0.15.7
DataBaseConfName	字符串	数据库配置文件名	比如对于mysql可以配置my.cnf，默认为空,表示不使用。原db_driver为mycnf	V1.0.15.7
CharsetName	字符串	数据库客户端字符集（包括发送和接受）	可为gbk，utf8或留空（由驱动决定）。默认为空，原db_driver为charset。对 ob_oracle 若配置文件和这个参数都进行了配置，已配置文件为准	V1.0.15.7
Compress	布尔	是否启用数据压缩	压缩客户端和服务端之间发送的所有信息，默认关闭，只对某些数据库起效，开启后能压缩则压缩。原db_driver为compress	V1.0.15.7
LibPath	字符串	驱动库所在路径	默认为空，从环境变量 获取	V1.0.15.7
LibName	字符串	驱动的动态库名	若配置则使用该值，未配置则使用默认动态库名	V1.0.15.7
EnableAutoReconnect	布尔	是否启用自动重连	默认关闭，开启后在执行sql前若连接已断开则会自动重连	V1.0.15.7
EnableAutoCommit	布尔	是否启用自动提交	物理db默认非自动提交(ob_oracle、goldendb), utfmdb只支持自动提交.	V1.0.15.13
LowerCaseNames	布尔	是否使用小写	开启时，需要数据库表名，字段名不会出现大小写混用情况。对于GetXXXByName需要传入小写字段名而不考虑数据库时间大小写；通过元数据获取到的字段名都为小写；元数据的GetTables GetColumns 入参大小写不敏感，内部会转为大写或小写处理。	V1.0.15.7
ShardNo	整数	分片号（远程mdb独有）	备注参见下面的远程mdb个性配置, 加在这是为了兼容老版本	V1.0.15.7
SqlFunction	整数	SQL功能号	备注参见下面的远程mdb个性配置, 加在这是为了兼容老版本	V1.0.15.7
MaxFieldLengthKB	整数	绑定执行中字段分配内存的最大值	在绑定执行中，字段分配的最大内存，单位为KB，不配置默认为64KB，配置范围为1~65536 KB，即1KB~64M，当前只支持goldendb数据库	V1.0.15.13
Args	数组	数据源个性配置数组		V1.0.15.7

数据源个性配置说明

ufmdb(远程mdb)

名称	类型	说明	备注	开始支持版本
ShardNo	整数	分片号（远程mdb独有）	不配置默认0，由请求包目的地的服务器分片号配置决定，需要确认是各个连接统一还是不同连接数据会传到不同的分片	V1.0.15.7
SqlFunction	整数	SQL功能号	不配默认使用508902，配了就用配置值	V1.0.15.7
DriverConfigFile	字符串	LdpTcpSdk配置文件	没有该配置项时 默认值为kltcpsdk.ini 当该文件存在且可读时 会载入 其内容格式参考《低延时分布式开发平台_客户端SDK使用指南.pdf》值	V1.0.15.7

ob_oracle

名称	类型	说明	备注	开始支持版本
OblbConfigFile	字符串	obproxy负载均衡的配置文件，默认为空	ob_oracle支持该功能的客户端驱动版本建议在2.2.8以上，为ob工程师推荐的版本。	V1.0.15.9

名称	类型	说明	备注	开始支持版本
ldp_dbclient	插件	my.cnf配置新增配置示例		
<pre>[client] default-character-set=utf8 init-command="set collation_connection=utf8_general_ci" init-command="set names utf8"</pre>				

ldp_dbclient插件my.cnf配置新增配置项说明

名称	类型	说明	备注
default-character-set	字符串	设置数据库默认字符集，当前设置为UTF-8	无
init-command	字符串	在 MySQL 中，可以通过设置 init-command 来确保每次连接时都执行特定的命令，如设置字符集或时间区	无

goldendb

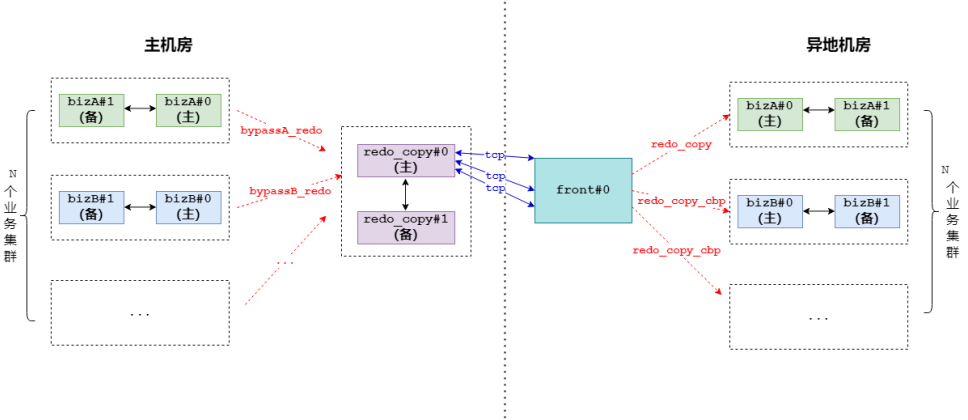
名称	类型	说明	备注	开始支持版本
GoldenDBlbMode	布尔型	是否开启 goldendb 负载均衡模式连接	不配置默认为false, 配置为true表示开启负载均衡连接, 该功能使用依赖于goldendb数据库客户端驱动版本, ZXCLLOUD-GoldenDB-ALL-Connector-C_V1.0.3(buildNumber:ZXCLLOUD-GOLDENDB-ALL-Connector-C.r59f0314)及以上	V1.0.15.13
GoldenDBlbIpList	字符串	在开启负载均衡模式后的连接串信息	开启负载均衡后, 必须填写该配置。形式为 ip1:port,ip2:port..., 每组ip:port之间采用英文逗号分割, 如 127.0.0.1:1234,127.0.0.2:2345	V1.0.15.13
GoldenDBStrategy	整型	goldendb 负载均衡策略	使用0: 随机策略, 会在1到4中, 产生随机数, 随机到2, 就会对CN2尝试建链, 随机到4就会对CN4尝试建链; 使用1: 轮询策略, 如果是第一次, 会选择1, CN1 尝试建链; 如果第二次建链, 会选择CN2尝试建链; 依次类推, 如果是第5次, 会重新从第一个, CN1开始建链; 另外, 如果第一次CN1建链失败, 会自动后移尝试CN2; 然后第二次建链, 会变成尝试CN3。简单来说, 轮询策略, 会每次从上次建链结束的下一个CN开始建链。使用2: 主备策略, 每次来建链, 都是优先级第一个, 如果第一个失败了, 会进行第二个, 第三个, 直到成功或者全部CN尝试结束报错(该说明由goldendb官方提供, 具体详见其官方说明), 不配置默认传0	V1.0.15.13

4.25. redo同步代理插件 ldp_redo_copy

ldp_redo_copy是一个redo消息同步代理插件，用于与异地机房核心之间redo消息同步。

插件名	支持平台	插件标识	说明
libldp_redo_copy.so	linux.x64、aarch64	ldp_redo_copy	redo同步代理插件

4.25.1. ldp_redo_copy 在系统中的位置及数据流向



4.25.2. 配置项说明

```
{
  "Lib": "ldp_redo_copy",
  "Instance": 0,
  "Timer": 0,
  "Transmitter": {
    "//00": "本地网卡的网络号，必填，支持网段/IP地址配置，不能填回环地址",
    "LocalNetworkNumber": "192.168.86.0",
    "//01": "发送端配置，数组，可以配置多个发送端，至少配置一个",
    "Transmitters": [
      {
        "//1": "功能号，目前可以配置的功能号为417和418，默认值为417",
        "FunctionID": 418,
        "//2": "缓存消息最大个数，默认为2048，合法区间为[64, 65536]",
        "MaxCacheMessageCount": 128,
        "//3": "心跳间隔时间，单位为毫秒，默认为1秒，合法区间为[50, 10000]",
        "HeartbeatIntervalMilli": 1000,
        "//4": "可以触发直接发送的最小消息长度，单位为字节，默认值为1500，合法区间为[1024, 65536]",
        "DirectSendLength": 1024,
        "//5": "压缩功能配置",
        "Compress": {
          "//6": "压缩算法，目前LDPMMSG支持ZSTD和LZ4压缩算法",
          "AlgoType": "ZSTD",
          "//7": "压缩阈值，消息内容长度超过时开启压缩",
          "Threshold": 1000
        },
        "//8": "remoter实例配置",
        "LdpRemoterInstance": {
          "//9": "沿用remoter插件中的backend含义，目前只支持TCP客户端",
          "Backend": {
            "//10": "Sites为数组，至少要配置一个，可以配置多个ip和端口",
            "Sites": [
              {
                "Host": "192.168.86.101",
                "Port": 11200
              },
              {
                "Host": "192.168.86.102",
                "Port": 11200
              },
              {
                "Host": "192.168.86.20",
                "Port": 11200
              }
            ]
          },
          "//11": "沿用remoter插件中的 Frontend 含义，服务端接入端，目前只支持Rcm类型服务端",
          "Frontend": {
            "//12": "旁路接收主题，必填",
            "RxBypassTopic": {
              "//13": "主题名，必填",
              "Name": "bypass_redo",
              "//14": "分区号，必填，用于标识能接收哪个分区",
              "PartitionNo": 62
            }
          }
        }
      }
    ]
  }
}
```

4.25.3. ldp_redo_copy 插件实例配置参数说明

配置项	类型	含义	备注
Transmitter	对象		
LocalNetworkNumber	字符串	本地网卡网络号	
Transmitters	数组	各Transmitter的配置	
FunctionID	整型	功能号	目前可以配置的功能号为417和418,默认值为417；由于业务旁路生成的redo消息存在两种情况，一种是使用相同的主题名称但是分片号不同；第二种为分片号相同但是主题不同；为了确保异地机房前置也能识别redo_copy转发这一类场景的业务核心的redo同步消息，如果原始消息分片号不同，则可以直接通过分片号进行区分；否则，需要通过不同的功能号进行区分。
MaxCacheMessageCount	整型	消息缓存的最大个数，超过该数量，暂停新的旁路消息接收，待缓存空出来之后继续接收	默认为32768,合法区间为[64, 65536)
HeartbeatIntervalMilli	整型	心跳间隔时间	单位毫秒,默认为1000, [50, 10000]
DirectSendLength	整型	可以触发直接发送的最小消息长度	单位字节,默认值为1500
Compress	对象	压缩功能配置	不配置表示不压缩
AlgoType	字符串	压缩算法	目前LDPMMSG支持ZSTD和LZ4压缩算法，不配置表示不压缩
Threshold	整型	压缩阈值	消息内容长度超过时开启压缩

LdpRemoterInstance	对象	remoter实例配置	
Backend	对象	TCP客户端	Backend含义沿用通信服务框架插件kdp_remoter中的概念，详见:4.20节《通信服务框架插件kdp_remoter》
Sites	数组		Sites里可以配置多个ip和端口，至少需要配置一个；如果存在多个Site，数据发送选择固定使用一个正常的Site发送，当出现发送失败时，会自动切换到一个新的正常的Site继续发送。 注：最多配置8个Site
Host	字符串	主机IP	必填
Port	整型	主机侦听端口	必填
Frontend	数组	服务端接入端	Frontend含义沿用通信服务框架插件kdp_remoter中的概念，详见:4.20节《通信服务框架插件kdp_remoter》
RxBypassTopic	对象	旁路接收主题	必填
Name	字符串	主题名	必填
PartitionNo	整形	分区号	必填，用于标识能接收哪个分区
SendWindowSize	整型	发送窗口大小	当发送进度与收到ACK确认进度的差值超过设置值，将暂停发送。默认值：2048，合法区间为[1,65535]。 kdp_redo_copy V1.0.15.3+
FlowControl	对象	流控配置	可选，有该配置表示开启了流控功能。+ kdp_redo_copy V1.0.15.4+
LimitRateMbps	整形	流控速率	单位是Mbps，即每秒最多可通过的数据比特数，超过该速率限制后，会暂缓发送数据，必填。+ kdp_redo_copy V1.0.15.4+
CycleTimeMilli	整形	流控周期	单位为毫秒，取值范围1-1000，即最小流控周期为1毫秒，最大流控周期为1秒, 如果配置CycleTimeMilli为10, LimitRateMbps为100, 说明一个流控周期内最多可通过1Mb的数据；默认为100毫秒。+ kdp_redo_copy V1.0.15.4+

4.25.4. 配置说明

1. 核心配置中的ShardNo字段与其对应的Transmitter配置中的PartitionNos必须一致。

4.25.5. 版本约束

节点	依赖插件的版本
主机房核心	kdp_rcm V1.1.15.20+
redo_copy节点	kdp_remoter V1.0.15.54+, kdp_rcm V1.1.15.20+
备机房核心	kdp_bizproc V1.0.15.82+

4.25.6. 其他说明

1. 本插件采用LDP_MSG支持的数据压缩发送。对于小数据包，压缩率低情况下，压缩可能无法带来性能改善。因压缩功能本身会额外增加CPU负担，并写入压缩协议头，增加数据处理时间，同时小包压缩可能额外增大数据长度，降低整体同步性能，故须综合考虑是否启用压缩功能；

4.25.7. 示例配置

示例： ldp_redo_copy一主两从 实现跨机房redo消息传输，同一机房包含的所有业务节点和ldp_redo_copy代理节点，共用同一个zk上的同一份配置。

```
{
  "Applications": {},
  "REFS": {
    "redo_copy": {
      "Components": {
        "Libs": [],
        "GlobalLibs": []
      },
      "Plugins": [
        {
          "Lib": "ldp_redo_copy",
          "Instance": 0,
          "Timer": 0,
          "..."
        }
      ]
    }
  }
}
```

```
    "LocalNetworkNumber": "192.168.86.0",
    "Transmitters": [
      {
        "FunctionID": 417,
        "MaxCacheMessageCount": 128,
        "HeartbeatIntervalMilli": 1000,
        "DirectSendLength": 1024,
        "Compress": {
          "AlgoType": "ZSTD",
          "Threshold": 1000
        },
        "Backend": {
          "Sites": [
            {
              "Host": "192.168.86.101",
              "Port": 11200
            },
            {
              "Host": "192.168.86.102",
              "Port": 11200
            },
            {
              "Host": "192.168.86.20",
              "Port": 11200
            }
          ]
        },
        "Frontend": {
          "RxBypassTopic": {
            "Name": "bypass_redo",
            "PartitionNo": 62
          }
        }
      }
    ],
    {
      "FunctionID": 418,
      "MaxCacheMessageCount": 128,
      "HeartbeatIntervalMilli": 1000,
      "DirectSendLength": 1024,
      "Compress": {
        "AlgoType": "ZSTD",
        "Threshold": 1000
      },
      "Backend": {
        "Sites": [
          {
            "Host": "192.168.86.101",
            "Port": 11200
          },
          {
            "Host": "192.168.86.102",
            "Port": 11200
          },
          {
            "Host": "192.168.86.20",
            "Port": 11200
          }
        ]
      },
      "Frontend": {
        "RxBypassTopic": {
          "Name": "bypass_redo",
          "PartitionNo": 62
        }
      }
    }
  ],
  {
    "FunctionID": 417,
    "MaxCacheMessageCount": 128,
    "HeartbeatIntervalMilli": 1000,
    "DirectSendLength": 1024,
    "Compress": {
      "AlgoType": "ZSTD",
      "Threshold": 1000
    },
    "Backend": {
      "Sites": [
        {
          "Host": "192.168.86.101",
          "Port": 11200
        },
        {
          "Host": "192.168.86.102",
          "Port": 11200
        },
        {
          "Host": "192.168.86.20",
          "Port": 11200
        }
      ]
    },
    "Frontend": {
      "RxBypassTopic": {
        "Name": "bypass_redo",
        "PartitionNo": 62
      }
    }
  }
],
{
  "FunctionID": 417,
  "MaxCacheMessageCount": 128,
  "HeartbeatIntervalMilli": 1000,
  "DirectSendLength": 1024,
  "Compress": {
    "AlgoType": "ZSTD",
    "Threshold": 1000
  },
  "Backend": {
    "Sites": [
      {
        "Host": "192.168.86.101",
        "Port": 11200
      },
      {
        "Host": "192.168.86.102",
        "Port": 11200
      },
      {
        "Host": "192.168.86.20",
        "Port": 11200
      }
    ]
  },
  "Frontend": {
    "RxBypassTopic": {
      "Name": "bypass_redo",
      "PartitionNo": 62
    }
  }
}
```

```

        "//8": "remoter实例配置",
        "LdpRemoterInstance": {
            "//9": "沿用remoter插件中的backend含义，目前只支持TCP客户端",
            "Backend": {
                "//10": "Sites为数组，至少要配置一个，可以配置多个ip和端口",
                "Sites": [
                    {
                        "Host": "192.168.86.101",
                        "Port": 11200
                    },
                    {
                        "Host": "192.168.86.102",
                        "Port": 11200
                    },
                    {
                        "Host": "192.168.86.20",
                        "Port": 11200
                    }
                ]
            },
            "//11": "沿用remoter插件中的 Frontend 含义，服务端接入端，目前只支持Rcm类型服务端",
            "Frontend": {
                "//12": "旁路接收主题，必填",
                "RxBypassTopic": {
                    "//13": "主题名，必填",
                    "Name": "bypass_redo",
                    "//14": "分区号，可选，用于标识能接收哪些分区",
                    "PartitionNo": 63
                }
            }
        }
    },
    {
        "Lib": "ldp_mproxy",
        "Instance": 0,
        "Timer": 0,
        "Network": {
            "ListenPorts": "11108"
        }
    },
    {
        "Lib": "ldp_logproxy",
        "Instance": 0,
        "Timer": 0,
        "LogLevel": 0
    },
    {
        "Lib": "ldp_remoter",
        "Instance": 0,
        "Timer": 0
    },
    {
        "Lib": "ldp_rcm",
        "Instance": 0,
        "Timer": 0,
        "ConfigUpdate": true
    }
]
},
"Derived": {
    "redo_copy": {
        "redo_copy#0": {},
        "redo_copy#1": {},
        "redo_copy#2": {}
    }
},
"Rcm": {
    "Transports": [
        {
            "Name": "Default",
            "Port": 12500
        }
    ],
    "Contexts": [
        {
            "Name": "Default",
            "Ip": "192.168.86.0",
            "IsSingleton": true,
            "IsRecord": false,
            "IsTotalOrder": false,
            "RecordDir": ".",
            "RepairPortStart": 12000,
            "RepairPortEnd": 12020
        }
    ],
    "MaterialRange": {
        "TransportId": {
            "MinValue": 1,
            "MaxValue": 65535
        },
        "ContextId": {
            "MinValue": 1,
            "MaxValue": 65535
        },
        "PartitionNo": {
            "MinValue": 1,
            "MaxValue": 65535
        },
        "MulticastAddr": {
            "MinValue": "238.0.0.1",
            "MaxValue": "238.0.0.255"
        },
        "MulticastPort": {
            "MinValue": 12500,
            "MaxValue": 12999
        },
        "SyncAddr": {
            "MinValue": "238.1.0.0",
            "MaxValue": "238.1.0.255"
        },
        "SyncPort": {
            "MinValue": 13000,

```

```
        "MaxValue": 13999
    },
    "LocalPort": {
        "MinValue": 14000,
        "MaxValue": 14999
    }
}
}
```

4.26. 核心转储文件解析插件 ldp_coreparser

ldp_coreparser是一个核心转储文件解析插件，用于自动和手动解析coredump文件。

插件名	支持平台	插件标识	说明
libldp_coreparser.so	linux.x64、aarch64	ldp_coreparser	核心转储文件解析插件

4.26.1. 示例配置

```
{
  "Lib": "ldp_coreparser",
  "Instance": 0,
  "Timer": 0,
  "/*": "核心应用工作路径列表，配置后将会自动解析目录下新生成的core文件",
  "WorkSpaceList": [
    {
      "/*": "核心节点的工作路径，如果core文件生成路径与核心节点的工作路径不相同，需要设置为core文件生成路径",
      "WorkSpace": "/home/uft30/ucrtbid52/workspace"
    },
    {
      "WorkSpace": "/home/uft30/uconvert99/workspace"
    }
  ],
  "/*": "定时解析core dump文件时间间隔，单位秒，默认为60",
  "AutoParserSec": 60,
  "/*": "消息结构的配置文件",
  "MsgConfig": "demo.cfg"
}
```

4.26.2. 配置项说明

名称	类型	说明	备注
WorkSpaceList	对象数组	核心应用工作路径列表	配置后将会定时自动扫描并解析目录下新生成的core文件
WorkSpace	字符串	核心节点的工作路径	需要配置绝对路径，如果core文件生成路径与核心节点的工作路径不相同，需要设置为core dump文件生成路径
AutoParserSec	整型	定时解析core dump文件时间间隔	单位秒，默认为60。每次扫描解析完成后，AutoParserSec秒后再次进行扫描解析
MsgConfig	字符串	消息结构的配置文件	可选。指定消息结构的配置文件（.cfg）。若提供该参数，程序将依据配置文件中定义的功能号及其对应的请求结构来解析codump文件。不配置此配置项时默认按照ldp-pack来解析codump文件。

4.26.3. 其他说明

- 1、本插件只解析核心节点中，因业务代码崩溃导致的core文件，不解析因平台代码导致的core文件，不解析因其他插件而产生的core文件。
- 2、解析结果将会保存在工作路径下的coreparser_result文件夹中。
- 3、解析结果文件名命名规则为`<core文件名>-<解析时间>-parser-result.json`，例如`core.83614-20250728-191256-ParserResult.json`。

4.26.4. 依赖说明

- 本插件依赖t2sdk独立库(libt2sdk.so)，用户需要在Cres的发布包中下载该so文件，将其放到LDP发布包/bin目录下，并额外设置环境变量LD_LIBRARY_PATH库路径为LDP发布包/bin目录。

4.27. 小表缓存 ldp_minitablecache

Ldp_minitablecache 小表缓存针对的是字段众多的大表，通过将经常访问的大表字段进行缩减，变成小表，来提高查询速度，达到低延迟的效果。小表缓存在恒生电子的演变包括：java版本的小表缓存和cres版本的小表缓存。实现c++依赖mab版本的小表缓存是因为它与内存数据库具有很好的耦合性，且很容易实现跨库的功能，在性能上有一定优势。

小表缓存功能包括下面几个方面：

- 1、订阅同步消息，执行缓存更新；
- 2、业务更新缓存表，由MC通知其他内存核心进行缓存更新。

库名	支持平台	插件标识	说明
libldp_minitablecache.so	linux.x64	libldp_minitablecache.so	小表缓存驱动库

4.27.1. minitablecache模块配置示例

```
{
  "Lib": "ldp_minitablecache",
  "Instance": 0,
  "Timer": 0,
  // 小表缓存数据源, 默认值是 AppName + "minitablecache"
  "LogicName": "ds_minitablecache",
  // 主题名
  "TopicName": "topic.mid.localcache",
  // Mc 配置文件
  "McConfigFile": "config.ini",
  // 表名相关内容
  "TableConfigFile": "CacheTableList.json",
  // 消费模式, 两种消费模式: 数据或者通知
  "ConsumptionMode": 0,
  // Mc 启动失败, 尝试重启的时间间隔
  "TimeInterval": 1,
  // Mc 启动失败, 最大重试次数
  "MaxAttempts": 100,
  // 查询缓存找不到记录时是否写警告日志, 不写
  "WriteLogWhenNotFound": false,
}
```

4.27.2. 配置项说明

名称	类型	说明	备注
LogicName	字符串	小表缓存数据源, 并作为数据库名	要求用户配置时名称不能相同, 默认值是 AppName + "minitablecache"
TopicName	字符串	主题名	必须配置, 无默认值
McConfigFile	Mc 配置文件名	无默认值, 不配置参数表示不使用 MC 相关功能	
ConsumptionMode	正整数	消费模式, 两种消费模式: 数据或者通知	默认值为0, 表示数据消费模式; 取值为0, 1
TimeInterval	正整数	Mc 启动失败, 尝试重启的时间间隔	默认值 1s, 范围 1s~50s
MaxAttempts	整数	Mc 启动失败, 最大重试次数	默认值 100, 范围 50~1000
WriteLogWhenNotFound	布尔	查询缓存找不到记录时是否写警告日志	默认值 false, 不写警告记录

4.27.3. McConfigFile、TableConfigFile文件格式

McConfigFile Mc配置文件格式(配置文件参见 Mc使用文档)

配置中 ; 号为注释

```
[mcsdk]
;group_name=group1
;server_address=192.168.86.32:22030
;发布队列的长度, 最小为 50, 最大为 100000
;msg_count=10000
;集群消费异步偏移量提交时, 可以指定距离上一次偏移量多少个消息就提交, 默认 100 个消息。
;offset_count=10
;集群消费异步偏移量提交时, 指定提交的时间间隔, 单位为毫秒, 默认值为 5000 毫秒。
;commit_offset_interval=5000
;用户回调返回 CONSUME_CUSTOMER 时会将消息加入 RPO 队列
consume_rpo=10000
```

TableConfigFile 缓存表配置文件（由开发工具生成）

```
{
  "CacheTables":
  [
    {
      // 表名
      "TableName": "test_minitable",
      "LogicName": "usersvr",
      "Statement": {
        "Select": "col_int,col_integer,col_number,col_varchar2, col_nvarchar2,col_varchar, transaction_no, position_str",
        "From": "test_minitable",
        "Where": ""
      },
      // 索引相关配置
      "Indexes": [
        {
          "IndexName": "idx_test_minitable1",
          "IndexInfo": [
            {
              "Name": "col_integer",
              "Type": 2
            }
          ]
        },
        {
          "IndexName": "idx_test_minitable2",
          "IndexInfo": [
            {
              "Name": "col_varchar2,col_varchar",
              "Type": 3
            }
          ]
        }
      ]
    }
  ]
}
```


4.28. partkeeper插件 ldp_partkeeper

partkeeper是一个LDP插件，在核心启用分区模式下使用，负责下发初始分区路由表给所有核心，并且在运行过程中，将异常退出的核心上的分区划分给其他存活的核心，或者人为实时调整出有效的分区路由表

插件名	支持平台	插件标识	说明
libldp_partkeeper.so	linux.x64	ldp20.partkeeper	partkeeper

4.28.1. 配置说明

partkeeper插件的配置格式

以下是partkeeper插件配置的一个抽象示例，后续对其具体展开解释

```
"Plugins":
[
  {
    "Lib": "ldp_partkeeper",
    "Instance": 0,
    "Timer": 1,
    "StartMode": 0,
    "ArbFile": "arb_result.json",
    "BackupDir": "./partkeeper_data",
    "MaxFileSizeBit": 1073741824,
    "BizStatusFreqMS": 30000,
    "AppStatusFreqMS": 30000,
    "ProgressTimeOutMS": 30000,
    "SyncTimeOutMS": 30000,
    "StartTimeOutMS": 30000,
    "HeartBeatTimeMS": 50,
    "SessionTimeOutMS": 3000,
    "ProgressTimeMS": 1000,
    "Remoter": {
      "HaMode": 2,
      "Frontend": {
        "Tcp": {
          "Acceptors": [
            {
              "Name": "keeper_tcp",
              "FactoryName": "Default",
              "Address": "192.168.86.102",
              "Port": 10554
            }
          ]
        }
      }
    }
  }
]
```

4.28.2. 配置项说明

名称	类型	说明	备注
StartMode	整型	启动模式,0=集群重启,1=partkeeper灾备恢复到上次停止时的状态	默认为0
ArbFile	字符串	本地初始分区路由表	只支持存放在本地,默认为arb_result.json
BackupDir	字符串	持久化文件保存路径	只支持存放在本地,默认为./partkeeper_data
MaxFileSizeBit	整型	单个持久化文件的大小上限	单位字节,默认1G
BizStatusFreqMS	整型	查询核心业务就绪状态的频率	单位ms,默认30000ms
AppStatusFreqMS	整型	查询核心分区就绪状态的频率	单位ms,默认30000ms
ProgressTimeMS	整型	partkeeper定时获取redo进度的频率	单位ms,默认1000ms
ProgressTimeOutMS	整型	查询redo进度的超时时间	单位ms,默认30000ms
SyncTimeOutMS	整型	等待同步完成超时时间	单位ms,默认30000ms
HeartBeatTimeMS	整型	心跳间隔	单位ms,默认50ms
SessionTimeOutMS	整型	连接超时时间	超过该时间没有收到任何消息后连接断开并自动登出,单位ms,默认3000ms
StartTimeOutMS	整型	集群启动超时时间	从第一个核心登录开始计时,单位ms,默认30000ms
Remoter	对象	remoter组件服务端配置	必须配置,使用remoter组件进行通讯

4.28.3. 初始分区路由表配置文件格式

```
{
  "ArbNo": 0,
  "KeeperQuery": 0,
  "mdb_1": {
    "ShardRoundNo": 0,
    "StartStatus": 0,
    "BizProcNode": {
      "mdb#0": {
        "GroupId": 1,
        "Age": 0
      }
    },
    "Partition": {
      "0": {
        "ArbNo": 0,
        "Writer": [ "multiproc_mdb#0_partgroup_1" ],
        "Reader": [ "multiproc_mdb#1_partgroup_2" ],
        "Status": 0,
        "MigrationStrategy": 0
      }
    },
    "PartGroup": {
      "multiproc_mdb#0_partgroup_1": {
        "NodeName": "mdb#0",
        "BizStatus": 2,
        "AppStatus": 0,
        "ArbStatus": 0,
        "GroupStatus": 0,
        "MinMessageNo": 0,
        "MaxMessageNo": 0,
        "Partition": [ 0, 1 ],
        "UnReadyPart": []
      }
    }
  }
}
```

4.28.4. 配置项说明

名称	类型	说明	备注
ArbNo	整型	仲裁序号	每次分区路由表的数据发生变更时递增(例如节点登入登出,分区0迁移到新核心),默认为0
KeeperQuery	整型	查询序号	每次partkeeper向核心查询信息的时递增(例如查询分区同步进度,注:业务就绪,分区就绪是核心主动上报的,不算查询),默认为0
分片名	字符串	分片名	对应整个系统的分片名
ShardRoundNo	整型	分片内轮次号	每次发生分区迁移时递增(例如分区0迁移到新核心),默认为0
StartStatus	整型	集群启动状态	0=集群未启动完成,1=等待初次迁移,2=集群启动完成,默认为0
BizProcNode	对象	核心节点相关配置	必须配置
节点名	对象	节点名	必须配置,一个核心节点的配置,带#组内编号
GroupId	整型	分区组下标	当前核心负责的写分区组下标,从1开始编号,不能有空缺,不能重复,默认自增,注:组下标必须是节点名组内编号+1,例如核心xst#0对应的GroupId为1
Age	整型	核心的年龄	对应核心的年龄(即核心节点生成和管理的轮次号),默认为0
Partition	对象	分片相关配置	可选
分片号	对象	一个分片号的配置	必须配置,分片号从0开始,必须存在0号分片
ArbNo	整型	仲裁序号	确定当前分片对应分片组时的仲裁序号,默认为0
Status	整型	分区迁移状态	0=正常,1=等待写核心停止处理,2=等待读核心同步完成,3=收到写核心确定,4=收到所有网关确认,5=收到所有读核心确认,6=分区迁移结束,默认为0
Writer	数组对象	写分区组	必须配置,持有当前分片写权限的分片组名
分片组名	字符串	分片组名	必须配置,分片组名,也可以只配置核心节点名,格式: multiproc_节点名_partgroup_分片组下标
MigrationStrategy	整型	迁移策略	0(默认)=手动迁移,1=自动迁移给redo序号最小的核心
PartGroup	对象	分片组相关配置	可选
分片组名	字符串	分片组名	必须配置,分片组名,格式: multiproc_节点名_partgroup_分片组下标
NodeName	字符串	核心节点名	可选,当前分片组对应的核心节点名
BizStatus	整型	业务就绪状态	1=业务未就绪,2=业务就绪; 业务未就绪的核心不处理除0号分区外的其他业务,默认为1
AppStatus	整型	平台就绪状态	0=分区未就绪,1=分区就绪; 分区未就绪的核心不处理该分区内的任意分区业务,默认为0

ArbStatus	整型	redo通道的读写权限	0=作为当前分区组所在行的写核心,1=作为当前分区组所在行的读核心,默认为0
GroupStatus	整型	分区组迁移状态	0=分区组写核心正常,1=分区组写核心断开,需要分区组迁移,2=作为数据同步源(由其他核心替代分区组的写核心),3=要向数据同步源同步分区组数据,默认为0
MinMessageNo	整型	起始redo序号	当前分区内轮次号[ShardRoundNo]下的起始redo序号,其他核心向其同步数据时作为起始redo序号,默认为-1
MaxMessageNo	整型	截止redo序号	当前分区内轮次号[ShardRoundNo]下的该核心的最大redo序号,其他核心向其同步数据时作为截止redo序号,默认为-1
Partition	数组对象	分区列表	可选,受Appstatus管理的分区列表
分区号	整数	分区号	可选,自动对应分片的Writer
UnReadyPart	数组对象	分区列表	可选,受Appstatus管理的分片列表,必定是分区未就绪的分区(例如原本1号分区已经就绪,但是新分配了2号分区,1号分区就在Partition内表示该分区已经继续,2号分区就在UnReadyPart内,表示分区未就绪,等核心汇报就绪后,将2号分区迁移到Partition内)
分区号	整数	分区号	可选,自动对应分片的Writer

4.29. partkeeper客户端插件 ldp_partkeeper_client

ldp_partkeeper_client 是一个LDP插件，用于对接 Partkeeper ， 在网格模式下， 核心、前置、todb/togw 会直接或间接用到该插件。

插件名	支持平台	插件标识	说明
libldp_partkeeper_client.so	linux.x64	ldp_partkeeper_client	partkeeper 客户端插件

4.29.1. 配置说明

ldp_partkeeper_client 插件的配置格式

以下是partkeeper_client插件配置的一个抽象示例，后续对其具体展开解释。

一般来说，只有 Remoter/Backends 与 AutoOnlineAfterCallbackReg 需要关注。Remoter/Backends 需要配置为 partkeeper 的地址。可配置多个 Sites（目前 partkeeper 仅支持冷备，所以应当配置一个启动中的地址，一个关掉的冷备地址）；AutoOnlineAfterCallbackReg 在网关使用 partkeeper_client 时需要配置为 true，核心或其他场景配置为 false。

```
"Plugins":
[
  {
    "Lib": "ldp_partkeeper_client",
    "Instance": 0,
    "Timer": 0,
    "/*": "向 Partkeeper 发送同步消息等待响应的超时时间, 默认 5 秒",
    "SendTimeoutSec": 5,
    "/*": "回调失败后的重试次数, 默认 3",
    "CallbackRetryTimes": 3,
    "/*": "回调失败后的重试间隔, 默认 1 秒",
    "CallbackDelaySec": 1,
    "/*": "是否开启首次注册回调函数时自动向 Partkeeper 发起登录, 注意网关节点应当开启, 核心节点应当关闭, 默认 false",
    "AutoOnlineAfterCallbackReg": false,
    "/*": "是否开启 Session 断开后自动向 partkeeper 重新登录的功能, 默认为 true",
    "AutoLoginAfterSessionTimeout": true,
    "/*": "Session 断开检查的频率, 一般不用修改, 默认 1000 毫秒",
    "SessionCheckIntervalMs": 1000,
    "/*": "Remoter 配置, 用于连接 Partkeeper",
    "Remoter": {
      "/*": "在 partkeeper_client 中需要配置为 true",
      "AutoStart": true,
      "/*": "Backend路由, 需要都指向 Backend1",
      "Routes": [
        {
          "Functions": "*",
          "Systems": "*",
          "Nodes": "*",
          "Backend": 1
        }
      ],
      "/*": "需要配置一个 Id 为 1 的 Backend, Type 为 TCP, Sites 配置指向 Partkeeper 地址",
      "Backends": [
        {
          "Id": 1,
          "Type": "TCP",
          "Sites": [
            {
              "Host": "192.168.86.102",
              "Port": 33201
            },
            {
              "Host": "192.168.86.103",
              "Port": 33201
            }
          ]
        },
        "HeartbeatIntervalSec": 1
      ]
    },
    "/*": "partkeeper_client 需要配置 2 非集群模式",
    "HaMode": 2,
    "/*": "partkeeper_client 需要配置 false",
    "EnableBackendBizStatusCtrl": false
  },
  "WatchShardList": ["proc_0"]
],
}
```

4.29.2. 配置项说明

名称	类型	说明	备注
SendTimeoutSec	整型	向 Partkeeper 发送同步消息等待响应的超时时间	单位秒, 默认 5 秒
CallbackRetryTimes	整型	回调失败后的重试次数	默认 3
CallbackDelaySec	整型	回调失败后的重试间隔	单位秒, 默认 1 秒
AutoOnlineAfterCallbackReg	布尔型	是否开启首次注册回调函数时自动向 Partkeeper 发起登录	网关节点应当开启, 核心节点应当关闭, 默认 false
AutoLoginAfterSessionTimeout	布尔型	是否开启 Session 断开后自动向 Partkeeper 重新登录的功能	默认 true
SessionCheckIntervalMs	整型	Session 断开检查的频率	一般不用修改, 默认 1000 毫秒
Remoter 对象配置			
Remoter	对象	remoter组件服务端配置	必须配置, 使用remoter组件进行通讯
AutoStart	布尔型	是否自动启动	在 partkeeper_client 中需要配置为 true
Routes	数组	Backend路由配置	需要都指向 Backend1
Backends	数组	Backend 配置	需要配置一个 Id 为 1 的 Backend, Type 为 TCP, Sites 配置指向 Partkeeper 地址
HaMode	整型	高可用模式	partkeeper_client 需要配置 2 非集群模式
EnableBackendBizStatusCtrl	布尔型	是否启用后端业务状态控制	partkeeper_client 需要配置 false
WatchShardList	数组	控制客户端监听的分片信息	网关todb类客户端默认监听所有分片,核心默认只监听所属分片,必须配置完整的分片信息,即集群名_分片号,通配符*表示监听所有分片

4.30. multiproc插件 ldp_multiproc

ldp_multiproc 是一个LDP插件, 是数据网格模式下用于处理多个 RCM 通道的管理插件, 提供结构给 bizproc 或 todb/togw 使用, 主要对接 lib_rcm 与 partkeeper_client 。

插件名	支持平台	插件标识	说明
libldp_multiproc.so	linux.x64	ldp_multiproc	multiproc 插件

4.30.1. 使用限制

使用 multiproc 时，需要进行以下限制：

- multiproc 最大支持的节点数为10；
- 配置中SyncIp需要配置为网段，且同一应用的 multiproc 网络核心必须在同一网段内部署
- 写分区组（arbStatus=0 的分区组）名字必须满足核心应用编号与分区组编号是加一关系，如 multiproc_xst#0_partgroup_1 作为写分区组合法，multiproc_xst#1_partgroup_1 作为写分区组不合法，因为 #1 要对应分区组 2
- 不支持对接多集群多分片，目前仅支持对接单集群单分片(修复版本： V1.2501.2.0)；
- 不支持集群中只有部分节点启动，存在一些节点从未启动时进行分区迁移(修复插件： ldp_partkeeper 修复版本： V1.2501.0.1)
- 目前不支持所有核心节点全部与 partkeeper 断开过的场景，这样会导致 partkeeper 上所有分区组的状态都不正确，无法选出有效的主核心(修复版本： V1.2501.1.0)

4.30.2. 配置说明

ldp_multiproc 插件的配置格式

```
"Plugins":
[
  {
    "Lib": "ldp_multiproc",
    "Instance": 0,
    "Timer": 0,
    "LogLevel": 1,
    "BizLogLevel": 1,
    "Network": {
      "/*": "绑定的本地ip",
      "SyncIp": "192.168.86.0",
      "/*": "组播地址",
      "SyncAddr": "237.34.0.2",
      "/*": "组播端口",
      "SyncPortStart": 42101,
      "/*": "旁路组播地址",
      "BypassSyncAddr": "236.34.0.2",
      "/*": "旁路端口",
      "BypassSyncPort": 42001
    },
    "Base": {
      "/*": "是否开启旁路, false: 不开启, true: 开启。默认不开启",
      "IsBypass": true,
      "/*": "旁路方向, 1: 发送, 2: 接收。默认无",
      "BypassDir": 2
    },
    "/*": "高可用模式, true: 主备, false: 单机",
    "HaMode": true,
    "/*": "是否仅启动bypass、不启动普通同步, 默认true。无论何种模式，旁路都支持单节点启动多个上下文接收",
    "/*": "同时启动普通同步指的是双中心同步，旁路收到后会自动转发给本中心其他节点，相当于跨集群的同步。他要求和主机房配置和拓扑一样",
    "/*": "仅bypass指的是收到后单纯的转发给业务处理，如写物理库。部署上可以简化，如主机房5台，旁路接收1台，这种模式下multiproc会启动keeper下发的所有分区组对应的接收旁路",
    "OnlyBypass": true
  },
]
```

4.30.3. 配置项说明

名称	类型	说明	备注
Network	对象	网络配置	包含多个子项，用于配置网络相关参数
Network/SyncIp	字符串	绑定的本机IP	用于指定插件绑定的本地网段。取值示例："192.168.86.0"
Network/SyncAddr	字符串	组播地址	用于数据同步的组播通信，配置需要在RCM物料"SyncAddr"范围内。取值示例："237.34.0.2"
Network/SyncPortStart	整型	组播端口	指定组播通信的起始端口，当前已基于配置的RCM物料范围自动生成（但仍需配置）。取值示例：42101
Network/BypassSyncAddr	字符串	旁路组播地址	用于旁路数据同步的组播通信，配置需要在RCM物料"MulticastAddr"范围内。取值示例："236.34.0.2"
Network/BypassSyncPort	整型	旁路端口	指定旁路组播通信的端口，当前已基于配置的RCM物料范围自动生成（但仍需配置）。取值示例：42001
Base	对象	基础配置	包含多个子项，用于配置基础参数
Base/IsBypass	布尔型	是否开启旁路	决定是否启用旁路功能。取值：true 或 false。默认值：false
Base/BypassDir	整型	旁路方向	指定旁路数据的传输方向。取值：1（发送），2（接收）。
HaMode	布尔型	高可用模式	决定是否启用高可用配置。取值：true（主备），false（单机）。默认值：true
OnlyBypass	布尔型	是否仅启动旁路	决定是否仅启动旁路而不启动普通同步。取值：true 或 false

4.31. 数据转换 ldp_datamapper

ldp_datamapper 数据转换插件支持对 LdpMsg 内部内容进行转换。目前支持的主体是 pack 的场景进行转换。支持用户进行二次开发。目前支持静态字典转换、适配函数转换。当前版本仅支持对字符串、嵌套包转换，不支持对 int、浮点数等域进行转换，不支持中文字符转换、包版本切换等功能。只支持简单的英文和字符转换，且只对第一个结果集的内容进行转换。

数据转换功能包括下面几个方面：

- 1、解析包体中的Pack包，对每个域进行静态字典转换。
- 2、解析包体中的Pack包，支持用于开发二次开发库，对包体直接进行转换。

IMPORTANT

限制：目前只支持单结果集。如果是多结果集，其他结果集会丢弃

库名	支持平台	插件标识	说明
libldp_datamapper.so	linux.x64	ldp_datamapper	数据转换插件

4.31.1. datamapper模块配置示例

```
{
  "Lib": "ldp_datamapper",
  "ConfigFile": "test.xml",
  "Instance": 0
}
```

4.31.2. 配置项说明

名称	类型	说明	备注
ConfigFile	字符串	数据转换使用的配置文件	必填，无默认值

4.31.3. DataMapperConfigFile文件格式

DataMapper配置文件格式 note中的内容为注释

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- datamapper示例配置 -->
<ldptrans>
  <!-- 函数配置 -->
  <funcs>
    <func functionid="620701;620702" biz_func="FuncFunc" rule="rule01" note="同时配置了rule和biz_func,优先使用biz_func"/>
    <func functionid="76*" rule="rule02" dictionary="dtDict2" note="功能号是76开头的前按照规则2转换"/>
  </funcs>

  <!-- 具体的转换规则配置 -->
  <rules>
    <rule name="rule01" note="规则配置1">
      <field name="id" dictionary="dtDict1" srcsplit="|" destsplit="," note="该字段使用数据字典转换，且有分割符"/>
      <field name="test" dictionary="dtDict2" biz_func="FieldFunc" note="同时配置了数据字典和字段转换函数，这里采用字段转换函数执行">
        <field name="fund_account" dictionary="dtDict2" />
      </rule>
      <rule name="rule02">
        <field name="operator_code" dictionary="dtDict1"/>
        <field name="fund_account" note="这里没配置字典，会尝试使用funcs中的字典去进行静态字典转换"/>
      </rule>
    </rules>

    <!-- 数据字典配置 -->
    <dictionaries>
      <dictionary name="dtDict1" note="静态数据字典">
        <entry srcvalue="1" destvalue="a"/>
        <entry srcvalue="2" destvalue="b"/>
        <entry srcvalue="s" destvalue="c"/>
        <entry srcvalue="t" destvalue="d"/>
        <entry srcvalue="w" destvalue="e"/>
        <entry srcvalue="z" destvalue="p"/>
        <entry srcvalue="abc" destvalue="xyz"/>
      </dictionary>
      <dictionary name="dtDict2" note="静态数据字典">
        <entry srcvalue="a" destvalue="1"/>
        <entry srcvalue="t" destvalue="s"/>
        <entry srcvalue="b" destvalue="d"/>
        <entry srcvalue="w" destvalue=" " />
        <entry srcvalue="x" destvalue="y"/>
        <entry srcvalue="z" destvalue="p"/>
      </dictionary>
    </dictionaries>

    <!-- 二次开发器 -->
    <adapters>
      <adapter name="adapter.so">
        <function>
          <bizfunc biz_func="FuncFunc" note="biz_func不能重名，多个adapter之间，biz_func也不能重名" type="0"/>
          <bizfunc biz_func="FieldFunc" type="1" note="0表示功能号级别转换，1表示域级别转换"/>
        </function>
      </adapter>
    </adapters>
  </ldptrans>
```

4.31.4. 规则说明

处理层级

消息转换分为两个层级：

- 1. 功能级处理 - 针对整个消息的处理（通过 biz_func 配置）
- 2. 字段级处理 - 针对消息中单个字段的处理（通过规则配置）

处理方式

字段级别支持两种处理方式：

- 1. 数据字典转换 - 通过静态映射表进行值的转换
- 2. 自定义函数 - 通过编写代码实现复杂的转换逻辑

数据字典配置

定义数据字典

在 <dictionaries> 节点下定义数据字典：

```
<dictionaries>
  <dictionary name="dtDict1" note="说明信息">
    <entry srcvalue="源值1" destvalue="目标值1"/>
    <entry srcvalue="源值2" destvalue="目标值2"/>
  </dictionary>
</dictionaries>
```

Table 1. 配置说明

参数	说明
name	字典名称，全局唯一标识
note	字典说明（可选）
srcvalue	源字段值
destvalue	转换后的目标值

在功能号上配置字典

在 <funcs> 节点中为功能号配置数据字典：

```
<func functionid="1111" dictionary="dtDict3" rule="rule03"/>
```

Table 2. 配置说明

参数	说明
functionid	功能号，支持精确匹配（如 620701;620702）和通配符（如 76**）
dictionary	指定保底使用的数据字典名称
rule	关联的规则名称

应用场景

当某个功能号下的所有字段都需要使用相同的字典转换时，可以在功能级配置公共字典。

在字段规则上配置字典

在 <rules> 节点中为具体字段配置数据字典：

```
<rule name="rule01">
  <field name="id" dictionary="dtDict1" srcsplit="|" destsplit=","/>
  <field name="fund_account" dictionary="dtDict2"/>
</rule>
```

Table 3. 配置说明

参数	说明
name	字段名称
dictionary	该字段使用的数据字典
srcsplit	源值的分隔符（可选）
destsplit	目标值的分隔符（可选）

优先级规则

功能级处理优先级

当功能号配置了 biz_func 时：

```
<func functionid="620701" biz_func="FuncFunc" dictionary="dtDict2"/>
```

处理顺序

- 1. 优先执行 biz_func 函数处理整个消息
- 2. 忽略后续的字典和规则配置
- 3. 直接返回处理结果

IMPORTANT

biz_func 具有最高优先级，一旦配置，整个消息将由该函数处理。

字段级处理优先级

当功能号未配置 biz_func，系统会进入字段级处理：

```
<rule name="rule01">
  <field name="test" dictionary="dtDict2" biz_func="FieldFunc"/>
</rule>
```

处理顺序（从高到低）

- 1. 字段级自定义函数（field.biz_func）
 - 如果字段配置了 biz_func，优先使用该函数处理
- 2. 字段级数据字典（field.dictionary）
 - 如果字段配置了 dictionary，使用该字典转换
- 3. 功能级公共字典（func.dictionary）
 - 如果字段未配置字典，但功能号配置了公共字典，使用公共字典
- 4. 不处理
 - 如果以上都未配置，该字段保持原值

用户适配函数限制和使用说明

动态库开发要求

用户可以自行编写适配函数动态库（demo 库），但需要遵循以下要求：

必须包含的头文件

```
#include "datamapper_biz.h"
```

导出

用户需要用 Export 导出，且必须保证，对应的函数类型入参正确。否则会导致崩溃

必须实现的函数

用户必须强制实现以下三个函数：

函数名	说明
OnInit	初始化函数，在加载动态库时调用
OnExit	退出函数，在卸载动态库时调用
GetVersion	版本获取函数，返回动态库版本信息

NOTE 具体函数签名和入参请参考头文件 datamapper_biz.h 中的定义。

功能号级别转换函数

对于功能号级别的转换函数（biz_func 配置在 <func> 节点上）：

处理方式

- 直接将转换后的结果写入 lpIFPacker 中即可
- 无需额外的内存管理操作

示例

```
EXPORT int32_t FuncFunc(FuncTransContext *lpContext)
{
    // 直接向 lpIFPacker 写入转换后的数据
    auto lpPack = lpContext->lpIF2Packer;
    lpPack->BeginPack();
    lpPack->AddField("Data");
    std::string data = std::to_string(44) + "-" + std::string(44, 'x');
    lpPack->AddStr(data.c_str());

    lpPack->EndPack();
    return 0;
}
```

字段级别转换函数

对于字段级别的转换函数（biz_func 配置在 <field> 节点上）：

处理方式

- 1. 转换后的结果需要用户通过 malloc 申请内存
- 2. 将申请的内存地址返回
- 3. 在 destBufLen 中填写对应的数据长度
- 4. 由 datamapper 负责统一释放内存

IMPORTANT

内存申请必须使用 malloc 函数

- 正确：使用 malloc 申请内存
- 错误：使用 new、calloc 或栈内存
- 原因：datamapper 内部使用 free 统一释放内存，必须与 malloc 配对使用

示例


```
EXPORT int32_t FieldFunc(FieldTransContext *lpContext)
{
    // 插件分配内存
    const char* result = "1";
    int32_t len = strlen(result) + 1;

    // 这里必须使用malloc而不是new
    char* temp = (char*)malloc(len);
    if (!temp) return -1;

    strcpy(temp, result);

    // 返回指针给调用者
    *(lpContext->lpszDestValue) = temp;
    *(lpContext->iDestBufLen) = len;

    return 0;
}
```

WARNING

内存管理注意事项

- 申请的内存由 `datamapper` 自动释放，用户无需手动释放
- 必须在 `destBufLen` 中正确填写数据长度
- 内存申请失败时应返回错误码

4.32. 流量管控服务插件 `ldp_flowcontrol`

`ldp_flowcontrol`是一个流量管控服务插件，加载管理接口如APM送入的黑白名单的规则，在前置接收外部请求时，根据请求的账号和功能区，或分片号，对符合限流规则的请求，进行拦截报错处理。黑白名单规则由账号，功能区，分片号，生效节点信息组成，可控制规则是否生效， 生效的规则通过插件存储到`zookeeper`上，同一个产品内多个插件节点间共用同一份数据，一处修改，同时生效。

插件名	支持平台	插件标识	说明
<code>libldp_flowcontrol.so</code>	<code>linux.x64</code>	<code>ldp_flowcontrol</code>	流量管控服务插件

4.32.1. 示例配置

```
{
    "/*": "流量管控服务库名，这个名字固定，不需要修改",
    "Lib": "ldp_flowcontrol",
    "/*": "实例编号",
    "Instance": 0,
    "/*": "黑名单匹配时返回的错误信息",
    "BlackList": {
        "FuncErr": "该请求的功能号当前已被暂停接收服务",
        "AccountErr": "该请求的账户当前已被加入黑名单，请求已被拒绝",
        "FuncAccountErr": "该请求的账户被限制开展该业务请求，请求已被拒绝"
    },
    "/*": "白名单匹配时返回的错误信息",
    "WhiteList": {
        "FuncErr": "该请求的请求当前已暂停接收服务",
        "AccountErr": "系统已暂停对外服务，请求已被拒绝",
        "FuncAccountErr": "该账户当前请求当前已暂停接收服务"
    },
    "/*": "被熔断限流时返回的错误信息",
    "Tps": {
        "FuncErr": "该业务请求流速已超限",
        "AccountErr": "该账户请求流速已超限",
        "FuncAccountErr": "该账户的该业务请求流速已超限"
    },
    "/*": "熔断限流token申请分配参数",
    "PreAllocTokenTps": 20,
    "PreAllocTokenTpm": 100,
    "PreAllocTokenTph": 1000,
    "ReqTokenTimeoutUs": 800,
    "TokensUpdateCount": 1000,
    "TokensUpdateTimeMs": 200
}
```

4.32.2. 配置项说明

名称	类型	说明	备注
<code>FuncErr</code>	字符串	检查账号不在黑名单时返回	
<code>AccountErr</code>	字符串	检查账号不在黑名单时返回	
<code>FuncAccountErr</code>	字符串	检查账号不在黑名单时返回	
<code>FuncErr</code>	字符串	检查功能区不在白名单时返回	
<code>AccountErr</code>	字符串	检查账号不在白名单时返回	
<code>FuncAccountErr</code>	字符串	检查账号和功能号不在白名单时返回	
<code>FuncErr</code>	字符串	检查功能区超过了熔断限流规则值时返回	
<code>AccountErr</code>	字符串	检查账号超过了熔断限流规则值时返回	
<code>FuncAccountErr</code>	字符串	检查请求的账号和功能号超	

		过了熔断限流规则值时返回	
PreAllocTokenTps	整形	单位为秒的规则，控制预分配token的个数，可以配置为1-1000, 实际计算值规则 $\text{tps}/1000 * \text{当前值}$ ，建议配置为20-200之间，配置数值太小，会导致申请token请求数过多，过大可能导致某个前置申请未使用	
PreAllocTokenTpm	整形	单位为秒以上2-3600秒的规则，每次预分配的token数	
PreAllocTokenTpH	整形	单位为小时以上的规则，每次预分配的token数	
ReqTokenTimeoutUs	整形	分配token请求的超时时间，一般为前置到核心请求时延，一般为几百到1毫秒	
TokensUpdateCount	整形	秒级别以上限流规格预分配token数变化后需要写到zk的阈值，能结束的误差大小	
TokensUpdateTimeMs	整形	秒级别以上限流规格预分配token数变化后定时同步到ZK时间，最小200毫秒	

4.32.3. 依赖说明

- 流量管控插件自身不能完成流量管控，请求不会自动进入到它里边，要实现流量管控的目标，需要前置或者其它节点调用它的接口来识别请求是否触发拦截规则，然后由节点自己实现请求的拦截处理。插件只提供接口。
- 流量管控插件如果只启用黑白名单功能,那么在前置部署即可。如果要启用限流熔断功能，那么还需要将flowcontrol部署到一个bizproc节点上，这个节点作为一个中心节点，总管流量分配。然后前置需求将内部功能号413请求路由到该节点。
- 流量管控插件需要有zookeeper作为中心节点，实现接口一处调用，各个插件都能生效的效果。

4.33. 文件同步 ldp_filesync

Ldp_filesync 一个基于 rsync 核心算法深度优化的插件，为开发者提供了一套完整、灵活、高性能的同步解决方案。将 rsync 的强大功能封装为简洁的 API 接口，支持任务化管理、实时监控和进度跟踪，让开发者能够轻松地将专业级文件同步能力集成到自己的应用中。

文件同步功能包括下面几个方面：

- 启动文件同步任务，将复杂的同步逻辑封装为简洁的参数，让开发者能够以极少的代码实现专业级的同步能力。
- 查询文件同步任务，在同步任务启动后，如何实时掌握任务执行状态、监控同步进度、及时发现和处理异常，成为确保数据同步可靠性的关键。
- 终止文件同步任务，批量终止等待状态和正在运行状态的任务。

库名	支持平台	插件标识	说明
libldp_filesync.so	linux.x64、aarch64	libldp_filesync.so	文件同步库

4.33.1. filesync模块配置示例

```
{
  "Lib": "ldp_filesync",
  "Instance": 0,
  "Timer": 0,
  // 计算数据文件MD5值的并发进程数量，默认为 20，范围 [1,100]
  "MD5Concurrent": 20,
  // 传输完成后是否检查文件存在和校验文件md5值，默认为 true
  "CheckFiles": true,
  // 添加任务时是否创建目标目录，默认为 true
  "CreateDestDir": true
}
```

4.33.2. 配置项说明

名称	类型	说明	备注
MD5Concurrent	整数	计算数据文件MD5值的并发进程数量	默认值 20，范围 [1,100]
CheckFiles	布尔	传输完成后是否检查文件存在和校验文件md5值	默认值 true，检查文件存在并校验文件md5值
CreateDestDir	布尔	添加任务时是否创建目标目录	默认值 true，创建目标目录

5. 运维管控平台

本章主要介绍运维管控平台的安装、部署和监控

5.1. 运维管控平台安装

5.1.1. SEE2.0运维控制台安装

下载地址：<http://rdc.hundsun.com/product/download/see2.htm>

安装步骤可参看

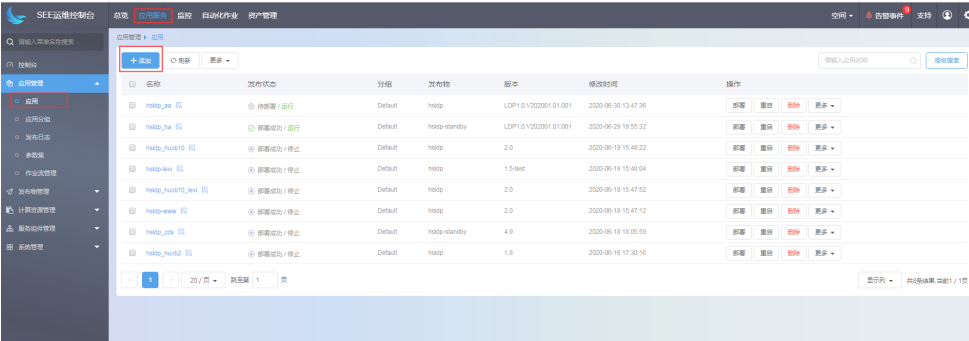
SEE2.0的安装手册 <http://rdcdocs.hundsun.com/pages/viewpage.action?pageId=13831426>

5.2. 部署

5.2.1. 发布物部署步骤

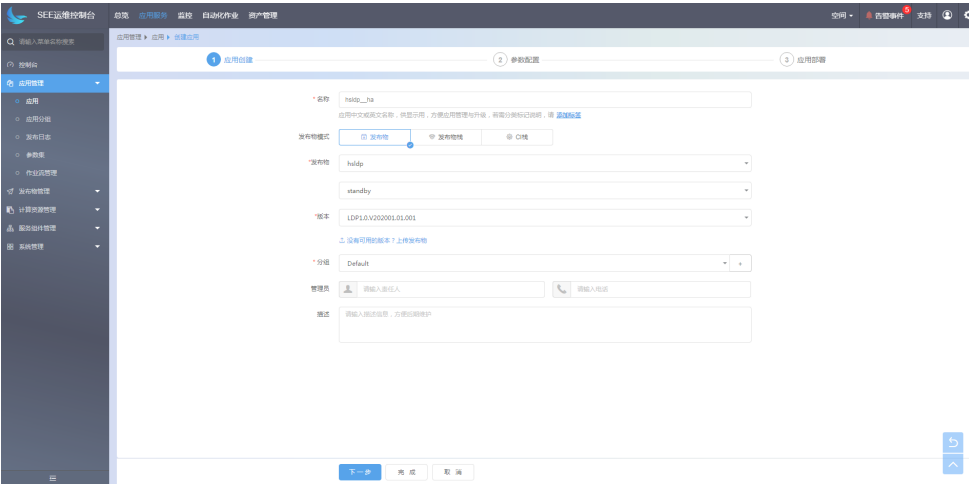
1. 添加应用

在应用服务-应用管理页面上点击添加。



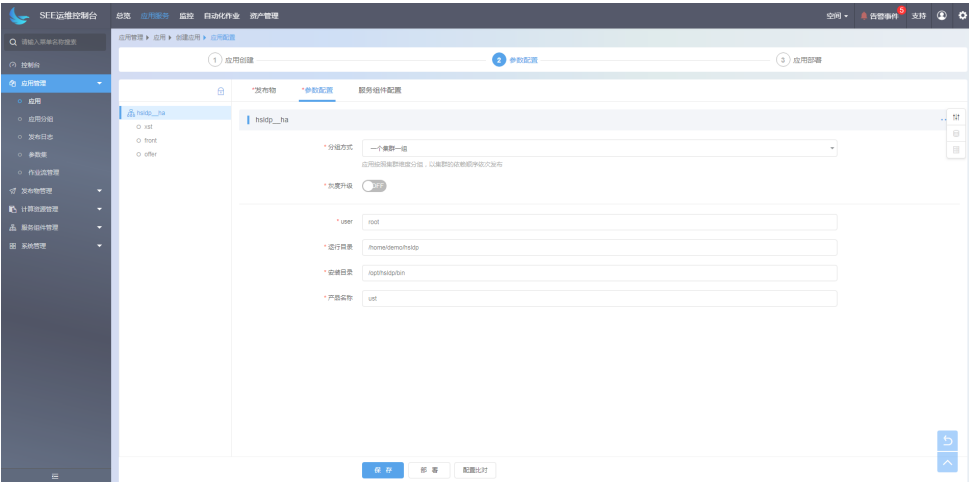
2. 应用创建

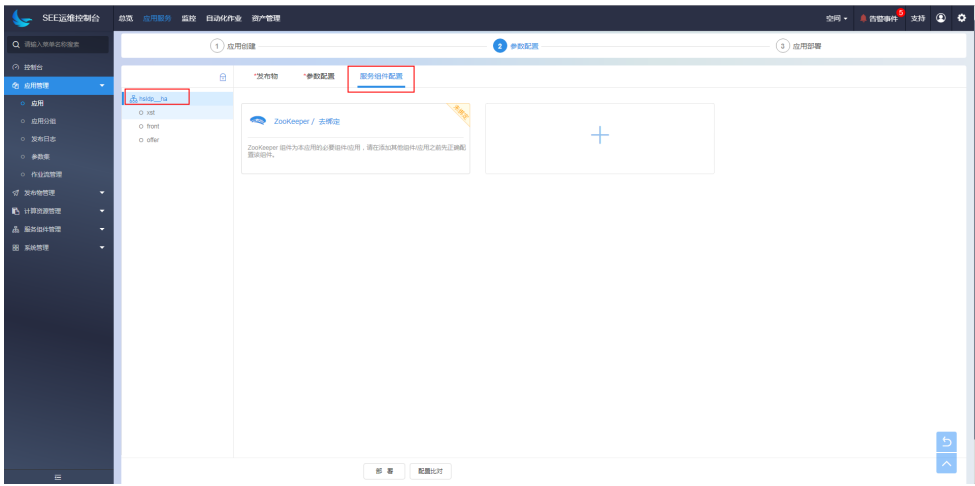
在应用创建页面上填写应用名称并选择发布物和对版本，点击下一步。



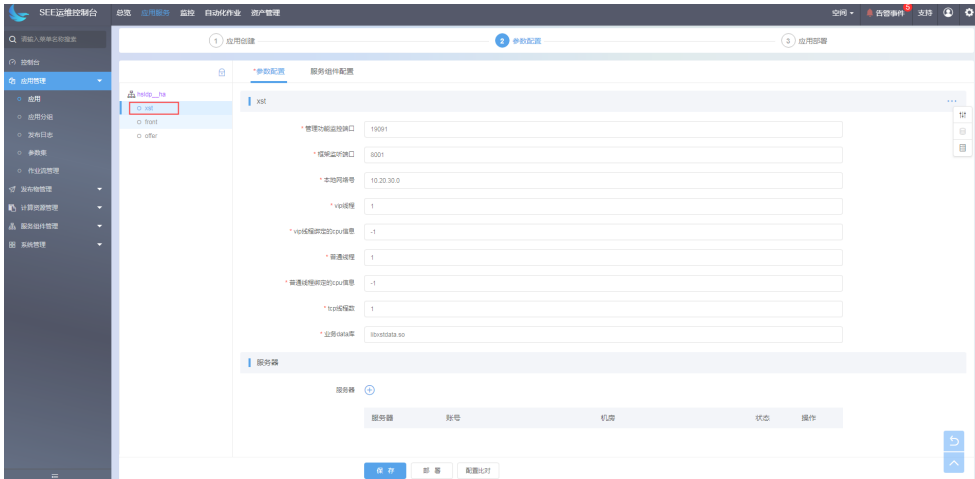
3. 应用配置

配置应用的参数和部署节点的参数，如部署的用户、安装运行目录、产品名称等。为了简化部署，运维监控平台屏蔽了一些很少需要改动的配置，若需要进行更详细的配置，可在发布物中的配置模版中进行修改。另外还需配置应用需要的zookeeper服务组件，运维管控平台会将ldp平台的配置自动写入绑定的zookeeper。

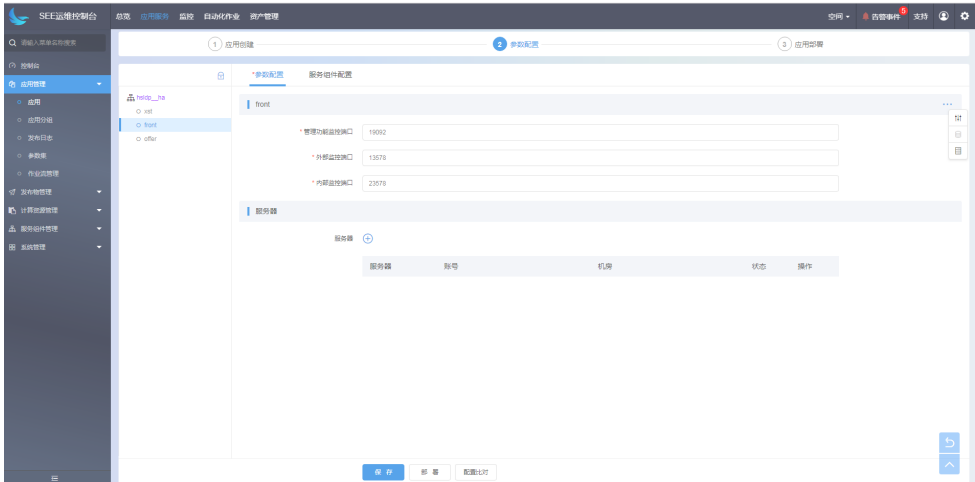




核心需要配置的参数如下，需要确保端口号不冲突，在页面下方需要选择部署节点的服务器。

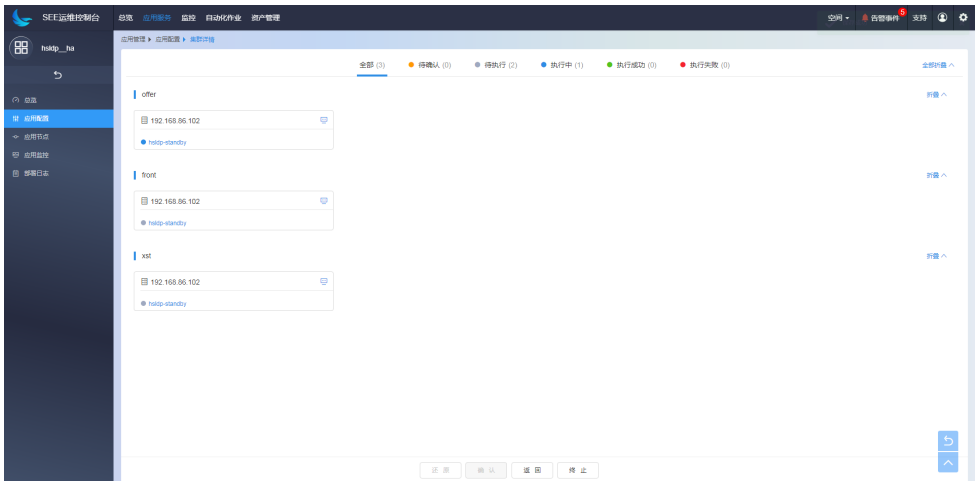


其中vip线程默认指PollingLoops配置为10000000的线程。即工作线程会轮询队列，连续轮询PollingLoops次（10000000次）以后，仍然没有请求，则工作线程睡眠SleepTimeUS微秒后再轮询队列，可通过发布物里的模版配置文件进行个性化修改。普通线程则表示默认PollingLoops配置等于0的线程，PollingLoops为0时，工作线程不再轮训队列，而是通过事件通知去从队列获取请求处理。前置配置参数页面：

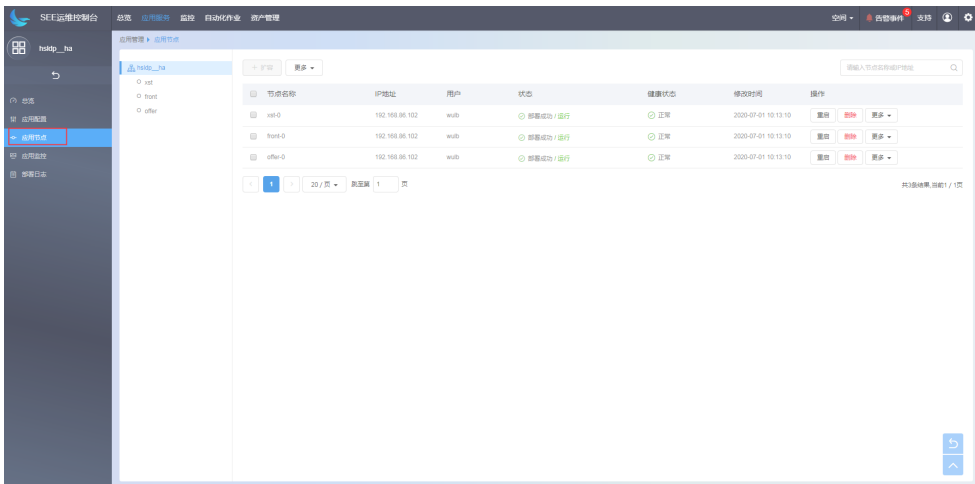


4.应用部署

各项参数配置完成后，即可开始部署。



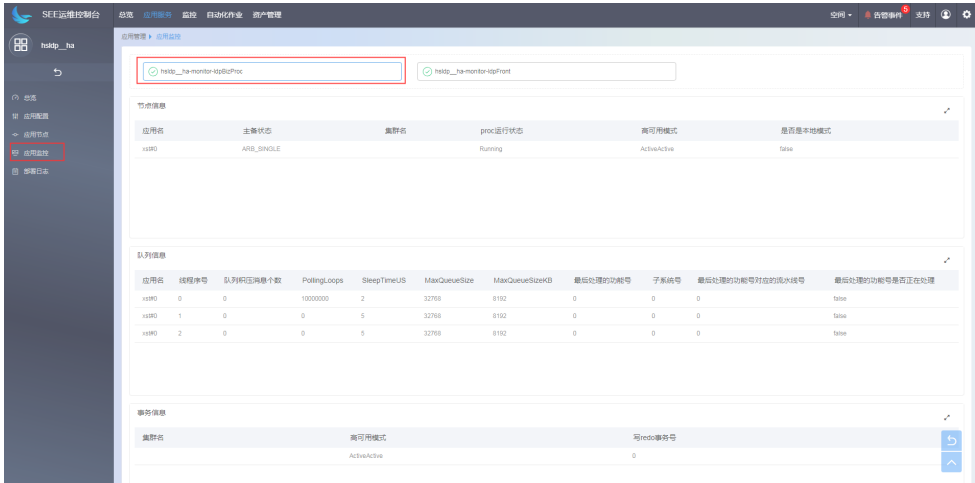
部署完成后，点击左侧菜单的应用节点即可查看每个部署节点的状态。



5.3. 监控

5.3.1. 通过应用服务查看监控

应用部署完成后会自动创建对应的监视器，在应用的应用监控菜单下可查看各个监视器对应的监控数据。

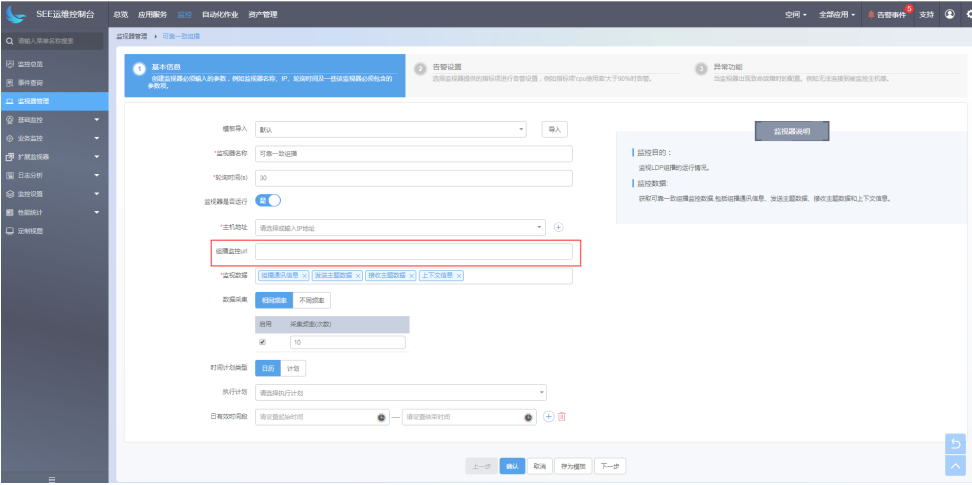


5.3.2. 通过手动创建监视器查看监控

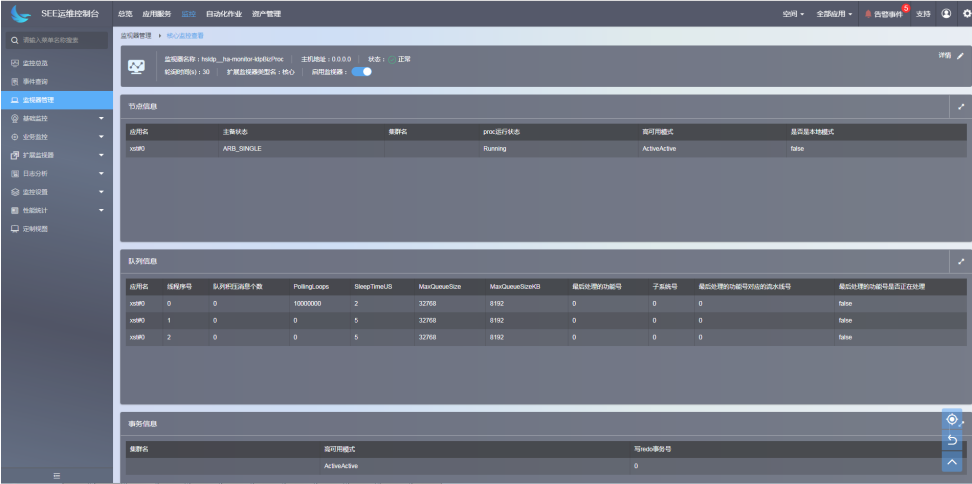
平台提供了可靠一致性组监视器、前置监视器、核心监视器三种基础监视器。



在监控-监视器管理页面添加需要的监视器，其中监控的url地址必填，url格式为http://ip:port/ldp，ip为部署节点的ip地址，端口为应用部署时的关联功能监控端口，组播和核心监视器若存在多个监控节点的url则需要使用分号隔开。主机地址目前暂时不使用，可任意填写，如0.0.0.0。



监视器配置完成后即可查看节点的监控数据。



6. 管理功能

本章主要是介绍各个插件的管理功能说明

6.1. 管理工具

1. web浏览器
2. ldp运维管控平台

6.2. 管理代理插件 ldp_mproxy

6.2.1. QueryMgrList查询插件管理功能列表

功能名称	QueryMgrList	版本号	V1.0.0.0	更新日期	20210610
功能备注					

功能说明	查询指定的插件提供的管理功能列表
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_mproxy&FuncName=QueryMgrList
入参说明	
plugin配置可以自由填充，实际设置的是需要查询的插件名。如果值为空，则表示查询节点加载的所有插件的管理功能列表。	
<pre>{ "/": "指定查询的插件名称", "plugin":"" }</pre>	
出参说明	
<pre>{ "ErrorNo": 0, "ErrorMsg": "", "Response": [{ "/": "插件名称", "PluginName": "", "/": "插件实例号", "InstanceNo": 1, "/": "插件的管理列表", "MgrList": ["f1","f2"] }, ...] }</pre>	
#ErrorNo表示返回码，0表示成功，其他表示错误，相应的错误信息由ErrorMsg字段描述	
#返回成功时，ErrorMsg字段和ErrorNo可不传	

6.2.2. GetConfigInfo 获取管理代理网络库的配置信息

功能名称	GetConfigInfo	版本号	V1.0.0.0	更新日期	20200630
功能备注					
功能说明	查询管理代理插件的配置信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_mproxy&FuncName=GetConfigInfo				
入参说明					
出参说明					
<pre>{ "ErrorNo": 0, "ErrorMsg": "", "Response": [{ "/": "监听端口", "ListenPorts": "8080", "/": "是否禁用Nagle算法", "EnableNagle": true, "/": "http连接处理线程数量", "NumThreads": 10, "/": "等待服务器操作系统接受的最大连接数", "ListenBackLog": 50, "/": "等待工作线程分派的可接受连接的最大数目", "ConnQueue": 50, "/": "允许客户端在后续HTTP请求中重用TCP连接，提高性能", "EnableKeepAlive": false, "/": "一个保持连接中两个请求之间的空闲超时，单位毫秒", "KeepAliveTimeoutMilli": "5000", "/": "错误日志文件的路径", "ErrLog": "error.log" }] }</pre>					

#ErrorNo表示返回码，0表示成功，其他表示错误，相应的错误信息由ErrorMsg字段描述

#返回成功时，ErrorMsg字段和ErrorNo可不传

#Response字段是应答具体信息，返回字段的具体含义请参考管理代理插件的配置项说明。

6.2.3. GetAppInfo 查询应用配置信息

功能名称	GetAppInfo	版本号	V1.0.0.0	更新日期	20200630
功能备注					
功能说明	查询应用配置信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_mproxy&FuncName=GetAppInfo				
入参说明					
出参说明					

```
{
  "State": 4, // 模块状态
  "Language": "zh_CN", // 语言
  "Charset": "GB2312", // 字符集
  "ProductName": "ldp", // 产品名
  "AppName": "wuyyp_xst#0", // 应用名+编号
  "ShortAppName": "wuyyp_xst", // 应用名
  "AppNo": 0, // 应用编号
  "HomeDirectory": "/home/wuyyp/ldp/workspace/bin", // Home目录
  "WorkingDirectory": "/home/wuyyp/ldp/workspace", // 工作目录
  "MessageDirectory": "/home/wuyyp/ldp/workspace/bin/message", // 消息目录
  "ComponentsDirectory": "/home/wuyyp/ldp/workspace/bin/components", // 组件目录
  "PluginsDirectory": "/home/wuyyp/ldp/workspace/bin/plugins", // 插件目录
  "AppDirectory": "/home/wuyyp/ldp/workspace/bin/wuyyp_xst.local", // 应用目录
  "AppModulesDirectory": "/home/wuyyp/ldp/workspace/bin/wuyyp_xst.local/modules", // 应用组件目录
  "ConfigFileName": "/home/wuyyp/ldp/workspace/ldp.json", // 配置文件
  "ZKConfigPath": "/ldp/ldp/config", // zk配置路径
  "AppInstance": "ldp-wuyyp_xst#0", // 应用实例
  "PidFile": "/var/tmp/ldp-wuyyp_xst#0.pid", // pid文件
  "ZoneName": "Zone1", // 机房名
  "ZoneType": 1, // 机房类型
  "ConfigMode": 0 // 配置类型 0: 配置从ZK上拉取 1: 配置从Keeper上拉取 2: 配置从本地读取
}
```

6.2.4. GetPluginList查询插件信息列表

功能名称	GetPluginList	版本号	V1.0.1.0	更新日期	20201016
功能备注					
功能说明	查询插件信息列表				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_mproxy&FuncName=GetPluginList				
入参说明					
出参说明					
<pre>{ "Response": [{ "PluginName": "ldp_mproxy", // 插件名称 "InstanceNo": 0, // 插件实例编号 "Version": "ldp_mproxy V1.0.1.0 Oct 16 2020 02:44:55" // 插件版本 }, ...] }</pre>					

6.2.5. GetThreadInfo查询所有注册线程的信息

功能名称	GetThreadInfo	版本号	V1.0.1.0	更新日期	20210905
功能备注	没有注册的线程，无线程信息。 不是通过注册绑核的线程，绑核信息无效。				
功能说明	查询所有注册线程的信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_mproxy&FuncName=GetThreadInfo				
入参说明					
出参说明					
<pre>{ "ThreadInfo": [{ "Owner": "ldp_bizproc", "Name": "bizproc_tcprun", "Priority": "Exclusive", "CpuNo": -1 }, { "Owner": "ldp_todb", "Name": "todb_time", "Priority": "Share", "CpuNo": 0 }] }</pre>					

6.2.6. GetAppsStatus查询所有插件状态

功能名称	GetAppsStatus	版本号	V1.0.1.0	更新日期	20230110
功能备注	每个插件需要向管理代理投递告警信息，并会在该管理功能展示。管理代理配置中"Timer"必须大于0				
功能说明	该管理功能只会展示收到的插件报警信息，如果当前插件报警全部消失，则Plugins下将删除该插件对象。"Status"为所有插件中等级最高的状态，等级从低到高依次为："OK","WARNING","PASUING","STARTING","ERROR"				

访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_mproxy&FuncName=GetAppsStatus
入参说明	
出参说明	
<pre>{ "Plugins": [{ "插件名": "ldp_front": [{ "子模块名": "Backend1", "SubModule": "Backend1", "告警等级": "ERROR", "MonitorLevel": "ERROR", "告警编号, 每一种报警编号不一样": "MonitorNo": 1, "告警描述": "Desc": "BackendId:1, is disconnected!" }, { "SubModule": "Backend2", "MonitorLevel": "WARNING", "MonitorNo": 2, "Desc": "BackendId:2 , some one connected!" }] }], "总状态": "Status": "ERROR" }</pre>	

6.2.7. GetAppJsonInfo查询应用实例配置

功能名称	GetAppJsonInfo	版本号	V1.0.9.1	更新日期	20230720
功能备注	应用实例配置				
功能说明	获取应用实例配置信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_mproxy&InstanceNo=0&FuncName=GetAppJsonInfo				
入参说明					
出参说明					

```
{
  "":"/":"应用实例配置信息，具体含义参考对应插件的配置说明",
  "AppInfo": {
    "Components": {
      "Libs": [],
      "GlobalLibs": [
        "tcp"
      ]
    },
    "Plugins": [
      {
        "Lib": "ldp_front",
        "Instance": 0,
        "Timer": 1,
        "Wan": {
          "Port": 23150
        },
        "Lan": {
          "Port": 23151
        },
        "Routes": [
          {
            "Functions": "**",
            "Backend": 1
          }
        ],
        "Backends": [
          {
            "Id": 1,
            "Type": "AS-TCP",
            "Sites": [
              {
                "Host": "192.168.86.20",
                "Port": 18111
              },
              {
                "Host": "192.168.86.102",
                "Port": 10001
              }
            ]
          }
        ]
      },
      {
        "Lib": "ldp_mproxy",
        "Instance": 0,
        "Timer": 0,
        "Network": {
          "ListenPorts": "9010"
        },
        "Performance": {
          "EnableKeepAlive": true,
          "KeepAliveTimeoutMilli": 5000
        }
      },
      {
        "Lib": "ldp_logproxy",
        "Instance": 0,
        "Timer": 0,
        "BizLogLevel": 1,
        "LogLevel": 1
      }
    ]
  }
}
```

6.2.8. GetSocketMgrInfo查询socket信息

功能名称	GetSocketMgrInfo	版本号	V1.0.0.0	更新日期	20240506
功能备注					
功能说明	查询socket信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_mproxy&FuncName=GetSocketMgrInfo				
入参说明					
<pre>{ }</pre>					
出参说明					

```
{
  "ErrorNo": 0,
  "ErrorMsg": "",
  "SocketInfo": [
    {
      "/*": "socket文件描述符",
      "Id":1,
      "/*": "加速标志",
      "AccFlag":"false",
      "/*": "加速堆栈名",
      "AccStackName":1,
      "/*": "socket创建者名",
      "Owner":"","
      "/*": "线程号",
      "ThreadId":1,
      "/*": "socket类型",
      "Type":"Tcp",
      "/*": "本地IP",
      "LocalIp":"192.168.86.101",
      "/*": "本地端口",
      "LocalPort":8999,
      "/*": "远程IP",
      "RemoteIp":"192.168.86.102",
      "/*": "远程端口",
      "RemotePort":8999
    },
    ...
  ]
}
```

#ErrorNo表示返回码，0表示成功，其他表示错误，相应的错误信息由ErrorMsg字段描述
#返回成功时，ErrorMsg字段和ErrorNo可不传

6.2.9. GetStatsInfo查询管理功能统计信息

功能名称	GetStatsInfo	版本号	V1.0.0.0	更新日期	20240506
功能备注					
功能说明	查询管理功能统计信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_mproxy&FuncName=GetStatsInfo				
入参说明					
{ }					
出参说明					
{ "ErrorNo": 0, "ErrorMsg": "", "Response": [{ "//": "请求数量", "reqNum":1, "//": "应答数量", "ansNum":1, "//": "错误应答数量", "errAnsNum":1, "//": "每秒吞吐量", "iThroughOut":0 }, ...] }					
#ErrorNo表示返回码，0表示成功，其他表示错误，相应的错误信息由ErrorMsg字段描述					
#返回成功时，ErrorMsg字段和ErrorNo可不传					

6.3. 日志代理插件 ldp_logproxy

6.3.1. SetLogLevel设置日志级别

功能名称	SetLogLevel	版本号	V1.0.0.0	更新日期	20200630
功能备注	日志级别				
功能说明	设置日志级别				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_logproxy&FuncName=SetLogLevel				
入参说明					
<pre>{ "/*": "设置日志级别，可只写一个，LogLevel：平台日志级别，支持输入字符串整数0-5，分别对应DEBUG-FATAL，枚举值可参考对应配置项说明", "LogLevel":nLevel, "/*": "BizLogLevel：业务日志级别，入参值同上LogLevel", "BizLogLevel":nLevel, "/*": "TSPointLevel：时延点位等级，支持输入字符串整数0-3", "TSPointLevel": nLevel }</pre>					
出参说明					

```
{
  "///": "返回设置的日志级别或者set failed",
  "LogLevel":nLevel,
  "BizLogLevel":nLevel,
  "TSPointLevel": nLevel
}
```

6.3.2. GetLogLevel获取日志级别

功能名称	GetLogLevel	版本号	V1.0.0.0	更新日期	20200630
功能备注	日志级别				
功能说明	获取日志级别				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetLogLevel				
入参说明					
无					
出参说明					
<pre>{ "///": "平台日志级别", "LogLevel":nLevel, "///": "业务日志级别", "BizLogLevel":nLevel, "///": "点位等级级别", "TSPointLevel": nLevel}</pre>					

6.3.3. GetMonitor监控日志

功能名称	GetMonitor	版本号	V1.0.0.0	更新日期	20210322
功能备注	监控日志				
功能说明	监控日志，综合输出线程、队列、各类内存池状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetMonitor				
入参说明					
无					
出参说明					

```

{
    "":"/": "同步模式",
    "SynMode": false,
    "":"/": "异步模式下Run线程循环次数",
    "RunThreadLoop": 1000,
    "":"/": "允许生成的日志类型，二进制每位代表一种类型",
    "LogType": 15,
    "":"/": "单个最大文件大小",
    "MaxFileSize": 10,
    "":"/": "保留最大天数",
    "MaxDays": 7,
    "":"/": "最大文件总量",
    "MaxTotalSize": 100,
    "":"/": "队列最大容量",
    "MaxQueueSize": 10240,
    "":"/": "队列已使用容量",
    "CurQueueSize": 0,
    "":"/": "队列剩余容量",
    "FreeQueueSize": 10240,
    "":"/": "timelog对象获取失败的次数",
    "GetTimeLogFailedCount": 0,
    "":"/": "traceidlog对象获取失败的次数",
    "GetTraceIdLogFailedCount": 0,
    "":"/": "内存池内存申请失败的次数",
    "GetMallocFailedCount": 0,
    "":"/": "入队列失败的次数",
    "GetPushQueueFailedCount": 0,
    "":"/": "使用过的日志内存池块数量",
    "MemPoolUsedBlockCount": 100,
    "":"/": "释放掉的日志内存池块数量",
    "MemPoolFreedBlockCount": 99,
    "":"/": "丢弃时延日志数量",
    "TimeLogDropCnt": 0,
    "":"/": "正在使用的日志内存池块的情况",
    "MemPoolBlocksInfo": [
        {
            "":"/": "内存池块编号",
            "BlockId": 0,
            "":"/": "内存池块要释放到的位置",
            "ToFreePos": 8,
            "":"/": "内存池块当前写的位置",
            "CurPos": 8
        }
    ],
    "":"/": "使用过的抓包内存池块数量",
    "MsglogMemPoolUsedBlockCount": 1,
    "":"/": "释放掉的抓包内存池块数量",
    "MsglogMemPoolFreedBlockCount": 0,
    "":"/": "正在使用的抓包内存池块的情况",
    "MsglogMemPoolBlocksInfo": [
        {
            "":"/": "内存池块编号",
            "BlockId": 0,
            "":"/": "内存池块要释放到的位置",
            "ToFreePos": 8,
            "":"/": "内存池块当前写的位置",
            "CurPos": 8
        }
    ],
    "":"/": "时延对象池预留的块数量",
    "TimeStampPoolReserveBlockCount": 4,
    "":"/": "时延对象池正在使用的块数量",
    "TimeStampPoolUsedBlockCount": 1,
    "":"/": "时延对象池申请过的块数量",
    "TimeStampPoolMallocBlockCount": 67,
    "":"/": "时延对象池释放掉的块数量",
    "TimeStampPoolFreeBlockCount": 66,
    "":"/": "时延对象池当前剩余数量大小",
    "TimeStampPoolCurrElementSize": 1024,
    "":"/": "时延对象池内部块使用情况",
    "TimeStampPoolUsedBlocksInfo": [
        {
            "":"/": "块编号",
            "BlockId": 66,
            "":"/": "块中剩余的数量",
            "CurrSize": 1024
        }
    ],
    "":"/": "TraceIDLog对象池，字段含义同时延对象池",
    "TraceIDLogPoolReserveBlockCount": 3,
    "TraceIDLogPoolUsedBlockCount": 1,
    "TraceIDLogPoolMallocBlockCount": 1,
    "TraceIDLogPoolFreeBlockCount": 0,
    "TraceIDLogPoolCurrElementSize": 1024,
    "TraceIDLogPoolUsedBlocksInfo": [
        {
            "BlockId": 0,
            "CurrSize": 1024
        }
    ],
    "":"/": "时延对象打点数量超过256时，保存打点信息的池，字段含义同时延对象池",
    "TimeInfoPoolReserveBlockCount": 3,
    "TimeInfoPoolUsedBlockCount": 1,
    "TimeInfoPoolMallocBlockCount": 1,
    "TimeInfoPoolFreeBlockCount": 0,
    "TimeInfoPoolCurrElementSize": 1024,
    "TimeInfoPoolUsedBlocksInfo": [
        {
            "BlockId": 0,
            "CurrSize": 1024
        }
    ]
}

```

6.3.4. GetLogMemPoolMonitor 平台日志内存池监控

功能名称	GetLogMemPoolMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	平台日志内存池（m_lpLogMemPool）监控				

功能说明	仅输出平台日志内存池的块使用情况，字段与 GetMonitor 中 MemPool* 部分一致
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetLogMemPoolMonitor
入参说明	
无	
出参说明	
<pre>{ "MemPoolUsedBlockCount": 100, "MemPoolFreedBlockCount": 99, "MemPoolBlocksInfo": [{ "BlockId": 0, "ToFreePos": 8, "CurPos": 8 }] }</pre>	

6.3.5. GetOneLogMemPooMonitor 队列元素内存池监控

功能名称	GetOneLogMemPooMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	队列元素 OneLog 内存池（m_lpOneLogMemPool）监控				
功能说明	仅输出 OneLog 内存池块使用情况，字段与 GetMonitor 中 OneLogMemPool* 一致				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetOneLogMemPooMonitor				
入参说明					
无					
出参说明					
<pre>{ "OneLogMemPoolUsedBlockCount": 1, "OneLogMemPoolFreedBlockCount": 0, "OneLogMemPoolBlocksInfo": [{ "BlockId": 0, "ToFreePos": 16, "CurPos": 16 }] }</pre>					

6.3.6. GetTimeLogMemPooMonitor 时延日志内存池监控

功能名称	GetTimeLogMemPooMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	时延专属 TimeLog 内存池（m_lpTimeLogMemPool）监控				
功能说明	仅输出 TimeLog 内存池块使用情况，字段与 GetMonitor 中 TimeLogMemPool* 一致				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetTimeLogMemPooMonitor				
入参说明					
无					
出参说明					
<pre>{ "TimeLogMemPoolUsedBlockCount": 0, "TimeLogMemPoolFreedBlockCount": 0, "TimeLogMemPoolBlocksInfo": [{ "BlockId": 0, "ToFreePos": 8, "CurPos": 8 }] }</pre>					

6.3.7. GetMsgLogMemPooMonitor 抓包内存池监控

功能名称	GetMsgLogMemPooMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	抓包 MsgLog 内存池（m_MsgLogMemPool）监控				
功能说明	仅输出 Msglog 内存池块使用情况，字段与 GetMonitor 中 MsglogMemPool* 一致				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetMsgLogMemPooMonitor				
入参说明					
无					
出参说明					

```
{
  "MsgLogMemPoolUsedBlockCount": 1,
  "MsgLogMemPoolFreedBlockCount": 0,
  "MsgLogMemPoolBlocksInfo": [
    {
      "BlockId": 0,
      "ToFreePos": 8,
      "CurPos": 8
    }
  ]
}
```

6.3.8. GetBinLogMemPoolMonitor 二进制日志内存池监控

功能名称	GetBinLogMemPoolMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	二进制 BinLog 内存池（m_lpBinLogMemPool）监控				
功能说明	仅输出 BinLog 内存池块使用情况，字段与 GetMonitor 中 BinLogMemPool* 一致				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetBinLogMemPoolMonitor				
入参说明					
无					
出参说明					
<pre>{ "BinLogMemPoolUsedBlockCount": 0, "BinLogMemPoolFreedBlockCount": 0, "BinLogMemPoolBlocksInfo": [{ "BlockId": 0, "ToFreePos": 8, "CurPos": 8 }] }</pre>					

6.3.9. GetTimeStampPoolMonitor 时延对象池监控

功能名称	GetTimeStampPoolMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	时延对象池（m_TimeStampPool）监控				
功能说明	单独查看时延对象池使用情况，字段含义同 GetMonitor 中 TimeStampPool*				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetTimeStampPoolMonitor				
入参说明					
无					
出参说明					
<pre>{ "TimeStampPoolReserveBlockCount": 4, "TimeStampPoolUsedBlockCount": 1, "TimeStampPoolMallocBlockCount": 67, "TimeStampPoolFreeBlockCount": 66, "CurrElementSize": 1024, "TimeStampPoolUsedBlocksInfo": [{ "BlockId": 66, "CurrSize": 1024 }] }</pre>					

6.3.10. GetTraceIDLogPoolMonitor TraceIDLog对象池监控

功能名称	GetTraceIDLogPoolMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	TraceIDLog 对象池（m_TraceIDLogPool）监控				
功能说明	单独查看 TraceIDLog 对象池使用情况，字段含义同 GetMonitor 中 TraceIDLogPool*				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetTraceIDLogPoolMonitor				
入参说明					
无					
出参说明					

```
{
  "TraceIDLogPoolReserveBlockCount": 3,
  "TraceIDLogPoolUsedBlockCount": 1,
  "TraceIDLogPoolMallocBlockCount": 1,
  "TraceIDLogPoolFreeBlockCount": 0,
  "CurrElementSize": 1024,
  "TraceIDLogPoolUsedBlocksInfo": [
    {
      "BlockId": 0,
      "CurrSize": 1024
    }
  ]
}
```

6.3.11. GetTimeInfoPoolMonitor 时延打点信息池监控

功能名称	GetTimeInfoPoolMonitor	版本号	V1.0.0.0	更新日期	20260415
功能备注	TimeInfoPool: 时延打点信息池监控				
功能说明	单独查看 TimeInfoPool 使用情况，字段含义同 GetMonitor 中 TimeInfoPool*				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetTimeInfoPoolMonitor				
入参说明					
无					
出参说明					
<pre>{ "TimeInfoPoolReserveBlockCount": 3, "TimeInfoPoolUsedBlockCount": 1, "TimeInfoPoolMallocBlockCount": 1, "TimeInfoPoolFreeBlockCount": 0, "TimeInfoPoolCurrElementSize": 1024, "TimeInfoPoolUsedBlocksInfo": [{ "BlockId": 0, "CurrSize": 1024 }] }</pre>					

6.3.12. GetPktCapPlugins 获取支持抓包的插件

功能名称	GetPktCapPlugins	版本号	V1.0.4.6	更新日期	20220221
功能备注	获取支持抓包的插件				
功能说明	获取支持抓包的插件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetPktCapPlugins				
入参说明					
无					
出参说明					
<pre>{ "Plugins": [{ "PluginName": "ldp_front", "Instance": 0, "LogFlag": false }, { "PluginName": "ldp_bizproc", "Instance": 0, "LogFlag": false }] }</pre>					

6.3.13. EnableMsgLog 开启/关闭抓包

功能名称	EnableMsgLog	版本号	V1.0.0.0	更新日期	20260119
功能备注	开启/关闭抓包				
功能说明	开启/关闭抓包				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=EnableMsgLog&SettingPluginName=kdp_front&SettingInstanceNo=0&LogFlag=1				
入参说明					


```
{
  "/*": "设置抓包的插件名，目前支持前置和核心",
  "SettingPluginName": "ldp_front",

  {
    "/*": "错误号，0表示成功",
    "ErrorNo": 0,
    "/*": "错误信息",
    "ErrorMsg": "",
    "/*": "抓包开关状态",
    "LogFlag": true
  }
}
```

6.3.14. EnableMsgLogList批量开启/关闭抓包

功能名称	EnableMsgLogList	版本号	V1.0.4.6	更新日期	20220221
功能备注	批量开启/关闭抓包				
功能说明	批量开启/关闭抓包				
访问链接	http://ipport/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=EnableMsgLogList&SettingPlugins=ldp_bizproc,ldp_front.1,ldp.&LogFlag=1				
入参说明					
<pre>{ "/*/*": "设置抓包的 插件名.实例编号 ， 以 ',' 隔开，实例编号不传默认为0", "SettingPlugins": "ldp_bizproc,ldp_front,ldp_bizproc.", "/*/*": "抓包开关，1是开启，0是关闭，不传此参数默认为0", "LogFlag": "0"}</pre>					
出参说明					
<pre>{ "Status": [{ "/*/*": "插件名", "Plugin": "ldp_bizproc", "/*/*": "实例编号", "Instance": 0, "/*/*": "开启状态，开启为true，关闭为false", "LogFlag": false }, { "/*/*": "出错时的错误号", "ErrorNo": 808, "/*/*": "错误信息", "ErrorMsg": "插件 ldp_front.1 不存在" }, { "ErrorNo": 102, "ErrorMsg": "无效的输入参数 (ParseParam=ldp.)" }], "/*/*": "最后一次出错的错误号，成功则为0", "ErrorNo": 102,}</pre>					

6.3.15. GetMsgLogFlag查看插件是否开启抓包

功能名称	GetMsgLogFlag	版本号	V1.0.0.0	更新日期	20210514
功能备注	查看插件是否开启抓包				
功能说明	查看插件是否开启抓包				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=GetMsgLogFlag				
入参说明					
无					
出参说明					
<pre>{ "Plugins": [{ "//": "插件名", "PluginName": "ldp_front", "//": "插件实例编号", "InstanceNo": 0, "//": "是否开启抓包, true是开启, false是关闭", "LogFlag": true }] }</pre>					

6.3.16. GetMsgLog获取缓存的抓包数据

功能名称	GetMsgLog	版本号	V1.0.0.0	更新日期	20220309
功能备注	获取抓包缓存数据，仅缓存最新的1000条抓包数据				
功能说明	获取抓包缓存数据，设置参数[StartIdx,Count]或Count，只设置Count时，默认获取抓包缓存数据前Count条。				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=GetMsgLog&StartIdx=1&Count=1				
入参说明					
<pre>{ </pre>					

```
{
  "Message": [
    {
      "/*/*": "自开启抓包功能的第几条",
      "Index": 1,
      "/*/*": "插件名",
      "PluginName": "ldp_front",
      "/*/*": "时间",
      "Time": "0803 134022.559444",
      "/*/*": "附加说明信息",
      "Remark": "FrontOnMessage",
      "/*/*": "头部字段参考ldpmsg头部定义",
      "MsgLen": "55",
      "FunctionID": "900003",
      "IsUstMsg": "False",
      "IsDelayAnalysis": "False",
      "IsCompress": "False",
      "IsTestOnly": "False",
      "MsgType": "Response",
      "Dest": "ToMaster",
      "ExtHeadLength": "0",
      "NodeNo": "0",
      "FixedSize": "2",
      "SystemNo": "0",
      "Token": "0",
      "ConnectionID": "1073807361",
      "UserDefined": "1",
      "/*/*": "可选头"
      "OptionalHead": {
        "/*/*": "一个对象为一个tag",
        "1": {
          "/*/*": "index和对应的值",
          "1": 10022
        },
        "5": {
          "/*/*": "二进制base64编码后的二进制数据",
          "0": "MTIzNAA=",
          "/*/*": "是否是二进制数据",
          "IsBinary": true
        }
      },
      "/*/*": "base64编码后的ldpmsg",
      "Data": "NwAAAKO7DQAABAAAAAgAAAAAAAAAAAAAAAAQAABQAEAAABbXf/1DwAAAXwH/AQLMnAEFBwoxMjM0AA=="
    }
  ],
  "/*/*": "缓存中的起始抓包数据编号",
  "StartIdx": 1,
  "/*/*": "缓存总量",
  "TotalSize": 3
}
```

6.3.17. ClearMsgLog清空抓包缓存

功能名称	ClearMsgLog	版本号	V1.0.0.0	更新日期	20210826
功能备注	清空抓包缓存				
功能说明	清空抓包缓存				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=ClearMsgLog				
入参说明					
无					
出参说明					
无					

6.3.18. SetFilter设置过滤条件

功能名称	SetFilter	版本号	V1.0.0.0	更新日期	20210826
功能备注	设置过滤条件				
功能说明	设置过滤条件，可多次设置，过滤条件项为或的关系				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=SetFilter&Functions=*				
入参说明					
<pre>{ "/*/*": "设置功能号过滤条件，支持？ *的模糊匹配", "Functions": "512000;513?01;*" }</pre>					
出参说明					
<pre>{ "/*/*": "错误号，0表示成功", "ErrorNo": 0, "/*/*": "错误信息", "ErrorMsg": "无效的输入参数" }</pre>					

6.3.19. GetFilter查询过滤条件

功能名称	GetFilter	版本号	V1.0.0.0	更新日期	20210826
功能备注	查询过滤条件				
功能说明	查询过滤条件				

访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetFilter
入参说明	
无	
出参说明	
<pre>{ "/*": "已经设置的功能号过滤条件", "Filter": ["5;102;10??;5?", "*"] }</pre>	

6.3.20. DeleteFilter删除过滤条件

功能名称	DeleteFilter	版本号	V1.0.0.0	更新日期	20210826
功能备注	删除过滤条件				
功能说明	删除过滤条件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=DeleteFilter&FilterIdx=1				
入参说明					
<pre>{ "/*": "要删除的过滤条件项，编号从1开始，为0则全部删除", "FilterIdx": "1" }</pre>					
出参说明					
<pre>{ "/*": "错误号，0表示成功", "ErrorNo": 0, "/*": "错误信息", "ErrorMsg": "无效的输入参数" }</pre>					

6.3.21. SetLogMerge设置日志合并

功能名称	SetLogMerge	版本号	V1.0.0.0	更新日期	20211111
功能备注	设置日志合并				
功能说明	设置日志合并				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=SetLogMerge				
入参说明					
<pre>{ "/*": "设置是否启用日志合并功能，默认启用，填false则暂停日志合并功能", "LogMergeStat": true }</pre>					
出参说明					
<pre>{ "Rsp": { "/*": "当前log日志合并功能状态，true表示启用", "SysMergeStat": true, "/*": "当前bizlog日志合并功能状态，true表示启用", "BizMergeStat": true } }</pre>					

6.3.22. GetLogMerge获取日志合并

功能名称	GetLogMerge	版本号	V1.0.0.0	更新日期	20211111
功能备注	获取日志合并				
功能说明	获取日志合并				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetLogMerge				
入参说明					
无					
出参说明					

<pre>{ "Stat": { "/*": "当前log日志合并功能状态, true表示启用", "SysMergeStat": true, "/*": "当前bizlog日志合并功能状态, true表示启用", "BizMergeStat": true } }</pre>
--

6.3.23. QueryFileData查询日志文件内容

功能名称	QueryFileData	版本号	V1.0.0.0	更新日期	20220209
功能备注	查询日志文件内容				
功能说明	查询日志文件内容，支持功能号过滤				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=QueryFileData&FileName=20220209-hsldp-kdp-front.msglog&StartIdx=1&Count=5&FunctionID=5;102;103				
入参说明					
<pre>{ "/*": "查询的日志文件名，目前只支持msglog类型的日志文件", "FileName": "20220209-hsldp-kdp-front.msglog", "/*": "查询范围的起始编号，当起始范围不在文件数据的编号范围，则输入参数无效", "StartIdx": "1", "/*": "查询指定数量的数据，为查询范围中编号最小的Count个", "Count": "5", "/*": "过滤的功能号，多个功能号以英文分号隔开", "FunctionID": "5;102;103", }</pre>					
出参说明					
<pre>{ "Message": [{ "/*": "自开启抓包功能的第几条", "Index": 1, "/*": "插件名", "PluginName": "ldp_front", "/*": "时间", "Time": "0803 134022.559444", "/*": "附加说明信息", "Remark": "FrontOnMessage", "/*": "头部字段参考ldpmsg头部定义", "MsgLen": "55", "FunctionID": "900003", "IsUstMsg": "False", "IsDelayAnalysis": "False", "IsCompress": "False", "IsTestOnly": "False", "MsgType": "Response", "Dest": "ToMaster", "ExtHeadLength": "0", "NodeNo": "0", "FixedSize": "2", "SystemNo": "0", "Token": "0", "ConnectionID": "1073807361", "UserDefined": "1", "/*": "可选头" "OptionalHead": { "/*": "一个对象为一个tag", "1": { "/*": "index和对应的值", "1": 10022 }, "5": { "/*": "二进制base64编码后的二进制数据", "0": "MTIzNAA=", "/*": "是否是二进制数据", "IsBinary": true } }, "/*": "base64编码后的ldpmsg", "Data": "NwAAAKO7DQABAAAAAaGAAAAAAAAAAAAAAAAQAABQAEAAAABbXf/1DwAAAXwH/AQLMnAEFBwoxMjM0AA==" }], "/*": "文件中的起始抓包数据编号", "StartIdx": 1, "/*": "符合条件数据的总量", "TotalSize": 4 }</pre>					

6.3.24. QueryFileList查询日志文件列表

功能名称	QueryFileList	版本号	V1.0.0.0	更新日期	20220209
功能备注	查询日志文件列表				
功能说明	查询当前应用下的日志文件列表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=QueryFileList&FileType=log				
入参说明					
<pre>{ "/*": "查询日志文件的类型， 包括log,bizlog,timelog,traceidlog,msglog", "FileType": "log", }</pre>					

出参说明
<pre>{ "FileList": ["20220209-hsldp-ldp_aa-front.log"] }</pre>

6.3.25. SetEnableTimeLogTag 设置日志tag输出

功能名称	SetEnableTimeLogTag	版本号	V1.0.0.0	更新日期	20260119
功能备注					
功能说明	设置日志tag输出				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=SetEnableTimeLogTag				
入参说明					
<pre>{ "":"/": "0: 表示关闭 其他数值: 表示开启 , 注意请尽量使用数字型, 字母等会按0处理", "Enable": "1", }</pre>					
出参说明					
<pre>{ "EnableTimeLogTag": true }</pre>					

6.3.26. GetEnableTimeLogTag 查看日志tag设置信息

功能名称	GetEnableTimeLogTag	版本号	V1.0.0.0	更新日期	20240627
功能备注					
功能说明	查看日志tag设置信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=GetEnableTimeLogTag				
入参说明					
无					
出参说明					
<pre>{ "EnableTimeLogTag": true }</pre>					

6.3.27. GetTimeLogFilter 查看时延日志过滤信息

功能名称	GetTimeLogFilter	版本号	V1.0.0.0	更新日期	20240627
功能备注					
功能说明	查看时延日志过滤信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_logproxy&FuncName=GetTimeLogFilter				
入参说明					
无					
出参说明					
<pre>{ "Filter": [{ "Functions": "101;102;301;1??", //功能号列表 "ServiceName": "ldp_bizproc;ldp_front", //插件名列表 "TName": "bizworker-1;bizproc_work_2;", //线程名称列表 "Type": "", //消息类型未配置 "ETimeThold": 500, //执行耗时 "QcntThold": 0 //队列积压数量 }, { "Functions": "*", "ServiceName": "*", "TName": "*", "Type": "", "ETimeThold": 500, //单位 ns "QcntThold": 10 }] }</pre>					

6.3.28. GetLogPath 获取日志文件存储路径

功能名称	GetLogPath	版本号	V1.0.0.0	更新日期	20251017

功能备注	
功能说明	获取日志文件存储路径
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetLogPath
入参说明	
无	
出参说明	
如果是相对路径则显示的是workspace路径拼接上填写的相对路径（即原先Log配置的路径为"./testlog"）变为绝对路径。	
<pre>{ "BinLog": "/home/lijianan/Sources/bin/log/", "BizLog": "/home/lijianan/Sources/bin/log/", "Log": "/home/lijianan/Sources/bin/../testlog/", "MsgLog": "/home/lijianan/Sources/bin/log/", "TimeLog": "/home/lijianan/Sources/bin/log/", "TraceIDLog": "/home/lijianan/Sources/bin/log/" }</pre>	

6.3.29. QueryMaskConfig 查询脱敏配置规则

功能名称	QueryMaskConfig	版本号	V1.0.0.0	更新日期	20251017
功能备注					
功能说明	查询脱敏配置规则				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=QueryMaskConfig				
入参说明					
无					
出参说明					
<pre>{ "version": "1.0", //配置文件版本号 "enabled": true, //是否启用脱敏功能 "default_mask": "*", //默认待替换成的值 "rules": [//脱敏规则数组 { "path": ["secret_*"], //脱敏匹配字段 "type": "HIDE" //脱敏类型 }]}</pre>					

6.3.30. UpdateMaskConfig 更新脱敏配置规则

功能名称	UpdateMaskConfig	版本号	V1.0.0.0	更新日期	20251017
功能备注					
功能说明	更新脱敏配置规则				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=UpdateMaskConfig				

入参说明

```
{  "//": "脱敏所需的配置文件或路径,文件后缀要求命名为.json",  "MaskProcessorConfigPath": "mask.json"}
```

出参说明

```
{  "//": "更新脱敏配置规则结果",  "Result": "Success"}
```

6.3.31. GetTsMonitor 获取打点监控信息

功能名称	GetTsMonitor	版本号	V1.0.0.0	更新日期	20251023
功能备注					
功能说明	获取打点监控信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetTsMonitor				
入参说明					
无					
出参说明					

```
{
    "///<": "事件通知阈值",
    "NotifySize": 12,
    "///<": "事件通知次数",
    "NotifyTimes": 0,
    "///<": "无锁中队列未被消费的总数",
    "CurTsCount": 0,
    "///<": "队列丢弃阈值",
    "QueueDropThreshold": 103,
    "///<": "时延打点内存池",
    "TimeInfoPool": {
        "///<": "时延对象池当前剩余数量大小",
        "ElementSize": 1024,
        "///<": "时延对象池正在使用的块数量",
        "UsedBlockSize": 1,
        "///<": "时延对象池预留的块数量",
        "ReserveBlockSize": 3,
        "///<": "时延对象池申请过的块数量",
        "MallocBlockCount": 1,
        "///<": "时延对象池释放掉的块数量",
        "FreeBlockCount": 0,
    }
},
    "///<": "内存申请长度（字节）",
    "MemTsMalloc": 28448,
    "///<": "内存回收长度（字节）",
    "MemTsRelease": 28448,
    "dlylog_ts_copy": {
        "///<": "绑核信息",
        "BindCpuNo": -1,
        "///<": "线程等待时长（毫秒）",
        "WaitMs": 1,
        "///<": "时延总数",
        "TsTotalCount": 4,
        "///<": "拷贝线程处理总个数",
        "TsCopyTotalCount": 4,
        "///<": "拷贝线程丢弃总个数",
        "TsCopyDropTotalCount": 0,
        "///<": "拷贝操作完成率",
        "TsCopyPercents(%)": 100.0,
    },
    "dlylog_ts_write": {
        "///<": "绑核信息",
        "BindCpuNo": -1,
        "///<": "线程等待时长（毫秒）",
        "WaitMs": 1,
        "///<": "时延写入个数",
        "TimeStampWriteCount": 4,
        "///<": "时延写入完成率",
        "TsWritePercents(%)": 100.0,
    },
    "WriteTsQueue": {
        "///<": "当前队列大小",
        "CurSize": 0,
        "///<": "最大积压总数",
        "HisMaxSize": 2,
        "///<": "历史积压总数（不代表时延个数）",
        "TotalCount": 2,
        "///<": "放入队列失败次数",
        "PushFailedCount": 0,
        "///<": "队列最大长度",
        "MaxQueueSize": 10240,
    }
}
```

6.3.32. GetLockMonitor 获取锁监控信息

功能名称	GetLockMonitor	版本号	V1.0.0.0	更新日期	20260414
功能备注					
功能说明	获取 logproxy 内部锁与内存池锁的统计信息，并返回当前锁相关配置项的生效值（便于核对运行时行为）				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetLockMonitor				
入参说明					
无					
出参说明					


```
{
  "":"/: "锁策略相关配置项（返回的是当前运行时生效值），含义同 plugins/log_proxy.adoc 中同名配置项",
  "":"/: "LockMode: 锁/队列模式（返回的是当前运行时生效值）",
  "":"/: "0=有锁+mutex, 1=有锁+spinlock, 2=无锁队列",
  "LockMode": 1,
  "":"/: "EnableLockStat: 是否启用锁耗时统计（LockCount/HoldTimeNs/MaxHoldTimeNs）",
  "EnableLockStat": false,

  "SafeQueue": {
    "":"/: "LockCount: 加锁次数（包含自旋锁/互斥锁）",
    "LockCount": 0,
    "":"/: "HoldTimeNs: 累计持锁时长，单位纳秒",
    "HoldTimeNs": 0,
    "":"/: "MaxHoldTimeNs: 单次持锁的最大时长，单位纳秒",
    "MaxHoldTimeNs": 0
  },
  "MsgLogMemPool": {
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  },
  "LogMemPool": {
    "Initied": true,
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  },
  "OneLogMemPool": {
    "Initied": true,
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  },
  "TimeLogMemPool": {
    "Initied": true,
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  },
  "BinLogMemPool": {
    "Initied": true,
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  },
  "TimeStampPool": {
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  },
  "TraceIDLogPool": {
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  },
  "TimeInfoPool": {
    "LockCount": 0,
    "HoldTimeNs": 0,
    "MaxHoldTimeNs": 0
  }
}
```

6.3.33. GetTsTlsInfo 获取线程存储信息

功能名称	GetTsTlsInfo	版本号	V1.0.0.0	更新日期	20251023
功能备注					
功能说明	获取线程存储信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetTsTlsInfo				
入参说明					
无					
出参说明					

```
{
  "":"/":"是否阻塞模式获取",
  "EnableBlocking": false,
  "":"/" : "线程局部循环数组大小",
  "TimeStampSize": 128,
  "":"/":"线程局部存储数量",
  "ThreadLocalCount": 3,
  "ThreadLocal": [
    {
      "":"/":"序号",
      "Index": 0,
      "":"/":"线程名称",
      "ThreadName": "rcm_rx_recv",
      "":"/":"获取内存失败次数",
      "GetMemFailedCount": 0,
      "":"/":"打点次数",
      "TimeStampCount": 3,
      "":"/":"无锁队列获取失败次数",
      "TimeStampWaitOrGetNullTimes": 0,
      "":"/":"未被消费的数据量",
      "TimeStampDistance": 0,
      "":"/":"生产者所在缓冲区的位置",
      "TimeStampPosFill": 3,
      "":"/":"消费者所在缓冲区的位置",
      "TimeStampPosOutput": 3
    },
    {
      "Index": 1,
      "ThreadName": "cres_main",
      "GetMemFailedCount": 0,
      "TimeStampCount": 1,
      "TimeStampWaitOrGetNullTimes": 0,
      "TimeStampCopyCount": 1,
      "TimeStampCopyDropCount": 0,
      "TimeStampDistance": 0,
      "TimeStampPosFill": 1,
      "TimeStampPosOutput": 1
    },
    {
      "Index": 2,
      "ThreadName": "cres_main",
      "GetMemFailedCount": 0,
      "TimeStampCount": 1,
      "TimeStampWaitOrGetNullTimes": 0,
      "TimeStampCopyCount": 1,
      "TimeStampCopyDropCount": 0,
      "TimeStampDistance": 0,
      "TimeStampPosFill": 1,
      "TimeStampPosOutput": 1
    }
  ]
}
```

6.3.34. GetArchiveMonitor 获取归档相关信息

功能名称	GetArchiveMonitor	版本号	V1.0.0.0	更新日期	20260311
功能备注					
功能说明	获取归档相关信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_logproxy&FuncName=GetArchiveMonitor				
入参说明					
无					
出参说明					
<pre>{ //是否开启压缩备份 "EnableCompress": true, //压缩过的总文件数量 "CompressFileCount": 4, //压缩失败的文件数量 "CompressFailedCount": 0, //耗时统计 单位 (ms) "TotalCompressTimeMs": 49, //最大压缩时间 单位 (ms) "MaxCompressTimeMs": 13, //最小压缩时间 单位 (ms) "MinCompressTimeMs":13, //平均压缩时间 单位 (ms) "AveCompressTimeMs":13, //最近一次压缩时间 单位 (ms) "LastCompressTimeMs":13 }</pre>					

6.4. zk访问插件 ldp_zkhandler

6.4.1. GetZookeeperInfo获取zk信息

功能名称	GetZookeeperInfo	版本号	V1.0.0.1	更新日期	20240226
功能备注	获取zk信息				
功能说明	获取zk信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_zkhandler&FuncName=GetZookeeperInfo				
入参说明					
无					

出参说明
<pre>{ "":"/": "连接状态1:已连接 2:会话超时 3:连接中", "ConnectStatus":1, "":"/": "zk服务端IP地址: 端口", "ZKHost":"192.168.86.102:12181" }</pre>

6.4.2. GetZkLockInfo 获取zk锁信息

功能名称	GetZkLockInfo	版本号	V1.0.0.1	更新日期	20240710
功能备注	获取zk锁信息				
功能说明	获取zk锁信息				
访问链接	http://ip:port/kjp?Pretty=1&PluginName=kjp_zkhandler&FuncName=GetZkLockInfo				
入参说明					
无					
出参说明					
<pre>{ "LockHandler": [{ "":"/": "zk锁索引", "Index": 0, "":"/": "zk锁持有者线程ID", "OwnerThreadID": 118414, "":"/": "zk锁持有者线程名", "OwnerThreadName": "arith_start", "":"/": "zk锁持有者创建的锁节点数据", "LockNodeData": { "":"/": "上锁者所在设备IP", "IP": "1.1.1.102", "":"/": "上锁者所在进程ID", "PID": 118257, "":"/": "上锁者所在线程ID", "TID": 118414, "":"/": "上锁时间", "Time": "Wed Jul 10 16:09:49 2024" }, "":"/": "zk锁状态, 有以下几种状态: Locked, Unlocked, Invalid", "LockStatus": "Locked", "":"/": "zk锁句柄连接状态, 有以下几种状态: Connected, Expired, Connecting", "ConnStatus": "Connected", "":"/": "重新创建会话标志位", "ReinitFlag": "true", "":"/": "zk锁上锁持续时间, 单位: 秒", "LockHoldingTimeSecs": 5.834223 }, { "":"/": "zk锁索引", "Index": 1, "":"/": "zk锁持有者线程ID", "OwnerThreadID": 0, "":"/": "zk锁持有者线程名", "OwnerThreadName": "", "":"/": "zk锁状态, 有以下几种状态: Locked, Unlocked, Invalid", "LockStatus": "Unlocked", "":"/": "zk锁句柄连接状态, 有以下几种状态: Connected, Expired, Connecting", "ConnStatus": "Connected", "":"/": "重新创建会话标志位", "ReinitFlag": "true", "":"/": "zk锁上锁持续时间, 单位: 秒", "LockHoldingTimeSecs": 0.0 }], "":"/": "zk锁句柄数量", "HandlerCount": 2, "":"/": "可使用句柄数量", "ValidCount": 1}</pre>					

6.5. 可靠一致组播插件 ldp_rcm

6.5.1. GetStatistics 获取组播统计信息

功能名称	GetStatistics	版本号	V1.1.4.6	更新日期	20211129
功能备注	获取组播统计信息				
功能说明	获取组播统计信息				
访问链接	http://ip:port/ldp?PluginName=ldp_rcm&FuncName=GetStatistics&Pretty=1				
入参说明					
Pretty=1 表示将json美化输出，为0表示不美化					
出参说明					
发送端统计信息					
{ "/": "版本编号", "RcmVersion": "1.1.4.6", "/": "文件前缀名", "FilePrefix": "hundsun", "/": "产品名", "ProductName": "hundsun", "/": "上下文"					


```

"//": "版本号",
"RcmVersion": "1.1.4.6",
"//": "文件前缀名",
"FilePrefix": "hundsun",
"//": "产品名",
"ProductName": "hundsun",
"//": "上下文",
"ctxs": [
{
"//": "上下文ID",
"ContextId": 100,
"//": "上下文名称",
"Context": "server24",
"txs": [],
"rx": {
"//": "接收端统计信息",
"Rms": [
{
"//": "接收端IP",
"IP": "192.168.86.101",
"//": "上下文IP",
"ContextId": 1,
"//": "上一次收到数据的时间戳",
"LastRecvMilli": 1495579672,
"//": "会话号",
"ConnectionId": 45935,
"//": "组播地址",
"Addr": "235.0.8.5",
"//": "组播端口",
"Port": 30061,
"//": "主题ID",
"TransportId": 1,
"//": "主题名",
"TopicName": "req",
"//": "分区号",
"PartitionNo": 1,
"//": "下一个待接收分片序号",
"NextSqn": 7007,
"//": "下一个待取消息分片序号",
"NextSqnPop": 7007,
"//": "已经确认的分片号",
"SqnAcked": 7007,
"//": "收到的最大分片序号",
"MaxSqn": 7007,
"//": "能补缺的最大分片序号",
"MaxSqnToNak": 7007,
"//": "已经发起补缺的最大分片号",
"MaxSqnLastNak": 0,
"//": "下一个待接收消息序号",
"NextMsgNo": 7007,
"//": "下一个待处理消息序号",
"NextMsgNoDeliver": 7007,
"//": "已经确认的消息序号",
"MsgNoAcked": 7007,
"//": "收到的最大消息序号",
"MaxMsgNo": 7007,
"//": "发起补缺的最大消息序号",
"MaxMsgNoNak": 7007,
"//": "最后发送ACK的时间",
"LastAckMilli": 1495579672,
"//": "最后发送NAK的时间",
"LastNakMilli": 0,
"//": "最后发送ACK的原因",
"LastAckReason": "AckInterval",
"//": "收到乱序包次数",
"UnorderedSqnCount": 59,
"//": "触发快速补缺次数",
"QuickNakCount": 0,
"//": "发送NAK次数",
"NakCount": 0,
"//": "接收端启动以来的整体的丢包率",
"PacketLossRate": 0.0,
"//": "接收端最近五秒丢包率",
"RecentlyFiveSecondsLossRate": 0.0,
"//": "会话状态",
"SessionPhase": "Started",
"//": "是否处于发送端发起的序号协商流程",
"IsInHandshaking": 0,
"//": "是否停止接收数据",
"StopRecv": 0
}
],
"//": "集群统计信息",
"Tier": {
"//": "已经接收并回调用户信息",
"Deliver": {
"//": "集群名",
"TierName": "Default",
"//": "节点角色",
"Role": "Single",
"//": "第一个消息序号",
"FirstMsgSqn": 0,
"//": "下一个排队消息序号",
"NextMsgSqn": 7007,
"//": "下一个待接收分片序号",
"NextSqn": 7007,
"//": "下一个处理消息序号",
"NextMsgSqnDeliver": 7007,
"//": "已经处理的消息数量",
"MsgsDelivered": 7007,
"//": "下一个持久化消息序号",
"NextMsgSqnRecord": 7007,
"//": "已经持久化的消息数量",
"MsgsRecorded": 7007,
"//": "应用上下文全局唯一Id",
"WaitContextId": 0,
"//": "所有主题分区 全局唯一Id",
"WaitTransportId": 0,
"//": "提交的消息序号",
"CommittedMsgSqn": 7007,
"//": "消息Rcm校验和失败次数统计",
"MsgCountRcmCheckFailed": 0,
"//": "消息RMS流创建失败次数统计",
"MsgCountRmsCheckFailed": 0,
"//": "消息回调失败次数统计",
"OnMessageCountFailed": 0
}
}
}

```

```

    },
    "///": "同步发送",
    "Send": {
        "///": "序号同步地址",
        "SyncIp": "0.0.0.0",
        "///": "序号同步端口",
        "SyncPort": 0,
        "///": "序号同步补缺端口",
        "SyncRepairPort": 0,
        "///": "当前待发送序号",
        "ToSendMsgSqn": 0,
        "///": "发送流控次数",
        "uToSendSqn": 0,
        "///": "发送流控次数",
        "FlowControlCount": 0,
        "///": "所有成员的最大待确认序号",
        "WaitAckMsgSqn": 0,
        "///": "待确认分片号, 主备模式下使用",
        "WaitAckSqn": 0,
        "///": "最近发送时间",
        "LastSendTickMilli": 0,
        "///": "应答消息展示",
        "Ack": []
    }
}
}
}
]
}

```

多进程上下文统计信息

```

{
    "///": "版本编号",
    "RcmVersion": "1.1.4.6",
    "///": "文件前缀名",
    "FilePrefix": "hundsun",
    "///": "产品名",
    "ProductName": "hundsun",
    "ctxMPs": [
        {
            "///": "当前启动状态",
            "State": 2,
            "///": "当前身份",
            "Role": "ACTIVE",
            "///": "该上下文ID",
            "ID": 100,
            "///": "该上下文名",
            "Name": "server1",
            "///": "上下文实例名",
            "InstanceName": "Instance1",
            "///": "消息排队状态",
            "Ordering": {
                "///": "已排队的连续消息的最大消息序号",
                "NextOrderedMsgSqn": 49562,
                "///": "已排队的连续消息的最大分片序号",
                "NextFragSqn": 198248,
                "///": "已预分配的已连续的消息序号",
                "NextReservedMsgSqn": 49562,
                "///": "已预分配的最大消息序号",
                "MaxReservedMsgSqn": 49562,
                "///": "在队列中最小的起始消息序号",
                "MinCachedMsgSqn": 0,
                "///": "在队列中最小的起始分片序号",
                "MinCachedFragSqn": 0
            },
            "///": "消息发送状态",
            "Send": {
                "///": "下一个准备从网络发送的排队序号",
                "ToSendMsgSqn": 49559,
                "///": "下一个准备从网络发送的分片序号",
                "ToSendFragSqn": 198236,
                "///": "下一个待确认的已接收的消息序号",
                "NextToAckMsgSqn": 49546,
                "///": "下一个待确认的已接收的分片序号",
                "NextToAckFragSqn": 198184,
                "///": "下一个待确认的已消费的消息序号",
                "NextToAckConsumedMsgSqn": 0,
                "///": "下一个同步发送的分片序号",
                "MainThreadToSendFragSqn": 0
            },
            "///": "消息递交状态",
            "Delivering": {
                "///": "下一个允许递交的排队序号",
                "NextMsgSqnCanDeliver": 49546,
                "///": "下一个递交的排队序号",
                "NextMsgSqnToDeliver": 0,
                "///": "下一个业务层处理完成的排队序号",
                "NextMsgSqnConsumed": 0,
                "///": "业务层处理返回失败次数",
                "OnMessageCountFailed": 0,
                "///": "重演中本轮重放起始排队序号",
                "ReplayStartPointThisTurn": 0
            },
            "///": "消息持久化状态",
            "Record": {
                "///": "下一个待持久化的排队序号",
                "NextMsgSqnRecord": 0,
                "///": "已持久化消息个数",
                "MsgsRecorded": 0
            },
            "///": "RPO控制状态",
            "RPO": {
                "///": "满足RPO条件的待确认排队序号",
                "RPOControlledMsgSqn": 49546
            },
            "///": "运行时状态",
            "RunningState": {
                "///": "启动完成时本地有效的消息序号",
                "MsgSqnFromLocalData": 18446744073709551615
            },
            "///": "消息同步者状态",
            "MsgSync": {
                "///": "接收线程循环次数",
                "RecvThreadLoopTimes": 49621,
                "///": "持久化线程循环次数",
                "PersistThreadLoopTimes": 2538
            }
        }
    ]
}

```

```

        "AsyncSendThreadLoopTimes": 72902
    },
    "/*": "消息复制发送者状态（作为主机）",
    "ReplicationTransmitter": {
        "/*": "所有备机状态",
        "Followers": [
            {
                "/*": "备机RCM上下文ID",
                "ContextId": 101,
                "/*": "备机RCM上下文名",
                "Name": "server2",
                "/*": "备机身份",
                "Role": "INACTIVE",
                "/*": "是否连接上",
                "Connected": true,
                "/*": "备机编号",
                "Index": 0,
                "/*": "来自该备机报文的最后一次接收时间",
                "LastRecvTickMill": 1681577097,
                "/*": "已确认接收的消息序号",
                "AckedMsgSqn": 49546,
                "/*": "已确认接收的分片序号",
                "AckedFragSqn": 198184,
                "/*": "已确认处理完成的消息序号",
                "ConsumedMsgSqn": 0,
                "/*": "满足RPO条件的已确认消息序号",
                "RPOControlledMsgSqn": 49546
            }
        ]
    },
    "/*": "消息复制接收者状态（作为备机）",
    "ReplicationReceiver": {
        "/*": "当前主机状态",
        "Leader": {
            "/*": "主机RCM上下文ID",
            "ContextId": 100,
            "/*": "主机RCM上下文名",
            "Name": "server1",
            "/*": "是否连接上",
            "Connected": false,
            "/*": "来自该主机报文的最后一次接收时间",
            "LastRecvTickMill": 2216611257
        },
        "/*": "下一个已连续的特接收分片序号",
        "NextFragSqn": 0,
        "/*": "最大待接收分片序号",
        "MaxFragSqn": 0,
        "/*": "下一个待排队的分片序号",
        "NextFragSqnToPop": 0,
        "/*": "最大允许补缺的分片序号",
        "MaxFragSqnCanNak": 0,
        "/*": "最近一次ACK时间",
        "LastAckTickMill": 0,
        "/*": "最近一次ACK中，已接收的分片序号",
        "LastAckedReceivedFragSqn": 0,
        "/*": "最近一次ACK中，已接收的排队序号",
        "LastAckedReceiveMsgSqn": 0,
        "/*": "最近一次ACK中，已处理完成的排队序号",
        "LastAckedConsumedMsgSqn": 0,
        "/*": "最近一次ACK发送成功时间",
        "LastAckSuccessMilli": 0,
        "/*": "最近一次ACK原因",
        "LastAckReason": 0,
        "/*": "累计ACK总数",
        "AckCount": 0,
        "/*": "最后一次NAK时间",
        "LastNakMilli": 0,
        "/*": "最后一次NAK成功发送时间",
        "LastNakSuccessMilli": 0,
        "/*": "累计NAK总数",
        "NakCount": 0,
        "/*": "快速NAK总数",
        "QuickNakCount": 0
    },
    "/*": "旁路发送状态",
    "Bypass": {
        "/*": "下一待从网络发送的排队序号",
        "ToSendMsgSqn": 0,
        "/*": "下一待确认的排队序号",
        "NextToAckMsgSqn": 0,
        "/*": "下一待从网络发送的分片序号",
        "ToSendSqn": 0,
        "/*": "下一待确认的分片序号",
        "NextToAckSqn": 0,
        "/*": "发送失败次数",
        "SendFailCount": 0
    },
    "/*": "旁路发送端状态",
    "BypassTx": {
        "/*": "旁路发送端地址",
        "BypassIp": "192.168.86.0",
        "/*": "旁路发送传输模式",
        "BypassTransMode": "udp_mc",
        "/*": "下一待从网络发送的排队序号",
        "ToSendMsgSqn": 0,
        "/*": "下一待确认的排队序号",
        "NextToAckMsgSqn": 0,
        "/*": "下一待从网络发送的分片序号",
        "ToSendSqn": 0,
        "/*": "下一待确认的分片序号",
        "NextToAckSqn": 0,
        "/*": "上一次发送数据的时间戳",
        "LastSendDataMilli": 2691061432,
        "/*": "上一次发送心跳的时间戳",
        "LastSendHeartbeatMilli": 2691080442,
        "/*": "发送失败次数",
        "SendFailCount": 0,
        "/*": "累计收到无效的Msg总数",
        "ReceivedInvalidMsgCount": 0,
        "/*": "应答消息展示",
        "Ack": [
            {
                "/*": "上下文Id",
                "ContextId": 0,
                "/*": "下一个待接收Ack的分片序号",

```

```

        "AckNextSqn": 0,
        "/*": "下一个待接收Ack的消息号",
        "AckNextMsgSqn": 0,
        "/*": "上一次收到数据的时间戳",
        "LastRecvMilli": 2691080442,
        "/*": "上一次处理Ack的时间戳",
        "LastOnAckMilli": 2691080442,
        "/*": "是否为背压节点",
        "IsBackPressureNode": true,
        "/*": "累计ACK总数",
        "AckCount": 0,
        "/*": "累计Nak总数",
        "NakCount": 0,
        "/*": "连接状态",
        "ConnectionStatus": ""
    }
}
],
"/*": "内存池信息",
"MemoryBlockPool": {
    "/*": "RCM消息生产者使用的内存池状态",
    "ProducerBlockPool": {
        "/*": "内存池名, 同时作为共享内存文件名前缀",
        "BlockPoolName": "server1_rx_syncmsg",
        "/*": "内存池类型",
        "MemoryType": "SHARED",
        "/*": "内存池每个标准内存块大小",
        "StdBlockSize (MB)": 16,
        "/*": "内存池内存块上线",
        "MaxBlockCount": 16383,
        "/*": "已申请未使用的内存空间大小",
        "StdBlock_NotUsed (MB)": 48,
        "/*": "已申请未使用的内存块个数",
        "StdBlock_NotUsedBlockCount": 3,
        "/*": "已使用的内存空间大小",
        "StdBlock_Used (MB)": 512,
        "/*": "已使用的内存块个数",
        "StdBlock_UsedBlockCount": 32,
        "/*": "已申请未使用的巨大内存块空间大小",
        "BigBlock_NotUsed (MB)": 0,
        "/*": "已申请未使用的巨大内存块数量",
        "BigBlock_NotUsedBlockCount": 0,
        "/*": "已使用的巨大内存块空间大小",
        "BigBlock_Used (MB)": 0,
        "/*": "已使用的巨大内存块数量",
        "BigBlock_UsedBlockCount": 0
    },
    "/*": "接收内存池状态",
    "RecvMsgBlockPool": {
        "BlockPoolName": "server1_rx_rcmmmsg",
        "MemoryType": "SHARED",
        "StdBlockSize (MB)": 8,
        "MaxBlockCount": 16383,
        "StdBlock_NotUsed (MB)": 24,
        "StdBlock_NotUsedBlockCount": 3,
        "StdBlock_Used (MB)": 8,
        "StdBlock_UsedBlockCount": 1,
        "BigBlock_NotUsed (MB)": 0,
        "BigBlock_NotUsedBlockCount": 0,
        "BigBlock_Used (MB)": 0,
        "BigBlock_UsedBlockCount": 0
    },
    "/*": "分片重组内存池状态",
    "RecvFragMsgBlockPool": {
        "BlockPoolName": "server1_rx_rcmmmsg_big",
        "MemoryType": "SHARED",
        "StdBlockSize (MB)": 16,
        "MaxBlockCount": 16383,
        "StdBlock_NotUsed (MB)": 48,
        "StdBlock_NotUsedBlockCount": 3,
        "StdBlock_Used (MB)": 0,
        "StdBlock_UsedBlockCount": 0,
        "BigBlock_NotUsed (MB)": 0,
        "BigBlock_NotUsedBlockCount": 0,
        "BigBlock_Used (MB)": 0,
        "BigBlock_UsedBlockCount": 0
    },
    "/*": "全局排队数组内存池状态",
    "MsgIndexBlockPool": {
        "BlockPoolName": "server1_rx_rcmmmsg_index",
        "MemoryType": "SHARED",
        "StdBlockSize (MB)": 6,
        "MaxBlockCount": 16383,
        "StdBlock_NotUsed (MB)": 18,
        "StdBlock_NotUsedBlockCount": 3,
        "StdBlock_Used (MB)": 6,
        "StdBlock_UsedBlockCount": 1,
        "BigBlock_NotUsed (MB)": 0,
        "BigBlock_NotUsedBlockCount": 0,
        "BigBlock_Used (MB)": 0,
        "BigBlock_UsedBlockCount": 0
    },
    "/*": "控制报文内存池状态",
    "MsgSyncCtrlBlockPool": {
        "BlockPoolName": "server1_rx_ctrl",
        "MemoryType": "COMMON",
        "StdBlockSize (MB)": 1,
        "MaxBlockCount": 16383,
        "StdBlock_NotUsed (MB)": 0,
        "StdBlock_NotUsedBlockCount": 0,
        "StdBlock_Used (MB)": 1,
        "StdBlock_UsedBlockCount": 1,
        "BigBlock_NotUsed (MB)": 0,
        "BigBlock_NotUsedBlockCount": 0,
        "BigBlock_Used (MB)": 0,
        "BigBlock_UsedBlockCount": 0
    },
    "SyncRecvWindowBlockPool": {
        "BlockPoolName": "server1_sync_wnd",
        "MemoryType": "COMMON",
        "StdBlockSize (MB)": 1,
        "MaxBlockCount": 16383,
        "StdBlock_NotUsed (MB)": 0,
        "StdBlock_NotUsedBlockCount": 0,
        "StdBlock_Used (MB)": 1,
        "StdBlock_UsedBlockCount": 1,

```



```
        "BigBlock_NotUsed(MB)": 0,
        "BigBlock_NotUsedBlockCount": 0,
        "BigBlock_Used(MB)": 0,
        "BigBlock_UsedBlockCount": 0
    },
    "/*": "旁路发送的接收内存池状态",
    "BypassTxRecvBlockPool": {},
    "/*": "旁路发送的控制报文内存池状态",
    "BypassTxCtrlBlockPool": {}
},
"/*": "RCM消息消费者状态",
"Consumers": [
    {
        "/*": "实例名",
        "InstanceName": "Consumer0",
        "/*": "是否启用消费",
        "EnabledConsume": true,
        "/*": "是否正在消费中",
        "IsInMessageCallback": false,
        "/*": "正在消费中的排队序号",
        "CurrentCallbackMsgSqn": 18446744073709551615,
        "/*": "正在消费中的大小",
        "CurrentCallbackMsgSize": 0,
        "/*": "正在消费中的开始回调时间戳",
        "CurrentCallbackBeginTickNs": 0,
        "/*": "正在消费中的处理花费时间",
        "CurrentCallbackCostNs": 0,
        "/*": "正在消费中的重试次数",
        "CurrentCallbackRetryTimes": 0,
        "/*": "上一次消费的排队序号",
        "LastCallbackMsgSqn": 18446744073709551615,
        "/*": "上一次消费的大小",
        "LastCallbackMsgSize": 0,
        "/*": "上一次消费的开始回调时间戳",
        "LastCallbackBeginTickNs": 0,
        "/*": "上一次消费的处理划分时间",
        "LastCallbackCostNs": 0,
        "/*": "上一次消费的重试次数",
        "LastCallbackRetryTimes": 0,
        "/*": "上一次消费的回调返回值",
        "LastCallbackReturn": 0,
        "/*": "耗时最长一笔的排队序号",
        "MaxTimeCallbackMsgSqn": 18446744073709551615,
        "/*": "耗时最长一笔的大小",
        "MaxTimeCallbackMsgSize": 0,
        "/*": "耗时最长一笔的开始回调时间戳",
        "MaxTimeCallbackBeginTickNs": 0,
        "/*": "耗时最长一笔的处理划分时间",
        "MaxTimeCallbackCostNs": 0,
        "/*": "耗时最长一笔的重试次数",
        "MaxTimeCallbackRetryTimes": 0,
        "/*": "耗时最长一笔的回调返回值",
        "MaxTimeCallbackReturn": 0,
        "/*": "重试最多一笔的排队序号",
        "MaxRetryCallbackMsgSqn": 18446744073709551615,
        "/*": "重试最多一笔的大小",
        "MaxRetryCallbackMsgSize": 0,
        "/*": "重试最多一笔的重试次数",
        "MaxRetryCallbackRetryTimes": 0,
        "/*": "平均处理耗时",
        "AveCallbackCostNs": 0,
        "/*": "处理返回成功的消息平均处理耗时",
        "AveCallbackSuccessCostNs": 0,
        "/*": "处理成功次数",
        "SumCallbackSuccessCount": 0,
        "/*": "耗时最小一笔的处理耗时",
        "MinCallbackCostNs": 18446744073709551615
    }
],
"/*": "RCM消息生产者状态",
"Producers": [
    {
        "/*": "实例名",
        "InstanceName": "Producer0",
        "/*": "是否正在同步中",
        "IsSyncing": false,
        "/*": "是否指定排队序号同步",
        "IsGivenMsgSqn": false,
        "/*": "正在同步的一笔的排队序号",
        "SyncingMsgSqn": 18446744073709551615,
        "/*": "正在同步的一笔的消息大小",
        "SyncingMsgSize": 0,
        "/*": "正在同步的一笔的起始时间戳",
        "SyncingBeginTickNs": 0,
        "/*": "正在同步的一笔的开始加锁时间戳",
        "SyncingGetLockTickNs": 0,
        "/*": "正在同步的一笔的开始发送时间戳",
        "SyncingBeforeSendTickNs": 0,
        "/*": "正在同步的一笔的发送完成时间戳",
        "SyncingAfterSendTickNs": 0,
        "/*": "正在同步的一笔的总耗时",
        "SyncingTotalTimeNs": 0,
        "/*": "同步完成的上一笔是否指定排队序号",
        "LastSyncIsGivenMsgSqn": false,
        "/*": "同步完成的上一笔的排队序号",
        "LastSyncMsgSqn": 49546,
        "/*": "同步完成的上一笔的消息大小",
        "LastSyncMsgSize": 5000,
        "/*": "同步完成的上一笔的开始时间戳",
        "LastSyncBeginTickNs": 1681577097082578,
        "/*": "同步完成的上一笔的加锁耗时",
        "LastSyncGetLockCostNs": 172,
        "/*": "同步完成的上一笔的排队耗时",
        "LastSyncOrderCostNs": 553,
        "/*": "同步完成的上一笔的发送耗时",
        "LastSyncSendCostNs": 13438,
        "/*": "同步完成的上一笔的等待满足RPO的ACK耗时",
        "LastSyncWaitRpoCostNs": 657024,
        "/*": "同步完成的上一笔的总耗时",
        "LastSyncTotalTimeNs": 671187,
        "/*": "耗时最大一笔是否指定排队序号",
        "MaxSyncIsGivenMsgSqn": false,
        "/*": "耗时最大一笔的排队序号",
        "MaxSyncMsgSqn": 9181,
        "/*": "耗时最大一笔的消息大小",
        "MaxSyncMsgSize": 5000,
        "/*": "耗时最长一笔的开始时间戳"
```

```
// 1. 获取新增组播统计信息
"MaxSyncBeginTickNs": 1681575314438146,
"/": "耗时最大一笔的加锁耗时",
"MaxSyncGetLockCostNs": 169,
"/": "耗时最大一笔的排队耗时",
"MaxSyncOrderCostNs": 425,
"/": "耗时最大一笔的发送耗时",
"MaxSyncSendCostNs": 13788,
"/": "耗时最大一笔的等待满足RPO的ACK耗时",
"MaxSyncWaitRpoCostNs": 5197786,
"/": "耗时最大一笔的总耗时",
"MaxSyncTotalTimeNs": 5212168,
"/": "累计总耗时",
"SumSyncTotalTimeNs": 2134812863,
"/": "累计同步消息数",
"SumSyncCount": 3126,
"/": "平均同步总耗时",
"AveSyncTotalCostNs": 682921,
"/": "平均加锁耗时",
"AveGetLockCostNs": 257,
"/": "平均排队耗时",
"AveOrderCostNs": 570,
"/": "平均发送耗时",
"AveSendCostNs": 13322,
"/": "平均等待满足RPO的ACK耗时",
"AveWaitRpoCostNs": 668770,
"/": "耗时最小一笔的总耗时",
"MinSyncTotalTimeNs": 293348,
"/": "排队序号连续的次数",
"ContinuousMsgCount": 3127,
"/": "排队序号不连续的次数",
"UnorderedMsgCount": 0,
"/": "指定排队序号的次数",
"GivenMsgSqnCount": 0,
"/": "不指定排队序号的次数",
"NotGivenMsgSqnCount": 3127,
"/": "等待RPO的消息个数",
"SyncWaitRpoMsgCount": 3127,
"/": "不等待RPO的消息个数",
"NotWaitRpoMsgCount": 0
}
}
}
}
```

6.5.2. GetAllStatistics 获取新增组播统计信息

功能名称	GetAllStatistics	版本号	V1.1.4.11	更新日期	20220110
功能备注	获取新格式组播统计信息				
功能说明	获取新格式组播统计信息				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetAllStatistics&Pretty=1				
入参说明					
Pretty=1 表示将json美化输出，为0表示不美化					
出参说明					
发送端统计信息 <pre>{ "/": "版本编号", "RcmVersion": "1.1.4.11", "/": "文件前缀名", "FilePrefix": "hundsun", "/": "产品名", "ProductName": "hundsun", "/": "上下文", "Contexts": [{ "/": "上下文ID", "ContextId": 1, "/": "上下文名称", "ContextName": "client23", "/": "集群名", "TierName": "", "/": "节点角色", "Role": "Single", "/": "内存统计信息", "Memory": { "/": "发送端其他块池 已分配但是未使用", "TransmitterOther_MemAllocButUnused(MB)": 0, "/": "发送端其他块池 已分配并且已使用", "TransmitterOther_MemoryUsed(MB)": 6, "/": "发送端索引块池 已分配但是未使用", "TransmitterIndex_MemAllocButUnused(MB)": 9, "/": "发送端索引块池 已分配并且已使用", "TransmitterIndex_MemoryUsed(MB)": 3, "/": "发送端消息块池 已分配但是未使用", "TransmitterMessage_MemAllocButUnused(MB)": 48, "/": "发送端消息块池 已分配并且已使用", "TransmitterMessage_MemoryUsed(MB)": 16, "/": "接收端消息块池 已分配但是未使用", "RecvMessage_MemAllocButUnused(MB)": 0, "/": "接收端消息块池 已分配并且已使用", "RecvMessage_MemoryUsed(MB)": 0, "/": "接收端索引块池 已分配但是未使用", "RecvIndex_MemAllocButUnused(MB)": 0, "/": "接收端索引块池 已分配并且已使用", "RecvIndex_MemoryUsed(MB)": 0, "/": "接收端其他块池 已分配但是未使用", "RecvOther_MemAllocButUnused(MB)": 0, "/": "接收端其他块池 已分配并且已使用", "RecvOther_MemoryUsed(MB)": 0 }, "/": "发送端统计信息", "Transmitters": [</pre>					

```
"/": "发送端地址",
"Address": "192.168.86.101:35005",
"/": "主题名",
"TopicName": "req",
"/": "分区号",
"PartitionNo": 1,
"/": "主题ID",
"TransportId": 1,
"/": "组播地址",
"Addr": "235.0.8.5",
"/": "组播端口",
"Port": 30061,
"/": "下个消息编号",
"NextMsgNo": 101,
"/": "当前最大消息编号",
"ToSendMsgNo": 101,
"/": "上次应答消息编号",
"LastAckMsgNo": 101,
"/": "下一个待持久化的消息号",
"NextMsgNoRecord": 101,
"/": "已持久化消息数",
"MsgRecorded": 101,
"/": "缓存中保存的消息的最小消息号",
"MinCachedMsgNo": 0,
"/": "数据消息恢复的最小消息号",
"MinRecoveryMsgNo": 0,
"/": "数据消息恢复的最大消息号",
"MaxRecoveryMsgNo": 0,
"/": "发送端创建时传入的自定义参数",
"TxArg": 0,
"/": "理论上最大可能的UDP包长度",
"MaxUdpSize": 1472,
"/": "组播非分片报文能携带的最大数据",
"AvailableSpace": 1448,
"/": "组播分片报文能携带的最大数据",
"RegularLength": 1432,
"/": "发送标志，底层是否发送数据",
"SendFlag": true,
"/": "应答对应主题",
"ReqTopic": "",
"/": "下一个待分配的分片号",
"NextSqn": 101,
"/": "当前待发送分片号",
"ToSendSqn": 101,
"/": "最后一次收到的应答序号",
"LastAckSqn": 101,
"/": "异步发送的分片数目",
"AsyncSendSqnCount": 37,
"/": "底层失败的分片数目",
"SendFailedCount": 0,
"/": "发送端编号",
"TransmitterIndex": 0,
"/": "节点角色",
"Role": "Single",
"/": "会话",
"Session": {
  "/": "会话号",
  "ConnectionId": 17983,
  "/": "会话状态",
  "SessionPhase": "Started",
  "/": "会话启动消息号",
  "StartMsgNo": 0,
  "/": "启动时间戳",
  "CreateTimeStamp": 342500630284046,
  "/": "上次发送时间",
  "LastSendMilli": 342547754,
  "/": "进入当前阶段时间",
  "CurPhaseTickMilli": 342503630,
  "/": "上次发送SYN时间",
  "LastSendSynTickMilli": 342502630,
  "/": "上次发送SET时间",
  "LastSendSetTickMilli": 0
},
"/": "接收端的数组，每个元素是一个",
"Ack": [
  {
    "/": "上下文Id",
    "ContextId": 100,
    "/": "上下文名",
    "ContentName": "server24",
    "/": "接收端IP地址",
    "Address": "192.168.86.101:30061",
    "/": "接收端待接收消息的sqn序号",
    "AckSqn": 101,
    "/": "接收端待接收的消息号",
    "AckMsgNo": 101,
    "/": "接收端最近回复时间",
    "LastRecvMilli": 342547754,
    "/": "接收端回复ACK报文个数",
    "AckCount": 43,
    "/": "接收端回复NAK报文个数",
    "NakCount": 1,
    "/": "新启动的接收端回复NAK时被流控的次数",
    "NakCountFlowControl": 0,
    "/": "已在线的接收端回复Nak被聚合的消息数",
    "OnlineNakMergeCount": 0,
    "/": "已在线的接收端恢复Nak被流控的次数",
    "OnlineNakFlowControlCount": 0,
    "/": "接收端的连接状态",
    "ConnectionPhase": "CONNECTED",
    "/": "是否主接收者",
    "IsMandatoryAcker": true
  }
]
}
```

接收端统计信息

```
{
  "/": "版本编号",
  "RcmVersion": "1.1.4.11",
  "/": "文件前缀名",
  "FilePrefix": "hmdsun"
```

```

        "ProductName": "hundsun",
        "Contexts": [
            {
                "ContextId": 100,
                "ContextName": "server24",
                "TierName": "",
                "Role": "Single",
                "Memory": {
                    "TransmitterOther_MemAllocButUnused(MB)": 0,
                    "TransmitterOther_MemoryUsed(MB)": 6,
                    "TransmitterIndex_MemAllocButUnused(MB)": 9,
                    "TransmitterIndex_MemoryUsed(MB)": 3,
                    "TransmitterMessage_MemAllocButUnused(MB)": 48,
                    "TransmitterMessage_MemoryUsed(MB)": 16,
                    "RecvMessage_MemAllocButUnused(MB)": 0,
                    "RecvMessage_MemoryUsed(MB)": 0,
                    "RecvIndex_MemAllocButUnused(MB)": 0,
                    "RecvIndex_MemoryUsed(MB)": 0,
                    "RecvOther_MemAllocButUnused(MB)": 0,
                    "RecvOther_MemoryUsed(MB)": 0
                },
                "Transmitters": [],
                "Receiver": {
                    "Rms": [
                        {
                            "TierName": "",
                            "ContextId": 1,
                            "ContextName": "client23",
                            "LastRecvMilli": 342547754,
                            "ConnectId": 17983,
                            "Ip": "192.168.86.101",
                            "Addr": "235.0.8.5",
                            "Port": 30061,
                            "TransportId": 1,
                            "TopicName": "req",
                            "PartitionNo": 1,
                            "NextSqn": 101,
                            "NextSqnPop": 101,
                            "SqnAcked": 101,
                            "MaxSqn": 101,
                            "MaxSqnToNak": 101,
                            "MaxSqnLastNak": 2,
                            "NextMsgNo": 101,
                            "NextMsgNoDeliver": 101,
                            "MsgNoAcked": 101,
                            "MaxMsgNo": 101,
                            "MaxMsgNoNak": 101,
                            "LastAckMilli": 342547754,
                            "LastNakMilli": 342509733,
                            "LastAckReason": "AckInterval",
                            "UnorderedSqnCount": 59,
                            "QuickNakCount": 1,
                            "NakCount": 1,
                            "PacketLossRate": 0.01,
                            "RecentlyFiveSecondsLossRate": 0.0,
                            "SessionPhase": "Started",
                            "IsInHandshaking": 0,
                            "StopRecv": 0,
                            "StartWithData": 0
                        }
                    ]
                }
            }
        ]
    },
}

```

```

        "///": "已经接收并回调用户信息",
        "Deliver": {
            "///": "第一个消息序号",
            "FirstMsgSqn": 0,
            "///": "下一个排队消息序号",
            "NextMsgSqn": 101,
            "///": "下一个待接收分片序号",
            "NextSqn": 101,
            "///": "下一个处理消息序号",
            "NextMsgSqnDeliver": 101,
            "///": "已经处理的消息数量",
            "MsgsDelivered": 101,
            "///": "下一个持久化消息序号",
            "NextMsgSqnRecord": 101,
            "///": "已经持久化的消息数量",
            "MsgsRecorded": 101,
            "///": "应用上下文全局唯一Id",
            "WaitContextId": 0,
            "///": "所有主题分区 全局唯一Id",
            "WaitTransportId": 0,
            "///": "提交的消息序号",
            "CommittedMsgSqn": 101,
            "///": "消息RCM校验和失败次数统计",
            "MsgCountRcmCheckFailed": 0,
            "///": "消息RMS流创建失败次数统计",
            "MsgCountRmsCheckFailed": 0,
            "///": "消息回调失败次数统计",
            "OnMessageCountFailed": 0
        },
        "///": "同步",
        "Sync": {
            "///": "同步发送",
            "Send": {
                "///": "序号同步地址",
                "SyncAddress": "",
                "///": "当前待发送序号",
                "ToSendMsgSqn": 0,
                "///": "发送流控次数",
                "uToSendSqn": 0,
                "///": "发送流控次数",
                "FlowControlCount": 0,
                "///": "所有成员的最大待确认序号",
                "WaitAckMsgSqn": 0,
                "///": "待确认分片号, 主备模式下使用",
                "WaitAckSqn": 0,
                "///": "最近发送时间",
                "LastSendTickMilli": 0,
                "///": "应答消息展示",
                "Ack": [
                    {
                        "///": "对端上下文ID",
                        "ContextId": 0,
                        "///": "对端上下文名",
                        "ContextName": "",
                        "///": "当前待确认序号",
                        "MsgSqn": 0,
                        "///": "待确认分片号",
                        "Sqn": 0,
                        "///": "最近收到确认的时间",
                        "LastRecvMilli": 0,
                        "///": "ack应答数",
                        "AckCount": 0,
                        "///": "补缺统计个数",
                        "NakSqnCount": 0,
                        "///": "连接状态",
                        "ConnectionStatus": "",
                        "///": "对端地址",
                        "Address": ""
                    }
                ]
            },
            "///": "同步接收",
            "Recv": {
                "///": "序号同步地址",
                "SyncAddress": "",
                "///": "已经确认的消息序号",
                "MsgSqnLastAck": 0,
                "///": "已经确认的分片序号",
                "SqnLastAck": 0,
                "///": "备收到的最大消息序号",
                "MaxMsgSqn": 0,
                "///": "收到的最大分片序号",
                "MaxSqn": 0,
                "///": "下一个待接收分片序号",
                "NextSqn": 0,
                "///": "备能发起补缺的最大消息序号",
                "MaxMsgSqnNak": 0,
                "///": "能补缺最大分片序号",
                "MaxSqnNak": 0,
                "///": "备最后发送数据的时间",
                "LastSendMilli": 0,
                "///": "备最后发送NAK的时间",
                "LastNakMilli": 0,
                "///": "上次收到数据的毫秒数",
                "LastRecvMilli": 0,
                "///": "统计sqn补缺次数",
                "NakSqnCount": 0,
                "///": "连接状态",
                "ConnectionStatus": ""
            },
            "Rebuilding": {
                "///": "下一个排队消息序号",
                "NextMsgSqn": 0,
                "///": "收到的最大分片序号",
                "MaxSqn": 0,
                "///": "能补缺最大分片序号",
                "MaxSqnNak": 0,
                "///": "下一个待接收分片序号",
                "NextSqn": 0,
                "///": "发起补缺的最大序号的最大分片号",
                "MaxSqnLastNak": 0,
                "///": "集群备节点开机重启从主节点同步数据容忍范围",
                "SyncMaxDiff": 0
            }
        }
    }
}

```

```
}  
}
```

6.5.3. GetContextInfo 获取普通上下文的信息

功能名称	GetContextInfo	版本号	V1.1.15.9	更新日期	20241011
功能备注	获取普通上下文的信息				
功能说明	获取普通上下文的信息，目前可以获取到上下文的Id、名字Name、身份角色Role、是否单机IsSingleton、集群名TierName、上下文是否启动IsStart、传输模式TransMode、所属中心Zone、仲裁优先级ArbPriority。				
访问链接	http://pport/kdp?PluginName=kdp_rcm&FuncName=GetContextInfo				
入参说明					
无					
出参说明					
查询到了上下文的信息，返回值如下 <pre>{ "Context": [{ "Id": 100, "Name": "server1", "Role": "SINGLE", "IsSingleton": true, "TierName": "server1", "IsStart": true, "TransMode": "udp_mc", "Zone": "", "ArbPriority": 0 }] }</pre>					

6.5.4. GetTxSessionInfo 获取发送端一侧的会话连接信息

功能名称	GetTxSessionInfo	版本号	V1.1.14.3	更新日期	20240102
功能备注	获取发送端一侧的会话连接信息				
功能说明	获取普通上下文，发送端一侧维护的会话连接状态等信息。可以通过此管理功能查询到，当前所有普通上下文的发送端的所有接收端的信息和连接状态等。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetTxSessionInfo				
入参说明					
无					
出参说明					

查询到了发送端的信息，返回值如下

```
{
  "/*": "发送端的数组，可能会有多个",
  "Transmitter": [
    {
      "/*": "发送端的所属的上下文的名字",
      "TxContextName": "client1",
      "/*": "发送端的所属的上下文的Id",
      "TxContextId": 1,
      "/*": "发送端发送的主题名",
      "TopicName": "req",
      "/*": "发送端发送的分区号",
      "PartitionNo": 1,
      "/*": "发送端发送端主题Id      (V1.1.14.9 新增)",
      "TransportId": 1,
      "/*": "下一个待排队消息序号（不含）      (V1.1.14.9 新增)",
      "NextMsgNo": 0,
      "/*": "下一个待向网络发送的消息序号（不含）      (V1.1.14.9 新增)",
      "ToSendMsgNo": 0,
      "/*": "已被确认的消息序号（不含）      (V1.1.14.9 新增)",
      "AckedMsgNo": 0,
      "/*": "已写入持久化文件的消息序号（不含）      (V1.1.14.9 新增)",
      "RecordedMsgNo": 0,
      "/*": "已写入持久化文件的消息数      (V1.1.14.9 新增)",
      "RecordedMsgCount": 0,
      "/*": "发送端缓存的有效消息的最小消息序号（含）      (V1.1.14.9 新增)",
      "MinCachedMsgNo": 0,
      "/*": "下一个待排队的分片序号（不含）      (V1.1.14.9 新增)",
      "NextFragNo": 0,
      "/*": "下一个待向网络发送的分片序号（不含）      (V1.1.14.9 新增)",
      "ToSendFragNo": 0,
      "/*": "已被确认的分片序号（不含）      (V1.1.14.9 新增)",
      "AckedFragNo": 0,
      "/*": "最近一次发送报文的时间戳      (V1.1.14.9 新增)",
      "LastSendTickMilli": 3776615698,
      "/*": "接收端的数组，如果有多个接收端此处会有多个",
      "Receivers": [
        {
          "/*": "接收端的上下文Id",
          "RxContextId": 100,
          "/*": "接收端的上下文名字",
          "RxContextName": "server1",
          "/*": "接收端的所使用的Ip",
          "RxIp": "1.1.1.102",
          "/*": "接收端的所使用的端口Port",
          "RxPort": "30111",
          "/*": "接收端当前的连接状态，是否与发送端正常连接。",
          "Connected": true,
          "/*": "最后一次接收到来自该接收端报文的时间戳（毫秒）",
          "LastRecvTickMill": 917502545,
          "/*": "已确认接收到的分片序号（不含）",
          "AckedFragNo": 9601,
          "/*": "已确认接收到的消息序号（不含）",
          "AckedMsgNo": 9601,
          "/*": "从该接收端接收到的补缺请求报文次数",
          "NakCount": 0,
          "/*": "从该接收端接收到的ACK确认报文次数",
          "AckCount": 566
        }
      ]
    }
  ]
}
```

6.5.5. GetRxTopicInfo 获取接收端一侧的会话连接信息

功能名称	GetRxTopicInfo	版本号	V1.1.14.9	更新日期	20240201
功能备注	获取接收端一侧的会话连接和消息序号等信息				
功能说明	获取普通上下文，接收端一侧维护的会话连接状态和消息序号等信息。可以通过此管理功能查询到，当前所有普通上下文的接收端的消息序号进度和所有对端发送端的信息和连接状态等。				
访问链接	http://ip:port/ldp?PluginName=ldp_rcm&FuncName=GetRxTopicInfo				
入参说明					
无					
出参说明					

查询到了接收端的信息，返回值如下

```
{
  "RxTopics": [
    {
      "RxContextName": "server1",
      "RxContextId": 1,
      "TopicName": "req",
      "PartitionNo": 1,
      "TransportId": 1,
      "TopicAddr": "235.0.2.1",
      "TopicPort": 30001,
      "NextMsgNoReceived": 1000,
      "NextMsgNoDelivered": 500,
      "AckedMsgNoReceived": 1000,
      "NextFragNoReceived": 2000,
      "AckedFragNoReceived": 2000,
      "NakCount": 1,
      "Transmitters": [
        {
          "TxContextId": 100,
          "TxContextName": "client1",
          "TxIp": "192.168.86.102",
          "AckCount": 123,
          "Connected": true,
          "LastRecvTickMilli": 804852723
        }
      ]
    }
  ]
}
```

6.5.6. GetLeaderSessionInfo 获取集群主机的一侧的会话连接信息

功能名称	GetLeaderSessionInfo	版本号	V1.1.14.3	更新日期	20240102
功能备注	获取集群主机的一侧的会话连接信息				
功能说明	获取集群主机的一侧的会话连接信息。返回的结果中只会包含集群身份是主机或者单机的上下文。如果上下文是单机模式也会在返回的列表里。注意：返回的结果是主机一侧维护的连接等相关信息。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetLeaderSessionInfo				
入参说明					
无					
出参说明					

查询到了集群主机一侧的会话连接信息，返回值如下

集群主机的返回值

```
{
  "///": "主机或者单机上下文的数组，每个元素是一个 上下文",
  "Leader": [
    {
      "///": "上下文所属的集群名",
      "TireName": "HaTest",
      "///": "上下文的Id",
      "LeaderContextId": 5,
      "///": "上下文的名字",
      "LeaderContextName": "HaNode2",
      "///": "集群身份，可能有ACTIVE和SINGLE",
      "Role": "ACTIVE",
      "///": "上下文是否是单机模式，true表示单机模式，false表示集群模式",
      "IsSingleton": false,
      "///": "作为集群主机时，所有集群内备机的连接状态等信息。可能有多个备机",
      "Followers": [
        {
          "///": "备机的上下文Id",
          "FollowerContextId": 4,
          "///": "备机的上下文名",
          "FollowerContextName": "HaNode1",
          "///": "主机一侧维护的与备机的连接状态",
          "Connected": true,
          "///": "主机收到来自备机的应答次数",
          "AckCount": 8,
          "///": "主机收到来自备机的请求补缺序号的次数",
          "NakSqnCount": 0,
          "///": "主机收到来自备机的请求补缺消息的次数",
          "NakDataCount": 0
        }
      ]
    }
  ]
}
```

单机模式的上下文返回值

```
{
  "///": "主机或者单机上下文的数组，每个元素是一个 上下文",
  "Leader": [
    {
      "///": "上下文所属的集群名",
      "TireName": "server1",
      "///": "上下文的Id",
      "LeaderContextId": 100,
      "///": "上下文的名字",
      "LeaderContextName": "server1",
      "///": "集群身份，单机模式肯定是SINGLE",
      "Role": "SINGLE",
      "///": "上下文是否是单机模式，true表示单机模式，false表示集群模式",
      "IsSingleton": true,
      "///": "由于是单机上下文所以没有备机，此处为空",
      "Followers": []
    }
  ]
}
```

6.5.7. GetContextInfoOfCtxMP 获取多进程上下文的信息

功能名称	GetContextInfoOfCtxMP	版本号	V1.1.14.3	更新日期	20240102
功能备注	获取多进程上下文的信息				
功能说明	获取多进程上下文的信息，展示多进程上下文的Id、Name、InstanceName、是否单机模式，和当前的身份角色等相关信息。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetContextInfoOfCtxMP				
入参说明					
无					
出参说明					
查询到了多进程上下文的信息，返回值如下 <pre>{ "///": "多进程上下文的数组，可能有多个", "ContextMultiProc": [{ "///": "多进程上下文的Id", "Id": 87, "///": "多进程上下文的上下文名", "ContextName": "cluster_consumer_node_0", "///": "多进程上下文的实例名", "InstanceName": "sync_demo", "///": "多进程上下文的身份，可能有INIT、SYNC、INACTIVE、ACTIVE、SINGLE、SEPARATE", "Role": "ACTIVE", "///": "多进程上下文是否是单机模式", "IsSingleton": false, "///": "多进程上下文所属的集群名", "TierName": "cluster_server_node", "///": "组播模式", "MulticastMode": "RealMulticast" }]}</pre>					

6.5.8. GetLeaderSessionInfoOfCtxMP 获取集群主机一侧的会话连接信息

功能名称	GetLeaderSessionInfoOfCtxMP	版本号	V1.1.15.10	更新日期	20241111
功能备注	仅主机或单机显示该管理功能，获取集群主机一侧的会话连接信息				

功能说明	获取集群主机一侧的会话连接信息，返回的是一个Leader数组，里面存放的是身份为主机或者单机的多进程上下文的相关会话信息。可以获取到作为主机的多进程上下文的名字、Id、集群名、角色、消息序号相关信息、以及备机的个数和主机一侧维护的与备机的连接等相关信息。
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetLeaderSessionInfoOfCtxMP
入参说明	
无	
出参说明	
<pre>{ "/*": "多进程上下文主机的数组，可能会有多个", "Leader": [{ "/*": "多进程上下文所属的集群名", "TierName": "cluster_server_node", "/*": "多进程上下文的Id", "LeaderContextId": 87, "/*": "多进程上下文的名字", "LeaderContextName": "cluster_consumer_node_0", "/*": "多进程上下文的身份，目前会有ACTIVE和SINGLE", "Role": "ACTIVE", "/*": "上下文是否是单机模式，true表示单机模式，false表示集群模式", "IsSingleton": false, "/*": "组播模式", "MulticastMode": "RealMulticast", "/*": "已排队连续消息的最大消息序号", "NextOrderedMsgSqn": 49562, "/*": "已排队连续消息的最大分片序号", "NextOrderedFragSqn": 198248, "/*": "下一个准备从网络发送的排队序号", "ToSendMsgSqn": 49559, "/*": "下一个准备从网络发送的分片序号", "ToSendFragSqn": 198236, "/*": "下一个待确认的已接收的消息序号", "NextToAckMsgSqn": 49546, "/*": "下一个待确认的已接收的分片序号", "NextToAckFragSqn": 198184, "/*": "下一个待确认的已消费的消息序号", "NextToAckConsumedMsgSqn": 0, "/*": "下一个允许递交的排队序号", "NextCanDeliverMsgSqn": 49546, "/*": "下一个递交的排队序号", "NextToDeliverMsgSqn": 0, "/*": "下一个业务层处理完成的排队序号", "NextConsumedMsgSqn": 0, "/*": "下一个待持久化的排队序号", "NextRecordMsgSqn": 0, "/*": "满足RPO条件的待确认排队序号", "RPOControlledMsgSqn": 49546, "/*": "主机一侧维护的备机的连接状态，可能有多个备机", "Followers": [{ "/*": "备机的上下文Id", "FollowerContextId": 88, "/*": "备机的上下文名", "FollowerContextName": "cluster_consumer_node_1", "/*": "备机的当前身份，可能会有SYNC和INACTIVE", "FollowerRole": "INACTIVE", "/*": "主机与备机的连接是否建立", "Connected": true, "/*": "主机最后一次收到备机应答的时间戳", "LastRecvTickMill": 929763846, "/*": "主机收到的备机已经Ack确认的消息号", "AckedMsgSqn": 0, "/*": "主机收到的备机已经Ack确认的分片号", "AckedFragSqn": 0, "/*": "主机收到的备机已经消费的消息号", "ConsumedMsgSqn": 0, "/*": "主机一侧维护的备机的RPO消息号", "RPOControlledMsgSqn": 0, "/*": "主机收到的备机的Nak补缺请求次数", "NakCount": 0, "/*": "该备机看到的主机状态", "ConnectedFromFollowerView": 0, "/*": "备机通信ID", "PeerId": 1465544272579919874, "/*": "备机状态汇报的连接的状态", "RcmReportState": 1, "/*": "最后一次收到该备机的状态汇报时间", "LastRecvRcmReportTickMill": 29949190, "/*": "该备机是否被拒绝加入集群", "RejectedFromCluster": false }] }] }</pre>	

6.5.9. AddAllMemberToCluster 强制将当前集群所有节点加入集群

功能名称	AddAllMemberToCluster	版本号	V1.1.14.4	更新日期	20240102
功能备注	【警告】仅作为无主机场景下的紧急运维手段。除非完全明确该操作带来的所有后续影响，否则不应轻易使用。若集群中剩余备机和原主机未达到同步，强制选主得到的新主机的数据会少于原主机。				
功能说明	强制将当前集群所有节点加入集群，这之后应当调用StartElection管理功能开始选主。 一个集群只需要调用任意一个节点的此管理功能。 若集群主机在线/集群中存在有RCM_ROLE_INACTIVE的备机/指定的RCM上下文不是备机/指定的RCM上下文正在竞选中，此管理功能调用均无效。 应当调用RCM仲裁对象（位于kdp_arb插件）所在进程的管理功能。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=AddAllMemberToCluster				

入参说明
<pre>{ "///": "指定RCM上下文名，对该RCM上下文所属集群进行操作", "ContextName": "xst#0_ldp_remoter_0" }</pre>
出参说明
<pre>{ "///": "成功", "ErrorNo": 0 }</pre>

6.5.10. StartElection 强制备机开始选主

功能名称	StartElection	版本号	V1.1.14.4	更新日期	20240102
功能备注	【警告】仅作为无主机场景下的紧急运维手段。除非完全明确该操作带来的所有后续影响，否则不应轻易使用。 若集群中剩余备机和原主机未达到同步，强制选主得到的新主机的数据会少于原主机。				
功能说明	若该备机身份为RCM_ROLE_INACTIVE，强制其开始选主。 集群中的每个备机都需要调用后才会开始选主。 如果调用时该备机身份为RCM_ROLE_SYNC，可能是还未完成到RCM_ROLE_INACTIVE的切换，该管理功能会阻塞等待至其切换为RCM_ROLE_INACTIVE后开始执行，至多等待1秒。 此管理功能不会等待选主结果，调用者应判断集群最终是否达到期望的状态。 若集群主机在线/等待切换结束后，指定的RCM上下文身份不是RCM_ROLE_INACTIVE/指定的RCM上下文正在竞选中，此管理功能调用均无效。 应当调用RCM仲裁对象（位于kdp_arb插件）所在进程的管理功能。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=StartElection				
入参说明	<pre>{ "///": "指定RCM上下文名，令其开始选主", "ContextName": "xst#0_ldp_remoter_0" }</pre>				
出参说明	<pre>{ "///": "成功", "ErrorNo": 0 }</pre>				

6.5.11. GetDynamicConfigInfo 获取动态配置信息

功能名称	GetDynamicConfigInfo	版本号	V1.1.14.9	更新日期	20240131
功能备注	获取Rcm动态配置相关信息				
功能说明	返回当前Rcm动态配置的相关信息。 只支持配置为服务端模式，版本1.1.14.9只支持配置服务端为ZK，否则返回的信息字段的值都为空("")。 如果没有开启动态配置，返回的信息字段的值都为空("")。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetDynamicConfigInfo				
入参说明	无				
出参说明	查询到动态配置信息，返回值如下： <pre>{ "RcmDynamicConfig": { "///": "本份动态配置首次创建时的基准版本号，最长24字节", "BenchmarkVersion": "1706064126.252713781" } }</pre>				

6.5.12. GetCfgUpdateInfo 获取动态配置更新统计信息

功能名称	GetCfgUpdateInfo	版本号	V1.1.15.14	更新日期	20250314
功能备注	获取Rcm动态配置更新的相关统计信息				
功能说明	返回Rcm动态配置更新的相关统计信息。 只支持配置为服务端模式，版本1.1.15.14只支持配置服务端为ZK，否则返回的信息字段的值都为空("")。 如果没有开启动态配置，返回的信息字段的值都为空("")。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetCfgUpdateInfo				
入参说明	无				
出参说明					

```
查询到动态配置更新统计信息，返回值如下：
{
    "///": "配置更新的处理时延（毫秒）",
    "DelayMilli": 1000,
    "///": "配置更新的回调发生次数",
    "CallbackCount": 10,
    "///": "实际处理配置更新的次数",
    "DealCount": 5,
    "///": "通知异步检查配置更新的次数（目标配置不存在时，主动通知异步检查配置更新）",
    "NotifyAsyncCheckCfgUpdateCnt": 0,

    "///": "提交型事务的最小耗时（毫秒）",
    "MinCommitTransTimeMS": 20,
    "///": "提交型事务的最大耗时（毫秒）",
    "MaxCommitTransTimeMS": 300,
    "///": "提交型事务总耗时（毫秒）",
    "SumCommitTransTimeMS": 2000,
    "///": "提交型事务的总次数",
    "CommitTransCount": 1,
    "///": "提交型事务的平均耗时（毫秒）",
    "AveCommitTransTimeMS": 150,

    "///": "回滚型事务的最小耗时（毫秒）",
    "MinCancelTransTimeMS": 20,
    "///": "回滚型事务的最大耗时（毫秒）",
    "MaxCancelTransTimeMS": 300,
    "///": "回滚型事务总耗时（毫秒）",
    "SumCancelTransTimeMS": 2000,
    "///": "回滚型事务的总次数",
    "CancelTransCount": 1,
    "///": "回滚型事务的平均耗时（毫秒）",
    "AveCancelTransTimeMS": 150,

    "///": "动态配置获取事务锁，加锁返回时（不论是否成功）的锁获取次数分布",
    "TransLockAcqMap": {
        "///": "尝试加锁次数为0次，一般是指一次加锁都没尝试时报错了，值表示发生次数",
        "0": 0,
        "///": "尝试加锁次数为1次，值表示发生次数",
        "1": 4,
        "///": "尝试加锁次数位于2~4次，值表示发生次数",
        "2-4": 2,
        "///": "尝试加锁次数位于5~32次，值表示发生次数",
        "5-32": 1,
        "///": "尝试加锁次数位于33~128次，值表示发生次数",
        "33-128": 0,
        "///": "尝试加锁次数位于129~600次，值表示发生次数",
        "129-600": 0,
        "///": "尝试加锁次数位于601次及以上，值表示发生次数",
        "601+": 0
    }
}
```

6.5.13. RecoverToMulticastSync 集群同步恢复为组播模式

功能名称	RecoverToMulticastSync	版本号	V1.1.14.11	更新日期	20240312
功能备注	【警告】应当在确认组播已恢复后再执行，否则切换回组播但仍然不通，会引起主备连接、主备同步再次中断。				
功能说明	组播断开但单播正常，且已经切换为单播模拟组播模式的主机，可以通过调用此管理功能恢复回组播模式 仅udp_mc模式有效 应当调用RCM集群同步对象（位于kdp_copy插件）所在进程的管理功能。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=RecoverToMulticastSync				
入参说明					
<pre>{ "///": "指定RCM上下文名", "ContextName": "xst#0_ldp_remoter_0" }</pre>					
出参说明					
<pre>{ "///": "成功", "ErrorNo": 0 }</pre>					

6.5.14. GetSyncLatencyInfoOfCtxMP 获取集群主备同步时延信息

功能名称	GetSyncLatencyInfoOfCtxMP	版本号	V1.1.14.11	更新日期	20240314
功能备注	获取集群主备同步的时延信息				
功能说明	获取集群主备同步的时延信息，对集群中每个备机都分别统计，只有主机才会统计这些信息。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetSyncLatencyInfoOfCtxMP				
入参说明					
无					
出参说明					

```
{
  "/*": "上下文主机的数组，可能会有多个",
  "Leader": [
    {
      "/*": "集群名",
      "TierName": "hundsun_SyncCluster",
      "/*": "主机的上下文Id",
      "LeaderContextId": 101,
      "/*": "主机的上下文名",
      "LeaderContextName": "hundsun_node1",
      "/*": "主机所属中心名",
      "Zone": "Zone_1",
      "/*": "上下文的主机身份，目前会有ACTIVE和SINGLE",
      "Role": "ACTIVE",
      "/*": "上下文是否为单机模式",
      "IsSingleton": false,
      "/*": "该集群中备机的数组，可能会有多个",
      "Followers": [
        {
          "/*": "备机的上下文Id",
          "FollowerContextId": 201,
          "/*": "备机的上下文名",
          "FollowerContextName": "hundsun_node2",
          "/*": "备机所属中心名",
          "Zone": "Zone_1",
          "/*": "备机的当前身份，可能会有SYNC和INACTIVE",
          "FollowerRole": "INACTIVE",
          "/*": "主机与备机的连接是否建立",
          "Connected": true,
          "/*": "最近一个消息的同步耗时（纳秒）",
          "LastSyncTimeNs": 30740,
          "/*": "最近一个同步的消息号",
          "LastSyncMsgSqn": 3879,
          "/*": "最小的一次同步耗时（纳秒）",
          "MinSyncTimeNs": 23835,
          "/*": "耗时最小的一笔对应的消息号",
          "MinSyncMsgSqn": 2825,
          "/*": "最大的一次同步耗时（纳秒）",
          "MaxSyncTimeNs": 2111688,
          "/*": "耗时最大的一笔对应的消息号",
          "MaxSyncMsgSqn": 2029,
          "/*": "被统计到的同步总耗时（纳秒）",
          "SumSyncTimeNs": 136632508,
          "/*": "被统计到的同步次数",
          "CapturedSyncCount": 3880,
          "/*": "平均的同步耗时（纳秒）",
          "AveSyncTimeNs": 35214
        }
      ]
    }
  ]
}
```

6.5.15. GetRecentSyncLatencyByTimeOfCtxMp 获取最近若干按时间划分的统计周期内集群主备同步时延信息

功能名称	GetRecentSyncLatencyByTimeOfCtxMp	版本号	V1.1.15.3	更新日期	20240626
功能备注	目前支持显示4个统计周期				
功能说明	获取集群主备同步的时延信息，对集群中每个备机都分别统计，只有主机才会统计这些信息。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetRecentSyncLatencyByTimeOfCtxMp				
入参说明					
无					
出参说明					

```
{
  "":"/": "时延数据数组，每一对 主机-备机 是该数组的一个元素",
  "SyncLatency": [
    {
      "":"/": "集群名",
      "TierName": "TestCluster",
      "":"/": "主机上下文ID",
      "LeaderContextId": 100,
      "":"/": "主机上下文名",
      "LeaderContextName": "hundsun_server1",
      "":"/": "主机所属中心名",
      "LeaderZone": "Z1",
      "":"/": "主机身份",
      "LeaderRole": "ACTIVE",
      "":"/": "是否为集群模式",
      "IsSingleton": false,
      "":"/": "备机上下文ID",
      "FollowerContextId": 101,
      "":"/": "备机上下文名",
      "FollowerContextName": "hundsun_server2",
      "":"/": "备机所属中心名",
      "FollowerZone": "Z2",
      "":"/": "备机身份",
      "FollowerRole": "INACTIVE",
      "":"/": "备机是否连接上",
      "Connected": true,
      "":"/": "每个统计周期的时间长度",
      "TimeMillofSingleStatisticalPeriod": 5000,
      "":"/": "基于历史耗时数据的当前耗时估计值",
      "EstimatedSyncTimeNs": 1562618,
      "":"/": "当前时间戳，纳秒",
      "CurrentTimeNs": 9637368023299437,
      "":"/": "统计周期数组",
      "RecentPeriods": [
        {
          "":"/": "该统计周期的开始时间戳，纳秒",
          "BeginTimeNs": 9637360704025856,
          "":"/": "该统计周期内采样到的耗时笔数",
          "CapturedSyncCount": 63116,
          "":"/": "该统计周期内的首笔消息的排队序号",
          "FirstMsgSqn": 4183174,
          "":"/": "该统计周期内的最后一笔消息的排队序号",
          "LastMsgSqn": 4405713,
          "":"/": "该统计周期内耗时最小一笔的同步耗时",
          "MinSyncTimeNs": 250454,
          "":"/": "该统计周期内耗时最小一笔的排队序号",
          "MinSyncMsgSqn": 4305072,
          "":"/": "该统计周期内耗时最大一笔的同步耗时",
          "MaxSyncTimeNs": 25099728,
          "":"/": "该统计周期内耗时最大一笔的排队序号",
          "MaxSyncMsgSqn": 4242334,
          "":"/": "该统计周期内每笔耗时的累加值",
          "SumSyncTimeNs": 97440321573,
          "":"/": "该统计周期内耗时的平均值",
          "AveSyncTimeNs": 1543829
        },
        {
          "BeginTimeNs": 9637355703030314,
          "CapturedSyncCount": 152528,
          "FirstMsgSqn": 3803157,
          "LastMsgSqn": 4183172,
          "MinSyncTimeNs": 239845,
          "MinSyncMsgSqn": 3811920,
          "MaxSyncTimeNs": 21250485,
          "MaxSyncMsgSqn": 4175533,
          "SumSyncTimeNs": 233629370199,
          "AveSyncTimeNs": 1531714
        },
        {
          "BeginTimeNs": 9637350702074774,
          "CapturedSyncCount": 145691,
          "FirstMsgSqn": 3426668,
          "LastMsgSqn": 3803145,
          "MinSyncTimeNs": 141573,
          "MinSyncMsgSqn": 3796952,
          "MaxSyncTimeNs": 16151254,
          "MaxSyncMsgSqn": 3486167,
          "SumSyncTimeNs": 228440433526,
          "AveSyncTimeNs": 1567979
        },
        {
          "BeginTimeNs": 9637345701035371,
          "CapturedSyncCount": 138216,
          "FirstMsgSqn": 3048318,
          "LastMsgSqn": 3426658,
          "MinSyncTimeNs": 195624,
          "MinSyncMsgSqn": 3086328,
          "MaxSyncTimeNs": 15259567,
          "MaxSyncMsgSqn": 3223072,
          "SumSyncTimeNs": 212357544087,
          "AveSyncTimeNs": 1536417
        }
      ]
    }
  ]
}
```

+

6.5.16. GetRecentSyncLatencyByCountOfCtxMp 获取最近若干按数量划分的统计周期内集群主备同步时延信息

功能名称	GetRecentSyncLatencyByCountOfCtxMp	版本号	V1.1.15.3	更新日期	20240626
功能备注	目前支持显示4个统计周期				
功能说明	获取集群主备同步的时延信息，对集群中每个备机都分别统计，只有主机才会统计这些信息。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=GetRecentSyncLatencyByCountOfCtxMp				
入参说明					

无	
	出参说明
	<pre>{ "SyncLatency": [{ "TierName": "TestCluster", "LeaderContextId": 100, "LeaderContextName": "hundsun_server1", "LeaderZone": "Z1", "LeaderRole": "ACTIVE", "IsSingleton": false, "FollowerContextId": 101, "FollowerContextName": "hundsun_server2", "FollowerZone": "Z2", "FollowerRole": "INACTIVE", "Connected": true, "CountOfSingleStatisticalPeriod": 1000, "EstimatedSyncTimeNs": 1562618, "CurrentTimeNs": 9637368023299437, "RecentPeriods": [{ "BeginTimeNs": 9637360704025856, "CapturedSyncCount": 2116, "FirstMsgSqn": 4183174, "LastMsgSqn": 4405713, "MinSyncTimeNs": 250454, "MinSyncMsgSqn": 4305072, "MaxSyncTimeNs": 25099728, "MaxSyncMsgSqn": 4242334, "SumSyncTimeNs": 97440321573, "AveSyncTimeNs": 1543829 }, { "BeginTimeNs": 9637355703030314, "CapturedSyncCount": 1000, "FirstMsgSqn": 3803157, "LastMsgSqn": 4183172, "MinSyncTimeNs": 239845, "MinSyncMsgSqn": 3811920, "MaxSyncTimeNs": 21250485, "MaxSyncMsgSqn": 4175533, "SumSyncTimeNs": 233629370199, "AveSyncTimeNs": 1531714 }, { "BeginTimeNs": 9637350702074774, "CapturedSyncCount": 1000, "FirstMsgSqn": 3426668, "LastMsgSqn": 3803145, "MinSyncTimeNs": 141573, "MinSyncMsgSqn": 3796952, "MaxSyncTimeNs": 16151254, "MaxSyncMsgSqn": 3486167, "SumSyncTimeNs": 228440433526, "AveSyncTimeNs": 1567979 }, { "BeginTimeNs": 9637345701035371, "CapturedSyncCount": 1000, "FirstMsgSqn": 3048318, "LastMsgSqn": 3426658, "MinSyncTimeNs": 195624, "MinSyncMsgSqn": 3086328, "MaxSyncTimeNs": 15259567, "MaxSyncMsgSqn": 3223072, "SumSyncTimeNs": 212357544087, "AveSyncTimeNs": 1536417 }] }] }</pre>

6.5.17. RejectFollowerFromCluster 拒绝指定备机加入集群

功能名称	RejectFollowerFromCluster	版本号	V1.1.15.3	更新日期	20240626
功能备注	【警告】该管理功能为非常规的运维操作，只应在明确后果时调用。				
功能说明	指定备机上下文名，将该备机从集群中移除（若已在集群中），且后续拒绝该备机加入集群。被从集群中移除后的备机，身份会保持SYNC，主机不会因为其接收处理慢而被阻塞。 只会对主机本次启动中，已连接过当前主机的备机生效。若不通过管理功能将该备机恢复，此状态的生效时间持续				

	至主机进程关闭或主备发生切换为止。 应当调用是主机且RCM集群同步对象（位于kdp_copy插件）所在进程的管理功能。 目前仅支持消息复制模式的RCM上下文集群。 若该备机已在集群中，将其移出集群的操作是异步的，通常会立即生效，但此时必须具备能够将其移出集群的前提条件（例如与ZK连接正常），若否则会直到前提条件满足（例如与ZK连接恢复）才将该备机移出集群。 本操作的反向操作见 AdmitFollowerToCluster 管理功能说明。
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=RejectFollowerFromCluster
入参说明	
<pre>{ "/*": "指定RCM上下文名", "ContextName": "xst#0_ldp_remoter_0" }</pre>	
出参说明	
<pre>{ "/*": "成功", "ErrorNo": 0 }</pre>	

6.5.18. AdmitFollowerToCluster 允许指定备机加入集群

功能名称	AdmitFollowerToCluster	版本号	V1.1.15.3	更新日期	20240626
功能备注	【警告】该管理功能为非常规的运维操作，只应在明确后果时调用。				
功能说明	若某个备机已被通过 <code>RejectFollowerFromCluster</code> 管理功能拒绝加入集群，指定该备机上下文名，此后允许该备机加入集群。 只会对主机本次启动中，已连接过当前主机的备机生效。 应当调用是主机且RCM集群同步对象（位于<code>kdp_copy</code>插件）所在进程的管理功能。 目前仅支持消息复制模式的RCM上下文集群。 该操作仅是撤销 <code>RejectFollowerFromCluster</code> 对该备机的影响，后续其是否能够真正加入集群，仍由该备机与主机的同步状态控制，不受本操作影响 本操作的反向操作见 <code>RejectFollowerFromCluster</code> 管理功能说明。				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm&FuncName=AdmitFollowerToCluster				
入参说明					
<pre>{ "/*": "指定RCM上下文名", "ContextName": "xst#0_ldp_remoter_0"}</pre>					
出参说明					
<pre>{ "/*": "成功", "ErrorNo": 0}</pre>					

6.6. fens插件 ldp_fens

6.6.1. SiteInfoStat 获取Site信息

功能名称	SiteInfoStat	版本号	V1.0.6.6	更新日期	20260119
功能备注	站点信息				
功能说明	获取站点信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_fens&FuncName=SiteInfoStat				
入参说明					
<pre>{ "/*": "Address和Port可以不指定", "/*": "指定时只显示特定站点信息，不指定时，显示所有站点信息", "Address": "127.0.0.1", "Port": "33330", "/*": "为true时，显示所有连接信息；为false时，仅显示已连接的连接信息", "ShowAllSite": "false"}</pre>					
出参说明					
<pre>{ "CoreBackSites": [{ "/*": "后端节点类型", "Type": "core", "/*": "后端节点所属组", "Group": "Default", "/*": "后端节点站点名", "SiteName": "", "/*": "后端节点地理定位", "GeographicLocation": 0, "/*": "后端节点制造商类型", "OperatorType": 0, "/*": "站点所属账户类别", "AccountCategory": "Default", "/*": "站点所属账户名称" }]}</pre>					


```

        "":"/":"站点所属类别优先级",
        "AccountCategoryPriority": 0,
        "":"/":"后端节点分区信息, 账户分区开启下显示",
        "AccountPartition": [
            {
                "AccountPartition no": 1
            }
        ],
        "ClusterName": "",
        "ConnectInfo": [
            {
                "":"/":"站点ip",
                "Addr": "192.168.87.137",
                "":"/":"站点域名",
                "DomainName": "",
                "":"/":"站点端口",
                "Port": 18111,
                "":"/":"站点状态",
                "Status": "Registered",
                "IsMaster": true
            }
        ]
    },
    ...
],
"FrontBackSites": [
    {
        "Type": "front",
        "Group": "Default",
        "SiteName": "Default_1_front_1",
        "GeographicLocation": 0,
        "OperatorType": 0,
        "":"/":"站点所属账户类别",
        "AccountCategories": [
            "Default1",
            "Default2"
        ],
        "AccountPartition": [
            {
                "AccountPartition no": 1
            }
        ],
        "ConnectInfo": [
            {
                "Addr": "192.168.87.137",
                "DomainName": "www.hundsun.com",
                "Port": 23150,
                "Status": "Registered",
                "":"/":"站点内外网属性",
                "bWan": true,
                "":"/":"站点负载",
                "Loads": 5,
                "TimeOutTimes": 0
            },
            {
                "Addr": "192.168.87.137",
                "DomainName": "www.hundsun.com",
                "Port": 23151,
                "Status": "Registered",
                "bWan": false,
                "Loads": 3,
                "TimeOutTimes": 0
            }
        ]
    },
    ...
],
"FrontProxyBackSites": [
    {
        "Type": "frontproxy",
        "Group": "Default",
        "SiteName": "",
        "GeographicLocation": 0,
        "OperatorType": 0,
        "AccountPartition": [
            {
                "AccountPartition no": 1
            },
            {
                "AccountPartition no": 2
            }
        ],
        "ConnectInfo": [
            {
                "Addr": "192.168.87.137",
                "DomainName": "www.hundsun.com",
                "Port": 13142,
                "Status": "Registered",
                "Loads": 1,
                "TimeOutTimes": 0
            },
            {
                "Addr": "192.168.87.137",
                "DomainName": "www.hundsun.com",
                "Port": 13143,
                "Status": "Registered",
                "Loads": 1,
                "TimeOutTimes": 0
            }
        ]
    },
    ...
]
}

```

6.6.2. AccountPartitionStat获取账户分区信息

功能名称	AccountPartitionStat	版本号	V1.0.6.6	更新日期	20220926
功能备注	账户分区				
功能说明	获取账户分区信息				

访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_fens&FuncName=AccountPartitionStat
入参说明	
无	
出参说明	
<pre>{ "AccountPartition": [{ "///": "组", "Group": "g1", "///": "只在开启了fens的账户分区后，以下信息才存在", "AccountPartition": [{ "///": "此组下某个分区", "AccountPartitionNo": 0, "///": "此分区下账户数", "Count": 2222 }, ...] }, ...] }</pre>	

6.6.3. BackSiteClassifierStat后端节点分类

功能名称	BackSiteClassifierStat	版本号	V1.0.0.0	更新日期	20200630
功能备注	后端节点分类				
功能说明	获取后端节点分类信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_fens&FuncName=BackSiteClassifierStat				
入参说明					
无					
出参说明					
<pre>{ "Site Classifier": [{ "///": "开启账户分区下才显示", "AccountPartition": 0, "///": "后端节点所属组", "Group": "g1", "///": "后端节点类型", "Type": "Front", "///": "后端节点名称", "SiteName": "front1" }, { "AccountPartition": 0, "Group": "g1", "Type": "Core", "SiteName": "core1" }, ... { "AccountPartition": "All", "Group": "g1", "Type": "Front", "SiteName": "front1" }, { "AccountPartition": "All", "Group": "g1", "Type": "Core", "SiteName": "core1" }] }</pre>					

6.6.4. AccountInfoCollectFinishStat账户信息收集完成状态

功能名称	AccountInfoCollectFinishStat	版本号	V1.0.4.6	更新日期	20220224
功能备注	账户信息收集完成状态				
功能说明	账户分区开启下用于获取核心后端账户收集完成状态及连接信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_fens&FuncName=AccountInfoCollectFinishStat				
入参说明					
无					
出参说明					

```
{
  "CoreAccountInfoCollectFinishStat": [
    {
      "/CoreConnInfo": [
        {
          "/CoreConnInfo": {
            "Addr": "1.1.1.102",
            "DomainName": "",
            "Port": 10001,
            "Status": "Registered"
          }
        },
        {
          "Addr": "1.1.1.101",
            "DomainName": "",
            "Port": 10001,
            "Status": "Registered"
          }
        }
      ],
      "CoreCollectFinishState": "Success"
    }
  ]
}
```

6.6.5. NodeNoAndSystemNoStat核心NodeNo和SystemNo映射关系

功能名称	NodeNoAndSystemNoStat	版本号	V1.0.6.0	更新日期	20220630
功能备注	核心NodeNo和SystemNo映射关系				
功能说明	节点定位开启下，用于获取核心NodeNo和SystemNo映射关系				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_fens&FuncName=NodeNoAndSystemNoStat				
入参说明					
无					
出参说明					
<pre>{ "NodeNoAndSystemNoStat": [{ "核心节点名", "NodeName": "192.168.86.102:8001", "核心NodeNo", "NodeNo": 2, "核心SystemNo", "SystemNo": 5, "核心所配置的账户分区号", "AccountPartition": 2 }] }</pre>					

6.6.6. FrontToBackStat获取FrontSites和BackSites映射关系

功能名称	FrontToBackStat	版本号	V1.0.6.0	更新日期	20220630
功能备注	获取FrontSites和BackSites映射关系				
功能说明	获取FrontSites和BackSites映射关系				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_fens&FuncName=FrontToBackStat				
入参说明					
无					
出参说明					

```
{
  "FrontToBackStat": [
    {
      "///": "前端站点信息",
      "FrontSiteNodeName": "192.168.86.102:33330",
      "GroupAttrMap": [
        {
          "///": "组信息分类",
          "GroupName": "Default",
          "FrontToCores": [
            {
              "///": "账号分区号",
              "AccountPartition": "2_front",
              "BackSitesRelates": [
                {
                  "FrontBackSites": {
                    "///": "站点类型",
                    "Type": "front",
                    "///": "站点名称",
                    "SiteName": "Front102",
                    "Initiators": [
                      {
                        "///": "连接ip地址",
                        "Addr": "192.168.86.102",
                        "///": "连接域名",
                        "DomainName": "www.hundsun.com",
                        "///": "连接端口",
                        "Port": 44000,
                        "///": "连接状态",
                        "Status": "Registered"
                      }
                    ]
                  },
                  "CoreBackSites": [
                    {
                      "///": "站点类型",
                      "Type": "core",
                      "///": "站点名称",
                      "SiteName": "Core102",
                      "Initiators": [
                        {
                          "///": "连接ip地址",
                          "Addr": "192.168.86.102",
                          "///": "连接域名",
                          "DomainName": "",
                          "///": "连接端口",
                          "Port": 8001,
                          "///": "连接状态",
                          "Status": "Registered"
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

6.7. 前置机插件 ldp_front

6.7.1. RouteStat 获取路由统计信息

功能名称	RouteStat	版本号	V1.0.0.0	更新日期	20260119
功能备注	路由统计信息				
功能说明	获取路由统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_front&FuncName=RouteStat				

入参说明

```
{  "///": "Backend实例索引，默认0；",  "BackendIndex": "0"}
```

出参说明

```
{  "///": "路由配置版本号",  "Version": 1,  "///": "路由未命中次数",  "RouteMissCount": 0,  "///": "Backend实例ID",  "BackendInstanceId": 0,  "Routes": [    {      "///": "功能号",      "Functions": "111?",      "///": "目标系统号",      "SystemNo": 1,      "///": "目标节点号",      "NodeNo": 1,      "///": "目标端ID",      "BackendID": 1,      "///": "转发次数",      "Count": 10432    }  ]}
```

6.7.2. ASStat获取对接的通过tcp交互的主备核心节点信息

功能名称	ASStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	对接的主备核心节点信息				
功能说明	获取对接的主备核心节点信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=ASStat				
入参说明					
<pre>{ "BackendIndex": "0"}</pre>					
出参说明					
<pre>{ "AS": [{ "Name": "配置的名称", "BackendInstanceId": 0, "ID": 2, "Product": "yy", "Cluster": "cc", "SendSuccessCount": 100, "SendFailCount": 50, "InterceptCount": 20, "ErrorRspCount": 100, "IsDynamicCreate": false, "BackendId": 3, "SendMasterSuccessCount": 0, "SendMasterFailCount": 0, "SendBackupSuccessCount": 0, "SendBackupFailCount": 0, "RecvCount": 0, "RecvRegRspCount": 0, "SendRegSuccCount": 0, "SendRegFailCount": 0, "UnReadyDropCount": 0, "Sites": [{ "Node": "10.20.3.4:5555", "DomainName": "www.test.com", "Master": true, "InCluster": true, "SiteId": 3, "State": "Connected", "ConnectTime": 12, "ConnectedTimeSec": 1595973863, "LastSendedTimeSec": 1595974121, "LastRecvedTimeSec": 1595973863, "InterceptTimes": 0, "SendSuccessTimes": 1403337, "SendFailTimes": 0, "RecvTimes": 443 }] }]}</pre>					

6.7.3. ASRcmStat获取对接的通过rcm交互的主备核心节点信息

功能名称	ASRcmStat	版本号	V1.0.0.0	更新日期	20200630
功能备注	对接的主备核心节点信息				
功能说明	获取对接的主备核心节点信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=ASRcmStat				

入参说明
<pre>{ "BackendIndex": "0" }</pre>
出参说明
<pre>{ "ASRcm": [{ "Name": "AS-RCM-1", "BackendInstanceId": 0, "ID": 1, "Cluster": "xst", "Product": "as-rcm", "TxTopic": "trade", "Sites": [{ "Node": "xst#0", "Master": true, "InCluster": true, "PartitionNo": 1 }, { "Node": "xst#1", "Master": false, "InCluster": true, "PartitionNo": 2 }] }] }</pre>

6.7.4. GWStat 获取对接的网关节点信息

功能名称	GWStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	对接的网关节点信息				
功能说明	获取对接的网关节点信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=GWStat				
入参说明	<pre>{ "BackendIndex": "0" }</pre>				
出参说明					

```
{
  "GW": [
    {
      "Name": "配置的名称",
      "BackendInstanceId": 0,
      "ID": 2,
      "BackendId": 2,
      "SendSuccessCount": 0,
      "SendFailCount": 0,
      "InterceptTimes": 0,
      "RecvCount": 0,
      "ErrorRspCount": 0,
      "Sites": [
        {
          "Node": "10.20.3.4:5555",
          "DomainName": "www.test.com",
          "SiteId": 3,
          "State": "Connected",
          "ConnectTimes": 12,
          "ConnectedTimeSec": 1595973863,
          "LastSendedTimeSec": 1595974121,
          "LastRecvedTimeSec": 1595973863,
          "InterceptTimes": 0,
          "SendSuccessTimes": 1403337,
          "SendFailTimes": 0,
          "RecvTimes": 443
        }
      ]
    }
  ]
}
```

6.7.5. WanClientStat 获取客户端连接信息

功能名称	WanClientStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	客户端连接信息				
功能说明	获取客户端连接信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_front&FuncName=WanClientStat				
入参说明					
<pre>{ "StartIndex": 0, "QueryCount": 100 }</pre>					
出参说明					

```
{
  "/*": "Wan客户端总连接个数",
  "WanClientCount": 1,
  "/*": "Wan客户端总连接上限",
  "MaxWanCount": 65535,
  "/*": "宿主插件名",
  "HostPlugin": "ldp_front",
  "/*": "服务端过滤的主机收到的去备消息或备机收到的去主消息个数",
  "AsModeDropTotalCount": 0,
  "/*": "服务端未就绪过滤的消息个数",
  "UnReadyDropTotalCount": 0,
  "/*": "连接的具体信息",
  "Connections": [
    {
      "/*": "端名",
      "FrontendName": "Wan",
      "/*": "连接号",
      "ConnectId": 1073807360,
      "/*": "连接id, 包含了重用计数",
      "ConnectIndex": 65536,
      "/*": "发送的报文数, 不包含心跳",
      "SendTimes": 3,
      "/*": "接收的报文数, 不包含心跳",
      "RecvTimes": 3,
      "/*": "网络连接上的时间戳, 自1970-1-1以来的秒数",
      "ConnectedTimeSec": 1718680577,
      "/*": "最近发送数据的时间戳, 自1970-1-1以来的秒数, 包含心跳",
      "LastSendedTimeSec": 1718680587,
      "/*": "最近接收数据的时间戳, 自1970-1-1以来的秒数, 包含心跳",
      "LastRecvTimeSec": 1718680587,
      "/*": "客户端ip",
      "Address": "143.2.3.4",
      "/*": "客户端端口",
      "Port": 12121,
      "/*": "协议",
      "Protocol": "IDP",
      "/*": "非法报文拦截次数",
      "InterceptTimes": 0,
      "/*": "收到除了心跳的消息个数",
      "RecvTimesNoHeartbeat": 0,
      "/*": "发送成功次数, 不包含心跳",
      "SendSuccessTimes": 0,
      "/*": "发送失败次数, 不包含心跳",
      "SendFailTimes": 0,
      "/*": "客户端类型",
      "ClientType": "FrontProxy",
      "/*": "客户端进程号",
      "ClientPid": 12345,
      "/*": "客户端版本",
      "ClientVersion": "tcp version V1.0.6.4 Oct 17 2022 13:30:33",
      "/*": "使用客户端代理的真实客户端ip地址",
      "ClientAddressByProxy": "192.168.87.137",
      "/*": "使用客户端代理的真实客户端端口",
      "ClientPortByProxy": 61178,
      "/*": "每秒实时接收数, 只有配置针对连接的流控才有效",
      "RecvRealTimeCount": 0,
      "/*": "每秒实时拒绝数, 只有配置针对连接的流控才有效",
      "RefuseRealTimeCount": 0,
      "/*": "每秒接收数峰值, 只有配置针对连接的流控才有效",
      "RecvPeakCount": 0,
      "/*": "每秒接收数峰值时间戳, 只有配置针对连接的流控才有效",
      "RecvPeakTimeSec": 0,
      "/*": "每秒拒绝数峰值, 只有配置针对连接的流控才有效",
      "RefusePeakCount": 0,
      "/*": "每秒拒绝数峰值时间戳, 只有配置针对连接的流控才有效",
      "RefusePeakTimeSec": 0
    }
  ]
}
```

6.7.6. LanClientStat 获取连接局域网端口的客户端连接信息

功能名称	LanClientStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	连接局域网端口的客户端连接信息				
功能说明	获取连接局域网端口的客户端连接信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=LanClientStat				
入参说明					
<pre>{ "/*": "起始连接号, 默认0; 不含重用计数", "StartIndex": 0, "/*": "想要查询的连接数, 默认100", "QueryCount": 100 }</pre>					
出参说明					


```
{
  "/*": "Lan客户端总连接个数",
  "LanClientCount": 1,
  "/*": "Lan客户端总连接上限",
  "MaxLanCount": 65535,
  "/*": "宿主插件名",
  "HostPlugin": "ldp_front",
  "/*": "服务端过滤的主机收到的去备消息或备机收到的去主消息个数",
  "AsModeDropTotalCount": 0,
  "/*": "服务端未就绪过滤的消息个数",
  "UnReadyDropTotalCount": 0,
  "/*": "连接的具体信息",
  "Connections": [
    {
      "/*": "端名",
      "FrontendName": "Lan",
      "/*": "连接号",
      "ConnectId": 1073807361,
      "/*": "连接id, 包含了重用计数",
      "ConnectIndex": 65536,
      "/*": "发送的报文数, 不包含心跳",
      "SendTimes": 3,
      "/*": "接收的报文数, 不包含心跳",
      "RecvTimes": 3,
      "/*": "网络连接上的时间戳, 自1970-1-1以来的秒数",
      "ConnectedTimeSec": 1718680577,
      "/*": "最近发送数据的时间戳, 自1970-1-1以来的秒数, 包含心跳",
      "LastSendedTimeSec": 1718680587,
      "/*": "最近接收数据的时间戳, 自1970-1-1以来的秒数, 包含心跳",
      "LastRecvTimeSec": 1718680587,
      "/*": "客户端ip",
      "Address": "143.2.3.4",
      "/*": "客户端端口",
      "Port": 12122,
      "/*": "协议",
      "Protocol": "IDP",
      "/*": "非法报文拦截次数",
      "InterceptTimes": 0,
      "/*": "收到除了心跳的消息个数",
      "RecvTimesNoHeartbeat": 0,
      "/*": "发送成功次数, 不包含心跳",
      "SendSuccessTimes": 0,
      "/*": "发送失败次数, 不包含心跳",
      "SendFailTimes": 0,
      "/*": "客户端类型",
      "ClientType": "FrontProxy",
      "/*": "客户端进程号",
      "ClientPid": 12345,
      "/*": "客户端版本",
      "ClientVersion": "tcp version V1.0.6.4 Oct 17 2022 13:30:33",
      "/*": "使用客户端代理的真实客户端ip地址",
      "ClientAddressByProxy": "192.168.87.137",
      "/*": "使用客户端代理的真实客户端端口",
      "ClientPortByProxy": 61178,
      "/*": "每秒实时接收数, 只有配置针对连接的流控才有效",
      "RecvRealTimeCount": 0,
      "/*": "每秒实时拒绝数, 只有配置针对连接的流控才有效",
      "RefuseRealTimeCount": 0,
      "/*": "每秒接收数峰值, 只有配置针对连接的流控才有效",
      "RecvPeakCount": 0,
      "/*": "每秒接收数峰值时间戳, 只有配置针对连接的流控才有效",
      "RecvPeakTimeSec": 0,
      "/*": "每秒拒绝数峰值, 只有配置针对连接的流控才有效",
      "RefusePeakCount": 0,
      "/*": "每秒拒绝数峰值时间戳, 只有配置针对连接的流控才有效",
      "RefusePeakTimeSec": 0
    }
  ]
}
```

6.7.7. FrontendStat 获取前置发送的包成功数量，失败数量，接受数量，拦截数量

功能名称	FrontendStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	前置发送的包成功数量，失败数量，接受数量，拦截数量				
功能说明	获取前置对接各个模块过程中，发送的包成功数量，失败数量，接受数量，拦截数量				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_front&FuncName=FrontendStat				
入参说明					
无					
出参说明					

```
{
  "Front": [
    {
      "name": "Frontend",
      "/*": "前置类型: TCP为对接客户端, PLUGIN为前置插件",
      "type": "TCP",
      "/*": "发送成功计数",
      "SendSuccessCount": 10,
      "/*": "发送失败计数",
      "SendFailCount": 0,
      "/*": "接受计数",
      "RecvCount": 10,
      "/*": "拦截计数",
      "InterceptCount": 0
    },
    {
      "name": "FrontPlugin",
      "/*": "前置类型: TCP为对接客户端, PLUGIN为前置插件",
      "type": "PLUGIN",
      "/*": "发送成功计数",
      "SendSuccessCount": 10,
      "/*": "发送失败计数",
      "SendFailCount": 0,
      "/*": "接受计数",
      "RecvCount": 10,
      "/*": "拦截计数",
      "InterceptCount": 0
    }
  ]
}
```

6.7.8. RcmBackendStat 获取对接双活（主备rcm）节点的发送消息个数

功能名称	RcmBackendStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	获取对接双活（主备rcm）的发送消息个数				
功能说明	获取前置对接双活（主备rcm）的发送消息个数				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=RcmBackendStat				
入参说明					
<pre>{ "/*": "Backend实例索引，默认0;", "BackendIndex": "0" }</pre>					
出参说明					
<pre>{ "RcmBackend": [{ "name": "AA-1", "id": 1, "/*": "类型有AA, AS-RCM", "type": "AA", "/*": "Backend实例ID", "BackendInstanceId": 0, "BackendId": 3, "/*": "收到消息个数", "RecvCount": 3, "/*": "错误应答个数", "ErrorRspCount": 0, "/*": "发送给主机成功个数", "SendMasterSuccessCount": 0, "/*": "发送给主机失败个数", "SendMasterFailCount": 0, "/*": "发送给备机成功个数", "SendBackupSuccessCount": 0, "/*": "发送给备机失败个数", "SendBackupFailCount": 0, "/*": "发送成功计数", "SendSuccessCount": 10, "/*": "发送失败计数", "SendFailCount": 0, "/*": "拦截计数", "InterceptCount": 0, "/*": "发送应答个数", "ToSendRcmRspCount": 0, "/*": "收到注册应答个数", "RecvRegRspCount": 0, "/*": "发送注册请求成功个数", "SendRegSuccCount": 0, "/*": "发送注册请求失败个数", "SendRegFailCount": 0, "/*": "服务端未就绪, 消息丢弃个数", "UnReadyDropCount": 0 }] }</pre>					

6.7.9. RcmContextStat 获取前置上下文接收消息个数

功能名称	RcmContextStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	获取前置上下文接收消息个数				
功能说明	获取前置上下文接收消息个数				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=RcmContextStat				
入参说明					

<pre>{ "":"/: "Backend实例索引, 默认0;", "BackendIndex": "0" }</pre>
出参说明
<pre>{ "RcmContext": [{ "name": "Rcmcontext", "":"/: "上下文名称", "context": "front#0_ldp_front_0_0", "":"/: "当前上下文收到报文个数", "RcmRecvCount": 0 }] }</pre>

6.7.10. PluginStat 获取对接插件节点的信息

功能名称	PluginStat	版本号	V1.0.4.3	更新日期	20211123
功能备注	获取对接插件节点的信息				
功能说明	根据前置配置中的插件，获取每个插件的发送接收信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=PluginStat				

入参说明

```
{  "":"/: "Backend实例索引, 默认0;",  "BackendIndex": "0"}
```

出参说明

```
{  "PluginBackend": [    {      "name": "PluginBackend",      "":"/: "类型",      "type": "PLUGIN",      "":"/: "插件名称",      "PluginName": "ldp_rcm",      "":"/: "发送成功计数",      "SendSuccessCount": 10,      "":"/: "发送失败计数",      "SendFailCount": 0,      "":"/: "收到消息次数",      "RecvCount": 0,      "":"/: "拦截计数",      "InterceptCount": 0    }  ]}
```

6.7.11. FlowControlStat 获取流控状态, 流控内存占用, 设置流控开关

功能名称	FlowControlStat	版本号	V1.0.0.2	更新日期	20260122
功能备注	获取流控状态, 流控内存占用, 设置流控开关				
功能说明	通过本管理功能可以获取流量控制、流量监控状态, 流控内存占用, 动态设置流量控制、流量监控开关				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=FlowControlStat				

入参说明

```
{  "":"/: "流量控制开关, true 开启流量控制, false 关闭流量控制, 默认开启",  "SetFlowControlStatus":true,  "":"/: "流量监控开关, true 开启流量监控, false 关闭流量监控, 默认关闭",  "SetFlowMonitorStatus":false}
```

出参说明

```
{  "FlowControlStat": {    "":"/: "当前流量控制开关状态, 布尔类型, true 开启流量控制, false 关闭流量控制",    "FlowControlStatus": true,    "":"/: "当前流量监控开关状态, 布尔类型, true 开启流量监控, false 关闭流量监控",    "FlowMonitorStatus": false,    "":"/: "当前流控内存占用, 单位Bytes",    "MemoryStatusBytes": 400000,    "":"/: "当前配置的流控功能实现版本",    "FlowControlVersion": 2  } }
```

6.7.12. AccessControlStat 设置和获取接入控制状态

功能名称	AccessControlStat	版本号	V1.0.0.0	更新日期	20200925
功能备注	设置和获取接入控制状态				

功能说明	通过本管理功能可以设置和获取接入控制状态
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=AccessControlStat
入参说明	
<pre>{ "/*": "设置接入控制开关，布尔类型，默认是true，即接入控制生效", "SetAccessControlStatus": true }</pre>	
出参说明	
<pre>{ "AccessControlStat": { "/*": "当前接入控制开关状态，布尔类型，默认是true，即接入控制生效", "AccessControlStatus": true, } }</pre>	

6.7.13. MsgQuantityStat 获取当前插件交互的消息积压情况

功能名称	MsgQuantityStat	版本号	V1.0.0.0	更新日期	20200925
功能备注					
功能说明	通过本管理功能可以获取当前插件交互的消息积压情况				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=MsgQuantityStat				
入参说明					
无					
出参说明					
<pre>{ "/*": "插件数量", "PluginCount": 1, "Plugin": [{ "/*": "插件名", "PluginName": "ldp_front", "/*": "插件索引", "PluginIndex": 1, "/*": "为本插件分配的连接号", "ConnectID": 257, "/*": "消息积压数", "MsgQuantity": 2952, "/*": "是否需要缓存", "IsNeedCache": false, "/*": "收到消息个数", "RecvCount": 10, "/*": "发送消息成功个数", "SendSuccCount": 10, "/*": "发送消息失败个数", "SendFailCount": 10 }] }</pre>					

6.7.14. BackendStat 获取当前插件接出模块内部状态信息

功能名称	BackendStat	版本号	V1.0.0.0	更新日期	20220129
功能备注					
功能说明	通过本管理功能可以获取当前插件接出模块内部状态信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&FuncName=BackendStat				

入参说明

```
{  "/*": "Backend实例索引，默认0;",  "BackendIndex": "0"}
```

出参说明

```
{  "/*": "路由未命中次数",  "RouteMissCount": 0,  "/*": "路由配置",  "Routes": [    {      "/*": "功能号",      "Functions": "**",      "/*": "目标系统号",      "SystemNo": "**",      "/*": "目标节点号",      "NodeNo": "**",      "/*": "目标端ID",      "BackendID": 1,      "/*": "转发次数",      "Count": 0    }  ],  "Backends": [    {      "/*": "该Backend对应的服务器地址，只有sc、mc才会有该字段，默认是sc的服务器地址"    }  ]}
```

```

//": "以00000000000000000000000000000000为字节的Zk服务端地址，共有4096个字节用于存放，默认是"0"的Zk服务端地址",
"ZkServerAddr": "192.168.86.102:6572",
"/": "Backend实例ID",
"BackendInstanceId": 0,
"BackendId": 1,
"/": "是否在运行期创建",
"IsDynamicCreate": false,
"/": "端接收消息计数",
"RecvCount": 0,
"/": "端发送成功消息计数",
"SendSuccessCount": 0,
"/": "端发送失败消息计数",
"SendFailCount": 0,
"/": "端接收拦截消息计数",
"InterceptCount": 0,
"/": "端接收错误应答消息计数",
"ErrorRespCount": 0,
"/": "端发送主核心成功消息计数",
"SendMasterSuccessCount": 0,
"/": "端发送主核心失败消息计数",
"SendMasterFailCount": 0,
"/": "端发送备核心成功消息计数",
"SendBackupSuccessCount": 0,
"/": "端发送备核心失败消息计数",
"SendBackupFailCount": 0,
"/": "收到注册应答个数",
"RecvRegRespCount": 0,
"/": "发送注册消息成功个数",
"SendRegSuccCount": 0,
"/": "发送注册消息失败个数",
"SendRegFailCount": 0,
"/": "未就绪，过滤消息个数",
"UnReadyDropCount": 0,
"/": "端配置名称",
"ConfigName": "xst",
"/": "端类型",
"Type": "AS_TCP",
"/": "端通信消息类型",
"MsgType": "LDP",
"/": "是否使用仲裁",
"IsArb": true,
"/": "产品名",
"ProductName": "ldp",
"/": "集群名",
"ClusterName": "xst",
"/": "端维护的节点个数",
"SiteInUse": 2,
"/": "是否开启回话",
"IsEnableSession": false,
"/": "是否优先去备",
"IsEnableQueryFirst": false,
"/": "节点列表",
"Sites": [
{
"/": "节点索引",
"SiteIndex": 0,
"/": "节点名称",
"SiteName": "xst#0",
"/": "节点IP",
"IP": "192.168.86.101",
"/": "节点端口",
"Port": 10111,
"/": "节点激活状态",
"ActiveStatus": true,
"/": "站点对应的域名",
"DomainName": "www.test.com",
"/": "节点ID",
"SiteId": 1,
"/": "节点连接状态",
"State": "Connected",
"/": "节点连接成功次数",
"ConnectTimes": 1,
"/": "节点接收消息计数",
"RecvCount": 0,
"/": "节点发送成功消息计数",
"SendSuccessCount": 0,
"/": "节点发送失败消息计数",
"SendFailCount": 0,
"/": "节点拦截消息计数",
"InterceptCount": 0
}
],
"/": "主核心节点索引",
"MasterSiteIndex": 1,
"/": "主核心节点名",
"MasterNode": "xst#0"
}
},
"RcmCompaction": {
"/": "使用RcmCompaction时的Rcm接收消息计数",
"RcmCompRecvCount": 0,
"/": "使用RcmCompaction时的Rcm接收错误消息计数",
"RcmCompRecvErrCount": 0
},
"/": "Arb Watch事件列表",
"Watches": [
{
"/": "Watch集群",
"Watcher": "ldp_xst",
"/": "事件信息",
"Events": [
{
"/": "端ID",
"BackendId": 1,
"/": "Arb路径",
"ArbPath": "/ldp/ldp/arb",
"/": "集群名",
"Cluster": "xst"
}
]
}
]
}
}
}

```

6.7.15. GetNetworkServiceConfig 获取前置的接入服务信息

功能名称	GetNetworkServiceConfig	版本号	V1.0.0.0	更新日期	20220920
功能备注	IP和Port信息和配置保持一致				
功能说明	通过本管理功能可以获取前置的接入服务信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_front&FuncName=GetNetworkServiceConfig				
入参说明					
无					
出参说明					
<pre>{ "Lan": { "Name": "Lan", "Tcp工厂名": "Tcp工厂名", "FactoryName": "Default", "发送等待时间": 0, "SendWaitTimeMilli": 0, "当配置中没有Address配置时, 此处为空字符串", "Address": "192.168.86.102", "ip黑名单": "192.168.86.102", "AddressBlackList": "192.168.86.102", "ip白名单": "192.168.86.102", "AddressWhiteList": "192.168.86.102", "端口": 23572, "Port": 23572, "SocketReuseAddress": false, "心跳间隔": 100, "HeartbeatIntervalSec": 100, "心跳次数": 2, "HeartbeatTimes": 2, "最大连接数": 1000, "MaxConnectCount": 1000, "最大发送缓存大小": 23, "MaxSendBuffSizeBytes": 23, "最小接收缓存大小": 512, "MinRecvBuffSizeBytes": 512, "是否为异步": false, "EnableAsyncSend": false, "配置是否启用直接recv接收": false, "EnableDirectRecv": false, "是否收发合并": false, "MergeSendRecvThread": false, "收发合并线程数": 1, "SendRecvThreadCount": 1, "收发分离发送线程": 2, "SendThreadCount": 2, "收发分离接收线程": 2, "RecvThreadCount": 2, "一个侦听最多允许多少连接": 1000, "MaxConnectPerAcceptor": 1000, "线程绑核": "", "ThreadCoreBind": "", "快速ack": false, "EnableQuickAck": false, "是否开启Nagle": true, "EnableNagle": true }, "Wan": { "Name": "Wan", "Tcp工厂名": "Tcp工厂名", "FactoryName": "Default", "发送等待时间": 0, "SendWaitTimeMilli": 0, "当配置中没有Address配置时, 此处为空字符串", "Address": "192.168.86.102", "端口": 13572, "Port": 13572, "地址重用": false, "SocketReuseAddress": false, "心跳间隔": 100, "HeartbeatIntervalSec": 100, "心跳次数": 2, "HeartbeatTimes": 2, "最大连接数": 1000, "MaxConnectCount": 1000, "最大发送缓存大小": 23, "MaxSendBuffSizeBytes": 23, "最小接收缓存大小": 512, "MinRecvBuffSizeBytes": 512, "是否为异步": false, "EnableAsyncSend": false, "配置是否启用直接recv接收": false, "EnableDirectRecv": false, "是否收发合并": false, "MergeSendRecvThread": false, "收发合并线程数": 1, "SendRecvThreadCount": 1, "收发分离发送线程": 2, "SendThreadCount": 2, "收发分离接收线程": 2, "RecvThreadCount": 2, "一个侦听最多允许多少连接": 1000, "MaxConnectPerAcceptor": 1000, "线程绑核": "", "ThreadCoreBind": "", "快速ack": false, "EnableQuickAck": false, "是否开启Nagle": true, "EnableNagle": true } }</pre>					

6.7.16. RpcClientStat 获取前置的rpc信息

功能名称	RpcClientStat	版本号	V1.0.0.0	更新日期	20240507
------	---------------	-----	----------	------	----------

功能备注	
功能说明	通过本管理功能可以获取前置的rpc信息
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_front&FuncName=RpcClientStat
入参说明	
无	
出参说明	
<pre>{ "Rpc": [{ "HostPlugin": "ldp_front", "RpcClientId": 0, "LastPacketId": 0, "FirstPacketId": 16776191, "StartPacketId": 0, "ReqCount": 0, "RspCount": 0, "IsWaiting": false, "TotalReqCount": 0, "TotalRspCount": 0, "TotalTimeoutRspCount": 0, "MaxPacketId": 16777215 }] }</pre>	

6.7.17. SetFrontRoute 设置前端路由

功能名称	SetFrontRoute	版本号	V1.0.15.16	更新日期	20250612
功能备注	设置前端路由配置				
功能说明	此功能用于设置前端路由配置,确保新路由的版本号和当前路由版本一致,每次更新后会使版本加一。需要在配置中将EnableDynamicRoutes设置为true。				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_backend&FuncName=SetFrontRoute				

入参说明

```
{  "/*": "路由配置版本号, 从管理功能RouteStat中获取",  "Version": 1,  "/*": "路由配置具体内容",  "Routes": [    {      "Functions": "*",      "SystemNo": "*",      "NodeNo": "*",      "BackendID": 1    }  ]}
```

出参说明

```
{  "/*": "操作结果",  "UpdateResult": "Success"}
```

6.8. 业务框架插件 ldp_bizproc

6.8.1. GetArbStatus 仲裁状态

功能名称	GetArbStatus	版本号	V1.0.0.0	更新日期	20200630
功能备注	仲裁状态				
功能说明	获取仲裁状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&FuncName=GetArbStatus				
入参说明					
无					
出参说明					

<pre>{ "":"/: "主备状态", "ArbStatus": "ARB_ACTIVE", "":"/: "高可用模式", "HaMode": "ActiveStandby", "":"/: "集群名", "TierName": "hc", "":"/:"仲裁状态最新变化完成时间", "ArbStatusChangeTime":"2026-01-01 00:00:00.000000", "":"/:"仲裁状态最新开始变化时间", "ArbStatusBeginChangeTime":"2026-01-01 00:00:00.000000" }</pre>

6.8.2. GetProcStatus proc状态

功能名称	GetProcStatus	版本号	V1.0.0.0	更新日期	20200630
功能备注	proc状态				
功能说明	获取proc状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetProcStatus				
入参说明					
无					
出参说明					
<pre>{ "":"/: "proc运行状态", "State": "Running", "":"/: "高可用模式", "HaMode": "ActiveStandby", "":"/: "是否是本地模式", "IsLocalMode": false}</pre>					

6.8.3. GetWorkQueueInfo 工作线程队列信息

功能名称	GetWorkQueueInfo	版本号	V1.0.1.0	更新日期	20201022
功能备注	工作线程队列信息				
功能说明	获取工作线程队列信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetWorkQueueInfo				
入参说明					
无					
出参说明					
<pre>{ "WorkQueueInfo": [{ "":"/: "线程序号", "ThreadId": 0, "":"/: "线程绑定cpu序号", "ThreadBindCpuId": 1, "":"/: "线程执行任务数", "ProcessedCount": 0, "":"/: "线程唤醒次数", "NotifyCount": 0, "":"/: "队列积压消息个数", "QueueCount": 0, "":"/: "队列别名", "QueueAlias": 1, "":"/: "线程轮询次数", "PollingLoops": 10000000, "":"/: "睡眠时间", "SleepTimeUS": 0, "":"/: "队列大小上限", "MaxQueueSize": 32768, "":"/: "队列使用内存", "InUseSizeKB": 4096, "":"/: "最后处理的功能号", "FunctionNo": 108, "":"/: "最后处理的功能号对应的子系统号", "SystemNo": 100, "":"/: "最后处理的功能号是否正在处理", "Running": false, "":"/: "工作队列历史缓存最大消息个数", "MaxHisQueueCount": 3, "":"/: "非工作线程调用ExecAllPreTasks时, 当前工作线程的状态", "NonWorkThreadStaus": false, "":"/: "非工作线程调用ExecAllPreTasks时, 当前工作线程的轮次号", "NonWorkThreadRoundNum": 3, "":"/: "工作线程调用ExecAllPreTasks时, 当前工作线程的状态", "WorkThreadStaus": false, "":"/: "工作线程调用ExecAllPreTasks时, 当前工作线程的轮次号", "WorkThreadRoundNum": 3 }] }</pre>					

6.8.4. GetPipelineQueueInfo 流水线线程队列信息

功能名称	GetPipelineQueueInfo	版本号	V1.0.1.0	更新日期	20201016
功能备注	流水线线程队列信息				

功能说明	获取流水线线程队列信息
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_bizproc&FuncName=GetPipelineQueueInfo
入参说明	
无	
出参说明	
<pre>{ "GetPipelineQueueInfo": [{ "ThreadId": 0, "ThreadBindCpuId": 1, "ProcessedCount": 0, "QueueCount": 0, "QueueAlias": 1, "PollingLoops": 10000000, "SleepTimeUS": 0, "MaxQueueSize": 32768, "InUseSizeKB": 4096, "FunctionID": 108, "SystemNo": 100, "NextOriPipelineStep": 2, "CurOriPipelineStep": 1, "CurVerPipelineStep": 1, "Running": false, "NonWorkThreadStaus": false, "NonWorkThreadRoundNum": 3, "WorkThreadStaus": false, "WorkThreadRoundNum": 3 }] }</pre>	

6.8.5. GetGlobalQueueInfo 全局队列信息

功能名称	GetGlobalQueueInfo	版本号	V1.0.5.1	更新日期	20220421
功能备注	全局队列信息				
功能说明	获取全局队列信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_bizproc&FuncName=GetGlobalQueueInfo				
入参说明					
无					
出参说明					
<pre>{ "GlobalQueueInfo": { "QueueCount": 0, "QueueSizeKB": 2048, "InUsedSizeKB": 0, "ProcessedCount": 0, "MaxHisQueueCount": 0, "QueueExpandCount": 0, "LocateFailCount": 0, "WorkThreadsInfo": [{ "ThreadId": 0, "QueueCount": 0, "MaxHisQueueCount": 0, "ProcessedCount": 0 }] } // 其他工作线程信息...</pre>					

6.8.6. GetTransactionNo 事务号

功能名称	GetTransactionNo	版本号	V1.0.0.0	更新日期	20200630
功能备注	事务号				
功能说明	获取事务号				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetTransactionNo				
入参说明					
无					
出参说明					
<pre>{ "/*": "高可用模式", "HaMode": "ActiveStandby", "/*": "是否本地模式", "IsLocalMode": false, "/*": "主备状态", "ArbStatus": "ARB_ACTIVE", "/*": "当前重放redo事务号, 多分区模式下不展示", "ReplayTransactionNo": 13, "/*": "当前写redo事务号", "WriterTransactionNo": 0, "/*": "写redo故障状态, 故障时, 无法更新redo文件", "IsWriteRedoFault": false, "/*": "多分区模式下各分区组信息", "NodeTransInfo": [{ "/*": "分区组编号, 与分区组一一对应", "PartGroupNo": 2, "/*": "分区组名", "PartGroupName": "multiproc_mdb#2_partgroup_3", "/*": "当前读分区组事务号 (非下一个), 非读就是-1", "ReplayTransactionNo": 18446744073709551615, "/*": "当前写分区组事务号 (非下一个), 非写就是-1", "WriterTransactionNo": 5 }, { "PartGroupNo": 1, "PartGroupName": "multiproc_mdb#2_partgroup_2", "ReplayTransactionNo": 4, "WriterTransactionNo": 18446744073709551615 }, { "PartGroupNo": 0, "PartGroupName": "multiproc_mdb#2_partgroup_1", "ReplayTransactionNo": 0, "WriterTransactionNo": 18446744073709551615 }] }</pre>					

6.8.7. GetBizDataInfo 业务数据组件信息

功能名称	GetBizDataInfo	版本号	V1.0.0.0	更新日期	20200630
功能备注	业务数据组件信息				
功能说明	获取业务数据组件信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetBizDataInfo				
入参说明					
无					
出参说明					
<pre>{ " GetBizDataInfo ": [{ "/*": "路径", "Path": "/home/solarflare/ldp/bin/hc.local/libxstdata.so", "/*": "版本", "Version": "xstdata V1.0.0.0", "/*": "业务版本号, 用于接口兼容", "BizVersion": 1, "/*": "子系统号", "SystemNo": 100, "/*": "是否初始化", "Inited": true }] }</pre>					

6.8.8. GetBizModuleInfo 业务功能组件信息

功能名称	GetBizModuleInfo	版本号	V1.0.0.0	更新日期	20211126
功能备注	业务功能组件信息				
功能说明	获取业务功能组件信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetBizModuleInfo				
入参说明					

无
出参说明
<pre>{ "GetBizModuleInfo": [{ "///": "子系统号", "SystemNo": 100, "BizModule": [{ "///": "路径", "Path": "/home/solarflare/ldp/bin/hc.local/modules/libxstfunc.so", "///": "版本", "Version": "xstfunc V1.0.0.0", "///": "是否初始化", "Inited": true }] }]}</pre>

6.8.9. GetFuncDealInfo 业务功能处理信息

功能名称	GetFuncDeallInfo	版本号	V1.0.0.0	更新日期	20200630
功能备注	业务功能处理信息				
功能说明	获取业务功能处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&FuncName=GetFuncDeallInfo				
入参说明					
<pre>{ "///": "需要查看功能的统计信息，默认所有。英文分号分隔，如“11;30?:319”，?号表示一个数字，如11??表示1100-1199范围,可以只出现?,如??表示0-99。'*'号代表支持所有的功能号", "FuncList": "123;1?:456346;*", "///": "是否只查询处理过的功能号, 'true'为开启上述筛选,其他输入为不筛选，默认值为 'false'", "ExecOnly": false}</pre>					
出参说明					

```
{
  " GetFuncDealInfo": [
    {
      "FunctionID": 5,
      "SystemNo": 100,
      "LocateMode": 0,
      "TotalExecNum": 0,
      "FailExecNum": 0,

      "SumExecTimeNs": 0,
      "MaxExecTimeNs": 0,
      "MinExecTimeNs": 0,
      "LastExecTimeNs": 0,
      "AveExecTimeNs": 0,

      "SumQueueTimeNs": 0,
      "MaxQueueTimeNs": 0,
      "MinQueueTimeNs": 0,
      "LastQueueTimeNs": 0,
      "AveQueueTimeNs": 0,

      "SumLocateTimeNs": 0,
      "MaxLocateTimeNs": 0,
      "MinLocateTimeNs": 0,
      "LastLocateTimeNs": 0,
      "AveLocateTimeNs": 0,

      "SumTotalTimeNs": 0,
      "MaxTotalTimeNs": 0,
      "MinTotalTimeNs": 0,
      "LastTotalTimeNs": 0,
      "AveTotalTimeNs": 0,

      "SumRedoTimeNs": 0,
      "MaxRedoTimeNs": 0,
      "MinRedoTimeNs": 0,
      "LastRedoTimeNs": 0,
      "AveRedoTimeNs": 0
    }
  ]
}
```

6.8.10. ClearFuncDealInfo 清除业务功能处理信息

功能名称	ClearFuncDealInfo	版本号	V1.0.0.0	更新日期	20200630
功能备注	业务功能处理信息				
功能说明	清除业务功能处理信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=ClearFuncDealInfo				
入参说明					
无					
出参说明					
<pre>{ "ClearFuncDealInfo": "OK" }</pre>					

6.8.11. GetFuncInfo 业务功能信息

功能名称	GetFuncFuncInfo	版本号	V1.0.0.0	更新日期	20200630
功能备注	业务功能信息				
功能说明	获取业务功能信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetFuncInfo				
入参说明					
无					

出参说明
<pre>{ "GetFuncInfo": [{ "/*": "功能名, 功能号0为平台内部使用功能号, 名字为ErrRsp", "FunctionName": "F3", "/*": "功能号", "FunctionNo": 3, "/*": "子系统号", "SystemNo": 0, "/*": "读写标志, 暂时未使用", "Flag": 0, "/*": "可运行的业务状态", "AllowBizStatus": 18446744073709551615, "/*": "定位模式", "LocateMode": 0, "/*": "请求结构", "ReqStruct": "", "/*": "请求可选字段", "ReqOptionalFields": "", "/*": "应答结构", "RspStruct": "", "/*": "应答可选字段", "RspOptionalFields": "" }] }</pre>

6.8.12. GetErrorProcessCount 错误处理个数

功能名称	GetErrorProcessCount	版本号	V1.0.1.0	更新日期	20210713
功能备注	错误处理个数				
功能说明	获取错误处理个数				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetErrorProcessCount				
入参说明					
无					
出参说明					
<pre>{ "/*": "处理不支持的功能号个数", "UnSpFuncCount": 0, "/*": "处理定位错误个数", "LocateFailCount": 0, "/*": "流水线调用错误个数", "PipelineFailCount": 0, "/*": "接收的应答消息个数", "RecvResponseCount": 0, "/*": "暂停状态下拦截的请求个数", "RejectCount": 0, "/*": "定位模式重试个数", "RetryCount": 0, "/*": "黑名单中忽略的请求总个数", "BlackListIgnoreCount": 0 }</pre>					

6.8.13. GetConnectInfo 获取连接信息

功能名称	GetConnectInfo	版本号	V1.0.0.0	更新日期	20200630
功能备注	获取连接信息				
功能说明	获取连接信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetConnectInfo				
入参说明					
<pre>{ "/*": "起始连接号, 默认值0", "StartIndex": 0, "/*": "查询个数, 默认值100", "QueryCount": 100, "/*": "端名, 默认值为空字符串,填写后会匹配对应端名的配置返回", "FrontendName": "", "/*": "选定查看的类型, 默认值为 'Connections'", "Type": "" }</pre>					
出参说明					

```
{
  "":"/": "宿主插件名",
  "HostPlugin": "ldp_bizproc",
  "":"/": "服务端过滤的主机收到的去备消息或备机收到的去主消息个数",
  "AsModeDropTotalCount": 0,
  "":"/": "服务端未就绪过滤的消息个数",
  "UnReadyDropTotalCount": 0,
  "GetConnectInfo": [
    {
      "":"/": "前端名",
      "FrontendName": "ActiveStandby",
      "":"/": "连接号",
      "ConnectId": 1073807360,
      "":"/": "连接索引",
      "ConnectIndex": 65536,
      "":"/": "发送次数",
      "SendTimes": 9883,
      "":"/": "接收次数",
      "RecvTimes": 9883,
      "":"/": "最近一次连接建立时间",
      "ConnectedTimeSec": 1712469299,
      "":"/": "最近一次发送完成时间",
      "LastSendedTimeSec": 1712477537,
      "":"/": "最近一次接收完成时间",
      "LastRecvedTimeSec": 1712477537,
      "":"/": "客户端地址",
      "Address": "192.168.88.157",
      "":"/": "客户端端口",
      "Port": 65096,
      "":"/": "协议",
      "Protocol": "IDP",
      "":"/": "非法报文拦截次数",
      "InterceptTimes": 0,
      "":"/": "收到除了心跳的消息个数",
      "RecvTimesNoHeartbeat": 0,
      "":"/": "发送成功次数, 不包含心跳",
      "SendSuccessTimes": 0,
      "":"/": "发送失败次数, 不包含心跳",
      "SendFailTimes": 0,
      "":"/": "客户端类型",
      "ClientType": "None",
      "":"/": "客户端进程号",
      "ClientPid": 0,
      "":"/": "客户端版本",
      "ClientVersion": "",
      "":"/": "使用客户端代理的真实客户端ip地址",
      "ClientAddressByProxy": "",
      "":"/": "使用客户端代理的真实客户端端口",
      "ClientPortByProxy": 0
    }
  ]
}
```

6.8.14. GetNetworkServiceConfig 网络配置信息

功能名称	GetNetworkServiceConfig	版本号	V1.0.0.0	更新日期	20200630
功能备注	网络配置信息				
功能说明	获取网络配置信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&FuncName=GetNetworkServiceConfig				
入参说明					
无					
出参说明					

```
{
  "ActiveStandby": {
    "Name": "ActiveStandby",
    "FactoryName": "Default",
    "Address": "服务器侦听地址",
    "AddressBlackList": "",
    "AddressWhiteList": "",
    "Port": 8001,
    "SocketReuseAddress": true,
    "HeartbeatIntervalSec": 20,
    "HeartbeatTimes": 3,
    "MaxConnectCount": 10000,
    "MaxSendBuffSizeBytes": 0,
    "MinRecvBuffSizeBytes": 512,
    "MergeSendRecvThread": true,
    "EnableAsyncSend": false,
    "SendWaitTimeMilli": 0,
    "SendRecvThreadCount": 1,
    "SendThreadCount": 0,
    "RecvThreadCount": 1,
    "MaxConnectPerAcceptor": 10000,
    "ThreadCoreBind": "7,8,9",
    "EnableQuickAck": false,
    "EnableNagle": true
  },
  "ActiveActive": {
    "GetProcRcmConfig": [
      {
        "ContextName": "xst#0_ldp_bizproc_0_0",
        "TierName": "xst_trade",
        "Reliability": "Tier",
        "Role": "RCM_SINGLE"
      }
    ]
  }
}
```

6.8.15. GetWorkQueueRequests 工作线程队列请求信息

功能名称	GetWorkQueueRequests	版本号	V1.0.1.0	更新日期	20201022
功能备注	工作线程队列请求信息				
功能说明	获取工作线程队列请求信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&FuncName=GetWorkQueueRequests				
入参说明					
<pre>{ "QueryCount": 100 }</pre>					
出参说明					

<pre>{ "GetWorkQueueRequests": [{ "ThreadId": 0, "ThreadBindCpuId": 1, "ProcessedCount": 0, "QueueCount": 0, "QueueAlias": 1, "Requests": [{ "FunctionID": 100, "Length": 128, "EnqueueTimeNs": 18042859938154846 }, { "FunctionID": 101, "Length": 128, "EnqueueTimeNs": 18042860938602624 }] }] }</pre>
--

6.8.16. GetPipelineQueueRequests 流水线线程队列请求信息

功能名称	GetPipelineQueueRequests	版本号	V1.0.1.0	更新日期	20201022
功能备注	流水线线程队列请求信息				
功能说明	获取流水线线程队列请求信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetPipelineQueueRequests				

入参说明

```
{  "QueryCount": 100}
```

出参说明

```
{  "GetPipelineQueueRequests": [    {      "ThreadId": 0,      "ThreadBindCpuId": 1,      "ProcessedCount": 0,      "QueueCount": 0,      "QueueAlias": 1,      "Requests": [        {          "FunctionID": 100,          "Length": 128,          "EnqueueTimeNs": 18042859938154846,          "CurOriPipelineStep": 1,          "CurVerPipelineStep": 2        },        {          "FunctionID": 101,          "Length": 128,          "EnqueueTimeNs": 18042860938602624,          "CurOriPipelineStep": 1,          "CurVerPipelineStep": 2        }      ]    }  ]}
```

6.8.17. GetRpcStatistics 获取RPC相关监控信息

功能名称	GetRpcStatistics	版本号	V1.0.3.0	更新日期	20210517
功能备注	RPC监控信息				
功能说明	获取RPC相关监控信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetRpcStatistics				
入参说明					


```
{
}
```

出参说明

```
{
  "/*": "路由未命中次数",
  "RouteMissCount": 0,
  "/*": "路由配置",
  "Routes": [
    {
      "/*": "功能号",
      "Functions": "900018",
      "/*": "目标系统号",
      "SystemNo": "*",
      "/*": "目标节点号",
      "NodeNo": "*",
      "/*": "目标端ID",
      "BackendID": 1,
      "/*": "转发次数",
      "Count": 0
    }
  ],
  "Backends": [
    {
      "/*": "宿主插件名",
      "HostPlugin": "ldp Bizproc",
      "/*": "端ID",
      "BackendId": 1,
      "/*": "是否在运行期创建",
      "IsDynamicCreate": false,
      "/*": "端接收消息计数",
      "RecvCount": 0,
      "/*": "端发送成功消息计数",
      "SendSuccessCount": 0,
      "/*": "端发送失败消息计数",
      "SendFailCount": 0,
      "/*": "端接收拦截消息计数",
      "InterceptCount": 0,
      "/*": "端接收错误应答消息计数",
      "ErrorRspCount": 0,
      "/*": "端发送主核心成功消息计数",
      "SendMasterSuccessCount": 0,
      "/*": "端发送主核心失败消息计数",
      "SendMasterFailCount": 0,
      "/*": "端发送备核心成功消息计数",
      "SendBackupSuccessCount": 0,
      "/*": "端发送备核心失败消息计数",
      "SendBackupFailCount": 0,
      "/*": "收到注册应答个数",
      "RecvRegRspCount": 0,
      "/*": "发送注册消息成功个数",
      "SendRegSuccCount": 0,
      "/*": "发送注册消息失败个数",
      "SendRegFailCount": 0,
      "/*": "未就绪, 过滤消息个数",
      "UnReadyDropCount": 0,
      "/*": "端配置名称",
      "ConfigName": "AS-TCP-1",
      "/*": "端类型",
      "Type": "AS TCP",
      "/*": "端通信消息类型",
      "MsgType": "LDP",
      "/*": "端维护的节点个数",
      "SiteInUse": 1,
      "/*": "是否使用仲裁",
      "IsArb": false,
      "/*": "是否开启回话",
      "IsEnableSession": false,
      "/*": "是否优先去备",
      "IsEnableQueryFirst": false,
      "Sessions": [
        {
          "/*": "节点索引",
          "SiteIndex": 0,
          "/*": "节点名称",
          "SiteName": "xst#0",
          "/*": "节点IP",
          "IP": "192.168.36.33",
          "/*": "节点端口",
          "Port": 8009,
          "/*": "节点激活状态",
          "ActiveStatus": false,
          "/*": "节点机房名",
          "ZoneName": "",
          "/*": "节点连接成功次数",
          "ConnectTimes": 0,
          "/*": "节点接收消息计数",
          "RecvCount": 0,
          "/*": "节点发送成功消息计数",
          "SendSuccessCount": 0,
          "/*": "节点发送失败消息计数",
          "SendFailCount": 0,
          "/*": "节点拦截消息计数",
          "InterceptCount": 0,
          "/*": "节点ID",
          "SiteId": 1,
          "/*": "节点连接状态",
          "State": "Disconnected"
        }
      ],
      "/*": "主核心节点索引",
      "MasterSiteIndex": 0,
      "/*": "主核心节点名",
      "MasterNode": "",
      "/*": "查询优先模式下上一次发送的SiteIndex",
      "ToBackupLastIndex": 0
    }
  ],
  "RcmCompaction": {
    "/*": "使用RcmCompaction时的Rcm接收消息计数",
    "RcmCompRecvCount": 0,
    "/*": "使用RcmCompaction时的Rcm接收错误消息计数",
    "RcmCompRecvErrCount": 0
  }
}
```

```

    },
    "/*": "Arb Watch事件列表",
    "Watches": [
        {
            "/*": "Watch集群",
            "Watcher": "ldp_test_xst1",
            "/*": "事件信息",
            "Events": [
                {
                    "/*": "端ID",
                    "BackendId": 1,
                    "/*": "Arb路径",
                    "ArbPath": "/ldp/ldp_test_0123tester/arb",
                    "/*": "集群名",
                    "Cluster": "xst1"
                }
            ]
        }
    ]
},
"RcmProxy": {
    "/*": "调用者名字",
    "UserName": "xst#0_ldp_bizproc_0_backend",
    "/*": "本机IP或者网段",
    "LocalNetworkNumber": "192.168.36.33",
    "/*": "协议类型",
    "ProtocolMode": "UDP-MULTICAST",
    "/*": "应答主题",
    "RspTopic": {
        "/*": "主题",
        "Topic": "ucrt.result",
        "/*": "分区号",
        "PartitionNo": 3,
        "/*": "主题ID",
        "TransportId": 3,
        "/*": "是否为私有主题",
        "IsPrivate": false,
        "/*": "该主题是否启用单播模式",
        "EnableUnicastMode": false
    },
    "Singleton": {
        "/*": "引用的上下文模板",
        "Ref": "Default",
        "/*": "Singleton通道号",
        "SingletonId": 0,
        "/*": "上下文名",
        "ContextName": "xst#0_ldp_bizproc_0_backend_Singleton",
        "/*": "上下文ID",
        "ContextId": 3,
        "/*": "是否自动启动",
        "IsAutoStart": true,
        "/*": "是否多进程上下文",
        "IsMsgSync": false,
        "/*": "是否创建Consumer",
        "IsConsumerCreate": false,
        "/*": "发送主题配置",
        "TxTopics": [
            {
                "/*": "发送主题名",
                "Name": "ucrt.rpc",
                "/*": "是否启用多分片模式",
                "AliaName": "ucrt.rpc-2",
                "/*": "发送分区号",
                "PartitionNos": []
            }
        ],
        "/*": "接收主题配置",
        "RxTopics": [],
        "/*": "发送旁路主题配置",
        "TxBypass": []
    },
    "Acceptor": {
        "/*": "发送成功计数 (不含心跳包)",
        "SendSuccessCount": 0,
        "/*": "发送失败计数 (不含心跳包)",
        "SendFailCount": 0,
        "/*": "发送注册包成功计数",
        "SendRegSuccCount": 0,
        "/*": "发送注册包失败计数",
        "SendRegFailCount": 0,
        "/*": "接收注册包消息计数",
        "RecvRegCount": 0,
        "/*": "非高可用上下文接收合法消息计数 (不含心跳包)",
        "RecvSingletonsCount": 0,
        "/*": "高可用上下文接收合法消息计数",
        "RecvTierCount": 0,
        "/*": "旁路上下文接收消息计数",
        "RecvBypassCount": 0,
        "/*": "非高可用上下文接收非法消息计数",
        "RecvSingletonsDropCount": 0,
        "/*": "高可用上下文接收非法消息计数",
        "RecvTierDropCount": 0
    },
    "/*": "客户端统计信息",
    "Initiator": [
        {
            "/*": "实际发送主题名",
            "Name": "ucrt.rpc",
            "/*": "别名",
            "AliaName": "ucrt.rpc-2",
            "/*": "是否属于集群通道",
            "IsTier": false,
            "/*": "Singleton通道号",
            "SingletonId": 0,
            "/*": "全双工通道连接状态",
            "IsConnected": true,
            "/*": "发送成功计数",
            "SendSuccessCount": 0,
            "/*": "发送失败计数",
            "SendFailCount": 0,
            "/*": "接收消息计数",
            "RecvCount": 0
        }
    ],
    "Publisher": [],
    "ContextHandle": [
        {

```

```

        "端名",
        "EndName": "",
        "上下文名",
        "ContextName": "xst#0_ldp_bizproc_0_backend_Singleton",
        "是否是多进程上下文",
        "IsMultiProc": false,
        "多进程上下文句柄是否有效",
        "IsMultiProcValid": false
    }
},
"Full duplex": [
    {
        "Singleton通道号",
        "SingletonId": 0,
        "initiator发送主题",
        "TxTopic": "ucrt.rpc",
        "initiator发送主题分区号",
        "TxPartitions": "",
        "initiator接收主题",
        "RxTopic": "ucrt.result",
        "initiator接收主题分区号",
        "RxPartition": "3",
        "initiator全双工通道连接状态",
        "IsConnected": true,
        "ConnectionInfo": [
            {
                "对端上下文名称",
                "PeerContext": "xst#0_ldp_bizproc_0_Singleton",
                "对端上下文ID",
                "PeerContextId": 1,
                "上行分区号",
                "UpPartition": 1,
                "上行本端就绪状态",
                "UplinkLocalStat": "Ready",
                "上行对端就绪状态",
                "UplinkLPeerStat": "Ready",
                "下行本端就绪状态",
                "DownlinkLocalStat": "Ready",
                "下行对端就绪状态",
                "DownlinkPeerStat": "Ready"
            }
        ]
    }
]
},
"Rpc": [
    {
        "宿主插件名",
        "HostPlugin": "ldp_bizproc",
        "RPC客户端ID",
        "RpcClientId": 0,
        "最近一次RPC的请求编号",
        "LastPacketId": 0,
        "一次批量RPC的第一笔请求编号",
        "FirstPacketId": 16776191,
        "一次业务请求的第一次RPC的请求编号",
        "StartPacketId": 0,
        "当前批量RPC的请求个数",
        "ReqCount": 0,
        "当前批量RPC已经接收到的应答个数",
        "RspCount": 0,
        "是否正在等待应答",
        "IsWaiting": false,
        "历史发送的请求总数",
        "TotalReqCount": 0,
        "历史接收的应答总数",
        "TotalRspCount": 0,
        "历史接收的超时应答总数",
        "TotalTimeoutRspCount": 0,
        "最大合法请求编号",
        "MaxPacketId": 16777215
    }
]
}
}

```

6.8.18. GetPushServiceStatistics主推服务统计信息

功能名称	GetPushServiceStatistics	版本号	V1.0.3.0	更新日期	20210517
功能备注	主推服务统计信息				
功能说明	获取主推服务统计信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_bizproc&FuncName=GetPushServiceStatistics				
入参说明					
<pre>{ }</pre>					
出参说明					
<pre>{ "PushService": [{ "主子": "主推服务名", "Name": "", "主子": "服务类型", "Type": "", "主子": "主推次数（包含主推失败的次数）", "PushCount": 0, "主子": "主推失败次数", "PushFailCount": 0 }] }</pre>					

6.8.19. GetOrSetRunningStatus获取或设置框架运行状态

功能名称	GetOrSetRunningStatus	版本号	V1.0.3.4	更新日期	20210713
功能备注	获取或设置框架运行状态				
功能说明	获取或设置框架运行状态。如果框架为暂停状态，则无法处理任何请求，且不会有错误应答				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetOrSetRunningStatus				

入参说明

```
{  "Action": "Pause/Resume"}
```

出参说明

```
{  {    "ErrorNo": 0,    "ErrorMsg": "",    "RunningStatus": "Pause/Running"  },  {    "PauseFlag": "Pause/Running"  } }
```

6.8.20. GetMsgnoStatus 获取消息序号处理状态

功能名称	GetMsgnoStatus	版本号	V1.0.0.0	更新日期	20210924
功能备注	高可靠上下文的消息序号处理状态				
功能说明	获取消息序号处理状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetMsgnoStatus				
入参说明					
无					
出参说明					
<pre>{ "MsgnoStat": [{ "TxContextId": 8, "TransportId": 4, "ExpectTxMsgno": 20000, "Thread": {} }] }</pre>					

6.8.21. GetBizFuncIdTsListStatus 获取当前支持时延分析功能号列表

功能名称	GetBizFuncIdTsListStatus	版本号	V1.0.0.13	更新日期	20220314
功能备注	获取当前支持时延分析功能号列表				
功能说明	获取当前支持时延分析功能号列表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetBizFuncIdTsListStatus				
入参说明					
无					
出参说明					
<pre>{ "":"/": "支持时延分析功能号列表", "BizFuncIdTsList": "123;1;456346" }</pre>					

6.8.22. SetBizFuncIdTsList 设置支持时延分析功能号列表

功能名称	SetBizFuncIdTsList	版本号	V1.0.0.13	更新日期	20220314
功能备注	设置支持时延分析功能号列表				
功能说明	设置支持时延分析功能号列表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=SetBizFuncIdTsList				

入参说明
入参支持"*": 全部功能号都打点; "1?": 支持模糊匹配; 支持"clear":表示之前设置的时延分析功能号清空。
<pre>{ "/*": "支持时延分析功能号列表", "TimeStampFuncList": "123;1?;456346" }</pre>
出参说明
<pre>{ "/*": "ErrorNo为0, 则代表设置成功; 若不为0, 则代表失败。ErrorMsg为具体失败信息", "ErrorNo": 0, "ErrorMsg": "业务框架设置时延分析配置成功" }</pre>

6.8.23. GetReqLogWritersInfo 获取请求落地功能的相关信息

功能名称	GetReqLogWritersInfo	版本号	V1.0.5.2	更新日期	20220520
功能备注	获取请求落地功能的相关信息				
功能说明	获取请求落地功能的相关信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetReqLogWritersInfo				
入参说明					
无					
出参说明					
<pre>{ "/*": "是否重演模式", "IsReplyMode": false, "/*": "使能写请求开关", "WriteFlag": true, "/*": "写请求路径", "WritePath": "./reqlog/", "/*": "单文件最大值", "MaxFileSizeMB": 1, "/*": "当前记录的消息号", "CurMsgNo": 0, "/*": "工作线程请求落地相关信息", "WorkThreadWritersInfo": { "/*": "工作线程编号", "0": { "/*": "是否开启", "WriteFlag": true, "/*": "当前写文件夹名", "CurWriteDir": "20241018-211111-475025868", "/*": "当前写文件名", "CurOperFile": "00000-000000000000.dat", "/*": "落地调用的总次数", "TotalCount": 0, "/*": "落地成功的次数", "SuccessCount": 0, "/*": "当前打开的文件写了多少, 单位MB", "CurrFileWriteMB": 0.0, "/*": "已经创建了多少个文件", "TotalWriteFileCount": 0, "/*": "写满需要关闭的文件数量", "NeedReleaseFileCount": 0 }, "1": { "/*": "是否开启", "WriteFlag": true, "/*": "当前写文件夹名", "CurWriteDir": "20241018-211111-475025868", "/*": "当前写文件名", "CurOperFile": "00001-000000000000.dat", "/*": "落地调用的总次数", "TotalCount": 0, "/*": "落地成功的次数", "SuccessCount": 0, "/*": "当前打开的文件写了多少, 单位MB", "CurrFileWriteMB": 0.0, "/*": "已经创建了多少个文件", "TotalWriteFileCount": 0, "/*": "写满需要关闭的文件数量", "NeedReleaseFileCount": 0 } } }</pre>					

6.8.24. GetRspLogWritersInfo 获取应答落地功能的相关信息

功能名称	GetRspLogWritersInfo	版本号	V1.0.5.2	更新日期	20241018
功能备注	获取应答落地功能的相关信息				
功能说明	获取应答落地功能的相关信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetRspLogWritersInfo				
入参说明					
无					
出参说明					

```
{
  "/*": "使能写应答开关",
  "WriteFlag": true,
  "/*": "写应答路径",
  "WritePath": ".\rsplog/",
  "/*": "单文件最大值",
  "MaxFileSizeMB": 1,
  "/*": "当前记录的消息号",
  "CurMsgNo": 0,
  "/*": "工作线程应答落地相关信息",
  "WorkThreadWritersInfo": {
    "/*": "工作线程编号",
    "0": {
      "/*": "是否开启",
      "WriteFlag": true,
      "/*": "当前写文件夹名",
      "CurWriteDir": "20241018-211111-475025868",
      "/*": "当前写文件名",
      "CurOperFile": "00000-000000000000.dat",
      "/*": "落地调用的总次数",
      "TotalCount": 0,
      "/*": "落地成功的次数",
      "SuccessCount": 0,
      "/*": "当前打开的文件写了多少, 单位MB",
      "CurrFileWriteMB": 0.0,
      "/*": "已经创建了多少个文件",
      "TotalWriteFileCount": 0,
      "/*": "写满需要关闭的文件数量",
      "NeedReleaseFileCount": 0
    },
    "1": {
      "/*": "是否开启",
      "WriteFlag": true,
      "/*": "当前写文件夹名",
      "CurWriteDir": "20241018-211111-475025868",
      "/*": "当前写文件名",
      "CurOperFile": "00001-000000000000.dat",
      "/*": "落地调用的总次数",
      "TotalCount": 0,
      "/*": "落地成功的次数",
      "SuccessCount": 0,
      "/*": "当前打开的文件写了多少, 单位MB",
      "CurrFileWriteMB": 0.0,
      "/*": "已经创建了多少个文件",
      "TotalWriteFileCount": 0,
      "/*": "写满需要关闭的文件数量",
      "NeedReleaseFileCount": 0
    }
  }
}
```

6.8.25. GetBizFuncBlackList 获取配置的需要忽略的功能号集合（黑名单）

功能名称	GetBizFuncBlackList	版本号	V1.0.5.3	更新日期	20220607
功能备注	获取配置的需要忽略的功能号集合（黑名单）				
功能说明	获取配置的需要忽略的功能号集合（黑名单）				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetBizFuncBlackList				
入参说明					
无					
出参说明					
<pre>{ "/*": "需要忽略的功能号集合（黑名单）", "FuncBlackList": "123;456;34;1??:6553?:677?:676756" }</pre>					

6.8.26. GetOrSetFunctionRunningStatus获取或设置功能号运行状态

功能名称	GetOrSetFunctionRunningStatus	版本号	V1.0.5.4	更新日期	20230110
功能备注	获取或设置功能号运行状态				
功能说明	获取或设置功能号运行状态。如果当前功能号A暂停使用，发送A功能号的请求会有错误应答。				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetOrSetFunctionRunningStatus				
入参说明					
<pre>{ "/*": "需要控制的功能号：必填", "FunctionID": 105, "/*": "功能号状态，支持 Pause 或 Resume;如果当前字段不填写，则默认为Pause", "Status": "Pause" }</pre>					
出参说明					

<pre>{ "ErrorNo": 102, "ErrorMsg": "xxxxxxxxxx", "GetPauseFuncInfo": [{ "FunctionID": 105 }] }</pre>
--

6.8.27. GetBizStatusInfo获取业务状态

功能名称	GetBizStatusInfo	版本号	V1.0.5.4	更新日期	20230714
功能备注	获取业务状态				
功能说明	获取业务状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&FuncName=GetBizStatusInfo				
入参说明					
出参说明					
<pre>{ "BizStatus": 1, "BizStatusBit": "0b1", "BizStatusCN": "初始状态" }</pre>					

6.8.28. GetShardNo获取分片号

功能名称	GetShardNo	版本号	V1.0.12.5	更新日期	20230919
功能备注	获取分片号				
功能说明	获取分片号				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&FuncName=GetShardNo				
入参说明					
出参说明					
<pre>{ "ShardNo": 1 }</pre>					

6.8.29. GetAsyncRedoQueueInfo 获取异步redo线程信息

功能名称	GetAsyncRedoQueueInfo	版本号	V1.0.8.513	更新日期	20231213
功能备注	获取异步redo线程信息				
功能说明	获取异步redo线程信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&FuncName=GetAsyncRedoQueueInfo				
入参说明					
无					
出参说明					
<pre>[{ "ThreadId": 0, "ThreadBindCpuId": 0, "ProcessedCount": 1, "QueueCount": 0, "PollingLoops": 100000, "SleepTimeUS": 1, "MaxQueueSize": -1, "Running": false, "MaxHisQueueCount": 1 }]</pre>					

6.8.30. GetAsyncRedoStatus 获取异步写redo信息

功能名称	GetAsyncRedoStatus	版本号	V1.0.8.513	更新日期	20231213
功能备注	获取异步写redo信息				
功能说明	获取异步写redo信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetAsyncRedoStatus				
入参说明					
无					
出参说明					
<pre>[{ "ThreadId": 1, "EventGenerator_LastNodeCount": 2048, "EventGenerator_TotalNodeCount": 2048, "EventGenerator_TotalBytes": 65536, "EventGenerator_BlockCount": 16, "EventGenerator_NodeSize": 32, "EventGenerator_BlockSize": 4096, "AllocGenerator_LastNodeCount": 2048, "AllocGenerator_TotalNodeCount": 2048, "AllocGenerator_TotalBytes": 21102592, "AllocGenerator_BlockCount": 16, "AllocGenerator_NodeSize": 40, "AllocGenerator_BlockSize": 5120 }]</pre>					

6.8.31. GetNodesWithFuncWhitelist 获取符合功能号白名单的节点信息

功能名称	GetNodesWithFuncWhitelist	版本号	V1.0.8.513	更新日期	20231213
功能备注	获取符合功能号白名单的节点信息				
功能说明	获取符合功能号白名单的节点信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetNodesWithFuncWhitelist				
入参说明					
无					
出参说明					
<pre>{ "RegNodeInfo": [{ "ContextId": 0, "ContextName": "client1", "AppName": "test", "TopicName": "req" }] }</pre>					

6.8.32. ClearWorkQueue 清空工作线程队列

功能名称	ClearWorkQueue	版本号	V1.0.15.5	更新日期	20240521
功能备注	清空工作线程队列				
功能说明	清空工作线程队列				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=ClearWorkQueue&Confirm=true				
入参说明					
<pre>{ "Confirm": "true" }</pre>					
出参说明					
<pre>{ "status": "ok" }</pre>					

--

6.8.33. GetTsRuleWhiteFuncList 获取当前支持条件时延功能号列表

功能名称	GetTsRuleWhiteFuncList	版本号	V1.0.15.13	更新日期	20240627
功能备注	获取当前支持条件时延功能号列表				
功能说明	获取当前支持条件时延功能号列表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetTsRuleWhiteFuncList				
入参说明					
无					
出参说明					
<pre>{ "///": "获取当前支持条件时延分析功能号列表", "TsRuleWhiteFuncList": "123;1;456346" }</pre>					

6.8.34. GetTsRuleBlackFuncList 获取需要忽略的条件功能号集合（黑名单）

功能名称	GetTsRuleBlackFuncList	版本号	V1.0.15.13	更新日期	20240627
功能备注	获取需要忽略的条件功能号集合（黑名单）				
功能说明	获取需要忽略的条件功能号集合（黑名单）				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetTsRuleBlackFuncList				
入参说明					
无					
出参说明					
<pre>{ "///": "获取需要忽略的条件功能号集合（黑名单）", "TsRuleBlackFuncList": "123;456;34;1?;6553?;677?;676756" }</pre>					

6.8.35. GetFuncDealInfoDetails 获取工作线程上的账号执行的统计明细

功能名称	GetFuncDealInfoDetails	版本号	V1.0.15.21	更新日期	20240715
功能备注	获取工作线程上的账号执行的统计明细				
功能说明	支持根据工作线程ID和账号信息进行过滤，若两个过滤条件都指定，则取交集。 如若账号与工作线程不绑定，会出现以下两种场景： 1. 查询某个账户信息，返回的结果中，包含有所有执行过该账户的工作线程的执行明细，按线程ID分开显示。 2. 查询某个线程ID，返回的结果中，相应账号的执行明细可能是部分，该账号的其余明细可能在其他工作线程上。 过滤场景说明： 有ThreadId，没有AccountInfo：则返回指定ThreadId上的账户明细。 没有ThreadId，有AccountInfo：则返回指定AccountInfo的账户明细。 既有ThreadId，又有AccountInfo：则两者取交集，即返回指定ThreadId上的指定账户的明细。 既没有ThreadId，又没有AccountInfo：则返回空。				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetFuncDealInfoDetails				
入参说明					
<pre>[source,json]</pre>					
<pre>{ "///": "若要查询多个工作线程ID数据，通过','分隔，若传""则返回空，如果某个ID非法，则被忽略", "ThreadId": "1,2,3", "///": "若要查询多个账户信息数据，通过','分隔，若传""则返回空，如果某个账户信息非法，则被忽略", "AccountInfo": "abc1,bcd2"}</pre>					
出参说明					

```
{
  " GetFuncDealInfoDetails": [
    {
      "///": "工作线程ID",
      "ThreadId": 1,
      "///": "账号信息",
      "AccountInfo": "abc1",
      "///": "功能号, 功能号为0时, 表示为业务框架错误应答的处理统计信息",
      "FunctionID": 5,
      "///": "子系统号",
      "SystemNo": 100,
      "///": "定位模式",
      "LocateMode": 0,
      "///": "累计执行总次数",
      "TotalExecNum": 0,
      "///": "累计执行失败次数",
      "FailExecNum": 0,

      "///": "累计执行时间, 单位: 纳秒, 不含排队",
      "SumExecTimeNs": 0,
      "///": "执行单次最大耗时",
      "MaxExecTimeNs": 0,
      "///": "执行单次最小耗时",
      "MinExecTimeNs": 0,
      "///": "最后一次执行时间, 单位: 纳秒",
      "LastExecTimeNs": 0,
      "///": "执行单次平均耗时",
      "AveExecTimeNs": 0,

      "///": "在队列积压的总时间, 单位: 纳秒",
      "SumQueueTimeNs": 0,
      "///": "在队列积压的最大时间, 单位: 纳秒",
      "MaxQueueTimeNs": 0,
      "///": "在队列积压的最小时间, 单位: 纳秒",
      "MinQueueTimeNs": 0,
      "///": "最后一次队列积压时间, 单位: 纳秒",
      "LastQueueTimeNs": 0,
      "///": "在队列积压的平均时间, 单位: 纳秒",
      "AveQueueTimeNs": 0,

      "///": "定位的总时间, 单位: 纳秒",
      "SumLocateTimeNs": 0,
      "///": "最大定位时间, 单位: 纳秒",
      "MaxLocateTimeNs": 0,
      "///": "最小定位时间, 单位: 纳秒",
      "MinLocateTimeNs": 0,
      "///": "最后一次定位时间, 单位: 纳秒",
      "LastLocateTimeNs": 0,
      "///": "平均定位时间, 单位: 纳秒",
      "AveLocateTimeNs": 0,

      "///": "累计执行时间, 单位: 纳秒, 含排队",
      "SumTotalTimeNs": 0,
      "///": "执行单次最大耗时",
      "MaxTotalTimeNs": 0,
      "///": "执行单次最小耗时",
      "MinTotalTimeNs": 0,
      "///": "最后一次执行时间, 单位: 纳秒",
      "LastTotalTimeNs": 0,
      "///": "执行单次平均耗时",
      "AveTotalTimeNs": 0,

      "///": "Redo累计耗时, 单位: 纳秒",
      "SumRedoTimeNs": 0,
      "///": "redo最大耗时, 单位: 纳秒",
      "MaxRedoTimeNs": 0,
      "///": "redo最小耗时, 单位: 纳秒",
      "MinRedoTimeNs": 0,
      "///": "redo最后一次调用耗时, 单位: 纳秒",
      "LastRedoTimeNs": 0,
      "///": "redo平均耗时",
      "AveRedoTimeNs": 0
    }
  ]
}
```

6

功能名称	EnableFuncDealInfoDetails	版本号	V1.0.15.21	更新日期	20240715
功能备注	开启/关闭统计账户执行明细功能				
功能说明	开启/关闭统计账户执行明细功能，默认关闭				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=EnableFuncDealInfoDetails				
入参说明					
<pre>{ "///": "是否开启统计账户执行明细，合法值：true/false，缺省为：false，传入其他值或没有入参则表示查询", "Enable": "true" }</pre>					
出参说明					
<pre>{ "///": "是否开启账户统计执行明细"， "false"表示关闭， "true"表示开启， "IsEnable": "false" }</pre>					

6.8.37. ExceptionReqInfo 查看异常请求消息消息

功能名称	ExceptionReqInfo	版本号	V1.0.15.21	更新日期	20240715
功能备注	查看异常请求消息消息				
功能说明	查看异常请求消息消息				

访问链接	http://port/ldp?Pretty=1&PluginName=ldp_bizproc&FuncName=ExceptionReqInfo
入参说明	
无	
出参说明	
<pre>{ "/*": "持久化文件中数据", "Storage": { "ExceptionMsgs": [{ "Type": "RcmMsgSqn", "Count": 3, "TotalLength": 29, "Details": [{ "TxContextId": 1, "Length": 2 }, { "TransportId": 5, "Length": 2 }, { "TxMsgNo": 1, "Length": 8 }] }], "WritePos": 57, "CountInMem": 1, "FileName": "xst#1_ldp_bizproc_0_Exception", "FileSize": 57, "Fd": 9, "Version": 1, "ReadPosWhenOpen": 24, "CountInFileWhenOpen": 0 }, "/*": "共享内存中, 每个线程的数据", "IdentifyInMems": [{ "ThreadNo": 0, "ExceptionMsgInShm": "xst#1_ldp_bizproc_0_Trace_0", "Type": "RcmMsgSqn", "Count": 3, "TotalLength": 29, "Details": [{ "TxContextId": 1, "Length": 2 }, { "TransportId": 5, "Length": 2 }, { "TxMsgNo": 1, "Length": 8 }] }, { "ThreadNo": 1, "ExceptionMsgInShm": "xst#1_ldp_bizproc_0_Trace_1", "Type": "InvalidType", "Count": 0, "TotalLength": 0, "Details": [] }, { "ThreadNo": 2, "ExceptionMsgInShm": "xst#1_ldp_bizproc_0_Trace_2", "Type": "InvalidType", "Count": 0, "TotalLength": 0, "Details": [] }] }</pre>	

6.8.38. GetTableAnalyseFunctionID 获取内存表分析调用功能号

功能名称	GetTableAnalyseFunctionID	版本号	V1.0.15.40	更新日期	20241113
功能备注	获取内存表分析调用功能号				
功能说明	获取内存表分析调用功能号				
访问链接	http://pport/ldp?Pretty=1&PluginName=ldp_bizproc&FuncName=GetTableAnalyseFunctionID				
入参说明					
无					
出参说明					
<pre>{ "FunctionID": 407 }</pre>					

6.8.39. GetConfig 获取插件配置

功能名称	GetConfig	版本号	V1.0.15.57	更新日期	20250317
功能备注	获取插件配置				
功能说明	获取插件配置				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetConfig				
入参说明					
无					
出参说明					
<pre>{ "/*": "允许备机处于ToInactive时接入的功能号列表", "AppStatusCtrlWhiteFuncList": "5;102" "/*": "是否强制写入binlog", "ForceWriteBinLog": false, "/*": "当前是否处于重演模式，true表示重演模式，false表示正常模式", "IsReplyMode": false, "/*": "二次定位功能号", "LocateFunction": 100, "/*": "配了Replay才有", "Replay": { "/*": "是否可重演", "EnableReproducible": true, "/*": "是否可重演", "Persistent": { "/*": "重演文件落地路径", "WritePath": "xxx", "/*": "文件最大大小", "MaxFileSizeMB": 100 } }, "/*": "不配Replay才有", "ReqLogWrite": { "/*": "是否写文件", "WriteFlag": true, "/*": "文件最大大小", "MaxFileSizeMB": 100, "WritePath" "xxx" }, "/*": "不配Replay才有", "RspLogWrite": { "/*": "是否写文件", "WriteFlag": true, "/*": "文件最大大小", "MaxFileSizeMB": 100, "WritePath" "xxx" } }</pre>					

6.8.40. GetReplayInfo 获取重演功能相关信息

功能名称	GetReplayInfo	版本号	V1.0.15.63	更新日期	20250414
功能备注	获取重演功能相关信息				
功能说明	获取重演功能相关信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetReplayInfo				
入参说明					
无					
出参说明					
<pre>{ "/*": "当前是否处于重演模式，true表示重演模式，false表示正常模式", "IsReplayMode": true, "/*": "是否可重演，只有存在Replay配置时才有意义", "IsReproducible": false, "/*": "重演id，只有IsReplayMode为true时才有意义", "ReplayId": 1234567, "/*": "重演文件持久化根目录", "PersistentRootPath": "replaylog", "/*": "重演文件持久化当前目录", "PersistentFullPath": "replaylog/xst/20250403-113653463"}</pre>					

6.8.41. CloseReplayMode 关闭重演模式（切换到正常模式）

功能名称	CloseReplayMode	版本号	V1.0.15.63	更新日期	20260119
功能备注	关闭重演模式（切换到正常模式）				
功能说明	关闭重演模式（切换到正常模式）				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=CloseReplayMode				
入参说明					
<pre>{ "/*": "当前重演id", "ReplayId": 1234567 }</pre>					

出参说明
<pre>{ "":"/": "当前是否处于重演模式，执行成功返回false， "IsReplayMode": false }</pre>

6.8.42. ModifyRpcTimeout 修改RPC超时时间

功能名称	ModifyRpcTimeout	版本号	V1.0.15.79	更新日期	20250612
功能备注	修改RPC调用的超时时间				
功能说明	此功能用于修改RPC调用的超时时间，超时时间以毫秒为单位				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&FuncName=ModifyRpcTimeout				
入参说明					
<pre>{ "":"/": "新的超时时间（毫秒）", "TimeoutMilli": 5000 }</pre>					
出参说明					
<pre>{ "":"/": "操作结果说明", "Result": "超时时间更新成功", "TimeoutMilli": 5000 }</pre>					

6.8.43. GetZoneSwitchHostStatus 查询主核心切换就绪状态

功能名称	GetZoneSwitchHostStatus	版本号	V1.0.15.80	更新日期	20250615
功能备注	查询主核心切换就绪状态				
功能说明	查询主核心切换就绪状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetZoneSwitchHostStatus				
入参说明					
无					
出参说明					
<pre>{ "":"/": "查询主核心切换就绪状态，-1=初始状态，0=就绪状态，1=未就绪状态", "Status": 0 }</pre>					

6.8.44. GetFilterQueuesInfo 获取去重功能去重队列相关信息

功能名称	GetFilterQueuesInfo	版本号	V1.0.15.773	更新日期	20250615
功能备注	获取去重功能去重队列相关信息				
功能说明	获取去重功能去重队列相关信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&FuncName=GetFilterQueuesInfo				
入参说明					
无					
出参说明					

```
{
  "/*": "当前的去重队列列表信息，",
  "FilterQueueInfo": [
    {
      "/*": "去重队列前置名信息，",
      "FrontName": "front#0",
      "/*": "去重队列前置轮次号信息，",
      "RoundNo": 1001,
      "/*": "去重队列前置时间戳信息，",
      "Stamp": 56789,
      "/*": "去重队列使用的内存字节数，",
      "MemSize": 524320,
      "/*": "去重队列老化信息号，前置重发最小序号，",
      "CleanNo": 0,
      "/*": "去重队列存储的最大序号，",
      "MaxNo": 2002,
      "/*": "去重队列失效后丢弃的重传包数，",
      "DroppedCount": 0,
      "/*": "去重队列去重的请求数，",
      "FilterCount": 0,
      "/*": "去重队列是否工作正常，",
      "Running": true,
      "/*": "去重队列不连续，没有收到的序号，最大展示100个，",
      "RedoQueue": [
        "/*": "去重队列里的空洞序号，",
        6,
        7
      ]
    }
  ]
}
```

6.8.45. GetAllNodePartInfo 查询核心分区信息

功能名称	GetAllNodePartInfo	版本号	V1.0.15.83	更新日期	20250925
功能备注	查询核心分区信息				
功能说明	查询多分区状态下核心的分区信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetAllNodePartInfo				
入参说明					
无					
出参说明					
<pre>{ "/*": "分区信息", "NodeInfo": [{ "/*": "节点号", "NodeNo": 1, "/*": "写分区号，表示可以进行写操作的分区（主）", "ActivePartNo": [0, 4, 7, 10], "/*": "读分区号，表示只能进行读操作的分区（备）", "StandbyPartNo": [1, 2, 3, 5, 6, 8, 9, 11, 12], "/*": "主备状态", "Role": "Active", "/*": "工作状态", "Status": "ArbActive" }] }</pre>					

6.8.46. GetBizStatusPartInfo 查询核心分区状态

功能名称	GetBizStatusPartInfo	版本号	V1.0.15.83	更新日期	20250925
功能备注	查询核心分区状态				
功能说明	查询多分区状态下核心的分区状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&FuncName=GetBizStatusPartInfo				
入参说明					
无					
出参说明					

<pre>{ "/*": "业务状态", "BizStatus": 1, "/*": "分区状态", "PartitionStatus": [{ "/*": "分区号", "PatitionNo": 0, "/*": "业务状态", "BizStatus": 1, "/*": "平台状态", "AppStatus": 1 }, { "PatitionNo": 1, "BizStatus": 1, "AppStatus": 1 }, { "PatitionNo": 2, "BizStatus": 1, "AppStatus": 1 }] }</pre>
--

6.8.47. FuncNaPartitionDealInfo 查询核心分区处理信息

功能名称	FuncNaPartitionDealInfo	版本号	V1.0.15.83	更新日期	20250925
功能备注	查询核心分区处理信息				
功能说明	查询多分区状态下核心分区处理的信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&FuncName=FuncNaPartitionDealInfo				
入参说明					
无					
出参说明					
<pre>{ "/*": "分区组处理信息", "GetPartitionDealInfo": [{ "/*": "分区号", "PatitionNo": 0, "/*": "执行次数", "TotalExecNum": 4, "/*": "执行失败次数", "FailExecNum": 1, "/*": "业务累计执行时间（单位：纳秒）", "SumExecTimeNs": 689453, "/*": "业务执行单次最大耗时（单位：纳秒）", "MaxExecTimeNs": 558842, "/*": "业务执行单次最小耗时（单位：纳秒）", "MinExecTimeNs": 31099, "/*": "业务最后一次请求执行时间（单位：纳秒）", "LastExecTimeNs": 558842, "/*": "平均每次执行耗时（单位：纳秒）", "AveExecTimeNs": 172363 }, { "PatitionNo": 1, "TotalExecNum": 0, "FailExecNum": 0, "SumExecTimeNs": 0, "MaxExecTimeNs": 0, "MinExecTimeNs": 18446744073709551615, "LastExecTimeNs": 0, "AveExecTimeNs": 0 }]}</pre>					

6.9. 回库插件 ldp_todb

6.9.1. GetMonitor查看回库插件状态

功能名称	GetMonitor	版本号	V1.0.0.1	更新日期	20200828
功能备注					
功能说明	查看回库插件状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=GetMonitor				
入参说明					
无					
出参说明					
<pre>{ "/*": "主备状态", "HostStatus": 1, "/*": "核心的产品名", "XstProductName": "ldp", "/*": "核心的集群名", "XstShortAppName": "xst", "/*": "核心的应用名", "XstAppName": "xst#0", "/*": "核心的应用编号", }</pre>					

```
"XstAppNo": 0,
"/":"/ 配置的回库地址",
"Address": "10.40.3.21:9001,",
"/":"/ redo文件路径",
"RedoPath": "/home/ldp/workspace/./ldpredo/current/",
"/":"/ 回库的redo类型",
"RedoTypes": 2,
"/":"/ 业务框架在zk上写的回库节点路径",
"ZkPath": "",
"/":"/ 共享内存文件名",
"ShmName": "ldp_xst#0_ldp_bizproc_0_todb",
"/":"/ 回库在zk上写的节点路径",
"ToddbZkPath": "",
"/":"/ 注册名",
"RegName": "usttodb",
"/":"/ 最大缓存数量（多线程不设置上限，超过最大数量时缓存速度变慢）",
"MaxCacheCount": "10000",
"/":"/ 重传发送间隔",
"ResendIntervalSec": 8,
"/":"/ 高可用模式 1:本地模式 2:仲裁模式 3:独立部署模式",
"HaMode": 1,
"/":"/ "zk状态是否有变化",
"ZKChanged": false,
"/":"/ "zk通知事件数",
"ZkEventsCount": 0,
"/":"/ "发送线程次数",
"WorkthreadLoops": 245,
"/":"/ "redo解析是否初始化",
"IsReaderInited": true,
"/":"/ "已经从redo文件中读取到的事务数",
"ReadTransCount": 1,
"/":"/ "已经从redo文件中读取到的最大事务号",
"MaxReadTransNo": 0,
"/":"/ "当前所有redo文件中最大的事务编号的下一个事务编号",
"GlobalMaxTransNo": 0,
"/":"/ "true: 连接的是老网关, false: 新网关",
"IsUstGW": true,
"/":"/ "网关发送主备模式, true为主备模式",
"GWMasterSlaveOn": false,
"/":"/ "定时线程次数",
"OnTimerThreadLoops": 122,
"/":"/ "非法应答数量",
"InvalidRspCount": 0,
"/":"/ "黑名单过滤数量",
"BlackListFilterCount": 0,
"/":"/ "回库黑名单功能号",
"BlackListFuncNo": 0,
"/":"/ "当前未确认的事务数",
"CacheCount": 1,
"/":"/ "最大重传次数",
"MaxResendCount": 3,
"/":"/ "单线程错误应答数量",
"ErrResponseCount": 0,
"/":"/ "落库失败记录数量",
"FailedToddbRecordCount": 0,
"/":"/ "单线程重发次数",
"ResendCount": 0,
"/":"/ "读取redo平均耗时, 单位毫秒",
"AveragReadMilli": 0,
"/":"/ "读取redo最大耗时, 单位毫秒",
"MaxReadMilli": 0,
"/":"/ "读取redo最大耗时时间点",
"MaxReadTime": 0,
"/":"/ "最近1s的读redo吞吐量",
"ReadRedoTps": 0,
"/":"/ "最大读redo吞吐率, 单位 笔/s",
"MaxReadRedoTps": 0,
"/":"/ "最小读redo吞吐率, 单位 笔/s",
"MinReadRedoTps": 0,
"/":"/ "5min内读redo吞吐率",
"Per5minReadRedoTps": 0,
"/":"/ "多线程模式",
"MultiThreadsMode": false,
"/":"/ "当前zk上同步的未处理最小事务号",
"ZkMinTransNo": 0,
"/":"/ "读取redo起始事务号",
"OrigTransNo": 0,
"/":"/ "线程数量",
"ThreadsCount": 1,
"/":"/ "是否开启业务错误自动重传",
"IsBizErrAutoResend": false,
"/":"/ "线程拥堵时的等待次数",
"JammedWaitLoop": 0,
"/":"/ "错误应答重发时间间隔",
"ErrRspResendInterval": 8,
"/":"/ "最近1s的发送吞吐量",
"SendTps": 0,
"/":"/ "最大发送吞吐率, 单位 笔/s",
"MaxSendTps": 0,
"/":"/ "最小发送吞吐率, 单位 笔/s",
"MinSendTps": 0,
"/":"/ "5min内发送吞吐率",
"Per5minSendTps": 0,
"/":"/ "最近1s的应答吞吐量",
"RspTps": 0,
"/":"/ "最大应答吞吐率, 单位 笔/s",
"MaxRspTps": 0,
"/":"/ "最小应答吞吐率, 单位 笔/s",
"MinRspTps": 0,
"/":"/ "5min内应答吞吐率",
"Per5minRspTps": 0,
"/":"/ "发送线程情况",
"ThreadsSendStatus": [
{
"/":"/ "是否是业务错误重发线程",
"IsReSendThread": ,
"/":"/ "线程号",
"ThreadNo": ,
"/":"/ "已处理数量",
"ManagedCount": ,
"/":"/ "业务失败数量",
"BizFailedCount": ,
"/":"/ "平台失败数量",
"FailedCount": ,
"/":"/ "超时数量",
"TimeoutCount": ,
```



```
        "":"/": "迁移数量（重放、黑名单）",
        "ReplayCount": ,
        "":"/": "发送数量",
        "SendCount": ,
        "":"/": "当前发送的这笔redo业务错误重发次数",
        "CurBizErrResendCount": 0,
        "":"/": "当前发送的这笔redo平台错误重发次数",
        "CurPlatErrResendCount": 0,
        "":"/": "当前发送的这笔redo超时重发次数",
        "CurTimeoutResendCount": 0,
        "":"/": "正在等待确认的事务号",
        "ExpectingRspTransNo": ,
        "":"/": "已经处理的事务号",
        "HandledTransNo": ,
        "":"/": "线程缓存数量",
        "CacheCount": ,
        "":"/": "发送消息平均耗时, 单位毫秒",
        "AveragSendMilli": 0,
        "":"/": "发送消息最大耗时, 单位毫秒",
        "MaxSendMilli": 0,
        "":"/": "发送消息最大耗时时间点",
        "MaxSendTime": 0,
        "":"/": "收应答平均耗时, 单位毫秒",
        "AveragRspMilli": 0,
        "":"/": "收应答最大耗时, 单位毫秒",
        "MaxRspMilli": 0,
        "":"/": "收应答最大耗时时间点",
        "MaxRspTime": 0,
        "":"/": "以下字段仅当IsReSendThread=false时存在",
        "":"/": "正在重试的todbkey的集合",
        "RetryingTodbKeys": [
            123, 456, 789
        ]
    },
    ],
}
```

6.9.2. ForceUpdateZKData主动触发回库读取zk数据

功能名称	ForceUpdateZKData	版本号	V1.0.0.1	更新日期	20200828
功能备注					
功能说明	主动触发回库读取zk数据				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=ForceUpdateZKData				
入参说明					
无					
出参说明					
无					

6.9.3. GetConnectionStatus查看网关连接状态

功能名称	GetConnectionStatus	版本号	V1.0.0.1	更新日期	202111130
功能备注					
功能说明	查看网关连接状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_toddb&FuncName=GetConnectionStatus				
入参说明					
无					
出参说明					
<pre>{ "":"/": "网关连接信息", "GWConnections": [{ "":"/": "网关地址", "IP": "10.40.3.21", "":"/": "站点对应的域名", "DomainName": "www.test.com", "":"/": "网关端口号", "Port": 9001, "":"/": "连接状态", "Status": "CONNECTED", "":"/": "发送次数", "SendCount": 2 }] }</pre>					

6.9.4. PerSecondReadCount 5min内每秒读redo吞吐量

功能名称	PerSecondReadCount	版本号	V1.0.0.1	更新日期	20211215
功能备注					
功能说明	5min内每秒读redo吞吐量				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_toddb&FuncName=PerSecondReadCount				
入参说明					
无					

出参说明					
<pre>{ "/*": "5min内每秒读redo吞吐量", "5minPerSecondReadCount": [] }</pre>					

6.9.5. PerSecondSendCount 5min内每秒发送吞吐量

功能名称	PerSecondSendCount	版本号	V1.0.0.1	更新日期	20211215
功能备注					
功能说明	5min内每秒发送吞吐量				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=PerSecondSendCount				
入参说明					
无					
出参说明					
<pre>{ "/*": "5min内每秒发送吞吐量", "5minPerSecondSendCount": [] }</pre>					

6.9.6. PerSecondRspCount 5min内每秒应答吞吐量

功能名称	PerSecondRspCount	版本号	V1.0.0.1	更新日期	20211215
功能备注					
功能说明	5min内每秒应答吞吐量				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=PerSecondRspCount				
入参说明					
无					
出参说明					
<pre>{ "/*": "5min内每秒应答吞吐量", "5minPerSecondRspCount": []}</pre>					

6.9.7. GetFuncNoDealInfo 功能号执行统计

功能名称	GetFuncNoDealInfo	版本号	V1.0.0.0	更新日期	20221101
功能备注					
功能说明					
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetFuncNoDealInfo				
入参说明					
无					
出参说明					
<pre>{ "FuncNoDealInfo": [{ "/*": "功能号", "FunctionNo": 0, "/*": "发送数量", "SendCount": 0, "/*": "业务错误应答数量", "BizErrCount": 0, "/*": "平台错误应答数量", "PlatErrCount": 0, "/*": "正确应答数量", "SuccessCount": 0, "/*": "白名单应答匹配数量", "whiteListMatchCount": 0 }]}</pre>					

6.9.8. GetErrNoWhiteListInfo 错误号白名单匹配信息

功能名称	GetErrNoWhiteListInfo	版本号	V1.0.0.0	更新日期	20221101
功能备注					
功能说明					
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetErrNoWhiteListInfo				

入参说明
无
出参说明
<pre>{ "ErrNoWhiteListInfo": [{ "///": "匹配到的错误号", "ErrorNo": -1, "///": "匹配数量", "MatchCount": 1 }] }</pre>

6.9.9. ResendBizErr 重传落地文件中的业务错误

功能名称	ResendBizErr	版本号	V1.0.0.0	更新日期	20221101
功能备注					
功能说明					
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=ResendBizErr				
入参说明					
无					
出参说明					
<pre>{ "///": "触发成功", "State": "OK" }</pre>					

6.9.10. LastManualResendInfo 上次重传落地文件错误消息的详细信息

功能名称	LastManualResendInfo	版本号	V1.0.0.0	更新日期	20221109
功能备注					
功能说明					
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=LastManualResendInfo				
入参说明					
无					
出参说明					
<pre>{ "///": "读了几笔事务", "ReadTransCount": 4, "///": "从文件读到的事务号列表", "TransNoFromFile": [1] }</pre>					

6.9.11. LastErrRspMsg 上次错误应答详细信息

功能名称	LastErrMsg	版本号	V1.0.0.0	更新日期	20240313
功能备注					
功能说明					
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=LastErrMsg				
入参说明					
无					
出参说明					
<pre>{ "///": "错误号", "errNo": 1, "///": "错误信息", "errMsg": ""}</pre>					

6.10. 下场插件 ldp_todb_v2

6.10.1. GetRedoDealInfo 查看下场插件线程处理redo的信息

功能名称	GetRedoDealInfo	版本号	V1.0.0.1	更新日期	20230629
功能备注					

功能说明	查看回库线程处理redo的情况
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetRedoDealInfo
入参说明	
无	
出参说明	

```

{
  "ThreadDealInfo": {
    "/*": "哪一个线程",
    "Thread_0": {
      "/*": "是否为重试线程, true为重试线程",
      "IsResend": false,
      "/*": "线程所属线程组",
      "QueueAlias": 1,
      "/*": "业务错误入重试文件队列失败次数, 非0可能是磁盘满",
      "BizErrPutFailedCount": 0,
      "/*": "大事务处理或者多账户事务全局因子线程组别名",
      "GlobalMultiThreadQueueAlias": 1,
      "/*": "超时检测处理耗时信息",
      "TimeoutCheck": {
        "/*": "检测次数",
        "CheckCount": 4222,
        "/*": "检测一次的最大耗时, 单位ns",
        "CheckMaxNs": 30239,
        "/*": "检测一次的最小耗时, 单位ns",
        "CheckMinNs": 1099,
        "/*": "检测一次的平均耗时, 单位ns",
        "CheckAvgNs": 2745
      },
      "/*": "集群名, 能连上核心时通过上下文获取核心apname, 否则使用topicname_partitionno",
      "mdb_redo_1": {
        "/*": "集群类型, 1表示网格集群, 0是非网格集群",
        "ClusterType": 1,
        "/*": "分区组编号",
        "GroupNo": 1,
        "/*": "分区组名称",
        "PartGroupName": "multiproc_mdb#0_partgroup_1",
        "/*": "线程的请求处理信息",
        "Req": {
          "/*": "处理的数量",
          "DealCount": 0,
          "/*": "处理过的最小事务号",
          "DealMinTransNo": 0,
          "/*": "处理过的最大事务号",
          "DealMaxTransNo": 0,
          "/*": "Filter1的最小值, 单位ns",
          "Filter1MinNs": 0,
          "/*": "Filter1的最大值, 单位ns",
          "Filter1MaxNs": 0,
          "/*": "Filter1的平均值, 单位ns",
          "Filter1AvgNs": 0,
          "/*": "Locate的最小值, 单位ns",
          "LocateMinNs": 0,
          "/*": "Locate的最大值, 单位ns",
          "LocateMaxNs": 0,
          "/*": "Locate的平均值, 单位ns",
          "LocateAvgNs": 0,
          "/*": "在队列中滞留时间的最小值, 单位ns",
          "QueueMinNs": 0,
          "/*": "在队列中滞留时间的最大值, 单位ns",
          "QueueMaxNs": 0,
          "/*": "在队列中滞留时间的平均值, 单位ns",
          "QueueAvgNs": 0,
          "/*": "Filter2的最小值, 单位ns",
          "Filter2MinNs": 0,
          "/*": "Filter2的最大值, 单位ns",
          "Filter2MaxNs": 0,
          "/*": "Filter2的平均值, 单位ns",
          "Filter2AvgNs": 0,
          "/*": "Process的最小值, 单位ns",
          "ProcessMinNs": 0,
          "/*": "Process的最大值, 单位ns",
          "ProcessMaxNs": 0,
          "/*": "Process的平均值, 单位ns",
          "ProcessAvgNs": 0,
          "/*": "redo处理的平均值, 单位ns",
          "TotalAvgNs": 0,
          "/*": "近 1 秒请求平均耗时, 单位ns",
          "Last1sAvgNs": 0,
          "/*": "近 1 秒请求处理耗时, 单位ns",
          "Last1CostNs": 0
        },
        "/*": "线程的应答处理信息",
        "Rsp": {
          "/*": "处理的数批",
          "DealCount": 0,
          "/*": "在队列中滞留时间的最小值, 单位ns",
          "QueueMinNs": 0,
          "/*": "在队列中滞留时间的最大值, 单位ns",
          "QueueMaxNs": 0,
          "/*": "在队列中滞留时间的最平均, 单位ns",
          "QueueAvgNs": 0,
          "/*": "Process的最小值, 单位ns",
          "ProcessMinNs": 0,
          "/*": "Process的最大值, 单位ns",
          "ProcessMaxNs": 0,
          "/*": "Process的平均值, 单位ns",
          "ProcessAvgNs": 0,
          "/*": "请求发送到应答处理结束的最小值, 单位ns",
          "AsyncProcessMinNs": 0,
          "/*": "请求发送到应答处理结束的最大值, 单位ns",
          "AsyncProcessMaxNs": 0,
          "/*": "请求发送到应答处理结束的平均值, 单位ns",
          "AsyncProcessAvgNs": 0,
          "/*": "应答处理的平均值, 单位ns",
          "TotalAvgNs": 0
        }
      }
    }
  }
}

```

6.10.2. ClearRedoDealInfo重置回库插件线程处理redo的信息

功能名称	ClearRedoDealInfo	版本号	V1.0.0.1	更新日期	20230629
功能备注					

功能说明	重置回库插件线程处理redo的统计信息
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=ClearRedoDealInfo
入参说明	
无	
出参说明	
<pre>{ "/*": "重置结果", "ClearResult": true }</pre>	

6.10.3. GetWorkCacheInfo回库工作线程redo缓存

功能名称	GetWorkCacheInfo	版本号	V1.0.0.1	更新日期	20241213
功能备注					
功能说明	查看回库工作线程redo缓存数据				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetWorkCacheInfo				
入参说明					
<pre>{ "/*": "待处理redo请求应答，查看第几页的数据，从0开始", "WaitProcessPageNo": "0", "/*": "每页几条数据", "WaitProcessCount": "10", "/*": "异步处理中redo请求，查看第几页的数据，从0开始", "ProcessingPageNo": "0", "/*": "每页几条数据", "ProcessingCount": "10",}</pre>					
出参说明					
<pre>{ "WorkCacheInfo": [{ "/*": "线程号", "ThreadNo": 0, "/*": "是否为重试线程", "IsResend": false, "/*": "当前线程是否正在处理", "Running": true, "/*": "当前或上一笔消息类型：Request、Response。当前线程没有处理过消息时为Invalid", "Type": "Response", "/*": "当前或上一笔的功能号", "FunctionID": 409, "/*": "当前或上一笔的集群名", "ClusterName": "mdb", "/*": "已处理总数量", "ProcessedCount": 202, "/*": "当前或上一笔的开始执行时间，hh:mm:ss.SSSus", "BeginProcessTime": "15:11:32.534961", "/*": "当前或上一笔的已经执行了的时间，Running为true时有效，单位微秒", "DurationUs": 10000, "/*": "线程队列满的次数", "QueueFullCount": 1, "/*": "线程队列满最新时间", "LastQueueFullTime": "14:09:19.980138", "RedoMsg": [{ "/*": "集群号", "ClusterNo": 0, "/*": "集群名，能连上核心时通过上下文获取核心appname，否则使用topicname_partitionno", "ClusterName": "mdb_redo_1", "/*": "待处理redo的数量，包括请求和应答", "WaitProcessSize": 0, "/*": "待处理的redo事务号集合", "WaitProcess": [], "/*": "异步处理中请求的数量", "ProcessingSize": 0, "/*": "异步处理中的redo事务号集合", "Processing": [], "/*": "集群类型，1表示网格集群，0是非网格集群", "ClusterType": 1, "/*": "分区组编号", "GroupNo": 1, "/*": "分区组名称", "PartGroupName": "multiproc_mdb#0_partgroup_1" }] }]}</pre>					

6.10.4. GetWorkCacheMemInfo回库工作线程内存池状态

功能名称	GetWorkCacheMemInfo	版本号	V1.0.0.1	更新日期	20230629
功能备注					
功能说明	查看回库工作线程内存池状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetWorkCacheMemInfo				
入参说明					

无
出参说明
<pre>{ "WorkCacheMemInfo": [{ "ThreadNo": 0, "IsResend": false, "MemInfo": { "MemPoolUsedBlockCount": 1, "MemPoolFreedBlockCount": 0, "MemPoolBlocksInfo": [{ "BlockId": 0, "ToFreePos": 8, "CurPos": 8 }] } }], "TodbMemPoolInfo": { "MemPoolUsedBlockCount": 0, "MemPoolFreedBlockCount": 0, "MemPoolBlocksInfo": [] }, "ResendMemPoolInfo": { "MemPool": { "MemPoolUsedBlockCount": 0, "MemPoolFreedBlockCount": 0, "MemPoolBlocksInfo": [] }, "ResidentMemPool": { "MemPoolUsedBlockCount": 1, "MemPoolFreedBlockCount": 0, "MemPoolBlocksInfo": [{ "BlockId": 0, "ToFreePos": 16, "CurPos": 112 }] } }, "PrimaryMemPoolInfo": { "RedoMergeMemPool": { "MemPoolUsedBlockCount": 0, "MemPoolFreedBlockCount": 0, "MemPoolBlocksInfo": [] }, "TodbMemPool": { "MemPoolUsedBlockCount": 0, "MemPoolFreedBlockCount": 0, "MemPoolBlocksInfo": [] } } }</pre>

6.10.5. GetFileQueueInfo回库文件队列的状态

功能名称	GetFileQueueInfo	版本号	V1.0.0.1	更新日期	20230803
功能备注					
功能说明	查看回库文件队列的状态				
访问链接	http://ip:port/ldap?Pretty=1&PluginName=ldap_todb&InstanceNo=0&FuncName=GetFileQueueInfo				

入参说明
无
出参说明

```
{
  "时间戳": 2402981210408436,
  "入线程队列成功的次数": 0,
  "SentSucc": 0,
  "入线程队列失败的次数": 0,
  "SentFail": 0,
  "放入文件队列失败的次数": 0,
  "PutFail": 0,
  "当前在写的文件索引": 0,
  "WriteIndex": 0,
  "当前在读的文件索引": 0,
  "ReadIndex": 0,
  "放入文件队列成功的次数": 0,
  "PutSucc": 0,
  "从文件队列获取成功次数": 0,
  "GetSucc": 0,
  "配置的最大文件个数": 16,
  "MaxFileCount": 16,
  "配置的最大文件大小": 1024,
  "MaxFileSizeMB": 1024,
  "文件所在目录": ".",
  "FileDirName": ".",
  "文件前缀": "todb_filequeue",
  "FileInfo": [
    {
      "文件编号": 0,
      "FileIdx": 0,
      "文件写位置": 7720,
      "WritePos": 7720,
      "文件读位置": 7720,
      "ReadPos": 7720
    },
    {
      "FileIdx": 1,
      "WritePos": 0,
      "ReadPos": 0
    }
  ]
}
```

6.10.6. GetSyncFileQueueInfo回库重试文件队列的状态

功能名称	GetSyncFileQueueInfo	版本号	V1.0.0.0	更新日期	20240523
功能备注					
功能说明	回库重试文件队列的状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&InstanceNo=0&FuncName=GetSyncFileQueueInfo				
入参说明					
无					
出参说明					
<pre>{ "时间戳": 2402981210408436, "ClockGettime": 2402981210408436, "入线程队列成功的次数": 0, "SentSucc": 0, "入线程队列失败的次数": 0, "SentFail": 0, "放入文件队列失败的次数": 0, "PutFail": 0, "当前在写的文件索引": 0, "WriteIndex": 0, "当前在读的文件索引": 0, "ReadIndex": 0, "放入文件队列成功的次数": 0, "PutSucc": 0, "从文件队列获取成功次数": 0, "GetSucc": 0, "配置的最大文件个数": 16, "MaxFileCount": 16, "配置的最大文件大小": 1024, "MaxFileSizeMB": 1024, "文件所在目录": ".", "FileDirName": ".", "文件前缀": "todb_filequeue", "FileInfo": [{ "文件编号": 0, "FileIdx": 0, "文件写位置": 7720, "WritePos": 7720, "文件读位置": 7720, "ReadPos": 7720 }, { "FileIdx": 1, "WritePos": 0, "ReadPos": 0 }] }</pre>					

6.10.7. GetMonitor回库综合监控信息

功能名称	GetMonitor	版本号	V1.0.0.1	更新日期	20260119
功能备注					
功能说明	查看回库综合监控信息				

访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&InstanceNo=0&FuncName=GetMonitor
入参说明	
<pre>{ "/*": "起始编号，默认为0", "StartNo": "0", "/*": "查询数量，默认为10", "Count": "10", "/*": "核心集群名，多个时按Delimiter分割符隔开", "ClusterName": "xst", "/*": "分割符，默认按','隔开", "Delimiter": ",", }</pre>	
出参说明	
<pre>{ "/*": "主备状态，主机为1，备机为2", "Role": 1, "/*": "从remoter收到redo数量", "RecvRemoterReqTotalCount": 0, "/*": "从remoter收到错误重试redo数量", "ResendRemoterReqTotalCount": 0, "/*": "从remoter收到的应答数量", "RecvRemoterRspTotalCount": 0, "/*": "从remoter收到的无效数据数量", "RecvRemoterInvalidTotalCount": 0, "/*": "下一个编号", "NextNo": 0, "/*": "集群的综合监控信息", "ClusterMonitor": [{ "/*": "编号", "No": 0, "/*": "集群名，能连上核心时通过上下文获取核心appname，否则使用topicname_partitionno", "ClusterName": "mddb redo 1", "/*": "当前处理完成的最小事务号，如果最后的redo都被Filter1过滤掉可能最小事务号会小一些", "MinTransNo": 0, "/*": "最大已回库事务号", "MaxTransNo": 4, "/*": "最后收到redo的事务号", "LastRecvTransNo": 0, "/*": "最后收到redo的时间", "LastRecvTime": 0, "/*": "从remoter收到的redo数量", "RecvRemoterReqCount": 0, "/*": "小于zk上最小事务号的redo数量", "IgnoreRecvReqCount": 0, "/*": "Filter1过滤掉的redo数量", "Filter1RejectCount": 0, "/*": "Filter1过滤掉的最大事务号", "Filter1RejectMaxTransNo": 3, "/*": "线程定位失败的次数", "LocateErrorCount": 0, "/*": "Filter2过滤掉的redo数量", "Filter2RejectCount": 0, "/*": "回库处理出错的次数", "DealErrorCount": 0, "/*": "组件处理失败重试的次数", "Failed2RetryCount": 0, "/*": "应答超时重试的次数", "Timeout2RetryCount": 0, "/*": "应答返回平台错误的重试次数", "PlatErrorRsp2RetryCount": 0, "/*": "应答返回业务错误的重试次数", "BizErrorRsp2RetryCount": 0, "/*": "拦截消息的重试次数", "Intercept2RetryCount": 0, "/*": "组件处理失败丢弃的次数", "Failed2DropCount": 0, "/*": "超时丢弃的次数", "Timeout2DropCount": 0, "/*": "应答平台错误的丢弃次数", "PlatErrorRsp2DropCount": 0, "/*": "应答业务错误的丢弃次数", "BizErrorRsp2DropCount": 0, "/*": "拦截消息的丢弃次数", "Intercept2DropCount": 0, "/*": "请求超时丢弃场景下，删除已返回但未处理应答的次数", "Timeout2DropDelRsp": 0, "/*": "请求超时重试场景下，删除已返回但未处理应答的次数", "Timeout2MoveDelRsp": 0, "/*": "该集群的业务报错定位字段总数", "BizErrorLocateValueCount": 11, "/*": "该集群的业务报错的事务总数", "BizErrorRspDropCount": "123", "/*": "该集群的业务重发的事务总数", "BizErrorRspTotalCount": "11313", "/*": "该集群剩余重试的事务数量", "BizErrorToResendTransCount": 132, "/*": "该集群剩余重试的账户数量", "BizErrorToResendLocateValueCount": 9, "/*": "该集群业务重发的已处理总数", "BizErrorHistoryProcessCount": 231, "/*": "该集群历史重试总数", "BizErrorHistoryTotalCount": 231, "/*": "集群类型，1表示网络集群，0是非网络集群", "ClusterType": 1, "/*": "分区组编号", "GroupNo": 1, "/*": "分区组名称", "PartGroupName": "multiproc_mddb#0_partgroup_1", "/*": "该集群还有多少事务没执行完，非网络集群该值不生效为0", "RemainRedoCount": 100 }], "/*": "错误重试redo信息", "ResendClusterMonitor": [{ "/*": "集群名，能连上核心时通过上下文获取核心appname，否则使用topicname_partitionno", "ClusterName": "mddb redo 1", }] }</pre>	

```

        "/*": "最后收到redo的事务号",
        "lastRecvTransNo": 0,
        "/*": "最后收到redo的时间",
        "lastRecvTime": 0,
        "/*": "从remoter收到的redo数量",
        "recvRemoterReqCount": 0,
        "/*": "小于zk上最小事务号的redo数量",
        "ignoreRecvReqCount": 0,
        "/*": "Filter1过滤掉的redo数量",
        "filter1RejectCount": 0,
        "/*": "Filter1过滤掉的最大事务号",
        "filter1RejectMaxTransNo": 3,
        "/*": "线程定位失败次数",
        "locateErrorCount": 0,
        "/*": "Filter2过滤掉的redo数量",
        "filter2RejectCount": 0,
        "/*": "回库处理出错的次数",
        "dealErrorCount": 0,
        "/*": "组件处理失败重试的次数",
        "failed2RetryCount": 0,
        "/*": "应答超时重试的次数",
        "timeout2RetryCount": 0,
        "/*": "应答返回平台错误的重试次数",
        "platErrorRsp2RetryCount": 0,
        "/*": "应答返回业务错误的重试次数",
        "bizErrorRsp2RetryCount": 0,
        "/*": "拦截消息的重试次数",
        "intercept2RetryCount": 0,
        "/*": "组件处理失败丢弃的次数",
        "failed2DropCount": 0,
        "/*": "超时丢弃的次数",
        "timeout2DropCount": 0,
        "/*": "应答平台错误的丢弃次数",
        "platErrorRsp2DropCount": 0,
        "/*": "应答业务错误的丢弃次数",
        "bizErrorRsp2DropCount": 0,
        "/*": "拦截消息的丢弃次数",
        "intercept2DropCount": 0,
        "/*": "请求超时丢弃场景下, 删除已返回但未处理应答的次数",
        "timeout2DropDelRsp": 0,
        "/*": "请求超时重试场景下, 删除已返回但未处理应答的次数",
        "timeout2MoveDelRsp": 0,
        "/*": "最新重试已完成的事务号",
        "resentTransNo": 2,
        "/*": "待处理的重试最大事务号",
        "maxToResendTransNo": 3,
        "/*": "该集群是否日间重试完成",
        "resendFinishFlag": false,
        "/*": "集群类型, 1表示网络集群, 0是非网络集群",
        "clusterType": 1,
        "/*": "分区组编号",
        "groupNo": 1,
        "/*": "分区组名称",
        "partGroupName": "multiproc_mdb#0_partgroup_1"
    }
},
"/*": "所有集群是否日间重试完成",
"resendFinishFlag": false,
"/*": "错误处理策略监控信息",
"resultHandler": {
    "/*": "组件错误",
    "failed": {
        "/*": "重发间隔, 单位: 秒",
        "resendIntervalSec": 3,
        "/*": "最大重发次数, 包括第一次正常发送",
        "maxSendCount": 3,
        "/*": "所有redo重发的次数",
        "resendCount": 0
    },
    "/*": "超时错误",
    "timeout": {
        "/*": "重发间隔, 单位: 秒",
        "resendIntervalSec": 8,
        "/*": "最大重发次数, 包括第一次正常发送",
        "maxSendCount": 3,
        "/*": "所有redo重发的次数",
        "resendCount": 0
    },
    "/*": "应答平台错误",
    "platErrorRsp": {
        "/*": "重发间隔, 单位: 秒",
        "resendIntervalSec": 6,
        "/*": "最大重发次数, 包括第一次正常发送",
        "maxSendCount": 0,
        "/*": "所有redo重发的次数",
        "resendCount": 0,
        "/*": "配置中的错误号范围的解析结果",
        "errorNo": "-61;-1;1"
    },
    "/*": "应答业务错误",
    "bizErrorRsp": {
        "/*": "重发间隔, 单位: 秒",
        "resendIntervalSec": 5,
        "/*": "最大重发次数, 包括第一次正常发送",
        "maxSendCount": 0,
        "/*": "所有redo重发的次数",
        "resendCount": 0,
        "/*": "配置中的错误号范围的解析结果, 同PlatErrorRsp, 两者只会存在一个",
        "errorNo": "-61;-1;1"
    },
    "/*": "拦截消息",
    "intercept": {
        "/*": "重发间隔, 单位: 秒",
        "resendIntervalSec": 0,
        "/*": "最大重发次数, 包括第一次正常发送",
        "maxSendCount": 0,
        "/*": "所有redo重发的次数",
        "resendCount": 0
    },
    "/*": "多账号事务消息",
    "multiAccount": {
        "/*": "重发间隔, 单位: 秒",
        "resendIntervalSec": 0,
        "/*": "最大重发次数, 包括第一次正常发送",
        "maxSendCount": 0,
        "/*": "所有redo重发的次数",

```

```
        "ReSendCount": 0
    }
},
"//": "重试线程错误处理策略监控信息",
"ResultHandler": {
    "//": "组件错误",
    "Failed": {
        "//": "重发间隔, 单位: 秒",
        "ResendIntervalSec": 3,
        "//": "最大重发次数, 包括第一次正常发送",
        "MaxSendCount": 3,
        "//": "所有redo重发的次数",
        "ReSendCount": 0
    },
    "//": "超时错误",
    "Timeout": {
        "//": "重发间隔, 单位: 秒",
        "ResendIntervalSec": 8,
        "//": "最大重发次数, 包括第一次正常发送",
        "MaxSendCount": 3,
        "//": "所有redo重发的次数",
        "ReSendCount": 0
    },
    "//": "应答平台错误",
    "PlatErrorRsp": {
        "//": "重发间隔, 单位: 秒",
        "ResendIntervalSec": 6,
        "//": "最大重发次数, 包括第一次正常发送",
        "MaxSendCount": 0,
        "//": "所有redo重发的次数",
        "ReSendCount": 0,
        "//": "配置中的错误号范围的解析结果",
        "ErrorNo": "-61;-1;1"
    },
    "//": "应答业务错误",
    "BizErrorRsp": {
        "//": "重发间隔, 单位: 秒",
        "ResendIntervalSec": 5,
        "//": "最大重发次数, 包括第一次正常发送",
        "MaxSendCount": 0,
        "//": "所有redo重发的次数",
        "ReSendCount": 0,
        "//": "配置中的错误号范围的解析结果, 同PlatErrorRsp, 两者只会存在一个",
        "ErrorNo": "-61;-1;1"
    },
    "//": "拦截消息",
    "Intercept": {
        "//": "重发间隔, 单位: 秒",
        "ResendIntervalSec": 0,
        "//": "最大重发次数, 包括第一次正常发送",
        "MaxSendCount": 0,
        "//": "所有redo重发的次数",
        "ReSendCount": 0
    },
    "//": "多账号事务消息",
    "MultiAccount": {
        "//": "重发间隔, 单位: 秒",
        "ResendIntervalSec": 0,
        "//": "最大重发次数, 包括第一次正常发送",
        "MaxSendCount": 0,
        "//": "所有redo重发的次数",
        "ReSendCount": 0
    }
},
"//": "所有集群剩余重试事务数",
"BizErrorToResendTransCount": 0,
"//": "所有集群重试事务总数",
"BizErrorRspTotalCount": 0,
"//": "处理的最大redo的大小, 单位字节",
"MaxRedoSizeByte": 100000,
"//": "当前是否有运维事务正在阻塞回库, 0: 没有, 1: 有, 在等待执行状态, 2: 有, 正在执行状态",
"IsSqlRedoBlockIng": 0,
"//": "当前运维事务阻塞回库时间, 单位毫秒, IsSqlRedoBlockIng > 1 才会展示",
"SqlRedoBlockTimeMs": 100,
"//": "当前阻塞回库的运维事务的事务号, IsSqlRedoBlockIng > 1 才会展示",
"SqlRedoTransNo": 100,
}
```

6.10.8. ForceStopRecv 停止接收某个核心分片消息

功能名称	ForceStopRecv	版本号	V1.0.0.1	更新日期	20230807
功能备注					
功能说明	停止接收某个核心分片消息, 传集群号 或者 主题和分区停止接收分片				
访问链接	http://pport/kfp?Pretty=1&PluginName=kfp_todb&FuncName=ForceStopRecv				

入参说明

```
{
  "//": "集群号, 对应GetClusterInfo管理功能的ClusterNo, 传集群号 或者 主题和分区停止接收分片",
  "ClusterNo": "0",
  "//": "主题",
  "Topic": "redo",
  "//": "分区号",
  "Partition": "2"
}
```

出参说明

```
{
  "//": "错误号",
  "ErrorNo": 0,
  "//": "错误信息",
  "ErrorMsg": "Success"
}
```

6.10.9. ForceStartRecv 开启接收某个核心分片消息

功能名称	ForceStartRecv	版本号	V1.0.0.1	更新日期	20230807
功能备注					
功能说明	开启接收某个核心分片消息，传集群号 或者 主题和分区开启接收分片				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=ForceStartRecv				
入参说明					
<pre>{ "/*":"集群号，对应GetClusterInfo管理功能的ClusterNo，传集群号 或者 主题和分区开启接收分片", "ClusterNo":"0", "/*":"主题", "Topic":"redo", "/*":"分区号", "Partition":"2" }</pre>					
出参说明					
<pre>{ "/*":"错误号", "ErrorNo": 0, "/*":"错误信息", "ErrorMsg": "Success" }</pre>					

6.10.10. ForceSendBizErrRedo 重新发送业务错误redo

功能名称	ForceSendBizErrRedo	版本号	V1.0.0.0	更新日期	20260119
功能备注					
功能说明	重新发送业务错误redo				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=ForceSendBizErrRedo				
入参说明					
<pre>{ "/*": "是否开启重传，为false时报错", "ResendFlag": true, "/*": "集群名", "ClusterName": ["ucrtbid", "ucrtcbp"], "/*": "定位字段信息", "LocateInfo": [{ "/*": "定位字段名称", "LocateField": "fund_account", "/*": "定位字段值", "LocateValue": "1111" }]}</pre>					
出参说明					
<pre>{ "/*": "错误号", "ErrorNo": 0, "/*": "错误信息", "ErrorMsg": "Success"}</pre>					

6.10.11. TerminateForceSendBizErrRedo 终止重发业务错误redo

功能名称	TerminateForceSendBizErrRedo	版本号	V1.0.0.0	更新日期	20241223
功能备注					
功能说明	终止发送业务错误redo				
访问链接	http://ip:port/kjp?Pretty=1&PluginName=kjp_todb&FuncName=TerminateForceSendBizErrRedo				
入参说明					
<pre>{ "/*": "是否停止重发，为false时报错", "ResendFlag": true, "/*": "超时时间", "TimeOutSec": 10}</pre>					
出参说明					
<pre>{ "/*": "错误号", "ErrorNo": 0, "/*": "错误信息", "ErrorMsg": "Success"}</pre>					

6.10.12. GetBizErrResendStatus 获取重发业务错误的状态

功能名称	GetBizErrResendStatus	版本号	V1.0.0.0	更新日期	20260119
功能备注					
功能说明	获取重发业务错误的状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetBizErrResendStatus				

入参说明

```
{  "":"/": "起始编号，默认为0",  "StartNo": "0",  "":"/": "查询数量，默认为10",  "Count": "10",  "":"/": "重试状态，默认查所有状态，状态，0：进行中，1：正常完成，2：中止完成",  "Status": "0"}
```

出参说明

```
{  "":"/": "结束编号，用于翻页",  "NextNo": 1,  "BizErrResendStatus": [    {      "":"/": "编号",      "No": 1,      "":"/": "集群名",      "ClusterName": "ucrtbid",      "":"/": "集群类型，1表示网格集群，0是非网格集群",      "ClusterType": 1,      "":"/": "分区组编号",      "GroupNo": 1,      "":"/": "分区组名称",      "PartGroupName": "multiproc_mdb#0_partgroup_1",      "":"/": "定位字段名",      "LocateField": "fund_account",      "":"/": "定位字段值",      "LocateValue": 1111,      "":"/": "线程组别名",      "QueueAlias": 3,      "":"/": "定位字段名描述",      "FieldDesc": "资金账户",      "":"/": "开始重试同步序号",      "StartMsgNo": 1,      "":"/": "开始重试事务号",      "StartTransNo": 1,      "":"/": "最新重试同步序号",      "LastMsgNo": 5,      "":"/": "最新重试的事务号",      "LastTransNo": 5,      "":"/": "结束的重试同步序号",      "EndMsgNo": 11,      "":"/": "结束的事务号",      "EndTransNo": 11,      "":"/": "重试已成功事务数量",      "ProcessedCount": 111,      "":"/": "需要重试的事务总数",      "TotalCount": 120,      "":"/": "开始时间",      "StartTime": "2024-11-21 11:12:11",      "":"/": "结束时间",      "EndTime": "2024-11-21 11:12:11",      "":"/": "耗时，单位毫秒",      "SpendTimeMilli": 12312,      "":"/": "状态，0：进行中，1：正常完成，2：中止完成",      "Status": 0    }  ]}
```

6.10.13. GetHistoryBizErrResendStatus 获取历史重发业务错误的状态

功能名称	GetHistoryBizErrResendStatus	版本号	V1.0.0.0	更新日期	20260119
功能备注					
功能说明	获取历史重发业务错误的状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetHistoryBizErrResendStatus				
入参说明					
<pre>{ "":"/": "起始编号，默认为0", "StartNo": "0", "":"/": "查询数量，默认为10", "Count": "10" }</pre>					
出参说明					

```
{
  "":"/": "结束编号，用于翻页",
  "NextNo": 1,
  "HistoryBizErrResendStatus": [
    {
      "":"/": "编号",
      "No": 1,
      "":"/": "集群名",
      "ClusterName": "ucrtbid",
      "":"/": "集群类型，1表示网络集群，0是非网络集群",
      "ClusterType": 1,
      "":"/": "分区组编号",
      "GroupNo": 1,
      "":"/": "分区组名称",
      "PartGroupName": "multiproc_mdb#0_partgroup_1",
      "":"/": "定位字段名",
      "LocateField": "fund_account",
      "":"/": "定位字段值",
      "LocateValue": 1111,
      "":"/": "线程组别名",
      "QueueAlias": 3,
      "":"/": "定位字段名描述",
      "FieldDesc": "资金账户",
      "":"/": "开始重试同步序号",
      "StartMsgNo": 1,
      "":"/": "开始重试事务号",
      "StartTransNo": 1,
      "":"/": "最新重试同步序号",
      "LastMsgNo": 5,
      "":"/": "最新重试的事务号",
      "LastTransNo": 5,
      "":"/": "结束的重试同步序号",
      "EndMsgNo": 11,
      "":"/": "结束的事务号",
      "EndTransNo": 11,
      "":"/": "重试已成功事务数量",
      "ProcessedCount": 111,
      "":"/": "需要重试的事务总数",
      "TotalCount": 120,
      "":"/": "开始时间",
      "StartTime": "2024-11-21 11:12:11",
      "":"/": "结束时间",
      "EndTime": "2024-11-21 11:12:11",
      "":"/": "耗时，单位毫秒",
      "SpendTimeMilli": 12312,
      "":"/": "状态，0：进行中，1：正常完成，2：中止完成",
      "Status": 0,
      "":"/": "历史账户重试成功的数量",
      "HistoryProcessedCount": 111,
      "":"/": "历史需要重试的数量",
      "HistoryTotalCount": 120,
      "":"/": "历史重试的时间",
      "HistorySpendTimeMilli": 12312
    }
  ]
}
```

功能备注	
功能说明	获取重发业务错误的状态统计信息
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetResendStatusStatistics

入参说明
无

出参说明
<pre>{ "":"/": "全部重发账户的数量", "TotalCount": 10, "":"/": "进行中的重发账户数量", "ProcessingCount": 4, "":"/": "正常完成的重发账户数量", "SuccesedCount": 5, "":"/": "暂停完成的重发账户数量", "PausedCount": 1 }</pre>

6.10.15. GetBizErrStatistics 获取重发业务错误的统计信息

功能名称	GetBizErrStatistics	版本号	V1.0.0.0	更新日期	20260119
功能备注					
功能说明	获取重发业务错误的统计信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetBizErrStatistics				
入参说明					

<pre>{ "":"/: "起始编号，默认为0", "StartNo": "0", "":"/: "数量，默认为10", "Count": "10", "":"/: "集群名，多个按分割符隔开", "ClusterName": "ucrtbid", "":"/: "定位字段名", "LocateField": "fund_account, 可选,当前仅支持一个", "":"/: "定位字段值", "LocateValue": "1111, 可选, 多个按分割符隔开, 小于等于50个", "":"/: "分割符，默认为,", "Delimiter": ",,", "":"/: "是否仅查询剩余要重试的账户，默认为0查询所有，为1时查询剩余", "ToResendQuery": "1" }</pre>
出参说明
<pre>{ "":"/: "下一页起始编号", "NextNo": 1, "BizErrStatistics":[{ "":"/: "编号", "No": 1, "":"/: "集群名", "ClusterName": "ucrtbid", "":"/: "集群类型，1表示网络集群，0是非网络集群", "ClusterType": 1, "":"/: "分区组编号", "GroupNo": 1, "":"/: "分区组名称", "PartGroupName": "multiproc_mdb#0_partgroup_1", "":"/: "定位字段名", "LocateField": "fund_account", "":"/: "定位字段值", "LocateValue": "11111", "":"/: "线程组别名", "QueueAlias": 3, "":"/: "定位字段名描述", "FieldDesc": "资金账户", "":"/: "该账户实际业务报错的事务总数", "BizErrorRspDropCount": 131, "":"/: "该账户业务重发的事务总数", "BizErrorRspTotalCount": 231, "":"/: "该账户剩余重试的事务数量", "BizErrorToResendTransCount": 132, "":"/: "该账户业务重发的已处理总数", "HistoryProcessedCount": 231, "":"/: "历史重试总数量", "HistoryTotalCount": 231, "":"/: "最后一次错误号，默认为0", "LastErrorNo": 111 }], "":"/: "所有账户重试完成数量", "AllHistoryProcessedCount": 0, "":"/: "所有账户需要重试总数量", "AllBizErrorRspTotalCount": 1, "":"/: "所有历史重试总数量", "AllHistoryTotalCount": 0, }</pre>

6.10.16. GetBizErrRefCount 获取重发错误引用计数

功能名称	GetBizErrRefCount	版本号	V1.0.0.0	更新日期	20260119
功能备注					
功能说明	获取重发错误引用计数				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetBizErrRefCount				

入参说明

```
{  "":"/: "数量，默认为10",  "Count": "10",}
```

出参说明

```
{  "BizErrRefCount": [    {      "":"/: "线程号",      "ThreadNo": 1,      "":"/: "集群号",      "ClusterNo": 0,      "":"/: "集群名",      "ClusterName": "mcib",      "":"/: "定位字段名称",      "LocateField": "fund_account",      "":"/: "定位字段值",      "LocateValue": "1",      "":"/: "线程组别名",      "QueueAlias": 3,      "":"/: "定位字段名描述",      "FieldDesc": "资金账户",      "":"/: "引用计数",      "RefCount": 1    }  ]}
```

6.10.17. GetVersion todb当前进程版本

功能名称	GetVersion	版本号	V1.0.0.1	更新日期	20241220
功能备注					
功能说明	查看todb当前进程版本				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_todb&FuncName=GetVersion				
入参说明					
无					
出参说明					
<pre>{ "Version": "v2" }</pre>					

6.10.18. GetBizErrorMsgInfo 获取业务报错信息

功能名称	GetBizErrorMsgInfo	版本号	V1.0.0.1	更新日期	20250418
功能备注					
功能说明	获取业务报错信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_todb&FuncName=GetBizErrorMsgInfo				
入参说明					
<pre>{ "/*": "查询起始编号，不传默认为0", "StartMsgNo": "0", "/*": "查询消息数量，不传默认为10，如果不够10条则认为到结尾", "MsgCount": "10", "/*": "查询超时时间，单位ms，不传默认为3000", "TimeoutMilli": "3000", "/*": "集群名，多个值按分隔符隔开，例如ucrtbid,ucrtcbp", "ClusterName": "ucrtbid", "/*": "定位字段名，目前只支持1个", "LocateField": "fund_account", "/*": "定位字段值，多个值按分隔符隔开，例如111,1111", "LocateValue": "", "/*": "过滤类型，默认为0，过滤字段按与的逻辑处理，为1，过滤字段按或的逻辑处理，目前只支持且和或的过滤，其他规则不支持", "FilterType": "0", "/*": "分割符，默认','分割", "Delimiter": ",", "/*": "0:重试未完成, 1: 重试完成, 2: 所有的", "IsResent": "2" }</pre>					
出参说明					
<pre>{ "/*": "报错事务详细信息", "BizErrorMsgInfo": [{ "/*": "消息编号", "MsgNo": 0, "/*": "事务号", "TransNo": 1, "/*": "事务长度", "TransLength": 68, "/*": "错误号", "ErrorNo": 131, "/*": "错误信息", "ErrorMsg": "logic error", "/*": "事务时间", "Time": "1031-112903.250748", "/*": "集群号", "ClusterNo": 1, "/*": "集群名", "ClusterName": "ucrtbid", "/*": "集群类型, 1表示网络集群, 0是非网络集群", "ClusterType": 1, "/*": "分区组编号", "GroupNo": 1, "/*": "分区组名称", "PartGroupName": "multiproc_mdb#0_partgroup_1", "/*": "定位字段值", "LocateValue": "1111", "/*": "定位字段名", "LocateField": "11", "/*": "线程组别名", "QueueAlias": 3, "/*": "定位字段名描述", "FieldDesc": "资金账户", "/*": "功能号", "FunctionNo": 333001, "/*": "线程号", "ThreadNo": 1, "/*": "重试是否完成", "IsResent": true, "/*": "重试是否丢弃", "IsDrop": false }], "/*": "下一个消息序号，方便后续前端缓存进行翻页", "NextMsgNo": 1, "/*": "满足查询条件的总数", "TotalCount": 1 }</pre>					

6.10.19. GetMsgBodyInfo 获取业务报错记录信息

功能名称	GetMsgBodyInfo	版本号	V1.0.0.1	更新日期	20241228
功能备注					
功能说明	获取业务报错记录信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todlb&FuncName=GetMsgBodyInfo&MsgNo=0&StartActionNo=0&ActionCount=10&TimeoutMill=3000				
入参说明					
<pre>{ "///<": "消息序号，默认为0", "MsgNo": "0", "///<": "起始操作编号，默认0", "StartActionNo": "0", "///<": "查询超时时间，单位ms，不传默认为3000", "TimeoutMilli": "3000", "///<": "操作数量，默认10", "ActionCount": "10" }</pre>					
出参说明					
<pre>{ "Actions": [{ "///<": "操作编号", "No": 0, "///<": "操作表名", "TableName": "entrust", "///<": "操作类型， 值为Create,Update,Delete,SyncSeq,ResetSeq等", "Type": "Create", "///<": "操作记录号， 序列号清表赋0", "RecordNo": 1, "///<": "操作记录信息， 字段名,字段值,...", "RecordInfo": { "fund_account": "111", "client_id": "111" } }], "///<": "下一个操作编号， 翻页使用", "NextActionNo": 1 }</pre>					

6.10.20. GetBizErrRedoInfo 获取业务错误redo信息

功能名称	GetBizErrRedoInfo	版本号	V1.0.0.0	更新日期	20240523
功能备注					
功能说明	获取业务错误redo信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todlb&FuncName=GetBizErrRedoInfo				

入参说明
无
出参说明
<pre>{ "BizErrRedoInfo": [{ "transNo": 1, "threadNo": 0, "clusterNo": 0, "newTransNo": 0, "msglen": 201 }] }</pre>

6.10.21. GetClusterInfo 查看配置集群信息

功能名称	GetClusterInfo	版本号	V1.0.0.1	更新日期	20230807
功能备注	查看配置集群信息				
功能说明	查看配置集群信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetClusterInfo				
入参说明					
无					
出参说明					
<pre>{ "clusterInfo": [{ "ClusterNo": 0, "TopicName": "redo", "PartitionNo": 2, "MinTransNo": 3, "TransportId": 4, "TxContextId": 1, "ClusterName": "redo_2", "FrontEndName": "todb_ljl#1_ldp_todb_0_receiver_0", "RecvStatus": true, "StartOnce": true, "ClusterType": 1, "GroupNo": 1, "PartGroupName": "multiproc_mdb#0_partgroup_1", "TurnNo": 5, "TurnSwitchTransNo": 20 }] }</pre>					

6.10.22. GetModuleInfo 查看加载的组件信息

功能名称	GetModuleInfo	版本号	V1.0.0.1	更新日期	20231214
功能备注					
功能说明	查看加载的组件信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetModuleInfo				
入参说明					
无					
出参说明					

<pre>{ "/*": "加载的所有组件信息", "ModuleInfo": [{ "/*": "组件名", "Name": "ldp_mdbredo", "/*": "组件版本", "Version": "ldp_mdbredo V1.0.14.3 Nov 30 2023 17:11:12", "/*": "组件加载路径", "Path": "/home/yangds/LDP1.0/Sources/workspace/./bin/todb.local/libldp_mdbredo.so" }] }</pre>

6.10.23. LastErrMsg 上次错误应答详细信息

功能名称	LastErrMsg	版本号	V1.0.0.0	更新日期	20240313
功能备注					
功能说明					
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=LastErrMsg				
入参说明					
无					
出参说明					
<pre>{ "/*": "错误号", "errNo": 1, "/*": "错误信息", "errMsg": "" }</pre>					

6.10.24. GetOnTimerTransNoTaskStatistics 获取事务号定时任务执行统计信息

功能名称	GetOnTimerTransNoTaskStatistics	版本号	V1.0.0.0	更新日期	20250615
功能备注	获取事务号定时任务执行统计信息				
功能说明	获取事务号定时任务执行统计信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetOnTimerTransNoTaskStatistics				
入参说明					
无					
出参说明					
<pre>{ "/*": "同步最大已回库事务号执行统计", "SynTransNoTaskStatistics": { "/*": "功能号", "FunctionID": 9991, "/*": "累计执行总次数", "TotalExecNum": 0, "/*": "累计执行失败次数", "FailExecNum": 0 } }</pre>					

6.10.25. GetTodbStatus 获取回库内部状态

功能名称	GetTodbStatus	版本号	V1.0.0.0	更新日期	20251216
功能备注	获取回库内部状态				
功能说明	获取回库内部状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetTodbStatus				
入参说明					
无					
出参说明					
<pre>{ "/*": "是否暂停", "IsPaused": false, "/*": "是否rcm仲裁", "IsRcmSupport": false, "/*": "Rcm角色", "Role": 17, "/*": "是否所有都是备机", "IsAnyInActive": false, "/*": "是否开启错误重试", "IsEnableBizErrResend": true, "/*": "全链路消息池大小", "MsgWriterPoolSize": 256}</pre>					

6.10.26. GetGlobalMultiThreadInfo 获取全局多线程框架信息

功能名称	GetGlobalMultiThreadInfo	版本号	V1.0.0.0	更新日期	20251216
功能备注	获取全局多线程框架信息				
功能说明	获取全局多线程框架信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetGlobalMultiThreadInfo				
入参说明					
无					
出参说明					
<pre>{ "GlobalMutliThreadInfo": [{ "//": "别名", "QueueAlias": 1, "//": "加锁竞争失败次数", "LockFailCount": 0, "//": "是否正在执行任务", "IsRunning": false, "//": "因子线程数量", "MultiThreadCount": 4 }]}</pre>					

6.10.27. GetMinHeaperInfo redo合并开启，获取当前最小合并事务号及集群剩余事务数统计信息

功能名称	GetMinHeaperInfo	版本号	V1.0.0.0	更新日期	20251209
功能备注	redo合并开启，获取当前最小合并事务号				
功能说明	redo合并开启，获取当前最小合并事务号及集群剩余事务数统计信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetMinHeaperInfo				
入参说明					
无					
出参说明					
<pre>{ "MinHeaper": [{ "//": "线程id", "ThreadNo": 0, "ClusterInfo": [{ "//": "集群id", "ClusterNo": 0, "//": "该线程的该集群的剩余事务的最小合并的事务号", "MinMergeTransNo": 18446744073709551615, "//": "该线程的该集群的剩余事务数", "TransCount": 0 }] }]}</pre>					

6.10.28. GetResendProcessingTransInfo 获取各工作线程正在执行重试的事务信息

功能名称	GetResendProcessingTransInfo	版本号	V2.0.1	更新日期	20260311
功能备注	获取各工作线程正在执行重试的事务信息				
功能说明	获取各工作线程正在执行重试的事务信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetResendProcessingTransInfo				
入参说明					
无					
出参说明					

```
[
{
  "/*": "线程号",
  "ThreadId": 1,
  "/*": "集群号",
  "ClusterNo": 1,
  "/*": "事务号",
  "TransNo": 10,
  "/*": "错误重试类型",
  "ErrorType": 2,
  "/*": "事务类型",
  "Type": 3,
  "/*": "重试次数",
  "ResendCount": 10,
  "/*": "是否表拆分，默认0表示否",
  "TableSplit": 1,
  "/*": "redo合并的最小事务号，如果没有合并就是transNo本身",
  "RedoMergeMinTransNo": 10,
  "/*": "定位域",
  "LocateField": "fund_account",
  "/*": "定位值",
  "LocateValue": "1234",
  "/*": "功能号",
  "FuncNo": 100
}
]
```

6.10.29. ForceDiscardTrans 丢弃指定事务

功能名称	ForceDiscardTrans	版本号	V2.0.1	更新日期	20260311
功能备注	丢弃指定事务				
功能说明	丢弃指定事务				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=ForceDiscardTrans				
入参说明					
<pre>{ "Request": { "TransInfo": [{ "/*": "集群号", "ClusterNo": 1, "/*": "事务号", "TransNo": 10, }] } }</pre>					
出参说明					
<pre>{ "/*": "错误码", "ErrorNo": 0, "/*": "错误信息", "ErrorMsg": "Sucess" }</pre>					

6.10.30. GetForceDiscardTransInfo 获取丢弃过的事务信息

功能名称	GetForceDiscardTransInfo	版本号	V2.0.1	更新日期	20260311
功能备注	获取丢弃过的事务信息				
功能说明	获取丢弃过的事务信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetForceDiscardTransInfo				
入参说明					
无					
出参说明					

```
[
{
  "线程号": "线程号",
  "ThreadId": 1,
  "集群号": "集群号",
  "ClusterNo": 1,
  "事务号": "事务号",
  "TransNo": 10,
  "错误重试类型": "错误重试类型",
  "ErrorType": 2,
  "事务类型": "事务类型",
  "Type": 3,
  "重试次数": "重试次数",
  "ResendCount": 10,
  "是否表拆分": "是否表拆分",
  "TableSplit": 1,
  "redo合并的最小事务号": "redo合并的最小事务号",
  "RedoMergeMinTransNo": 10,
  "定位域": "定位域",
  "LocateField": "fund_account",
  "定位值": "定位值",
  "LocateValue": "1234",
  "功能号": "功能号",
  "FuncNo": 100,
  "丢弃时间": "丢弃时间",
  "DiscardTime": "2026-03-10 16:43:00"
}
]
```

6.10.31. GetMinHeapInsertFailInfo 获取插入最小堆失败事务信息

功能名称	GetMinHeapInsertFailInfo	版本号	V2.0.1	更新日期	20260407
功能备注	获取插入最小堆失败事务信息				
功能说明	获取插入最小堆失败事务信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=GetMinHeapInsertFailInfo				
入参说明					
无					
出参说明					
<pre>{ "MinHeapInsertFailInfo": [{ "线程号", "ThreadNo": 1, "集群号", "ClusterNo": 0, "todb进程启动以来该集群失败的总次数", "FailCount": 3476, "该集群最后一次失败的时间", "LastFailTime": "16:26:14.967301", "该集群最后一次失败的事务号", "LastTransNo": 5, "该集群最后一次失败的事务索引", "LastTransIndex": 14579, "该集群最后一次失败的事务的最小合并事务号", "LastMinMergeTransNo": 5 }]}</pre>					

6.10.32. ldp_mdbrdo_GetDataMgrInfo 直连数据库组件加载的业务库信息

功能名称	ldp_mdbrdo_GetDataMgrInfo	版本号	V1.0.0.1	更新日期	20230629
功能备注					
功能说明	查看直连数据库组件加载的业务库信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbrdo_GetDataMgrInfo				
入参说明					
无					
出参说明					

```
{
  "DataMgrInfo": [
    {
      "BizType": "ldpmdb",
      "LibName": "s_ldpmdb_datamgr",
      "TabelDefine": []
    },
    {
      "/*": "业务类型",
      "BizType": "ldpmdb",
      "/*": "库名",
      "LibName": "s_ldpmdb_datamgr",
      "/*": "表定义信息",
      "TabelDefine": [
        {
          "/*": "表号",
          "RecordCategory": 104,
          "/*": "表定义",
          "RecordDefine": "struct user{char fund_account[19];char password[12];int8_t money_count;double average_business_price;double balance;int32_t login_time;char user_id[19];}; struct varfields{char Name[61];};"
        }
      ]
    }
  ]
}
```

6.10.33. ldp_mdbrdo_GetTableInfo 直连数据库组件表配置信息

功能名称	ldp_mdbrdo_GetTableInfo	版本号	V1.0.0.1	更新日期	20240423
功能备注					
功能说明	查看直连数据库组件表配置信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbrdo_GetTableInfo				
入参说明					
无					
出参说明					
<pre>{ "tables": [{ "/*": "是否是异地机房配置", "IsRemoteZone": false, "/*": "表号", "Category": 104, "/*": "内存表表名", "MdbTableName": "user", "/*": "物理表表名", "LogicTableName": "user_table", "/*": "是否回库", "EnableTableTodb": true, "/*": "是否清库", "EnableTableClear": false, "/*": "是否过滤业务初始化", "EnableTableInitDate": true, "/*": "是否空转空格", "ToSpace": false, "/*": "物理数据库类型", "DriverType": 70, "FieldsRename": [{ "/*": "内存表字段名", "MdbFieldName": "fund_account", "/*": "物理表字段名", "LogicFieldName": "fund_account", }], /*": "不需要回库的字段", "FieldsFilter": [], "/*": "配置的索引名", "LogicIndexName": "idx_name", "/*": "加载的索引字段名", "UniqueIndex": ["fund_account"], /*": "当前表允许回库的集群名", "ClusterTodb": ["mdb_test"] }]}</pre>					

6.10.34. ldp_mdbrdo_GetMonitor 直连数据库组件表自身监控

功能名称	ldp_mdbrdo_GetMonitor	版本号	V1.0.0.1	更新日期	20240423
功能备注					
功能说明	查看直连数据库组件表配置信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbrdo_GetMonitor				
入参说明					
无					
出参说明					

<pre>{ "/*": "处理失败的次数", "FailedCount": 0, "/*": "无定位字段的次数", "NoLocateCount": 0, "/*": "非异地机房redoProcess数量", "LocalZoneCount": 0, "/*": "异地机房redoProcess数量", "RemoteZoneCount": 0, "/*": "上一次数据库执行错误信息", "LastExecErrMsg":"" }</pre>

6.10.35. ldp_mdbredo_GetRedoMergeInfo 获取直联数据库事务合并信息

功能名称	ldp_mdbredo_GetRedoMergeInfo	版本号	V1.0.0.1	更新日期	20240423
功能备注					
功能说明	获取直联数据库事务合并信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbredo_GetRedoMergeInfo				
入参说明					
无					
出参说明					
<pre>{ "RedoMerges": [{ "//": "集群号", "ClusterNo": 0, "//": "redo合并执行器编号", "RedoMergeNo": 0, "//": "redo合并缓存个数", "CacheCount": 1 }] }</pre>					

6.10.36. ldp_mdbredo_GetMdbRedoMergeInfo 获取内存数据库redo合并信息

功能名称	ldp_mdbredo_GetMdbRedoMergeInfo	版本号	V1.0.0.1	更新日期	20240423
功能备注					
功能说明	获取内存数据库redo合并信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbredo_GetMdbRedoMergeInfo				
入参说明					
无					
出参说明					
<pre>{ "RedoMergers": [{ "/*": "redo合并执行器编号", "RedoMergerNo": 0, "/*": "原始redo最大积压数量", "RedoBacklogMaxCount": 5000, "/*": "redo合并理论内存占用上限,可能会超", "MaxRedoMergeMemoryGB": 8, "/*": "redo合并日志总大小上限,单位:兆", "TotalMergeRedoMaxSizeMB": 4096, "/*": "redo合并后单个redo日志最大大小,单位:字节", "PerRedoMaxSize": 12000000, "/*": "redo合并后单个redo日志中单表操作记录数上限,至少512及以上,否则可能存在内存问题", "RedoMaxRecordCountPerTable": 512, "/*": "redo合并结果集返回等待时长,单位:毫秒", "ResultReturnWaitTimeMs": 1000, "/*": "执行redo合并的操作字符串,C-创建记录,U-更新记录,D-删除记录,Q-清表", "RedoMergeAction": "CUDQ", "/*": "开启redo合并结果集是否写Debug日志,默认: false", "IsMergeRedoWriteDebugLog": false, "/*": "是否开启redo合并,默认true", "EnableRedoMerge": false, "/*": "Redo合并执行总次数", "MergeExecuteCount": 0, "/*": "本次redo合并执行开始时间,结果集返回后清零重新统计", "RedoMergeStartMs": 0, "/*": "当前redo合并组件执行状态,0-就绪状态,1-初始化状态,2-停止状态,4-已出错状态", "RedoMergeStatus": 0, "/*": "当前已接收的原始redo日志最大事务号", "MaxOriginalTransactionNo": 0, "/*": "已接收的原始redo日志最小事务号", "MinOriginalTransactionNo": 1, "/*": "当前已执行redo合并的redo日志最大事务号", "MaxMergeRedoTransactionNo": 0, "/*": "当前回调结果集的最后一个事务号", "LastCallbackTransactionNo": 0, "/*": "记录数据上场后的第一个事务号", "FirstMergeRedoTransactionNo": 0, "/*": "记录redo合并模块中缓存未发送到todb的最小事务号", "CachedMinTransactionNo": 18446744073709551615, "/*": "成功发送到todb模块的redo数量", "RedoSendTodbCount": 0, "/*": "执行过redo合并的redo数量", "MergeRedoCount": 0, "/*": "获取接收的redo数量", "RecvRedoCount": 21700 }] }</pre>					


```

    redoMergeCount": 31700,
    "/*": "获取找到多个可合并置已完成的次数",
    "MultiEableMergeCount": 31700,
    "/*": "执行过事务合并的数量",
    "TransactionMergeCount": 0,
    "/*": "统计记录操作合并出错的次数(期望为0)",
    "MergeErrorHappenedCount": 0,
    "/*": "统计redo标志为TRANS_HEAD_FLAG_MULTI_ACCOUNT的事务数量",
    "MultiAccountTCCount": 0,
    "/*": "统计redo标志为FLAG_UPDATE_RECORD_PHY_KEY的事务数量",
    "UpdatePhyKeyCount": 0,
    "/*": "统计redo标志为 TRANS_HEAD_FLAG_UNSTORE 的事务数量",
    "ReloadTblTCCount": 0,
    "/*": "统计分片重组redo的事务数量",
    "FragmentRedoCount": 0,
    "/*": "统计合并后事务有过相同事务号的数量",
    "MergeRedoSameTransactionCount": 0,
    "/*": "统计合并结果集相关对象",
    "RedoMergeObjs": {
        "/*": "原始Redo日志实际积压数量",
        "OriginalRedoSize": 0,
        "/*": "统计redo合并结果集链表的数量",
        "ListRedoMergeResultSize": 0,
        "/*": "统计不需要合并的redo的数量",
        "ArrRedoDataItemSize": 0,
        "/*": "统计拆分表redo合并结果集链表的数量",
        "ListSequenceOpeSize": 0,
        "/*": "统计 redo 拆分合并后结果集数量",
        "MultimapRedoResultSize": 0
    },
    "/*": "统计内存池相关信息",
    "MemPoolObjs": [
        {
            "/*": "内存池所有者:redo合并器",
            "MemoryPoolOwner": "RedoMergeMemPool",
            "/*": "统计已使用的块数",
            "UsedBlockCount": 0,
            "/*": "统计已释放的块数",
            "FreedBlockCount": 0,
            "/*": "统计当前块索引",
            "CurrBlockIndex": 0,
            "/*": "统计当前内存池的实际内存占用情况",
            "CurrMemPoolByte": 0,
            "/*": "获取当前内存缓存区占用字节数",
            "CurrBufferMemPoolByte": 0,
            "/*": "统计调用get次数统计",
            "UserMmapCount": 0,
            "/*": "统计调用release次数统计",
            "UserMunmapCount": 0
        },
        {
            "/*": "内存池所有者:redo分片重组器",
            "MemoryPoolOwner": "RedoFragmentMemPool",
            "/*": "统计已使用的块数",
            "UsedBlockCount": 0,
            "/*": "统计已释放的块数",
            "FreedBlockCount": 0,
            "/*": "统计当前块索引",
            "CurrBlockIndex": 0,
            "/*": "统计当前内存池的实际内存占用情况",
            "CurrMemPoolByte": 0,
            "/*": "获取当前内存缓存区占用字节数",
            "CurrBufferMemPoolByte": 0,
            "/*": "统计调用get次数统计",
            "UserMmapCount": 0,
            "/*": "统计调用release次数统计",
            "UserMunmapCount": 0
        }
    ],
    "/*": "分片重组信息",
    "RedoFragmentMerger": {
        "/*": "分片重组器详细统计(备库)",
        "/*": "缓冲状态:当前缓冲中的事务数量",
        "BufferedTxCount": 0,
        "/*": "缓冲状态:当前缓冲总大小(字节)",
        "BufferedTotalSize": 0,
        "/*": "缓冲状态:当前缓冲中最小的TransferId",
        "MinBufferedTransferId": 0,
        "/*": "回放状态:下个要回放的传输ID",
        "NextTransactionNo": 59,
        "/*": "回放状态:故障恢复的待处理起始点",
        "MinTransactionNo": 18446744073709551615,
        "/*": "回放状态:故障恢复前已处理的最大点",
        "MaxTransactionNo": 18446744073709551615,
        "/*": "累计统计:累计收到的分片数",
        "TotalReceivedFragments": 0,
        "/*": "累计统计:累计完成重组的事务数",
        "TotalCompletedTx": 0,
        "/*": "累计统计:累计完整redo(未分片)数量",
        "TotalCompleteRedo": 0,
        "/*": "累计统计:累计重组redo数量",
        "TotalReassembledRedo": 0,
        "/*": "累计统计:累计重组redo字节数",
        "TotalReassembledBytes": 0,
        "/*": "累计统计:单个事务最大分片数量",
        "MaxFragmentCount": 0,
        "/*": "累计统计:单个事务最大重组大小(字节)",
        "MaxReassembledSize": 0,
        "/*": "错误统计:内存分配失败次数",
        "AllocFailCount": 0,
        "/*": "错误统计:跳过的分片数量",
        "SkippedFragmentCount": 0,
        "BufferedTransactions": [
            {
                "/*": "缓冲中的事务详情(最多显示100个)",
                "/*": "事务ID",
                "TxId": 0,
                "/*": "该事务第一个分片对应的TransferId",
                "FirstFragmentTransferId": 0,
                "/*": "该事务redo总大小(字节)",
                "TotalSize": 0,
                "/*": "该事务分片数量",
                "FragmentCount": 0,
                "/*": "该事务当前缓冲区容量(字节)",
                "BufferCapacity": 0
            }
        ]
    }
}

```

```
    },
    "高优先级表名信息列表",
    "HighPriorityTables": [
      {
        "表名",
        "TableName": "account"
      }
    ]
  },
]
}
```

6.10.37. ldp_gwredo_GetDataMgrInfo 网关组件加载的业务库信息

功能名称	ldp_gwredo_GetDataMgrInfo	版本号	V1.0.0.1	更新日期	20240423
功能备注					
功能说明	查看网关组件加载的业务库信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_GetDataMgrInfo				
入参说明					
无					
出参说明					
<pre>{ "DataMgrInfo": [{ "BizType": "ldpmdb", "LibName": "s_ldpmdb_datamgr", "TabelDefine": [] }, { "业务类型", "BizType": "ldpmdb", "库名", "LibName": "s_ldpmdb_datamgr", "表定义信息", "TabelDefine": [{ "表号", "RecordCategory": 104, "表定义", "RecordDefine": "struct user{char fund_account[19];char password[12];int8_t money_count;double average_business_price;double balance;int32_t login_time;char user_id[19];}; struct varfields{char Name[61];};" }] }] }</pre>					

6.10.38. ldp_gwredo_GetTableInfo 网关组件表配置信息

功能名称	kdp_gwredo_GetTableInfo	版本号	V1.0.0.1	更新日期	20240423
功能备注					
功能说明	查看网关组件表配置信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=kdp_gwredo_GetTableInfo				
入参说明					
无					
出参说明					
<pre>{ "tables": [{ "是否是异地机房配置", "IsRemoteZone": false, "表号", "Category": 104, "内存表表名", "MdbTableName": "user", "物理表表名", "LogicTableName": "user_table", "是否回库", "EnableTableTodb": true, "是否清库", "EnableTableClear": false, "是否过滤业务初始化", "EnableTableInitDate": true, "是否空转空格", "ToSpace": false, "当前表允许回库的集群名", "ClusterTodb": ["mdb_test"] }]}</pre>					

6.10.39. ldp_gwredo_GetMonitor 网关组件表自身监控

功能名称	ldp_gwredo_GetMonitor	版本号	V1.0.0.1	更新日期	20240423
功能备注					

功能说明	网关组件表自身监控
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=kdp_gwredo_GetMonitor
入参说明	
无	
出参说明	
<pre>{ "/*": "处理失败的次数", "FailedCount": 0, "/*": "无定位字段的次数", "NoLocateCount": 0, "/*": "非异地机房redoProcess数量", "LocalZoneCount": 0, "/*": "异地机房redoProcess数量", "RemoteZoneCount": 0 }</pre>	

6.10.40. ldp_gwredo_GetTableLocate gwredo按表定位监控信息

功能名称	kdp_gwredo_GetTableLocate	版本号	V1.0.0.1	更新日期	20241220
功能备注					
功能说明	查看gwredo按表定位监控信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=kdp_gwredo_GetTableLocate				
入参说明					
无					
出参说明					
<pre>{ "TableLocate": { "/*": "普通表 --- 无特殊功能，只按照线程分配", "NormalTable": [{ "/*": "表名", "Name": "user_table", "/*": "表号", "Category": 104, "/*": "线程号", "ThreadNo": 3, "/*": "集群号", "ClusterNo": 0, "/*": "集群名", "ClusterName": "mdb", "/*": "收到的最大事务号", "RecvMaxTransNo": 3, "/*": "上次处理的事务号", "LastProcessTransNo": 3, "/*": "处理数", "ProcessCount": 100, "/*": "过滤数", "FilterCount": 0 }], "/*": "低频表", "LowFrequencyTable": [{ "Name": "user_table", "Category": 104, "ThreadNo": 3, "/*": "集群号", "ClusterNo": 0, "/*": "集群名", "ClusterName": "mdb", "ProcessCount": 100, "FilterCount": 0, "/*": "记录数", "RecordCount": 100 }] }, "/*": "特殊标（内置表）", "SpecialTable": [{ "Name": "user_table", "Category": 104, "ThreadNo": 3, "/*": "集群号", "ClusterNo": 0, "/*": "集群名", "ClusterName": "mdb", "RecvMaxTransNo": 3, "LastProcessTransNo": 3, "ProcessCount": 100, "FilterCount": 0 }] }</pre>					

6.10.41. ldp_mdbredo_GetTableLocate mdbredo按表定位监控信息

功能名称	ldp_mdbredo_GetTableLocate	版本号	V1.0.0.1	更新日期	20241220
功能备注					

功能说明	查看mdbredo按表定位监控信息
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbredo_GetTableLocate
入参说明	
无	
出参说明	
<pre>{ "TableLocate": { "//": "普通表 --- 无特殊功能，只按照线程分配", "NormalTable": [{ "//": "表名", "Name": "user_table", "//": "表号", "Category": 104, "//": "线程号", "ThreadNo": 3, "//": "集群号", "ClusterNo": 0, "//": "集群名", "ClusterName": "mclb", "//": "收到的最大事务号", "RecvMaxTransNo": 3, "//": "上次处理的事务号", "LastProcessTransNo": 3, "//": "处理数", "ProcessCount": 100, "//": "过滤数", "FilterCount": 0 }], "//": "低频表", "LowFrequencyTable": [{ "Name": "user_table", "Category": 104, "ThreadNo": 3, "//": "集群号", "ClusterNo": 0, "//": "集群名", "ClusterName": "mclb", "ProcessCount": 100, "FilterCount": 0, "//": "记录数", "RecordCount": 100 }], "//": "特殊标（内置表）", "SpecialTable": [{ "Name": "user_table", "Category": 104, "ThreadNo": 3, "//": "集群号", "ClusterNo": 0, "//": "集群名", "ClusterName": "mclb", "RecvMaxTransNo": 3, "LastProcessTransNo": 3, "ProcessCount": 100, "FilterCount": 0 }] } }</pre>	

6.10.42. ldp_gwredo_GetRedoMergeInfo 获取gwredo事务合并信息

功能名称	ldp_gwredo_GetRedoMergeInfo	版本号	V1.0.0.1	更新日期	20240423
功能备注					
功能说明	获取mdbredo事务合并信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_GetRedoMergeInfo				
入参说明					
无					
出参说明					
<pre>{ "RedoMerges": [{ "//": "集群号", "ClusterNo": 0, "//": "redo合并执行器编号", "RedoMergeNo": 0, "//": "redo合并缓存个数", "CacheCount": 1 }]}</pre>					

6.10.43. ldp_gwredo_GetMdbRedoMergeInfo 获取内存数据库redo合并信息

功能名称	ldp_gwredo_GetMdbRedoMergeInfo	版本号	V1.0.0.1	更新日期	20240423

功能备注	
功能说明	获取内存数据库redo合并信息
访问链接	http://ip:port/kdp?Pretty=1&PhgInName=kdp_todb&FuncName=kdp_gwredo_GetMdbRedoMergeInfo
入参说明	
无	
出参说明	
<pre>{ "RedoMergers": [{ "/*": "redo合并执行器编号", "RedoMergerNo": 0, "/*": "原始redo最大积压数量", "RedoBacklogMaxCount": 5000, "/*": "redo合并理论内存占用上限, 可能会超", "MaxRedoMergeMemoryGB": 8, "/*": "redo合并日志总大小上限, 单位:兆", "TotalMergeRedoMaxSizeMB": 4096, "/*": "redo合并后单个redo日志最大大小, 单位:字节", "PerRedoMaxSize": 12000000, "/*": "redo合并后单个redo日志中单表操作记录数上限, 至少512及以上, 否则可能存在内存问题", "RedoMaxRecordCountPerTable": 512, "/*": "redo合并结果集返回等待时长, 单位:毫秒", "ResultReturnWaitTimeMs": 1000, "/*": "执行redo合并的操作字符串, C-创建记录, U-更新记录, D-删除记录, Q-清表", "RedoMergeAction": "CUDQ", "/*": "开启redo合并结果集是否写Debug日志, 默认: false", "IsMergeRedoWriteDebugLog": false, "/*": "是否开启redo合并, 默认true", "EnableRedoMerge": false, "/*": "Redo合并执行总次数", "MergeExecuteCount": 0, "/*": "本次redo合并执行开始时间, 结果集返回后清零重新统计", "RedoMergeStartMs": 0, "/*": "当前redo合并组件执行状态, 0-就绪状态, 1-初始化状态, 2-停止状态, 4-已出错状态", "RedoMergeStatus": 0, "/*": "当前已接收的原始redo日志最大事务号", "MaxOriginalTransactionNo": 0, "/*": "已接收的原始redo日志最小事务号", "MinOriginalTransactionNo": 1, "/*": "当前已执行redo合并的redo日志最大事务号", "MaxMergeRedoTransactionNo": 0, "/*": "当前回调结果集的最后一个事务号", "LastCallbackTransactionNo": 0, "/*": "记录数据上场后的第一个事务号", "FirstMergeRedoTransactionNo": 0, "/*": "记录redo合并模块中缓存未发送到todb的最小事务号", "CachedMinTransactionNo": 18446744073709551615, "/*": "成功发送到todb模块的redo数量", "RedoSendTodbCount": 0, "/*": "执行过redo合并的redo数量", "MergeRedoCount": 0, "/*": "获取接收的redo数量", "RecvRedoCount": 31700, "/*": "获取找到多个可合并置已完成的次数", "MultiEableMergeCount": 31700, "/*": "执行过事务合并的数量", "TransactionMergeCount": 0, "/*": "统计记录操作合并出错的次数 (期望为0)", "MergeErrorHappenedCount": 0, "/*": "统计redo标志为TRANS_HEAD_FLAG_MULTI_ACCOUNT的事务数量", "MultiAccountTCCount": 0, "/*": "统计redo标志为FLAG_UPDATE_RECORD_PHY_KEY的事务数量", "UpdatePhyKeyCount": 0, "/*": "统计redo标志为 TRANS_HEAD_FLAG_UNSTORE 的事务数量", "ReloadTblTCCount": 0, "/*": "统计分片重组redo的事务数量", "EfragmentRedoCount": 0, "/*": "统计合并后事务有过相同事务号的数量", "MergeRedoSameTransactionCount": 0, "/*": "统计合并结果集相关对象", "RedoMergeObjs": { "/*": "原始Redo日志实际积压数量", "OriginalRedoSize": 0, "/*": "统计redo合并结果集链表的数量", "ListRedoMergeResultSize": 0, "/*": "统计不需要合并的redo的数量", "ArrRedoDataItemSize": 0, "/*": "统计拆分成redo合并结果集链表的数量", "ListSequenceOpeSize": 0, "/*": "统计 redo 拆分合并后结果集数量", "MultimapRedoResultSize": 0 }, "/*": "统计内存池相关信息", "MemPoolObjs": [{ "/*": "内存池拥有者: redo合并器", "MemoryPoolOwner": "RedoMergeMemPool", "/*": "统计已使用的块数", "UsedBlockCount": 0, "/*": "统计已释放的块数", "FreedBlockCount": 0, "/*": "统计当前块索引", "CurrBlockIndex": 0, "/*": "统计当前内存池的实际内存占用情况", "CurrMemPoolByte": 0, "/*": "获取当前内存缓存区占用字节数", "CurrBufferMemPoolByte": 0, "/*": "统计调用get次数统计", "UserMmapCount": 0, "/*": "统计调用release次数统计", "UserMunmapCount": 0 }, { "/*": "内存池拥有者: redo分片重组器", "MemoryPoolOwner": "RedoFragmentMemPool", "/*": "统计已使用的块数", "UsedBlockCount": 0, </pre>	

```
        "///<": "统计已释放的块数",
        "FreedBlockCount": 0,
        "///<": "统计当前块索引",
        "CurrBlockIndex": 0,
        "///<": "统计当前内存池的实际内存占用情况",
        "CurrMemPoolByte": 0,
        "///<": "获取当前内存缓存区占用字节数",
        "CurrBufferMemPoolByte": 0,
        "///<": "统计调用get次数统计",
        "UserMmapCount": 0,
        "///<": "统计调用release次数统计",
        "UserMunmapCount": 0
    }
},
"///<": "分片重组信息",
"RedoFragmentMerger": {
    "///<": "分片重组器详细统计（备库）",
    "///<": "缓冲状态：当前缓冲中的事务数量",
    "BufferedTxCount": 0,
    "///<": "缓冲状态：当前缓冲总大小（字节）",
    "BufferedTotalSize": 0,
    "///<": "缓冲状态：当前缓冲中最小的TransferId",
    "MinBufferedTransferId": 0,
    "///<": "回放状态：下个要回放的传输ID",
    "NextTransactionNo": 59,
    "///<": "回放状态：故障恢复的待处理起始点",
    "MinTransactionNo": 18446744073709551615,
    "///<": "回放状态：故障恢复前已处理的最大点",
    "MaxTransactionNo": 18446744073709551615,
    "///<": "累计统计：累计收到的分片数",
    "TotalReceivedFragments": 0,
    "///<": "累计统计：累计完成重组的事务数",
    "TotalCompletedTx": 0,
    "///<": "累计统计：累计完整redo（未分片）数量",
    "TotalCompleteRedo": 0,
    "///<": "累计统计：累计重组redo数量",
    "TotalReassembledRedo": 0,
    "///<": "累计统计：累计重组redo字节数",
    "TotalReassembledBytes": 0,
    "///<": "累计统计：单个事务最大分片数量",
    "MaxFragmentCount": 0,
    "///<": "累计统计：单个事务最大重组大小（字节）",
    "MaxReassembledSize": 0,
    "///<": "错误统计：内存分配失败次数",
    "AllocFailCount": 0,
    "///<": "错误统计：跳过的分片数量",
    "SkippedFragmentCount": 0,
    "BufferedTransactions": [
        {
            "///<": "缓冲中的事务详情（最多显示100个）",
            "///<": "事务ID",
            "TxId": 0,
            "///<": "该事务第一个分片对应的TransferId",
            "FirstFragmentTransferId": 0,
            "///<": "该事务redo总大小（字节）",
            "TotalSize": 0,
            "///<": "该事务分片数量",
            "FragmentCount": 0,
            "///<": "该事务当前缓冲区容量（字节）",
            "BufferCapacity": 0
        }
    ]
},
"///<": "高优先级表表名信息列表",
"HighPriorityTables": [
    {
        "///<": "表名",
        "TableName": "account"
    }
]
},
]
}
```

6.10.44. ldp_gwredo_GetLocateFieldInfo 获取gwredo定位字段信息

功能名称	ldp_gwredo_GetLocateFieldInfo	版本号	V1.0.0.1	更新日期	20250610
功能备注					
功能说明	获取mdbredo事务合并信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_GetLocateFieldInfo				
入参说明					
无					
出参说明					
<pre>{ "///": "0使用“FieldLocate”， 1使用“FieldLocateThreadAffinity”，其他不支持", "LocateMode": 1, "LocateField": [{ "///": "定位字段名", "FieldName": "fund_account", "///": "定位字段名描述", "FieldDesc": "资金账户" }, { "FieldName": "stock_account", "FieldDesc": "股东账户" }] }</pre>					

6.10.45. ldp_mdbredo_GetLocateFieldInfo 获取mdbredo定位字段信息

--	--	--	--	--	--

功能名称	ldp_mdbrdo_GetLocateFieldInfo	版本号	V1.0.0.1	更新日期	20250610
功能备注					
功能说明	获取mdbrdo事务合并信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_toddb&FuncName=ldp_mdbrdo_GetLocateFieldInfo				
入参说明					
无					
出参说明					
<pre>{ "":"/": "0使用\"FieldLocate\", 1使用\"FieldLocateThreadAffinity\", 其他不支持", "LocateMode": 1, "LocateField": [{ "":"/": "定位字段名", "FieldName": "fund_account", "":"/": "定位字段名描述", "FieldDesc": "资金账户" }, { "FieldName": "stock_account", "FieldDesc": "股东账户" }] }</pre>					

6.10.46. ldp_gwredo_GetTransTableSplitInfo 获取事务级表并发处理信息

功能名称	ldp_gwredo_GetTransTableSplitInfo	版本号	V1.0.0.1	更新日期	20250725
功能备注					
功能说明	获取事务级表并发处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_toddb&FuncName=ldp_gwredo_GetTransTableSplitInfo				
入参说明					
无					
出参说明					
<pre>{ "TransTableSplitInfo": [{ "":"/": "工作线程编号", "ThreadNo": 0, "":"/": "工作线程所属线程组", "QueueAlias": 4, "":"/": "集群名", "ClusterName": "mddb", "":"/": "子线程编号", "SubThreadNo": 0, "":"/": "子线程处理的所有表名, 多个按: 隔开", "TotalProcessedTableName": "account;entrust", "":"/": "子线程处理的最新表名, 多个按: 隔开", "LastProcessedTableName": "account", "":"/": "处理数量", "DealTotalCount": 0, "":"/": "处理失败数量", "DealFailedCount": 0, "":"/": "处理最小的事务号", "DealMinTransNo": 0, "":"/": "处理最大的事务号", "DealMaxTransNo": 0, "":"/": "最新的处理时间, 单位ns", "ProcessLastNs": 0, "":"/": "最小的处理时间, 单位ns", "ProcessMinNs": 0, "":"/": "最大的处理时间, 单位ns", "ProcessMaxNs": 0, "":"/": "平均的处理时间, 单位ns", "ProcessAvgNs": 0 }] }</pre>					

6.10.47. ldp_mdbrdo_GetTransTableSplitInfo 获取事务级表并发处理信息

功能名称	ldp_mdbrdo_GetTransTableSplitInfo	版本号	V1.0.0.1	更新日期	20250731
功能备注					
功能说明	获取事务级表并发处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_toddb&FuncName=ldp_mdbrdo_GetTransTableSplitInfo				
入参说明					
无					
出参说明					

```
{
  "TransTableSplitInfo": [
    {
      "ThreadNo": 0,
      "QueueAlias": 4,
      "ClusterName": "mcb",
      "SubThreadNo": 0,
      "TotalProcessedTableName": "account;entrust",
      "LastProcessedTableName": "account",
      "DealTotalCount": 0,
      "DealFailedCount": 0,
      "DealMinTransNo": 0,
      "DealMaxTransNo": 0,
      "ProcessLastNs": 0,
      "ProcessMinNs": 0,
      "ProcessMaxNs": 0,
      "ProcessAvgNs": 0
    }
  ]
}
```

6.10.48. ldp_mdbrdo_GetBatchSqlProcInfo 获取批量sql处理信息

功能名称	ldp_mdbrdo_GetBatchSqlProcInfo	版本号	V1.0.0.1	更新日期	20250725
功能备注					
功能说明	获取批量sql处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbrdo_GetBatchSqlProcInfo				
入参说明					
无					
出参说明					


```
{
  "/*": "批量sql处理信息",
  "BatchSqlProcInfo": [
    {
      "/*": "线程号",
      "ThreadNo": 0,
      "/*": "线程组别名",
      "QueueAlias": 2,
      "/*": "集群名",
      "ClusterName": "xxx",
      "/*": "子线程号, -1代表在工作线程执行的"
      "SubThreadNo": -1,
      "/*": "处理的总数量"
      "DealTotalCount": 17,
      "/*": "处理失败的数量",
      "DealFailedCount": 1,
      "/*": "已处理最小的事务号",
      "DealMinTransNo": 1,
      "/*": "已处理最大的事务号",
      "DealMaxTransNo": 58608,
      "/*": "处理最新耗时",
      "ProcessLastNs": 4644401,
      "/*": "处理最小耗时",
      "ProcessMinNs": 1240840,
      "/*": "处理最大耗时",
      "ProcessMaxNs": 9643157,
      "/*": "处理平均耗时",
      "ProcessAvgNs": 2094556,
      "/*": "最新处理表细节",
      "LastProcessedTableDetail": [
        {
          "/*": "表名",
          "TableName": "uses_serial_counter",
          "/*": "事务中该表插入记录的数量",
          "InsertCount": 1,
          "/*": "事务中该表更新记录的数量",
          "UpdateCount": 0,
          "/*": "事务中该表删除记录的数量",
          "DeleteCount": 0
        }
      ],
      "/*": "历史处理表细节",
      "HistoryProcessedTableDetail": [
        {
          "/*": "表名",
          "TableName": "uses_serial_counter",
          "/*": "该表插入记录的总数量",
          "InsertTotalCount": 1,
          "/*": "该表单事务插入记录的最大数量",
          "InsertMaxCount": 1,
          "/*": "该表单事务插入记录的最小数量",
          "InsertMinCount": 1,
          "/*": "该表单事务插入记录的平均数量",
          "InsertAvgCount": 1,
          "/*": "该表单事务插入记录的最新数量",
          "InsertLastCount": 1,
          "/*": "该表更新记录的总数量",
          "UpdateTotalCount": 0,
          "/*": "该表单事务更新记录的最大数量",
          "UpdateMaxCount": 0,
          "/*": "该表单事务更新记录的最小数量",
          "UpdateMinCount": 0,
          "/*": "该表单事务更新记录的平均数量",
          "UpdateAvgCount": 0,
          "/*": "该表单事务更新记录的最新数量",
          "UpdateLastCount": 0,
          "/*": "该表删除记录的总数量",
          "DeleteTotalCount": 0,
          "/*": "该表单事务删除记录的最大数量",
          "DeleteMaxCount": 0,
          "/*": "该表单事务删除记录的最小数量",
          "DeleteMinCount": 0,
          "/*": "该表单事务删除记录的平均数量",
          "DeleteAvgCount": 0,
          "/*": "该表单事务删除记录的最新数量",
          "DeleteLastCount": 0
        }
      ]
    }
  ]
}
```

6.10.49. ldp_mdbredo_GetBigTransProcInfo 获取批量sql处理信息

功能名称	ldp_mdbredo_GetBigTransProcInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取批量sql处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_toddb&FuncName=ldp_mdbredo_GetBigTransProcInfo				
入参说明					
无					
出参说明					

```
{
  "":"/": "批量sql处理信息",
  "BigTransProcInfo": [
    {
      "":"/": "线程号",
      "ThreadNo": 0,
      "":"/": "线程组别名",
      "QueueAlias": 2,
      "":"/": "集群名",
      "ClusterName": "xxx",
      "":"/": "子线程号, -1代表在工作线程执行的",
      "SubThreadNo": -1,
      "":"/": "处理的总数量",
      "DealTotalCount": 17,
      "":"/": "处理失败的数量",
      "DealFailedCount": 1,
      "":"/": "已处理最小的事务号",
      "DealMinTransNo": 1,
      "":"/": "已处理最大的事务号",
      "DealMaxTransNo": 58608,
      "":"/": "处理最新耗时",
      "ProcessLastNs": 4644401,
      "":"/": "处理最小耗时",
      "ProcessMinNs": 1240840,
      "":"/": "处理最大耗时",
      "ProcessMaxNs": 9643157,
      "":"/": "处理平均耗时",
      "ProcessAvgNs": 2094556,
      "":"/": "最新处理表细节",
      "LastProcessedTableDetail": [
        {
          "":"/": "表名",
          "TableName": "uses_serial_counter",
          "":"/": "事务中该表插入记录的数量",
          "InsertCount": 1,
          "":"/": "事务中该表更新记录的数量",
          "UpdateCount": 0,
          "":"/": "事务中该表删除记录的数量",
          "DeleteCount": 0
        }
      ],
      "":"/": "历史处理表细节",
      "HistoryProcessedTableDetail": [
        {
          "":"/": "表名",
          "TableName": "uses_serial_counter",
          "":"/": "该表插入记录的总数量",
          "InsertTotalCount": 1,
          "":"/": "该表单事务插入记录的最大数量",
          "InsertMaxCount": 1,
          "":"/": "该表单事务插入记录的最小数量",
          "InsertMinCount": 1,
          "":"/": "该表单事务插入记录的平均数量",
          "InsertAvgCount": 1,
          "":"/": "该表单事务插入记录的最新数量",
          "InsertLastCount": 1,
          "":"/": "该表更新记录的总数量",
          "UpdateTotalCount": 0,
          "":"/": "该表单事务更新记录的最大数量",
          "UpdateMaxCount": 0,
          "":"/": "该表单事务更新记录的最小数量",
          "UpdateMinCount": 0,
          "":"/": "该表单事务更新记录的平均数量",
          "UpdateAvgCount": 0,
          "":"/": "该表单事务更新记录的最新数量",
          "UpdateLastCount": 0,
          "":"/": "该表删除记录的总数量",
          "DeleteTotalCount": 0,
          "":"/": "该表单事务删除记录的最大数量",
          "DeleteMaxCount": 0,
          "":"/": "该表单事务删除记录的最小数量",
          "DeleteMinCount": 0,
          "":"/": "该表单事务删除记录的平均数量",
          "DeleteAvgCount": 0,
          "":"/": "该表单事务删除记录的最新数量",
          "DeleteLastCount": 0
        }
      ]
    }
  ]
}
```

6.10.50. ldp_gwredo_GetBigTransProcInfo 获取批量sql处理信息

功能名称	ldp_gwredo_GetBigTransProcInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取批量sql处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_GetBigTransProcInfo				
入参说明					
无					
出参说明					

```
{
  "BigTransProcInfo": [
    {
      "/*": "工作线程编号",
      "ThreadNo": 0,
      "/*": "工作线程所属线程组",
      "QueueAlias": 4,
      "/*": "集群名",
      "ClusterName": "mcb",
      "/*": "子线程编号",
      "SubThreadNo": 0,
      "/*": "子线程处理的所有表名，多个按：隔开",
      "TotalProcessedTableName": "account;entrust",
      "/*": "子线程处理的最新表名，多个按：隔开",
      "LastProcessedTableName": "account",
      "/*": "处理数量",
      "DealTotalCount": 0,
      "/*": "处理失败数量",
      "DealFailedCount": 0,
      "/*": "处理最小的事务号",
      "DealMinTransNo": 0,
      "/*": "处理最大的事务号",
      "DealMaxTransNo": 0,
      "/*": "最新的处理时间，单位ns",
      "ProcessLastNs": 0,
      "/*": "最小的处理时间，单位ns",
      "ProcessMinNs": 0,
      "/*": "最大的处理时间，单位ns",
      "ProcessMaxNs": 0,
      "/*": "平均的处理时间，单位ns",
      "ProcessAvgNs": 0
    }
  ]
}
```

6.10.51. ldp_md bred o _GetMultiAccountProcInfo 获取多账户事务处理信息

功能名称	kdp_md bred o _GetMultiAccountProcInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取多账户事务处理信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_todb&FuncName=kdp_md bred o _GetMultiAccountProcInfo				
入参说明					
无					
出参说明					

```
{
  "多账户事务处理信息",
  "MultiAccountProcInfo": [
    {
      "线程号",
      "ThreadNo": 0,
      "线程组别名",
      "QueueAlias": 2,
      "集群名",
      "ClusterName": "xxx",
      "子线程号, -1代表在工作线程执行的",
      "SubThreadNo": -1,
      "处理的总数量",
      "DealTotalCount": 17,
      "处理失败的数量",
      "DealFailedCount": 1,
      "已处理最小的事务号",
      "DealMinTransNo": 1,
      "已处理最大的事务号",
      "DealMaxTransNo": 58608,
      "处理最新耗时",
      "ProcessLastNs": 4644401,
      "处理最小耗时",
      "ProcessMinNs": 1240840,
      "处理最大耗时",
      "ProcessMaxNs": 9643157,
      "处理平均耗时",
      "ProcessAvgNs": 2094556,
      "最新处理表细节",
      "LastProcessedTableDetail": [
        {
          "表名",
          "TableName": "uses_serial_counter",
          "事务中该表插入记录的数量",
          "InsertCount": 1,
          "事务中该表更新记录的数量",
          "UpdateCount": 0,
          "事务中该表删除记录的数量",
          "DeleteCount": 0
        }
      ],
      "历史处理表细节",
      "HistoryProcessedTableDetail": [
        {
          "表名",
          "TableName": "uses_serial_counter",
          "该表插入记录的总数量",
          "InsertTotalCount": 1,
          "该表单事务插入记录的最大数量",
          "InsertMaxCount": 1,
          "该表单事务插入记录的最小数量",
          "InsertMinCount": 1,
          "该表单事务插入记录的平均数量",
          "InsertAvgCount": 1,
          "该表单事务插入记录的最新数量",
          "InsertLastCount": 1,
          "该表更新记录的总数量",
          "UpdateTotalCount": 0,
          "该表单事务更新记录的最大数量",
          "UpdateMaxCount": 0,
          "该表单事务更新记录的最小数量",
          "UpdateMinCount": 0,
          "该表单事务更新记录的平均数量",
          "UpdateAvgCount": 0,
          "该表单事务更新记录的最新数量",
          "UpdateLastCount": 0,
          "该表删除记录的总数量",
          "DeleteTotalCount": 0,
          "该表单事务删除记录的最大数量",
          "DeleteMaxCount": 0,
          "该表单事务删除记录的最小数量",
          "DeleteMinCount": 0,
          "该表单事务删除记录的平均数量",
          "DeleteAvgCount": 0,
          "该表单事务删除记录的最新数量",
          "DeleteLastCount": 0
        }
      ]
    }
  ]
}
```

6.10.52. ldp_gwredo_GetMultiAccountProcInfo 获取多账户事务处理信息

功能名称	ldp_gwredo_GetMultiAccountProcInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取多账户事务处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_GetMultiAccountProcInfo				
入参说明					
无					
出参说明					

<pre>{ "MultiAccountProcInfo": [{ "/*": "工作线程编号", "ThreadNo": 0, "/*": "工作线程所属线程组", "QueueAlias": 4, "/*": "集群名", "ClusterName": "mcb", "/*": "子线程编号", "SubThreadNo": 0, "/*": "子线程处理的所有表名, 多个按: 隔开", "TotalProcessedTableName": "account;entrust", "/*": "子线程处理的最新表名, 多个按: 隔开", "LastProcessedTableName": "account", "/*": "处理数量", "DealTotalCount": 0, "/*": "处理失败数量", "DealFailedCount": 0, "/*": "处理最小的事务号", "DealMinTransNo": 0, "/*": "处理最大的事务号", "DealMaxTransNo": 0, "/*": "最新的处理时间, 单位ns", "ProcessLastNs": 0, "/*": "最小的处理时间, 单位ns", "ProcessMinNs": 0, "/*": "最大的处理时间, 单位ns", "ProcessMaxNs": 0, "/*": "平均的处理时间, 单位ns", "ProcessAvgNs": 0 }] }</pre>

6.10.53. ldp_mdbredo_GetTodbAttributeInfo 获取回库属性信息

功能名称	ldp_md bredo_GetTodbAttributeInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取多账户事务处理信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_md bredo_GetTodbAttributeInfo				
入参说明					
无					
出参说明					
<pre>{ "TodbAttributeInfo": { "/*": "是否过滤上场数据", "EnableInitDataFilter": true, "/*": "是否过滤日间数据", "EnableTradeDataFilter": false, "/*": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": true, "/*": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false } }</pre>					

6.10.54. ldp_gwredo_GetTodbAttributeInfo 获取回库属性信息

功能名称	ldp_gwredo_GetTodbAttributeInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取多账户事务处理信息				
访问链接	http://p:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_GetTodbAttributeInfo				
入参说明					
无					
出参说明					
<pre>{ "TodbAttributeInfo": { "//": "是否过滤上场数据", "EnableInitDataFilter": true, "//": "是否过滤日间数据", "EnableTradeDataFilter": false, "//": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": true, "//": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false } }</pre>					

6.10.55. ldp_mdbredo_GetTableAttributeInfo 获取回库表属性信息

功能名称	klp_md bredo_GetTableAttributeInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取回库表属性信息				

访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbredo_GetTableAttributeInfo
入参说明	
无	
出参说明	
<pre>{ "TableAttributeInfo": [{ "/*": "内存表名", "MdbTableName": "account", "/*": "是否过滤上场数据", "EnableInitDataFilter": true, "/*": "是否过滤日间数据", "EnableTradeDataFilter": false, "/*": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": false, "/*": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false, "/*": "指定回库的核心集群名", "ClusterTodb": [{ "/*": "集群名", "ClusterName": "*mdb*" }] }] }</pre>	

6.10.56. ldp_gwredo_GetTableAttributeInfo 获取回库表属性信息

功能名称	ldp_gwredo_GetTableAttributeInfo	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	获取回库表属性信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_GetTableAttributeInfo				
入参说明					
无					
出参说明					
<pre>{ "TableAttributeInfo": [{ "/*": "内存表名", "MdbTableName": "account", "/*": "是否过滤上场数据", "EnableInitDataFilter": true, "/*": "是否过滤日间数据", "EnableTradeDataFilter": false, "/*": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": false, "/*": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false, "/*": "指定回库的核心集群名", "ClusterTodb": [{ "/*": "集群名", "ClusterName": "*mdb*" }] }]}</pre>					

6.10.57. ldp_mdbredo_UpdateTodbAttribute 更新回库属性信息

功能名称	ldp_mdbredo_UpdateTodbAttribute	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	更新回库表属性信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbredo_UpdateTodbAttribute				
入参说明					
<pre>{ "TodbAttributeInfo": { "//": "是否过滤上场数据", "EnableInitDataFilter": true, "//": "是否过滤日间数据", "EnableTradeDataFilter": false, "//": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": true, "//": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false } }</pre>					
出参说明					

```
{
  "TodbAttributeInfo": {
    "///": "是否过滤上场数据",
    "EnableInitDataFilter": true,
    "///": "是否过滤日间数据",
    "EnableTradeDataFilter": false,
    "///": "备机房是否过滤上场数据",
    "EnableInitDataFilterOfRemote": true,
    "///": "备机房是否过滤日间数据",
    "EnableTradeDataFilterOfRemote": false
  }
}
```

6.10.58. ldp_gwredo_UpdateTodbAttribute 更新回库属性信息

功能名称	ldp_gwredo_UpdateTodbAttribute	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	更新回库表属性信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_UpdateTodbAttribute				
入参说明					
<pre>{ "TodbAttributeInfo": { "///": "是否过滤上场数据", "EnableInitDataFilter": true, "///": "是否过滤日间数据", "EnableTradeDataFilter": false, "///": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": true, "///": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false } }</pre>					
出参说明					
<pre>{ "TodbAttributeInfo": { "///": "是否过滤上场数据", "EnableInitDataFilter": true, "///": "是否过滤日间数据", "EnableTradeDataFilter": false, "///": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": true, "///": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false } }</pre>					

6.10.59. ldp_mdbredo_UpdateTableAttribute 更新回库表属性信息

功能名称	ldp_mdbredo_UpdateTableAttribute	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	更新回库表属性信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_mdbredo_UpdateTableAttribute				
入参说明					
<pre>{ "TableAttributeInfo": [{ "///": "内存表名", "MdbTableName": "account", "///": "是否过滤上场数据", "EnableInitDataFilter": true, "///": "是否过滤日间数据", "EnableTradeDataFilter": false, "///": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": false, "///": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false, "///": "指定回库的核心集群名", "ClusterTodb": [{ "///": "集群名", "ClusterName": "*mdb*" }] }] }</pre>					
出参说明					

```
{
  "TableAttributeInfo": [
    {
      "/*": "内存表名",
      "MdbTableName": "account",
      "/*": "是否过滤上场数据",
      "EnableInitDataFilter": true,
      "/*": "是否过滤日间数据",
      "EnableTradeDataFilter": false,
      "/*": "备机房是否过滤上场数据",
      "EnableInitDataFilterOfRemote": false,
      "/*": "备机房是否过滤日间数据",
      "EnableTradeDataFilterOfRemote": false,
      "/*": "指定回库的核心集群名",
      "ClusterTodb": [
        {
          "/*": "集群名",
          "ClusterName": "*mdb*"
        }
      ]
    }
  ]
}
```

6.10.60. ldp_gwredo_UpdateTableAttribute 更新回库表属性信息

功能名称	ldp_gwredo_UpdateTableAttribute	版本号	V1.0.0.1	更新日期	20251216
功能备注					
功能说明	更新回库表属性信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_todb&FuncName=ldp_gwredo_UpdateTableAttribute				
入参说明					
<pre>{ "TableAttributeInfo": [{ "/*": "内存表名", "MdbTableName": "account", "/*": "是否过滤上场数据", "EnableInitDataFilter": true, "/*": "是否过滤日间数据", "EnableTradeDataFilter": false, "/*": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": false, "/*": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false, "/*": "指定回库的核心集群名", "ClusterTodb": [{ "/*": "集群名", "ClusterName": "*mdb*" }] }] }</pre>					
出参说明					
<pre>{ "TableAttributeInfo": [{ "/*": "内存表名", "MdbTableName": "account", "/*": "是否过滤上场数据", "EnableInitDataFilter": true, "/*": "是否过滤日间数据", "EnableTradeDataFilter": false, "/*": "备机房是否过滤上场数据", "EnableInitDataFilterOfRemote": false, "/*": "备机房是否过滤日间数据", "EnableTradeDataFilterOfRemote": false, "/*": "指定回库的核心集群名", "ClusterTodb": [{ "/*": "集群名", "ClusterName": "*mdb*" }] }] }</pre>					

6.11. 报盘框架插件 ldp_offerproc

6.11.1. GetOfferArbStatus 仲裁状态

功能名称	GetOfferArbStatus	版本号	V1.0.0.0	更新日期	20220516
功能备注	仲裁状态				
功能说明	获取仲裁状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_offerproc&FuncName=GetOfferArbStatus				
入参说明					
无					
出参说明					

<pre>{ "///": "主备状态, ARB_NONE: 无状态; ARB_ACTIVE: 主机; ARB_INACTIVE: 备机", "ArbStatus": "ARB_ACTIVE", "///": "报盘是否有仲裁状态, true: 有仲裁状态; false: 无仲裁状态", "ExistArbStatus": true }</pre>

6.11.2. GetProxyShardNo 获取报盘代理分片序号

功能名称	GetProxyShardNo	版本号	V1.0.0.0	更新日期	20220516
功能备注	报盘代理分片序号				
功能说明	获取报盘代理分片序号				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetProxyShardNo				
入参说明					
无					
出参说明					
<pre>{ "///": "报盘代理分片序号", "ProxyShardNo": 1 }</pre>					

6.11.3. GetOfferTradeTopic 获取报盘交易主题

功能名称	GetOfferTradeTopic	版本号	V1.0.0.0	更新日期	20220516
功能备注	报盘交易主题				
功能说明	获取报盘交易主题				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferTradeTopic				
入参说明					
无					
出参说明					
<pre>{ "///": "报盘交易组播主题", "TradeTopic": "report", "///": "报盘交易组播分区号", "TradePartitionNo": 1 }</pre>					

6.11.4. GetOfferTimeStamp 获取时间戳

功能名称	GetOfferTimeStamp	版本号	V1.0.0.0	更新日期	20220516
功能备注	获取时间戳标志				
功能说明	获取时间戳标志				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferTimeStamp				
入参说明					
无					
出参说明					
<pre>{ "///": "时间戳标志", "TimeStamp": false, }</pre>					

6.11.5. GetOfferGroupInfo 获取报盘组信息

功能名称	GetOfferGroupInfo	版本号	V1.0.0.60	更新日期	20230103
功能备注	报盘组信息				
功能说明	获取报盘组信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferGroupInfo				
入参说明					
无					
出参说明					

```
{
  "OfferGroupInfo": [
    {
      "///": "报盘类型",
      "OfferType": 2,
      "///": "报盘组库名",
      "LibName": "libdemo_offer.so",
      "///": "业务头文件版本(offer_biz.h)",
      "BizVersion": 1,
      "///": "报盘组库版本",
      "Version": "demo_offer V1.0.1.1",
      "///": "OnMessgae调用次数",
      "OnMsgExecNum": 0,
      "///": "OnPluginLpc调用次数",
      "OnLpcExecNum": 0,
    }
  ]
}
```

6.11.6. GetPluginRole 获取报盘框架插件角色

功能名称	GetPluginRole	版本号	V1.0.0.0	更新日期	20220516
功能备注	报盘框架插件角色				
功能说明	获取报盘框架插件角色				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetPluginRole				
入参说明					
无					
出参说明					
<pre>{ "///": "报盘框架插件角色, OFFER: 报盘; PROXY: 代理; OFFER & PROXY: 报盘和代理", "PluginRole": "OFFER" }</pre>					

6.11.7. GetOfferBaseInfo 获取报盘对象信息

功能名称	GetOfferBaseInfo	版本号	V1.0.0.0	更新日期	20220516
功能备注	报盘对象信息				
功能说明	获取报盘对象信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferBaseInfo				
入参说明					
无					
出参说明					

```
{
  "OfferBaseInfo": [
    {
      "///": "报盘序号",
      "OfferIndex": 1,
      "///": "报盘名",
      "OfferName": "OfferABC",
      "///": "报盘类型",
      "OfferType": 1,
      "///": "交易请求是否入工作线程",
      "UseThread": true,
      "///": "报盘是否已启动",
      "Started": true,
      "///": "OnMessgae调用次数",
      "OnMsgExecNum": 0,
      "///": "OnPluginIpc调用次数",
      "OnLpcExecNum": 0,
      "///": "MSG消息处理个数",
      "TotalMsgExecNum": 0,
      "///": "MSG消息, 最后一笔业务处理耗时, 单位: 纳秒",
      "LastMsgBizExecTimeNs": 0,
      "///": "MSG消息, 最大业务处理耗时, 单位: 纳秒",
      "MaxMsgBizExecTimeNs": 0,
      "///": "MSG消息, 最小业务处理耗时, 单位: 纳秒",
      "MinMsgBizExecTimeNs": 0,
      "///": "MSG消息, 平均业务处理耗时, 单位: 纳秒",
      "AveMsgBizExecTimeNs": 0,
      "///": "MSG消息, 最后一笔业务处理耗时 (加锁前打点), 单位: 纳秒",
      "LastMsgLockBizExecTimeNs": 0,
      "///": "MSG消息, 最大业务处理耗时 (加锁前打点), 单位: 纳秒",
      "MaxMsgLockBizExecTimeNs": 0,
      "///": "MSG消息, 最小业务处理耗时 (加锁前打点), 单位: 纳秒",
      "MinMsgLockBizExecTimeNs": 0,
      "///": "MSG消息, 平均业务处理耗时 (加锁前打点), 单位: 纳秒",
      "AveMsgLockBizExecTimeNs": 0,
      "///": "MSG消息, 最后一笔平台处理耗时, 单位: 纳秒",
      "LastMsgFrameExecTimeNs": 0,
      "///": "MSG消息, 最大平台处理耗时, 单位: 纳秒",
      "MaxMsgFrameExecTimeNs": 0,
      "///": "MSG消息, 最小平台处理耗时, 单位: 纳秒",
      "MinMsgFrameExecTimeNs": 0,
      "///": "MSG消息, 平均平台处理耗时, 单位: 纳秒",
      "AveMsgFrameExecTimeNs": 0,
      "///": "MSG消息, 最后一笔总体耗时, 单位: 纳秒",
      "LastMsgTotalExecTimeNs": 0,
      "///": "MSG消息, 最大总体耗时, 单位: 纳秒",
      "MaxMsgTotalExecTimeNs": 0,
      "///": "MSG消息, 最小总体耗时, 单位: 纳秒",
      "MinMsgTotalExecTimeNs": 0,
      "///": "MSG消息, 平均总体耗时, 单位: 纳秒",
      "AveMsgTotalExecTimeNs": 0
    }
  ]
}
```

6.11.8. GetOfferRegisterInfo 获取报盘注册信息

功能名称	GetOfferRegisterInfo	版本号	V1.0.0.0	更新日期	20220516
功能备注	报盘注册信息				
功能说明	获取报盘注册信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferRegisterInfo				
入参说明					
无					
出参说明					
<pre>{ "OfferRegisterInfo": [{ "///": "报盘序号", "OfferIndex": 1, "///": "报盘名", "OfferName": "OfferABC", "///": "交易组播分区号", "PartitionNo": 1, "///": "交易组播主题", "TopicName": "report", "///": "是否为本地报盘", "IsLocal": true, "///": "报盘是否连接", "IsConnected": true }] }</pre>					

6.11.9. GetOfferTimerInfo 获取报盘定时器信息

功能名称	GetOfferTimerInfo	版本号	V1.0.0.0	更新日期	20220516
功能备注	报盘定时器信息				
功能说明	获取报盘定时器信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferTimerInfo				
入参说明					
无					
出参说明					

```

{
    "OfferTimerInfo": [
        {
            "/*": "报盘序号",
            "OfferIndex": 1,
            "/*": "报盘名",
            "OfferName": "OfferABC",
            "/*": "定时间隔, 单位: 毫秒",
            "TimerMilli": 5000,
            "/*": "定时执行开始时间, 单位: 毫秒",
            "StartTime": 1228377552,
            "/*": "功能号",
            "FunctionNo": 0,
            "/*": "定时任务类型, RegTimer: 源于RegTimer接口注册; PostMsg: 源于PostMsg接口注册",
            "TimerType": "RegTimer",
            "/*": "是否为周期任务",
            "Periodicity": true,
            "/*": "用户自定义数据地址",
            "UserAddr": 0
        }
    ]
}

```

6.11.10. GetOfferStatisticsInfo 获取报盘业务统计信息

功能名称	GetOfferStatisticsInfo	版本号	V1.0.0.60	更新日期	20230103
功能备注	报盘业务统计信息				
功能说明	获取报盘业务统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_offerproc&FuncName=GetOfferStatisticsInfo				

入参说明

无

出参说明

```

{
    "OfferStatisticsInfo": [
        {
            "/*": "报盘序号",
            "OfferIndex": 1,
            "/*": "报盘名",
            "OfferName": "OfferABC",
            "/*": "报盘类型",
            "OfferType": 1,
            "/*": "报盘状态, None: 无状态; Start: 报盘启动; Offline: 报盘不在线; Online: 报盘在线; Open: 开市; Halt: 暂停开市; Close: 闭市",
            "OfferStatus": "None",
            "/*": "交易所状态",
            "ExchangeStatus": 0,
            "/*": "申报个数",
            "ReportNum": 1,
            "/*": "申报成功个数",
            "ReportSucceedNum": 1,
            "/*": "申报失败个数",
            "ReportFailedNum": 0,
            "/*": "重复申报个数",
            "RepeatNum": 1,
            "/*": "确认个数",
            "ConfirmNum": 1,
            "/*": "确认回报成功个数",
            "ConfirmSucceedNum": 1,
            "/*": "确认回报失败个数",
            "ConfirmFailedNum": 0,
            "/*": "成交个数",
            "DealNum": 1,
            "/*": "成交回报成功个数",
            "DealSucceedNum": 1,
            "/*": "成交回报失败个数",
            "DealFailedNum": 0,
            "SeatsStatisticsInfo": [
                {
                    "/*": "席位号",
                    "SeatId": "1",
                    "/*": "平台类型",
                    "PlatformType": "现货几种竞价交易",
                    "/*": "申报个数",
                    "ReportNum": 1,
                    "/*": "申报成功个数",
                    "ReportSucceedNum": 1,
                    "/*": "申报失败个数",
                    "ReportFailedNum": 0,
                    "/*": "重复申报个数",
                    "RepeatNum": 1,
                    "/*": "确认个数",
                    "ConfirmNum": 1,
                    "/*": "确认回报成功个数",
                    "ConfirmSucceedNum": 1,
                    "/*": "确认回报失败个数",
                    "ConfirmFailedNum": 0,
                    "/*": "成交个数",
                    "DealNum": 1,
                    "/*": "成交回报成功个数",
                    "DealSucceedNum": 1,
                    "/*": "成交回报失败个数",
                    "DealFailedNum": 0,
                }
            ]
        }
    ]
}

```

6.11.11. GetOfferQueueInfo 获取工作线程队列信息

功能名称	GetOfferQueueInfo	版本号	V1.0.0.0	更新日期	20220516
------	-------------------	-----	----------	------	----------

功能备注	工作线程队列信息
功能说明	获取工作线程队列信息
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferQueueInfo
入参说明	
无	
出参说明	
<pre>{ "OfferQueueInfo": [{ "/*": "报盘序号", "OfferIndex": 1, "/*": "报盘名", "OfferName": "OfferABC", "/*": "报盘类型", "OfferType": 1, "/*": "报盘是否已启动", "Started": true, "/*": "队列请求个数", "QueueCount": 0, "/*": "队列使用内存", "InUseSizeKB": 0 }] }</pre>	

6.11.12. GetOfferQueueReqs 获取工作线程队列请求

功能名称	GetOfferQueueReqs	版本号	V1.0.0.0	更新日期	20220516
功能备注	工作线程队列请求				
功能说明	获取工作线程队列请求				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferQueueReqs				
入参说明					
<pre>{ "/*": "查询个数，默认值100", "QueryCount": 100 }</pre>					
出参说明					
<pre>{ "OfferQueueReqs": [{ "/*": "报盘序号", "OfferIndex": 1, "/*": "报盘名", "OfferName": "OfferABC", "/*": "报盘类型", "OfferType": 1, "/*": "报盘是否已启动", "Started": true, "/*": "队列请求个数", "QueueCount": 3, "/*": "请求", "Requests": [{ "/*": "功能号", "FunctionNo": 0, "/*": "请求类型, TIMERSG: 定时请求; TRADEMSG: 交易请求; POSTMSG: 异步请求; FRAMEMSG: 框架请求", "ReqType": "TIMERSG" }, { "FunctionNo": 2, "ReqType": "TRADEMSG" }, { "FunctionNo": 3, "ReqType": "POSTMSG" }] }] }</pre>					

6.11.13. GetOfferPushServiceInfo 获取主推服务信息

功能名称	GetOfferPushServiceInfo	版本号	V1.0.0.0	更新日期	20220516
功能备注	主推服务信息				
功能说明	获取主推服务信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferPushServiceInfo				
入参说明					
无					
出参说明					

```
{
  "OfferPushServiceInfo": [
    {
      "///": "主推服务别名",
      "Name": "PushCore",
      "///": "主推服务类型, PLUGIN: 插件主推; RCM: 组播主推",
      "Type": "RCM",
      "///": "组播主题",
      "Topic": "return",
      "///": "组播主题分区",
      "PartitionNo": 1,
      "///": "插件名",
      "PluginName": "",
      "///": "实例编号",
      "Instance": 0,
      "///": "主推总次数",
      "PushCount": 5,
      "///": "主推失败次数",
      "PushFailCount": 0
    }
  ]
}
```

6.11.14. GetOfferRegFailedInfo 获取报盘注册失败信息

功能名称	GetOfferRegFailedInfo	版本号	V1.0.0.0	更新日期	20220516
功能备注	报盘注册失败信息				
功能说明	获取报盘注册失败信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferRegFailedInfo				
入参说明					
无					
出参说明					
<pre>{ "RegisterFailedCount": 0 }</pre>					

6.11.15. GetOfferRcmProxyConfigStat 获取报盘Rcm代理配置信息

功能名称	GetOfferRcmProxyConfigStat	版本号	V1.0.0.16	更新日期	20220516
功能备注	报盘Rcm代理配置信息				
功能说明	获取报盘Rcm代理配置信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_offerproc&FuncName=GetOfferRcmProxyConfigStat				
入参说明					
无					
出参说明					

```
{
  "/*": "字段描述详见rcm代理开发手册",
  "OfferRcmProxyConfigStat": [
    {
      "offer#0_ldp_offerproc_0_RcmProxy_Config": {
        "LocalNetworkNumber": "192.168.86.101",
        "ProtocolMode": "UDP-MULTICAST",
        "RspTopic": {},
        "Tier": {
          "Ref": "Tier",
          "TierName": "ldp_unittest_wangsong_offer_as_offer_ldp_offerproc_0_1",
          "ContextName": "offer#0_ldp_offerproc_0_Tier",
          "ContextId": 4,
          "TxTopics": [],
          "RxTopics": [
            {
              "Name": "manager",
              "PartitionNos": [
                {
                  "PartitionNo": 1,
                  "TransportId": 8
                }
              ]
            }
          ]
        }
      }
    },
    {
      "offer#0_ldp_offerproc_0_RcmProxy_Config": {
        "LocalNetworkNumber": "192.168.86.101",
        "ProtocolMode": "UDP-MULTICAST",
        "RspTopic": {
          "Topic": "response",
          "PartitionNo": 1,
          "TransportId": 10
        },
        "Singleton": {
          "Ref": "OfferDefault",
          "ContextName": "offer#0_ldp_offerproc_0_Singleton",
          "ContextId": 5,
          "TxTopics": [
            {
              "Name": "return",
              "AliaName": "",
              "PartitionNos": []
            }
          ],
          "RxTopics": [
            {
              "Name": "report",
              "PartitionNos": [
                {
                  "PartitionNo": 1,
                  "TransportId": 9
                }
              ]
            }
          ]
        }
      }
    }
  ]
}
```

6.11.16. GetOfferRcmProxyStatistics 获取报盘Rcm代理收发包统计信息

功能名称	GetOfferRcmProxyStatistics	版本号	V1.0.0.16	更新日期	20220516
功能备注	报盘Rcm代理收发包统计信息				
功能说明	获取报盘Rcm代理收发包统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_offerproc&FuncName=GetOfferRcmProxyStatistics				
入参说明					
无					
出参说明					

```
{
  "/*": "字段描述详见rcm代理开发手册",
  "OfferRcmProxyStatistics": [
    {
      "offer#0_ldp_offerproc_0_RcmProxy_Statistics": {
        "TierRecvInvalidCount": 0,
        "Initiator": [],
        "Acceptor": {
          "SendSuccessCount": 0,
          "SendFailCount": 0,
          "RecvCount": 3
        },
        "Publisher": []
      }
    },
    {
      "offer#0_ldp_offerproc_0_RcmProxy_Statistics": {
        "SingletonRecvInvalidCount": 0,
        "Initiator": [
          {
            "name": "return",
            "AliaName": "",
            "IsTier": false,
            "IsConnected": true,
            "SendSuccessCount": 6,
            "SendFailCount": 0,
            "RecvCount": 0
          }
        ],
        "Acceptor": {
          "SendSuccessCount": 0,
          "SendFailCount": 0,
          "RecvCount": 1
        },
        "Publisher": []
      }
    }
  ]
}
```

6.11.17. GetOfferRcmProxyFull duplexStat 获取报盘Rcm代理全双工通道状态

功能名称	GetOfferRcmProxyFull duplexStat	版本号	V1.0.0.16	更新日期	20220516
功能备注	报盘Rcm代理全双工通道状态				
功能说明	获取报盘Rcm代理全双工通道状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_offerproc&FuncName=GetOfferRcmProxyFull duplexStat				
入参说明					
无					
出参说明					
<pre>{ "/*": "字段描述详见rcm代理开发手册", "OfferRcmProxyFull duplexStat": [{ "offer#0_ldp_offerproc_0_RcmProxy_Full duplex": [] }, { "offer#0_ldp_offerproc_0_RcmProxy_Full duplex": [{ "TxTopic": "return", "TxPartitions": "", "RxTopic": "response", "RxPartition": "1", "IsConnected": true, "ConnectionInfo": [{ "PeerContext": "bizproc_xst#0_ldp_bizproc_0_HighReliable_Singleton", "UpPartition": 1, "UplinkLocalStat": "Ready", "UplinkLPeerStat": "Ready", "DownlinkLocalStat": "Ready", "DownlinkPeerStat": "Ready" }] }] }]}</pre>					

6.11.18. GetOfferRpcStatistics 获取报盘Rpc统计信息

功能名称	GetOfferRpcStatistics	版本号	V1.0.0.16	更新日期	20220516
功能备注	报盘Rpc统计信息				
功能说明	获取报盘Rpc统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_offerproc&FuncName=GetOfferRpcStatistics				
入参说明					
无					
出参说明					


```
{
  "/*": "字段描述详见ldprc开发指南",
  "OfferRpcStatistics": {
    "/*": "下一个rpc客户端Id",
    "NextClientId": 1,
    "/*": "rpc客户端已使用数量",
    "RpcClientUsedCount": 0,
    "/*": "rpc客户端总数量",
    "RpcClientTotalCount": 1,
    "RpcClients": [
      {
        "RpcClientId": 0,
        "LastPacketId": 1,
        "FirstPacketId": 16776191,
        "StartPacketId": 1,
        "ReqCount": 0,
        "RspCount": 0,
        "IsWaiting": false,
        "TotalReqCount": 1,
        "TotalRspCount": 1,
        "TotalTimeoutRspCount": 0
      }
    ],
    "RouteMissCount": 0,
    "Routes": [
      {
        "Functions": "454",
        "SystemNo": "*",
        "NodeNo": "*",
        "BackendID": 1,
        "Count": 0
      },
      {
        "Functions": "*",
        "SystemNo": "*",
        "NodeNo": "*",
        "BackendID": 2,
        "Count": 1
      }
    ],
    "Backends": [
      {
        "BackendId": 1,
        "RecvCount": 0,
        "SendSuccessCount": 0,
        "SendFailCount": 0,
        "InterceptCount": 0,
        "ErrorRspCount": 0,
        "Type": "PLUGIN",
        "ConfigName": "PLUGIN-1",
        "ConfigInst": 0,
        "ConfigPlugin": "ldp_bizproc",
        "PluginName": "ldp_bizproc",
        "PluginInst": 0,
        "PluginState": 4
      },
      {
        "BackendId": 2,
        "RecvCount": 1,
        "SendSuccessCount": 1,
        "SendFailCount": 0,
        "InterceptCount": 0,
        "ErrorRspCount": 0,
        "Type": "PLUGIN",
        "ConfigName": "PLUGIN-2",
        "ConfigInst": 0,
        "ConfigPlugin": "ldp_bizproc",
        "PluginName": "ldp_bizproc",
        "PluginInst": 0,
        "PluginState": 4
      }
    ],
    "RcmCompaction": {
      "RcmCompRecvCount": 0,
      "RcmCompRecvErrCount": 0
    },
    "Watches": []
  }
}
```

6.11.19. NotifyOfferStatusRecovery 紧急恢复报盘状态

功能名称	NotifyOfferStatusRecovery	版本号	V1.0.0.20	更新日期	20220516
功能备注	紧急恢复报盘状态				
功能说明	紧急恢复报盘状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_offerproc&FuncName=NotifyOfferStatusRecovery				
入参说明					
无					
出参说明					
<pre>{ "/*": "错误码, 0表示恢复正常, 否则失败", "ErrorNo": 0, "/*": "错误信息", "ErrorMsg": "" }</pre>					

6.11.20. GetOfferStatusInfoInFile 获取落地文件中的报盘状态

功能名称	GetOfferStatusInfoInFile	版本号	V1.0.0.20	更新日期	20220516

功能备注	落地文件中的报盘状态
功能说明	获取落地文件中的报盘状态
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_offerproc&FuncName=GetOfferStatusInfoInFile
入参说明	
无	
出参说明	
<pre>{ "OfferStatusInfoInFile": [{ "///": "报盘序号", "OfferIndex": 5, "///": "报盘名", "OfferName": "offer00005", "///": "报盘类型", "OfferType": 1, "///": "交易请求是否入工作线程", "UseThread": false }, { "OfferIndex": 6, "OfferName": "offer00006", "OfferType": 1, "UseThread": false }] }</pre>	

6.11.21. GetStatusQueryInfo 获取报盘状态查询信息

功能名称	GetStatusQueryInfo	版本号	V1.0.0.20	更新日期	20220516
功能备注	报盘状态查询信息				
功能说明	获取报盘状态查询信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_offerproc&FuncName=GetStatusQueryInfo				
入参说明					
无					
出参说明					
<pre>{ "///": "是否向管理系统查询报盘状态", "IsQuery": true, "///": "是否查询完成", "IsQueryFinish": true }</pre>					

6.12. clean插件 ldp_clean

6.12.1. GetCleanStat 获取清理状态

功能名称	GetCleanStat	版本号	V1.0.0.0	更新日期	20210811
功能备注	清理状态				
功能说明	获取清理状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_clean&FuncName=GetCleanStat				
入参说明					
<pre>{ "///": "待查询的组", "GroupIndex": 2 }</pre>					
出参说明					

```
{
  "GroupIndex": 2,
  "///<": "轮次号",
  "RoundIndex": 1628590746088646,
  "///<": "当前清理阶段",
  "GroupCleanPhase": "Success",
  "///<": "本组清理次数",
  "GroupCleanCount": 7,
  "Node": [
    {
      "///<": "节点名",
      "NodeName": "xst#0_ldp_bizproc",
      "///<": "轮次号",
      "RoundIndex": 1628590746088646,
      "///<": "本节点当前清理状态",
      "CleanStat": "Success",
      "///<": "本节点清理次数",
      "CleanCount": 7
    },
    {
      "NodeName": "xst#0_ldp_todb",
      "RoundIndex": 1628590746088646,
      "CleanStat": "Success",
      "CleanCount": 7
    },
    {
      "NodeName": "xst#1_ldp_bizproc",
      "RoundIndex": 1628590746088646,
      "CleanStat": "Success",
      "CleanCount": 7
    },
    {
      "NodeName": "xst#1_ldp_todb",
      "RoundIndex": 1628590746088646,
      "CleanStat": "Success",
      "CleanCount": 7
    }
  ]
}
```

6.12.2. RequestClean 启动清理

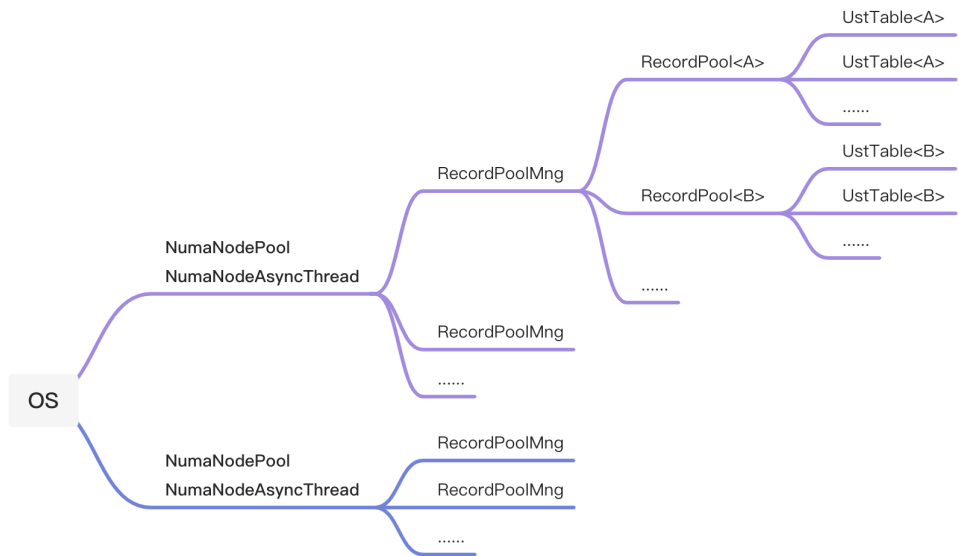
功能名称	RequestClean	版本号	V1.0.0.0	更新日期	20210811
功能备注	启动清理				
功能说明	启动清理				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_clean&FuncName=RequestClean				
入参说明					
<pre>{ "///": "待启动的组", "GroupIndex": 2 }</pre>					
出参说明					
<pre>{ "Resp Msg": "组 2 清理流程启动" }</pre>					

6.12.3. TerminateClean 清理中断

功能名称	TerminateClean	版本号	V1.0.0.0	更新日期	20210811
功能备注	清理中断				
功能说明	清理中断				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_clean&FuncName=TerminateClean				
入参说明					
<pre>{ "///": "待中断的组", "GroupIndex": 2 }</pre>					
出参说明					
<pre>{ "Rsp Msg": "组 2 清理被终止" }</pre>					

6.13. 记录池插件 ldp_recordpool

为了更好的理解记录池插件的管理功能，先简单介绍一下记录池插件的架构及内部几个主要模块。



如图所示，记录池插件主要分4个模块

- node
- mng
- pool
- table

node---1:N---mng---1:N---pool---1:N---table

左边模块相对于右边模块来说是上级模块, 1个上级模块可以有1个或多个下级模块。

使用内存时，是由下级逐级向上申请的，比如：

- table需要记录时，会向pool申请；
- pool中内存用完时，向mng申请；
- mng内存用完时，向node申请；
- node内存用完时，向操作系统申请。

使用管理功能时，返回的是json格式的数据。当需要返回的数据量较大时，会提示“缓存不足”，此时，需要使用管理功能“ChangeMngBufSize”调整管理功能使用的缓存的大小，避免出现错误。

特别的，当某个管理功能需要输入密码操作时，说明该功能不是一个常用的功能，或者是一个很消耗资源的功能，必须要谨慎使用。

6.13.1. GetNodesStatSummary

功能名称	GetNodesStatSummary	版本号	V1.0.0.2	更新日期	20211217
功能备注					
功能说明	查看node概要信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=GetNodesStatSummary				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ "NodesStatSummary": [{ "node": 0, // NUMA节点号 "pool_mng_num": 6, // 下属mng数量 "total": 2147491744, // 内存总量 "used": 1879048528 // 已分配量 }] }</pre>					

6.13.2. GetAndDumpNodesStatDetail

功能名称	GetAndDumpNodesStatDetail	版本号	V1.0.0.2	更新日期	20211217
功能备注					
功能说明	查看node详细信息, 并导出到json文件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=GetAndDumpNodesStatDetail				
入参说明					

Pretty	1-美化json输出,0-不美化
出参说明	
<pre>{ "NodesStatDetail": [{ "this": "0x27acdf0", // this指针 "node": 0, // NUMA节点号 "tid": "7f798405f380", // 线程号 "cpu": 6, // 创建此数据的cpu号 "huge": false, // 是否使用大页内存 "wait type": "condition variable", // 异步扩容线程等待唤醒方式 "bind_cpu": 7, // 绑核号 "step": 1073741824, // 扩容步长 "refill": "7f7974481700", // 异步扩容线程号 "pool_mngs": [// 下属mng列表 "0x27ad2f0", // mng线程号, 可以认为是mng的标识 "0x7f7910019e10", "0x7f790c019e40", "0x7f7914019e40", "0x7f7904019df0", "0x7f792c092b70"], "mem_tail": { // 内存节点 "detail": [// 详细信息 { "this": "0x7f78a3ffe000", // 本节点指针 "prev": "0x7f7931261000", // 前向节点指针 "next": "(nil)", // 后向节点指针 "size": 1073745872, // 总量 "ctor": 0, // 已构造量 "len": 805306512, // 已用量 "alloc": 0 // 分配器编号 }], "summary": { // 概要信息 "size": 2147491744, "ctor": 0, "len": 1879048528 } }, "acquire": [// 下属请求内存的信息 { "tid": "7f79225fd700", // 请求者的线程号 "succ": { // 成功信息 "cnt": 1, // 次数 "size": 0, // 内存量 "prefault": 268435504 // 预热过的内存量 }, "fail": { // 失败信息 "cnt": 0, "size": 0, "prefault": 0 } }], "alloc": [// 本节点申请内存的信息 { "tid": "7f7920dfa700", // 引发申请的下属线程号 "succ": { // 成功信息 "cnt": 1, // 次数 "size": 0, // 内存量 "prefault": 1073741824 // 需预热的内存量 }, "fail": { // 失败信息 "cnt": 0, "size": 0, "prefault": 0 } }] }] }</pre>	

6.13.3. GetPoolMngsStatSummary

功能名称	GetPoolMngsStatSummary	版本号	V1.0.0.2	更新日期	20211217
功能备注					
功能说明	查看mng概要信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=GetPoolMngsStatSummary				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					

```
{
  "PoolMngsStatSummary": [
    {
      "tid": "7f798405f380",          // 线程号
      "pool_num": 55,                // 池数量
      "pools": [                    // 池名列表
        "13CFensCfgField"           // 池名
      ],
      "refill": [                    // 异步扩容信息
        {
          "name": "16CTransparamField", // 异步扩容池名
          "cnt": 1                      // 异步扩容次数
        }
      ],
      "total": 805306368,             // 可用内存量
      "used": 739857296               // 已用内存量
    }
  ]
}
```

6.13.4. DumpPoolMngsStatDetail

功能名称	DumpPoolMngsStatDetail	版本号	V1.0.0.2	更新日期	20211217
功能备注	需要密码访问,数据量较大,谨慎使用!				
功能说明	导出mng详细信息到json文件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=DumpPoolMngsStatDetail&password=WrongPassword				
入参说明					
Pretty	1-美化json输出,0-不美化				
password	密码				
出参说明					
<pre>{ "ErrorNo": "ErrorNo非0表示错误" "ErrorNo": 0, "ErrorMsg": "success" }</pre>					

6.13.5. DumpTablesStatDetail

功能名称	DumpTablesStatDetail	版本号	V1.0.0.2	更新日期	20211217
功能备注	需要密码访问,数据量特别巨大(可能是几十,上百MB,甚至会有几GB),极有可能导致程序停止响应,慎用!				
功能说明	导出table详细信息到json文件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=DumpTablesStatDetail&password=WrongPassword&verbose=0				
入参说明					
Pretty	1-美化json输出, 0-不美化				
password	密码				
verbose	1-空表也导出 数据量极大,可能会到GB级别,其他值-忽略空表				
出参说明					
参照DumpPoolMngsStatDetail					

6.13.6. GetPoolsStatSummaryByMng

功能名称	GetPoolsStatSummaryByMng	版本号	V1.0.0.2	更新日期	20211217
功能备注					
功能说明	查看指定mng下pool的概要信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=GetPoolsStatSummaryByMng&mng_tid=7f82d6357380				
入参说明					
Pretty	1-美化json输出, 0-不美化				
mng_tid	mng标识(通过GetPoolMngsStatSummary获取)				
出参说明					

将注释转换为字段上方的注释字符串格式，如下示例：

```
{
  "PoolsStatSummaryByMng": [
    [
      {
        "/*": "池号",
        "id": 35,
        "/*": "池名",
        "name": "9CFFarelog",
        "/*": "记录大小",
        "size": 32,
        "/*": "记录对其",
        "align": 8,
        "/*": "申请内存步长",
        "step": 16384,
        "/*": "异步扩容步长",
        "async_step": 2048,
        "/*": "池的初始大小(取插件的配置值或者通过CUSTTable2接口设置)",
        "pool_min_sz": 64,
        "/*": "pool_min_sz值的来源.true-通过CUSTTable2接口设置,false-取插件的配置值",
        "cust_pool_min_sz": false,
        "/*": "async_step值的来源.true-通过CUSTTable2接口设置,false-取插件的配置值",
        "cust_async_step": false,
        "/*": "可用且已构造内存量",
        "ctor": 202752,
        "/*": "槽位节点",
        "slot": {
          "total": 131072,
          "used": 2048
        },
        "/*": "普通内存节点",
        "tail": {
          "/*": "总量",
          "total": 524288,
          "/*": "已构造量",
          "ctor": 524288,
          "/*": "已用量",
          "used": 321536
        }
      },
      {
        "/*": "池号",
        "id": 36,
        "/*": "池名",
        "name": "P12CBfare2Field",
        "/*": "记录大小",
        "size": 8,
        "/*": "记录对其",
        "align": 8,
        "/*": "申请内存步长",
        "step": 1024,
        "/*": "异步扩容步长",
        "async_step": 2048,
        "/*": "池的初始大小(取插件的配置值或者通过CUSTTable2接口设置)",
        "pool_min_sz": 64,
        "/*": "pool_min_sz值的来源.true-通过CUSTTable2接口设置,false-取插件的配置值",
        "cust_pool_min_sz": false,
        "/*": "async_step值的来源.true-通过CUSTTable2接口设置,false-取插件的配置值",
        "cust_async_step": false,
        "/*": "可用且已构造内存量",
        "ctor": 16384,
        "/*": "普通内存节点",
        "tail": {
          "/*": "总量",
          "total": 24576,
          "/*": "已构造量",
          "ctor": 24576,
          "/*": "已用量",
          "used": 8192
        },
        "/*": "快速分配节点",
        "fast": {
          "/*": "总量",
          "total": 1597440,
          "/*": "已构造量",
          "ctor": 0,
          "/*": "已用量",
          "used": 1591808
        }
      }
    ]
  ]
}
```

6.13.7. GetPoolStatDetailByMngAndId

功能名称	GetPoolStatDetailByMngAndId	版本号	V1.0.0.2	更新日期	20211217
功能备注					
功能说明	查看指定pool的详细信息				
访问链接	http://ppport/ldp?Pretty=1&PluginName=ldp_recordpool&FuncName=GetPoolStatDetailByMngAndId&mng_tid=7f82d6357380&pool_id=0				
入参说明					
Pretty	1-美化json输出, 0-不美化				
mng_tid	mng标识(通过GetPoolMngsStatSummary获取)				
pool_id	pool标识(通过GetPoolsStatSummaryByMng获取)				
出参说明					
将注释转换为字段上方的注释字符串格式，如下示例：					

```

{
  "PoolStatDetailByMngAndId": [
    {
      "//": "this指针",
      "this": "0x27af0f0",
      "//": "创建该结构的线程号",
      "tid": "7f798405f380",
      "//": "创建该结构的cpu",
      "cpu": 6,
      "//": "池名",
      "name": "11CSeatsField",
      "//": "池号",
      "id": 5,
      "//": "记录大小",
      "size": 32,
      "//": "记录对其",
      "align": 4,
      "//": "申请内存的步长",
      "step": 1024,
      "//": "异步扩容的步长",
      "async_step": 2048,
      "//": "池的初始大小(取插件的配置值或者通过CUSTTable2接口设置)",
      "pool_min_sz": 64,
      "//": "pool_min_sz值的来源.true-通过CUSTTable2接口设置,false-取插件的配置值",
      "cust_pool_min_sz": false,
      "//": "async_step值的来源.true-通过CUSTTable2接口设置,false-取插件的配置值",
      "cust_async_step": false,
      "//": "初始化标志",
      "init": "true",
      "//": "所属mng指针",
      "mng": "0x27ad2f0",
      "//": "可用且已构造内存量",
      "ctor": 114688,
      "//": "当前使用的内存节点指针",
      "current": "0x7f7944aeb0c8",
      "//": "普通内存结点",
      "mem_tail": {
        "//": "详细信息",
        "detail": [
          {
            "//": "本节点指针",
            "this": "0x7f79320c3328",
            "//": "前向节点指针",
            "prev": "(nil)",
            "//": "后向节点指针",
            "next": "0x7f7943fafd48",
            "//": "总大小",
            "size": 32768,
            "//": "已构造大小",
            "ctor": 32768,
            "//": "已使用大小",
            "len": 32768,
            "//": "分配器编号",
            "alloc": 254
          }
        ],
        "//": "概要信息",
        "summary": {
          "//": "总大小",
          "size": 950272,
          "//": "已构造大小",
          "ctor": 950272,
          "//": "已使用大小",
          "len": 835584
        }
      },
      "//": "槽位节点",
      "slot_tail": {
        "detail": [
          {
            "//": "本节点指针",
            "this": "0x7f79320c12f8",
            "//": "前向节点指针",
            "prev": "(nil)",
            "//": "后向节点指针",
            "next": "(nil)",
            "//": "总大小",
            "size": 8192,
            "//": "已构造大小",
            "ctor": 0,
            "//": "已使用大小",
            "len": 5120,
            "//": "分配器编号",
            "alloc": 254
          }
        ],
        "summary": {
          "//": "总大小",
          "size": 8192,
          "//": "已构造大小",
          "ctor": 0,
          "//": "已使用大小",
          "len": 5120
        }
      },
      "//": "快速分配节点",
      "fast_tail": {
        "detail": [
          {
            "//": "本节点指针",
            "this": "0x7f7943f67cb8",
            "//": "前向节点指针",
            "prev": "0x7f7943f5fc88",
            "//": "后向节点指针",
            "next": "(nil)",
            "//": "总大小",
            "size": 32768,
            "//": "已构造大小",
            "ctor": 0,
            "//": "已使用大小",
            "len": 8192,
            "//": "分配器编号",
            "alloc": 254
          }
        ],
        "summary": {

```



```
        "len": 401408
    }
}
}
}
}
```

6.13.8. GetTablesStatSummary

功能名称	GetTablesStatSummary	版本号	V1.0.0.3	更新日期	20221013
功能备注					
功能说明	查看表的整体信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=GetTablesStatSummary				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
将注释转换为字段上方的注释字符串格式，如下示例：					
<pre>{ "TablesStatSummary": { "/*": "数组，按线程号罗列", "threads": [{ "/*": "线程号，16进制表示的pthread_t", "tid": "7f8ba77fe700", "/*": "表总数量", "tbl_total": 17392384, "/*": "非空表的数量", "tbl_nonempty": 0, "/*": "空表的数量", "tbl_empty": 17392384, "/*": "所有记录数 = 未使用 + 已使用", "rcd_total": 0, "/*": "已使用记录数", "rcd_used": 0, "/*": "所有记录数占用内存", "mem_total": 0, "/*": "已用记录数占用的内存", "mem_used": 0, "/*": "总内存 = 所有表对象自身占用内存 + 所有记录数占用内存", "tbl_mem_sz": 3617615872, "/*": "数组，按表名罗列", "tables": [{ "/*": "表名，P开头表示指针表，否则就是普通表，数字部分请忽略，剩余部分为结构体的名字", "name": "P8CEntrust", "/*": "表总数量", "total": 3652160, "/*": "空表的数量", "empty": 3180150, "/*": "非空表的数量", "nonempty": 472010, "/*": "所有表信息", "/*": "所有记录数 = 未使用 + 已使用", "rcd_total": 7761696, "/*": "已使用记录数", "rcd_used": 3071766, "/*": "总内存 = 所有表对象自身占用内存 + 所有记录数占用内存", "mem_total": 920664448, "/*": "所有记录数占用内存", "mem_rcd": 62093568, "/*": "所有表对象自身占用内存", "mem_tbl_obj": 858570880, "/*": "非空表信息，字段含义同上", "nonempty_rcd_total": 7761696, "nonempty_rcd_used": 3071766, "nonempty_mem_total": 278274148, "nonempty_mem_rcd": 62093568, "nonempty_mem_tbl_obj": 216180580, "/*": "空表信息，字段含义同上", "empty_rcd_total": 0, "empty_rcd_used": 0, "empty_mem_total": 642390300, "empty_mem_rcd": 0, "empty_mem_tbl_obj": 642390300 }] }] } }</pre>					

6.13.9. GetTableStatSummaryByName

功能名称	GetTableStatSummaryByName	版本号	V1.0.0.2	更新日期	20211217
功能备注					
功能说明	查看某个表的概要信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=GetTableStatSummaryByName&table_name=12COcodeField				
入参说明					

Pretty	1-美化json输出, 0-不美化
table_name	表名(通过GetTablesStatSummary获取)
出参说明	
将注释转换为字段上方的注释字符串格式，如下示例： <pre>{ "TableStatSummaryByName": { "//": "按线程罗列", "threads": [{ "//": "线程号", "tid": "7F7974481700", "//": "表总数", "tbl_total": 8192, "//": "非空表数", "tbl_nonempty": 0, "//": "空表数", "tbl_empty": 8192, "//": "可用记录数", "rcd_total": 0, "//": "已用记录数", "rcd_used": 0, "//": "可用记录数占用的内存量", "mem_total": 0, "//": "已用记录数占用的内存量", "mem_used": 0 }], "//": "表编号", "id": 30, "//": "表名", "name": "8CEntrust", "//": "记录大小", "size": 600, "//": "记录对其", "align": 8 } }</pre>	

6.13.10. ChangeMngBufSize

功能名称	ChangeMngBufSize	版本号	V1.0.0.2	更新日期	20211217
功能备注					
功能说明	调整管理功能使用buf的大小				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=ChangeMngBufSize&size=128MB				
入参说明					
Pretty	1-美化json输出, 0-不美化				
size	正整数数字[单位] (方括号表示该部分可选, 单位有效值: KB, MB, 折算成字节数后, 不得超过2147483647)				
出参说明					
<pre>{ "ErrorNo": 0, "ErrorMsg": "success" }</pre>					

6.13.11. GetRecordNames

功能名称	GetRecordNames	版本号	V1.0.0.3	更新日期	20220225
功能备注					
功能说明	展示已经注册到记录池插件中的业务数据类型				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_recordpool&FuncName=GetRecordNames				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ "RecordNames": [{ "//": "业务数据类型的名字, 含义需参考C++接口typeid().name()", "name": "12OrderRequest", "//": "记录池插件给这个业务数据类型分配的编号, 编号从0开始顺序递增", "id": 2 }, { "name": "4Fool", "id": 1 }, { "name": "9FooTable1", "id": 0 }]}</pre>					

6.14. locate插件 ldp_locate

6.14.1. GetLocateStatistics 获取定位统计信息

功能名称	GetLocateStatistics	版本号	V1.0.0.0	更新日期	20220609
功能备注	获取定位统计信息				
功能说明	获取定位统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_locate&FuncName=GetLocateStatistics				
入参说明					
出参说明					
<pre>{ "LocateStatistics": { "/" : "未命中计数", "MissCount": 0, "Statistics": [{ "/" : "规则名称", "RuleName": "Default", "/" : "默认NodeNo命中次数", "DefHitCount": 0, "Items": [{ "ShardingKeys": "abc600;", "NodeNo": 80, "/" : "命中次数", "HitCount": 1 }] }, "/" : "总命中次数", "TotalHitCount": 1], { "/" : "规则名称", "RuleName": "NewLocate", "/" : "默认NodeNo命中次数", "DefHitCount": 0, "/" : "如果配置了Key为Account", "ShardingKeys" : "Account", "/" : "如果配置了Value为status", "Value" : "status", "Items": [{ "Account": "abc600;", "status": 80, "/" : "命中次数", "HitCount": 1 }] }, "/" : "总命中次数", "TotalHitCount": 1 } }</pre>					

6.14.2. GetRulesInfo 获取定位规则信息

功能名称	GetRulesInfo	版本号	V1.0.0.0	更新日期	20220616
功能备注	获取定位规则信息				
功能说明	获取定位规则信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_locate&FuncName=GetRulesInfo				
入参说明					
出参说明					

```
{
  "LocateRules": [
    {
      "Name": "Default",
      "DefaultNodeNo": 10000,
      "Rules": [
        {
          "ShardingKeys": "abc600;",
          "NodeNo": 80
        },
        {
          "ShardingKeys": "abc500;Abc600;",
          "NodeNo": 90
        }
      ]
    },
    {
      "Name": "Locate",
      "DefaultNodeNo": 10000,
      "ShardingKeys": "ABC",
      "Value": "EFG",
      "Rules": [
        {
          "ABC": "abc600;",
          "EFG": 80
        },
        {
          "ABC": "abc500;Abc600;",
          "EFG": 90
        }
      ]
    }
  ]
}
```

6.14.3. GetLocateStatisticsSum 获取定位统计信息总和

功能名称	GetLocateStatisticsSum	版本号	V1.0.0.0	更新日期	20250915
功能备注	获取定位规则信息				
功能说明	获取定位规则信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_locate&FuncName=GetLocateStatisticsSum				
入参说明					
RuleName	如果入参为空，则返回所有的规则信息，否则只返回指定的规则信息				
<pre>{ "RuleName": "Account2PartitionNo" }</pre>					
出参说明					
Name	规则名				
TotalHitCount	当前所有的命中次数				
<pre>{ "LocateRules": [{ "Name": "Default", "TotalHitCount": 1 }, { "Name": "Account2PartitionNo", "TotalHitCount": 10 }, { "Name": "NodeNoStatus", "TotalHitCount": 51 }, { "Name": "MemoryStatus", "TotalHitCount": 1 }] }</pre>					

6.14.4. Locate 定位

功能名称	Locate	版本号	V1.0.0.0	更新日期	20250915
功能备注	根据Shardingkey定位对应的Value值				
功能说明	根据Shardingkey定位对应的Value值				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_locate&FuncName=Locate				
入参说明					
RuleName	RuleName表示规则名，不存在，则为Default规则。				
ShardingKey	ShardingKey为对应的key值				
<pre>{ "RuleName": "Default", "ShardingKey": "2" }</pre>					

出参说明	
error_no	具体错误号
error_info	具体错误信息
nodeno	具体Value值
<pre>{ "error_no": 0, "error_info": "Locate success", "nodeno": 13 }</pre>	

6.14.5. GetSpecificRuleInfo 定位

功能名称	Locate	版本号	V1.0.0.0	更新日期	20250915
功能备注	根据规则名获取对应的规则信息				
功能说明	根据规则名获取对应的规则信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_locate&FuncName=GetSpecificRuleInfo				
入参说明					
RuleName	表示规则名，不存在则返回所有规则名和规则数量，但不会返回其对应的规则内容；存在则返回指定规则的数量以及对应的规则内容。				
StartIndex	表示规则起始字段，从0开始（默认为0）				
RecordCount	表示返回规则笔数。				
<pre>{ "RuleName": "Default", "StartIndex": 1, "RecordCount": 3}</pre>					
出参说明					
RuleCount	表示当前规则总数。				
StartIndex	表示规则起始字段，从0开始（默认为0）				
RecordCount	表示返回规则笔数。				
<pre>{ "LocateRules": [{ "Name": "Default", "ShardingKeys": "szFundAccount", "Value": "partition_id", "RuleCount": 7, "StartIndex": 1, "RecordCount": 3, "Rules": [{ "szFundAccount": "20-40;", "partition_id": 2 }, { "szFundAccount": "45-60;", "partition_id": 2 }, { "szFundAccount": "61-80;", "partition_id": 3 }] }]}</pre>					

6.15. cres代理插件 ldp_cres_agent

6.15.1. GetStatistics 获取统计信息

功能名称	GetStatistics	版本号	V1.0.0.0	更新日期	20220824
功能备注	获取统计信息				
功能说明	获取统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_cres_agent&FuncName=GetStatistics				
入参说明					
出参说明					

```
{
  "Statistics": {
    "Statistics": {
      "UseCache" : false,
      "LdpToCres": {
        "TimeoutMill": 3000,
        "MsgCacheCount": 0,
        "ReqDest": {
          "Plugin": "ldp_agent",
          "Instance": 0
        },
        "Stat": {
          "InterceptCount": 0,
          "ReqSuccessCount": 0,
          "ReqFailCount": 0,
          "RspSuccessCount": 66919,
          "RspFailCount": 0
        }
      },
      "CresToLdp": {
        "TimeoutMill": 3000,
        "MsgCacheCount": 0,
        "ReqDest": {
          "Plugin": "ldp_bizproc",
          "Instance": 0
        },
        "Stat": {
          "InterceptCount": 0,
          "ReqSuccessCount": 66919,
          "ReqFailCount": 0,
          "RspSuccessCount": 0,
          "RspFailCount": 0
        }
      }
    }
  }
}
```

6.15.2. GetLoadInfo 加载信息

功能名称	GetLoadInfo	版本号	V1.0.0.0	更新日期	20220824
功能备注	加载信息				
功能说明	加载信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_cres_agent&FuncName=GetLoadInfo				

入参说明

出参说明

```
{
  "LoadInfo": {
    "/*": "原始启动参数",
    "OriginalParam": "-start mainsvr -f gw_agent_ar.xml -p 12345678 -s 0 -t ar",
    "/*": "组装后启动参数",
    "ProcessedParam": "hsserver -start mainsvr -f gw_agent_ar.xml -p 12345678 -s 0 -t ar -c"
  }
}
```

6.16. 表分析插件 ldp_table_analyse

6.16.1. GetDatabaseName 获取数据库名称

功能名称	GetDatabaseName	版本号	V1.0.0.0	更新日期	20220919
功能备注	获取数据库名称				
功能说明	获取数据库名称				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=GetDatabaseName				
入参说明					
无					
出参说明					
<pre>{ "Resp": { "//":"功能id", "FunctionID": 403, "//":"消息类型 ", "MsgType": 1, "//":"操作类型", "OperationType": 1, "//":"数据库名称", "DataBasesName": ["crdt_ldpcrdt_liuc"] } }</pre>					

6.16.2. GetTableNames 获取所有表名

功能名称	GetTableNames	版本号	V1.0.0.0	更新日期	20220919
功能备注	获取所有表名				
功能说明	获取所有表名				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=GetTableNames				
入参说明					
无					
出参说明					
<pre>{ "Resp": { "功能id", "FunctionID": 403, "消息类型", "MsgType": 1, "操作类型", "OperationType": 2, "数据库名称", "DataBaseName": "crdt_ldpcrdt_liuc", "表名信息", "TablesInfo": [{ "记录大小, 值为0. 当前不关心", "RecordSize": 0, "表名", "TableName": "BusiEntrustParm" }, { "RecordSize": 0, "TableName": "CTransferEntrust" }] } }</pre>					

6.16.3. GetTableHeader 获取某个表描述信息

功能名称	GetTableHeader	版本号	V1.0.0.0	更新日期	20220919
功能备注	获取某个表描述信息				
功能说明	获取某个表描述信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=GetTableHeader				
入参说明					
<pre>{ "待查看的表名称": "BusiEntrustParm"}</pre>					
出参说明					
<pre>{ "Resp": { "输入参数"BusiEntrustParm"表的描述信息": { "BusiEntrustParm": { "ExchangeType": { "FieldType": "char[5]", "FieldSize": 5, "ReadOnly": true }, "CashMargin": { "FieldType": "char", "FieldSize": 1, "ReadOnly": false } } } }</pre> 如果输入参数错误, 会提示 "ErrorNo" , "ErrorInfo".					

6.16.4. GetWorkThreadInfo 获取工作线程信息

功能名称	GetWorkThreadInfo	版本号	V1.0.0.0	更新日期	20220919
功能备注	获取工作线程信息				
功能说明	获取工作线程信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=GetWorkThreadInfo				
入参说明					
无					
出参说明					

```
{
  "WorkThreadInfo": [
    {
      "/*": "工作线程PID信息",
      "PID": 140184420808448,
      "/*": "工作线程编号",
      "WTNo": 1
    },
    {
      "PID": 140184429201152,
      "WTNo": 0
    }
  ]
}
```

6.16.5. GetTableRecord 查询记录

功能名称	GetTableRecord	版本号	V1.0.7.2	更新日期	20260119
功能备注	查询记录				
功能说明	查询记录				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=GetTableRecord				
入参说明					
<pre>{ "/*": "待查看的表名称", "/*": "通过 GetTableNames 获取所有表名, 得到参数输入信息", "TableName": "BusiEntrustParm", "/*": "记录的数量", "/*": "不配置默认值 0, 显示1条记录信息", "RecordCount": "1", "/*": "不配置默认值 0, 配置 1显示记录总条数, 其他值总条数位置显示-1", "TotalCountEnable": "1"}</pre>					
出参说明					
<pre>{ "Resp": { "FunctionID": 403, "MsgType": 1, "OperationType": 4, "/*": "数据库名", "DataBaseName": "crdt_ldpcrdt_liuc", "/*": "表名", "TableName": "BusiEntrustParm", "/*": "记录数", "RecordCount": 1, "/*": "记录总数", "TotalCount": 10, "TableRecordes": [{ "ExchangeType": "1", "Direction": "88", "OrderCommand": "343", "ExchIndex": "0", "EntrustBs": "1", "EntrustProp": "0", "EntrustType": "9", "CheckDataValidIndex": "0", "CheckArgIndex": "0", "CheckAcctIndex": "0", "CalcBusiParamIndex": "0", "CheckBusiIndex": "0", "UpdateBusiIndex": "0", "OfferTypeIndex": "0", "SeatSerialIndex": "0", "ReturnDealIndex": "0", "CashMargin": "1", "ThreadId": 0, "TableId": 0, "RecordId": 0 }] }}</pre> 如果输入参数错误, 会提示 "ErrorNo", "ErrorInfo".					

6.16.6. CreateTableAuthInfo 创建表的权限文件

功能名称	CreateTableAuthInfo	版本号	V1.0.7.2	更新日期	20221121
功能备注	创建表的权限文件				
功能说明	创建表的权限文件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=CreateTableAuthInfo				
入参说明					
无					
出参说明					


```
{
  "Resp": "create AuthFileDefault.json success!"
}
说明：
权限文件创建在程序工作目录，文件名为AuthFileDefault.json，当工作目录下存在该文件时，创建会失败。
文件内容含义参考如下：
key-value模式： value表示key是否是只读的。 "1":表示是只读 "0":表示是可写； 其他值可以配置但无法识别，权限按"DefaultRW"配置执行。
关键字"DefaultRW" 表示没有配置或配置错误的字段权限， 1:表示是只读 0:表示是可写；默认值为0。
举例：当前"DefaultRW"配置为0，表示未配置的字段可写。 即"science"字段在web显示时就是可写的。"chinese"web显示只读，"sports"web显示可写。
```

6.16.7. GetAuthInfo 显示表的权限文件

功能名称	GetAuthInfo	版本号	V1.0.7.2	更新日期	20221121
功能备注	显示表的权限文件				
功能说明	显示表的权限文件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=GetAuthInfo				
入参说明					
无					
出参说明					
<pre>{ "Resp": { "TableName": { "///": "表名", "Score": { "///": "表字段权限", "id": "1", "chinese": "1", "english": "1", "math": "1", "science": "1", "sports": "1" }, "tagHeaderFileNode": { "szB1": { "iV1aue4": "1" }, "cFileName": "1", "llWhere": "1", "cTest": "1", "i64": "1", "ui32": "1", "ui64": "1", "iCommon": "1", "b11": "1" } } } }</pre>					

6.16.8. ReLoadAuthInfo 重新加载表的权限文件

功能名称	ReLoadAuthInfo	版本号	V1.0.7.2	更新日期	20221121
功能备注	重新加载表的权限文件				
功能说明	重新加载表的权限文件				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&FuncName=ReLoadAuthInfo				
入参说明					
无					
出参说明					

```
{
  "Resp": {
    "TableName": {
      "//": "表名",
      "Score": {
        "//": "表字段权限",
        "id": "1",
        "chinese": "1",
        "english": "1",
        "math": "1",
        "science": "1",
        "sports": "1"
      },
      "tagHeaderFileNode": {
        "szBl": {
          "iVlaue4": "1"
        },
        "cFileName": "1",
        "szTestStruct": {
          "cFirst": "1",
          "bFlag": "1",
          "iTest": "1",
          "OK": "1",
          "szStaticVar": "1",
          "sID": "1",
          "cAddress": "1",
          "llWhere": "1",
          "cTest": "1",
          "iPrice": "1",
          "iPrices": "1",
          "i64": "1",
          "ui32": "1",
          "ui64": "1",
          "b11": "1"
        }
      }
    }
  }
}
```

6.16.9. ExecSql 执行sql

功能名称	ExecSql	版本号	V1.0.0.0	更新日期	20220919
功能备注	执行指定sql语句				
功能说明	执行指定sql语句				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_tableAnalyse&InstanceNo=0&FuncName=ExecSql				
入参说明					
<pre>{ "//": "待执行sql语句", "SQL": "SELECT * from CCrdtEntrustView LIMIT 10;" }</pre>					
出参说明					

```
{
  "/*": "执行的sql语句",
  "Resp": "SELECT * from CCrdtEntrustView LIMIT 10;",
  "Data": {
    "/*": "列名称",
    "TableColumns": [
      "CurrDate",
      "CurrTime",
      "Busiflag",
      "EntrustNo",
      "OpBranchNo",
      "OperatorNo",
      "OpEntrustWay",
      "BatchNo",
      "StockCode",
      "EntrustBs",
      "EntrustAmount",
      "EntrustPrice",
      "EntrustType",
      "ExchReturnTime",
      "FundAccount"
    ],
    "/*": "对应列名内容",
    "TableRecords": [
      [
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        "",
        ""
      ]
    ],
    "/*": "执行的数量",
    "ExecuteCount": 0,
    "/*": "执行时间",
    "ExecuteTime": 22385,
    "/*": "命中数量",
    "HitCount": 0,
    "/*": "执行结果",
    "ExecResult": "success!"
  }
}
```

6.16.10. DumpTable 执行导出指定表

功能名称	DumpTable	版本号	V1.0.0.0	更新日期	20251208
功能备注	导出指定TableName数据到CSV文件				
功能说明	根据TableName执行sql查询导出数据到CSV文件				
访问链接	http://ip:port/lkdp?Pretty=1&PluginName=lkp_tableAnalyse&InstanceNo=0&FuncName=DumpTable				
入参说明					
<pre>{ "/*": "待导出的表名", "TableName": "Foobar" }</pre>					
出参说明					
<pre>{ "/*": "生成的执行的导出任务Id", "DumpId": 1008, "/*": "返回路径:时间 + DumpId", "DumpFiles": "/home/backup/20251111172748_1008", }</pre>					

6.16.11. DumpAllTable 执行所有的表导出

功能名称	DumpAllTable	版本号	V1.0.0.0	更新日期	20251208
功能备注	导出所有的内存表数据到CSV文件				
功能说明					
访问链接	http://ip:port/lkdp?Pretty=1&PluginName=lkp_tableAnalyse&InstanceNo=0&FuncName=DumpAllTable				
入参说明					
无					
出参说明					

<pre>{ "///": "生成的执行的导出任务Id", "DumpId": 1008, "///": "返回路径:时间 + DumpId", "DumpFiles": "/home/backup/20251111172748_1008", }</pre>

6.16.12. DumpTableStatus 查看导出结果

功能名称	DumpTableStatus	版本号	V1.0.0.0	更新日期	20251208
功能备注	查询指定的导出任务的执行结果状态				
功能说明	查询导出结果				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_tableAnalyse&InstanceNo=0&FuncName=DumpTableStatus				
入参说明					
<pre>{ "///": "执行的导出任务Id", "DumpId": 1008, }</pre>					
出参说明					
<pre>{ "///": "执行的导出任务Id", "DumpId": 1008, "///": "导出表名, 或者ALL", "TableName": "FooBar", "///": "执行的状态, done表示完成, Failed表示失败, Dumping表示在导出中", "Status": "done", "///": "需要导出的总表数", "TotalTables": 1, "///": "已经导出的总行数", "DumpedRows": 3000000, "///": "错误码, 失败时有", "ErrorNo": 9603, "///": "错误信息, 失败时有", "ErrorInfo": "No that dumpId!", }</pre>					

6.16.13. TerminateDump 终止导出任务

功能名称	TerminateDump	版本号	V1.0.0.0	更新日期	20251208
功能备注	终止导出任务				
功能说明	终止导出任务				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_tableAnalyse&InstanceNo=0&FuncName=TerminateDump				
入参说明					
<pre>{ "///": "要终止的导出任务Id", "DumpId": 1008, }</pre>					
出参说明					
<pre>{ "///": "要终止的导出任务Id", "DumpId": 1008, "///": "执行的状态, done表示完成", "Status": "done", "///": "错误码, 失败时有", "ErrorNo": 9603, "///": "错误信息, 失败时有", "ErrorInfo": "No that dumpId!", }</pre>					

6.17. front_proxy插件 ldp_front_proxy

6.17.1. BackendStat 获取后端状态信息

功能名称	BackendStat	版本号	V1.0.6.0	更新日期	20220926
功能备注	后端站点信息				
功能说明	获取后端站点信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_front_proxy&FuncName=BackendStat				
入参说明					
无					
出参说明					

```
{
  "BackendStat": [
    {
      "BackendAddr": "192.168.87.137",
      "Port": 23150,
      "Connected": false,
      "Load": 2
    },
    {
      "BackendAddr": "192.168.87.137",
      "Port": 23151,
      "Connected": false,
      "Load": 2
    }
  ]
}
```

6.17.2. CountInfo获取计数信息

功能名称	CountInfo	版本号	V1.0.6.0	更新日期	20220926
功能备注	计数信息				
功能说明	获取计数信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front_proxy&FuncName=CountInfo				
入参说明					
无					
出参说明					
<pre>{ "CountInfo": { "//": "被动连接数量", "PassConnCount": 0, "//": "空闲代理客户端数量", "FreeBackendCount": 100 } }</pre>					

6.18. 守护插件 ldp_daemon

6.18.1. GetGuardListInfo 获取配置守护列表信息

功能名称	GetGuardListInfo	版本号	V1.0.0.0	更新日期	20230517
功能备注	获取配置守护列表信息				
功能说明	获取配置守护列表信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=ldp_daemon&InstanceNo=0&FuncName=GetGuardListInfo				
入参说明					
无					
出参说明					
<pre>{ "GuardList": [{ //启动顺序 "SetupSequence": 0, //是否自动运行 "AutoRun": 1, //回到次数 "CallbakCnt": 0, //控制打印输出 "PrintCnt": 0, //产品名称 "ProductName": "ldpTAS", //应用名 "AppName": "TA#0", //启动命令 "Cmd": "/home/gongshang/crdt20220722/ldp/bin/hsldp startdaemon ldpTAS TA#0 " }] }</pre>					

6.18.2. GetWatchListInfo 获取配置观察列表信息

功能名称	GetWatchListInfo	版本号	V1.0.0.0	更新日期	20230517
功能备注	获取配置观察列表信息				
功能说明	获取配置观察列表信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_daemon&InstanceNo=0&FuncName=GetWatchListInfo				
入参说明					
无					

出参说明

```
{
  "WatchList": [
    {
      //检测次数
      "SetupSequence": 0,
      //是否自动运行
      "AutoRun": 0,
      //回到次数
      "CallbakCnt": 0,
      //控制打印输出
      "PrintCnt": 1,
      //产品名称
      "ProductName": "ldpTAS",
      //应用名
      "AppName": "TF#0",
      //启动命令
      "Cmd": "/home/workspace/hsl dp startdaemon ldpTAS TF#0"
    }
  ]
}
```

6.18.3. GetShareMemInfo 获取共享内存信息

功能名称	GetShareMemInfo	版本号	V1.0.0.0	更新日期	20230517
功能备注	获取共享内存信息				
功能说明	获取共享内存信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_daemon&InstanceNo=0&FuncName=GetShareMemInfo				

入参说明

无

出参说明

```
{
  "ProcInfos": [
    {
      //角色
      "Role": 0,
      //启动计数
      "SetupCount": 0,
      //启动时间
      "StartTime": 3205997418,
      //进程pid
      "Pid": 102255,
      //进程逻辑状态
      "Status": 1,
      //超时时间
      "Timeout": 32059980252313539,
      //产品名
      "ProductName": "ldpTAS",
      //应用名
      "AppName": "DM#0",
      //异常后启动命令
      "Cmd": "onload -p latency taskset -c 3 hsl dp startdaemon ldpTAS DM#0 "
    },
    {
      "Role": 0,
      "SetupCount": 0,
      "StartTime": 3205997426,
      "Pid": 102259,
      "Status": 1,
      "Timeout": 32059980335934590,
      "ProductName": "ldpTAS",
      "AppName": "TA#0",
      "Cmd": "onload -p latency taskset -c 3 /home/gongshang/crdt20220722/ldp/bin/hsl dp startdaemon ldpTAS
TA#0 "
    }
  ],
  "ProcInfos-All": [
    {
      "Role": 0,
      "SetupCount": 0,
      "StartTime": 3205997418,
      "Pid": 102255,
      "Status": 1,
      "Timeout": 32059980252313539,
      "ProductName": "ldpTAS",
      "AppName": "DM#0",
      "Cmd": "onload -p latency taskset -c 3 hsl dp startdaemon ldpTAS DM#0 "
    },
    {
      "Role": 0,
      "SetupCount": 0,
      "StartTime": 3205997426,
      "Pid": 102259,
      "Status": 1,
      "Timeout": 32059980335934590,
      "ProductName": "ldpTAS",
      "AppName": "TA#0",
      "Cmd": "onload -p latency taskset -c 3 /home/gongshang/crdt20220722/ldp/bin/hsl dp startdaemon ldpTAS
TA#0 "
    },
    {
      "Role": 0,
      "SetupCount": 0,
      "StartTime": 3200033894,
      "Pid": 0,
      "Status": 0,
      "Timeout": 32000345010349819,
      "ProductName": "ldpTAS",
      "AppName": "TT#2",
      "Cmd": "/usr/bin/onload -p latency taskset -c 3 hsl dp startdaemon ldpTAS TT#2 "
    }
  ],
  "ProcCnt": 3
}
```

6.18.4. GetVariablesInfo 获取变量信息

功能名称	GetVariablesInfo	版本号	V1.0.0.0	更新日期	20230517
功能备注	获取变量信息				
功能说明	获取变量信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_daemon&InstanceNo=0&FuncName=GetVariablesInfo				
入参说明					
无					
出参说明					
<pre>{ //启动程序用户名 "UserName": "gongshang", //产品id "m_id": "ldpTAS#DM#0", //进程索引号 "m_iProcIndex": 0, //检测执行计数 "m_CurrentTickCount": 5746, //锁名称 "m_szProcessMutexName": "gongshang_ldpTAS_lock_daemon_creation", //共享内存名称 "m_szShareMemName": "gongshang_ldpTAS_daemon_ldp_ctrl_share_mem", //共享内存锁名称 "m_szShareMemProcCtrlMutexName": "gongshang_ldpTAS_daemon_ldp_ctrl", //回调地址值 "callback": [] }</pre>					

6.19. 通讯服务插件 ldp_remoter

6.19.1. GetAsRcmBackendsStatistics 获取 REMOTER 所有 AS-RCM 端的状态信息

功能名称	GetAsRcmBackendsStatistics	版本号	V1.0.0.0	更新日期	20250703
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 AS-RCM 端的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetAsRcmBackendsStatistics				
入参说明					
无					
出参说明					
<pre>{ "Remoter": [{ "/*": "宿主插件名", "HostPlugin": "ldp_front", "/*": "Backend实例ID", "BackendInstanceId": 0, "/*": "端ID", "BackendId": 1, "/*": "是否通过 CreateBackend 接口创建的", "IsDynamicCreate": false, "/*": "端接收消息计数", "RecvCount": 0, "/*": "端发送成功消息计数", "SendSuccessCount": 0, "/*": "端发送失败消息计数", "SendFailCount": 0, "/*": "端接收拦截消息计数", "InterceptCount": 0, "/*": "端接收错误应答消息计数", "ErrorRspCount": 0, "/*": "端发送主核心成功消息计数", "SendMasterSuccessCount": 0, "/*": "端发送主核心失败消息计数", "SendMasterFailCount": 0, "/*": "端发送备核心成功消息计数", "SendBackupSuccessCount": 0, "/*": "端发送备核心失败消息计数", "SendBackupFailCount": 0, "/*": "预发送（实际被拦截）的应答消息计数", "ToSendRcmRspCount": 0, "/*": "端接收注册应答消息计数", "RecvRegRspCount": 0, "/*": "端成功发送注册请求消息计数", "SendRegSuccCount": 0, "/*": "端失败发送注册请求消息计数", "SendRegFailCount": 0, "/*": "端丢弃服务端就绪消息计数", "UnReadyDropCount": 0, "/*": "端类型", "Type": "AS_RCM", "/*": "是否使用仲裁", "IsArb": false, "/*": "是否开启会话模式", "IsEnableSession": true, "/*": "是否开启查询优先模式", "IsEnableQueryFirst": false, "/*": "是否开启多分片模式", "IsEnableMultiShard": true, "/*": "优先去备模式", "QueryFirstMode": 1, "/*": "是否使用Rcm代理", "IsRcmProxy": true, "/*": "集群名", "Cluster": "", "/*": "产品名", "Product": "", "/*": "发送主题别名", "AliasName": "trade-3", "/*": "发送主题", "TxTopic": "trade", "/*": "是否启用重发机制", "IsEnableResend": true, "/*": "节点（会话）列表", "Sessions": [{ "/*": "服务端分片号", "NodeNo": 50, "/*": "实际发送主题名", "CombineTx": "trade_50", "/*": "查询优先模式下上一次发送的SiteIndex", "ToBackupLastIndex": 0, "/*": "节点索引", "SiteIndex": 0, "/*": "会话发送的注册消息计数", "SendRegCount": 0, "/*": "会话接收的注册消息计数", "RecvRegCount": 0, "/*": "会话发送消息计数", "SendCount": 0, "/*": "累计重发消息计数", "CacheSendCount": 0, "/*": "当前重发队列消息个数", "ResendQSize": 0, "/*": "重发队列消息个数上限", "MaxResendQSize": 0, "/*": "丢弃重发消息个数", "ResendDropCount": 348, "/*": "最近一次重发消息时间，精确到毫秒", "LastResendTimestamp": 1749888781214, "/*": "最近一次重发消息个数", "LastCacheMsgCount": 100, "/*": "当前重发队列消息最大序号", "MaxSeqNo": 2013. }] }] }</pre>					


```
        "":"/": "对端上下文ID",
        "PeerCtxId": 4,
        "":"/": "发送主题ID",
        "TransportId": 5,
        "":"/": "实际发送分区号",
        "TxPartiNo": 4,
        "":"/": "节点连接状态",
        "IsConnected": true,
        "":"/": "服务端节点机房名",
        "ZoneName": "Zone1",
        "":"/": "remoter会话版本号",
        "Version": 1,
        "":"/": "会话获取的站点主备状态",
        "ArbStatus": "Active",
        "":"/": "会话获取的站点平台状态",
        "AppStatus": "LdpReady",
        "":"/": "会话获取的站点业务状态",
        "BizStatus": 1,
        "":"/": "会话获取的站点节点名",
        "SessionNodeName": "xst#0",
        "":"/": "会话获取的站点启动时间戳",
        "LaunchTimestamp": 1698974508,
        "":"/": "会话接收分区号",
        "RxPartiNo": 4,
        "":"/": "核心的运行模式，网格，主备，还是双活",
        "HaMode": 3
    }
}
}
```

6.19.2. GetAsTcpBackendsStatistics 获取 REMOTER 所有 AS-TCP 端的状态信息

功能名称	GetAsTcpBackendsStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 AS-TCP 端的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetAsTcpBackendsStatistics				
入参说明					
无					
出参说明					

```

{
  "Remoter": [
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 1,
      "IsDynamicCreate": false,
      "RecvCount": 0,
      "SendSuccessCount": 0,
      "SendFailCount": 0,
      "InterceptCount": 0,
      "ErrorRspCount": 0,
      "SendMasterSuccessCount": 0,
      "SendMasterFailCount": 0,
      "SendBackupSuccessCount": 0,
      "SendBackupFailCount": 0,
      "RecvRegRspCount": 0,
      "SendRegSuccCount": 0,
      "SendRegFailCount": 0,
      "UnReadyDropCount": 0,
      "ConfigName": "xx",
      "Type": "AS_TCP",
      "MsgType": "LDP",
      "SiteInUse": 1,
      "IsArb": true,
      "IsEnableSession": true,
      "IsEnableQueryFirst": false,
      "QueryFirstMode": 1,
      "ProductName": "ldp",
      "ClusterName": "xst",
      "Sessions": [
        {
          "SiteIndex": 0,
          "SiteName": "xst#0",
          "IP": "192.168.86.101",
          "DomainName": "www.test.com",
          "Port": 10111,
          "ActiveStatus": true,
          "ZoneName": "Zone1",
          "ConnectTimes": 1,
          "RecvCount": 0,
          "SendSuccessCount": 0,
          "SendFailCount": 0,
          "InterceptCount": 0,
          "SiteId": 1,
          "State": "LoggedIn",
          "Version": 1,
          "ArbStatus": "Active",
          "AppStatus": "LdpReady",
          "BizStatus": 1,
          "SessionNodeName": "xst#0",
          "NodeNo": 50,
          "LaunchTimestamp": 1702345841
        }
      ],
      "MasterSiteIndex": 0,
      "MasterNode": "xst#0",
      "ToBackupLastIndex": 0
    }
  ]
}

```

6.19.3. GetFunctionNoList 获取 REMOTER 所有开启会话模式的 Backend 获取到的功能号列表

功能名称	GetFunctionNoList	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有开启会话模式的 Backend 获取到的功能号列表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_remoter&InstanceNo=0&FuncName=GetFunctionNoList				
入参说明					
无					
出参说明					
<pre>{ "Remoter": [{ "HostPlugin": "ldp_front", "BackendInstanceId": 0, "BackendId": 1, "FuncLists": [{ "NodeName": "xst#0", "NodeInfo": "Singleton_Rx.4", "FuncNo": 103, "AllowBizStatus": 18446744073709551615 }] }]}</pre>					

6.19.4. GetGwBackendsStatistics 获取 REMOTER 所有 GW 端的状态信息

功能名称	GetGwBackendsStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 GW 端的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetGwBackendsStatistics				
入参说明					
无					
出参说明					

```
{
  "Remoter": [
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 6,
      "IsDynamicCreate": false,
      "RecvCount": 0,
      "SendSuccessCount": 0,
      "SendFailCount": 0,
      "InterceptCount": 0,
      "ErrorRspCount": 0,
      "RecvRegRspCount": 0,
      "SendRegSuccCount": 0,
      "SendRegFailCount": 0,
      "UnReadyDropCount": 0,
      "ConfigName": "TcpClient4",
      "Type": "GM",
      "MsgType": "LDP",
      "SiteInUse": 1,
      "IsArb": false,
      "IsEnableSession": false,
      "IsEnableQueryFirst": false,
      "Sessions": [
        {
          "SiteIndex": 0,
          "SiteName": "192.168.86.102:59999",
          "IP": "192.168.86.102",
          "DomainName": "www.test.com",
          "Port": 59999,
          "ActiveStatus": true,
          "ConnectTimes": 1,
          "RecvCount": 1,
          "SendSuccessCount": 1,
          "SendFailCount": 0,
          "InterceptCount": 0,
          "SiteId": 6,
          "State": "LogedIn"
        }
      ]
    }
  ]
}
```

6.19.5. GetPluginBackendsStatistics 获取 REMOTER 所有 PLUGIN 客户端的状态信息

功能名称	GetPluginBackendsStatistics	版本号	V1.0.0.0	更新日期	20231024
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 PLUGIN 客户端的状态信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_remoter&InstanceNo=0&FuncName=GetPluginBackendsStatistics				
入参说明					
无					
出参说明					

```
{
  "Remoter": [
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 4,
      "IsDynamicCreate": false,
      "RecvCount": 0,
      "SendSuccessCount": 0,
      "SendFailCount": 0,
      "InterceptCount": 0,
      "ErrorRspCount": 0,
      "RecvRegRspCount": 0,
      "SendRegSuccCount": 0,
      "SendRegFailCount": 0,
      "UnReadyDropCount": 0,
      "Type": "PLUGIN",
      "ConfigName": "xx",
      "ConfigInst": 0,
      "ConfigPlugin": "ldp_mproxy",
      "PluginName": "ldp_mproxy",
      "PluginInst": 0,
      "PluginState": 4
    }
  ]
}
```

6.19.6. GetPluginSvrStatistics 获取 REMOTER 所有 PLUGIN 服务端的状态信息

功能名称	GetPluginSvrStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 PLUGIN 服务端的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetPluginSvrStatistics				
入参说明					
无					
出参说明					
<pre>{ "Remoter": [{ "HostPlugin": "ldp_bizproc", "AsModeDropCount": 0, "UnReadyDropCount": 0, "PluginCount": 1, "Plugin": [{ "PluginName": "ldpbackend_inner_plugin", "PluginIndex": 0, "ConnectID": 1207959552, "MsgQuantity": 0, "IsNeedCache": false, "RecvCount": 1, "SendSuccCount": 1, "SendFailCount": 0 }] }] }</pre>					

6.19.7. GetRcmBackendsStatistics 获取 REMOTER 所有 RCM 端的状态信息

功能名称	GetRcmBackendsStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 RCM 端的状态信息.				

访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetRcmBackendsStatistics
入参说明	
无	
出参说明	
<pre>{ "Remoter": [{ "HostPlugin": "ldp_front", "BackendInstanceId": 0, "RecvCount": 1, "SendSuccessCount": 1, "SendFailCount": 0, "InterceptCount": 0, "ErrorRspCount": 0, "ToSendRcmRspCount": 0, "RecvRegRspCount": 0, "SendRegSuccCount": 0, "SendRegFailCount": 0, "UnReadyDropCount": 0, "Type": "RCM", "ForwardMode": "TOKEN", "PartitionNos": "1", "IsArb": false, "IsEnableSession": false, "IsEnableQueryFirst": false, "IsEnableMultiShard": false, "IsRcmProxy": true, "Cluster": "", "Product": "", "AliasName": "AllRcmTypeTest_4-4", "TxTopic": "AllRcmTypeTest_4", "Sessions": [], "MasterSiteIndex": 0, "MasterNode": "", "ToBackupLastIndex": 0 }] }</pre>	

6.19.8. GetRcmProxyStatistics 获取 REMOTER 所有 RcmProxy 的状态信息

功能名称	GetRcmProxyStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 RcmProxy 的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetRcmProxyStatistics				
入参说明					
无					
出参说明					
<pre>{ "Remoter": [{ "HostPlugin": "ldp_front", "BackendInstanceId": 0, "Type": "Backend", "UserName": "xst#0_ldp_front_0_backend", "Acceptor": { "SendSuccessCount": 0, "SendFailCount": 0, </pre>					

```

        "SendRegSuccCount": 0,
        "SendRegFailCount": 0,
        "RecvRegCount": 0,
        "RecvSingletonsCount": 0,
        "RecvTierCount": 0,
        "RecvBypassCount": 0,
        "RecvSingletonsDropCount": 0,
        "RecvTierDropCount": 0
    },
    "Initiator": [
        {
            "Name": "query",
            "IsTier": false,
            "SingletonId": 0,
            "IsConnected": true,
            "SendSuccessCount": 2401,
            "SendFailCount": 0,
            "RecvCount": 2401
        },
        {
            "Name": "trade_50",
            "IsCombineTx": true,
            "IsTier": false,
            "SingletonId": 0,
            "IsConnected": true,
            "SendSuccessCount": 2401,
            "SendFailCount": 0,
            "RecvCount": 2401
        }
    ],
    "Publisher": [],
    "ContextHandle": [
        {
            "EndName": "",
            "ContextName": "xst#0_ldp_front_0_backend_Singleton",
            "IsMultiProc": false,
            "IsMultiProcValid": false
        }
    ],
    "Fullduplex": [
        {
            "SingletonId": 0,
            "TxTopic": "query",
            "TxPartitions": "",
            "RxTopic": "result",
            "RxPartition": "1",
            "IsConnected": true,
            "ConnectionInfo": [
                {
                    "PeerContext": "xst#0_ldp_bizproc_0_Singleton",
                    "PeerContextId": 4,
                    "UpPartition": 4,
                    "UplinkLocalStat": "Ready",
                    "UplinkPeerStat": "Ready",
                    "DownlinkLocalStat": "Ready",
                    "DownlinkPeerStat": "Ready"
                }
            ]
        },
        {
            "SingletonId": 0,
            "TxTopic": "trade_50",
            "TxPartitions": "",
            "RxTopic": "result",
            "RxPartition": "1",
            "IsConnected": true,
            "ConnectionInfo": [

```

```
        "PeerContext": "xst#0_ldp_bizproc_0_Singleton",
        "PeerContextId": 4,
        "UpPartition": 4,
        "UplinkLocalStat": "Ready",
        "UplinkPeerStat": "Ready",
        "DownlinkLocalStat": "Ready",
        "DownlinkPeerStat": "Ready"
    }
}
}
```

6.19.9. GetRcmSvrStatistics 获取 REMOTER 所有 RCM 服务端的状态信息

功能名称	GetRcmSvrStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER RCM 服务端的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetRcmSvrStatistics				
入参说明					
无					
出参说明					
<pre>{ "Remoter": [{ "HostPlugin": "ldp_bizproc", "ContextInfo": [{ "IsTier": false, "IsAutoStart": false, "EndName": "xst#0_ldp_bizproc_0_Singleton_0", "ContextName": "xst#0_ldp_bizproc_0_Singleton", "AsModeDropCount": 0, "UnReadyDropCount": 0, "UnReadyDropTotalCount": 0, "AsModeDropTotalCount": 0 }] }] }</pre>					

6.19.10. GetRegisterEndInfo 获取 REMOTER 所有已注册的端列表

功能名称	GetRegisterEndInfo	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有已注册的端列表				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetRegisterEndInfo				
入参说明					
无					
出参说明					


```
{
  "Remoter": [
    {
      "HostPlugin": "ldp_front",
      "RegType": "EndName",
      "RegInfo": "xxx",
      "EndId": 4,
      "EndType": "BackendPlugin"
    },
    {
      "HostPlugin": "ldp_front",
      "RegType": "EndName",
      "RegInfo": "Lan",
      "EndId": 0,
      "EndType": "FrontendTcp"
    },
    {
      "HostPlugin": "ldp_front",
      "RegType": "ConnId",
      "RegInfo": "3758096388",
      "EndId": 4,
      "EndType": "BackendPlugin"
    },
    {
      "HostPlugin": "ldp_front",
      "RegType": "ConnId",
      "RegInfo": "1090519040",
      "EndId": 0,
      "EndType": "FrontendTcp"
    },
    {
      "HostPlugin": "ldp_bizproc",
      "RegType": "EndName",
      "RegInfo": "xst#0_ldp_bizproc_0_Singleton",
      "EndId": 0,
      "EndType": "FrontendRcm"
    }
  ]
}
```

6.19.11. GetRemoterConfig 获取 REMOTER 中的配置信息

功能名称	GetRemoterConfig	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 中的配置信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetRemoterConfig				
入参说明					
无					
出参说明					

```

{
  "Remoter": [
    {
      "HostPlugin": "ldp_front",
      "Remoter对象配置",
      "RemoterConfig": {
        "UserName": "xst#0_ldp_front_0_backend",
        "LocalNetworkNumber": "192.168.86.102",
        "ProtocolMode": "UDP-MULTICAST",
        "RspTopic": {
          "Topic": "result",
          "PartitionNo": 1,
          "TransportId": 4
        },
        "Singleton": {
          "Ref": "Default",
          "SingletonId": 0,
          "ContextName": "xst#0_ldp_front_0_backend_Singleton",
          "ContextId": 3,
          "IsAutoStart": true,
          "IsMsgSync": false,
          "IsConsumerCreate": false,
          "TxTopics": [
            {
              "Name": "trade",
              "IsMultiShardTx": true,
              "PartitionNos": []
            }
          ],
          "RxTopics": [],
          "TxBypass": []
        }
      },
      "HostPlugin": "ldp_bizproc",
      "Remoter对象配置",
      "RemoterConfig": {
        "UserName": "xst#0_ldp_bizproc_0",
        "LocalNetworkNumber": "192.168.86.102",
        "ProtocolMode": "UDP-MULTICAST",
        "RspTopic": {},
        "Tier": {
          "Ref": "Tier",
          "TierName": "xst",
          "ContextName": "xst#0_ldp_remoter_0",
          "ContextId": 7,
          "IsAutoStart": false,
          "IsMsgSync": true,
          "IsConsumerCreate": true
        },
        "ZkArbConfig": {
          "MasterNode": "xst#0",
          "CurrentRole": "Active",
          "CurrentNode": "xst#0",
          "Tcp": {
            "Ip": "192.168.86.102",
            "Port": 53647
          },
          "Rcm": [
            {
              "TransportId": 5,
              "Topic": "trade",
              "PartitionNo": 4
            }
          ]
        }
      }
    ]
  ],
  "Frontend中的RcmProxy配置（非原始配置）",
  "RcmSvrConfig_RcmProxy": {
    "UserName": "xst#0_ldp_bizproc_0",
    "LocalNetworkNumber": "192.168.86.102",
    "ProtocolMode": "UDP-MULTICAST",
    "RspTopic": {},
    "Tier": {
      "Ref": "Tier",
      "TierName": "xst",
      "ContextName": "xst#0_ldp_remoter_0",
      "ContextId": 7,
      "IsAutoStart": false,
      "IsMsgSync": true,
      "IsConsumerCreate": true
    },
    "ZkArbConfig": {
      "MasterNode": "xst#0",
      "CurrentRole": "Active",
      "CurrentNode": "xst#0",
      "Tcp": {
        "Ip": "192.168.86.102",
        "Port": 53647
      },
      "Rcm": [
        {
          "TransportId": 5,
          "Topic": "trade",
          "PartitionNo": 4
        }
      ]
    }
  ]
}

```

6.19.12. GetRoutesStatistics 获取 REMOTER 所有路由的状态信息

功能名称	GetRoutesStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					

功能说明	通过本管理功能可以获取 REMOTER 所有路由的状态信息
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_remoter&InstanceNo=0&FuncName=GetRoutesStatistics
入参说明	
无	
出参说明	
<pre>{ "Remoter": [{ "HostPlugin": "ldp_front", "BackendInstanceId": 0, "RouteMissCount": 0, "Routes": [{ "Functions": "**", "SystemNo": "**", "NodeNo": "**", "BackendID": 1, "Count": 0 }] }] }</pre>	

6.19.13. GetRpcClientsStatistics 获取 REMOTER 所有 RPC 客户端的状态信息

功能名称	GetRpcClientsStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 RPC 客户端的状态信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_remoter&InstanceNo=0&FuncName=GetRpcClientsStatistics				
入参说明					
无					
出参说明					
<pre>{ "Remoter": [{ "HostPlugin": "ldp_bizproc", "RpcClientId": 1, "LastPacketId": 0, "FirstPacketId": 0, "StartPacketId": 0, "ReqCount": 0, "RspCount": 0, "IsWaiting": false, "TotalReqCount": 0, "TotalRspCount": 0, "TotalTimeoutRspCount": 0, "MaxPacketId": 16777215 }]}</pre>					

6.19.14. GetStatusInfo 获取 REMOTER 对象的状态信息

功能名称	GetStatusInfo	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 对象的状态信息。				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_remoter&InstanceNo=0&FuncName=GetStatusInfo				
入参说明					
无					
出参说明					

```
{
  "Remoter": [
    {
      "HostPlugin": "ldp bizproc",
      "EnableAppStatusCtrl": true,
      "EnableBizStatusCtrl": true,
      "EnableMsgTypeCtrl": true,
      "EnableBackendAppStatusCtrl": false,
      "EnableBackendBizStatusCtrl": false,
      "ArbStatus": "Active",
      "AppStatus": "LdpReady",
      "BizStatus": 1,
      "ZoneName": "Zone2"
    }
  ]
}
```

6.19.15. GetTcpBackendsStatistics 获取 REMOTER 所有 TCP 端的状态信息

功能名称	GetTcpBackendsStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 TCP 端的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetTcpBackendsStatistics				
入参说明					
无					
出参说明					

```
{
  "Remoter": [
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 3,
      "IsDynamicCreate": false,
      "RecvCount": 1,
      "SendSuccessCount": 1,
      "SendFailCount": 0,
      "InterceptCount": 0,
      "ErrorRspCount": 0,
      "RecvRegRspCount": 0,
      "SendRegSuccCount": 0,
      "SendRegFailCount": 0,
      "UnReadyDropCount": 0,
      "ForwardMode": "LoadBalance",
      "ConfigName": "TcpClient3",
      "Type": "TCP",
      "MsgType": "LDP",
      "SiteInUse": 1,
      "IsArb": false,
      "IsEnableSession": false,
      "IsEnableQueryFirst": false,
      "Sessions": [
        {
          "SiteIndex": 0,
          "SiteName": "192.168.86.102:37909",
          "IP": "192.168.86.102",
          "DomainName": "www.test.com",
          "Port": 37909,
          "ActiveStatus": true,
          "ConnectTimes": 1,
          "RecvCount": 1,
          "SendSuccessCount": 1,
          "SendFailCount": 0,
          "InterceptCount": 0,
          "SiteId": 3,
          "State": "Connected"
        }
      ]
    }
  ]
}
```

6.19.16. GetTcpFactoryConfig 获取 REMOTER 所有 TCP 工厂配置

功能名称	GetTcpFactoryConfig	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 TCP 工厂配置				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetTcpFactoryConfig				
入参说明					
无					
出参说明					

```
{
  "Remoter": [
    {
      "HostPlugin": "ldp_remoter",
      "FactoryName": "PublicFactory",
      "EnableAsyncSend": false,
      "EnableDirectRecv": false,
      "EnableQuickAck": true,
      "EnableNagle": false,
      "SendRecvThreadCount": 0,
      "SendThreadCount": 2,
      "RecvThreadCount": 2,
      "MaxConnectPerAcceptor": 65535,
      "ThreadCoreBind": ""
    }
  ]
}
```

6.19.17. GetTcpSvrStatistics 获取 REMOTER 所有 TCP 服务端的状态信息

功能名称	GetTcpSvrStatistics	版本号	V1.0.0.0	更新日期	20230523
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 所有 TCP 服务端的状态信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_remoter&InstanceNo=0&FuncName=GetTcpSvrStatistics				
入参说明					
<pre>{ "Type": "Acceptors", "StartIndex": 0, "QueryCount": 100, "FrontendName": "" }</pre>					
出参说明					
<pre>{ "Remoter": [{ "HostPlugin": "ldp_bizproc", "AsModeDropTotalCount": 0, "UnReadyDropTotalCount": 0, "AcceptorProtocolType": 255, "Connections": [{ "FrontendName": "ActiveStandby", "ConnectId": 1073807360, "ConnectIndex": 65536, "SendTimes": 1326, "RecvTimes": 1326, "ConnectedTimeSec": 1684840686, "LastSendedTimeSec": 1684841790, "LastRecvedTimeSec": 1684841790, "Address": "192.168.86.102", "Port": 53136, "Protocol": "LDP", "InterceptTimes": 0, "RecvTimesNoHeartbeat": 1105, "SendSuccessTimes": 1105, "SendFailTimes": 0, "ClientType": "None", "ClientPid": 0, "ClientVersion": "", "ClientAddressByProxy": "", "ClientPortByProxy": 0 }] }], }</pre>					

```

        "Type": "Type = Acceptors时",
        "Acceptors": [
            {
                "Name": "ActiveStandby",
                "FrontendName": "ActiveStandby",
                "RecvCount": 1105,
                "SendSuccCount": 1105,
                "SendFailCount": 0
            }
        ],
        "Type": "Type = Config时",
        "ActiveStandby": {
            "Name": "ActiveStandby",
            "FactoryName": "Default",
            "SendWaitTimeMilli": 0,
            "Address": "",
            "AddressBlackList": "",
            "AddressWhiteList": "",
            "Port": 10101,
            "SocketReuseAddress": true,
            "HeartbeatIntervalSec": 20,
            "HeartbeatTimes": 3,
            "MaxConnectCount": 10000,
            "MaxSendBuffSizeBytes": 0,
            "MinRecvBuffSizeBytes": 512,
            "InitialSendBuffSizeBytes": 4194304,
            "EnableAsyncSend": false,
            "EnableDirectRecv": false,
            "MergeSendRecvThread": true,
            "SendRecvThreadCount": 4,
            "SendThreadCount": 4,
            "RecvThreadCount": 4,
            "MaxConnectPerAcceptor": 10000,
            "ThreadCoreBind": "",
            "EnableQuickAck": true,
            "EnableNagle": false
        },
        "Type": "Type = ConnectionsByName时",
        "ConnectionsByName": [
            {
                "ConnectName": "A#4",
                "FrontendName": "ActiveStandby",
                "ConnectId": 1073807360,
                "ConnectIndex": 65536,
                "SendTimes": 1326,
                "RecvTimes": 1326,
                "ConnectedTimeSec": 1684840686,
                "LastSendedTimeSec": 1684841790,
                "LastRecvedTimeSec": 1684841790,
                "Address": "192.168.86.102",
                "Port": 53136,
                "Protocol": "IDP",
                "InterceptTimes": 0,
                "RecvTimesNoHeartbeat": 1105,
                "SendSuccessTimes": 1105,
                "SendFailTimes": 0,
                "ClientType": "None",
                "ClientPid": 0,
                "ClientVersion": "",
                "ClientAddressByProxy": "",
                "ClientPortByProxy": 0,
                "IsLastSendWhenFuzzyMatch": false
            }
        ]
    }
}

```

6.19.18. GetTopo 获取 REMOTER 连接拓扑信息

功能名称	GetTopo	版本号	V1.0.0.0	更新日期	20230921
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 连接拓扑信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_remoter&InstanceNo=0&FuncName=GetTopo				
入参说明					
无					
出参说明					
<pre>{ "Remoter": [{ "HostPlugin": "ldp_bizproc", "BackendInstanceId": 0, "Direction": 0, "ClientProduct": "ldp", "ClientApp": "front#0", "ClientPlugin": "ldp_front#0", "ServerProduct": "ldp", "ServerApp": "xst#0", "ServerPlugin": "ldp_bizproc#0", "Protocol": "TCP", "ServerAddr": "192.168.87.138:20101" }]}</pre>					

6.19.19. GetPartitionRouterInfo 获取 REMOTER 分区转发信息及路由表信息

功能名称	GetPartitionRouterInfo	版本号	V1.0.0.0	更新日期	20250925
功能备注					
功能说明	通过本管理功能可以获取 REMOTER 分区转发信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_remoter&InstanceNo=0&FuncName=GetPartitionRouterInfo				
入参说明					
无					
出参说明					


```

{
  "Remoter": [
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 2,
      "MessageRouterInfo": []
    },
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 4,
      "MessageRouterInfo": [
        {
          "ClusterName": "mdb",
          "SharedNo": 1,
          "PartitionInfo": []
        }
      ]
    },
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 6,
      "MessageRouterInfo": [
        {
          "ClusterName": "mdb",
          "SharedNo": 1,
          "PartitionInfo": []
        }
      ]
    },
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 7,
      "MessageRouterInfo": []
    },
    {
      "HostPlugin": "ldp_front",
      "BackendInstanceId": 0,
      "BackendId": 8,
      "MessageRouterInfo": [
        {
          "ClusterName": "mdb",
          "SharedNo": 1,
          "PartitionInfo": [
            {
              "PatitionNo": 1,
              "SendSuccessCount": 1,
              "SendFailCount": 0
            }
          ]
        },
        {
          "RouterTableInfo": [
            {
              "PartitionNo": 0,
              "RouterTable": [
                {
                  "PartGroupName": "multiproc_mdb#0_partgroup_1",
                  "NodeName": "mdb#0",
                  "AppStatus": 0,
                  "CurrentRole": 0,
                  "MaxMessageNo": 18446744073709551615,
                  "vecUnReadyPartiNo": "",
                  "PartRouterArbStatus": 0,
                  "GroupStatus": 1
                }
              ]
            },
            {
              "PartGroupName": "multiproc_mdb#1_partgroup_1",
              "NodeName": "mdb#1",
              "AppStatus": 0,
              "CurrentRole": 1,
              "MaxMessageNo": 2,
              "vecUnReadyPartiNo": "",
              "PartRouterArbStatus": 2045597024,
              "GroupStatus": 32632
            }
          ]
        }
      ]
    }
  ],
  {
    "HostPlugin": "ldp_front",
    "BackendInstanceId": 0,
    "BackendId": 9,
    "MessageRouterInfo": [
      {
        "ClusterName": "mdb",
        "SharedNo": 1,
        "PartitionInfo": []
      }
    ]
  }
]
}

```

6.20. MDB数据组件 uftmdb_data

6.20.1. GetMdbInitInfo 获取内存数据库上场信息

功能名称	GetMdbInitInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取内存数据库上场信息				
功能说明	获取内存数据库上场信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_bizproc&InstanceNo=0&FuncName=uftmdb_data_GetMdbInitInfo				

入参说明

出参说明

```
{  "/*": "上场信息，注意只有主机才有该信息",  "MdbInitInfo": [    {      "/*": "上场业务类型",      "BizType": "uftmdb",      "/*": "上场次数",      "Count": 1,      "/*": "上场开始时间",      "StartTime": "2023-11-24 11:12:21",      "/*": "上场状态， 值为， 0： 未上场， 1： 上场中， 2： 上场完成， 3： 上场失败",      "Status": 2 ,      "/*": "需加载表的总数量",      "LoadTotalCount": 10,      "/*": "已加载表的总数量",      "LoadedCount": 10,      "/*": "上场总耗时，单位毫秒",      "SpendTimeMilli": 300000,      "/*": "上场结束时间",      "EndTime": "2023-11-24 11:17:21"    }  ] }
```

6.20.2. GetMdbTableInfo 获取表信息

功能名称	GetMdbTableInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取表信息				
功能说明	获取表信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_bizproc&InstanceNo=0&FuncName=uftmdb_data_GetMdbTableInfo				
入参说明					
出参说明					

```

{
  "Tables": [
    {
      // 表名
      "Name": "xxx",
      // 表号
      "Category": 111,
      // 普通表的TableType字段为"NormalTable", 序列表的TableType字段为"SequenceTable"
      "TableType": "NormalTable",
      // 该表索引个数
      "IndexCount": 2,
      // 该表关联记录个数
      "RefRecordCount": 2,
      // 该表关联索引个数
      "RefIndexCount": 1,
      // 对该表的操作是否写Redo, 可以通过这个字段来判断本表是不是主备不同步表, 在运行期允许修改, 因此可能会发生变化
      "WriteRedo": 1,
      // 表特性: ReadCommitted-读已提交(一贯模式); ReadByGlobalVersion-写时生成全局版本号, 读时根据全局版本号访问(重演新增)
      "TableFeature": "ReadByGlobalVersion",
      // 该表的记录数据大小, 未对齐
      "RecordDataSize": 256,
      // 该表的记录数据大小, 对齐后
      "RecordDataSizeAligned": 256,
      // 该表的首版本记录数
      "RecordCount": 333333,
      // 表是否上场, 值为0:不上场, 1:上场
      "IsInit": 1,
      //上场业务类型, 不上场时为空串
      "BizType": "uftmdb",
      // 上场开始时间, 不上场时为空串
      "InitStartTime": "2023-11-24 11:12:21",
      // 上场状态, 值为0:未上场, 1:上场中, 2:上场完成, 3:上场失败
      "InitStatus": 0,
      // 上场耗时, 不上场时为0
      "InitSpendTimeMilli": 1111,
      // 上场结束时间, 不上场时为空串
      "InitEndTime": "2023-11-24 11:12:22",
      // 数据源
      "DbAlias": "usersvr",
      // 上场sql
      "Sql": "select * from xxx",
      // 该表的临时记录数和修改版本记录数之和
      "TempRecordCount": 0,
      // 该表总共使用的内存大小, 单位MB
      "TotalMemoryMB": 32,
      // 该表的首版本记录使用的内存大小
      "RecordMemoryMB": 2,
      // 该表的ROWID使用的内存大小
      "RowIdMemoryMB": 0,
      // 该表的临时记录数和修改版本记录使用的内存大小
      "TempRecordMemoryMB": 0,
      // 该表的索引使用内存大小
      "IndexMemoryMB": 30,
      // 该表的变长字段使用的内存大小
      "VarFieldMemoryMB": 0,
      "Files": [
        {
          // 表的记录对应的文件
          "TableFile": "/dev/shm/uftmdb.table.mdb_secu.table1",
          // 文件大小, 单位MB
          "FileSizeMB": 2,
          // 文件版本
          "FileVersion": 6
        },
        {
          // 表的临时记录对应的文件
          "TableFile": "/dev/shm/uftmdb.tbtmp.mdb_secu.table1",
          // 文件大小, 单位MB
          "FileSizeMB": 0,
          // 文件版本
          "FileVersion": 6
        },
        {
          // 表的变长字段对应的文件
          "TableFile": "/dev/shm/uftmdb.tblvf.mdb_secu.table1",
          // 文件大小, 单位MB
          "FileSizeMB": 0,
          // 文件版本
          "FileVersion": 6
        },
        {
          // 表的索引对应的文件
          "IndexFile": "/dev/shm/uftmdb.index.mdb_secu.table1",
          // 文件大小, 单位MB
          "FileSizeMB": 30,
          // 文件版本
          "FileVersion": 6
        }
      ],
      // 创建记录插入表级索引时, 每个索引发生冲突的次数
      // (在一次插入索引的过程中, 发生冲突后只要不超时, 会重试, 统计时只要发生过冲突, 统一只记录一次)
      "InsertKeyConflicts": [
        {
          // 索引名称
          "IndexName": "SetIndex",
          // 冲突的次数
          "Count": 3
        }
      ],
      // 调用UpdateRecord出现了冲突的UpdateRecord次数
      "UpdateConflictCount": 2,
      // 该表的分区字段, 有多个分区字段时用';'分割
      "PartitionField": "",
      // 其他事务控制器创建的记录, 未提交时, 本事务控制器对记录进行更新, 导致报错的次数
      "UpdateWhenRecordIsCreatingCount": 2,
      // 其他事务控制器更新记录的非索引字段, 未提交时, 本事务控制器对记录进行更新, 导致报错的次数
      "UpdateWhenRecordIsUpdatingCount": 0,
      // 其他事务控制器更新记录的索引字段, 未提交时, 本事务控制器对记录进行更新, 导致报错的次数
      "UpdateWhenRecordIsUpdatingIndexCount": 2
    }
  ]
}

```

6.20.3. GetMdbSequenceInfo 获取序列号信息

功能名称	GetMdbSequenceInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取序列号信息				
功能说明	获取序列号信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbSequenceInfo				
入参说明					
出参说明					
<pre>{ "Sequences": [{ // 序列的编号 "No": 0, // 起始号 "Start": 1, // 增长步长 "Increment": 1, // 序列同步间隔，设置为10000的话相差一万就同步一次 "SyncInterval": 10000, // 当前值 "Current": 100, // 上次同步的值 "LastSync": 1 }] }</pre>					

6.20.4. GetUfmdbCtrlInfo 获取内存数据库控制信息

功能名称	GetUfmdbCtrlInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取内存数据库控制信息				
功能说明	获取内存数据库控制信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetUfmdbCtrlInfo				
入参说明					
出参说明					
<pre>{ // 数据库名称 "DataBase": "mdb_secu", // 内存数据库内置用户名，拥有最高权限用户，当显示为空的时候为未配置内置用户名 "SystemUsername": "sys", // 是否开启了自动写事务的功能，Yes-开启了自动写事务，No-没有开启自动写事务 "AutoBeginWriteTrans": "Yes", // 下一个将要使用的全局版本号 "NextWriteXid": 4, // 表示主/备 主为Leader，备为Standby "Role": "Leader", // 是否为分区模式 "IsPartMode": true, // 当前节点ID "NodeId": 0, // 当前数据库的下一事务号，一个数据库下有一个或多个进程，每个进程有多个事务控制器，事务控制器有开始事务，提交事务等操作，提交事务之后， // 事务号就会递增 "CommitSqn": 100, "AllNodeInfos": [// 所有节点事务号信息，通过遍历这个数组可以获取所有节点的事务号信息 { // 节点ID "NodeId": 0, // 该节点的下一传输号 "CommitSqn": 100, // 该节点回放时要求的起始传输号 "MinTransactionNo": 0, // 该节点回放时要求过滤传输号的上水位线 "MaxTransactionNo": -1, // 该节点的下一事务号 "TxId": 4 }], "RedoFragmentMergerInfos": [//所有节点的redo重组信息 { // 节点ID "NodeId": 0, // 分片重组器详细统计（备库） // 缓冲状态：当前缓冲中的事务数量" "BufferedTxCount": 0, // 缓冲状态：当前缓冲总大小（字节） "BufferedTotalSize": 0, // 缓冲状态：当前缓冲中最小的TransferId "MinBufferedTransferId": 0, // 下一传输号 "NextTransactionNo": 0, // 要求的起始传输号 "MinTransactionNo": 0, // 要求过滤传输号的上水位线 "MaxTransactionNo": -1, // 累计统计：累计收到的分片数 "TotalReceivedFragments": 0, // 累计统计：累计完成的事务数 "TotalCompletedTx": 0, // 累计统计：累计完整redo（未分片）数量 "TotalCompleteRedo": 0, // 累计统计：累计重组redo数量</pre>					

```

        "TotalReassembledRedo": 0,
        // 累计统计: 累计重组redo字节数
        "TotalReassembledBytes": 0,
        // 累计统计: 单个事务最大分片数量
        "MaxFragmentCount": 0,
        // 累计统计: 单个事务最大重组大小(字节)
        "MaxReassembledSize": 0,
        // 错误统计: 内存分配失败次数
        "AllocFailCount": 0,
        // 跳过的分片数量
        "SkippedFragmentCount": 0,
        // 已经返回但未被释放的被重组的redo数量
        "NeedFreeFragmentRedoCount": 0,
        "BufferedTransactions": [ {
            // 缓冲中的事务详情(最多显示100个)
            // 事务ID
            "TxId": 0,
            // 该事务第一个分片对应的TransferId
            "FirstFragmentTransferId": 0,
            // 该事务已缓存的redo总大小(字节)
            "TotalSize": 0,
            // 该事务已缓存的分片数量
            "FragmentCount": 0,
            // 该事务当前缓冲区容量(字节)
            "BufferCapacity": 0
        }
    ]
},
// 当前数据库的工作进程数, 目前是单进程数据库, 未来会支持多进程数据库
"ProcessCount": 1,
// 当前数据库总共创建了多少个表
"TableCount": 233,
// 当前数据库总共占用了多少内存
"UsedMemoryMB": 1024,
// 当前数据库分区权限开启级别, 0: 不开启, 1: 开启校验, 但仅记录日志, 不报错阻断业务流程, 2: 开启校验, 记录日志, 报错且阻断业务流程
"CheckRecordPartition": 0,
// 当前数据库时间戳打点等级(内部打点, 用于查看内部各个函数消耗的时间, 0表示关闭, 1和2分别表示不同的打点粒度)
"Trace": 0,
// 当前数据库是否开启权限校验控制, true为开启校验, false为关闭校验, 开启之后, SQL对于不支持的语句报错, 并且对于APM调用的SQL语句会进行权限校验
"AccessControl": true,
// 是否开启通过SQL插入/更新产生的字符/字符串类型字段的null/空串替换成空格, true为开启, false为关闭
"EmptyToSpaceConversion": false,
// 是否开启Redo分片功能: false-不开启, true-开启
"RedoFragmentEnabled": true,
// 单个Redo分片的大小, 单位KB, 范围[1, 1048576]
"RedoFragmentEverySize": 10240,
// 记录慢SQL阈值, 单位微秒
"SlowSQLTimeMicro": 0,
// 记录变长字段的读存行为, 0 - 变长字段存储值null读取时给出null, 1 - 变长字段存储值null读取时给出""
"IsVarcharNullReturnEmpty": 0,
// 慢事务的时间阈值, 单位微秒
"SlowTransactionTimeMicro": 500000,
// 当前数据库对应的数据库控制文件AdminFile, 每个数据库只有一个, 存储了数据库控制结构体
"AdminFile": "/dev/shm/uftmdb.admin.mdb_secu",
// 当前数据库对应的数据库控制文件AdminFile大小, 单位字节
"FileSizeBytes": 3012763,
// 当前数据库对应的数据库控制结构体版本号, 结构体定义改动时, 结构体版本号就要增加
"FileVersion": 3,
// 当前数据库的主控进程号, 对于数据库的一些操作, 比如创建数据库, 清理数据库, 重置数据库等只有主控进程能做, 其他的进程都是事务进程, 目前是单进程数据库, 唯一存在的那个进程同时也是主控进程
"AdminPid": 111,
// 当前加锁进程号, 某个进程修改数据库控制结构体的字段的时候需要加锁 0表示没有加锁
"LockPid": 222,
// 心跳超时时间, 单位毫秒, 判断进程是否心跳超时的情况下使用, 超过这个阈值就说明这个进程出现异常了
"HeartbeatTimeoutMill": 6000,
// 最大进程数, 目前单进程数据库, 这个值为1
"MaxProcessCount": 1,
// Sql引擎池中空闲的Sql引擎数
"FreeSqlEngineCount": 1,
// 每个事务控制器的Sql缓存条数上限
"SqlCacheSizeLimit": 3,
// 当前所有事务控制器的Sql缓存条数总和
"SqlCacheCountExist": 2,
// 所有事务控制器最大的redo的估算大小与实际大小差值
"DiffPredictRedoSize": 0,
// 总的GC处理项
"TotalGCActionCount": 10000,
// 已经被处理的GC项
"ProcessedGCActionCount": 9000,
// 剩余未处理的GC项
"LeftGCActionCount": 1000,
// GC循环扫描次数
"GCThreadLoopTimes": 2345,
// 更新索引字段提交时, 遇到其他事务控制器读取该记录, 陷入等待的次数
"CommitUpdateIndexWaitTimes": 239628,
// 更新索引字段提交, 总次数
"CommitUpdateIndexTotalTimes": 239628,
// 更新索引字段提交时, 遇到其他事务控制器读取该记录, 陷入等待, 平均每次等待的时间(ns)
"CommitUpdateIndexUseTimeAvg": 624,
// 更新索引字段提交时, 遇到其他事务控制器读取该记录, 陷入等待, 最大等待的时间(ns)
"CommitUpdateIndexUseTimeMax": 285444,
// 更新索引字段提交时, 遇到其他事务控制器读取该记录, 陷入等待设置的超时时间(ms)
"UpdateIndexWaitReadTimeoutMilli": 10,
// 写Redo出错后的策略, 0表示写Redo出错后继续写Redo, 1表示写Redo出错后停止写Redo 出现9表示该数据库的事务控制器策略不一致, 一部分是0, 一部分是1
"WriteRedoPolicy": 1,
// 是否启用预分配内存, 0表示不启用, 1表示启用
"PreAllocMemory": 0,
// Undo日志的截断大小, 单位MB
"UndoTruncateSizeMB": 1024,
// 本数据库使用的地址空间范围, 进行内存映射时, 地址空间的范围
"AddressSpaceRange": "0x10000000-0x20000000, 0x50000000-0x60000000",
// SQL向带精度实数字段存入数据时如何取整. 0表示不调整, 1表示四舍五入
"RoundMode": 1,
// 序列集合编号的范围区间
"SequenceSetRangeMinMax": "[1, 2000]",
// 序列集合的数量
"SequenceSetCount": 0,
// 序列表起始表号
"SeqTableStartCategory": 30001,
// 序列号当发生从远端预取错误时, 跳过该序列号紧接的多少毫秒的预取
"SeqSkipMillisecondWhenAllocError": 1000,
// MDB运维功能开关, true表示数据库运维功能开, false表示数据库运维功能关
"IsMdbMaintenanceOpen": true,

```

```
// MDB状态: "MDBRunning"--就绪状态 "MDBIniting"--数据库初始化中 "MDBMaintenance"--数据库运维中
"MDBMaintenanceWaiting"--数据库运维等待中
    "MdbStatus": "MDBRunning",
    // 以下4个字段当MdbStatus不为MDBRunning时存在
    // 运维等待的表名
    "WaitingTableName": "tablename",
    // 运维等待的表号, 范围为0~65535
    "WaitingCategory": 0,
    // 运维等待的事务号, 范围为0~255
    "WaitingTransCtrlNo": 0,
    // 运维等待的操作, 清表-ClearTable, 其他-Unknown
    "WaitingAction": "ClearTable",
    // 被运维等待堵塞的事务控制器相关信息
    "BlockingTransCtrlsInfo": [
        {
            // 被运维等待堵塞的事务控制器ID
            "BlockedTransCtrlID": 9,
            // 当前事务控制器已经被堵塞了多少时间(毫秒)
            "BlockedTimeMs": 999
        },
        {
            "BlockedTransCtrlID": 12,
            "BlockedTimeMs": 999
        }
    ],
    // 是否启用内存数据库共享内存使用保护, 0表示不启用, 1表示启用
    "ShmUseProtect": 1,
    // true-禁用全局版本本号表, false-启用全局版本本号表
    "DisableRRTable": true,
    // 用于控制事务控制器创建时, 它的读全局版本本号的初始值, uint64_t类型, 默认为18446744073709551615, 当其值为
    18446744073709551614时表示初始值为数据库当前最大的全局版本号, 其它值则直接赋值不特殊处理
    "InitReadGlobalVersion": 18446744073709551614,
    // Redo事务头结构体的版本, 结构体有变动时增加
    "RedoVersion": 6,
    "Processes": [ // 当前数据库所有的进程的信息
        {
            // 进程序号
            "ProcessNo": 1,
            // 系统进程号, 此进程号对应的进程在系统中实际的进程号
            "SystemPid": 1111,
            // 进程状态, 有三种可以取的值, 分别为Normal、Invalid、CleanUp
            "Status": "Normal",
            // 此进程上次心跳的时间, 单位毫秒
            "LastHeartbeatMill": 11111,
            // 该进程的事务控制器个数
            "TransCtrlCount": 22
        }
    ],
    // 并发回放配置信息
    "ParallelReplayConfig": {
        // 模式 0-禁用(展示为disable) 1-普通同步式并发回放(展示为normalSync)
        // 在配置文件中, 使用整数值
        // 在管理功能中, 展示为名称字符串
        "Mode": "normalSync",
        // 并发线程数量
        "ThreadCount": 4,
        "PollingLoops": 1000,
        "SleepTimeUS": 0,
        // 绑核配置
        "ThreadBindCpu": "80,81,82,83,84,85,86,87"
    }
}
```

6.20.5. GetMdbTransCtrlInfo 获取事务管理器信息

功能名称	GetMdbTransCtrlInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取事务管理器信息				
功能说明	获取事务管理器信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=utfmdb_data_GetMdbTransCtrlInfo				

入参说明

--

出参说明

```
{
    "TransCtrls": [
        {
            // 事务控制器ID
            "TCId": 11,
            // 该事务控制器所属进程的进程序号
            "ProcessNo": 1,
            // 该事务控制器的编号
            "TransCtrlNo": 0,
            // 该事务控制器正在等待的记录, 比如这个事务控制器要操作一条记录, 但是这条记录正在被进行操作, 需要等待这条记录被操作完成 值为
            18446744073709551615时 (uint64的最大值) 表示没有正在等待的记录
            "WaitRecord": 11,
            // 该事务控制器正在等待的事务控制器ID, 比如当前事务控制器进行的事务需要等待另一个事务控制器的事务完成再进行操作 值为uint16的
            最大值时表示没有正在等待的事务控制器
            "WaitTCId": 2,
            // 当前事务提交标志, 0表示未提交, 1表示已提交
            "IsCommitting": 1,
            // 当前事务控制器正在进行的事务的传输号 值为uint64的最大值时表示不在事务中
            "CommitSqn": 100,
            // 最后未完结事务的起始传输号
            "MinTransactionNo":-1,
            // 最后已发出的传输号
            "MaxTransactionNo":100,
            // 当前事务控制器正在进行的事务的事务号
            "TxId":100,
            // 最后传输号(包括不写redo的事务)
            "LastCommittingTransaction" : 100,
            // 最大的redo的估算大小与实际大小差值
            "DiffPredictRedoSize":0,
            // 是否在事务中, 0表示不在事务中, 1表示在事务中
            "IsInTrans": 1,
            // 是否在写事务中, 0表示不在写事务中, 1表示在写事务中
```

```

    "IsInWriteTrans": 1,
    // 是否在保存点中, 0表示不在保存点, 1表示在保存点中
    "IsInSavePoint": 1,
    // 事务超时时间, 单位毫秒
    "TransactionTimeout": 5,
    // 事本事务控制器中分区权限校验失败的次数
    "PartitionAccessDenyCount": 0,
    // 非预期的写事务回滚次数, 忘记结束写事务, 在读事务结束时回滚
    "RollbackWriteTransInEndReadTransCount": 0,
    // 非预期的保存点提交次数, 忘记处理保存点, 随写事务提交而提交
    "CommitSavePointInCommitWriteTransCount": 0,
    // 非预期的保存点回滚次数, 忘记处理保存点, 随写事务回滚而回滚
    "RollbackSavePointInRollbackWriteTransCount": 0,
    // 本事务控制器提交事务的次数
    "CommitWriteTransactionCount": 12,
    // 允许读取的最大全局版本号, 0-不允许读取, 18446744073709551615-可读最新版
    "CurrentReadXid": 18446744073709551615,
    // CurrentReadXid字段的设置次数
    "ReadXidSetCnt": 0,
    // CurrentReadXid字段的最近一次设置时间, 当ReadXidSetCnt>0时有意义
    "LastReadXidSetTime": "1970-01-01 08:00:00.000000000",
    // 当前进行中的事务的全局版本号的写入值 (当前写事务进行中, 且写入全局表时有意义, 否则为0)
    "CurrentWriteXid": 0,
    // 最近一次通知全局版本号的值, NoticeWriteXidCnt>0时有意义
    "LastNoticeWriteXid": 0,
    // 已通知的全局版本号变化的次数
    "NoticeWriteXidCnt": 0,
    // 最近一次通知全局版本号的时间, NoticeWriteXidCnt>0时有意义
    "LastNoticeWriteXidTime": "1970-01-01 08:00:00.000000000",
    // 当前数据库 (非本事务控制器) 的最新的全局版本号
    "CurrentGlobalVersion": 11,
    // 当前事务控制器对于全局表的操作信息
    "RRTableOpStat": {
        // 更新记录索引字段的提交次数-总次数
        "UpdateIndexTotalCommitCnt": 10,
        // 更新记录索引字段的提交次数-正常提交次数
        "UpdateIndexNormalCommitCnt": 2,
        // 更新记录索引字段的提交次数-超时后强制覆盖次数
        "UpdateIndexTimeoutOverWriteCommitCnt": 8
    },
    // 最近一次写Redo的信息
    "LastRedoInfo": {
        "RedoSizeBytes": 100, // Redo大小
        "WriteRedoUseTimeNs": 100000, // 写Redo耗时
        "WriteRedoStartTime": "1001.135034.131" // 写Redo开始的时刻 (mmdd.hhmmss.mmm)
    },
    "RedoFragmentStat": {
        //"/": "Redo分片统计信息",
        //"/": "发送端统计 (主库): 发生redo分片的事务数量",
        "FragmentedTxCount": 0,
        //"/": "发送端统计 (主库): 发送的分片数量",
        "FragmentSentCount": 0,
        //"/": "发送端统计 (主库): 发送的分片总字节数 (不包含头部)",
        "FragmentSentBytes": 0
    },
    "Undo": {
        // 事务控制器的Undo文件信息
        // 对应的Undo文件的路径和名称
        "UndoFile": "/dev/shm/ufitmdb.undo.mdb_secu.[TCId].1",
        // Undo文件文件大小, 单位MB
        "FileSizeMB": 32,
        // Undo文件头版本
        "FileVersion": 2,
        // Undo文件占用的内存, 单位MB
        "MemoryMB": 32,
        // Undo文件日志条数
        "LogItemCount": 100,
        // Undo文件有效数据长度, 不包含Undo头
        "LogItemBytes": 100000,
        // 事务提交或回滚处理进度
        "AppliedBytes": 5000
    },
    // 并发回放统计
    "ReplayStat": {
        // 主线程 (处理业务功能号409的那个线程)
        "Primary": {
            // 时间间隔 (纳秒) 1s左右
            "Nano": 1000081441,
            // 事务数
            "Txn": 218545,
            // 事务字节数
            "TxBytes": 793362472,
            // 创建记录次数
            "ActionCreateRecord": 143274,
            // 更新记录次数
            "ActionUpdateRecord": 237893,
            // 插入索引次数
            "ActionInsertIntoIndex": 453796,
            // 重插索引次数
            "ActionReinsertIndex": 56467,
            // ART索引操作次数
            "ArtOp": 56472,
            "CbSyncIdx": 7708077,
            "CbReplay": 84849702,
            // 状态 initial-初始态 failed-初始化失败 succeeded-初始化成功
            "InitStat": "succeeded",
            // 类型 callback-409功能号线程 indexSync并发索引线程
            "ReplayMode": "callback"
        },
        // 并发回放线程0 (根据实际配置, 会有1、2、3...)
        "Thread0": {
            "Nano": 1000060596,
            "ArtOp": 112974,
            "ArtWaitRowid": 60034,
            "ArtInsert": 27699586,
            "IdxReplay": 41483288,
            "IdxWaitRedo": 55656067,
            "InitStat": "initial",
            "ReplayMode": "indexSync",
            // 内部线程编号
            "ThreadIdx": 0,
            // 绑定cpu
            "BindCpu": 80,
            // 是否停止
            "IsStop": false,
            // 绑定的事务控制器的编号
            "TcId": 70
        }
    }
}

```

```
    }
    }
    ]
}
```

6.20.6. GetRecordInfo 获取记录信息

功能名称	GetRecordInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取记录信息				
功能说明	获取记录信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=utfmdb_data_GetRecordInfo				
入参说明					
出参说明					
<pre>{ "RecordInfo": [{ "Category": 103, "RecordName": "entrust", "MemoryUsage": 4096, "RecordCount": 0, "LoadRecordTime(us)": 0 }, { "Category": 104, "RecordName": "account", "MemoryUsage": 4096, "RecordCount": 0, "LoadRecordTime(us)": 0 }] }</pre>					

6.20.7. GetContextObjectPoolInfo 获取上下文packer池信息

功能名称	GetContextObjectPoolInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取上下文packer池信息				
功能说明	获取上下文packer池信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=utfmdb_data_GetContextObjectPoolInfo				
入参说明					
出参说明					
<pre>{ "ContextObjectPoolInfo": [{ "ThreadNo": 0, "PoolMgrInfo": [{ "PoolMgrType": "Packer", "PoolInfo": [{ "PackerType": "V1", "FreeCount": 1024, "GetCount": 0, "ReleaseCount": 0, "NewCount": 0, "HoldCountOfNew": 0, "ErrorCount": 0 }, { "PackerType": "V2", "FreeCount": 1024, "GetCount": 0, "ReleaseCount": 0, "NewCount": 0, "HoldCountOfNew": 0, "ErrorCount": 0 }, { "PackerType": "V3", "FreeCount": 1024, "GetCount": 0, "ReleaseCount": 0, "NewCount": 0, "HoldCountOfNew": 0, "ErrorCount": 0 }] }] }]}</pre>					


```
,
{
  "PackerType": "v4",
  "FreeCount": 1024,
  "GetCount": 0,
  "ReleaseCount": 0,
  "NewCount": 0,
  "HoldCountOfNew": 0,
  "ErrorCount": 0
}
]
},
{
  "PoolMgrType": "UnPacker",
  "PoolInfo": [
    {
      "PackerType": "v1",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v2",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v3",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v4",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    }
  ]
},
{
  "PoolMgrType": "RsPacker",
  "PoolInfo": [
    {
      "PackerType": "v1",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v2",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v3",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v4",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    }
  ]
},
{
  "PoolMgrType": "RsUnPacker",
  "PoolInfo": [
    {
      "PackerType": "v1",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v2",
      "FreeCount": 1024,
      "GetCount": 0,
      "ReleaseCount": 0,
      "NewCount": 0,
      "HoldCountOfNew": 0,
      "ErrorCount": 0
    },
    {
      "PackerType": "v3",
```

```
        "FreeCount": 1024,
        "GetCount": 0,
        "ReleaseCount": 0,
        "NewCount": 0,
        "HoldCountOfNew": 0,
        "ErrorCount": 0
    },
    {
        "PackerType": "V4",
        "FreeCount": 1024,
        "GetCount": 0,
        "ReleaseCount": 0,
        "NewCount": 0,
        "HoldCountOfNew": 0,
        "ErrorCount": 0
    }
]
}
]
}
}
```

6.20.8. GetDatamgrLibInfo 获取Datamgr库信息

功能名称	GetDatamgrLibInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取Datamgr库信息				
功能说明	获取Datamgr库信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufmdb_data_GetDatamgrLibInfo				
入参说明					
出参说明					
<pre>{ "DatamgrLibInfo": [{ "//": "库名", "LibName": "libs_uftmdb_datamgr.so", "//": "库版本", "LibVersion": "1.0.0.0", "//": "加载库所在路径", "LibPath": "/home/xxx/ldp_mdb/workspace/./bin/pub/libs_uftmdb_datamgr.so", "//": "开发工具对应的平台发布版本", "PlatformVersion": "LDP1.0.V202302.00.000" }]}</pre>					

6.20.9. GetMDBLibInfo 获取内存数据库版本信息

功能名称	GetMDBLibInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取内存数据库版本信息				
功能说明	获取内存数据库版本信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufmdb_data_GetMDBLibInfo				
入参说明					
出参说明					
<pre>{ "//": "库名", "LibName": "libuftmdb.so", "//": "内存数据库版本", "Version": "V4.0.0.11 Aug 24 2023 17:21:48", "//": "加载库所在路径", "LibPath": "/home/wangsong/ldp_mdb/workspace/./bin/components/libuftmdb.so", "//": "内存数据库主备状态。0: 主机。1: 备机", "MdbMode": 0}</pre>					

6.20.10. GetMemUsage 获取内存使用情况

功能名称	GetMemUsage	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取内存使用情况				
功能说明	获取内存使用情况				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufmdb_data_GetMemUsage				
入参说明					
出参说明					

```
{
  "///": "表工厂内存使用总量 (Bytes) ",
  "TableFactory": 8388608,
  "///": "Daemon类的内存使用总量 (Bytes) ",
  "UftMdbDaemon": 2160736,
  "///": "上下文内存使用总量 (Bytes) ",
  "Context": 85390480
}
```

6.20.11. GetPoolInfo 获取内存池信息

功能名称	GetPoolInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取内存池信息				
功能说明	获取内存池信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetPoolInfo				
入参说明					
出参说明					
<pre>{ "PoolInfo": [{ "///": "上下文编号", "ContextId": 0, "///": "内存池编号, 每个上下文都有5层内存池", "No": 0, "///": "内存池块大小", "MemoryBlockSize": 256, "///": "内存池块的个数", "PoolSize": 3200, "///": "内存池当前空闲块数", "PoolCurCount": 3200, "///": "错误次数", "ErrCount": 0 }, { "ContextId": 0, "No": 1, "MemoryBlockSize": 512, "PoolSize": 3200, "PoolCurcount": 3200, "ErrCount": 0 }, { "ContextId": 0, "No": 2, "MemoryBlockSize": 768, "PoolSize": 3200, "PoolCurcount": 3200, "ErrCount": 0 }, { "ContextId": 0, "No": 3, "MemoryBlockSize": 1024, "PoolSize": 3200, "PoolCurcount": 3200, "ErrCount": 0 }, { "ContextId": 0, "No": 4, "MemoryBlockSize": 1280, "PoolSize": 3200, "PoolCurcount": 3200, "ErrCount": 0 }] }</pre>					

6.20.12. GetPoolIncInfo 获取扩容内存池信息

功能名称	GetPoolIncInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取扩容内存池信息				
功能说明	获取扩容内存池信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetPoolIncInfo				
入参说明					
出参说明					

```
{
  "PoolIncInfo": [
    {
      "/*": "上下文编号",
      "ContextId": 0,
      "/*": "内存池编号",
      "No": 5,
      "/*": "块大小",
      "MemoryBlockSize": 2560,
      "/*": "内存池块的个数",
      "PoolSize": 0,
      "/*": "内存池当前空闲块数",
      "PoolCurCount": 0,
      "/*": "错误次数",
      "ErrCount": 0,
      "/*": "向系统申请的块数",
      "SystemMalloc": 0,
      "/*": "向操作系统申请的大小",
      "SystemMallocSize(m)": 0.0,
      "/*": "向系统申请的总个数",
      "SystemTotalMalloc": 0,
      "/*": "向系统申请总的内存块大小",
      "SystemTotalMallocSize": 0,
      "/*": "最大内存块大小",
      "SystemTotalMaxBlockSize": 0,
      "/*": "最小内存块大小",
      "SystemTotalMinBlockSize": 0,
      "/*": "平均内存块大小",
      "SystemTotalAveBlockSize": 0
    }
  ]
}
```

6.20.13. SetFunctionLogLevel 设置功能号日志级别

功能名称	SetFunctionLogLevel	版本号	V1.0.5.4	更新日期	20230823
功能备注	设置功能号日志级别				
功能说明	设置功能号日志级别				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_SetFunctionLogLevel&Function=1111&LogLevel=1				
入参说明					
<pre>{ "/*": "功能号", "Function": "1111", "/*": "日志级别, 0: 调试, 1: 信息, 2: 告警, 3: 错误, 4: 重要, 5: 致命错误", "LogLevel": "1" }</pre>					
出参说明					
<pre>{ "/*": "日志级别", "LogLevel": 1 }</pre>					

6.20.14. GetFunctionLogLevel 获取功能号日志级别

功能名称	GetFunctionLogLevel	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取功能号日志级别				
功能说明	获取功能号日志级别				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetFunctionLogLevel				

入参说明

出参说明

```
{
  "FunctionLogLevelInfo": [
    {
      "/*": "未指定功能号",
      "Function": "all_function",
      "/*": "日志级别",
      "LogLevel": "LEVEL_DEBUG"
    },
    {
      "/*": "指定功能号",
      "/*": "功能号",
      "Function": "1111",
      "/*": "日志级别",
      "LogLevel": "LEVEL_INFO"
    },
    {
      "/*": "功能号",
      "Function": "2222",
      "/*": "日志级别",
      "LogLevel": "LEVEL_DEBUG"
    }
  ]
}
```

6.20.15. SetRSDealTimeStatus 设置因子处理耗时状态

功能名称	SetRSDealTimeStatus	版本号	V1.0.5.4	更新日期	20230823
功能备注	设置因子处理耗时状态				
功能说明	设置因子处理耗时状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_SetRSDealTimeStatus&Status=1				
入参说明					
<pre>{ "///": "状态", "Status": "1",}</pre>					
出参说明					
<pre>{ { "///": "注：当请求中Status传入为0或者1时", "///": "状态", "Status": "start/stop", }, { "///": "注：当请求中Status传入不为0和1时", "Status": "Error Status, 0:Stop,1:Start", }, { "///": "注：当请求中Status不存在时", "///": "错误号", "ErrorNo": 102, "ErrorMsg": "无效的输入参数(lack of field [Status])" } }</pre>					

6.20.16. GetRSDealTimeStatus 获取因子处理耗时状态

功能名称	GetRSDealTimeStatus	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取因子处理耗时状态				
功能说明	获取因子处理耗时状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetRSDealTimeStatus				
入参说明					
出参说明					
<pre>{ "///": "状态, Start、Stop", "Status": "Stop" }</pre>					

6.20.17. GetRSDealTimeInfo 获取因子处理耗时信息

功能名称	GetRSDealTimeInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取因子处理耗时信息				
功能说明	获取因子处理耗时信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetRSDealTimeInfo				
入参说明					
出参说明					
<pre>{ "RSDealTimeInfo": [{ "///": "功能号", "FunctionId": "3333", "///": "执行总耗时 (us) ", "ExecSumTime": 111, "///": "最大执行耗时 (us) ", "MaxExecTime": 111, "///": "执行次数", "ExecCount": 3 }] }</pre>					

6.20.18. ClearRSDealTimeInfo 清空因子处理耗时信息

功能名称	ClearRSDeaTimeInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	清空因子处理耗时信息				
功能说明	清空因子处理耗时信息				

访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_ClearRSDealTimeInfo
入参说明	
出参说明	
<pre>{ "ClearRSDealTimeInfo": "OK" }</pre>	

6.20.19. SetMdbTrace 设置Mdb时延跟踪

功能名称	SetMdbTrace	版本号	V1.0.5.4	更新日期	20230823
功能备注	设置Mdb时延跟踪				
功能说明	设置Mdb时延跟踪(当Trace由0变为1/2时，将会清空过去生成的时延分析文件)				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_SetMdbTrace&Trace=0				
入参说明					
<pre>{ "Trace": "0" }</pre>					
出参说明					
<pre>{ "Trace": "0" }</pre>					

6.20.20. SetMdbSqlCacheSizeLimit 设置每个事务控制器的Sql缓存条数上限

功能名称	SetMdbSqlCacheSizeLimit	版本号	V1.0.5.4	更新日期	20241220
功能备注	设置每个事务控制器的Sql缓存条数上限				
功能说明	设置每个事务控制器的Sql缓存条数上限				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_SetMdbSqlCacheSizeLimit&SqlCacheSizeLimit=111				

入参说明

```
{  "///": "设置每个事务控制器的Sql缓存条数上限，支持范围：[0, intmax]，0表示关闭Sql缓存；由于前端传入的是字符串，内部会转成整型值，不以数值开始的字符串会转化为0",  "SqlCacheSizeLimit": "111"}

```

出参说明

```
{  "SqlCacheSizeLimit": "111"}

```

6.20.21. GetMdbTraceInfo 获取Mdb时延跟踪信息

功能名称	GetMdbTraceInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取Mdb时延跟踪信息				
功能说明	获取Mdb时延跟踪信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbTraceInfo				
入参说明					
出参说明					
<pre>{ "Data": { "tcs": [{ "Id": 2, "trace": [{ "Method": "CreateRecord", "TopTimes": [47756] }] }] } }</pre>					

```

        "///<": "平均时间",
        "AveTime": 47756,
        "///<": "最小值",
        "Min": 47756,
        "///<": "标准差",
        "Stddev": 0,
        "///<": "从小到大排序，位于百分之十位置的数值，其他p开头的以此类推",
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "///<": "函数执行总次数",
        "Count": 1
    },
    {
        "Method": "GetNextSequence",
        "TopTimes": [
            600
        ],
        "AveTime": 600,
        "Min": 600,
        "Stddev": 0,
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "Count": 1
    },
    {
        "Method": "BeginTransaction",
        "TopTimes": [
            1841
        ],
        "AveTime": 1841,
        "Min": 1841,
        "Stddev": 0,
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "Count": 1
    },
    {
        "Method": "CommitTransaction",
        "TopTimes": [
            3564
        ],
        "AveTime": 3564,
        "Min": 3564,
        "Stddev": 0,
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "Count": 1
    },
    {
        "Method": "BeginWriteTransaction",
        "TopTimes": [
            145
        ],
        "AveTime": 145,
        "Min": 145,
        "Stddev": 0,
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "Count": 1
    },
    {
        "Method": "CommitWriteTransaction",
        "TopTimes": [
            41871
        ],
        "AveTime": 41871,
        "Min": 41871,
        "Stddev": 0,
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "Count": 1
    },
    {
        "Method": "CreateSavePoint",
        "TopTimes": [
            139
        ],
        "AveTime": 139,
        "Min": 139,
        "Stddev": 0,
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "Count": 1
    },
    {
        "Method": "DeleteSavePoint",
        "TopTimes": [
            64
        ],
        "AveTime": 64,
        "Min": 64,
        "Stddev": 0,
        "p10": -1,
        "p50": -1,
        "p90": -1,
        "p95": -1,
        "p99": -1,
        "Count": 1
    }
]

```

```
        "p95": -1,
        "p99": -1,
        "Count": 1
    }
}
]
```

6.20.22. GetMdbAddressSpaceInfo 获取Mdb地址空间信息

功能名称	GetMdbAddressSpaceInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取Mdb地址空间信息				
功能说明	获取Mdb地址空间信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbAddressSpaceInfo				
入参说明					
出参说明					
<pre>{ "AddressSpaceInfo": { "//": "地址空间范围", "Range": "0x100000000000-0x900000000000", "//": "地址空间大小 (GB)", "TotalGB": 8192 } }</pre>					

6.20.23. GetMdbLongTimeTransactionHistory 获取超长事务告警历史记录

功能名称	GetMdbLongTimeTransactionHistory	版本号	V1.0.5.4	更新日期	20230823
功能备注	最多存储10000条超长事务告警的信息，重启后告警信息清空				
功能说明	获取超长事务告警历史记录				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbLongTimeTransactionHistory&Count=30				
入参说明					
<pre>{ "///": "查询个数，默认值100", "Count": "30"}</pre>					
出参说明					

<pre>{ "/*": "内存数据库名称", "DbName": "uftmdb_zhangjh", "/*": "慢事务触发的总次数", "SlowTransactionCount": 23, "/*": "超长事务信息", "LongTimeTransactionInfo": [{ "/*": "表示事务耗时超时是提交还是回滚，分别对应Commit和Rollback", "TransactionType": "Commit", "/*": "事务控制器ID", "TransCtrlID": 0, "/*": "该事务开始时刻的时间戳", "BeginTime": 1713768225, "/*": "事务执行时间，单位毫秒", "UseTimeMS": 3000, "/*": "事务关联的功能号", "FunctionID": 900001, "/*": "事务号，注意：假如TransactionType为Commit，那么本字段表示当前事务的事务号", "TransactionNo": 2, "/*": "本事务中进行过操作的所有表，数组形式（以此表是否在Undo日志中出现为准）", "Tables": [{ "/*": "表名", "Name": "contract", "/*": "表号", "Category": 101 }] }, { "/*": "表示事务耗时超时是提交还是回滚，分别对应Commit和Rollback", "TransactionType": "Rollback", "/*": "事务控制器ID", "TransCtrlID": 0, "/*": "该事务开始时刻的时间戳", "BeginTime": 1713773594, "/*": "事务执行时间，单位毫秒", "UseTimeMS": 1022, "/*": "事务关联的功能号", "FunctionID": 900001, "/*": "事务号，注意：假如TransactionType为Rollback，由于回滚不分配事务号，取AdminFileHead中的下一事务号，表示大致位置", "TransactionNo": 19, "/*": "本事务中进行过操作的所有表，数组形式（以此表是否在Undo日志中出现为准）", "Tables": [{ "/*": "表名", "Name": "contract", "/*": "表号", "Category": 101 }, { "Name": "entrust", "Category": 103 }] }] }</pre>	
功能说明	设置Mdb权限校验，此功能针对单个节点生效，开启之后，对于不支持的SQL语句会拒绝执行并报错，在通过APM调用SQL时会进行鉴权
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=uftmdb_data_SetMdbAccessControl&AccessControl=0
入参说明	
<pre>{ "/*": "表示是否开启权限校验总开关，默认开启。0：关闭权限校验， 1：开启权限校验", "AccessControl": "0" }</pre>	
出参说明	
<pre>{ "AccessControl": "0" }</pre>	

6.20.25. GetExternErrorInfo 获取业务周边外部错误号映射信息

功能名称	GetExternErrorInfo	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取LS服务功能号和内部错误号到业务周边外部错误号的映射信息				
功能说明	获取LS服务功能号和内部错误号到业务周边外部错误号的映射信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=uftmdb_data_GetExternErrorInfo				
入参说明					
出参说明					

```
{
  "ExternErrorInfo": [
    {
      "FunctionID": 339653,
      "ErrorNo": 144435,
      "ExternCode": -60
    },
    {
      "FunctionID": 339307,
      "ErrorNo": -61,
      "ExternCode": -61
    }
  ]
}
```

6.20.26. SetMdbMaintenance 设置数据库运维功能开关

功能名称	SetMdbMaintenance	版本号	V1.0.5.4	更新日期	20230823
功能备注	设置数据库运维功能开关				
功能说明	数据库运维功能开关，1表示开启功能，0表示关闭功能，默认为1开启。当开启功能时：可保持表创建和数据库清表的并发安全。(1)先清表再对这张表插入，会报错；先插入记录再清表，会等待插入操作完成再执行清表。清表时认为数据库在运维中，运维过程中数据库无法处理事务。(2)创建表过程中认为数据库在运维中，运维过程中数据库无法处理事务。				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_SetMdbMaintenance&EnableMaintenance=0				
入参说明					
<pre>{ "///": "数据库运维功能开关，默认开启。0--关闭数据库运维功能 1--开启数据库运维功能", "EnableMaintenance": "0"}</pre>					
出参说明					
<pre>{ "EnableMaintenance": "0"}</pre>					

6.20.27. GetMdbSqlExecInfo 获取慢sql执行统计信息

功能名称	GetMdbSqlExecInfo	版本号	V1.0.5.4	更新日期	20241001
功能备注	获取慢sql执行统计信息				
功能说明	获取慢sql执行统计信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbSqlExecInfo&Count=30				
入参说明					
<pre>{ "Count": "30", "MinExcuteNum": 1, "MaxExcuteNum": 10 }</pre>					
出参说明					
<pre>{ "DbName": "dbtest", "MaxSlowSQLNumInTC": 10000, "ExecSqlTotalCount": 10, "ExecSqlSuccCount": 0, "ExecSqlFailCount": 10, "SlowSQLCount": 10, "SqlExecInfo": [{ "Sql": "xxx", "AvgExecTimeMicro": 500, "MaxExecTimeMicro": 200, "MinExecTimeMicro": 100, "ExecTime": 10, "FailTime": 1, "MissIndexTime": 2 }] }</pre>					

6.20.28. GetMdbSqlExecDetail 获取慢sql执行明细

功能名称	GetMdbSqlExecDetail	版本号	V1.0.5.4	更新日期	20241001
功能备注	获取慢sql执行明细				
功能说明	获取慢sql执行明细				

访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbSqlExecDetail&Count=30
入参说明	
<pre>{ "Count": "30", "SQL": "select * from Table;" }</pre>	
出参说明	
<pre>{ "DbName": "dbtest", // 数据库名称 "SqlExecDetail": [// 数组形式 { "Sql": "xxx", // 执行的sql语句 "ExecDetail": [// 数组形式 { "ArgsIn": [// 输入参数, 数组形式 "arg1=v1", // 参数1和值, '='分隔 "arg2=v21" // 参数2和值, '='分隔], "ArgsOut": [// 输出参数, 数组形式 "arg1=v1" // 参数1和值, '='分隔], "ExecTimeMicro": 100, // 执行耗时(单位微秒) "BeginTime": "1001.135034.131", // 执行时刻(形式: mmdd.hhmmss.mmm) "ErrorNo": 0, // 执行结果: 0表示成功, 其他表示错误号 "FuncId": 310053, // 功能号 "IsHitCache": false, // 是否命中Sql缓存(true表示命中) "IsHitIndex": true, // 是否命中索引(true表示命中) "HitIndexInfo": [// 命中索引信息, IsHitIndex为true时存在 { "Category": 100, // 表号 "TableName": "tablexxx", // 表名 "IndexNo": 0, // 索引编号 "IndexName": "Indexxxx", // 索引名称 "IndexField": "studentID", // 索引字段 "TableRecordCount": 10000 // 表记录数量 }], "SQL_ScanRecordCount": 200, // 扫描记录数量 "TransCtrlID": 1 // 事务控制器ID }] }] }</pre>	

6.20.29. GetMdbSqlCacheDetail 获取Sql缓存明细

功能名称	GetMdbSqlCacheDetail	版本号	V1.0.5.4	更新日期	20241220
功能备注	获取Sql缓存明细				
功能说明	获取Sql缓存明细				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbSqlCacheDetail&Count=100				
入参说明					
<pre>{ "Count": "100"}</pre>					
出参说明					
<pre>{ "SqlCacheDetail": [{ "SQL": "select * from `GSqlTableRecord` limit 11;\n", // 执行的sql语句 "TransCtrlID": 0, // 事务控制器ID "UseTimes": 2, // 缓存被使用的次数 "CreateTime": "2024-12-20 15:56:16.712", // 缓存创建时间 "LastTime": "2024-12-20 15:56:16.713" // 缓存最后使用时间 }]}</pre>					

6.20.30. SetMdbSqlHistoryThreshold 设置Mdb记录慢Sql阈值

功能名称	SetMdbSqlHistoryThreshold	版本号	V1.0.5.4	更新日期	20241002
功能备注	设置Mdb记录慢Sql阈值				
功能说明	设置Mdb记录慢Sql阈值				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_SetMdbSqlHistoryThreshold&SqlHistoryThreshold=100000				
入参说明					

<pre>{ "///": "记录慢SQL阈值，单位微秒，默认值100000", "SqlHistoryThreshold": "100000" }</pre>
出参说明
<pre>{ "SqlHistoryThreshold": 100000 }</pre>

6.20.31. GetSqlFunctionID 获取MDB SQL调用功能号

功能名称	GetSqlFunctionID	版本号	V1.0.15.30	更新日期	20241113
功能备注	获取MDB SQL调用功能号				
功能说明	获取MDB SQL调用功能号				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&FuncName=utfmdb_data_GetSqlFunctionID				
入参说明					
无					
出参说明					
配置了SqlFunctions(可选),配置优先级: "SqlFunctions"> "SqlFunction"					
<pre>{ "///": "保证兼容性，FunctionID展示FunctionIDs中的第0个功能号", "FunctionID": 508902, "FunctionIDs": [508902, 508903]}</pre>					

6.20.32. GetLoadTableLevel 获取内存表加载的优先级

功能名称	GetLoadTableLevel	版本号	V1.0.5.4	更新日期	20250319
功能备注	获取内存表加载的优先级				
功能说明	获取内存表加载的优先级				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&InstanceNo=0&FuncName=utfmdb_data_GetLoadTableLevel				
入参说明					
无					
出参说明					
<pre>{ "ReloadTableInfo": [{ "///": "表名", "TableName": "father1", "///": "表加载的优先级，数字越小优先级越高", "ReloadLevel": 0, "///": "包含关系的父表 (1:1、1:n)", "ConParent": [], "///": "包含关系的子表 (1:1、1:n)", "ConChild": ["son1", "son1", "son1", "son1"], "///": "引用关系的父表 (n:1、n:n)", "RefParent": [], "///": "引用关系的子表 (n:1、n:n)", "RefChild": [] }]}</pre>					

6.20.33. GetSequenceNoInfo 获取序列号线程|进程|远程的对应关系

功能名称	GetSequenceNoInfo	版本号	V1.0.5.4	更新日期	20250402
功能备注	获取序列号线程，进程，远程的对应关系				
功能说明	获取序列号线程，进程，远程的对应关系				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&InstanceNo=0&FuncName=utfmdb_data_GetSequenceNoInfo				
入参说明					
无					
出参说明					

```
{
  "/*": "序列号线程，进程，远程的对应关系",
  "SequenceNoInfo": [
    {
      "/*": "序列号线程表号",
      "ThreadSeqType": 1,
      "/*": "序列号进程表号",
      "ProcessSeqType": 1001,
      "/*": "序列号远程表号，为0表示不存在这一项",
      "iRpcSeqSetNo": 0
    },
    {
      "ThreadSeqType": 2,
      "ProcessSeqType": 1002,
      "iRpcSeqSetNo": 0
    },
    {
      "ThreadSeqType": 3,
      "ProcessSeqType": 1003,
      "iRpcSeqSetNo": 2003
    }
  ],
  "/*": "是否在重演中",
  "ReplayMode": false,
  "/*": "是否支持可重演",
  "EnableReproducible": false,
  "/*": "申请序列号段回调次数",
  "AllocSequenceRangeCount": 1136,
  "/*": "申请序列号段回调从HashMap中获取的次数",
  "AllocFromHashMapCount": 0,
  "/*": "申请序列号段回调从进程中获取的次数",
  "AllocFromProcessCount": 13,
  "/*": "申请序列号段回调从远程中获取的次数",
  "AllocFromRpcCount": 1123,
  "/*": "序列号段分配器相关信息",
  "SequenceRangeAllocatorArray": [
    {
      "/*": "最近一次从HashMap中获取号段相关信息",
      "LastHashMapAllocInfo": {
        "/*": "获取到的最小值",
        "LastHashMapAllocResultMin": 0,
        "/*": "获取到的最大值",
        "LastHashMapAllocResultMax": 0,
        "/*": "错误号",
        "LastHashMapAllocResultErrorNo": 0
      },
      "/*": "最近一次从进程中获取号段相关信息",
      "LastProcessAllocInfo": {
        "/*": "获取到的最小值",
        "LastProcessAllocResultMin": 0,
        "/*": "获取到的最大值",
        "LastProcessAllocResultMax": 0,
        "/*": "错误号",
        "LastProcessAllocResultErrorNo": 0
      },
      "/*": "最近一次从远程中获取号段相关信息",
      "LastRpcAllocInfo": {
        "/*": "获取到的最小值",
        "LastRpcAllocResultMin": 51400,
        "/*": "获取到的最大值",
        "LastRpcAllocResultMax": 51500,
        "/*": "错误号",
        "LastRpcAllocResultErrorNo": 0
      },
      "/*": "当前分配器的状态，可取值为normal_status, write_files, rpc_call, illegal_status",
      "AllocatorStatus": "normal_status"
    }
  ]
}
```

6.20.34. SetMdbLongTimeTransactionThreshold 设置内存数据库慢事务的时间阈值

功能名称	SetMdbLongTimeTransactionThreshold	版本号	V1.0.5.4	更新日期	20250811
功能备注	设置内存数据库慢事务的时间阈值				
功能说明	设置内存数据库慢事务的时间阈值				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufmdb_data_SetMdbLongTimeTransactionThreshold				
入参说明					
<pre>{ "/*": "慢事务阈值，单位微秒，默认值500000，传入int32范围内且大于0的值", "SlowTransactionThreshold": 500000 }</pre>					
出参说明					
<pre>{ "/*": "慢事务阈值，单位微秒", "LongTimeTransactionThreshold": 500000 }</pre>					

6.20.35. ExportTableToCsvBySQL 导出内存数据库的表记录到CSV文件中

功能名称	ExportTableToCsvBySQL	版本号	V1.0.5.4	更新日期	20250725
功能备注	导出内存数据库的表记录到CSV文件中				
功能说明	导出内存数据库的表记录到CSV文件中				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufmdb_data_ExportTableToCsvBySQL				
入参说明					

<pre>{ "Request": { "TaskID": "wq9e032e2ejwo32ej0ej92", // 由APM生成并传入，唯一标识一个任务ID "Tables": [{ "Table": "u_client", // 此字段传入表名 "SQL": "select * from uses_entrust", // 此字段传入SQL语句，可选传入 }, { "Table": "u_fund_real", // 此字段传入表名，必须传入 }] } }</pre>
出参说明
<pre>{ "TaskID": "wejdsa0dj3jd0djsjdoqwjd", // TaskID "nErrorNo": 0, // 错误号，为0表示成功 "ExportInfo": [{ "Table": "u_client", // 表名 "FilePath": "/home/xxx/u_client-1753421164696450066.csv" // 文件路径 }, { "Table": "u_fund_real", "FilePath": "/home/xxx/u_fund_real-1753421164696450066.csv" }] }</pre>

6.20.36. ExportTableToCsvStatus 查看导出任务的进度和状态

功能名称	ExportTableToCsvStatus	版本号	V1.0.5.4	更新日期	20250725
功能备注	查看导出任务的进度和状态				
功能说明	查看导出任务的进度和状态				
访问链接	http://ppport/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufmdb_data_ExportTableToCsvStatus				
入参说明					
<pre>{ "FuncName": "ExportTableToCsvStatus", "Param": { "TaskID": "x23dqwe92039e9032ie0923ei" // 任务ID } }</pre>					
出参说明					
<pre>{ "TaskID": "wejdsa0dj3jd0djsjdoqwjd", "ExportToFileThreadCount": 4, // 表示当前配置的导出线程个数 "RunningExportThreadCount": 0, // 表示当前有多少个导出线程正在执行 "ExportTableToCsvStatus": [{ "Table": "u_client", // 表名 "FilePath": "/home/xxx/u_client-1753421164696450066.csv", // 文件路径 "WriteRecordCount": 3777198, // 总共写了多少条记录 "status": "finished", // 状态，finished已完成，exporting正在导出，error表示出错，waiting表示等待导出 "ErrorNo": 0, // 错误号，0表示成功 "TimeCostNs": 4729753119, // 此表导出的耗时，单位ns "ErrorMsg": "" // 错误信息 }, { "Table": "u_fund_real", "FilePath": "/home/xxx/u_client-1753421164696450066.csv", "WriteRecordCount": 1000002, "status": "finished", "ErrorNo": 0, "TimeCostNs": 2282873480, "ErrorMsg": "" }] }</pre>					

6.20.37. GetMdbTableDefineInfo 获取 MDB 的表定义信息

功能名称	GetMdbTableDefineInfo	版本号	V1.0.515.1067	更新日期	20251128
功能备注	获取 MDB 的表定义信息				
功能说明	获取 MDB 的表定义信息				
访问链接	http://ppport/ldp?Pretty=1&PluginName=ldp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMdbTableDefineInfo				
入参说明					
<pre>{ "/*": "指定表名的字段信息(不区分大小写)", "TableName": "U_CLIENT" }</pre>					
说明:					

- 不传递表名或者传递空串则默认展示所有表的字段信息。
- 目前只支持传递一个表名

出参说明

```
{
  "Tables": [
    {
      // 表名
      "TableName": "xxx",
      // 表号
      "Category": 111,
      "Fields": [
        {
          // 字段名
          "FieldName": "branch_no",
          // 字段类型, "TINYINT"对应int8_t
          "FieldType": "TINYINT",
          // 字段类型对应字节数, int8_t字节=1
          "FieldByte": 1,
          // 浮点数精度, 未设置时为255
          "FieldScale": 255
        },
        {
          // 字段名
          "FieldName": "msg",
          // 字段类型, "VARCHAR"对应可变长度字符串
          "FieldType": "VARCHAR",
          // 字段类型对应字节数, VARCHAR的最大长度
          "FieldByte": 1024,
          // 浮点数精度, 未设置时为255 (虽然VARCHAR不适用此属性, 但保留以保持一致性)
          "FieldScale": 255
        },
        {
          // 字段名
          "FieldName": "age",
          // 字段类型, "SMALLINT"对应int16_t
          "FieldType": "SMALLINT",
          // 字段类型对应字节数, int16_t字节=2
          "FieldByte": 2,
          // 浮点数精度, 未设置时为255
          "FieldScale": 255
        },
        {
          // 字段名
          "FieldName": "salary",
          // 字段类型, "INT"对应int32_t
          "FieldType": "INT",
          // 字段类型对应字节数, int32_t字节=4
          "FieldByte": 4,
          // 浮点数精度, 未设置时为255
          "FieldScale": 255
        },
        {
          // 字段名
          "FieldName": "id",
          // 字段类型, "BIGINT"对应int64_t
          "FieldType": "BIGINT",
          // 字段类型对应字节数, int64_t字节=8
          "FieldByte": 8,
          // 浮点数精度, 未设置时为255
          "FieldScale": 255
        },
        {
          // 字段名
          "FieldName": "unsigned_age",
          // 字段类型, "USMALLINT"对应uint16_t
          "FieldType": "USMALLINT",
          // 字段类型对应字节数, uint16_t字节=2
          "FieldByte": 2,
          // 浮点数精度, 未设置时为255
          "FieldScale": 255
        },
        {
          // 字段名
          "FieldName": "unsigned_id",
          // 字段类型, "UBIGINT"对应uint64_t
          "FieldType": "UBIGINT",
          // 字段类型对应字节数, uint64_t字节=8
          "FieldByte": 8,
          // 浮点数精度, 未设置时为255
          "FieldScale": 255
        },
        {
          // 字段名
          "FieldName": "price",
          // 字段类型, "FLOAT"对应float
          "FieldType": "FLOAT",
          // 字段类型对应字节数, float字节=4
          "FieldByte": 4,
          // 浮点数精度, 保留2位小数
          "FieldScale": 2
        },
        {
          // 字段名
          "FieldName": "weight",
          // 字段类型, "DOUBLE"对应double
          "FieldType": "DOUBLE",
          // 字段类型对应字节数, double字节=8
          "FieldByte": 8,
          // 浮点数精度, 保留3位小数
          "FieldScale": 3
        },
        {
          // 字段名
          "FieldName": "name",
          // 字段类型, "CHAR"对应定长字符串
          "FieldType": "CHAR",
          // 字段类型对应字节数, 例如定长50个字符
          "FieldByte": 50,
          // 浮点数精度, 未设置时为255 (虽然CHAR不适用此属性, 但保留以保持一致性)
          "FieldScale": 255
        }
      ]
    }
  ],
}
```

```
{
    // 字段名
    "FieldName": "ch",
    // 字段类型, "CHAR"对应定长字符串
    "FieldType": "CHAR",
    // 字段类型对应字节数, 字节 =0 表示类型为 char
    "FieldByte": 0,
    // 浮点数精度, 未设置时为255 (虽然CHAR不适用此属性, 但保留以保持一致性)
    "FieldScale": 255
}
]
```

管理功能项出参说明

名称	类型	说明	备注
TableName	字符串	表名	内存数据库的表名
Category	整型	表号	内存数据库的表名对应的表号, 唯一对应
FieldName	字符串	字段名	表中的字段名称
FieldType	字符串	字段类型	对应的数据类型, 如"TINYINT"、"VARCHAR"等 "TINYINT" (int8_t) "SMALLINT" (int16_t) "INT" (int32_t) "BIGINT" (int64_t) "UTINYINT" (uint8_t) "USMALLINT" (uint16_t) "UINT" (uint32_t) "UBIGINT" (uint64_t) "FLOAT" (float) "DOUBLE" (double) "CHAR" (char\char 数组) "VARCHAR" (varchar): 定义的最大字节长度
FieldByte	整型	字节大小	字段类型对应字节数 "TINYINT" (int8_t): 1字节 "SMALLINT" (int16_t): 2字节 "INT" (int32_t): 4字节 "BIGINT" (int64_t): 8字节 "UTINYINT" (uint8_t): 1字节 "USMALLINT" (uint16_t): 2字节 "UINT" (uint32_t): 4字节 "UBIGINT" (uint64_t): 8字节 "FLOAT" (float): 4字节 "DOUBLE" (double): 8字节 "CHAR" (char\char 数组): - 字符(char)的长度 恒=0 - 字符数组的长度 !=0, 例如`char [1]`的`FieldByte`为1 "VARCHAR" (varchar): 定义的最大字节长度

6.20.38. GetFileRecordProgress 获取从文件上场，从文件下场以及从文件二次上场的进度信息

功能名称	GetFileRecordProgress	版本号	V1.0.5.4	更新日期	20251224
功能备注	获取从文件上场，从文件下场以及从文件二次上场的进度信息				
功能说明	获取从文件上场，从文件下场以及从文件二次上场的进度信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&InstanceNo=0&FuncName=ufimdb_data_GetFileRecordProgress				
入参说明					

<pre>{ "FuncName": "GetFileRecordProgress", "Param": { "Type": "Load", // 表示查询的是文件上场,下场还是二次上场的进度,分别传入Load,Export和Reload,此字段必须传入 "Table": "entrust" // 内存表名称,此字段若不传入,则展示所有执行过当前Type选中操作的表的信息 } }</pre>
出参说明
<pre>{ "Type": "Load", // 表示当前查询的类型 "Tables": [// 展示目标表的执行情况 { "Table": "entrust", // 表名 "ThreadCount": 4, // 线程数量 "ThreadInfo": [// 每个线程的状态 { "nErrorNo": 0, // 错误号,0表示没有出错 "status": 2, // 执行状态,0表示运行中,1表示出 // 错,2表示成功 "strFilePath": "/home/jiangyl/load/*entrust*", // 文件的路径 "uProcessRecordCount": 0, // 处理的记录数量 "uTimeCountNs": 8537119 // 执行的耗时,单位纳秒 }, { "nErrorNo": 0, "status": 2, "strFilePath": "/home/jiangyl/load/*entrust*", "uProcessRecordCount": 1, "uTimeCountNs": 9324431 }, { "nErrorNo": 0, "status": 2, "strFilePath": "/home/jiangyl/load/*entrust*", "uProcessRecordCount": 1, "uTimeCountNs": 9149884 }, { "nErrorNo": 0, "status": 2, "strFilePath": "/home/jiangyl/load/*entrust*", "uProcessRecordCount": 1, "uTimeCountNs": 6266186 }] }, { "Table": "contract", "ThreadCount": 1, "ThreadInfo": [{ "nErrorNo": 10002, "status": 1, "strFilePath": "/home/jiangyl/trunk-env/workspace/./LoadCsvFiles/asdasdasd/ddd/*contract*", "uProcessRecordCount": 0, "uTimeCountNs": 102769 }] }] }</pre>

6.20.39. GetMDBTableIndexInfo 获取 MDB 的表对应索引信息

功能名称	GetMDBTableIndexInfo	版本号	V4.2501.0.1	更新日期	20260113
功能备注	获取 MDB 的表对应索引信息				
功能说明	获取 MDB 的表对应索引信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMDBTableIndexInfo				
入参说明					
<pre>{ "///": "指定表名的索引信息, 字符串类型", "TableName": "mytable", "///": "指定对应表的索引名, 字符串类型, 不区分大小写", "IndexName": "idx1_Key", "///": "分页参数 - 每页记录数, 整型int64_t, 默认50, 无效值则采用默认值, 有效范围: [1, INT64_MAX]", "PageSize": 50, "///": "分页参数 - 页码, 整型int64_t, 默认1, 无效值则采用默认值, 有效范围: [1, INT64_MAX]", "PageIndex": 1 }</pre>					
说明:					
- 不传递表名或者传递空串则默认展示当前页的表索引信; - 指定索引名只在指定表名的场景下才生效, 不指定索引名则默认展示该表的所有索引; - 若未传入指定表名, 直接传入索引名则报错; - 目前只支持传递单个表名、单个索引名; - 不支持展示IIF_Array、IIF_Hash此类索引(此类属于外部注册索引, 暂无使用); - 在指定表名或指定表名+索引名场景下, 忽略分页参数, 均为默认1					
出参说明					

```

{
    // 分页信息 - 总记录数
    "TotalCount": 4,
    // 分页信息 - 当前页码
    "CurrentPage": 1,
    // 分页信息 - 每页记录数
    "PageSize": 4,
    // 表索引信息列表
    "Tables": [
        {
            // 表名
            "TableName": "S_MgrTable1",
            // 表号
            "Category": 1,
            // 索引列表, 当前表没有索引
            "Indexes": []
        },
        {
            // 表名
            "TableName": "S_MgrTable2",
            // 表号
            "Category": 2,
            // 索引列表, 当前表没有索引
            "Indexes": []
        },
        {
            // 表名
            "TableName": "S_MgrTable3",
            // 表号
            "Category": 3,
            // 索引列表, 当前表没有索引
            "Indexes": []
        },
        {
            // 表名
            "TableName": "S_MgrTable4",
            // 表号
            "Category": 4,
            // 索引列表
            "Indexes": [
                {
                    // 索引名
                    "IndexName": "idx_set_int16_value",
                    // 索引类型, "Set"表示唯一索引
                    "IndexType": "Set",
                    // 索引字段列表, "int16_value asc"表示按 int16_value 字段升序排列
                    "IndexFieldList": "int16_value asc"
                },
                {
                    // 索引名
                    "IndexName": "idx_artset_id_name",
                    // 索引类型, "ArtSet"表示ART唯一索引
                    "IndexType": "ArtSet",
                    // 索引字段列表, "idx_set_id desc"表示按 idx_set_id 字段降序排列
                    "IndexFieldList": "idx_set_id desc"
                },
                {
                    // 索引名
                    "IndexName": "idx_artmutiset_id2_name",
                    // 索引类型, "MultiSet"表示非唯一索引
                    "IndexType": "MultiSet",
                    // 索引字段列表, "idx_mutiset_id2 asc"表示按 idx_mutiset_id2 字段升序排列
                    "IndexFieldList": "idx_mutiset_id2 asc"
                },
                {
                    // 索引名
                    "IndexName": "idx_artmutiset_id2_name",
                    // 索引类型, "ArtMultiSet"表示ART非唯一索引
                    "IndexType": "ArtMultiSet",
                    // 索引字段列表, "idx_mutiset_id2 asc"表示按 idx_mutiset_id2 字段升序排列
                    "IndexFieldList": "idx_mutiset_id2 asc"
                },
                {
                    // 索引名
                    "IndexName": "idx_levelset_name",
                    // 索引类型, "LevelSet"表示分级唯一索引
                    "IndexType": "LevelSet",
                    // 索引字段列表, 多个字段组合索引用竖线分隔, 每个组合用逗号分隔
                    "IndexFieldList": "idx_set_id_1 asc,idx_set_id_2 asc,idx_set_id_3 asc | idx_set_id_1
asc,idx_set_id_2 asc | idx_set_id_2 asc,idx_set_id_3 asc | idx_set_id_1 asc,idx_set_id_3 asc"
                },
                {
                    // 索引名
                    "IndexName": "idx_artlevelset_name",
                    // 索引类型, "ArtLevelSet"表示ART分级唯一索引
                    "IndexType": "ArtLevelSet",
                    // 索引字段列表, 多个字段组合索引用竖线分隔, 每个组合用逗号分隔
                    "IndexFieldList": "idx_set_id_1 asc,idx_set_id_2 asc,idx_set_id_3 asc | idx_set_id_1
asc,idx_set_id_2 asc | idx_set_id_2 asc,idx_set_id_3 asc | idx_set_id_1 asc,idx_set_id_3 asc"
                },
                {
                    // 索引名
                    "IndexName": "idx_levelmutiset_name",
                    // 索引类型, "LevelMultiSet"表示ART分级非唯一索引
                    "IndexType": "LevelMultiSet",
                    // 索引字段列表, 多个字段组合索引用竖线分隔, 每个组合用逗号分隔
                    "IndexFieldList": "idx_mutiset_id_1 asc,idx_mutiset_id_2 asc,idx_mutiset_id_3
asc,idx_mutiset_id_4 asc | idx_mutiset_id_1 asc,idx_mutiset_id_2 asc | idx_mutiset_id_3 asc,idx_mutiset_id_4 asc"
                },
                {
                    // 索引名
                    "IndexName": "idx_artlevelmutiset_name",
                    // 索引类型, "ArtLevelMultiSet"表示ART分级非唯一索引
                    "IndexType": "ArtLevelMultiSet",
                    // 索引字段列表, 多个字段组合索引用竖线分隔, 每个组合用逗号分隔
                    "IndexFieldList": "idx_mutiset_id_1 asc,idx_mutiset_id_2 asc,idx_mutiset_id_3
asc,idx_mutiset_id_4 asc | idx_mutiset_id_1 asc,idx_mutiset_id_2 asc | idx_mutiset_id_3 asc,idx_mutiset_id_4 asc"
                }
            ]
        }
    ]
}

```

6.20.40. GetTransactionNo 获取事务号信息

功能名称	GetTransactionNo	版本号	V1.0.0.0	更新日期	20260210
功能备注	获取事务号信息，注意获取的所有事务号都为下一事务号				
功能说明	获取事务号信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_bizproc&InstanceNo=0&FuncName=ufimdb_data_GetTransactionNo				
入参说明					
无					
出参说明					
<pre>{ "///": "当前是否为网格模式，非网格模式仅在TransactionNo展示事务号", "IsPartitionMode": true, "///": "下一事务号，网格模式展示主通道事务号，主备模式展示主机事务号，备机展示重放事务号", "TransactionNo": 3, "///": "网格模式各节点事务号信息，主备模式下为空", "TransactionNoInfo": [{ "///": "节点号", "NodeId": 0, "///": "下一个事务号", "TransactionNo": 3 }]}</pre>					

6.20.41. GetMonitorField 获取监控字段

功能名称	GetMonitorField	版本号	V1.0.0.0	更新日期	20260415
功能备注	获取监控字段				
功能说明	获取监控字段				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp Bizproc&InstanceNo=0&FuncName=ufimdb_data_GetMonitorField				
入参说明					
无					
出参说明					
<pre>{ "PackerPoolLeakCount": 0 }</pre>					

6.21. 前置消息转换组件 front_transfer

6.21.1. GetMsgConvertStatistics 获取消息转换统计信息

功能名称	GetMsgConvertStatistics	版本号	V1.0.5.4	更新日期	20230823
功能备注	获取消息转换统计信息				
功能说明	获取消息转换统计信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front_transfer&InstanceNo=0&FuncName=GetMsgConvertStatistics				
入参说明					
出参说明					

```
{
  "":"/": "消息转换统计信息",
  "MsgConvertStatistics": [
    {
      "":"/": "功能号",
      "FuncNo": 34567,
      "":"/": "转换顺序规则: 0: 按照消息的顺序转换, 1: 按照配置文件中定义的顺序转换",
      "TransByName": 0,
      "":"/": "是否携带token: 0: 没有携带token, 1: 携带token",
      "HasToken": 0,
      "":"/": "是否无需解包: 0: 转换需要解包, 1: 转换不需要解包",
      "NoPack": 0,
      "":"/": "LDP多结果集是否转换嵌套包: 0: 多结果集转多结果集, 1: 多结果集转嵌套包, 2: 第一个结果集平铺, 后面的结果集转嵌套包",
      "Nesting": 0,
      "":"/": "LDP消息转ESB消息总计数",
      "Ldp2EsbTotal": 0,
      "":"/": "LDP消息转ESB消息请求包成功数",
      "Ldp2EsbReqSucc": 0,
      "":"/": "LDP消息转ESB消息请求包失败数",
      "Ldp2EsbReqFail": 0,
      "":"/": "LDP消息转ESB消息应答包成功数",
      "Ldp2EsbRspSucc": 0,
      "":"/": "LDP消息转ESB消息应答包失败数",
      "Ldp2EsbRspFail": 0,
      "":"/": "LDP消息转ESB消息错误包成功数",
      "Ldp2EsbErrSucc": 0,
      "":"/": "LDP消息转ESB消息错误包失败数",
      "Ldp2EsbErrFail": 0,
      "":"/": "ESB消息转LDP消息总计数",
      "Esb2LdpTotal": 0,
      "":"/": "ESB消息转LDP消息请求包成功数",
      "Esb2LdpReqSucc": 0,
      "":"/": "ESB消息转LDP消息请求包失败数",
      "Esb2LdpReqFail": 0,
      "":"/": "ESB消息转LDP消息应答包成功数",
      "Esb2LdpRspSucc": 0,
      "":"/": "ESB消息转LDP消息应答包失败数",
      "Esb2LdpRspFail": 0,
      "":"/": "ESB消息转LDP消息错误包成功数",
      "Esb2LdpErrSucc": 0,
      "":"/": "ESB消息转LDP消息错误包失败数",
      "Esb2LdpErrFail": 0,
      "":"/": "LdpPack消息转LdpStruct消息总计数",
      "LdpPack2LdpStructTotal": 0,
      "":"/": "LdpPack消息转LdpStruct消息请求包成功数",
      "LdpPack2LdpStructReqSucc": 0,
      "":"/": "LdpPack消息转LdpStruct消息请求包失败数",
      "LdpPack2LdpStructReqFail": 0,
      "":"/": "LdpPack消息转LdpStruct消息应答包成功数",
      "LdpPack2LdpStructRspSucc": 0,
      "":"/": "LdpPack消息转LdpStruct消息应答包失败数",
      "LdpPack2LdpStructRspFail": 0,
      "":"/": "LdpPack消息转LdpStruct消息错误包成功数",
      "LdpPack2LdpStructErrSucc": 0,
      "":"/": "LdpPack消息转LdpStruct消息错误包失败数",
      "LdpPack2LdpStructErrFail": 0,
      "":"/": "LdpStruct消息转LdpPack消息",
      "LdpStruct2LdpPackTotal": 0,
      "":"/": "LdpStruct消息转LdpPack消息请求包成功数",
      "LdpStruct2LdpPackReqSucc": 0,
      "":"/": "LdpStruct消息转LdpPack消息请求包失败数",
      "LdpStruct2LdpPackReqFail": 0,
      "":"/": "LdpStruct消息转LdpPack消息应答包成功数",
      "LdpStruct2LdpPackRspSucc": 0,
      "":"/": "LdpStruct消息转LdpPack消息应答包失败数",
      "LdpStruct2LdpPackRspFail": 0,
      "":"/": "LdpStruct消息转LdpPack消息错误包成功数",
      "LdpStruct2LdpPackErrSucc": 0,
      "":"/": "LdpStruct消息转LdpPack消息错误包失败数",
      "LdpStruct2LdpPackErrFail": 0
    }
  ]
}
```

6.22. RCM网络代理插件 ldp_rcm_agent

6.22.1. GetStatistics 获取RCM网络代理统计信息

功能名称	GetStatistics	版本号	V1.0.15.0	更新日期	20240604
功能备注	获取RCM网络代理统计信息				
功能说明	获取RCM网络代理统计信息				
访问链接	http://ip:port/ldp?PluginName=ldp_rcm_agent&FuncName=GetStatistics&Pretty=1				
入参说明					
Pretty=1 表示将json美化输出，为0表示不美化					
出参说明					
rcm_agent统计信息 { "///": "版本信息", "RcmAgentVersion": "ldp_rcm_agent V1.0.15.4", "///": "udp 接收端", "UdpReceivers": { "///": "udp 接收端数组", "Receivers": [{ "///": "本地绑定地址", "LocalAddress": "10.19.37.107:0", "///": "接收的组播地址端口", "MulticastAddress": "225.3.14.15:24220", "///": "本套接字发送统计", "Send Statics": { "///": "原始数据包个数", "ODATA": 8909, "///": "地址数据和本套接字" } }] } }					

```
        "RDATA": 0,
        "Heartbeat": 0,
        "NAK": 0,
        "SYN": 12,
        "FIN": 2,
        "SET": 6,
        "RST": 0,
        "Unknown": 0
    },
    "Recv Statics": {
        "ODATA": 0,
        "RDATA": 0,
        "Heartbeat": 8927,
        "NAK": 0,
        "SYN": 6,
        "FIN": 0,
        "SET": 6,
        "RST": 0,
        "Unknown": 0
    }
},
"ErrCount": 0,
"TxErrCount": 0,
"FromUnknown": 0,
"NoUMCount": 0,
"IncompleteRCM": 0,
"PktErrCount": 0
},
"//": "作为Tcp包发送端",
"TcpClient": {
    "Connections": [
        {
            "ServerAddress": "10.20.30.125",
            "ServerPort": 24086,
            "State": "Connected"
        }
    ],
    "ErrorCount": 0,
    "ShortCount": 0,
    "RxCount": 73069,
    "TxCount": 195507
},
"//": "作为Tcp包接收端",
"TcpServer": {
    "Address": "10.19.37.107",
    "Port": 24086,
    "RxCount": 2684086,
    "TxCount": 855105,
    "ErrorCount": 0,
    "TxErrorCount": 0,
    "ErrorPktCount": 0,
    "FullCount": 0
},
"//": "udp 发送端",
"UdpSender": {
    "Senders": [
        {
            "mcAddr": "225.3.14.7:24207",
            "SrcTransportId": 8,
            "DestTransportId": 8,
            "Send Statics": {
                "ODATA": 275650,
                "RDATA": 0,
                "Heartbeat": 2802,
                "NAK": 0,
                "SYN": 1,
                "FIN": 0,
                "SET": 1,
            }
        }
    ]
}
```

```

        "RST包个数",
        "RST": 0,
        "RST包个数",
        "Unknown": 0
    },
    "Recv Statics": {
        "原始数据包个数",
        "ODATA": 0,
        "数据包个数",
        "RDATA": 0,
        "心跳包个数",
        "HEARTBEAT": 0,
        "NAK包个数",
        "NAK": 0,
        "SYN包个数",
        "SYN": 0,
        "FIN包个数",
        "FIN": 0,
        "SET包个数",
        "SET": 0,
        "RST包个数",
        "RST": 0,
        "RST包个数",
        "Unknown": 0
    }
}
},
"接收出错包个数",
"RxErrCount": 0,
"发送出错包个数",
"TxErrCount": 0,
"包转发来源节点信息",
"NoUMCount": 0,
"包转发来源节点信息",
"PktErrCount": 0
},
"Agent转发配置项",
"RcmAgentCfgMgr": [
    {
        "普通主题转发项",
        "Key": "Zone2_#ucrt.result_biz_60_#Zone1",
        "包转发来源节点信息",
        "From": {
            "源Zone",
            "Zone": "Zone2",
            "源Transport编号",
            "TransportId": 41,
            "源主题名字",
            "Topic": "ucrt.result_biz",
            "源分区编号",
            "partitionNo": 60,
            "源组播地址",
            "Addr": "225.3.14.26",
            "源组播端口",
            "Port": 24240
        },
        "包转发目的节点信息",
        "To": {
            "目的Zone",
            "Zone": "Zone1",
            "目的Transport编号",
            "TransportId": 41,
            "目的主题名字",
            "Topic": "ucrt.result_biz",
            "目的分区编号",
            "partitionNo": 60,
            "目的组播地址",
            "Addr": "225.3.14.26",
            "目的组播端口",
            "Port": 24240
        }
    },
    {
        "集群内数据同步转发项",
        "Key": "Zone1_#bizA_#Zone2",
        "包转发来源节点信息",
        "From": {
            "源Zone",
            "Zone": "Zone1",
            "集群名称",
            "TierName": "bizA",
            "集群数据同步组播地址",
            "Addr": "225.3.14.103",
            "集群数据同步组播端口",
            "Port": 24403
        },
        "包转发目的节点信息",
        "To": {
            "目的Zone",
            "Zone": "Zone2",
            "集群名称",
            "TierName": "bizA",
            "集群数据同步组播地址",
            "Addr": "225.3.14.103",
            "集群数据同步组播端口",
            "Port": 24403
        }
    }
]
}

```

6.22.2. GetLocalAgentStatus 获取RCM网络代理数据包统计信息

功能名称	GetLocalAgentStatus	版本号	V1.0.15.18	更新日期	20250120
功能备注	获取RCM网络代理转发出错数据包统计信息				
功能说明	获取RCM网络代理转发出错数据包统计信息				
访问链接	http://ip:port/kdp?PluginName=kdp_rcm_agent&FuncName=GetLocalAgentStatus&Pretty=1				

入参说明
Pretty=1 表示将json美化输出，为0表示不美化
出参说明
<pre>rcm_agent统计信息 { "/*": "版本信息", "RcmAgentVersion": "ldp_rcm_agent V1.0.15.18", "/*": "源Zone", "LocalZoneName": "Z0", "/*": "本地绑定地址", "LocalTcpIp": "192.168.86.101", "/*": "本地绑定端口", "LocalTcpPort": 37531, "/*": "本地Udp绑定地址", "LocalUdpIp": "192.168.86.101", "Udp": { "/*": "Udp接收端", "Receiver": { "/*": "发送出错包个数", "TxErrorCount": 0, "/*": "接收出错包个数", "RxErrorCount": 0, "/*": "不完整的RCM包个数", "ErrorPacketCount": 0 }, "/*": "Udp发送端", "Sender": { "/*": "发送出错包个数", "TxErrorCount": 0, "/*": "接收出错包个数", "RxErrorCount": 0, "/*": "不完整的RCM包个数", "ErrorPacketCount": 0 } }, "Tcp": { "/*": "Tcp接收端", "Receiver": { "/*": "发送出错包个数", "TxErrorCount": 0, "/*": "接收出错包个数", "RxErrorCount": 0, "/*": "不完整的RCM包个数", "ErrorPacketCount": 0, "/*": "网络拥塞发送出错包个数", "CongestionCount": 0 }, "/*": "Tcp发送端", "Sender": { "/*": "发送出错包个数", "TxErrorCount": 0, "/*": "接收出错包个数", "RxErrorCount": 0, "/*": "不完整的RCM包个数", "ErrorPacketCount": 0, "/*": "网络拥塞发送出错包个数", "CongestionCount": 0 } } }</pre>

6.22.3. GetClientConnectionStatus 获取RCM网络代理客户端统计信息

功能名称	GetClientConnectionStatus	版本号	V1.0.15.18	更新日期	20250120
功能备注	当前网络代理作为客户端，获取RCM网络代理统计信息				
功能说明	当前网络代理作为客户端，获取RCM网络代理统计信息				
访问链接	http://p.port/ldp?PluginName=ldp_rcm_agent&FuncName=GetClientConnectionStatus&Pretty=1				

入参说明
Pretty=1 表示将json美化输出，为0表示不美化
出参说明
<pre>rcm_agent统计信息 { "/*": "版本信息", "RcmAgentVersion": "ldp_rcm_agent V1.0.15.18", "/*": "源Zone", "LocalZone": "Z0", "/*": "客户端信息", "TcpClient": { "/*": "对端代理信息", "RemoteSites": [{ "/*": "对端代理所在Zone", "ZoneName": "Z1", "/*": "对端代理绑定地址", "Ip": "192.168.87.138", "/*": "对端代理绑定端口", "Port": 37533, "/*": "与对端代理是否已建立连接", "Connected": true }] } }</pre>

6.22.4. GetServerConnectionStatus 获取RCM网络代理服务端统计信息

--	--	--	--	--	--

功能名称	GetServerConnectionStatus	版本号	V1.0.15.18	更新日期	20250120
功能备注	当前网络代理作为服务端，获取RCM网络代理统计信息				
功能说明	当前网络代理作为服务端，获取RCM网络代理统计信息				
访问链接	http://ip:port/ldp?PluginName=ldp_rcm_agent&FuncName=GetServerConnectionStatus&Pretty=1				
入参说明					
Pretty=1 表示将json美化输出，为0表示不美化					
出参说明					
<pre>rcm_agent统计信息 { "///": "版本信息", "RcmAgentVersion": "ldp_rcm_agent V1.0.15.18", "///": "源Zone", "LocalZone": "Z0", "///": "服务端信息", "TcpServer": { "///": "对端代理信息", "RemoteSites": [{ "///": "对端代理绑定地址", "Ip": "192.168.87.138", "///": "对端代理绑定端口", "Port": 55848, "///": "与对端代理是否已建立连接", "Connected": true }] } }</pre>					

6.23. 重演工具插件 ldp_replay

6.23.1. PrepareReplay 日初数据库备份

功能名称	PrepareReplay	版本号	V1.0.0.0	更新日期	20250328
功能备注	日初数据库备份				
功能说明	将核心的日初数据库备份到重演工具				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_replay&InstanceNo=0&FuncName=PrepareReplay				
入参说明					
<pre>{ "///": "重演准备标识，PrepareFlag为1，则为日出模式；只缓存数据库；PrepareFlag为2，则为日终模式；缓存数据库的同时，拷贝核心的重演数据到本地；", "PrepareFlag": 1, "///": "交易日字段", "TradeDate": "20240328", "///": "需要持久化的表名，如果没有，则使用配置中的默认表进行缓存", "TableBackup": [] }</pre>					
出参说明					
<pre>{ "ErrorNo": 0, "ErrorMsg": "Success" }</pre>					

6.23.2. PrepareReplay 日终数据库备份

功能名称	PrepareReplay	版本号	V1.0.0.0	更新日期	20250328
功能备注	日终数据库备份				
功能说明	将核心的日终数据库备份到重演工具				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_replay&InstanceNo=0&FuncName=PrepareReplay				
入参说明					

<pre>{ "///": "重演准备标识, PrepareFlag为1, 只缓存数据库, 用于日初; PrepareFlag为2, 缓存数据库的同时, 拷贝核心的重演数据到本地, 用于日终", "PrepareFlag":2, "///": "交易日字段", "TradeDate":"20250328", "///": "需要持久化的表名, 如果没有, 则使用配置中的默认表进行缓存", "TableBackup":[], "///": "远程核心信息", "ReplayAppArray": [{ "AppName":"usesbid92", "Ip":"10.20.163.124", "Port":22, "UserName":"zhongzx" }, { "AppName":"usescbp92", "Ip":"10.20.163.124", "Port":22, "UserName":"zhongzx" }] }</pre>
出参说明
<pre>{ "ErrorNo": 0, "ErrorMsg": "Success" }</pre>

6.23.3. StartReplay 开始重演

功能名称	StartReplay	版本号	V1.0.0.0	更新日期	20250328
功能备注	开始重演				
功能说明	重演工具开始重演				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_replay&InstanceNo=0&FuncName=StartReplay				

入参说明

```
{  "///": "任务来源",  "SourceType": "apm",  "///": "关闭数据准备相关逻辑",  "PrepareDisabled": false,  "ReplayParams":  [    {      "///": "交易日字段",      "TradeDate":"20250327",      "///": "待重演核心信息",      "ReplayAppArray":      [        {          "AppName": "usesbid92",          "///": "重演请求发送间隔",          "SendIntervalMs": 1,          "///": "重演暂停在指定序号的消息发送之后, -1不做暂停",          "PauseNo": -1        }      ]    }  ]}
```

出参说明

```
{  "///": "唯一重演标识",  "ReplayFlag": 34663224206515035,  "ErrorNo": 0,  "ErrorMsg": "Success"}
```

6.23.4. StopReplay 停止重演

功能名称	StopReplay	版本号	V1.0.0.0	更新日期	20250328
功能备注	停止重演				
功能说明	停止重演				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_replay&InstanceNo=0&FuncName=StopReplay				
入参说明					
<pre>{ "///": "唯一重演标识", "ReplayFlag": 2342534523452345235, "///": "是否停止重演", "StopReplay": "true" }</pre>					
出参说明					

<pre>{ "ErrorNo": 0, "ErrorMsg": "Success" }</pre>
--

6.23.5. PauseReplay 暂停重演

功能名称	PauseReplay	版本号	V1.0.0.0	更新日期	20250328
功能备注	暂停重演				
功能说明	调用PauseReplay管理功能后，重演工具会暂停当前的请求发送				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_replay&InstanceNo=0&FuncName=PauseReplay				
入参说明					
<pre>{ "/*": "唯一重演标识", "ReplayFlag": 2342534523452345235, "/*": "是否暂停重演", "PauseReplay": "true" }</pre>					
出参说明					
<pre>{ "ErrorNo": 0, "ErrorMsg": "Success" }</pre>					

6.23.6. ResumeReplay 继续重演

功能名称	ResumeReplay	版本号	V1.0.0.0	更新日期	20250328
功能备注	继续重演				
功能说明	调用ResumeReplay管理功能后，重演工具会继续上次暂停的请求发送				
访问链接	http://ip:port/kjp?Pretty=1&PluginName=kjp_replay&InstanceNo=0&FuncName=ResumeReplay				
入参说明					
<pre>{ "/*": "唯一重演标识", "ReplayFlag": 2342534523452345235, "/*": "是否继续重演", "ResumeReplay": "true", "/*": "指定继续重演的核心，不配置则全部核心继续重演", "ResumeAppArray" : [{ "AppName" : "usesbid32", "/*" : "暂停在指定序号的消息发送之后，-1不做暂停", "PauseNo": -1 }] }</pre>					
出参说明					
<pre>{ "ErrorNo": 0, "ErrorMsg": "Success" }</pre>					

6.23.7. GetReplayResult 展示重演结果

功能名称	GetReplayResult	版本号	V1.0.0.0	更新日期	20250328
功能备注	展示重演结果				
功能说明	调用GetReplayResult管理功能后，重演工具会将应答的数据结果返回到界面				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_replay&InstanceNo=0&FuncName=GetReplayResult				
入参说明					
<pre>{ "/*": "唯一重演标识", "ReplayFlag": 23434524362562456456 }</pre>					
出参说明					

```
{
  "":"/: "重演任务级别状态",
  "Status": "FINISHED",
  "":"/: "重演开始时间",
  "StartTime": "20250327184347282",
  "":"/: "重演结束时间",
  "EndTime": "20250327184552615",
  "":"/: "重演结果信息",
  "TradeDayResult": [
    {
      "":"/: "交易日字段",
      "TradeDate": "20250327",
      "":"/: "重演交易日级别状态",
      "Status": "FINISHED",
      "":"/: "重演结果",
      "Result": "SUCCESS",
      "":"/: "核心级的应答比对结果",
      "AppsResult": [
        {
          "":"/: "核心名",
          "AppName": "usescbp92",
          "":"/: "重演核心级别状态",
          "Status": "FINISHED",
          "":"/: "重演结果",
          "Result": "SUCCESS",
          "":"/: "该核心的总请求数",
          "TotalSourceReqCount": "13025",
          "":"/: "该核心的总应答数",
          "TotalSourceRspCount": "103",
          "":"/: "应答比对结果",
          "AppResultArray": [
            {
              "":"/: "核心启动的时间戳目录名",
              "FoldName": "20250327-183654726",
              "":"/: "应答比对结果",
              "Result": {
                "":"/: "发送结果状态",
                "SendResult": "SEND_FINISH",
                "":"/: "发送结果信息",
                "SendResultInfo": "Success",
                "":"/: "发送笔数",
                "SendCount": "182",
                "":"/: "总共发送笔数",
                "TotalSendCount": "473",
                "":"/: "该时间戳目录下的原始请求数",
                "SourceReqCount": "196",
                "":"/: "该时间戳目录下请求过滤数",
                "FilterReqCount": "14",
                "":"/: "该核心的总请求过滤数",
                "TotalFilterReqCount": "30",
                "":"/: "比较结果状态",
                "CompareResult": "COMPARE_FINSIH",
                "":"/: "比较结果信息",
                "CompareResultInfo": "Success",
                "":"/: "接收应答个数",
                "RecvRspCount": "0",
                "":"/: "总共接收应答个数",
                "TotalRecvRspCount": "81",
                "":"/: "该时间戳目录下原始应答个数",
                "SourceRspCount": "1",
                "":"/: "该时间戳目录下应答过滤数",
                "FilterRspCount": "1",
                "":"/: "该核心的总应答过滤数",
                "TotalFilterRspCount": "21",
                "":"/: "该时间戳目录下的比较次数",
                "CompareCount": "0",
                "":"/: "该核心的总比较次数",
                "TotalCompareCount": "81",
                "":"/: "该时间戳目录下的比较成功次数",
                "CompareSuccessCount": "0",
                "":"/: "该核心的总比较成功次数",
                "TotalCompareSuccessCount": "81",
                "":"/: "该时间戳目录下的比较失败次数",
                "CompareFailedCount": "0",
                "":"/: "该核心的总比较失败次数",
                "TotalCompareFailedCount": "0",
              }
            }
          ]
        }
      ]
    }
  ]
}
```

6.23.8. GetReplayProgress 重演进度展示

功能名称	GetReplayProgress	版本号	V1.0.0.1	更新日期	20250722
功能备注	重演进度展示				
功能说明	获取当前重演的实时进度				
访问链接	http://pport/kdp?Pretty=1&PluginName=kdp_replay&InstanceNo=0&FuncName=GetReplayProgress				
入参说明					
<pre>{ "":"/: "唯一重演标识", "ReplayFlag": 23434524362562456456 }</pre>					
出参说明					

```
{
  "":"/: "重演状态",
  "Status": "FINISHED",
  "":"/: "重演开始时间",
  "StartTime": "20250408182339337",
  "":"/: "重演结束时间",
  "EndTime": "20250408182613936",
  "":"/: "交易日级别的重演进度",
  "TradeDayProgress": [
    {
      "":"/: "交易日字段",
      "TradeDate": "20250327",
      "":"/: "交易日状态",
      "Status": "FINISHED",
      "Result": "SUCCESS",
      "":"/: "交易日文件路径",
      "Path": "/home/hundsun/uft30/replay/workspace/replay_data/20250719",
      "":"/: "需发送的请求数",
      "TotalSourceReqCount": 504461,
      "":"/: "已发送的请求数, 包含被过滤的",
      "TotalSendCount": 435495,
      "":"/: "需比对的应答数",
      "TotalSourceRspCount": 139090,
      "":"/: "已比较的应答数, 包含被过滤的",
      "TotalCompareCount": 108289,
      "":"/: "已接收的应答数",
      "TotalRecvCount": 108537,
      "":"/: "重演进度",
      "Progress": [
        {
          "":"/: "核心名",
          "AppName": "usescbp92",
          "":"/: "核心状态",
          "Status": "FINISHED",
          "":"/: "核心重演结果",
          "Result": "SUCCESS",
          "":"/: "需发送的请求数",
          "SourceReqCount": 11430,
          "":"/: "已发送的请求数",
          "SendCount": 11427,
          "":"/: "过滤的请求数",
          "FilterReqCount": 3,
          "":"/: "需比对的应答数",
          "SourceRspCount": 4,
          "":"/: "已比较的应答数",
          "CompareCount": 3,
          "":"/: "过滤的应答数",
          "FilterRspCount": 1,
          "":"/: "比对成功应答数",
          "CompareSuccessCount": 2,
          "":"/: "比对失败应答数",
          "CompareFailedCount": 1,
          "":"/: "核心重演发送进度",
          "ReplaySendPercent": "100",
          "":"/: "核心重演比对进度",
          "ReplayComparePercent": "100",
          "":"/: "各时间戳进度信息",
          "TimeStampInfo": [
            {
              "":"/: "时间戳字段",
              "TimeStampName": "20250327-183654726",
              "":"/: "重演发送进度",
              "ReplaySendPercent": "100",
              "":"/: "重演比较进度",
              "ReplayComparePercent": "100"
            }
          ]
        }
      ]
    }
  ]
}
```

6.23.9. GetBaseReplayConfig 获取基准重演配置

功能名称	GetReplayConfig	版本号	V1.0.0.0	更新日期	20250328
功能备注	获取基准重演配置				
功能说明	获取基准重演配置				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_replay&InstanceNo=0&FuncName=GetBaseReplayConfig				
入参说明					
无					
出参说明					

```
{
  "Lib": "ldp_replay",
  "Instance": 0,
  "Timer": 0,
  "BaseAppConfig":
  [
    {
      "AppName": "uconvert64",
      "NodeNo": 55,
      "SendMsgFilter":
      [
        {
          "FilterFuncId": "1;2;77??;",
        }
      ],
      "RspCompare":
      {
        "FunctionIdBlackList": "1;2;77??;",
        "FunctionIdWhiteArray":
        [
          {
            "FunctionIdsList": "789;23??;",
            "FieldsBlackArray": [
              "fund_account",
              "clinet_id"
            ],
            "FieldsWhiteArray": [
              "fund_account",
              "clinet_id"
            ]
          }
        ]
      },
      "OnStageFuncInfo":
      [
        {
          "FunctionId": "961200",
          "SetAppStatusFid": 960006,
          "TimeOutSec": 60
        }
      ],
      "OffStageFuncInfo":
      [
        {
          "FunctionId": "123;234;234;",
          "TimeOutSec": 10
        }
      ]
    }
  ],
  "Dss":
  {
    "LogicName": "usersvr",
    "Server": "192.168.86.102",
    "Port": 123,
    "UserName": "admin",
    "PassWord": "admin",
    "DbType": 0
  },
  "TableBackup":
  [
    "price",
    "fund_account"
  ],
  "Remoter":
  {
    "AutoStart": false,
    "Routes":
    [
      {
        "Functions": "5;102???",
        "Systems" : "2",
        "Backend": 1
      }
    ],
    "RspTopic":
    {
      "Name": "replay_rsp",
      "EnableUnicastMode": false
    },
    "Backends" :
    [
      {
        "Id": 1,
        "Type": "AS-RCM",
        "TxTopic": "trade",
        "EnableMutiShard": "true",
        "EnableQueryFisrt": false
      }
    ]
  }
}
```

6.23.10. GetReplayConfig 获取某次重演配置

功能名称	GetReplayConfig	版本号	V1.0.0.0	更新日期	20250328
功能备注	获取某次重演配置				
功能说明	获取某次重演配置				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_replay&InstanceNo=0&FuncName=GetReplayConfig				
入参说明					
<pre>{ "":"/": "唯一重演标识", "ReplayFlag": 23434524362562456456}</pre>					

出参说明

```
{
  "Request": {
    "ReplayParams": [
      {
        "TradeDate": "20250327",
        "ReplayAppArray": [
          {
            "AppName": "usesbid92",
            "sendIntervalMs": 1
          },
          {
            "AppName": "usescbp92",
            "sendIntervalMs": 1
          }
        ]
      }
    ],
    "SourceName": "任务来源",
    "SourceId": "apm",
    "FuncName": "StartReplay",
    "InstanceNo": "0",
    "PluginName": "ldp_replay",
    "Pretty": "1",
    "FuncName": "StartReplay",
    "InstanceNo": "0",
    "PluginName": "ldp_replay",
    "Pretty": "1",
    "StartTimestamp": "20250327184347282",
    "EndTimestamp": "20250327184552615"
  }
}
```

6.23.11. GetTradeDayInfo 展示每个交易日状态和核心状态

功能名称	GetTradeDayInfo	版本号	V1.0.0.0	更新日期	20250328
功能备注	展示每个交易日状态和核心状态				
功能说明	展示每个交易日状态和核心状态				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_replay&InstanceNo=0&FuncName=GetTradeDayInfo				
入参说明					
无					
出参说明					
<pre>{ "TradeDayInfo": [{ "TradeDay": "20250327", "TradeDayStatus": "ALLOWED", "AppInfo": [{ "AppName": "usesbid92", "AppStatus": "ALLOWED" }, { "AppName": "usescbp92", "AppStatus": "ALLOWED" }] }] }</pre>					

6.23.12. GetReplayStatus 展示历史重演的状态

功能名称	GetReplayStatus	版本号	V1.0.0.0	更新日期	20250328
功能备注	展示历史重演的状态				
功能说明	展示所有历史的交易日信息状态，每个交易日下的核心名和核心状态				
访问链接	http://ldp?Pretty=1&PluginName=ldp_replay&InstanceNo=0&FuncName=GetReplayStatus				
入参说明					
无					
出参说明					

```
{
  "":"/": "历史重演的状态信息",
  "ReplayStatus": [
    {
      "":"/": "重演唯一标识",
      "ReplayFlag": "20693334960154728",
      "":"/": "重演状态",
      "Status": "FINISHED",
      "":"/": "重演开始时间",
      "StartTime": "20250327184347282",
      "":"/": "重演结束时间",
      "EndTime": "20250327184552615",
      "":"/": "该次重演的配置, 参考GetReplayConfig",
      "ReplayConfig": {
        "Request": {
          "ReplayParams": [
            {
              "TradeDate": "20250327",
              "ReplayAppArray": [
                {
                  "AppName": "usesbid92",
                  "sendIntervalMs": 1
                },
                {
                  "AppName": "usescbp92",
                  "sendIntervalMs": 1
                }
              ]
            }
          ],
          "SourceType": "apm",
          "FuncName": "StartReplay",
          "InstanceNo": "0",
          "PluginName": "ldp_replay",
          "Pretty": "1"
        },
        "FuncName": "StartReplay",
        "InstanceNo": "0",
        "PluginName": "ldp_replay",
        "Pretty": "1",
        "StartTimeStamp": "20250327184347282",
        "EndTimeStamp": "20250327184552615"
      },
      "":"/": "交易日的状态",
      "TradeDayStatus": [
        {
          "":"/": "交易日字段",
          "TradeDate": "20250327",
          "":"/": "重演状态",
          "Status": "FINISHED",
          "":"/": "重演结果",
          "Result": "SUCCESS",
          "":"/": "核心状态",
          "AppsStatus": [
            {
              "":"/": "核心名",
              "AppName": "usescbp92",
              "":"/": "重演状态",
              "Status": "FINISHED",
              "":"/": "重演结果",
              "Result": "SUCCESS"
            }
          ]
        }
      ]
    }
  ]
}
```

6.23.13. GetAppInfo 获取核心的信息

功能名称	GetAppInfo	版本号	V1.0.0.0	更新日期	20250328
功能备注	获取核心的信息				
功能说明	获取核心的信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_replay&InstanceNo=0&FuncName=GetAppInfo				
入参说明					
无					
出参说明					
<pre>{ "":"/": "核心信息", "AppsInfo": [{ "":"/": "核心名", "AppName": "usesbid92", "":"/": "核心部署机器的IP", "Ip": "10.20.163.124", "":"/": "端口号", "Port": 22, "":"/": "用户名", "UserName": "zhongzx", "":"/": "核心落的持久化文件路径", "PersistentPath": "/home/zhongzx/plugin/workspace/replaylog/usesbid92/" }] }</pre>					

6.23.14. GetReplayDiffInfo 获取重演比对结果文件信息

功能名称	GetReplayDiffInfo	版本号	V1.0.0.0	更新日期	20250722
------	-------------------	-----	----------	------	----------

功能备注	获取重演比对结果文件信息
功能说明	调用GetReplayDiffInfo管理功能后，重演工具会将重演结果文件路径等信息返回到界面
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_replay&InstanceNo=0&FuncName=GetReplayDiffInfo
入参说明	
<pre>{ "":"/": "唯一重演标识", "ReplayFlag": 23434524362562456456 }</pre>	
出参说明	
<pre>{ "DiffInfo": [{ "":"/": "交易日", "TradeDate": "20250705", "":"/": "核心名", "AppName": "usesbid62", "":"/": "比对结果文件信息", "ResultArray": [{ "":"/": "时间戳目录名", "FoldName": "20250711-160749947", "":"/": "比对结果文件路径", "Path": ".../CompareResult.csv", "":"/": "比对结果文件内容，文件大小小于1M时展示", "Detail": "基准全局序号,功能号,线程号,错误信息,基准应答消息,重演应答消息..." }] }, { "":"/": "时间戳目录名", "TradeDate": "20250705", "":"/": "比对结果文件路径", "AppName": "usescbp62", "":"/": "比对结果文件信息", "ResultArray": [{ "":"/": "时间戳目录名", "FoldName": "20250711-160800796", "":"/": "比对结果文件路径", "Path": ".../CompareResult.csv" }] }] }</pre>	

6.24. dbclient插件 ldp_dbclient

6.24.1. GetPoolInfoByDataSource 获取指定数据源的连接池信息

功能名称	GetPoolInfoByDataSource	版本号	V1.0.0.0	更新日期	20250609
功能备注	获取指定数据源的连接池信息				
功能说明	获取指定数据源的连接池信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_dbclient&InstanceNo=0&FuncName=GetPoolInfoByDataSource				
入参说明					
<pre>{ "":"/": "需要获取连接池信息的数据源名称", "DataSource": "ob_oracle"}</pre>					
出参说明					
<pre>{ "":"/": "连接池是否开启,true/false" "EnableConnectionPool": true, "":"/": "连接池上分配的连接数" "TotalConnCount": 80, "":"/": "连接池上当前空闲的连接数（可使用）" "FreeConnCount": 30, "":"/": "连接池上已经被使用的连接数" "UsedConnCount": 50, "":"/": "连接池的初始连接数" "InitConnCount": 10, "":"/": "连接池的最大连接数" "MaxConnCount": 100, "":"/": "连接池清理空闲连接的阈值" "CleanThreshold": 50, "":"/": "连接池大小，表示可以存放的连接数，实际获取的连接数为TotalConnCount" "PoolSize": 100, "":"/": "物理连接失败累计次数，在获取数据源的新连接成功后重置为0" "ConnectFailedCount": 100, "":"/": "历史连接峰值，重启才会重置" "MaxUsedConnCount": 100}</pre>					

6.24.2. GetPoolInfo 获取所有数据源的连接池信息

功能名称	GetPoolInfo	版本号	V1.0.0.0	更新日期	20250609
功能备注	获取所有数据源的连接池信息				

功能说明	获取所有数据源的连接池信息
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_dbclient&InstanceNo=0&FuncName=GetPoolInfo
入参说明	
出参说明	
<pre>{ "/*": "连接池数" "TotalPoolCount": 2, "/*": "连接池具体信息" "PoolInfoByDataSource": [{ "/*": "连接池是否开启,true/false" "EnableConnectionPool": true, "/*": "数据源名称" "DataSourceName": "ob_oracle", "/*": "连接池上分配的连接数" "TotalConnCount": 80, "/*": "连接池上当前空闲的连接数（可使用）" "FreeConnCount": 30, "/*": "连接池上已经被使用的连接数" "UsedConnCount": 50, "/*": "连接池的初始连接数" "InitConnCount": 10, "/*": "连接池的最大连接数" "MaxConnCount": 100, "/*": "连接池清理空闲连接的阈值" "CleanThreshold": 50, "/*": "连接池大小，表示可以存放的连接数，实际获取的连接数为TotalConnCount" "PoolSize": 100, "/*": "连接失败累计次数，在获取数据源的新连接成功后重置为0" "ConnectFailedCount": 100, "/*": "历史连接峰值，重启才会重置" "MaxUsedConnCount": 100 }] }</pre>	

6.24.3. GetConnectionInfo 获取指定数据源的连接信息

功能名称	GetConnectInfo	版本号	V1.0.0.0	更新日期	20250609
功能备注	获取指定数据源的连接信息				
功能说明	获取指定数据源的连接信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_dbclient&InstanceNo=0&FuncName=GetConnectionInfo				

入参说明

```
{  "/*": "需要获取连接池信息的数据源名称",  "DataSource": "ob_oracle"}
```

出参说明

```
{  "/*": "连接池是否开启"  "EnableConnectionPool": true,  "ConnectionInfoByDataSource": [    {      "/*": "连接池中连接编号"      "ConnectionNo": 1,      "/*": "连接状态，0 空闲， 1 被使用， 2 断开（连接在连接池中），3 断开（连接不在池中，使用中）"      "ConnectionStatus": 0,      "/*": "连接上最近一次的SQL"      "LastSQL": "select 1 from dual",      "/*": "连接上最近一次的错误号"      "LastErrNo": 232,      "/*": "连接上最近一次的错误信息"      "LastErrorInfo": ""    },  ],}
```

6.24.4. GetExecStats 获取各驱动类型的执行统计信息

功能名称	GetExecStats	版本号	V1.0.0.0	更新日期	20250609
功能备注	获取各驱动类型的执行统计信息（成功/失败/0 行等）				
功能说明	按驱动类型统计 SQL/语句执行情况，用于监控执行质量和排查问题				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_dbclient&InstanceNo=0&FuncName=GetExecStats				
入参说明					
<pre>{ "/*": "当前实现不依赖入参，可为空对象", "/*": "如后续扩展，可增加统计粒度或时间范围等字段" }</pre>					
出参说明					

```

{
    "DriverTotals": [
        {
            "DriverType": "DS_TYPE_ORACLE",
            "ExecSuccessCount": 1000,
            "ExecFailedCount": 10,
            "QueryZeroRowsCount": 200,
            "UpdateZeroRowsCount": 50,
            "UncommittedPrepareRollbackCount": 3
        }
    ]
}

```

6.25. redo同步代理插件 ldp_redo_copy

6.25.1. GetStatistics 获取redo同步代理插件统计信息

功能名称	GetStatistics	版本号	V1.0.15.1	更新日期	20250617
功能备注	获取redo同步统计信息				
功能说明	获取redo同步统计信息				
访问链接	http://ip:port/klp?PluginName=ldp_redo_copy&InstanceNo=0&FuncName=GetStatistics&Pretty=1				

入参说明

Pretty=1 表示将json美化输出，为0表示不美化

出参说明

```

ldp_redo_copy统计信息
{
    "RedoCopyVersion": "ldp_redo_copy V1.0.15.1",
    "ProductName": "ldp_redocopy",
    "AppName": "redo_copy#0",
    "TransmitterInfo": {
        "RoleInfo": {
            "Role": 1,
            "RoleName": "ArbActive"
        },
        "AllTransmitterInfo": {
            "AllTransmitterCount": 1,
            "AllTransmitterSendCount": 0,
            "AllTransmitterSendKBytes": 0,
            "AllTransmitterSendHBCount": 534,
            "AllTransmitterFrontendRecvCount": 0,
            "AllTransmitterFrontendRecvKBytes": 0,
            "AllTransmitterBackendRecvCount": 534,
            "AllTransmitterBacklogCount": 0,
            "LimitRateMbps": 100,
            "LastSendRateMbps": 0.0,
            "AvgSendRateMbps": 0.0,
            "MaxSendRateMbps": 0.0,
            "TimerThreadRunTimeMilli": 60253628
        },
        "Transmitters": [
            {
                "Id": 0,
                "FrontendName": "rcm_frontend_0",
                "State": "READY",
                "TopicName": "bypass_redo_bizA",
                "PartitionNo": 62,
                "ShardNo": 62,
                "FunctionID": 417,
                "MaxCacheMessageCount": 128,
                "LastStateTimeUs": 5533345425607,
                "LastStateTime": "20250614-15:35:31.393971",
                "StartMsgNo": 0,
                "MaxMsgNo": 0
            }
        ]
    }
}

```

```
// 说明：从RCM上下文接收的redo数据数量",
"NextMsgNo": 0,
"/":"/": "下一个待发送的消息序号",
"ToSendMsgNo": 0,
"/":"/": "确认的消息序号",
"AckMsgNo": 0,
"/":"/": "最近一次发送数据时间（单位：微秒）",
"LastSendTimeUs": 5533890676693,
"/":"/": "最近一次发送数据时间",
"LastSendTime": "20250614-15:44:36.645057",
"/":"/": "消息重传次数",
"RetransmitCount": 0,
"/":"/": "缓存满的次数",
"CacheFullCount": 0,
"/":"/": "重置起始消息号的次数",
"ResetStartMsgNoCount": 0,
"/":"/": "重置起始消息号失败的次数",
"ResetStartMsgNoFailCount": 0,
"/":"/": "从缓存中获取消息失败的次数",
"GetMsgFailCount": 0,
"/":"/": "ToSendMsgNo 小于 AckMsgNo的次数",
"ToSendLessAckCount": 0,
"Send": {
    "/":"/": "发送成功数据数量",
    "SendCount": 0,
    "/":"/": "直接发送成功数据数量",
    "DirectSendCount": 0,
    "/":"/": "发送心跳的数量",
    "SendHBCount": 534,
    "/":"/": "发送失败数量数量",
    "SendFailedCount": 0,
    "/":"/": "压缩前数据大小（单位：KB）",
    "OriginalKBytes": 0,
    "/":"/": "压缩后数据大小，实际发送数据大小（单位：KB）",
    "SendKBytes": 0,
    "/":"/": "单位时间发送消息增长条数",
    "SendGrowthCounterPerSecond": 0,
    "/":"/": "被限速的次数",
    "LimitRateCount": 0,
    "/":"/": "最近一秒发送速率，每秒兆比特数",
    "LastSendRateMbps": 0.0,
    "/":"/": "最近30秒发送的平均速率，每秒兆比特数",
    "AvgSendRateMbps": 0.0,
    "/":"/": "最大发送速率，每秒兆比特数",
    "MaxSendRateMbps": 0.0
},
"FrontendRecv": {
    "/":"/": "从RCM上下文接收的redo数据数量",
    "FrontendRecvCount": 0,
    "/":"/": "从RCM上下文接收的redo数据大小（单位：KB）",
    "FrontendRecvKBytes": 0,
    "/":"/": "单位时间从RCM上下文接收消息增长条数",
    "RecvGrowthCounterPerSecond": 0
},
"BackendRecv": {
    "/":"/": "从Backend接收的Ack数量",
    "RecvCount": 534,
    "/":"/": "从Backend接收的Ack大小（单位：KB）",
    "RecvKBytes": 25,
    "/":"/": "单位时间从Backend接收消息增长条数",
    "RecvGrowthCounterPerSecond": 1
},
"/":"/": "数据积压数量",
"BacklogCount": 0,
"/":"/": "触发流控的次数",
"TriggeredFlowControlCount": 0,
"/":"/": "最近一次触发流控的时间（单位：微秒）",
"LastTriggeredFlowControlTimeUs": 5533324707389,
"/":"/": "最近一次触发流控的时间",
"LastTriggeredFlowControlTime": "20250614-15:35:10.675753",
"/":"/": "压缩率",
"CompressionRatio": 0.0,
"/":"/": "压缩后较压缩前更大的消息数",
"CompressionBiggerCount": 0,
"/":"/": "最新N笔数据平均处理耗时（此处N默认取发送窗口大小）",
"AvgTransmitterTimeUs": 0,
"/":"/": "最新N笔数据平均请求应答耗时",
"AvgAckRespTimeUs": 0,
"/":"/": "当前发送数据轮次号，发生重传时加1",
"Epoch": 160,
"/":"/": "动态发送窗口大小，最大值为配置的发送窗口值，发生丢包时，该值会变为1，后续每接收到一个新的ACK会递增1",
"DynamicSendWindowSize": 1173,
"/":"/": "收到 RCM_TOPIC_MESSAGE_LOSS 事件的次数，该次数大于0表示主题发送端和接收端消息轮次不一致出现协商失败，主机房
大概率发送过清流重启，因此需要人工干预，清除备机房对应主题消息然后执行重启",
"RcmTopicMessageLossEventCount": 10
}
}
}
```

6.25.2. SetLimitRateMbps设置流控限速值

功能名称	SetLimitRateMbps	版本号	V1.0.15.4	更新日期	20251008
功能备注	流控限速值				
功能说明	设置流控限速值				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_redo_copy&InstanceNo=0&FuncName=SetLimitRateMbps				
入参说明					
<pre>{ "Request": { "//": "设置流控限速值，要求为非负整数", "LimitRateMbps": nValue } }</pre>					
出参说明					

```
{
  "":"/: "返回设置的流控限速值或者set failed",
  "LimitRateMbps":nValue
}
```

6.26. 前置错误转换组件 front_transfer

6.26.1. GetErrorTransferConfig 获取错误转换配置信息

功能名称	GetErrorTransferConfig	版本号	V1.0.0.1	更新日期	20250626
功能备注	获取错误转换配置信息				
功能说明	获取错误转换配置信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&InstanceNo=0&FuncName=kdp_front_error_transfer_GetErrorTransferConfig				
入参说明					
出参说明					
<pre>{ "":"/: "范围类型配置", "rangeMappings": [{ "startErrorCode": "1000", "endErrorCode": "1999", "mappedErrorCode": "1000", "mappedErrorMessage": "系统繁忙，请稍后重试" }, { "startErrorCode": "2000", "endErrorCode": "2999", "mappedErrorCode": "KEEP_ORIGINAL_CODE", "mappedErrorMessage": "业务处理异常，请联系客服" }], "":"/: "枚举类型配置", "enumMappings": [{ "errorCode": 3001, "mappedErrorCode": 3001, "mappedErrorMessage": "参数格式不正确" }, { "errorCode": 3003, "mappedErrorCode": 3003, "mappedErrorMessage": "{}" }]}</pre>					

6.27. 前置账户定位组件 front_locate

6.27.1. GetAccountsMissLocate 获取账户定位失败信息

功能名称	GetAccountsMissLocate	版本号	V1.0.0.1	更新日期	20251212
功能备注	获取账户定位失败信息				
功能说明	获取账户定位失败信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_front&InstanceNo=0&FuncName=kdp_front_locate_GetAccountsMissLocate				
入参说明					
出参说明					
<pre>{ "":"/: "总账户定位失败个数", "TotalMissCount": 4, "AccountsInfo": [{ "":"/: "账户名", "FundAccount": "2025", "":"/: "该账户定位失败次数", "MissCount": 1 }, { "FundAccount": "2021", "MissCount": 1 }, { "FundAccount": "2023", "MissCount": 1 }, { "FundAccount": "2123", "MissCount": 1 }] }</pre>					

6.27.2. ClearAccountsMissLocate 清空定位失败账户信息

功能名称	ClearAccountsMissLocate	版本号	V1.0.0.1	更新日期	20251212
------	-------------------------	-----	----------	------	----------

功能备注	清空定位失败账户信息
功能说明	清空定位失败账户信息
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_front&FuncName=ldp_front_locate_ClearAccountsMissLocate
入参说明	
<pre>{ "Clear": true }</pre>	
出参说明	
<pre>{ "/*": "清空结果", "ClearResult": "Ok" }</pre>	

6.28. 核心转储文件解析插件 ldp_coreparser

6.28.1. GetWorkSpaceList 获取核心应用工作路径列表

功能名称	GetWorkSpaceList	版本号	V1.0.0.1	更新日期	20250728
功能备注	获取核心应用工作路径列表				
功能说明	获取核心应用工作路径列表				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_coreparser&FuncName=GetWorkSpaceList				
入参说明					
出参说明					
<pre>{ "/*": "核心应用工作路径列表", "WorkSpaceList": [{ "/*": "核心节点的工作路径", "WorkSpace": "/home/uft30/ucrtbid52/workspace" }, { "WorkSpace": "/home/uft30/uconvert99/workspace" }] }</pre>					

6.28.2. UpdateWorkSpaceList 更新核心应用工作路径列表

功能名称	UpdateWorkSpaceList	版本号	V1.0.0.1	更新日期	20250728
功能备注	更新核心应用工作路径列表				
功能说明	更新核心应用工作路径列表				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_coreparser&FuncName=UpdateWorkSpaceList				
入参说明					
<pre>{ "/*": "核心应用工作路径列表", "WorkSpaceList": [{ "/*": "核心节点的工作路径", "WorkSpace": "/home/uft30/ucrtbid52/workspace" }, { "WorkSpace": "/home/uft30/uconvert99/workspace" }] }</pre>					
出参说明					
<pre>{ "/*": "错误号, 0表示成功, 其他表示失败", "ErrorNo": 101, "/*": "错误信息", "ErrorInfo": "" }</pre>					

6.28.3. ExecParserCoreDumpFile 主动解析指定的核心转储文件

功能名称	ExecParserCoreDumpFile	版本号	V1.0.0.1	更新日期	20250728
功能备注	主动解析指定的核心转储文件				

注	
功能说明	主动解析指定的核心转储文件
访问链接	http://ppport/kdp?Pretty=1&PluginName=kdp_coreparser&FuncName=ExecParserCoreDumpFile&CoreDumpFileFullPath=/home/uft30/ucrtbid52/workspace/core.60427
入参说明	
<pre>{ "/CoreDumpFileFullPath": "/home/uft30/ucrtbid52/workspace/core.60427" }</pre>	
出参说明	
<pre>{ "FileName": "/home/uft30/ucrtbid52/workspace/core.87869", "CreateName": "2025-07-28 15:21:06", "ParserTime": "2025-07-28 15:56:26.232573", "StartCommandLine": "hsldp startdaemon uft30 mdb#0 -s 192.168.74.41:4245", "ParserResult": "Success", "ThreadInfos": [{ "ThreadId": 1, "IsCrashThread": true, "FunctionNo": 910011, "MsgType": "Request", "MsgLen": 110, "Dest": "ToMaster", "ExtHeadLength": 0, "NodeNo": 0, "FixedSize": 78, "SystemNo": 0, "Token": 0, "ConnectionID": 1207959552, "UserDefined": 0, "BizThreadNo": 0, "Body": [{ "fund_account": "999", "branch_no": "888", "Name": "777" }] }] }</pre>	

6.28.4. GetParserResultInfo 获取解析结果统计信息

功能名称	GetParserResultInfo	版本号	V1.0.0.1	更新日期	20250728
功能备注	获取解析结果统计信息				
功能说明	获取解析结果统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_coreparser&FuncName=GetParserResultInfo				
入参说明					
出参说明					
<pre>{ "ParserResultInfo": [{ "/*": "core dump文件名", "FileName": "/home/uft30/ucrtbid52/workspace/core.87869", "/*": "core dump文件创建时间, 年月日 hh:mm:ss", "CreateName": "2025-07-28 15:21:06", "/*": "解析时间, 年月日 hh:mm:ss.SSSSSSus", "ParserTime": "2025-07-28 15:56:26.232573", "/*": "启动命令行", "StartCommandLine": "hsl dp startdaemon uft30 mdb#0 -s 192.168.86.102:2999", "/*": "解析结果", "ParserResult": "Success", "ThreadInfos": [{ "/*": "线程id", "ThreadId": 1, "/*": "是否是崩溃线程", "IsCrashThread": true, "/*": "功能号", "FunctionNo": 910011, "/*": "消息类型", "MsgType": "Request", "/*": "消息长度", "MsgLen": 110, "/*": "去主去备标识", "Dest": "ToMaster", "/*": "扩展头长度", "ExtHeadLength": 0, "/*": "节点号", "NodeNo": 0, "/*": "固定字段长度", "FixedSize": 78, "/*": "系统号", "SystemNo": 0, "/*": "令牌标识", "Token": 0, "/*": "连接号", "ConnectionID": 1207959552, "/*": "用户自定义", "UserDefined": 0, "/*": "核心工作线程号", "BizThreadNo": 0, "Body": [{ "fund_account": "999", "branch_no": "888", "Name": "777" }] }] }] }</pre>					

6.29. 小表缓存插件 ldp_minitablecache

6.29.1. SetClusterInternalSync 管理集群内同步功能

功能名称	SetClusterInternalSync	版本号	V1.0.0.1	更新日期	20250801
功能说明	提供单表/全表同集群内同步，操作类型填reinit，根据dst_group_name比较，如果集群名一致，则更新缓存，TableName为""表示全表刷新，指定表名为单表刷新				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_minitablecache&FuncName=SetClusterInternalSync				
入参说明					
Pretty	1-美化json输出, 0-不美化				
DstGropName	集群名，不允许传空值"",不匹配 Status 返回 -1				
TableName	单表 or 全表, ""为全表，指定表名为单表刷新				
ParamOperType	操作类型填 4 (reinit)，这里是安全性设置，需要用户确保是更新操作				
出参说明					

```
{
  "Result": [
    {
      // 同步消息状态，正常创建返回值为0，异常返回值为-1
      "Status": 0,
    }
  ]
}
```

6.29.2. SetSystemInternalSync 管理集群内同步功能

功能名称	SetSystemInternalSync	版本号	V1.0.0.1	更新日期	20250801
功能说明	提供单表/全表同系统内同步，操作类型填reinit，TableName为""表示全表刷新，指定表名为单表刷新				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_minitablecache&FuncName=SetSystemInternalSync				
入参说明					
Pretty	1-美化json输出, 0-不美化				
TableName	单表 or 全表, ""为全表，指定表名为单表刷新				
ParamOperType	操作类型填 4 (reinit)，这里是安全性设置，需要用户确保是更新操作				
出参说明					
<pre>{ "Result": [{ // 同步消息状态，正常创建返回值为0，异常返回值为-1 "Status": 0, }]}</pre>					

6.29.3. GetMdbSaveInfo 获取存储小表缓存 mdb 信息

功能名称	GetMdbSaveInfo	版本号	V1.0.0.1	更新日期	20250801
功能说明	获取数据源名：直接操作小表缓存更新成功、失败次数，本版本不支持				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_minitablecache&FuncName=GetMdbSaveInfo				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ // 数据源名 "MdbLogicName": "obtest", // 表相关信息 "Tables": [] }</pre>					

6.29.4. GetStaticMiniTableCacheInfo 获取缓存表静态信息

功能名称	GetStaticMiniTableCacheInfo	版本号	V1.0.0.1	更新日期	20250801
功能说明	获取本地表名、db表名、字段信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_minitablecache&FuncName=GetStaticMiniTableCacheInfo				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					


```
{
  "LoadStaticInfo": [
    {
      // 本地缓存表名, 业务查询时使用的表名
      "LocalTableName": "obtest",
      // db表名
      "TableName": "test",
      // 字段信息, 多字段使用 ',' 号隔开
      "Select": 0,
      // 索引名信息
      "Indexs": [
        {
          "Name": "col_int",
          "Type": 0
        },
        {
          "Name": "col_integer",
          "Type": 1
        },
        {
          "Name": "col_varchar2,col_varchar",
          "Type": 2
        }
      ]
    }
  ]
}
```

6.29.5. GetDynamicMiniTableCacheInfo 获取缓存表动态信息

功能名称	GetDynamicMiniTableCacheInfo	版本号	V1.0.0.1	更新日期	20250801
功能说明	1、第一次加载数据时间, 加载记录条数;2、最后一次加载数据时间, 加载记录条数;3、缓存表加载的次数; 4、mc数据更新次数、指令更新次数;5、本地更新同步次数;6、缓存表数据加载状态;（加载中、加载完成、刷新中、刷新完成）;				
访问链接	http://192.168.86.102:55555/klp?Pretty=1&PluginName=klp_minitablecache&InstanceNo=0&FuncName=GetDynamicMiniTableCacheInfo				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ "LoadDynamicInfo": [{ // 缓存表表名 "LoadMiniTableCache": "sds", // 第一次加载数据时间, 加载记录条数 "StartTime": "2023-11-24 11:17:21", "StartLoadCount": 0, // 最后一次加载数据时间, 加载记录条数 "EndTime": "2023-11-24 11:17:21", "EndLoadCount": 0, // 缓存表加载的次数 "LoadMiniTableCacheCount": 1, // mc数据更新次数 "McUpdateCount": 0, // mc指令更新次数 "McCommandCount": 0, // 本地更新同步次数 "LocalUpdateCount": 12, // 缓存表数据加载状态:（0:加载中、1:加载完成、2:刷新中、3:刷新完成） "LoadStatus": 0 }] }</pre>					

6.29.6. SetConsumptionMode 设置消费模式

功能名称	SetConsumptionMode	版本号	V1.0.0.1	更新日期	20250801
功能说明	1、支持数据和通知两种模式，0为数据模式，1为通知模式				
访问链接	http://192.168.86.102:55555/ldp?Pretty=1&PluginName=ldp_minitablecache&InstanceNo=0&FuncName=SetConsumptionMode				
入参说明					
Pretty	1-美化json输出, 0-不美化				
ConsumptionMode	支持数据和通知两种模式，0为数据模式，1为通知模式				
出参说明					
<pre>{ // 设置成功返回值为0，异常返回值为-1 "Status": 0 }</pre>					

6.30. partkeeper插件 ldp_partkeeper

6.30.1. GetKeeperInfo 获取Keeper插件状态

功能名称	GetKeeperInfo	版本号	V1.0.2501.0.1	更新日期	20260122
功能备注	获取Keeper插件状态				

功能说明	获取Keeper插件状态
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetKeeperInfo
入参说明	
无	
出参说明	
<pre>{ "RejectCount": 0, "StartMode": 0, "BackupDir": ".\\partkeeper_data", "MaxFileSize": 1024000, "BizStatusFreq": 30000, "AppStatusFreq": 30000, "ProgressTimeOut": 30000, "InActiveSynChangeFreq": 3000, "SyncTimeOut": 30000, "StartClusterTimeOut": 30000, "HeartBeatTime": 50, "SessionTimeOut": 3000, "ProgressTime": 1000, "Remoter": {} }</pre>	

6.30.2. GetProcNodeList 获取核心节点状态

功能名称	GetProcNodeList	版本号	V1.0.0.0	更新日期	20250920
功能备注	获取核心节点状态				
功能说明	获取核心节点状态				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetProcNodeList				
入参说明					
无					
出参说明					
<pre>{ "ArbNo": 0, "QueryNo": 0, "ArbFile": "arb_result.json", "ShardNoGroup": [{ "ClusterName": "mclb_1", "ShardRoundNo": 0, "ClusterStartStatus": "Start", "DisasterStartStatus": "Start", "BizProcNode": [{ "NodeName": "proc#0", "Online": "Voter", "LastRecvTime": "2025-09-20 14:07:20.123", "Age": 0, "LastArbNo": 0, "ConnectId": 11342, "ClientType": "Proc", "PartCount": 1, "MainGroupId": 1 }] }]}</pre>					

6.30.3. GetWatcherList 获取todb节点状态

功能名称	GetWatcherList	版本号	V1.0.15.501	更新日期	20251029
------	----------------	-----	-------------	------	----------

功能备注	获取todb节点状态
功能说明	获取todb节点状态
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetWatcherList
入参说明	
无	
出参说明	
<pre>{ "":"/": "U32,当前仲裁序号", "ArbNo": 0, "":"/": "U32,当前查询序号", "QueryNo": 0, "":"/": "Str,本地仲裁文件名", "ArbFile": "arb_result.json", "ShardNoGroup": [{ "":"/": "Str,集群名_分片号", "ClusterName": "mcb_1", "":"/": "U64,分片内轮次号,从0开始", "ShardRoundNo": 0, "":"/": "Str,集群启动状态, Init=集群未启动完成, FirstChange=等待初次迁移, Start=集群启动完成", "ClusterStartStatus": "Start", "":"/": "Str,灾备恢复状态, Init=灾备恢复未完成, FirstChange=等待初次迁移, Start=灾备恢复完成", "DisasterStartStatus": "Start", "WatcherNode": [{ "":"/": "Str,节点名", "NodeName": "todb#0", "":"/": "Str,工作状态, UnRegister=未登录, DisConnect=连接断开, Watcher=连接正常, 但不参与读写核心迁移, Voter=参与读写核心迁移", "Online": "Watcher", "":"/": "Str,最近一次收到网络包的时间", "LastRecvTime": "2025-09-20 14:07:20.123", "":"/": "U32,收到的核心最新年龄", "Age": 0, "":"/": "U32,收到的核心最大确认仲裁序号龄", "LastArbNo": 0, "":"/": "Str,客户端身份类型, 对于todb来说固定为Toddb", "ClientType": "Toddb", "":"/": "U32,Remoter连接号", "ConnectId": 11342 }] }], }</pre>	

6.30.4. GetGatewayList 获取监听的网关列表

功能名称	GetGatewayList	版本号	V1.0.0.0	更新日期	20250920
功能备注	获取监听的网关列表				
功能说明	获取监听的网关列表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetGatewayList				
入参说明					
无					
出参说明					
<pre>{ "":"/": "U32,当前仲裁序号", "ArbNo": 0, "":"/": "U32,当前查询序号", "QueryNo": 0, "":"/": "Str,本地仲裁文件名", "ArbFile": "arb_result.json", "Gateway": [{ "":"/": "Str,网关节点名", "NodeName": "front#0", "":"/": "Str,工作状态,UnRegister=未登录,DisConnect=连接断开,Watcher=连接正常,但不参与读写核心迁移,Voter=参与读写核心迁移", "Online": "Watcher", "":"/": "U32,收到的最新年龄", "Age": 0, "":"/": "Str,最近一次收到网络包的时间", "LastRecvTime": "2025-09-20 14:07:20.123", "":"/": "U32,收到的核心最大确认仲裁序号龄", "LastArbNo": 0, "":"/": "Str,客户端身份类型, 对于前置来说固定为Front", "ClientType": "Front", "":"/": "U32,Remoter连接号", "ConnectId": 11342, }]}</pre>					

6.30.5. GetPartitionInfo 获取分区和核心的对应关系

功能名称	GetPartionInfo	版本号	V1.2501.1.0	更新日期	20260311
功能备注	获取分区和核心的对应关系				

功能说明	获取分区和核心的对应关系
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetPartionInfo
入参说明	
无	
出参说明	
<pre>{ "":"/":"U32,当前仲裁序号", "ArbNo": 0, "":"/":"U32,当前查询序号", "QueryNo": 0, "":"/":"Str,本地仲裁文件名", "ArbFile": "arb_result.json", "ShardNoGroup": [{ "":"/":"Str,集群名_分片号", "ClusterName": "mcb_1", "":"/":"U64,分片内轮次号,从0开始", "ShardRoundNo": 0, "":"/":"Str,集群启动状态,Init=集群未启动完成,FirstChange=等待初次迁移,Start=集群启动完成", "ClusterStartStatus": "Start", "":"/":"Str,灾备恢复状态,Init=灾备恢复未完成,FirstChange=等待初次迁移,Start=灾备恢复完成", "DisasterStartStatus": "Start", "Partition": [{ "":"/":"Str,分区号", "PartNo": "0", "":"/":"U32,每个分区最近一次读写核心迁移对应的轮次号", "ArbNo": 0, "":"/":"Str,分区组迁移状态,WriterConn=无需迁移,BeginChange=要迁移,CheckSyncSource=确定数据源中,FindSyncSource=确定数据源,WaitDataSource=等待数据源就绪,DataSourceWork=数据源就绪,WaitingSync=等待同步完成,SyncSuccess=同步完成,WaitAppReady=等待平台就绪,AppReadySucc=所有核心均平台就绪,WaitAdminPartChange=等待分区全部迁移,WaitWriterLogon=等待写核心重连,CancelSource=取消数据源中,NoneDataSource=取消数据源成功,WriterPrepare=等待写核心接管,WriterRecover=写核心接管成功,WaitLastWriter=无备机时等待原核心重启,WaitLastDataSource=无备机时等待原数据源重启,AdminGroupChangeWaitWriterSync=紧急运维迁移redo通道的写权限给新数据源时等待数据源同步完成", "GroupStatus": "WriterConn", "":"/":"Str,分区迁移状态,Normal=分区正常,WaitWriterStop=等待写核心停止处理,WaitReaderSync=等待读核心重放到特殊redo,RecvWriterStop=写核心停止处理该分区成功,RecvGatewayStop=网关停止转发该分区成功,RecvReaderSync=读核心重放到新轮次,PartChangeEnd=分区迁移结束", "PartStatus": "Normal", "":"/":"I32,迁移策略,AdminOrder=手动迁移,AutoChange=自动迁移给redo序号最小的核心", "MigrationStrategy": "AdminOrder", "":"/":"Str,持有写权限的分区组名,只有1个值", "Writer": ["multiproc_proc#0_partgroup_1"], "":"/":"Str,持有读权限的分区组名", "Reader": ["multiproc_proc#1_partgroup_1","multiproc_proc#2_partgroup_1"] },], },], }</pre>	

6.30.6. GetPartGroupInfo 获取分区组和核心的对应关系

功能名称	GetPartGroupInfo	版本号	V1.0.2501.0.1	更新日期	20260122
功能备注	获取分区组和核心的对应关系				
功能说明	获取分区组和核心的对应关系				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetPartGroupInfo				
入参说明					
无					
出参说明					

```
{
    "":"/":"U32,当前仲裁序号",
    "ArbNo": 0,
    "":"/":"U32,当前查询序号",
    "QueryNo": 0,
    "":"/":"Str,本地仲裁文件名",
    "ArbFile": "arb_result.json",
    "ShardNoGroup": [
        {
            "":"/":"Str,集群名_分片号",
            "ClusterName": "mdb_1",
            "":"/":"U64,分片内轮次号,从0开始",
            "ShardRoundNo": 0,
            "":"/":"Str,集群启动状态,Init=集群未启动完成,FirstChange=等待初次迁移,Start=集群启动完成",
            "ClusterStartStatus": "Start",
            "":"/":"Str,灾备恢复状态,Init=灾备恢复未完成,FirstChange=等待初次迁移,Start=灾备恢复完成",
            "DisasterStartStatus": "Start",
            "PartGroup": [
                {
                    "":"/":"Str,分区组名",
                    "GroupName": "multiproc_proc#0_partgroup_1",
                    "":"/":"Str,管理该分区组的核心节点名",
                    "NodeName": "proc#0",
                    "":"/":"Str,业务就绪状态,UnReady0=未选定0号分区的写核心,UnReady=等待核心业务就绪,Ready=业务就绪,注:目前非0号分区的核心都是UnReady的",
                    "BizStatus": "Ready",
                    "":"/":"Str,平台就绪状态,UnReady=分区未就绪,Ready=分区就绪",
                    "AppStatus": "Ready",
                    "":"/":"Str,读写状态,Writer=写核心,Reader=读核心",
                    "ArbStatus": "Writer",
                    "":"/":"Str,主机上报的该分区组的同步状态,UnSync=同步中,Sync=同步完成",
                    "SyncStatus": "UnSync",
                    "":"/":"Str,分区组迁移状态,Normal=写核心正常,RouterChange=需要分区组迁移,DataSource=作为数据同步源,SyncData=待同步数据,PrepareDS=准备成为数据源,PrepareW=准备恢复写核心",
                    "GroupStatus": "Normal",
                    "":"/":"U64,当前核心已经同步数据到该轮次号",
                    "ShardRoundNo": 0,
                    "":"/":"U64,上述轮次下,核心上报的最大redo序号",
                    "LastMsgNoInRound": 0,
                    "":"/":"U64,当前轮次号下的最小redo数据进度值",
                    "MinMessageNo": 0,
                    "":"/":"U64,redo数据最大进度值",
                    "MaxMessageNo": 0,
                    "":"/":"I32,状态受AppStatus控制的分区",
                    "AppPart": [0,1],
                    "":"/":"I32,分区读写迁移后,新分配的处于分区未就绪状态的分区,不受AppStatus管理",
                    "UnReadyPart": [2],
                    "":"/":"U32,等待平台就绪的仲裁序号",
                    "CheckBizArbNo": 0,
                    "":"/":"U32,等待分区就绪的仲裁序号",
                    "CheckAppArbNo": 0,
                    "":"/":"U32,等待备机确认同步状态变更的仲裁序号",
                    "WaitInActiveSynArbNo": 0,
                    "":"/":"U32,最近一次收到数据同步完成的仲裁序号",
                    "LastSybArbNo": 仲裁序号,
                    "":"/":"U32,最近一次收到redo进度的查询序号",
                    "LastRedoQueryNo": 查询序号
                },
            ],
        },
    ],
}
```

6.30.7. GetPartGroupRowInfo 获取分区组所在行和分区的对应关系

功能名称	GetPartGroupRowInfo	版本号	V1.0.0.0	更新日期	20250920
功能备注	获取分区组所在行和分区的对应关系				
功能说明	获取分区组所在行和分区的对应关系				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetPartGroupRowInfo				
入参说明					
无					
出参说明					

```
{
  "":"/":"U32,当前仲裁序号",
  "ArbNo": 0,
  "":"/":"U32,当前查询序号",
  "QueryNo": 0,
  "":"/":"Str,本地仲裁文件名",
  "ArbFile": "arb_result.json",
  "ShardNoGroup": [
    {
      "":"/":"Str,集群名_分片号",
      "ClusterName": "mdb_1",
      "":"/":"U64,分片内轮次号,从0开始",
      "ShardRoundNo": 0,
      "":"/":"Str,集群启动状态,Init=集群未启动完成,FirstChange=等待初次迁移,Start=集群启动完成",
      "ClusterStartStatus": "Start",
      "":"/":"Str,灾备恢复状态,Init=灾备恢复未完成,FirstChange=等待初次迁移,Start=灾备恢复完成",
      "DisasterStartStatus": "Start",
      "PartGroupRow": [
        {
          "":"/":"U32,分区组所在行,从1开始",
          "RowId": 1,
          "":"/":"Str,分区组迁移状态,WriterConn=无需迁移,BeginChange=要迁移,CheckSyncSource=确定数据源中,FindSyncSource=确定数据源,WaitDataSource=等待数据源就绪,DataSourceWork=数据源就绪,WaitingSync=等待同步完成,SyncSuccess=同步完成,WaitAppReady=等待平台就绪,AppReadySucc=所有核心均平台就绪,WaitAdminPartChange=等待分区全部迁移,WaitWriterLogon=等待写核心重连,CancelSource=取消数据源中,NoneDataSource=取消数据源成功,WriterPrepare=等待写核心接管,WriterRecover=写核心接管成功,WaitLastWriter=无备机时等待原核心重启,WaitLastDataSource=无备机时等待原数据源重启,AdminGroupChangeWaitWriterSync=紧急运维迁移redo通道的写权限给原核心时等待原核心同步完成,AdminGroupChangeWaitDataSourceSync=紧急运维迁移redo通道的写权限给新数据源时等待数据源同步完成",
          "Status": "WriterConn",
          "":"/":"Str,持有写权限的分区组名",
          "Writer": "multiproc_proc#0_partgroup_1",
          "":"/":"Str,持有读权限的分区组名",
          "Reader":["multiproc_proc#1_partgroup_1","multiproc_proc#2_partgroup_1"],
          "":"/":"I32,当前分区组所在行管理的分区列表",
          "Partition": [0,1,2],
          "":"/":"I32,分区迁移时有效,等待管理端迁移的分区列表",
          "OutputPart": [],
          "":"/":"U64,向数据源同步消息时有效,数据源的最大消息序号",
          "SyncMsgNo": 0,
          "":"/":"Str,分区迁移时有效,上次查询redo序号/数据同步完成的时间",
          "LastQueryTime": "2025-09-20 14:07:20.123",
          "":"/":"U32,等待数据同步完成的仲裁序号",
          "CheckSybArbNo": 0,
          "":"/":"U32,分区组迁移中消息,等待应答的仲裁序号",
          "CheckRTArbNo": 0,
          "":"/":"U32,等待redo进度的查询序号",
          "CheckRedoQueryNo": 0
        },
      ],
    },
  ],
}
```

6.30.8. GetHistoryPart 获取指定仲裁序号的的历史分区路由表

功能名称	GetHistoryPart	版本号	V1.0.0.0	更新日期	20250920
功能备注	获取指定仲裁序号的的历史分区路由表				
功能说明	获取指定仲裁序号的的历史分区路由表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetHistoryPart				
入参说明					
<pre>{ "":"/":"U32,查询的仲裁序号,默认为-1,表示查询最新仲裁序号", "ArbNo": -1 }</pre>					
出参(失败)说明					
<pre>{ "":"/":"U32,失败的错误号", "ErrorNo": 102, "":"/":"Str,失败的错误信息", "ErrorInfo": "查询不到对应的分区路由表" }</pre>					
出参说明					

```
{
  "":"/":"I32,版本信息,目前固定为0",
  "Version": 版本信息,
  "":"/":"I32,100=核心发出,101=网关发出,102=partkeeper发出,103=运维发出",
  "MsgType": 消息类型,
  "":"/":"I32,消息类型,参照事件ID说明",
  "EventType": 事件ID,
  "":"/":"U32,partkeeper端维护的轮次号,简称仲裁序号",
  "ArbNo": 0,
  "":"/":"U64,生成该分区路由表的时间,毫秒级",
  "Time": 1758261458743,
  "":"/":"Str,生成该分区路由表的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "TimeStr": "2025-09-19 13:57:38.743",
  "ShardNoGroup": [
    {
      "":"/":"Str,集群名_分片号",
      "ClusterName": "mdb_1",
      "":"/":"U64,分片内轮次号,从0开始",
      "ShardRoundNo": 0,
      "":"/":"Str,集群启动状态,Init=集群未启动完成,FirstChange=等待初次迁移,Start=集群启动完成",
      "StartStatus": "Start",
      "BizProcNode": [
        {
          "":"/":"Str,节点名",
          "NodeName": "todb#0",
          "":"/":"Str,工作状态,UnRegister=未登录,DisConnect=连接断开,Watcher=连接正常,但不参与读写核心迁移,Voter=参与读写核心迁移",
          "Online": "Watcher"
        }
      ],
      "Partition": [
        {
          "":"/":"Str,分区号",
          "PartNo": "0",
          "":"/":"U32,每个分区最近一次读写核心迁移对应的轮次号",
          "ArbNo": 0,
          "":"/":"Str,分区迁移状态,Normal=分区正常,WaitWriterStop=等待写核心停止处理,WaitReaderSync=等待读核心重放到特殊redo,RecvWriterStop=写核心停止处理该分区成功,RecvGatewayStop=网关停止转发该分区成功,RecvReaderSync=读核心重放到新轮次,PartChangeEnd=分区迁移结束",
          "PartStatus": "None",
          "":"/":"Str,持有写权限的分区组名,只有1个值",
          "Writer": ["multiproc_proc#0_partgroup_1"],
          "":"/":"Str,持有读权限的分区组名",
          "Reader": ["multiproc_proc#1_partgroup_1","multiproc_proc#2_partgroup_1"]
        }
      ],
      "PartGroup": [
        {
          "":"/":"Str,分区组名",
          "GroupName": "multiproc_proc#0_partgroup_1",
          "":"/":"Str,管理该分区组的核心节点名",
          "NodeName": "proc#0",
          "":"/":"Str,业务就绪状态,UnReady0=未选定0号分区的写核心,UnReady=等待核心业务就绪,Ready=业务就绪",
          "BizStatus": "Ready",
          "":"/":"Str,平台就绪状态,UnReady=分区未就绪,Ready=分区就绪",
          "AppStatus": "Ready",
          "":"/":"Str,读写状态,Writer=写核心,Reader=读核心",
          "ArbStatus": "Writer",
          "":"/":"Str,分区组迁移状态,Normal=写核心正常,RouterChange=需要分区组迁移,DataSource=作为数据同步源,SyncData=待同步数据,PrepareDS=准备成为数据源,PrepareW=准备恢复写核心",
          "GroupStatus": "None",
          "":"/":"U64,当前轮次号下的最小redo数据进度值",
          "MinMessageNo": 0,
          "":"/":"U64,redo数据最大进度值",
          "MaxMessageNo": 0,
          "":"/":"I32,分区读写迁移后,新分配的处于分区未就绪状态的分区,不受AppStatus管理",
          "UnReadyPart": [2]
        }
      ]
    }
  ]
}
```

6.30.9. GetHistoryRound 获取指定分片内轮次号的的历史分区路由表

功能名称	GetHistoryRound	版本号	V1.0.0.0	更新日期	20250920
功能备注	获取指定分片内轮次号的的历史分区路由表				
功能说明	获取指定分片内轮次号的的历史分区路由表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetHistoryRound				
入参说明					
<pre>{ "":"/":"Str,集群名_分片号", "ClusterName": "mdb_1", "":"/":"U64,查询的轮次号,默认为-1,查询最新轮次号", "ShardRoundNo": -1,}</pre>					
出参(失败)说明					
<pre>{ "":"/":"U32,失败的错误号", "ErrorNo": 102, "":"/":"Str,失败的错误信息", "ErrorInfo": "查询不到对应的分区路由表"}</pre>					
出参说明					

```
{
    "":"/":"I32,版本信息,目前固定为0",
    "Version": 版本信息,
    "":"/":"I32,100=核心发出,101=网关发出,102=partkeeper发出,103=运维发出",
    "MsgType": 消息类型,
    "":"/":"I32,消息类型,参照事件ID说明",
    "EventType": 事件ID,
    "":"/":"U32,partkeeper端维护的轮次号,简称仲裁序号",
    "ArbNo": 0,
    "":"/":"U64,生成该分区路由表的时间,毫秒级",
    "Time": 1758261458743,
    "":"/":"Str,生成该分区路由表的时间串,YYYY-MM-DD HH:MM:SS,sss",
    "TimeStr": "2025-09-19 13:57:38.743",
    "ShardNoGroup": [
        {
            "":"/":"Str,集群名_分片号",
            "ClusterName": "mdb_1",
            "":"/":"U64,分片内轮次号,从0开始",
            "ShardRoundNo": 0,
            "":"/":"Str,集群启动状态,Init=集群未启动完成,FirstChange=等待初次迁移,Start=集群启动完成",
            "StartStatus": "Start",
            "BizProcNode": [
                {
                    "":"/":"Str,节点名",
                    "NodeName": "todb#0",
                    "":"/":"Str,工作状态,UnRegister=未登录,DisConnect=连接断开,Watcher=连接正常,但不参与读写核心迁移,Voter=参与读写核心迁移",
                    "Online": "Watcher"
                }
            ],
            "Partition": [
                {
                    "":"/":"Str,分区号",
                    "PartNo": "0",
                    "":"/":"U32,每个分区最近一次读写核心迁移对应的轮次号",
                    "ArbNo": 0,
                    "":"/":"Str,分区迁移状态,Normal=分区正常,WaitWriterStop=等待写核心停止处理,WaitReaderSync=等待读核心重放到特殊redo,RecvWriterStop=写核心停止处理该分区成功,RecvGatewayStop=网关停止转发该分区成功,RecvReaderSync=读核心重放到新轮次,PartChangeEnd=分区迁移结束",
                    "PartStatus": "None",
                    "":"/":"Str,持有写权限的分区组名,只有1个值",
                    "Writer": ["multiproc_proc#0_partgroup_1"],
                    "":"/":"Str,持有读权限的分区组名",
                    "Reader": ["multiproc_proc#1_partgroup_1","multiproc_proc#2_partgroup_1"]
                }
            ],
            "PartGroup": [
                {
                    "":"/":"Str,分区组名",
                    "GroupName": "multiproc_proc#0_partgroup_1",
                    "":"/":"Str,管理该分区组的核心节点名",
                    "NodeName": "proc#0",
                    "":"/":"Str,业务就绪状态,UnReady0=未选定0号分区的写核心,UnReady1=等待核心业务就绪,Ready=业务就绪",
                    "BizStatus": "Ready",
                    "":"/":"Str,平台就绪状态,UnReady=分区未就绪,Ready=分区就绪,DataSource=作为数据同步源,SyncData=待同步数据",
                    "AppStatus": "Ready",
                    "":"/":"Str,读写状态,Writer=写核心,Reader=读核心",
                    "ArbStatus": "Writer",
                    "":"/":"Str,分区组迁移状态,Normal=写核心正常,RouterChange=需要分区组迁移,DataSource=作为数据同步源,SyncData=待同步数据,PrepareDS=准备成为数据源,PrepareW=准备恢复写核心",
                    "GroupStatus": "None",
                    "":"/":"U64,当前轮次号下的最小redo数据进度值",
                    "MinMessageNo": 0,
                    "":"/":"U64,redo数据最大进度值",
                    "MaxMessageNo": 0,
                    "":"/":"I32,分区读写迁移后,新分配的处于分区未就绪状态的分区,不受AppStatus管理",
                    "UnReadyPart": [2]
                }
            ]
        }
    ]
}
```

6.30.10. SendAdminMsg 运维人员进行手动分区迁移

功能名称	SendAdminMsg	版本号	V1.0.2501.0.1	更新日期	20260122
功能备注	运维人员进行手动分区迁移,将待迁移的分区划分到新的核心,注意,进行分区迁移时,必须保证所有通道上的写核心存活,或者能够正常重启,否则会导致其他核心等待下线的核心的轮次递增而无法完成本轮轮次切换,进而无法接收新轮次的数据。允许进行批量分区迁移(同时对多个分区进行迁移),但是注意正常迁移时,只允许将多个迁出核心的多个分区迁移给同一个核心;异常迁移时,只允许将一个核心持有的多个分区迁出给多个核心;禁止同时进行正常迁移和异常迁移。				
功能说明	这里下发的指令是被异步执行的,因此无法立即获取该指令的结果,需要通过生成的指令序号OrderId,到GetAdminMsgResult管理功能号去查询该指令的实际执行结果。详细使用说明参见专项说明的 partkeeper运维指令介绍章节。				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp20.partkeeper&FuncName=SendAdminMsg				

入参说明(分区迁移)

```
{
    "ShardNoGroup": [
        {
            "":"/":"Str,待迁出的分区号",
            "PartNo": "0",
            "":"/":"Str,待迁入的核心节点名",
            "NodeName": "proc#0"
        }
    ]
}
```


入参说明(分区组迁移)
<pre>{ "ShardNoGroup": [{ "/*": "Str, 集群名_分片号", "ClusterName": "mcb_1", "PartGroup": [{ "/*": "Str, 将redo通道的主机状态转移给该分区组", "GroupName": "multiproc_proc#0_partgroup_2", "/*": "Str, 该分区组状态迁移后的状态, PrepareDS=准备成为数据同步源", "GroupStatus": "PrepareDS" }] }] }</pre>
出参(失败)说明
<pre>{ "/*": "U32, 失败的错误号", "ErrorNo": 102, "/*": "Str, 失败的错误信息", "ErrorInfo": "没有给定ClusterName" }</pre>
出参说明(指令结果通过GetAdminMsgResult查询)
<pre>{ "/*": "I32, 版本信息, 目前固定为0", "Version": 0, "/*": "Str, admin=管理端下达", "NodeName": "admin", "/*": "I32, 100=核心发出, 101=网关发出, 102=partkeeper发出, 103=运维发出", "MsgType": 103, "/*": "I32, 消息类型, 参照事件ID说明", "EventType": 1099, "/*": "U32, partkeeper端维护的轮次号, 简称仲裁序号", "ArbNo": 0, "/*": "U32, 年龄", "Age": 0, "/*": "U32, 连接号", "SockIndex": 0, "/*": "U64, 当前指令对应的编号, 用于查询指令结果", "OrderId": 0, "/*": "U64, 生成该管理指令的时间, 毫秒级", "OrderTime": 1758261458743, "/*": "Str, 生成该指令的时间串, YYYY-MM-DD HH:MM:SS,sss", "TimeStr": "2025-09-19 13:57:38.743", "ShardNoGroup": [{ "/*": "Str, 集群名_分片号", "ClusterName": "mcb_1", "Partition": [{ "/*": "Str, 待迁出的分区号", "PartNo": "0", "/*": "Str, 待迁入的核心节点名", "NodeName": "proc#0" }] }] }</pre>

6.30.11. GetAdminMsgResult 获取管理指令的结果

功能名称	GetAdminMsgResult	版本号	V1.0.15.501	更新日期	20251029
功能备注	获取管理指令的结果				
功能说明	在SendAdminMsg下达的管理端指令,通过本管理功能可以查询到指令结果				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp20.partkeeper&FuncName=GetAdminMsgResult				
入参说明	<pre>{ "/*": "U64, 待查询的指令序号, -1表示查询最新处理的一条指令结果", "OrderId": -1 }</pre>				
出参(失败)说明	<pre>{ "/*": "U32, 失败的错误号", "ErrorNo": 0, "/*": "Str, 失败的错误信息", "ErrorInfo": "无法找到该指令序号的迁移结果, 可能指令消息还未处理" }</pre>				
出参说明(分区迁移结果)					

```
{
  "":"/":"I32,100=核心发出,101=网关发出,102=partkeeper发出,103=运维发出",
  "MsgType": 103,
  "":"/":"U64,当前指令对应的编号,用于查询指令结果",
  "OrderId": 1,
  "":"/":"Str,集群名_分片号",
  "ClusterName": "mclb_1",
  "":"/":"U64,分片内轮次号,从1开始",
  "ShardRoundNo": 1,
  "":"/":"Str,生成该指令的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "BeginTimeStr": "2025-10-29 11:41:28.705",
  "":"/":"Str,指令结果的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "EndTimeStr": "2025-10-29 11:41:28.895",
  "":"/":"Str,指令处理的整体耗时,ms",
  "Time": 190,
  "Partition": [
    {
      "":"/":"Str,待迁出的分区号",
      "PartNo": "0",
      "":"/":"Str,待迁出的核心节点名",
      "OldWriter": "proc#0",
      "":"/":"Str,待迁入的核心节点名",
      "NewWriter": "proc#1",
      "":"/":"U64,指令完成时,迁出分区组的redo数据起始进度值",
      "MinMessageNo": 18446744073709551615,
      "":"/":"U64,指令完成时,迁出分区组的redo数据最大进度值",
      "MaxMessageNo": 100
    }
  ],
  "":"/":"I32,分区迁移失败时存在,失败的错误码",
  "ErrorNo": 7,
}

{
  "":"/":"I32,100=核心发出,101=网关发出,102=partkeeper发出,103=运维发出",
  "MsgType": 103,
  "":"/":"U64,当前指令对应的编号,用于查询指令结果",
  "OrderId": 1,
  "":"/":"Str,集群名_分片号",
  "ClusterName": "mclb_1",
  "":"/":"U64,分片内轮次号,从1开始",
  "ShardRoundNo": 1,
  "":"/":"Str,生成该指令的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "BeginTimeStr": "2025-10-29 11:41:28.705",
  "":"/":"Str,指令结果的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "EndTimeStr": "2025-10-29 11:41:28.895",
  "":"/":"Str,指令处理的整体耗时,ms",
  "Time": 190,
  "PartGroup": [
    {
      "":"/":"Str,发生状态变更的分区组名",
      "GroupName": "multiproc_proc#0_partgroup_2",
      "":"/":"Str,变更前的分区组状态,Normal=写核心正常,RouterChange=需要分区组迁移,DataSource=作为数据同步源,SyncData=待同步数据,PrepareDS=准备成为数据源,PrepareW=准备恢复写核心",
      "OldStatus": "Normal",
      "":"/":"Str,变更后的分区组状态,Normal=写核心正常,RouterChange=需要分区组迁移,DataSource=作为数据同步源,SyncData=待同步数据,PrepareDS=准备成为数据源,PrepareW=准备恢复写核心",
      "NewStatus": "PrepareDS"
    }
  ],
  "":"/":"I32,分区迁移失败时存在,失败的错误码",
  "ErrorNo": 7,
  "":"/":"Str,分区迁移失败时存在,失败的错误信息",
  "ErrorInfo": "管理端进行分区迁移失败,原因 分区原本就归属核心[proc#3]无需迁移"
}
```

6.30.12. GetRouterChange 获取指定分片下的所有分区迁移历史

功能名称	GetRouterChange	版本号	V1.0.15.501	更新日期	20251029
功能备注	获取指定分片下的所有分区迁移历史				
功能说明	获取指定分片下的所有分区迁移历史				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp20.partkeeper&FuncName=GetRouterChange				
入参说明					
<pre>{ "":"/":"Str,集群名_分片号", "ClusterName": "mdb_1" }</pre>					
出参(失败)说明					
<pre>{ "":"/":"U32,失败的错误号", "ErrorNo": 0, "":"/":"Str,失败的错误信息", "ErrorInfo": "无法找到该指令序号的迁移结果,可能指令消息还未处理" }</pre>					
出参说明					

```
{
  "":"/":"Str,集群名_分片号",
  "ClusterName": "mcb_1",
  "":"/":"U64,分片内轮次号,从1开始",
  "ShardRoundNo": 1,
  "":"/":"Str,集群启动状态,Init=集群未启动完成,FirstChange=等待初次迁移,Start=集群启动完成",
  "StartStatus": "Start",
  "RouterChange": [
    {
      "":"/":"U64,当前指令对应的编号,用于查询指令结果",
      "OrderId": 1,
      "":"/":"U64,分片内轮次号,从1开始",
      "ShardRoundNo": 1,
      "":"/":"Str,生成该指令的时间串,YYYY-MM-DD HH:MM:SS,sss",
      "BeginTimeStr": "2025-10-29 11:41:28.705",
      "":"/":"Str,指令结果的时间串,YYYY-MM-DD HH:MM:SS,sss",
      "EndTimeStr": "2025-10-29 11:41:28.895",
      "":"/":"Str,指令处理的整体耗时,ms",
      "Time": 190,
      "Partition": [
        {
          "":"/":"Str,待迁出的分区号",
          "PartNo": "0",
          "":"/":"Str,待迁出的核心节点名",
          "OldWriter": "proc#0",
          "":"/":"Str,待迁入的核心节点名",
          "NewWriter": "proc#1",
          "":"/":"U64,指令完成时,迁出分区组的redo数据起始进度值",
          "MinMessageNo": 18446744073709551615,
          "":"/":"U64,指令完成时,迁出分区组的redo数据最大进度值",
          "MaxMessageNo": 100
        }
      ],
      "":"/":"I32,分区迁移失败时存在,失败的错误码",
      "ErrorNo": 7,
      "":"/":"Str,分区迁移失败时存在,失败的错误信息",
      "ErrorInfo": "管理端进行分区迁移失败,原因 分区原本就归属核心[proc#3]无需迁移"
    }
  ]
}
```

功能名称	GetStatistics	版本号	V1.0.0.0	更新日期	20250928
功能备注	获取统计信息				
功能说明	获取Keeper客户端的统计信息，包含完整分区表				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_partkeeper_client&FuncName=GetStatistics				
入参说明					
无					
出参说明					

```

{
    "":"/": "str, Keeper客户端版本",
    "KeeperClientVersion": "ldp_partkeeper_client V1.0.15.1",
    "":"/": "str, 应用名称",
    "AppName": "mdb#0",
    "":"/": "str, 分区键",
    "ShardNoKey": "mdb 1",
    "":"/": "bool, 是否在线",
    "IsOnline": true,
    "":"/": "bool, 客户端角色",
    "ClientRole": true,
    "":"/": "uint64, 本地仲裁编号",
    "LocalArbNo": 4,
    "":"/": "uint64, 本地年龄",
    "LocalAge": 2,
    "":"/": "uint64, 最后仲裁事件",
    "LastArbEvent": 1051,
    "":"/": "str, 最后仲裁事件描述",
    "LastArbEventStr": "keeper 下发集群启动初始路由表",
    "":"/": "str, 最后仲裁事件时间戳",
    "LastArbEventTimestamp": "2025-09-28 11:32:38.521",
    "":"/": "bool, 回调线程是否运行",
    "CallbackThreadRunning": true,
    "":"/": "uint64, 回调线程队列大小",
    "CallbackThreadQueueSize": 0,
    "":"/": "int32, Keeper连接状态",
    "KeeperConnected": 1,
    "":"/": "str, Keeper最后连接时间",
    "KeeperLastConnectedTime": "2025-09-28 11:31:51.661907",
    "":"/": "str, Keeper最后断开连接时间",
    "KeeperLastDisconnectedTime": "1970-01-01 08:00:00.000000",
    "":"/": "uint64, 会话超时时间, 毫秒",
    "SessionTimeoutMs": 600000,
    "":"/": "uint64, 最后消息时间戳",
    "LastMessageTimestamp": 3279669764,
    "":"/": "array, 注册的回调",
    "RegisteredCallbacks": [],
    "":"/": "object, 分区信息, key 对应 ShardNoKey",
    "mdb 1": {
        "":"/": "uint64, 轮次号",
        "ShardRoundNo": 0,
        "":"/": "object, 分区信息",
        "Partition": {
            "0": {
                "":"/": "uint64, 仲裁编号",
                "ArbNo": 2,
                "":"/": "int32, 状态 (0=正常, 1=等待取消写入者, 2=等待读取者同步, 3=收到写入者应答, 4=收到网关应答, 5=收到读取者应
                答, 6=迁移结束)",
                "Status": 0,
                "":"/": "array, 写入者, 只有一个",
                "Writer": [
                    "multiproc_mdb#0_partgroup 1"
                ],
                "":"/": "array, 读取者, 有多个",
                "Reader": [
                    "multiproc_mdb#2_partgroup 1",
                    "multiproc_mdb#1_partgroup 1"
                ],
                "":"/": "str, 状态描述",
                "statusStr": "NORMAL (0)"
            },
            ...
        },
        "":"/": "object, 节点信息",
        "BizProcNode": {
            "mdb#0": {
                "":"/": "int32, 在线状态 (0=未登录, 1=与keeper连接断开, 2=与keeper连接正常但不参与选举, 3=参与选举)",
                "Online": 3
            },
            ...
        },
        "":"/": "object, 分区组信息",
        "PartGroup": {
            "multiproc_mdb#0_partgroup 1": {
                "":"/": "str, 节点名称",
                "NodeName": "mdb#0",
                "":"/": "int32, 应用状态 (0=未就绪, 1=就绪)",
                "AppStatus": 1,
                "":"/": "int32, 业务状态 (0=未选定写入者, 1=业务未就绪, 2=业务就绪)",
                "BizStatus": 1,
                "":"/": "int32, 分区组状态 (0=写核心连接正常, 1=需要分区组迁移, 2=作为数据同步源, 3=同步数据到数据源, 4=数据同步源下
                线, 5=准备数据同步源, 6=准备恢复写核心)",
                "GroupStatus": 0,
                "":"/": "int32, 仲裁状态 (0=写入者, 1=读取者, 2=未确定读写状态)",
                "ArbStatus": 0,
                "":"/": "uint64, 最大消息编号",
                "MaxMessageNo": 10,
                "":"/": "uint64, 最小消息编号",
                "MinMessageNo": 0,
                "":"/": "int32, 当前角色 (0=写入者, 1=读取者)",
                "i32CurrentRole": 0,
                "":"/": "object, 未准备好的分区",
                "UnReadyPart": {},
                "":"/": "str, 应用状态描述",
                "appStatusStr": "Ready (1)",
                "":"/": "str, 业务状态描述",
                "bizStatusStr": "BIZ_NOT_READY (1)",
                "":"/": "str, 组状态描述",
                "groupStatusStr": "NORMAL (0)",
                "":"/": "str, 仲裁状态描述",
                "arbStatusStr": "WRITER (0)",
                "":"/": "str, 当前角色描述",
                "i32CurrentRoleStr": "WRITER (0)"
            },
            ...
        },
        ...
    }
}

```

6.31.2. GetPartitionInfo 获取分区信息

功能名称	GetPartitionInfo	版本号	V1.0.0.0	更新日期	20250928
功能备注	获取分区信息				

功能说明	获取Keeper客户端的分区信息
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_partkeeper_client&FuncName=GetPartitionInfo
入参说明	
无	
出参说明	
<pre>{ "/*": "str, Keeper客户端版本", "KeeperClientVersion": "ldp_partkeeper_client V1.0.15.1", "/*": "str, 应用名称", "AppName": "mdb#0", "/*": "str, 分区键", "ShardNoKey": "mdb_1", "/*": "bool, 路由表是否在线", "RouteTableOnline": true, "/*": "uint64, 本地仲裁编号", "LocalArbNo": 4, "/*": "uint64, 本地年龄", "LocalAge": 2, "/*": "uint64, 最后仲裁事件 (1001=登录应答, 1051=集群启动初始路由表, 1009=核心节点登入登出, 1033=分区组主节点变更, 1011=分区 迁移事件, 1025=核心业务状态变更, 1031=分区组平台状态变更, 1037=节点与keeper断开连接, 1047=主分区组下线, 1049=分区组迁移状态重置, 1057=取消分区写权限, 1061=分区迁移完成, 901=连接断开超时, 902=重连并登录完成)", "LastArbEvent": 1051, "/*": "str, 最后仲裁事件描述", "LastArbEventStr": "keeper 下发集群启动初始路由表", "/*": "str, 最后仲裁事件时间戳", "LastArbEventTimestamp": "2025-09-28 11:32:38.521", "/*": "bool, 路由表是否已初始化", "RouteTableInited": true, "/*": "object, 分区信息, key 对应 ShardNoKey", "mdb_1": { "/*": "uint64, 轮次号", "ShardRoundNo": 0, "/*": "object, 分区信息", "Partition": { "0": { "/*": "uint64, 仲裁编号", "ArbNo": 2, "/*": "int32, 状态 (0=正常, 1=等待取消写入者, 2=等待读取者同步, 3=收到写入者应答, 4=收到网关应答, 5=收到读取者应 答, 6=迁移结束)", "Status": 0, "/*": "array, 写入者, 只有一个", "Writer": ["multiproc_mdb#0_partgroup_1"], "/*": "array, 读取者, 有多个", "Reader": ["multiproc_mdb#2_partgroup_1", "multiproc_mdb#1_partgroup_1"], "/*": "str, 状态描述", "statusStr": "NORMAL (0)" }, ... }, ... }, ... }</pre>	

6.31.3. GetPartGroupInfo 获取分区组信息

功能名称	GetPartGroupInfo	版本号	V1.0.0.0	更新日期	20250928
功能备注	获取分区组信息				
功能说明	获取Keeper客户端的分区组信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_partkeeper_client&FuncName=GetPartGroupInfo				
入参说明					
无					
出参说明					

```
{
  "":"/: "str, Keeper客户端版本",
  "KeeperClientVersion": "ldp_partkeeper_client V1.0.15.1",
  "":"/: "str, 应用名称",
  "AppName": "mdb#0",
  "":"/: "str, 分区键",
  "ShardNoKey": "mdb 1",
  "":"/: "uint64, 本地仲裁编号",
  "ArbNo": 4,
  "":"/: "uint64, 本地年龄",
  "LocalAge": 2,
  "":"/: "uint64, 最后仲裁事件",
  "LastArbEvent": 1051,
  "":"/: "str, 最后仲裁事件描述",
  "LastArbEventStr": "keeper 下发集群启动初始路由表",
  "":"/: "str, 最后仲裁事件时间戳",
  "LastArbEventTimestamp": "2025-09-28 11:32:38.521",
  "":"/: "bool, 路由表是否已初始化",
  "RouteTableInited": true,
  "":"/: "object, 分区信息, key 对应 ShardNoKey",
  "mdb 1": {
    "":"/: "uint64, 轮次号",
    "ShardRoundNo": 0,
    "":"/: "object, 分区组信息",
    "PartGroup": {
      "multiproc_mdb#0_partgroup 1": {
        "":"/: "str, 节点名称",
        "NodeName": "mdb#0",
        "":"/: "int32, 应用状态 (0=未就绪, 1=就绪)",
        "AppStatus": 1,
        "":"/: "int32, 业务状态 (0=未选定写入者, 1=业务未就绪, 2=业务就绪)",
        "BizStatus": 1,
        "":"/: "int32, 分区组状态 (0=写核心连接正常, 1=需要分区组迁移, 2=作为数据同步源, 3=同步数据到数据源, 4=数据同步源下线, 5=准备数据同步源, 6=准备恢复写核心)",
        "GroupStatus": 0,
        "":"/: "int32, 仲裁状态 (0=写入者, 1=读取者, 2=未确定读写状态)",
        "ArbStatus": 0,
        "":"/: "uint64, 最大消息编号",
        "MaxMessageNo": 10,
        "":"/: "uint64, 最小消息编号",
        "MinMessageNo": 0,
        "":"/: "int32, 当前角色 (0=写入者, 1=读取者)",
        "i32CurrentRole": 0,
        "":"/: "object, 未准备好的分区",
        "UnReadyPart": {},
        "":"/: "str, 应用状态描述",
        "appStatusStr": "Ready (1)",
        "":"/: "str, 业务状态描述",
        "bizStatusStr": "BIZ_NOT_READY (1)",
        "":"/: "str, 组状态描述",
        "groupStatusStr": "NORMAL (0)",
        "":"/: "str, 仲裁状态描述",
        "arbStatusStr": "WRITER (0)",
        "":"/: "str, 当前角色描述",
        "i32CurrentRoleStr": "WRITER (0)"
      },
      ...
    }
  }
}
```

6.32. 数据转换插件 ldp_datamapper

6.32.1. GetAdapterInfo 获取适配器信息

功能名称	GetAdapterInfo	版本号	V1.0.0.1	更新日期	20251114
功能备注	获取适配器信息				
功能说明	展示配置中所有的适配器信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_datamapper&InstanceNo=0&FuncName=GetAdapterInfo				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ //适配器总数 "total": 1, "adapters": [{ "":"/: "适配器名称", "adapterName": "adapter.so", "":"/: "适配器版本号", "version": "AdapterDemo V1.1", "fieldTranslators": { "total": 1, "functions": [{ "name": "FieldFunc" }] }, "funcTranslators": { "total": 1, "functions": [{ "name": "FuncFunc" }] } }] }</pre>					

6.32.2. GetDictInfo 获取字典相关信息

功能名称	GetDictInfo	版本号	V1.0.0.1	更新日期	20251114
功能备注	获取字典相关信息				
功能说明	获取字典相关信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_datamapper&InstanceNo=0&FuncName=GetDictInfo				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ "total": 1, "dictionaries": [{ "dictName": "dtDict", "entryCount": 6, "accessCount": 0, "entries": [{ "srcValue": "1", "destValue": "a" }, { "srcValue": "90", "destValue": "ee" }, { "srcValue": "78", "destValue": "dd" }, { "srcValue": "56", "destValue": "cc" }, { "srcValue": "34", "destValue": "bb" }, { "srcValue": "12", "destValue": "aa" }] }] }</pre>					

6.32.3. GetFunctionInfo 获取每个功能号的处理规则

功能名称	GetFunctionInfo	版本号	V1.0.0.1	更新日期	20251114
功能备注	获取每个功能号的处理规则				
功能说明	获取每个功能号的处理规则				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_datamapper&InstanceNo=0&FuncName=GetFunctionInfo				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ "FunctionInfo": { "total": 4, "functions": [{ "functionId": "1111", "ruleName": "rule03", "bizFunc": "", "dictName": "dtDict3" }, { "functionId": "620701", "ruleName": "rule01", "bizFunc": "FuncFunc", "dictName": "" }, { "functionId": "620702", "ruleName": "rule01", "bizFunc": "FuncFunc", "dictName": "" }, { "functionId": "76**", "ruleName": "rule02", "bizFunc": "FuncFunc", "dictName": "dtDict2" }] }</pre>					

6.32.4. GetRuleInfo 获取规则的配置

功能名称	GetRuleInfo	版本号	V1.0.0.1	更新日期	20251114
------	-------------	-----	----------	------	----------

功能备注	获取规则的配置
功能说明	获取规则的配置
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_datamapper&InstanceNo=0&FuncName=GetRuleInfo
入参说明	
Pretty	1-美化json输出, 0-不美化
出参说明	
<pre>{ "total": 1, "rules": [{ "ruleName": "rule01", "fieldCount": 5, "fields": [{ "fieldName": "Data4", "dictName": "dtDict6", "srcSplit": " ", "destSplit": "", "bizFunc": "", "needSplit": true }] }] }</pre>	

6.33. 流量管控服务插件 flowcontrol

6.33.1. GetBlackRulesInfo 获取黑名单规则信息

功能名称	GetBlackRulesInfo	版本号	V1.0.0.0	更新日期	20251125
功能备注	获取黑名单规则信息				
功能说明	获取黑名单规则信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=GetBlackRulesInfo				
入参说明					
JSON Schema:					
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "Nodes": { "type": "string", "description": "请求参数，查询节点名。如果为所有节点，填写 'ALL'。", "examples": ["front#0", "ALL"] }, "Offset": { "type": "integer", "minimum": 0, "description": "从哪一条记录开始查询。例如，Offset为100表示从第101条记录开始。", "examples": [100] }, "PageSize": { "type": "integer", "minimum": 1, "maximum": 100, "description": "最多取多少条记录。例如，PageSize为100表示最多取100条记录。", "examples": [100] } }, "required": ["Nodes", "Offset", "PageSize"], "additionalProperties": false }</pre>					
入参示例:					
<pre>{ "Nodes" : "front#0", // 请求参数，查询节点名，如果为所有，就填"ALL" "Offset" : 100, // 从101条开始 "PageSize" : 100 // 最多取100条 }</pre>					
出参说明					
JSON Schema:					


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "BlackList": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "ID": {
            "type": "string",
            "description": "规则ID",
            "examples": ["123456-11-333"]
          },
          "Name": {
            "type": "string",
            "description": "规则名称",
            "examples": ["示例黑名单3"]
          },
          "Enable": {
            "type": "boolean",
            "description": "是否应用此规则",
            "examples": [true, false]
          },
          "Functions": {
            "type": "string",
            "description": "功能号名单，用分号分隔。可为空。",
            "examples": ["1000;1001;20??;3350021", ""]
          },
          "Accounts": {
            "type": "string",
            "description": "账号名单，用分号分隔。可为空。",
            "examples": ["0001;0002;0003;0000000500", ""]
          },
          "NodeNos": {
            "type": "string",
            "description": "生效的分片号列表，用分号分隔。可为空。",
            "examples": ["1;2;3", ""]
          },
          "Nodes": {
            "type": "string",
            "description": "生效的节点名列表，用分号分隔。可为空。",
            "examples": ["Front#0;front#1", ""]
          }
        }
      },
      "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes"]
    }
  },
  "required": ["BlackList"],
  "additionalProperties": false
}
```

出参示例：

```
{
  "BlackList": [
    {
      "ID": "123456-11-333", // 规则Id
      "Name": "示例黑名单3", // 规则名
      "Enable": true, // 是否应用
      "Functions": "1000;1001;20??;3350021", // 功能号名单
      "Accounts": "0001;0002;0003;0000000500", // 账号名单
      "NodeNos": "1;2;3", // 生效的分片号列表
      "Nodes": "Front#0;front#1" // 生效的节点名列表
    }
  ]
}
```

6.33.2. GetWhiteRulesInfo 获取白名单规则信息

功能名称	GetWhiteRulesInfo	版本号	V1.0.0.0	更新日期	20251125
功能备注	获取白名单规则信息				
功能说明	获取白名单规则信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=GetWhiteRulesInfo				

入参说明

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "Nodes": {
      "type": "string",
      "description": "请求参数，查询节点名。如果为所有节点，填写 'ALL'。",
      "examples": ["front#0", "ALL"]
    },
    "Offset": {
      "type": "integer",
      "minimum": 0,
      "description": "从哪一条记录开始查询。例如，Offset为100表示从第101条记录开始。",
      "examples": [100]
    },
    "PageSize": {
      "type": "integer",
      "minimum": 1,
      "maximum": 100,
      "description": "最多取多少条记录。例如，PageSize为100表示最多取100条记录。",
      "examples": [100]
    }
  },
  "required": ["Nodes", "Offset", "PageSize"],
  "additionalProperties": false
}
```

入参示例:

```
{
  "Nodes" : "front#0", // 请求参数，查询节点名，如果为所有，就填"ALL"
  "Offset" : 100,      // 从101条开始
  "PageSize" : 100     // 最多取100条
}
```

出参说明

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "WhiteList": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "ID": {
            "type": "string",
            "description": "规则ID",
            "examples": ["wabcl"]
          },
          "Name": {
            "type": "string",
            "description": "规则名称",
            "examples": ["示例白名单模板"]
          },
          "Enable": {
            "type": "boolean",
            "description": "是否启用此规则",
            "examples": [false, true]
          },
          "Functions": {
            "type": "string",
            "description": "功能号列表，用分号分隔。可为空。",
            "examples": ["1000;1001;20??;40??", ""]
          }
        },
        "Accounts": {
          "type": "string",
          "description": "账号列表，用分号分隔。可为空。",
          "examples": ["0001;0002;0003;0000000500", ""]
        },
        "NodeNos": {
          "type": "string",
          "description": "分片号列表，用分号分隔。可为空。",
          "examples": ["1", ""]
        },
        "Nodes": {
          "type": "string",
          "description": "生效节点值，只能是前置节点或者列表。如果为空字符串，表示规则尚未生效，只是模板。",
          "examples": ["", "fronthq#0;fronth1#1"]
        }
      },
      "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes"]
    }
  },
  "required": ["WhiteList"],
  "additionalProperties": false
}
```

出参示例:

```
{
  "WhiteList": [
    {
      "ID": "wabcl",
      "Name" : "示例白名单模板",
      "Enable": false,
      "Functions": "1000;1001;20??;40??",
      "Accounts": "0001;0002;0003;0000000500",
      "NodeNos": "1",
      "Nodes": ""
    }
  ]
}
```

6.33.3. GetActiveRulesInfo 获取生效规则信息

功能名称	GetActiveRulesInfo	版本号	V1.0.0.0	更新日期	20251125
功能备注	获取生效规则信息				
功能说明	获取生效规则信息				
访问链接	http://ip:port/kjp?Pretty=1&PluginName=kjp_flowcontrol&InstanceNo=0&FuncName=GetActiveRulesInfo				
入参说明					
JSON Schema:					
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "Type": { "type": "string", "description": "规则类型, 可以是 'WhiteList' 或 'BlackList'。", "enum": ["WhiteList", "BlackList"], "examples": ["WhiteList", "BlackList"] } }, "required": ["Type"], "additionalProperties": false }</pre>					
入参示例:					
<pre>{ "Type" : "WhiteList" , // 或者 "BlackList" }</pre>					
出参说明					
JSON Schema:					
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "RuleInfo": { "type": "array", "items": { "type": "object", "properties": { "ID": { "type": "string", "description": "规则ID", "examples": ["wabcl"] }, "Name": { "type": "string", "description": "规则名称", "examples": ["示例白名单"] }, "Enable": { "type": "boolean", "description": "是否启用此规则", "examples": [true, false] }, "Functions": { "type": "string", "description": "功能号列表, 用分号分隔。可为空。", "examples": ["1000;1001;20??;40??", ""] }, "Accounts": { "type": "string", "description": "账号列表, 用分号分隔。可为空。", "examples": ["0001;0002;0003;00000000500", ""] }, "NodeNos": { "type": "string", "description": "分片号列表, 用分号分隔。可为空。", "examples": ["1", ""] }, "Nodes": { "type": "string", "description": "生效节点值, 只能是前置节点或者列表。如果为空字符串, 表示规则尚未生效, 只是模板。", "examples": ["", "fronthq#0;fronth1#1"] }, "ActiveTime": { "type": "string", "format": "date-time", "description": "规则的生效时间", "examples": ["2025-11-24 10:19:45"] }, "RuleType": { "type": "string", "description": "规则的类型, 可以是 'WhiteList' 或 'BlackList'。", "enum": ["WhiteList", "BlackList"], "examples": ["BlackList", "WhiteList"] } }, "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes", "ActiveTime", "RuleType"] } } }, "required": ["RuleInfo"], "additionalProperties": false }</pre>					
出参示例:					

```
{
  "RuleInfo": [
    {
      "ID": "wabc1",
      "Name" : "示例白名单",
      "Enable": true,
      "Functions": "1000;1001;20??;40??",
      "Accounts": "0001;0002;0003;0000000500",
      "NodeNos": "1",
      "Nodes": "",
      "ActiveTime" : "2025-11-24 10:19:45", // 生效时间
      "RuleType" : "BlackList" // 规则的类型
    }
  ]
}
```

6.33.4. UpdateRulesInfo 更新规则信息

功能名称	UpdateRulesInfo	版本号	V1.0.0.0	更新日期	20251125
功能备注	更新或者新增规则信息				
功能说明	更新规则信息				
访问链接	http://ip:port/klp?Pretty=1&PluginName=klp_flowcontrol&InstanceNo=0&FuncName=UpdateRulesInfo				

入参说明

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "RuleInfo": {
      "type": "object",
      "properties": {
        "BlackList": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "ID": {
                "type": "string",
                "description": "规则ID",
                "examples": ["1234567890-1234567890-1234567890-1234567890"]
              },
              "Name": {
                "type": "string",
                "description": "规则名称",
                "examples": ["黑名单账户3"]
              },
              "Enable": {
                "type": "boolean",
                "description": "是否启用此规则",
                "examples": [true]
              },
              "Functions": {
                "type": "string",
                "description": "功能列表，用逗号分隔",
                "examples": [""]
              },
              "Accounts": {
                "type": "string",
                "description": "账号列表，用分号分隔",
                "examples": ["90001;90002;90003"]
              },
              "NodeNos": {
                "type": "string",
                "description": "分片号列表，用分号分隔",
                "examples": [""]
              }
            },
            "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes"]
          },
          "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes"]
        },
        "WhiteList": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "ID": {
                "type": "string",
                "description": "规则ID",
                "examples": ["123456-11-333"]
              },
              "Name": {
                "type": "string",
                "description": "规则名称",
                "examples": ["系统接入白名单"]
              },
              "Enable": {
                "type": "boolean",
                "description": "是否启用此规则",
                "examples": [true]
              },
              "Functions": {
                "type": "string",
                "description": "功能列表，用逗号分隔",
                "examples": ["6?;8?;1001;200?;30?;335002"]
              },
              "Accounts": {
                "type": "string",
                "description": "账号列表，用分号分隔",
                "examples": [""]
              },
              "NodeNos": {
                "type": "string",
                "description": "分片号列表，用分号分隔",
                "examples": ["1"]
              },
              "Nodes": {
                "type": "string",
                "description": "生效节点值，只能是前置节点或者列表。如果为空字符串，表示规则尚未生效，只是模板",
                "examples": ["ALL", ""]
              }
            },
            "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes"]
          },
          "required": ["BlackList", "WhiteList"]
        }
      },
      "required": ["RuleInfo"],
      "additionalProperties": false
    }
  }
}
```

入参示例：

```
"RuleInfo": {
  "BlackList": [
    {
      "ID": "1234567890-1234567890-1234567890-1234567890",
      "Name" : "黑名单账户3",
      "Enable": true,
      "Functions": "",
      "Accounts": "90001;90002;90003", //账号列表
      "NodeNos": "",
      "Nodes": "fronthq#0;fronthl#1", //带分片的，生效节点值只能是前置节点或者列表，如果为"", 那么说说明还没有生效，只是模板
    }
  ],
  "WhiteList": [
    {
      "ID": "123456-11-333", //规则Id
      "Name" : "系统接入白名单",
      "Enable": true,
      "Functions": "6?;8?;1001;200?;30??;335002",
      "Accounts": "",
      "NodeNos": "1",
      "Nodes": "ALL"
    }
  ]
},
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "Result": {
      "type": "array",
      "items": {
        "type": "object",
        "oneOf": [
          {
            "properties": {
              "ID": {
                "type": "string",
                "description": "规则ID",
                "examples": ["123456-11-333", "1234567890-1234567890-1234567890-1234567890"]
              },
              "Result": {
                "type": "string",
                "description": "执行结果，如果成功则为 'ok'. ",
                "examples": ["ok"]
              }
            },
            "required": ["ID", "Result"]
          },
          {
            "properties": {
              "ID": {
                "type": "string",
                "description": "规则ID",
                "examples": ["123456-11-333", "1234567890-1234567890-1234567890-1234567890"]
              },
              "ErrorNo": {
                "type": "string",
                "description": "错误编号，如果执行失败则包含具体的错误编号。",
                "examples": ["7951"]
              },
              "ErrorInfo": {
                "type": "string",
                "description": "错误信息，如果执行失败则包含具体的错误信息。",
                "examples": ["运行时出错"]
              }
            },
            "required": ["ID", "ErrorNo", "ErrorInfo"]
          }
        ]
      }
    },
    "required": ["Result"],
    "additionalProperties": false
  }
}
```

出参示例：

```
{
  "Result" : [
    {
      "ID": "123456-11-333",
      "Result": "ok"
    },
    {
      "ID": "1234567890-1234567890-1234567890-1234567890",
      "ErrorNo": "7951",
      "ErrorInfo": "运行时出错"
    }
  ]
  // 没在里边的，说明没有被执行，就是失败
}
```

6.33.5. DeleteRulesInfo 删除规则信息

功能名称	DeleteRulesInfo	版本号	V1.0.0.0	更新日期	20251125
功能备注	删除规则信息				
功能说明	删除规则信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=DeleteRulesInfo				
入参说明					
JSON Schema:					

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "RuleInfo": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "ID": {
            "type": "string",
            "description": "规则ID",
            "examples": ["1234567890-1234567890-1234567890-1234567890", "123456-11-333"]
          }
        },
        "required": ["ID"]
      }
    },
    "required": ["RuleInfo"],
    "additionalProperties": false
  }
}
```

入参示例:

```
"RuleInfo": [
  {
    "ID": "1234567890-1234567890-1234567890-1234567890", //规则Id
  },
  {
    "ID": "123456-11-333", //规则Id
  }
]
```

出参说明

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "Result": {
      "type": "array",
      "items": {
        "type": "object",
        "oneOf": [
          {
            "properties": {
              "ID": {
                "type": "string",
                "description": "规则ID",
                "examples": ["123456-11-333", "1234567890-1234567890-1234567890-1234567890"]
              },
              "Result": {
                "type": "string",
                "description": "执行结果, 如果成功则为 'ok'. ",
                "examples": ["ok"]
              }
            },
            "required": ["ID", "Result"]
          },
          {
            "properties": {
              "ID": {
                "type": "string",
                "description": "规则ID",
                "examples": ["123456-11-333", "1234567890-1234567890-1234567890-1234567890"]
              },
              "ErrorNo": {
                "type": "string",
                "description": "错误编号, 如果执行失败则包含具体的错误编号.",
                "examples": ["7951"]
              },
              "ErrorInfo": {
                "type": "string",
                "description": "错误信息, 如果执行失败则包含具体的错误信息.",
                "examples": ["运行时出错"]
              }
            },
            "required": ["ID", "ErrorNo", "ErrorInfo"]
          }
        ]
      }
    },
    "required": ["Result"],
    "additionalProperties": false
  }
}
```

出参示例:

```
{
  "Result" : [
    {
      "ID": "123456-11-333",
      "Result": "ok"
    },
    {
      "ID": "1234567890-1234567890-1234567890-1234567890",
      "ErrorNo": "7951",
      "ErrorInfo": "运行时出错"
    }
  ] // 没在里边的, 说明没有被执行, 就是失败
}
```

--

6.33.6. GetDSConfig 获取规则使能情况

功能名称	GetDSConfig	版本号	V1.0.0.0	更新日期	20251125
功能备注	获取规则使能情况				
功能说明	获取规则使能情况				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=GetDSConfig				
入参说明					
无					
出参说明					
JSON Schema:					
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "isEnabled": { "type": "boolean", "description": "是否开启熔断限流功能。true表示开启，false表示关闭。", "examples": [true, false] } }, "required": ["isEnabled"], "additionalProperties": false }</pre>					
出参示例:					
<pre>{ "isEnabled": true }</pre>					

6.33.7. SetDSConfig 设置规则使能情况

功能名称	SetDSConfig	版本号	V1.0.0.0	更新日期	20251125
功能备注	设置规则使能情况				
功能说明	设置规则使能情况				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=SetDSConfig				
入参说明					
JSON Schema:					
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "isEnabled": { "type": "boolean", "description": "是否开启熔断限流功能。true表示开启，false表示关闭。", "examples": [true, false] } }, "required": ["isEnabled"], "additionalProperties": false }</pre>					
入参示例:					
<pre>{ "isEnabled": true //true 开启 false 关闭 }</pre>					
出参说明					
JSON Schema:					
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "isEnabled": { "type": "boolean", "description": "是否开启熔断限流功能。true表示开启，false表示关闭。", "examples": [true, false] } }, "required": ["isEnabled"], "additionalProperties": false }</pre>					
出参示例:					

6.33.8. GetStatistics 获取统计信息

功能名称	GetStatistics	版本号	V1.0.0.0	更新日期	20251125
功能备注	获取统计信息				
功能说明	获取统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=kfp_flowcontrol&InstanceNo=0&FuncName=GetStatistics				
入参说明					
无					
出参说明					

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "AllStatistics": {
      "type": "object",
      "properties": {
        "MissCount": {
          "type": "integer",
          "description": "规则装载失败计数",
          "examples": [0]
        },
        "DelRules": {
          "type": "integer",
          "description": "待删除规则数，因删除或者更新过的规则需要异步清理",
          "examples": [40]
        },
        "Statistics": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "RuleType": {
                "type": "string",
                "description": "规则类型，可以是 'BlackList', 'WhiteList' 或 'Tps'",
                "enum": ["BlackList", "WhiteList", "Tps"],
                "examples": ["BlackList", "WhiteList", "Tps"]
              },
              "DefHitCount": {
                "type": "integer",
                "description": "通过记录数（对于 Tps 类型，为通过记录数）",
                "examples": [612420, 602420]
              },
              "HitCount": {
                "type": "integer",
                "description": "丢包记录数（对于 Tps 类型，为限流次数）",
                "examples": [0]
              },
              "Items": {
                "type": "array",
                "items": {
                  "type": "object",
                  "properties": {
                    "HitCount": {
                      "type": "integer",
                      "description": "规则命中记录数（对于 Tps 类型，为限流次数）",
                      "examples": [612420, 0]
                    },
                    "WarnTimes": {
                      "type": "integer",
                      "description": "告警次数",
                      "examples": [0, 5]
                    },
                    "ID": {
                      "type": "string",
                      "description": "规则ID",
                      "examples": ["222-0", "193-1234567890-222-0", "1234567890-999-0"]
                    },
                    "HitTime": {
                      "type": "string",
                      "format": "date-time",
                      "description": "最近一次命中时间（对于 Tps 类型，为流控最近一次命中，通过时间）",
                      "examples": ["2025-12-08 09:45:33"]
                    },
                    "DefHitTime": {
                      "type": "string",
                      "format": "date-time",
                      "description": "最近一次通过时间（对于 Tps 类型，为流控最近一次不通过时间）",
                      "examples": ["2025-12-08 09:45:34"]
                    },
                    "WarnTime": {
                      "type": "string",
                      "format": "date-time",
                      "description": "最近一次告警时间",
                      "examples": ["2025-12-08 09:45:34", ""]
                    }
                  },
                  "required": ["HitCount", "WarnTimes", "ID", "HitTime", "DefHitTime", "WarnTime"]
                }
              }
            },
            "required": ["RuleType", "DefHitCount", "HitCount", "Items"]
          },
          "required": ["MissCount", "DelRules", "Statistics"]
        },
        "required": ["AllStatistics"],
        "additionalProperties": false
      }
    }
  }
```

出参示例：

```
{
  "AllStatistics": {
    "MissCount": 0, //规则装载失败计数
    "DelRules": 40, //待删除规则数，因删除或者更新过的规则需要异步清理
    "Statistics": [
      {
        "RuleType": "BlackList", //规则类型
        "DefHitCount": 612420, //通过记录数
        "HitCount": 0, //丢包记录数
        "Items": [
          {
            "HitCount": 612420, //规则命中记录数
            "WarnTimes": 0,
            "ID": "222-0",
            "HitTime": "2025-12-08 09:45:33", // 黑名单最近一次命中时间
            "DefHitTime": "2025-12-08 09:45:34", // 黑名单最近一次通过时间
            "WarnTime": "" // 最近一次告警时间
          },
          {
            "HitCount": 0,
            "WarnTimes": 0,
            "ID": "193-1234567890-222-0",
            "HitTime": "1970-01-01 00:00:00", // 黑名单最近一次命中时间
            "DefHitTime": "2025-12-08 09:45:34", // 黑名单最近一次通过时间
            "WarnTime": "" // 最近一次告警时间
          }
        ]
      },
      {
        "RuleType": "WhiteList",
        "DefHitCount": 612420, //通过记录数
        "HitCount": 0, //丢包记录数
        "Items": [
          {
            "HitCount": 0,
            "WarnTimes": 0,
            "ID": "1234567890-999-0",
            "HitTime": "2025-12-08 09:45:33", // 白名单最近一次命中时间
            "DefHitTime": "2025-12-08 09:45:34", // 白名单最近一次不通过时间
            "WarnTime": "" // 最近一次告警时间
          },
          {
            "HitCount": 0,
            "WarnTimes": 0,
            "ID": "1234567890-222-0",
            "HitTime": "2025-12-08 09:45:33", // 白名单最近一次命中时间
            "DefHitTime": "2025-12-08 09:45:34", // 白名单最近一次不通过时间
            "WarnTime": "" // 最近一次告警时间
          }
        ]
      }
    ],
    {
      "RuleType": "Tps",
      "DefHitCount": 612420, //通过记录数
      "HitCount": 0, //限流次数
      "Items": [
        {
          "HitCount": 0, //限流次数
          "WarnTimes": 5, // 告警次数
          "ID": "1234567890-999-0",
          "HitTime": "2025-12-08 09:45:33", // 流控最近一次命中时间
          "DefHitTime": "2025-12-08 09:45:34", // 流控最近一次不通过时间
          "WarnTime": "2025-12-08 09:45:34" // 最近一次告警时间
        },
        {
          "HitCount": 0, //限流次数
          "WarnTimes": 5, // 告警次数
          "ID": "1234567890-222-0",
          "HitTime": "2025-12-08 09:45:33", // 流控最近一次命中时间
          "DefHitTime": "2025-12-08 09:45:34", // 流控最近一次通过时间
          "WarnTime": "2025-12-08 09:45:34" // 最近一次告警时间
        }
      ]
    }
  ]
}
```

6

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "Result": {
      "type": "string",
      "description": "操作结果，表示操作是否成功。通常为 '成功' 或 '失败'。",
      "examples": ["成功", "失败"]
    },
    "required": ["Result"],
    "additionalProperties": false
  }
}
```

出参示例:

```
{
  "Result" : "成功"
}
```

6.33.10. GetStatus 获取插件工作情况

功能名称	GetStatus	版本号	V1.0.0.0	更新日期	20251125
功能备注	获取插件工作情况				
功能说明	获取插件工作情况				

访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=GetStatus
入参说明	
无	
出参说明	
JSON Schema: <pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "Status": { "type": "string", "description": "服务状态, 可以是 'Normal'、'Initing' 或 'Runtime Error'。分别表示正常状态、初始化中、运行中出错。如果有JSON解析报错, 也是该状态。", "enum": ["Normal", "Initing", "Runtime Error"], "examples": ["Normal", "Initing", "Runtime Error"] } }, "required": ["Status"], "additionalProperties": false }</pre>	
出参示例: <pre>{ "Status": "Normal" // "Initing"、"Runtime Error"分别表示正常状态, 初始化中, 运行中出错, 如果有Json解析报错, 也是该状态 }</pre>	

6.33.11. GetTpsRulesInfo 获流控规则信息

功能名称	GetTpsRulesInfo	版本号	V1.0.0.0	更新日期	20260121
功能备注	获流控规则信息				
功能说明	获流控规则信息				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=GetTpsRulesInfo				
入参说明					
JSON Schema:					
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "Nodes": { "type": "string", "description": "请求参数, 查询节点名。如果为所有节点, 填写'ALL'。", "examples": ["front#0", "ALL"] }, "Offset": { "type": "integer", "minimum": 0, "description": "从哪一条记录开始查询。例如, Offset为100表示从第101条记录开始。", "examples": [100] }, "PageSize": { "type": "integer", "minimum": 1, "maximum": 100, "description": "最多取多少条记录。例如, PageSize为100表示最多取100条记录。", "examples": [100] } }, "required": ["Nodes", "Offset", "PageSize"], "additionalProperties": false }</pre>					
入参示例:					
<pre>{ "Nodes" : "front#0", // 请求参数, 查询节点名, 如果为所有, 就填"ALL" "Offset" : 100, // 从101条开始 "PageSize" : 100 // 最多取100条 }</pre>					
出参说明					
JSON Schema:					

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "Tps": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "ID": {
            "type": "string",
            "description": "规则ID",
            "examples": ["wabc1"]
          },
          "Name": {
            "type": "string",
            "description": "规则名称",
            "examples": ["示例白名单模板"]
          },
          "Enable": {
            "type": "boolean",
            "description": "是否启用此规则",
            "examples": [false, true]
          },
          "Functions": {
            "type": "string",
            "description": "功能号列表，用分号分隔。可为空。",
            "examples": ["1000;1001;20??;40??", ""]
          },
          "Accounts": {
            "type": "string",
            "description": "账号列表，用分号分隔。可为空。",
            "examples": ["0001;0002;0003;0000000500", ""]
          },
          "NodeNos": {
            "type": "string",
            "description": "分片号列表，用分号分隔。可为空。",
            "examples": ["1", ""]
          },
          "Nodes": {
            "type": "string",
            "description": "生效节点值，只能是前置节点或者列表。如果为空字符串，表示规则尚未生效，只是模板。",
            "examples": ["", "fronthq#0;fronth1#1"]
          },
          "Fc": {
            "type": "object",
            "description": "流控规则配置",
            "properties": {
              "Duration": {
                "type": "integer",
                "description": "流控时间窗口，单位为秒。例如30秒为一个时间维度，第一笔交易为开始。",
                "examples": [30]
              },
              "Warning": {
                "type": "integer",
                "description": "警告阈值，当交易笔数达到该值时，系统会记录一条警告日志。",
                "examples": [5000]
              },
              "Limit": {
                "type": "integer",
                "description": "限制阈值，当交易笔数超过该值时，后续交易将被丢弃。",
                "examples": [10000]
              }
            },
            "required": ["Duration", "Warning", "Limit"]
          }
        },
        "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes", "Fc"]
      }
    },
    "required": ["Tps"],
    "additionalProperties": false
  }
}
```

出参示例：

6

```
{
  "Tps": [
    {
      "ID": "111111111-abc3-222222222-333333333-333333333",
      "Name" : "示例流控模板",
      "Enable": true,
      "Functions": "60000;1000;1001;200?;30??;335002;600010;4444",
      "Accounts": "0000000500",
      "NodeNos": "1",
      "Nodes": "",
      "Fc": {
        {
          "Duration": 30, // 30秒为时间维度，第一笔为开始
          "Warning": 5000, // 达到5000笔写一条日志
          "Limit": 10000 // 超过10000笔，丢弃
        }
      },
    },
    {
      "ID": "abc2",
      "Name" : "示例流控",
      "Enable": false,
      "Functions": "",
      "Accounts": "90001;90002;90003;50001;00000005001",
      "NodeNos": "1",
      "Nodes": "front#0;front#1",
      "Fc": {
        {
          "Duration": 1, // 秒为时间维度
          "Warning": 5000, // 达到5000笔写一条日志
          "Limit": 10000 // 超过10000笔，丢弃
        }
      },
    },
    {
      "ID": "abc14353",
      "Name" : "示例流控模板",
      "Enable": true,
      "Functions": "1000;1001;20??;335002;40??;3333",
      "Accounts": "0001;0002;0003;00000005001",
      "NodeNos": "1",
      "Nodes": "",
      "Fc": {
        {
          "Duration": 3600, // 3600秒为时间维度，第一笔为开始
          "Warning": 50000, // 达到5000笔写一条日志
          "Limit": 100000 // 超过100000笔，丢弃
        }
      },
    }
  ]
}
```

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "RuleInfo": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "ID": {
            "type": "string",
            "description": "规则ID",
            "examples": ["wabc1"]
          },
          "Name": {
            "type": "string",
            "description": "规则名称",
            "examples": ["示例流控"]
          },
          "Enable": {
            "type": "boolean",
            "description": "是否启用此规则",
            "examples": [true, false]
          },
          "Functions": {
            "type": "string",
            "description": "功能号列表，用分号分隔。可为空。",
            "examples": ["", "1000;1001;20??;40??"]
          },
          "Accounts": {
            "type": "string",
            "description": "账号列表，用分号分隔。可为空。",
            "examples": ["0001;0002;0003;0000000500", "90001;90002;90003;50001;00000005001"]
          },
          "NodeNos": {
            "type": "string",
            "description": "分片号列表，用分号分隔。可为空。",
            "examples": ["1", ""]
          },
          "Nodes": {
            "type": "string",
            "description": "生效节点值，只能是前置节点或者列表。如果为空字符串，表示规则尚未生效，只是模板。",
            "examples": ["", "front#0;front#1"]
          },
          "Fc": {
            "type": "object",
            "description": "流控规则配置",
            "properties": {
              "Duration": {
                "type": "integer",
                "description": "流控时间窗口，单位为秒。例如1秒为一个时间维度。",
                "examples": [1]
              },
              "Warning": {
                "type": "integer",
                "description": "警告阈值，当交易笔数达到该值时，系统会记录一条警告日志。",
                "examples": [5000]
              },
              "Limit": {
                "type": "integer",
                "description": "限制阈值，当交易笔数超过该值时，后续交易将被丢弃。",
                "examples": [10000]
              }
            },
            "required": ["Duration", "Warning", "Limit"]
          },
          "ActiveTime": {
            "type": "string",
            "format": "date-time",
            "description": "规则的生效时间",
            "examples": ["2025-09-09 13:12:18"]
          }
        },
        "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes", "Fc", "ActiveTime"]
      }
    },
    "required": ["RuleInfo"],
    "additionalProperties": false
  }
}
```

出参示例：

```
{
  "RuleInfo": [
    {
      "ID": "abc2",
      "Name": "示例流控",
      "Enable": true,
      "Functions": "",
      "Accounts": "90001;90002;90003;50001;00000005001",
      "NodeNos": "1",
      "Nodes": "front#0;front#1",
      "Fc": {
        "Duration": 1, // 秒为时间维度
        "Warning": 5000, // 达到5000笔写一条日志
        "Limit": 10000 // 超过10000笔，丢弃
      },
      "ActiveTime": "2025-09-09 13:12:18"
    }
  ]
}
```

6.33.13. UpdateTpsInfo 更新流控规则信息

功能名称	UpdateTpsInfo	版本号	V1.0.0.0	更新日期	20260122
功能备注	更新流控规则信息				
功能说明	更新流控规则信息，支持批量更新				

访问链接	http://p.port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=UpdateTpsInfo
入参说明	
JSON Schema:	
<pre>{ "\$schema": "http://json-schema.org/draft-07/schema#", "type": "object", "properties": { "Tps": { "type": "array", "items": { "type": "object", "properties": { "ID": { "type": "string", "description": "规则ID", "examples": ["abc14353"] }, "Name": { "type": "string", "description": "规则名称", "examples": ["示例流控模板"] }, "Enable": { "type": "boolean", "description": "是否启用此规则", "examples": [true, false] }, "Functions": { "type": "string", "description": "功能号列表, 用分号分隔。可为空。", "examples": ["1000;1001;20??;335002;40??;3333", ""] }, "Accounts": { "type": "string", "description": "账号列表, 用分号分隔。可为空。", "examples": ["0001;0002;0003;00000005001", ""] }, "NodeNos": { "type": "string", "description": "分片号列表, 用分号分隔。可为空。", "examples": ["1", ""] }, "Nodes": { "type": "string", "description": "生效节点值, 只能是前置节点或者列表。如果为空字符串, 表示规则尚未生效, 只是模板。", "examples": ["", "front#0;front#1"] }, "Fc": { "type": "object", "description": "流控规则配置", "properties": { "Duration": { "type": "integer", "description": "流控时间窗口, 单位为秒。例如3600秒为一个时间维度。", "examples": [3600] }, "Warning": { "type": "integer", "description": "警告阈值, 当交易笔数达到该值时, 系统会记录一条警告日志。", "examples": [50000] }, "Limit": { "type": "integer", "description": "限制阈值, 当交易笔数超过该值时, 后续交易将被丢弃。", "examples": [100000] } }, "required": ["Duration", "Warning", "Limit"] } }, "required": ["ID", "Name", "Enable", "Functions", "Accounts", "NodeNos", "Nodes", "Fc"] } }, "required": ["Tps"], "additionalProperties": false } }</pre>	
入参示例:	
<pre>{ "Tps": [{ "ID": "abc14353", "Name": "示例流控模板", "Enable": true, "Functions": "1000;1001;20??;335002;40??;3333", "Accounts": "0001;0002;0003;00000005001", "NodeNos": "1", "Nodes": "", "Fc": { "Duration": 3600, // 3600秒为时间维度, 第一笔为开始 "Warning": 50000, // 达到50000笔写一条日志 "Limit": 100000 // 超过100000笔, 丢弃 } }] }</pre>	
出参说明	
JSON Schema:	


```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "Result": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "ID": {
            "type": "string",
            "description": "规则ID",
            "examples": ["123456-11-333", "1234567890-1234567890-1234567890", "abc14353"]
          },
          "ErrorNo": {
            "type": "integer",
            "description": "错误编号, 0表示成功, 其他表示错误",
            "examples": [0, 7951]
          },
          "ErrorInfo": {
            "type": "string",
            "description": "错误信息",
            "examples": ["ok", "运行时出错", "规则数超过10000"]
          }
        },
        "required": ["ID", "ErrorNo", "ErrorInfo"]
      }
    },
    "required": ["Result"],
    "additionalProperties": false
  }
}
```

出参示例:

```
{
  "Result": [
    {
      "ID": "123456-11-333",
      "ErrorNo": 0,
      "ErrorInfo": "ok"
    },
    {
      "ID": "1234567890-1234567890-1234567890-1234567890",
      "ErrorNo": 7951,
      "ErrorInfo": "运行时出错"
    },
    {
      "ID": "abc14353",
      "ErrorNo": 7951,
      "ErrorInfo": "规则数超过10000"
    }
  ]
}
// 没在里边的, 说明没有被执行, 就是失败
```

6

功能名称	GetTokenInfo	版本号	V1.0.0.0	更新日期	20260122
功能备注	获取指定令牌的详细信息				
功能说明	根据令牌ID获取令牌的详细信息, 包括开始时间、总令牌数和已使用令牌数				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_flowcontrol&InstanceNo=0&FuncName=GetTokenInfo				

入参说明

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "ID": {
      "type": "string",
      "description": "请求参数, 规则UUID",
      "examples": ["193-1234567890-222-0"]
    }
  },
  "required": ["ID"],
  "additionalProperties": false
}
```

入参示例:

```
{
  "ID": "193-1234567890-222-0" // 请求参数, 规则UUID
}
```

出参说明

JSON Schema:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "ID": {
      "type": "string",
      "description": "请求参数, 规则UUID",
      "examples": ["193-1234567890-222-0"]
    },
    "StartTime": {
      "type": "integer",
      "description": "令牌开始时间, 毫秒级时间戳",
      "examples": [170990099000]
    },
    "TotalTokens": {
      "type": "integer",
      "description": "总令牌数",
      "examples": [10000]
    },
    "UsedTokens": {
      "type": "integer",
      "description": "已使用令牌数",
      "examples": [9088]
    }
  },
  "required": ["ID", "StartTime", "TotalTokens", "UsedTokens"],
  "additionalProperties": false
}
```

出参示例:

```
{
  "ID": "193-1234567890-222-0", // 请求参数, 规则UUID
  "StartTime": 170990099000, // 令牌开始时间, 毫秒级时间戳
  "TotalTokens": 10000, // 总令牌数
  "UsedTokens": 9088 // 已使用令牌数
}
```

6.34. 文件同步插件 ldp_filesync

6.34.1. GetTasksInfo 获取文件同步任务信息

功能名称	GetTasksInfo	版本号	V1.0.0.1	更新日期	20251227
功能说明	根据指定的任务id查询任务信息，如果没有指定任务id，则查询全部任务信息				
访问链接	http://10.20.151.34:5526/ldp?Pretty=1&PluginName=ldp_filesync&InstanceNo=0&FuncName=GetTasksInfo				
入参说明					
Pretty	1-美化json输出, 0-不美化				
出参说明					
<pre>{ "TaskList": [{ //任务rsync命令参数 "TaskParam": "rsync -av /home/hundsun/file_list.csv 10.20.151.32:/home/hundsun/loading_data/ ", //任务id "TaskId": "1", //任务中所有文件数量 "TaskFileCounts": 1, //任务中已传输文件数量 "TaskFileCompletes": 0, //任务中已存在不用传输的文件数量 "TaskFileExists": 0, //任务中传输发生错误的数量 "TaskFileErrors": 0, //任务中检查文件数量 "TaskCheckCounts": 1, //任务中检查完成文件数量 "TaskCheckCompletes": 0, //任务中检查错误文件数量 "TaskCheckErrors": 0, //任务状态, 0待执行, 1运行中, 2已完成, 3失败, 4已传输, 5已终止 "TaskStatus": 1, //任务的错误描述 "TaskErrorMsg": "", //任务中文件信息列表 "TaskFileList": [{ //文件名称 "FileName": "file_list.csv", //文件已传输字节 "FileTransferredSize": "", //文件总大小 "FileSize": "25,966", //文件已传输字节与文件大小的百分比 "FileTransferredPercent": "", //文件传输速率 "FileSpeed": "", //相应.md5文件中的md5值 "FileExpectMd5": "", //传输完成后文件的md5值 "FileActualMd5": "", //md5值比对结果, 0表示没有对比, 1表示成功, 2表示成功 "FileCompareMd5": 0, //检查文件是否存在. 0表示没有检查, 1表示检查存在, 2表示检查不存在 "FileCheckExist": 0, //文件错误描述 "FileErrorMsg": "" }] }]}</pre>					

6.35. multiproc插件 ldp_multiproc

6.35.1. GetMuitprocStatistics 获取统计信息

功能名称	GetMultiprocStatistics	版本号	V1.2501.0.1	更新日期	20260122
功能备注	获取统计信息				
功能说明	获取当前multiproc的统计信息				
访问链接	http://ip:port/ldp?Pretty=1&PluginName=ldp_multiproc&InstanceNo=0&FuncName=GetMultiprocStatistics				
入参说明					
无					
出参说明					
核心与回库使用该管理功能时，返回字段存在差异，以下分别给出两种场景的完整示例说明。					
核心使用管理功能返回示例					

```

{
    "/*": "Multiproc 管理信息列表",
    "Multiproc": [
        {
            "/*": "管理器名称, 核心场景固定为 ILdpMultiproc",
            "ManagerName": "ILdpMultiproc",
            "/*": "节点名",
            "NodeName": "mdb#1",
            "/*": "分片号, 核心场景通常为数字编号",
            "ShardNo": "1",
            "/*": "原始节点名",
            "OriginNodeName": "mdb#1",
            "/*": "与partkeeper的连接状态",
            "IsOnline": true,
            "/*": "当前 multiproc 中包含的节点数量",
            "NodeCount": 3,
            "/*": "主分区组上下文名",
            "MasterPartGroupContextName": "multiproc_mdb#1_partgroup_2",
            "/*": "主分区组编号",
            "MasterPartGroupNo": "2",
            "/*": "当前 multiproc 系统所处状态码: 0 未初始化 1 初始化中 2 运行中 3 重新登录等待初始化",
            "StageNo": 2,
            "/*": "当前 multiproc 系统所处状态: UNINIT 未初始化 INIT 初始化中 READY 运行中 RELOGIN_WAIT_INIT 重新登录等待初始
化",
            "StageMode": "READY",
            "/*": "是否为观察者, 核心场景为 false",
            "IsObserver": false,
            "/*": "是否为与 keeper 断开连接后的超时重连场景",
            "IsTimeoutReconnect": false,
            "/*": "当前是否处于等待最后消费消息确认阶段",
            "IsWaitingLastConsumedMsg": true,
            "/*": "启动确认阶段已收到的首条消息数量",
            "ReplayFirstMessageCame": 0,
            "/*": "当前 multiproc 消费的轮次号",
            "CurrentRoundNo": 0,
            "/*": "当前 multiproc 中已完成同步的数量",
            "FinishedSyncCount": 2,
            "/*": "当前 multiproc 中需要完成同步的数量",
            "NeedFinishedCount": 2,
            "/*": "当前 multiproc 中等待进入下一轮次的投票数量",
            "WaitNextVoted": 0,
            "/*": "分区组状态列表, 核心场景返回完整仲裁状态信息",
            "PartGroupStatuss": [
                {
                    "/*": "分区组名称",
                    "PartGroupName": "multiproc_mdb#1_partgroup_1",
                    "/*": "所属层级名称",
                    "TierName": "mdb_partgroup_1",
                    "/*": "当前分区组处理的轮次号",
                    "CurrentRoundNo": 0,
                    "/*": "分区组核心状态 Writer 写核心 Reader 读核心",
                    "PartGroupArbStatus": "Reader",
                    "/*": "当前分区组所属集群任期号",
                    "CurrentClusterTerm": 0,
                    "/*": "启动状态码",
                    "StartStatusNo": 2,
                    "/*": "启动状态含义",
                    "StartStatusMode": "ArbStartStatus",
                    "/*": "当前角色状态码",
                    "CurrentRoleNo": 3,
                    "/*": "当前角色状态含义",
                    "CurrentRoleMode": "ARB_FOLLOWER",
                    "/*": "单机状态码",
                    "SingleStatusNo": 0,
                    "/*": "单机状态含义",
                    "SingleStatusMode": "NoneStatus"
                },
                {
                    "PartGroupName": "multiproc_mdb#1_partgroup_2",
                    "TierName": "mdb_partgroup_2",
                    "CurrentRoundNo": 0,
                    "PartGroupArbStatus": "Writer",
                    "CurrentClusterTerm": 0,
                    "StartStatusNo": 2,
                    "StartStatusMode": "ArbStartStatus",
                    "CurrentRoleNo": 2,
                    "CurrentRoleMode": "ARB_LEADER",
                    "SingleStatusNo": 2,
                    "SingleStatusMode": "LeaderStatus"
                },
                {
                    "PartGroupName": "multiproc_mdb#1_partgroup_3",
                    "TierName": "mdb_partgroup_3",
                    "CurrentRoundNo": 0,
                    "PartGroupArbStatus": "Reader",
                    "CurrentClusterTerm": 0,
                    "StartStatusNo": 2,
                    "StartStatusMode": "ArbStartStatus",
                    "CurrentRoleNo": 3,
                    "CurrentRoleMode": "ARB_FOLLOWER",
                    "SingleStatusNo": 0,
                    "SingleStatusMode": "NoneStatus"
                }
            ]
        }
    ]
}

```

回库使用管理功能返回示例

```
{
  "":"/: "Multiproc 管理信息列表",
  "Multiproc": [
    {
      "":"/: "管理器名称, 回库场景通常为回库集群分片的管理器名称",
      "ManagerName": "togw#0_mdb_1",
      "":"/: "节点名",
      "NodeName": "togw#0",
      "":"/: "分片号, 回库场景通常为业务分片标识",
      "ShardNo": "mdb_1",
      "":"/: "原始节点名",
      "OriginNodeName": "togw#0",
      "":"/: "与partkeeper的连接状态",
      "IsOnline": true,
      "":"/: "当前 multiproc 中包含的节点数量",
      "NodeCount": 3,
      "":"/: "主分区组上下文名, 回库场景可能为空字符串",
      "MasterPartGroupContextName": "",
      "":"/: "主分区组编号",
      "MasterPartGroupNo": "1",
      "":"/: "当前 multiproc 系统所处状态码: 0 未初始化 1 初始化中 2 运行中 3 重新登录等待初始化",
      "StageNo": 2,
      "":"/: "当前 multiproc 系统所处状态: UNINIT 未初始化 INIT 初始化中 READY 运行中 RELOGIN_WAIT_INIT 重新登录等待初始
化",
      "StageMode": "READY",
      "":"/: "是否为观察者, 回库场景为 true",
      "IsObserver": true,
      "IsTimeoutReconnect": true,
      "IsTimeoutReconnect": true,
      "":"/: "当前是否处于等待最后消费消息确认阶段, 回库场景通常为 false",
      "IsWaitingLastConsumedMsg": false,
      "":"/: "启动确认阶段已收到的首条消息数量",
      "ReplayFirstMessageCame": 3,
      "":"/: "当前 multiproc 消费的轮次号",
      "CurrentRoundNo": 0,
      "":"/: "当前 multiproc 中已完成同步的数量",
      "FinishedSyncCount": 0,
      "":"/: "当前 multiproc 中需要完成同步的数量",
      "NeedFinishedCount": 3,
      "":"/: "当前 multiproc 中等待进入下一轮次的投票数量",
      "WaitNextVoted": 0,
      "":"/: "分区组状态列表, 回库场景仅返回基础分区组信息",
      "PartGroupStatuss": [
        {
          "":"/: "分区组名称",
          "PartGroupName": "multiproc_mdb#0_partgroup_1",
          "":"/: "当前分区组处理的轮次号",
          "CurrentRoundNo": 0
        },
        {
          "PartGroupName": "multiproc_mdb#1_partgroup_2",
          "CurrentRoundNo": 0
        },
        {
          "PartGroupName": "multiproc_mdb#2_partgroup_3",
          "CurrentRoundNo": 0
        }
      ]
    }
  ]
}
```

6.35.2. GetPartGroupRCMRouteAndMaxMsgNoList 获取分区信息

功能名称	GetPartGroupRCMRouteAndMaxMsgNoList	版本号	V1.0.0.0	更新日期	20260122
功能备注	无				
功能说明	无				
访问链接	http://ip:port/kdp?Pretty=1&PluginName=kdp_multiproc&InstanceNo=0&FuncName=GetPartGroupRCMRouteAndMaxMsgNoList				
入参说明					
无					
出参说明					
<pre>{ [] }</pre>					

7. 专项说明

本章主要介绍分布式低延时开发平台的专项功能

7.1. 快照环境搭建与使用

平台针对机器异常场景，提供快照恢复功能，保证系统的高可用性。

快照由主核心负责，通过核心提供的日切备份管理功能触发，生成本地快照文件，然后将快照信息登记到zk上，后续可通过脚本拉取快照进行快速恢复。

7.1.1. 配置说明

生成快照的能力，默认不开启，必须通过“MDB数据组件配置项”中快照的配置项（文档在本文的“业务框架插件”——“MDB数据组件配置项说明”），进行显式的开启。

主、备核心都必须显式开启快照功能。开启后
主核心具备了生成快照的能力，且在管理功能的触发下，可生成快照。
备核心具备了生成快照的能力，但是即使触发，也会拒绝生成快照。当主核心挂掉，备核心如果身份切换为主核心，那么就能生成

快照。

快照功能如何通过配置开启

必须检查的配置项，显式列出的，都是至关重要的。

- SnapshotInterval

必须确保该配置项!!!不存在!!!

- SnapCommitSqnNum

必须确保该配置项!!!存在!!!

该配置项控制在生成快照的过程中，写盘的频繁程度
该值越小，写盘越频繁，性能越低，但可靠性高。
该值越高，写盘次数越少，性能变好，但可靠性变差

在目前快照由管理功能主动触发的情况下，该值可以配置在100000或以上。因为主动触发的情况下，在快照生成完成时，会确保写盘。

- 其它的配置项，阅读对应章节介绍。

7.1.2. 启动说明

- 使用快照场景启动时，依赖bin目录下的快照工具(snapshot_tools)和快照工具脚本(snapshot_tools.sh)。
- 用户需要在当前使用的启动脚本中添加（即：使用快照启动，需要显式修改启动脚本）

```
source $HSLDP_HOME/snapshot_tools.sh ldp xst#0 -s ip:port User MaxRetries
```

解释：

source : 是linux命令，相当于 . 或者 sh

\$HSLDP_HOME/snapshot_tools.sh: 固定，不改动

ldp: 产品名

xst#0: 应用名

-s ip:port: zk信息

User: 用于scp拷贝快照的用户名，比如：主核心部署在 192.168.88.188 上，那么快照就生成在这台机器上，最终是要从这台机器上使用scp拷贝快照的，User就是scp使用的用户名，比如：root, admin

MaxRetries: 重试次数，如果失败了，重试这么多次，然后就放弃。

这行代码的作用，用以导出变量\$SNAP_FILE_NAME，然后根据导出变量是否为空来决定是否以快照方式启动，如果以快照方式启动，需要在启动命令后添加-f \$SNAP_FILE_NAME。具体请参照启动脚本示例。

- 拉取快照时会通过scp从"生成快照的核心机器"拉取快照文件，默认需要手动输入"生成快照的核心机器"的密码，如果需要避免手动输入密码，请先建立SSH的信任关系。

启动脚本示例

```
#!/bin/bash

cwd=`pwd`
export HSLDP_HOME=$cwd/./bin
export PATH=$HSLDP_HOME:$PATH
export COMPONENT=$HSLDP_HOME/components
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$COMPONENT
export LANG="zh_CN.UTF-8"

# source $HSLDP_HOME/snapshot_tools.sh 产品名 应用名[#编号] -s zk服务器地址 生成快照的核心机器的用户名 scp最大重试次数（可选默认3次）
source $HSLDP_HOME/snapshot_tools.sh ldp xst#0 -s ip:port User MaxRetries

# SNAP_FILE_NAME 为 snapshot_tools.sh 中导出的快照名
if [ -n "$SNAP_FILE_NAME" ]; then
    echo "以加载快照的方式启动核心，快照文件为: $SNAP_FILE_NAME"
    hsl dp startdaemon ldp xst#0 -s ip:port -f $SNAP_FILE_NAME
else
    echo "以普通方式启动核心"
    hsl dp startdaemon ldp xst#0 -s ip:port
fi
```

快照拉取策略

拉取步骤：

1. 去zk上查询相应的快照信息
2. 如果zk上有快照信息，读取这个快照的信息，如：快照文件在哪个机器上，在这个机器上的完整路径是啥等信息
3. 使用scp命令去拷贝快照

快照场景通过拉取策略判断是否拉取快照，如果判定需要拉取快照，脚本将会尝试通过scp的方式拉取快照文件，成功获取到快照文件后，将自动执行清理、解压、以快照方式启动等操作。拉取策略根据ULID决定，本地ULID通过共享内存读取，ZK的ULID通过快照数据库名节点值读取，快照具体拉取策略如下：

- 本地没有ULID，zk上没有快照，正常启动
- 本地没有ULID，zk上有快照，拉取快照启动
- 本地有ULID，ZK上没有快照，正常启动
- 本地有ULID，ZK上有快照，但本地ULID比快照ULID新，正常启动
- 本地有ULID，ZK上有快照，但本地ULID和快照ULID相同，正常启动
- 本地有ULID，ZK上有快照，但快照ULID比本地ULID新，拉取快照启动

7.1.3. 功能说明

1、当被触发生成快照并成功生成后，会在配置的SnapshotDir目录下生成一个快照文件，快照名格式为"uftmdb.[dbname].[ULID].[Lsn].snap"，例如 "uftmdb.test.20231211-155314911.327.snap"

快照名含义：

uftmdb：固定

test：数据库名字

20231211-155314911：数据库的编号，该编号表示这个数据库首次创建的时间。格式为： 年 月 日 时 分 秒 毫秒

327：这个快照所包含的最新的事务号（这个事务号的数据，已经含在快照内）

2、本地快照生成后，会将快照信息写到Zk服务器，具体包含如下信息：

- 创建快照根节点：/ldp/snapshot
- 创建快照数据库名节点：/ldp/snapshot/dbname节点，存快照名信息，节点值为最新的ULID，例如/ldp/snapshot/test，节点值如下

```
{
  "ULID": "20231211-155314911"
}
```

- 创建快照文件临时节点（保留最新的两个）：/ldp/snapshot/dbname/snapfile， 例如 /ldp/snapshot/test/uftmdb.test.20231211-155314911.1.snap，节点值如下

```
{
  "//": "快照机Ip",
  "Ip": [
    "10.20.137.139",
    "192.168.86.20"
  ],
  "//": "快照文件大小",
  "SnapLength": 71628,
  "//": "快照文件所在目录",
  "SnapDir": "/home/test/ldp_mdb/workspace/",
  "//": "应用名",
  "AppName": "snapmdb#0"
}
```

3、通过拉取快照，成功启动的核心，会在工作目录下会保留这个快照文件，例如uftmdb.test.20231211-155314911.1.snap

7.1.4. 快照功能约束

- 快照场景仅支持ZooKeeper方式启动，不支持本地配置启动。
- 快照功能必须显式开启
- 生成快照必须显式触发
- 只有主核心才可以生成快照

7.2. 7x24日切与备份

该功能自版本 LDP1.0.V202401.14.000 开始支持，首先需要确认已经升级。

7.2.1. 配置

需要确认如下涉及的配置已经启用，才能支持本功能。

rcm配置

修改部署配置中rcm的上下文配置，在"Default"模版上下文配置里，配置 "AllowDataRotate": true，如下示例。

```
{
  "Name": "Default",
  "IsSingleton": false,
  "IsObserver": false,
  "SyncIp": "192.168.86.0",
  "SyncAddr": "236.1.2.123",
  "SyncPort": 60999,
  "SyncMode": 1,
  "SyncRepairPortStart": 45000,
  "SyncRepairPortEnd": 50000,
  "Bypass": {
    "TransMode": "udp_mc",
    "Topic": "bypass_redo",
    "PartitionNo": 62
  },
  "//": "需要添加的配置",
  "AllowDataRotate": true
}
```

MDB数据组件配置

参看 [快照功能如何通过配置开启](#)

7.2.2. 信息获取

本章节描述如何获得操作过程中需要使用到的数据信息，这些信息都是通过LDP核心的管理功能获取。同时假定已知如何使用LDP核心提供的管理功能。

核心身份判断

插件名	功能名
ldp_bizproc	GetArbStatus

应答数据

```
{
  "身份标识": ARB_SINGLE-单主; ARB_ACTIVE-集群主; ARB_INACTIVE-集群备",
  "ArbStatus": "ARB_ACTIVE",
  "集群名",
  "TierName": "secu"
}
```

7.2.3. 操作步骤

步骤1.1. MakeTag（预备及检查）

目标	插件名	功能名
主机房主核心	ldp_mproxy	MakeTag

请求数据

```
{
  "Request": {
    "唯一请求编号，由请求发起方生成填入，相同的Id表示同一个请求，不同的Id表示不同的请求",
    "如果使用同一个TagId发送MakeTag请求，那么得到的应答数据始终不变（里面的每一个字段的值都不变）",
    "tagId为快照的唯一编号，建议每次生成快照使用新的tagId，原因是MakeTag调用同一个tagId获取的数据为首次调用的返回数据，快照的
    实际数据也是首次调用时刻的数据无法更新",
    "TagId": "202507251148567"
  }
}
```

应答数据

```
{
  "错误码 0表示成功 其它表示失败",
  "ErrorNo": 0,
  "错误描述信息",
  "ErrorMsg": "Successfully",
  "请求中的原样带回",
  "TagId": "202507251148567",
  "集群信息",
  "Clusters": [
    {
      "集群名",
      "ClusterName": "zad",
      "本集群当前最大事务号",
      "MaxTxNo": 2,
      "本集群当前最大重演消息编号",
      "GlobalMsgId": 56
    }
  ]
}
```

步骤1.2. 等待todb回库完成

目标	插件名	功能名
主机房主todb核心	ldp_todb	GetMonitor

请求数据

无额外请求数据

应答数据（仅列出关键字段）

```
"集群名 必须与MakeTag应答中的集群名相同 才是这个集群的回库信息",
"ClusterName": "zad",
"已回库的事务号 大于等于 MakeTag应答中的MaxTxNo 才说明回库完成",
"MaxTransNo": 2,
"该标志必须为true才行",
"ResendFinishFlag": true,
```

步骤1.3. 查看同步备机房进度

如果不是主机房+异地机房这种配置，那么这步可以跳过

这步仅是确认，不是强制要求必须要等待备机房已完成MaxTxNo的同步

如果已完成同步，可马上进入下一步骤

如果一段可容忍的等待时候后（举例：检查3次，每次间隔30秒），还是没有同步完毕，也可以进入下一步骤

目标	插件名	功能名
主机房主redo_copy核心	ldp_redo_copy	GetStatistics

请求数据

无额外请求数据

应答数据（仅列出关键字段）

```
"主题名 该主题名必须用于前述集群向备机房同步redo的主题",
"TopicName": "bypass_redo",
"备机房已确认收到的消息序号 该号 大于等于 MakeTag应答中的MaxTxNo 说明已同步",
"AckMsgNo": 2,
```

步骤2. SwitchTag（发起日切与备份请求）

目标	插件名	功能名
----	-----	-----

目标房主核心	插件名	功能名
--------	-----	-----

请求数据

```
{
  "FuncName": "SwitchTag",
  "PluginName": "ldp_mproxy",
  "Request": {
    "TagId": "20250725114857",
    "Clusters": [{
      "ClusterName": "zad",
      "MaxTxNo": 2,
      "GlobalMsgId": 56
    }]
  }
}
```

应答数据

```
{
  "ErrorNo": 0,
  "ErrorMsg": "Successfully"
}
```

步骤3. QuerySwitchTag（查询日切与备份的结果）

目标	插件名	功能名
主机房主核心	ldp_mproxy	QuerySwitchTag

请求数据

```
{
  "Request": {
    "TagId": "20250802140607"
  }
}
```

应答数据

```
{
  "ErrorNo": 0,
  "ErrorMsg": "Successfully",
  "Status": 0,
  "Plugins": [
    {
      "Status": 0,
      "SnapPackedTxid": 2,
      "PluginName": "uftmdb_data",
      "Directories": [
        "/home/feis/test/mdb/ldp_mdb2/workspace/tags/20250802140607/uftmdb_data/uftmdb.zaddb.20250812-204658098.2.snap",
        "Progress": 100
      ],
      {
        "PluginName": "ldp_rcm",
        "Directories": [
          "/home/feis/test/mdb/ldp_mdb2/workspace/tags/20250802140607/ldp_rcm_0/Rcm_20250802140607"
        ],
        "Status": 0,
        "ErrorNo": 0,
        "ErrorMsg": ""
      }
    ]
  }
}
```

应答说明

1、当 ErrorNo=0 时

- Status=0 全部完成

建议整个目录备份，在上述的列子中，使用路径/home/feis/test/mdb/ldp_mdb2/workspace/tags/20250802140607

- Status=1 部分完成

部分还在进行中，需要间隔时间后再次查询

- Status=2 失败

需要重做

2、当 ErrorNo=其它 时，表示请求失败，需要重做

3、在各个模块的明细里，如果存在Progress标签，则表示当前的进度（0表示未开始，100表示完成）

7.2.4. 补充说明

前段描述的是如何对于主机房的主核心发起日切与备份操作，还需要对其它核心操作。

- 主机房的备核心

直接使用 主机房的主核心 的MakeTag的应答数据，进行后续操作

- 备机房的主核心

注意：此处分2种情况。

- 1、备机房与主机房是同一集群的，则备机房没有主核心，跳过此步骤。
- 2、备机房与主机房是不同集群的，则备机房存在主核心，其操作过程类似 主机房的主核心 的全过程

- 备机房的备核心

直接使用 备机房的主核心的MakeTag的应答数据（如果备机房存在主核心时），进行后续操作

但是，当备机房没有主核心时，使用主机房的主核心的MakeTag的应答数据。且前提是，备机房已经完成同步的事务号，必须 大于 等于 MakeTag应答中的MaxTxNo。

7.2.5. 失败处理

1、MakeTag失败

- 检查是否已经升级到要求的版本
- 是否进行了前文描述的配置的修改
- 启动前是否已经清理了上一个版本遗留的数据（参考cleansldp）
- 系统是否已经正常运行，并产生了事务数据

2、SwitchTag失败

- 首先确认问题1中所述是否正确完成。
- 留存请求和答应数据，联系LDP开发进行排查

3、QuerySwitchTag失败

- 如果是查询本身失败，可再次查询，或者联系LDP开发进行排查
- 如果查到得到了应答数据，表示日切与备份仅部分成功，或者全部失败。联系LDP开发进行排查。！！！请勿直接使用新的TagId再次发起新的日切与备份！！！

7.2.6. 约束与限制

1. 当前主核心生成快照后，将快照信息以临时节点数据的方式存储在zk上，其它机器可以读取此信息使用快照启动（参看 [启动说明](#)）。如果该主核心一旦发生过进程退出，那么zk上的快照信息会自动删除（因为是临时节点），那么其它机器将无法使用快照启动。
2. 日切时，主核心生成快照，各核心不生成快照（各核心会响应一个成功的应答，但是进度值Progress为0）

7.3. 风险提示

7.3.1. zk上lock节点

操作：如果第三方工具或者人为手动在zk节点上创建lock等永久节点等操作

影响风险：由于程序中操作zk需要创建lock节点，才能更新某些对应节点，如果额外或者错误的lock节点存在，可能会导致程序更新对应节点失败。

规避措施：推荐使用see平台去访问zk

7.3.2. LDP1.0.V202501.00.000 遗留风险

1、机构场景下，当Ust类型的tcp连接，开启抓包时，如果开启了抓包，可能导致崩溃。规避手段：不开启抓包功能

2、机构场景下，ldp消息cleardatasets后rewriteDataset，消息可选头信息被丢失。规避手段：避免使用ldp消息cleardatasets接口。

3、网格场景下，如果keeper故障且被清流，整个集群无法恢复分区仲裁结果，导致无法进行分区迁移，核心重启会出现无法服务的情况。规避手段：避免在核心存在的情况下，清理keeper节点

4、网格场景下，（主）核心与（备）核心断连，partkeeper状态当前未更新，导致可能出现双主情况。规避手段：无

5、网格场景下，各核心如果同步慢或者启动完，可能有业务异常（主备节点数据不一致），todb回库给物理也有数据不一致的情况。原因是partkeeper在上场结束后没有做轮次号+1。规避手段：保证主备节点历史数据同步完后再做业务。

6、网格场景下，当前在查询超时时间内的核心进行选举，存在REDO大的核心应答返回慢但无法参与到数据源选举的问题。规避手段：如果长时间未见分区迁移成功，需要分区迁移是否成功

7、单个前置处理巨量请求（tps>10w）且开启全链路情况下，性能下降很厉害，可能会下降50%。规避手段：启动多个前置

8、本地模式 hslp GetZkHandler函数不严谨返回未初始化句柄导致mproxy奔溃。规避手段：不启用本地模式

9、因子模块无法正常加载，且业务忽略了因子层的报错（配置IgnoreRsLoadError为1），导致后续调用加载失败的因子模块会导致节点启动失败。规避手段：无

10、非多分片场景，一个Backend对应的节点数量最多只能16个，超过该数量可能导致节点崩溃。规避手段：不要超过16个限制

11、1主3备情况下，前置核心交易请求应答总体性能未达到8w以上。规避手段：无

12、redo合并开启之后，无法记录回库链路的时延日志。规避手段：无

13、数据一致性校验偶现报错，导致流程卡死，原因待明确。规避手段：无

14、主备机房双集群场景下，备机房上场回库慢，原因待明确。规避手段：无

- 15、todb回物理库场景、EnableClear、EnableInitDate、ToSpace配置全局不生效，且EnableInitDate未补充文档说明，EnableClear只是配置读取，清表当前实际不支持。规避手段：无
- 16、todb回物理库场景、配置全局因子线程，大事务批量执行发生报错后，未转单行处理。规避手段：不开启大事务或者不配置全局因子线程
- 17、todb节点和核心节点有极小概率出现退出时卡住，原因是内部对象可能先于工作线程停止工作，导致接口调用阻塞。规避手段：先关闭其他可正常停止的应用，尽量最后再强制终止卡住的进程
- 18、首次部署启动redo_copy集群，部分节点出现概率性启动失败；规避手段：可以通过重复启动解决
- 19、修改redo_copy已存在发送端Transmitter旁路接收主题配置RxBypassTopic，会产生一个有多个接收主题的RCM上下配置。规避手段：保证配置修改在盘后进行，修改后清除zk上的动态rcm配置然后再重启即可
- 20、停止ldpadmin轻量管控台可能会崩溃，规避方式：使用ldp的apm平台代替
- 21、网格场景下，重新连接或新启动的核心轮次号补缺控制仅持续到启动从keeper上查询的补缺终点，在补缺到查询的补缺终点后，可能仍存在需要轮次号补缺逻辑控制的数据。规避手段：运维操作中，在集群稳健状态下（主备机数据同步完成，rcm管理功能监控二者一致）进行分区迁移
- 22、网格场景下，回库节点退出时可能出现超过2分钟才能退出的现象，原因：启动中执行了hsldp Stop multiproc在等到分区路由表（120秒超时）。规避手段：可启动完成之后再退出。（修复插件：ldp_multiproc 修复版本：V1.0.15.5701）

7.3.3. LDP1.0.V202501.02.000 遗留风险

1、主干分支在 LDP1.0.V202302.01.002 版本之后前置代理（准确说是在这个版本引入了remoter，框架在启动时会加载remoter）配置中components中如果配置了tcp库或者ldptcpsdk之后，在进程停止时会有概率发生崩溃（运行时无影响）。规避方案：无。

7.4. 支持业务基于物理数据库开发

LDP平台在LDP1.0.V202401.13.000及以上版本支持基于物理数据库进行业务开发，即平台提供统一数据库驱动，通过调用驱动提供的API连接不同物理数据库并使用相关SQL对物理库进行操作来实现业务开发。平台提供了插件库和独立库两种方式，业务可根据需要选择使用插件库或者独立库开发模式。插件开发模式下，使用LDP开发工具可辅助业务开发降低业务开发难度。DB插件开发模式下，为了满足某些高性能场景，支持创建缓存表，即数据的读取在缓存中进行以提升读取性能。当前仅支持基于ob-oracle的业务开发

7.4.1. 功能依赖程序项说明

功能说明	程序项名称	程序项发布版本
统一数据库驱动插件	bin/plugins/libdp_dbclient.so	LDP1.0.V202401.13.000
统一数据库驱动独立库	bin/components/libdbclient.so	LDP1.0.V202401.13.000
小表缓存插件	bin/plugins/libdp_minitablecache.so	LDP1.0.V202401.14.000
内存数据库	bin/components/libutmdb.so	LDP1.0.V202401.14.000
MC客户端插件	bin/plugins/libdp_mcsdk.so	LDP1.0.V202401.14.000
负载均衡转发	bin/plugins/libdp_remoter.so	LDP1.0.V202401.15.000

7.4.2. 插件开发模式使用说明

插件开发模式依赖LDP平台，ldp_dbclient插件提供统一的物理数据库开发接口和相关头文件，可调用相关接口实现业务相关功能，详情参见《低延时分布式开发平台_dbclient开发指南》中的2.2章节。

插件开发模式也可使用平台提供的开发工具进行业务开发，开发工具中提供相应的界面和内置宏用于简化业务开发。推荐使用平台开发工具进行业务开发。

开发工具安装包获取：公司共享磁盘 \\Tx1\技术平台开发工具\$\LDP开发工具

并将开发工具升级至LDP1.0.V202401.13.000及之后版本，开发工具DB业务开发使用详情参见《低延时分布式开发平台_开发工具使用指南》中的2.6.47章节。

插件开发模式需在核心进程中配置ldp_dbclient插件，该插件封装各个数据库的驱动，提供统一的API接口，插件详细配置参见 本文档4.24章节：统一数据库访问 ldp_dbclient。

另外，针对DB开发模式中对性能要求较高的操作，平台提供了缓存表操作，可将性能要求较高且操作频繁的表创建为缓存表，表数据的获取优先从缓存表中获取，在更新物理DB的时候同步更新缓存并同步更新其他节点，保证各个节点以及缓存表中的数据一致性。

缓存表也可配合开发工具使用，开发工具缓存表的使用详情参见《低延时分布式开发平台_开发工具使用指南》中的2.6.48章。

需要在核心进程中配置ldp_minitablecache插件，该插件使用mdb来实现缓存表，详细使用参见 本文档4.27章节：小表缓存 ldp_minitablecach

DB开发和缓存表完整配置参见发布包：workspace/db.json

7.4.3. 独立库开发模式使用说明

平台提供libdbclient.so独立库，可独立于LDP平台进行基于物理数据库的业务功能开发，详情参见：《低延时分布式开发平台_dbclient开发指南》中的2.1章节 和发布包 dev/demo/dbclient 开发示例

7.4.4. 平台负载均衡使用说明

DB开发业务场景，需要由平台提供负载均衡的路由能力，和无状态无身份的处理模式。libldp_remoter.so和libldp_bizproc.so在LDP1.0.V202401.15.000版本新增对应的能力，并新增相同的配置项

- 1、libldp_bizproc插件需要配置HaMode = 0，并配置ShardNo分片号。详见bizproc的说明文档
- 2、前置转发核心和RPC启用负载均衡需配置Backend Type为RCM,并配置ForwardMode为"LoadBalance"，可以支持在单分片或者多分配模式下启动，并且只支持无机房模式和本地机房模式。如果多分片需配置DefaultNodeNo,即默认分片号，如果请求中没带分片号，那么会用这个值去匹配bizproc的ShardNo。详见remoter的说明文档

7.4.5. 相关约束说明

dbclient相关使用约束

- 1、当前仅支持对接ob-oracle数据库
- 2、依赖统一SQL配置文件，若使用转换功能，需要将统一SQL发布包config目录下的unisql.config放到运行目录的config文件夹下，并修改配置文件：unisql.splitsql.transform.multisql值调整为1，unisql.remove.tablespace的值调整为1

3、libobcInt.so，libunisql.so 不随平台发布包发布，详细下载地址和配置参见 本文档4.24章节：统一数据库访问 ldp_dbclient

ldp_minitablecache相关使用约束

- 1、缓存表加载仅支持从ob-oracle数据库加载
- 2、缓存表中必须包含 transaction_no 和 position_str 字段（要求加载的物理库中也必须有同样的字段）

transaction_no：值为全局递增的，保证新记录获取到的值比老的大，保证事务顺序更新

position_str：全局唯一，用于缓存同步时定位到记录（建议该字段创建缓存表的索引，如果不是索引的话会走全表遍历，影响其他订阅节点的更新速度）

- 3、缓存数据同步时各个字段的必填情况：

（1）table_name

必填字段：在进行全表刷新时该字符串为空串，其余情况都需要填缓存中存在的表名；否则报错；

（2）param_oper_type

必填字段：与字符'0'，'1'，'2'，'3'，'4'比较，不匹配报错；

（3）transaction_no

必填字段：值为全局递增的，保证新记录获取到的值比老的大，或者报错

（4）position_str

数据消费模式，必填；用于匹配具体的行（以及查询影子表中的事物号），目前没有强制要求该值为索引字段；

通知消费模式，position_str 不存在可以使用 where_str，都不传的情况下插入、更新、删除全表，插入可能发生主键冲突

（5）where_str

非必填字段，通知消费模式，position_str 不存在可以使用 where_str，两者都不存在，都不传的情况下插入、更新、删除全表，插入可能发生主键冲突

（6）business_data

非必填字段：只有在数据消费模式进行 insert、update 操作时，为必填字段

- 4、缓存表必须存在全局唯一索引

- 5、业务使用时通过 jres 同步发送的表名为数据库表名，同小表缓存配置文件中的 From 字段的值，内存查询时使用内存名，同小表缓存配置文件中的 TableName 字段的值

- 6、事务连接池的上限为 200，获取超过 200 以上的事务控制，直接报错

- 7、MC3.0数据同步的主题名固定为:topic.mid.localcache

- 8、同步数据时，使用的表名必须和小表缓存中配置的表名一致

- 9、插件配置中 dbclient 组件放在 minitablecache 之前

- 10、配置文件中，需要在GlobalLibs配置项中加上unisql

- 11、存在多个ldp_minitablecache配置时，配置的 LogicName 不能相同

- 12、小表缓存有sql建表、事务控制、Execsql函数功能修改以及数据同步，需要同步升级LDP平台libuftmdb.so[V4.0.15.32]、libldp_dbclient.so[V1.0.15.8]、libldp_mcsdk.so[V1.0.15.22],另外需要同步升级Cres平台libt2sdk.so[V1.0.0.191],统一SQL LightDB1.0-unisql-V202501-01-003中的libunisql.so

7.5. 支持业务在服务网格模式下运行

LDP平台在LDP1.0.V202401.16.000及以上版本支持业务在服务网格模式下运行，即平台提供除一主多备模式外，新增多节点相互备份，各节点负责不同分区写和作为其它分区的备身份同步redo，实现各节点数据一致，在节点故障时，迁移它的写分区到其它分区上，从而实现高可用的新模式。平台提供了将请求根据账号等信息路由到分区的能力。分区作为路由管理单位，可以在后台进行管理，管理的信息将在路由节点如前置上生效，实现不同核心分别处理不同分区写请求，各节点可同时处理读请求。
网格模式相比主备模式，业务库需要基于新的平台库开发编译，运行也需要基于LDP1.0.V202401.16.000及以上平台运行。

7.5.1. 功能依赖程序项说明

功能说明	程序项名称	程序项发布版本
LDP框架	bin/hslp	LDP1.0.V202401.16.000
LDP框架基础插件	bin/plugins/libdp_os.so	LDP1.0.V202401.16.000
LDP清理插件	bin/cleanhslp	LDP1.0.V202401.16.000
分区路由管理插件	bin/plugins/libdp_partkeeper.so	LDP1.0.V202401.16.000
分区路由管理客户端插件	bin/plugins/libdp_partkeeper_client.so	LDP1.0.V202401.16.000
可靠组播插件	bin/plugins/libdp_rcm.so	LDP1.0.V202401.16.000
多通道复制插件	bin/plugins/libdp_multiproc.so	LDP1.0.V202401.16.000
分区定位插件	bin/components/libdp_locate.so	LDP1.0.V202401.16.000

bizproc相关使用约束

- 1、当前bizproc启动时，必须partkeeper已经启动成功，并且在partkeeper上有该应用名配置，否则启动失败。
- 2、当前bizproc启动时，其它bizproc (同一个网格里) 必须也同时启动，启动间隔事件，如果超过30秒，那么会启动失败。

multiproc相关使用约束

- 1、multiproc只支持单个集群，多个集群会导致回库问题。

todb和togw相关使用约束

- 1、目前仅支持同时处理单机房内单个开启分区模式的集群+多个不开启分区模式的集群回库。
- 2、不支持redo合并。
- 3、todb 和 togw不能在同一台机器。

7.6. partkeeper运维指令介绍

7.6.1. 正常分区迁移，用于调整核心的负载

数据网格模式下，系统运行过程中，运维人员发现部分核心的业务负载很高，可以通过 分区迁移 指令，将高负载核心上的部分分区迁移到低负载的核心上。

- 该功能自 LDP1.0.V202501.00.000 版本开始支持；
- 批量正常分区迁移功能自 LDP1.0.V202501.01.000 版本开始支持；
- 注：运维指令和正常分区迁移相同，由partkeeper底层自动识别下达的分区迁移指令是属于正常分区迁移还是异常分区迁移；

运维指令

指令下达（参见partkeeper管理功能号SendAdminMsg）

<http://ip:port/ldp?Pretty=1&PluginName=ldp20.partkeeper&FuncName=SendAdminMsg>

入参说明(分区迁移)

```
{
  "ShardNoGroup": [
    {
      "/*": "Str, 集群名_分片号",
      "ClusterName": "mdb_1",
      "Partition": [
        {
          "/*": "Str, 待迁出的分区号",
          "PartNo": "0",
          "/*": "Str, 待迁入的核心节点名",
          "NodeName": "proc#0"
        }
      ]
    }
  ]
}
```

查看指令结果（参见partkeeper管理功能号GetAdminMsgResult）

<http://ip:port/ldp?Pretty=1&PluginName=ldp20.partkeeper&FuncName=GetAdminMsgResult>

出参说明(分区迁移)

```
{
  "/*": "I32, 100=核心发出, 101=网发出, 102=partkeeper发出, 103=运维发出",
  "MsgType": 103,
  "/*": "U64, 当前指令对应的编号, 用于查询指令结果",
  "OrderId": 1,
  "/*": "Str, 集群名_分片号",
  "ClusterName": "mdb_1",
  "/*": "U64, 分片内轮次号, 从1开始",
  "ShardRoundNo": 1,
  "/*": "Str, 生成该指令的时间串, YYYY-MM-DD HH:MM:SS, sss",
  "BeginTimeStr": "2025-10-29 11:41:28.705",
  "/*": "Str, 指令结果的时间串, YYYY-MM-DD HH:MM:SS, sss",
  "EndTimeStr": "2025-10-29 11:41:28.895",
  "/*": "Str, 指令处理的整体耗时, ms",
  "Time": 190,
  "Partition": [
    {
      "/*": "Str, 待迁出的分区号",
      "PartNo": "0",
      "/*": "Str, 待迁出的核心节点名",
      "OldWriter": "proc#0",
      "/*": "Str, 待迁入的核心节点名",
      "NewWriter": "proc#1",
      "/*": "U64, 指令完成时, 迁出分区组的redo数据起始进度值",
      "MinMessageNo": 18446744073709551615,
      "/*": "U64, 指令完成时, 迁出分区组的redo数据最大进度值",
      "MaxMessageNo": 100
    }
  ],
  "/*": "I32, 分区迁移失败时存在, 失败的错误码",
  "ErrorNo": 7,
  "/*": "Str, 分区迁移失败时存在, 失败的错误信息",
  "ErrorInfo": "管理端进行分区迁移失败, 原因 分区原本就归属核心 [proc#3] 无需迁移"
}
```

使用限制

- 一次运维指令只能修改一个分片下的分区列表；
- 当前分片下, 上次的运维指令已经结束；
- 迁出分区的核心必须是在线核心，且状态正常(即正确分片下, 核心存在分片下)；
- 迁入分区的核心必须是在线核心，且状态正常；

- 迁出分区的核心和迁入分区的核心不能相同；
- 迁出的分区号是已经存在的，并且归属于迁出核心；
- 一条批量迁移指令中，不允许同时将多个分区迁移给不同的核心；

7.6.2. 异常分区迁移，将异常下线核心的分区划分到其他在线核心

数据网格模式下，系统运行过程中，某个核心异常后，该核心上负载的分区所对应的业务没有人处理，需要将这些分区划分给其他现有的核心进行接管处理。

- 该功能自 LDP1.0.V202501.00.000 版本开始支持；
- 批量异常分区迁移功能自 LDP1.0.V202501.01.000 版本开始支持；
- 异常分区自动迁移的功能自 LDP1.0.V202501.01.000 版本开始支持；
- 注：运维指令和正常分区迁移相同，由partkeeper底层自动识别下达的分区迁移指令是属于正常分区迁移还是异常分区迁移；

运维指令

指令下达（参见partkeeper管理功能号SendAdminMsg）

<http://ip:port/ldp?Pretty=1&PluginName=ldp20.partkeeper&FuncName=SendAdminMsg>

入参说明(分区迁移)

```
{
  "ShardNoGroup": [
    {
      "/*": "Str,集群名_分片号",
      "ClusterName": "mdb_1",
      "Partition": [
        {
          "/*": "Str,待迁出的分区号",
          "PartNo": "0",
          "/*": "Str,待迁入的核心节点名",
          "NodeName": "proc#0"
        }
      ]
    }
  ]
}
```

查看指令结果（参见partkeeper管理功能号GetAdminMsgResult）

<http://ip:port/ldp?Pretty=1&PluginName=ldp20.partkeeper&FuncName=GetAdminMsgResult>

出参说明(分区迁移)

```
{
  "/*": "I32,100=核心发出,101=网关发出,102=partkeeper发出,103=运维发出",
  "MsgType": 103,
  "/*": "U64,当前指令对应的编号,用于查询指令结果",
  "OrderId": 1,
  "/*": "Str,集群名_分片号",
  "ClusterName": "mdb_1",
  "/*": "U64,分片内轮次号,从1开始",
  "ShardRoundNo": 1,
  "/*": "Str,生成该指令的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "BeginTimeStr": "2025-10-29 11:41:28.705",
  "/*": "Str,指令结果的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "EndTimeStr": "2025-10-29 11:41:28.895",
  "/*": "Str,指令处理的整体耗时,ms",
  "Time": 190,
  "Partition": [
    {
      "/*": "Str,待迁出的分区号",
      "PartNo": "0",
      "/*": "Str,待迁出的核心节点名",
      "OldWriter": "proc#0",
      "/*": "Str,待迁入的核心节点名",
      "NewWriter": "proc#1",
      "/*": "U64,指令完成时,迁出分区组的redo数据起始进度值",
      "MinMessageNo": 18446744073709551615,
      "/*": "U64,指令完成时,迁出分区组的redo数据最大进度值",
      "MaxMessageNo": 100
    }
  ],
  "/*": "I32,分区迁移失败时存在,失败的错误码",
  "ErrorNo": 7,
  "/*": "Str,分区迁移失败时存在,失败的错误信息",
  "ErrorInfo": "管理端进行分区迁移失败,原因 分区原本就归属核心[proc#3]无需迁移"
}
```

使用限制

- 一次运维指令只能修改一个分片下的分区列表；
- 当前分片下,上次的运维指令已经结束；
- 如果上次是异常分区迁移，则在对该异常核心的所有分区迁移完成前，禁止执行其他运维指令；
- 迁出分区的核心必须已经完成新的数据源接管，且其他核心同步完成(即在线核心已经完成异常通道上的redo数据同步)；
- 迁入分区的核心必须是在线核心，且状态正常；
- 迁出分区的核心和迁入分区的核心可以是相同的，表示分区依旧由原核心接管(异常核心已经重启)；
- 迁出的分区号是已经存在的，并且归属于迁出核心；
- 一条批量迁移指令中，不允许同时迁移多个核心的分区；

7.6.3. 分区组迁移,原redo通道主机无法恢复，为其分配新的主机

正常情况下，核心异常后，该核心持有的redo通道上的主机状态会被partkeeper自动迁移给现有的其他核心；但是如果其他核心无法

满足接管主机状态的条件时，则该主机状态会一直维持在原来的异常核心上，直到这个核心恢复正常后重新接管主机状态；

- 该功能自 LDP1.0.V202501.01.000 版本开始支持；

其他核心无法接管主机状态的场景

- 所有核心均和partkeeper断开连接；
- 现有的在线核心在异常redo通道上的处于同步中状态（即redo数据不全）；
- 常见的业务场景有：有一个核心从未启动过。因为该核心从未启动，所以其他核心在该通道上永远处于同步中状态，无法接管主机状态。

只有redo通道上的主机正常时，才允许对该通道上的分区进行分区迁移。如果异常核心一直无法恢复，并且其他核心也不满足自动接管主机状态的条件，但又要将异常核心上的分区迁移给现有的其他核心处理。就需要通过 分区组迁移 令一个现有的核心先接管通道（执行过分区组迁移的redo通道通常处于数据丢失状态，原核心后续就不要再重启了，另外迁出的分区也可能存在数据丢失的（丢失redo数据，表示在原主机上可能已经被修改了数据,但是当前系统内该数据修改的操作被回退了））。

运维指令

指令下达（参见partkeeper管理功能号SendAdminMsg）

<http://ip:port/ldp?Pretty=1&PluginName=ldp20.partkeeper&FuncName=SendAdminMsg>

入参说明(分区组迁移)

```
{
  "ShardNoGroup": [
    {
      "/*": "Str,集群名 分片号",
      "ClusterName": "mdb_1",
      "PartGroup": [
        {
          "/*": "Str,将redo通道的主机状态转移给该分区组",
          "GroupName": "multiproc_proc#0_partgroup_2",
          "/*": "Str,该分区组状态迁移后的状态,PrepareDS=准备成为数据同步源",
          "GroupStatus": "PrepareDS"
        }
      ]
    }
  ]
}
```

查看指令结果（参见partkeeper管理功能号GetAdminMsgResult）

<http://ip:port/ldp?Pretty=1&PluginName=ldp20.partkeeper&FuncName=GetAdminMsgResult>

出参说明(分区组迁移)

```
{
  "/*": "I32,100=核心发出,101=网发出,102=partkeeper发出,103=运维发出",
  "MsgType": 103,
  "/*": "U64,当前指令对应的编号,用于查询指令结果",
  "OrderId": 1,
  "/*": "Str,集群名 分片号",
  "ClusterName": "mdb_1",
  "/*": "U64,分片内轮次号,从1开始",
  "ShardRoundNo": 1,
  "/*": "Str,生成该指令的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "BeginTimeStr": "2025-10-29 11:41:28.705",
  "/*": "Str,指令结果的时间串,YYYY-MM-DD HH:MM:SS,sss",
  "EndTimeStr": "2025-10-29 11:41:28.895",
  "/*": "Str,指令处理的整体耗时,ms",
  "Time": 190,
  "PartGroup": [
    {
      "/*": "Str,发生状态变更的分区组名",
      "GroupName": "multiproc_proc#0_partgroup_2",
      "/*": "Str,变更前的分区组状态,Normal=写核心正常,RouterChange=需要分区组迁移,DataSource=作为数据同步源,SyncData=待同步数据,PrepareDS=准备成为数据源,PrepareW=准备恢复写核心",
      "OldStatus": "Normal",
      "/*": "Str,变更后的分区组状态,Normal=写核心正常,RouterChange=需要分区组迁移,DataSource=作为数据同步源,SyncData=待同步数据,PrepareDS=准备成为数据源,PrepareW=准备恢复写核心",
      "NewStatus": "PrepareDS"
    }
  ],
  "/*": "I32,分区迁移失败时存在,失败的错误码",
  "ErrorNo": 7,
  "/*": "Str,分区迁移失败时存在,失败的错误信息",
  "ErrorInfo": "管理端进行分区迁移失败,原因 分区原本就归属核心[proc#3]无需迁移"
}
```

使用限制

- 一次运维指令只能修改一个分片下的一个分区组的状态；
- 当前分片下,上次的运维指令已经结束；
- 当前redo通道必须处于等待原主机恢复状态；
- 接管redo通道的分区组所对应的核心必须在线，且轮次号与partkeeper相同（允许redo序号比原主机小）；

8. FAQ

- 在\$BaseDir/bin/目录下，可使用strings hsldp | grep V1.0 查询版本号信息
- 如果配置了正确的环境变量，界面和日志打印还是undefined error
解决方法：设置系统语言LANG=zh_CN.UTF-8
- 用ILdpMsgWriter创建消息时发生错误


```
(gdb) bt
#0  CLdpMsgWriter::Create (this=0x12e5a50, lpIAllocator=<optimized out>, nFixedSize=48,
    nBizHeadSize=<optimized out>, lpRefHead=0x3)
    at ../../utilities/ldp_message_impl.cpp:689
#1  0x0000000000400f8a in main (argc=1, argv=0x7ffe15557788)
    at /home/xingjianwei/projects/Project3/demo.cpp:138
(gdb)
```

原因: ldp_message.h版本不匹配

4. 出现下图报错往往是配置中端口已被占用产生冲突, 检查配置文件中的各个可能冲突的端口配置并修改。

```
Acceptor创建失败 CreateAcceptor=20117(tcp_server.cpp,505,Start)
```

5. 出现类似以下的写事务超时日志时:

1120 152618.584317 10115 WARN 2929 3647 NULL 写事务时间超长警告(LongTime WriteTransaction(Commit), TCId=26, UseTime(us)=1461126, TransactionNo=18446744073709551615, FunctionID=4294967295, Table=No Table,.../Include/bizx_log_define.h:64,Log)

它的TransCtrlNo为UINT64_MAX,FunctionID为UINT32_MAX,Table信息为No Table,说明这是一个只读的操作但是开了一个写事务去做,比如开启写事务执行select语句,这里虽然是写事务超时,但是并没有进行任何修改,用户需要避免被这个日志误导

6. Rpc功能号约束: 需要单独的线程组来处理Rpc功能号, 防止出现死锁

例子: 以序列号Rpc申请为例, 在线程1中调用了获取序列号, 触发远程序列号申请, 此Rpc功能号若也分配到线程1中处理, 则会造成死锁

9. 修改记录

版本	日期	说明
V1.0.0.0	20210909	报盘框架管理功能注释中添加字典说明、定时间隔注释加单位、获取工作线程队列信息去掉内存上线、配置中修改业务so名
V1.0.0.1	20211014	更新fens增加动态更新参数, fens增加单独参数表, 更新fens管理功能
V1.0.0.2	20211112	更新cleankdp工具说明, 增加重演模式; 新增日志合并功能的相关配置
V1.0.0.3	20211130	回库增加管理功能GetConnectionStatus, 修改GetMonitor, 增加BizErrResendMode配置; RCM新增插件配置ConfigUpdate说明; 前置管理功能删除ConnectStat, 新增FrontendStat,PluginStat,RcmBackendStat, RcmContextStat, 修改ASStat,GWStat, WanClientStat,LanClientStat; 核心修改管理功能GetBizModuleInfo格式
V1.0.0.4	20211201	日志代理增加日志类型说明、记录操作系统日志功能、时延分析功能和日志合并说明, 修改几处文字错误
V1.0.0.5	20211215	fens部分更新1、新增对账户信息收集超时控制说明, 核心返回账户信息结构变更说明等 2、在Routes配置中新增Nodes字段, 增加Routes、Functions、Systems、Nodes字段的注释; Bizproc部分更新, EnableNagle默认值改为false; tool工具部分更新, 增加kdpEnvPrepare脚本的使用说明; 回库修改管理功能GetMonitor增加吞吐量统计, 新增三个统计5min内每秒吞吐量的管理功能
V1.0.0.6	20211222	1、日志代理增加日志合并阈值2、增加路由解析
V1.0.0.7	20211229	增加记录池插件的管理功能描述; 新增防止组播窜包校验开关, 配置项为IsCheckMsg
V1.0.0.8	20220110	rcm管理功能字段调整和更新; 新增产品名校验开关, 配置项为IsCheckProductName
V1.0.0.9	20220129	核心和前置增加rcm代理的配置和字段解析。增加自动创建RCM配置说明。增加前置BackendStat管理功能说明。
V1.0.0.10	20220221	日志代理增加获取支持抓包插件和批量开启/关闭插件抓包管理功能
V1.0.0.11	20220224	fens新增AccountInfoCollectFinishStat管理项;核心增加高可靠通道配置, 完善Singleton配置
V1.0.0.12	20220309	修改日志代理GetMonitor管理功能, 修改GetMsgLog管理功能说明
V1.0.0.13	20220311	增加支持通过配置"BizFuncLibs"字段指定特定的功能组件; 业务组件增加支持时延分析字段; 增加获取时延分析和设置时延分析管理功能
V1.0.0.14	20220314	更新调整RCM可靠组播配置项说明; 增加OnMessage回调失败次数统计, RCM管理功能新增OnMessageCountFailed字段
V1.0.0.15	20220329	业务框架新增字段PriorityGetTransNo, 用于优先获取事务号
V1.0.0.16	20220411	报盘框架增加字段IsQuery及rpc相关配置, 用于查询管理系统及增加rpc和rcm_proxy相关管理功能
V1.0.0.17	20220421	业务框架新增请求可靠功能、管理功能GetGlobalQueueInfo查看核心全局可靠队列当前状态;fens管理功能SiteInfoStat新增ShowAllSite入参, 允许查看所有连接信息;修改业务框架GetFuncDealInfo管理功能, 增加定位总耗时和平均耗时以及修改其他部分描述;增加随机端口预留范围定义, 获

		取，设置等
V1.0.0.18	20220426	删除rcm插件展示管理功能中的关于rpc模块的展示部分
V1.0.0.19	20220428	rcm添加主题UserDefined配置说明，上下文UserDefined和预设主题"Rx"配置说明
V1.0.0.20	20220516	报盘框架增加管理功能紧急恢复报盘状态、获取落地文件状态信息、获取当前是否为查询管理系统模式恢复状态及是否恢复完成
V1.0.0.21	20220520	业务框架新增主备请求落地，配置项为ReqLogWrite，以及管理功能GetReqLogWritersInfo查看请求落地信息
V1.0.0.22	20220526	rcm配置项TransMode新增rdma_mc通信模式
V1.0.0.23	20220606	新增配置项，启用或禁用UM层发送端套接字收发分离功能
V1.0.0.24	20220607	业务框架新增黑名单功能（需要忽略的功能号）FuncBlackList，以及管理功能 GetBizFuncBlackList, GetErrorProcessCount增加黑名单统计信息 BlackListIgnoreCount字段。增加重演模式配置开关 IsReplyMode
V1.0.0.25	20220609	新增节点定位插件说明
V1.0.0.26	20220613	rcm新增上下文配置项RecvBindAddr
V1.0.0.27	20220614	修改配置项，UM层发送端套接字收发分离功能，改成默认为false
V1.0.0.29	20220705	更新清理脚本说明：新增脚本日志，增加日志保留天数参数，增加主备重演备份目录参数，增加清理脚本检查模式
V1.0.0.30	20220706	业务框架新增配置项EnableRedoSupplement，支持Redo跳号截断和跳号补充功能；管理功能支持查看写Redo故障状态
V1.0.0.31	20220811	rcm管理功能新增RCM校验和与RMS流创建失败消息次数统计
V1.0.0.32	20220816	检查并更新各Rcm配置项默认值与最新代码保持一致
V1.0.0.33	20220822	回库新增RedoTypes配置， GetMonitor增加单线程模式重发次数统计
V1.0.0.34	20220824	新增kdp_cres_agent插件相关说明；管理功能WanClientStat，增加MaxWanCount字段，管理功能LanClientStat，增加MaxLanCount字段;修改前置配置对MaxConnectCount的注释
V1.0.0.35	20220831	fens支持站点使能
V1.0.0.36	20220831	业务框架"Singleton"通道配置支持数组
V1.0.0.37	20220914	清理脚本增加清理 todb 模式，用于 todb 重启后重头开始落库
V1.0.0.38	20220919	业务框架增加 记录池内存表 的查看功能
V1.0.0.39	20220920	管理代理增加身份注册功能，前置增加 GetNetworkServiceConfig 管理功能
V1.0.0.40	20220923	增加Rcm的TxMaxHistoryMsgCount和TxMaxHistoryMsgTimeMill配置说明
V1.0.0.41	20220924	配置项EnableQuickAck默认值调整为1
V1.0.0.42	20220924	Rcm新增配置项IsReplay，支持上下文开启重演模式；新增配置项FlushOnStop，支持上下文停止时将缓存区数据写入到持久化文件
V1.0.0.43	20220924	新增配置项NakDetectMicro，支持接收端补缺检测功能
V1.0.0.44	20220926	fens支持前置代理，新增前置代理插件说明，fens管理功能 AccountPartitionStat，SiteInfoStat调整
V1.0.0.45	20221013	记录池插件新增配置项说明，更新部分管理功能示例json及说明
V1.0.0.46	20221018	更新前置LanClientStat和WanClientStat管理功能
V1.0.0.47	20221018	业务框架新增配置 LdpErrorRspWithReqMsg，支持回复平台错误应答时携带请求消息
V1.0.0.48	20221019	增加配置上传命令行工具（kdpconfig）使用说明
V1.0.0.49	20221021	清理脚本增加keeper注释
V1.0.0.50	20221025	增加rcm线程说明
V1.0.0.51	20221031	更新Rcm的配置项RecvBindAddr配置项关于跨平台的注意说明
V1.0.0.52	20221031	todb增加管理功能GetErrNoWhiteListInfo、GetFuncNoDeallInfo，修改管理功能OnMonitor个别字段；增加白名单、错误重传时间相关配置
V1.0.0.53	20221101	清理脚本增加清理 todb 错误信息文件；增加todb管理功能 ResendBizErr， OnMonitor管理功能增加IsBizErrAutoResend字段
V1.0.0.54	20221103	回库ProductName配置增加场景限制的使用说明
V1.0.0.55	20221107	todb增加clean时的备份目录配置，增加管理功能LastManualResendInfo
V1.0.0.56	20221122	rcm插件新增上下文级配置SendBindCoreId的说明

V1.0.0.57	20221206	日志插件MaxQueueSize指定队列大小，管理功能GetMonitor增加统计信息。包括队列满、内存申请失败、时延对象申请失败、traccid对象申请失败
V1.0.0.58	20221206	增加核心全局队列部分字段和工作队列字段，增加kdpPing工具使用说明
V1.0.0.59	20221214	rcm插件管理功能新增接收端的丢包率字段PacketLossRate、RecentlyFiveSecondsLossRate
V1.0.0.60	20221227	业务框架管理功能GetFuncDealInfo增加redo耗时统计字段
V1.0.0.61	20230103	报盘框架管理功能GetOfferGroupInfo增加BizVersion业务头文件版本字段，GetOfferStatisticsInfo增加席位的监控信息
V1.0.0.62	20230110	业务框架增加管理功能GetOrSetFunctionRunningStatus，支持暂停和启动功能号
V1.0.0.63	20230110	管理代理增加管理功能GetAppsStatus;增加kdp_operation脚本解析
V1.0.0.64	20230111	新增Rcm上下文配置项IsTxRecord和IsRxRecord
V1.0.0.65	20230113	业务框架增加配置TimeLogAlias，支持设置时延服务别名
V1.0.0.66	20230113	报盘框架增加配置TimeLogAlias，支持设置时延服务别名
V1.0.0.67	20230206	增加业务参数列表功能 --args 业务参数列表
V1.0.0.68	20230328	新增Rcm上下文配置项EnableShmBusyWait和ShmInterruptEventName
V1.0.0.69	20230414	新增Rcm主题配置项NakFlowControlCount、NakFlowControlTime配置，上下文配置项新增NakRequestInterval、TrigQuickNakCount说明
V1.0.0.70	20230424	修改nproxy的NumThreads错误示例配置，正确类型是整型
V1.0.0.71	20230524	业务框架增加主备模式下，多进程上下文的功能描述
V1.0.0.72	20230524	业务框架增加多线程框架的共享模式配置
V1.0.0.73	20230524	业务框架增加守护相关配置
V1.0.0.74	20230524	todb修改配置选项
V1.0.0.75	20230608	增加kdpmdb_tools工具使用说明
V1.0.0.76	20230615	remoter插件增加ThreadNamePrefix配置，支持设置TCP工厂线程名前缀配置
V1.0.0.77	20230616	todb插件增加gwredo组件配置，以及应答处理配置
V1.0.0.78	20230628	bizproc插件增加MDB数据组件配置说明
V1.0.0.79	20230629	todb插件增加开启时延日志说明，增加组件DataMgrDir配置说明，增加部分回库管理功能
V1.0.0.80	20230717	bizproc插件增加管理功能GetFuncDealInfo入参，只查看某些功能的统计信息
V1.0.0.81	20230717	增加模板引用功能说明
V1.0.0.82	20230718	nproxy插件增加配置项RequestTimeoutMill用于设置读取http请求头的超时时间，修改KeepAliveTimeoutMill和EnableKeepAlive的错误说明
V1.0.0.83	20230720	nproxy插件新增管理功能GetAppJsonInfo
V1.0.0.84	20230803	日志代理抓包管理功能GetMsgLog和QueryFidleData输出字段新增kdpmsg头部字段和可选头信息
V1.0.0.85	20230803	回库插件配置说明文档更新，调整错误处理配置，增加文件队列配置信息，增加在途配置。回库管理功能说明完善
V1.0.0.86	20230803	新增节点定位规则
V1.0.0.87	20230805	新增业务框架多分片模式，业务状态控制规则以及管理功能
V1.0.0.88	20230806	新增todb管理功能GetClusterInfo ForceStartRecv ForceStopRecv，修改部分配置
V1.0.0.89	20230806	新增业务框架功能号分组FunctionThreadAffinity配置
V1.0.0.89	20230821	节点定位新增字符串范围规则
V1.0.0.90	20230823	管理代理废弃参数EnableKeepAlive
V1.0.0.91	20230827	RCM管理功能增加多进程上下文信息
V1.0.0.92	20230828	增加Mdb数据组件管理功能
V1.0.0.93	20230912	数组件据增加Mdb时延跟踪管理功能
V1.0.0.94	20230912	todb增加主备仲裁方式说明
V1.0.0.95	20230917	核心增加GetShardNo功能号

V1.0.0.95	20231012	合并分支：核心修改ProtocolMode为ProtocolMode； 主干前置核心默认配置修改
V1.0.0.96	20231013	todb_v2_gwredo组件增加配置MaxPackerSize;前置修改"ASRcm"管理功能的"AS"字段为"ASRcm"
V1.0.0.97	20231019	todb_v2增加拦截消息数量统计
V1.0.0.98	20231024	remoter配置更新
V1.0.0.99	20231027	tcp工厂线程数配置描述整理
V1.0.0.100	20231027	todb管理功能名和说明调整
V1.0.0.101	20231028	补充remoter的AutoStart配置字段描述
V1.0.0.102	20231028	增加平台配置项加密配置说明
V1.0.0.103	20231028	修正配置名称为Transaction, 管理功能增加对事务超时时间展示
V1.0.0.104	20231102	前置代理支持路由转发模式，增加节点定位路径规则，message文件夹下.level文件加载,ldp_mdtools解析工具模式新增支持模式3
V1.0.0.105	20231107	todb_v2 所有管理功能的CLusterName参数说明修改， GetFileQueueInfo参数命名调整
V1.0.0.106	20231110	增加mdb使用场景限制
V1.0.0.107	20231124	remoter增加EnableBackendAppStatusCtrl和EnableBackendBizStatusCtrl配置项；lcoate支持备注
V1.0.0.108	20231128	修改Components的配置项说明
V1.0.1.109	20231129	todb_v2 增加配置平台错误和业务错误号范围，修改部分todb配置项错误说明
V1.0.0.110	20231201	修改核心，添加配置，支持指定设置到HighReliable的msgno是否为连续最大序号，默认true
V1.0.0.111	20231201	表分析插件支持继承、枚举、位域、模板
V1.0.0.112	20231123	完善rcm的SyncMode配置项说明
V1.0.0.113	20231204	修改ldpmdb_tools使用说明，增加mode=1/2模式，增加命令行模式
V1.0.0.114	20231205	logproxy的管理功能GetMonitor增加时延打点池的信息
V1.0.0.115	20231207	数据组件增加上场监控管理功能信息GetMdbInitInfo， 并且 GetMdbTableInfo附加上场信息
V1.0.0.116	20231212	front增加配置项EnableReplyErrRsp
V1.0.0.117	20231214	todb_v2 增加管理功能GetModuleInfo展示加载的组件信息
V1.0.0.118	20231215	核心支持异步写redo
V1.0.0.119	20231218	核心RPC增加相关配置说明
V1.0.0.120	20231218	新增Rcm上下文IsMandatoryAcker和UseSharedMemory配置项说明
V1.0.0.121	20231219	新增内存数据库默认批量遍历大小配置DefaultBatchSize
V1.0.0.122	20231222	增加rcm的AddAllMemberToCluster与StartElection管理功能说明
V1.0.0.123	20231223	回库组件(mdb、gw)增加配置项“TablesTodb”，表示必须回库的mdb表名
V1.0.0.124	20231227	Remoter插件管理功能结构重构
V1.0.0.125	20231228	回库增加配置TablesTodb、 TablesDontTodb使用说明
V1.0.0.126	20240105	rcm新增5个管理功能GetContextInfo、GetTxSessionInfo、GetLeaderSessionInfo、GetContextInfoOfRtxMP、GetLeaderSessionInfoOfRtxMP的说明;增加核心管理功能的备注
V1.0.0.127	20240123	增加ldpRedoParser工具使用说明，增加配置项outFileMerge
V1.0.0.128	20240126	Remoter增加EnableUnicastMode配置说明
V1.0.0.129	20240130	GetMdbTableInfo管理功能修改示例文件大小， SetMdbTrace新增说明
V1.0.0.130	20240130	remoter支持设置RCM最大排队序号； hsl dp支持-f参数指定快照名;增加查询请求跨机房转发说明
V1.0.0.131	20240131	Rcm添加GetDynamicConfigInfo管理功能说明
V1.0.0.132	20240204	数据组件增加切主功能号配置ToActiveFuncList
V1.0.0.133	20240204	增加快照使用指南
V1.0.0.134	20240205	Rcm GetTxSessionInfo管理功能新增了序号相关字段,新增了 GetRxTopicInfo的管理功能； 核心管理功能注释； fens增加配置 EnableQuerySystemNo、 EnableQueryNodeNo、 EnableQueryAccPart、

		IsPrivate
V1.0.0.135	20240205	logproxy SetLogLevel管理功能说明完善,核心管理功能 GetOrSetRunningStatus修改应答说明
V1.0.0.136	20240226	修改平台安装包下载地址;前置查询请求跨机房转发-配置项说明重复内容过多; mldbdata管理功能SetRSDealTimeStatus修改
V1.0.0.137	20240306	修复Rcm插件的管理功能GetRxTopicInfo的部分描述有误的问题
V1.0.0.138	20240312	新增上下文EnableSyncMulticastAdaption和EnableRcmProtocolCheck配置项及说明,增加RecoverToMulticastSync管理功能说明
V1.0.0.139	20240313	todbv1 v2新增管理功能获取上一次错误应答信息 LastErrMsg, mldbredo管理功能GetMonitor增加字段LastExecErrMsg: 上一次数据库执行错误信息)
V1.0.0.140	20240313	hskdp支持-a命令行参数,配了默认不加速,不配默认加速
V1.0.0.141	20240314	Rcm新增上下文EnableSyncLatencyMetric配置项说明,增加 GetSyncLatencyInfoOfCtxMP管理功能说明
V1.0.0.142	20240325	MDB数据组件示例配置中增加功能号相关示例配置
V1.0.0.143	20240325	Rcm插件多进程上下文管理功能新增Producer模块字段说明和新增Rcm配置 CheckNameConflict的说明
V1.0.0.144	20240327	核心GetBizStatusInfo管理功能完善;增加UseRcmProxy字段描述
V1.0.0.145	20240328	增加前置消息转换组件说明
V1.0.0.146	20240403	增加组播环境测试工具说明
V1.0.0.147	20240407	修复管理功能GetMdbSequenceInfo、GetShardNo、GetConnectInfo、GetRpcStatistics的部分描述有误的问题;业务框架插件增加管理功能 GetNodesWithFuncWhitelist、MDB数据组件增加管理功能 GetMdbAddressSpaceInfo;TierName为非必填,机房信息更新,增加机房修改工具,增加管理功能appinfo机房信息
V1.0.0.148	20240422	MDB数据组件增加管理功能GetMdbLongTimeTransactionHistory获取超长事务告警历史记录、SetMdbAccessControl设置Mdb权限校验
V1.0.0.149	20240423	toddb增加异地机房相关配置"Remote","ClusterToddb",管理功能GetMonitor增加异地机房redo统计, GetTableInfo增加表集群回库配置、异地机房标识,补充gwredo组件管理功能跟mldbredo组件一致, mldbredo组件 GetTableInfo修改英文错别字
V1.0.0.150	20240424	新增AccessControl的字段和说明
V1.0.0.151	20240506	数据组件去掉最大redo文件大小配置MaxRedoFileSizeMB及redo文件路径; mproxy管理功能增加GetSocketMgrInfo、GetStatsInfo, todv_v2管理功能GetRedoDealInfo修改跟程序对应不上字段;核心管理功能 SetBizFuncIdTsList增加clear字段注释;增加前置管理功能RpcClientStat, 优先去备模式;增加风险提醒
V1.0.0.152	20240513	新增前置消息转换组件管理功能GetMsgConvertStatistics获取消息转换统计信息
V1.0.0.153	20240513	添加核心异地机房说明
V1.0.0.154	20240520	rcm插件新增上下文的SyncRepairDataMode配置项及说明; mldb数据组件增加管理功能GetExternErrorInfo获取业务周边外部错误号映射信息
V1.0.0.155	20240521	数据组件新增配置池最大数量PackerMaxCount;核心添加清空工作线程队列管理功能;mldb数据组件管理功能GetUfimdbsCtrlInfo新增字段; mldb数据组件新增配置EnableFormatConvert、XmlFileName,新增对外错误应答信息转换相关配置说明
V1.0.0.156	20240522	Rcm新增 OutOfSyncTimes、ArbElectionWaitMemberMilli 配置说明和 AsynchronousSqn配置对多进程上下文的支持和说明
V1.0.0.157	20240522	Rcm_Agent 新增 LocalNetworkNumberUdp 配置说明
V1.0.0.158	20240523	todv_v2业务错误重发功能,增加管理功能GetBizErrRedoInfo、ForceSendBizErrRedo、GetSyncFileQueueInfo,修改管理功能GetMonitor、GetRedoDealInfo,增加配置BizErrResend以及多进程上下文
V1.0.0.159	20240527	异地核心改为采用async同步旁路收到消息,不控rpo
V1.0.0.160	20240605	mldb数据组件新增配置项ShmUseProtect及其说明, mldb数据组件管理功能 GetUfimdbsCtrlInfo新增字段ShmUseProtect
V1.0.0.161	20240606	rcm_agent插件新增管理功能GetStatistics获取RCM网络代理统计信息
V1.0.0.162	20240606	rcm_agent插件新增TcpSockBufferSizeKBytes,UdpSockBufferSizeKBytes; 修改locate优先级
V1.0.0.163	20240617	toddb获取文件队列信息管理功能补充;修复klp_front管理功能LanClientStat和WanClientStat出参描述有误的问题
V1.0.0.164	20240620	zk_handler增加ZkLockCount配置

V1.0.0.165	20240626	hskdp和cleanhskdp支持-l参数，配置该参数后将使用指定文件名的本地配置文件，而不会使用服务器上的配置文件;nproxy插件管理功能GetAppInfo新增字段ConfigMode
V1.0.0.166	20240626	ldp_rcm插件新增 TimeMilOfSingleStatisticalPeriod, CountOfSingleStatisticalPeriod 配置项说明，新增 GetRecentSyncLatencyByTimeOfCtxMp, GetRecentSyncLatencyByCountOfCtxMp, RejectFollowerFromCluster, AdmitFollowerToCluster 管理功能说明， GetLeaderSessionInfoOfCtxMP 管理功能新增 "RejectedFromCluster" 字段；locate本地配置模式说明
V1.0.0.167	20240627	log_proxy插件，增加时延日志tag使能，规则配置及管理功能说明
V1.0.0.168	20240710	log_proxy插件增加配置项LogThreadBindCpu
V1.0.0.169	20240710	zk_handler插件，增加 GetZkLockInfo 管理功能说明
V1.0.0.170	20240711	调整若干核心管理功能的名称及描述信息
V1.0.0.171	20240715	添加核心配置项LoadStrategy
V1.0.0.172	20240716	todbv1增加配置PlatErrMaxResendCount BizErrMaxResendCount, 修改MaxResendCount配置说明，管理功能GetMonitor增加 CurBizErrResendCount CurPlatErrResendCount CurTimeoutResendCount
V1.0.0.173	20240716	添加核心管理功能EnableFuncDealInfoDetails和GetFuncDealInfoDetails
V1.0.0.174	20240716	新增 Rcm配置项 ShmMemoryFileHole 说明
V1.0.0.175	20240718	logproxy插件 front插件 修改配置说明单词拼写错误，修改核心配置 RxTopics 的描述
V1.0.0.176	20240718	Rcm网络代理跨机房转发，对于仅指定Topic的转发场景，配置项说明改为按Topic+ Partition对应转发；优化fens的使用说明，增加fens运行模式章节
V1.0.0.177	20240805	remoter插件配置中，TCP类型的Backend增加EnableSession配置项
V1.0.0.178	20240808	修改表分析插件配置相关说明
V1.0.0.179	20240820	新增 Rcm插件的EnableSyncThreadBusyWait、SyncBindCoreId、RepairBindCoreId、SendLockMode配置项说明,以及支持多个轻量管控平台的配置说明 新增文件前缀和产品名可观测项
V1.0.0.180	20240914	新增日志字段解析说明；新增异常消息跳号说明
V1.0.0.181	20240915	新增RCM插件的 EnableAccSocket、OnloadStackName、TxOnloadStackName、RxOnloadStackName、BypassOnloadStackName、SyncRepairBindCoreId、EnableSyncRepairThread 配置项说明
V1.0.0.182	20240915	cleanhskdp新增-E选项，支持清理异常消息标识的持久化文件和共享内存
V1.0.0.183	20240918	front新增ZkServerAddr配置项说明， front新增ID为7的json作为配置项示例，管理功能BackendStat新增ZkServerAddr
V1.0.0.184	20240919	todbv1管理功能GetMonitor新增字段RetryingTodbKeys
V1.0.0.185	20241001	mdb数据组件新增管理功能GetMdbSqlExecInfo和GetMdbSqlExecDetail；mdb数据组件新增配置项SqlHistoryThreshold；mdb数据组件新增管理功能SetMdbSqlHistoryThreshold
V1.0.0.186	20241011	Rcm GetContextInfo管理功能新增了相关字段：上下文是否启动IsStart、传输模式TransMode、所属中心Zone、仲裁优先级ArbPriority
V1.0.0.187	20241014	添加说明：ldp使用onload启动时禁止使用latency-best配置文件；增加ldpLocateTool工具；优先去备模式增加3；todbv2的gwredo业务组件增加配置项ShardNo
V1.0.0.188	20241015	新增 Rcm插件配置项 CallBackMsgCountEachRound 说明
V1.0.0.189	20241018	核心新增GetRspLogWritersInfo管理功能，增加RspLogWrite配置项，清理脚本增加-Q选项
V1.0.0.190	20241024	MDBData新增配置项ReadTimeOutSec和WriteTimeOutSec，指定数据库读写超时，增加内存数据库冲突等待信息
V1.0.0.191	20241026	获取事务管理器信息增加非预期的写事务,保存点提交/回滚次数统计. 获取内存数据库控制信息增加是否开启自动写事务的状态信息.
V1.0.0.192	20241103	mdb数据组件增加配置项：是否开启自动写事务
V1.0.0.193	20241107	mdb数据组件新增配置项EnableForceSqlAuth和RoundMode，mdb数据组件管理功能GetUfmdbCtrlInfo新增字段RoundMode；ldpmdb_tools增加 mode=4根据togw业务错误日志解析redo
V1.0.0.194	20241111	核心支持同步redo开启压缩
V1.0.0.195	20241111	GetLeaderSessionInfoOfCtxMP 新增获取到作为主机的多进程上下文消息序号相关信息

V1.0.0.196	20241112	GetStatistics 管理功能在多线程上下文 ctxMps 中复用旁路发送端的管理功能 BypassTx，文档中新增本次修改管理功能说明
V1.0.0.197	20241113	添加管理功能，支持查询表分析和mdb的功能号
V1.0.0.198	20241113	记录池插件管理功能添加字段
V1.0.0.199	20241113	mbdata新增配置，支持rpo!=0
V1.0.0.200	20241115	新增todb、togw按表并发配置和相关管理功能，新增todb落库前检查配置开关,新增运维工具kdpClusterTool使用说明.
V1.0.0.201	20241118	补充RCM的TransceiverSeparation功能不支持在vma加速模式下开启的说明，增加内存数据库管理功能的相关字段。kdp_rcm插件新增DefaultMainZone、OnlyMainZoneActive配置项描述，FAQ增加说明;cleanhskdp新增支持-L和-x参数
V1.0.0.202	20241201	补充todb_v2中异地机房配置数组Remote的配置项说明
V1.0.0.203	20241202	梳理平台的绑核逻辑
V1.0.0.204	20241202	核心添加队列优先级配置；修复todbv1的配置项PlatErrMaxResendCount含义描述有误的问题
V1.0.0.205	20241202	增加配置EnableRdtsc
V1.0.0.206	20241209	核心增加配置StartTime和StopTime，表示定时功能执行时间段
V1.0.0.207	20241210	新增RCM上下文配置项ProducerMemPreallocateCount;remoter的backend中TCP类型增加"ForwardMode"字段，且增加管理功能；修复部分管理功能和和实际名称不一致的缺陷LDP-4399
V1.0.0.208	20241212	mbdata添加obproxy负载均衡相关的配置，用以构造连接串
V1.0.0.209	20241212	kdpClusterTool新增ErrorNo对应的处理措施
V1.0.0.210	20241212	增加守护插件的使用描述；todbv2管理功能GetWorkCacheInfo新增部分字段，并调整其中RedoMsg的格式为数组;增加AutoCreate字段描述
V1.0.0.211	20241214	mb数据组件管理功能GetUfmbCtrlInfo新增字段SeqTableStartCategory、SequenceSetRangeMinMax、SequenceSetCount；mb数据组件管理功能GetMdbTableInfo新增字段TableType
V1.0.0.212	20241215	补充kdpmb_tool解析redo文件的结果说明
V1.0.0.213	20241216	db_driver支持ob负载均衡功能配置文件示例和参数说明;增加kdp_mdbredo_GetTableLocate 和 kdp_gwredo_GetTableLocate,GetVersion.
V1.0.0.214	20241220	mb数据组件管理功能GetMdbSqlCacheDetail、SetMdbSqlCacheSizeLimit
V1.0.0.215	20241230	增加业务日志版本号；hskdp支持水鉴权；kdp_zonetool支持-c命令行
V1.0.0.216	20250109	增加redo合并和错误重试相关配置及管理功能，增加WriteRedo字段的说明
V1.0.0.217	20250110	完善使用手册中hskdp平台注册的rcm线程信息
V1.0.0.218	20250113	添加管理功能持久化相关配置；修改locate规则
V1.0.0.219	20250116	新增SqlFunctions功能号配置说明和对应管理功能 ufmb_data_GetSqlFunctionID同步调整
V1.0.0.220	20250117	更新索引字段遇到其他事务控制器在读取该记录，等待超时的设置及管理功能的展示
V1.0.0.221	20250120	新增Rcm_agent管理功能：GetLocalAgentStatus，GetClientConnectionStatus，GetServerConnectionStatus
V1.0.0.222	20250121	补充回库kdp_todb_proc_info表说明
V1.0.0.223	20250312	增加日间重试完成判断的字段说明 新增rcm插件配置项"ConfigUpdateDelayMilli"和"PrettyUploadConfig"说明 新增rcm插件GetCfgUpdateInfo管理功能
V1.0.0.224	20250317	支持备机处于toinactive状态允许特殊请求接入
V1.0.0.225	20250318	增加日志tag等级配置说明、管理功能说明
V1.0.0.226	20250318	管理功能GetUfmbCtrlInfo增加GC相关信息；数据组件新增管理功能 GetLoadTableLevel
V1.0.0.227	20250320	remoter插件新增配置参数项MsgCacheSize和CacheTimeout的说明；remoter插件管理功能在AS-RCM模式下新增管理功能参数CacheSendCount，用于统计缓存发送成功次数
V1.0.0.228	20250402	更新Rcm上下文配置项TransceiverSeparation备注，新增SyncRepairPortStart和SyncRepairPortEnd配置项说明
V1.0.0.229	20250403	更新Rcm主题分区配置项Addr备注，增加主题分区Addr配置约束
V1.0.0.230	20250407	为支持重演功能MDB增加的全局版本号相关的管理功能查看项

V1.0.0.231	20250410	修改日志版本的日志等级默认值、增加kdp_cres_agent配置项
V1.0.0.233	20250412	uifmdb_data管理功能GetMdbTransCtrlInfo增加全局表操作的统计信息
V1.0.0.234	20250414	核心框架支持重演
V1.0.0.235	20250418	uifmdb_data管理功能添加新的管理功能查看字段
V1.0.0.236	20250418	新增重演工具插件说明
V1.0.0.237	20250418	todb管理接口GetMonitor回报新增字段 BizErrorToResendTransCount,BizErrorToResendLocateValueCount GetBizErrStatistics接口入参新增字段ToResendQuery，出参新增字段 LastErrorNo GetBizErrorMsgInfo接口入参新增字段IsResent，出参新增字段 TotalCount，IsResent kdpClusterTool工具新增update_todb命令说明
V1.0.0.238	20250512	增加redo合并信息的debug日志
V1.0.0.239	20250515	mdbredo组件新增配置项IndexFieldsBlackList
V1.0.0.240	20250515	修正MyCnfName配置写法
V1.0.0.241	20250609	新增kdp_dbclient插件说明
V1.0.0.242	20250610	新增定位字段线程亲和性配置说明；核心新增字段ZoneAutoStart
V1.0.0.243	20250614	新增redo合并管理功能说明 kdpClusterTool工具update_todb命令新增-c参数说明 front新增管理功能SetFrontRoute，用于动态修改路由配置;新增配置 EnableDynamicRoutes用于开启该管理功能 bizproc新增管理功能ModifyRpcTimeout，用于修改RPC超时时间 todbV2新增管理功能GetOnTimerTransNoTaskStatistics，获取事务号定时任 务执行统计信息 todbV2新增配置对象OnTimerTransNoTasks事务号定时任务 todbV2新增OnTimerTransNoTasks事务号定时任务说明 bizproc新增管理功能GetZoneSwitchHostStatus查询主核心切换就绪状态
V1.0.0.244	20250616	新增redo合并管理功能说明
V1.0.0.245	20250617	新增redo_copy插件描述
V1.0.0.246	20250624	新增remoter配置项：InitialSendBuffSizeBytes 重演工具新增配置项：PrepareDisabled，管理功能StartReplay新增入参： PrepareDisabled 重演工具配置SpecialFuncInfo中增加类 型"WaitResponseWhiteList"和"WaitResponseBlackList"说明 重演工具管理接口支持按序号暂停重演功能说明
V1.0.0.247	20250629	新增redo合并管理功能说明
V1.0.0.248	20250703	12.003版本配置变更合入主干，修订前置缓存remoter插件MsgCacheSize 和CacheTimeout配置项的含义和说明，新增配置参数项 ResendBlackFuncList
V1.0.0.249	20250703	核心去重功能，前置新增去重队列配置开关及参数，和管理功能查看去 重队列信息
V1.0.0.250	20250706	redo合并相关配置补充
LDP1.0.V202401.13.005	20250709	新增前置错误转换组件front_error_transfer说明
LDP1.0.V202401A.13.005	20250710	kdp_remoter插件新增输出参数项MaxResendQSize
LDP1.0.V202401A.13.005	20250718	新增前置错误转换组件front_error_transfer说明
LDP1.0.V202401A.13.005	20250722	kdp_replay插件管理接口增加GetReplayDiffInfo，修改GetReplayProgress
LDP1.0.V202401A.13.005	20250726	增加管理功能可查看获取找到多个可合并置已完成的次数
LDP1.0.V202401.14.000	20250715	因子模块未正常加载时，支持启动报错，避免盘中出现问题
LDP1.0.V202401.14.000	20250715	redo_copy模块去除 RepeatAckCount 配置项，并修订了部分配置中的默认 值
LDP1.0.V202401.14.000	20250723	新增rcm配置修改工具rcmcfg_change_tool使用说明 remoter新增配置"MaxMsgReaderNum","UseMsgReaderPool"用于优化 RpcClient固定申请1024个MsgReader占用内存问题
LDP1.0.V202401.14.000	20250725	togw回库新增事务级表并发相关配置，数据组件新增导出表数据的配置 项和管理功能
LDP1.0.V202401.14.000	20250725	redo合并增加管理功能可查看获取找到多个可合并置已完成的次数
LDP1.0.V202401.14.000	20250728	新增kdp_coreparser核心转储文件解析插件和kdp_coreparsertool核心转储文 件解析工具 kdp_coreparser核心转储文件解析插件新增管理功能 UpdateWorkSpaceList、GetWorkSpaceList、ExecParserCoreDumpFile、 GetParserResultInfo

LDP1.0.V202401.14.000	20250729	新增Rcm配置项AllowDataRotate说明
LDP1.0.V202401.14.000	20250804	新增ldp_minitablecache插件介绍以及管理功能介绍
LDP1.0.V202401.14.000	20250808	增加dbclient插件的unisql依赖说明
LDP1.0.V202401A.13.006	20250811	增加管理功能设置慢事务的阈值
LDP1.0.V202401.15.000	20250818	新增bizproc插件的配置项DockerAdressConvert说明；增加rcm负载均衡模式的配置用例。
LDP1.0.V202401.15.000	20250821	增加dbclient插件的ObblConfigFile负载均衡配置参数说明；数组组件定时任务相关配置中新增全链路开关EnableTrace
LDP1.0.V202401.15.000	20250822	1.补全记录池插件的配置项说明;2.修改快照相关说明以匹配当前实现;3.增加7x24日切与备份
LDP1.0.V202401.15.000	20250822	新增todb批量sql和表并发相关配置及管理功能
LDP1.0.V202401.15.000	20250827	1.redo_copy新增发送窗口配置项，修改最大缓存默认值为32768； 2.增加显示轮次号和动态发送窗口管理功能
LDP1.0.V202401.15.000	20250905	补充快照启动约束
LDP1.0.V202401.16.000	2025015	增加ldp_locate管理功能以及配置字段。
LDP1.0.V202401A.13.011	20250917	togw回库新增事务级表并发相关配置，数据组件新增导出表数据的配置项和管理功能
LDP1.0.V202401A.13.011	20250917	修改bizproc中关于ReplayId的错误说明
LDP1.0.V202401.16.000	20250923	使用指南增加EmptyToSpaceConversion管理功能说明
LDP1.0.V202401.16.000	20250925	bizproc增加数据网格相关管理功能和配置说明
LDP1.0.V202401.16.000	20250925	使用指南增加网格相关管理功能说明
LDP1.0.V202401.17.000	20251016	修改front模块配置中流量控制说明
LDP1.0.V202401.17.000	20251016	增加logproxy中binlog traceidlog描述，修改timelog描述
LDP1.0.V202401.17.000	20251016	bizproc插件新增配置项TimerZonesRelayRoleStart； fens的采用集群定位的核心后端增加配置项ZkServerAddr； fens支持自定义的站点账户类型
LDP1.0.V202401.17.000	20251016	log_proxy 新增日志文件可配置功能；增加GetLogPath管理功能获取日志文件配置路径
LDP1.0.V202401.17.000	20251017	增加ldpRedoParser 支持日志级别可配置功能说明
LDP1.0.V202401.17.000	20251017	增加SQL执行总览通过执行次数进行过滤
LDP1.0.V202401.17.000	20251017	redo_copy管理功能增加显示信息RemTopicMessageLossEventCount
LDP1.0.V202401.17.000	20251017	使用指南增加redo合并修改物理库索引的管理功能说明
LDP1.0.V202401.17.000	20251017	使用指南增加修改物理库索引和回库按请求记录时延日志的说明
LDP1.0.V202401.17.000	20251022	补充说明7x24日切查询结果
LDP1.0.V202401.15.001M3	20251010	Remoter插件Backend增加IgnoreNodeNoInMsg配置
LDP1.0.V202401.18.000	20251111	redo_copy 新增全局流控功能
LDP1.0.V202401.17.002	20251114	新增ldp_datamapper相关配置
LDP1.0.V202501.00.000	20251127	回库的GetMonitor接口新增字段RemainRedoCount
LDP1.0.V202401.17.001	20251028	使用指南增加binlog脱敏配置说明
LDP1.0.V202401.17.001	20251030	Remoter插件Backend增加ConnectMode和ReconnectSwitchAddr配置
LDP1.0.V202401.17.001	20251031	bizproc插件GetTransactionNo功能新增网格多分区模式下展示内容;工具ldp_locatetool增加字段-r
LDP1.0.V202401.17.001	20251101	todb的管理接口GetMonitor添加字段GroupNo, PartGroupName和ClusterType字段
LDP1.0.V202401.17.001	20251101	todb的配置字段PartGroupCount的算法发生改变
LDP1.0.V202401.17.001	20251108	修改todb的管理接口GetMonitor,添加字段RemainRedoCount
LDP1.0.V202401.17.001M1	20251112	增加bizproc配置项FilterPartFunctions
LDP1.0.V202401.17.002	20251111	新增table_analyse管理功能及配置DumpDir
LDP1.0.V202401.17.002	20251114	新增ldp_datamapper相关配置
LDP1.0.V202501.00.000	20251204	新增前置流程版本配置项
LDP1.0.V202501.00.000	20251207	优化db_driver和dbclient差异描述
LDP1.0.V202501.00.000	20251207	补充GetMdbTableDefineInfo管理功能

LDP1.0.V202501.00.000	20251208	rcm主题分区地址配置项 Addr支持配置域名
LDP1.0.V202501.00.000	20251209	增加kdpmdb_tools支持指定事务号范围解析redo
LDP1.0.V202501.00.000	20251210	kdp_remoter插件MsgCacheSize配置项增加RPO非0场景的约束说明
LDP1.0.V202501.00.000	20251210	kdp_remoter插件新增配置EnableAppStatusCtrlWhenNoWhileList,kdp_bizproc插件新增配置RoleChangeStateTimeoutMs，用于主备切换优化
LDP1.0.V202501.00.000	20251210	kdp_bizproc新增配置"Sync"用于适配DB模式
LDP1.0.V202501.00.000	20251210	kdp_bizproc管理功能GetTransactionNo相关字段说明修改
LDP1.0.V202501.00.000	20251210	kdp_bizproc管理功能GetGlobalQueueInfo增加线程信息相关内容展示
LDP1.0.V202501.00.000	20251211	增加core解析工具对struct类型的coredump文件解析说明
LDP1.0.V202501.00.000	20251211	增加核心配置EnableTraceRcmRedo，支持备机开关409功能号的timelog
LDP1.0.V202501.00.000	20251211	增加流控管理插件的配置和管理功能说明
LDP1.0.V202501.00.000	20251212	todb、front支持域名的方式连接管理功能补充;增加front_locate插件的管理功能
LDP1.0.V202401.17.001	20251212	增加kdp_locatetool工具-r字段说明
LDP1.0.V202501.00.000	20251212	增加kdp_dbclient支持goldendb相同配置参数说明
LDP1.0.V202501.00.000	20251212	fens支持使用域名的方式连接后端站点，fens的管理功能SiteInfoStat、AccountInfoCollectFinishStat、NodeNoAndSystemNoStat、FrontToBackStat支持显示后端站点的域名
LDP1.0.V202501.00.000	20251212	front支持多上下文配置新增；front压缩消息解压性能优化配置新增；相关管理功能增加BackendInstanceId字段
LDP1.0.V202501.00.000	20251212	todb新增管理接口GetMinHeapInfo
LDP1.0.V202501.00.000	20251205	修改todb的管理接口适配apm
LDP1.0.V202501.00.000	20251216	补充回库大事务需求相关管理功能描述
LDP1.0.V202501.00.000	20251216	补充redo合并管理功能
LDP1.0.V202501.00.000	20251217	补充MDB大事务管理功能
LDP1.0.V202501.00.000	20251217	1.增加并发回放相关的配置、说明、管理功能;2.额外补充MakeTag接口说明
LDP1.0.V202501.00.000	20251218	db_driver支持obclient2.2.x(x≥8)之后版本
LDP1.0.V202501.00.000	20251218	增加大事务相关的管理功能
LDP1.0.V202501.00.000	20251218	增加文件上下场相关的管理功能和配置说明
LDP1.0.V202501.00.000	20251224	给todb的GetClusterInfo接口添加字段
LDP1.0.V202501.00.000	20251227	增加filesync的使用说明
LDP1.0.V202501.00.000	20251230	增加数据组件LoadMode配置说明
LDP1.0.V202501.00.000	20251231	multiproc插件配置更新，补充multiproc使用约束
LDP1.0.V202501.00.000	20250105	补充版本遗留风险
LDP1.0.V202501.00.000	20250106	补充版本遗留风险（multiproc）
LDP1.0.V202501.00.001	20260115	kdp_dbclient插件中对ConnPoolWaitTimeOutMill参数描述修正
LDP1.0.V202501.00.001	20260115	数据组件新增DataSourceAccessPolicy配置项
LDP1.0.V202501.00.001	20260119	数据组件ExportTableToCsvStatus删除入参;管理功能按照最新的规范进行统一修改
LDP1.0.V202501.00.001	20260119	管理功能按照最新的规范进行统一修改;bizproc的GetArbStatus增加身份切换时间
LDP1.0.V202501.00.001	20260121	redo合并管理功能更新
LDP1.0.V202501.00.001	20260121	todb添加配置LinkLatencyMilli
LDP1.0.V202501.00.001	20260121	flowcontrol新增管理功能，原有管理功能修改；mdb_tools支持解析大事务，新增配置项fragment和M
LDP1.0.V202501.00.001	20260122	添加EnableInitDate，重试线程次数配置及相关管理功能等描述
LDP1.0.V202501.00.001	20260122	前置修改管理功能FlowControlStat
LDP1.0.V202501.00.001	20260122	flowcontrol新增配置说明
LDP1.0.V202501.00.001	20260122	Backend增加Onload配置开关；remoter插件增加tcp客户端协议连接控制
LDP1.0.V202501.00.001	20260123	增加timelog性能提升需要相关的配置项

LDP1.0.V202501.00.001	20260202	ldpClusterTool 支持ZK鉴权
LDP1.0.V202501.00.001	20260203	移除此前版本中未及时修复缺陷引入的规格约束
LDP1.0.V202501.00.001	20260203	补充Rpc功能号的约束说明
LDP1.0.V202501.00.001	20260113	补充GetMDBTableIndexInfo管理功能
LDP1.0.V202501.00.001	20260113	redo合并内存不受控制上涨，补充配置约束至使用文档
LDP1.0.V202501.00.001	20260121	redo合并管理功能更新
LDP1.0.V202501.01.000	20260302	补充全局表并发管理功能新增字段
LDP1.0.V202501.01.000	20260303	合并分支multiproc管理功能文档
LDP1.0.V202501.01.000	20260304	补充GetMDBTableIndexInfo管理功能,支持分页功能
LDP1.0.V202501.01.000	20260309	remoter插件增加EnableRcmSendLock配置
LDP1.0.V202501.01.000	20260310	核心插件工作线程流水线线程管理功能，新增字段。
LDP1.0.V202501.01.000	20260311	日志归档相关的配置和管理功能更新。
LDP1.0.V202501.01.000	20260311	todb添加丢弃事务相关的管理功能
LDP1.0.V202501.01.000	20260311	todbV2管理功能GetRedoDealInfo的Req新增近1秒请求平均耗时和近1笔请求处理耗时
LDP1.0.V202501.01.000	20260312	UfimdbsCtrlInfo管理功能新增正在被ClearTable堵塞的事务控制器已经对应被堵塞的时间
LDP1.0.V202501.01.000	20260312	新增表不回库及多账户性能优化相关管理功能
LDP1.0.V202501.01.000	20260312	GetReplayProgress中，展示核心内部相关进展数据
LDP1.0.V202501.01.000	20260313	filesync管理功能，增加文件检查数据
LDP1.0.V202501.01.000	20260316	合并分支00.002修改，mdb_data增加管理功能GetTransactionNo，同时修改bizproc的GetTransactionNo管理功能写redo事务号备注修改
LDP1.0.V202501.01.000	20260318	新增partkeeper分区自动迁移的配置和管理功能；新增partkeeper_client选择订阅的配置；新增partkeeper批量迁移的注意事项
LDP1.0.V202501.01.000	20260319	删除网格模式的bizproc管理功能GetAllNodePartInfo中的MaxTransNo字段
LDP1.0.V202501.02.000	20260331	数据组件新增一个配置项VarcharNullReturnEmpty, 变长字段存储为null值时，mdb返回""
LDP1.0.V202501.02.000	20260403	todb插件支持独立部署，不依赖核心，增加相关配置项和更新相关管理功能
LDP1.0.V202501.02.000	20260407	todb添加管理接口GetMinHeapInsertFailInfo
LDP1.0.V202501.02.000	20260413	db_driver增加libmysqlclient.so的白名单
LDP1.0.V202501.02.000	20260414	新增ldp_bizproc_tool和parse_skipped_msg.sh的说明文档;增加LoadBalance配置解析
LDP1.0.V202501.02.000	20260414	filesync新增终止同步任务描述，补充管理功能任务状态
LDP1.0.V202501.02.000	20260414	新增logproxy性能分析与性能提升相关配置项
LDP1.0.V202501.02.000	20260414	新增MDB分区权限校验相关配置项和管理功能字段；
LDP1.0.V202501.02.000	20260415	新增ldp_bizproc插件使用限制；跳号限制说明修改
LDP1.0.V202501.02.000	20260415	ufimdbs_data新增管理功能ufimdbs_data_GetMonitorField
LDP1.0.V202501.02.000	20260415	ldpmdbs_tools支持解析分区号Action
LDP1.0.V202501.02.000	20260416	修改todb配置ClusterName的描述
LDP1.0.V202501.02.000	20260420	完善multiproc管理功能说明
LDP1.0.V202501.02.000	20260424	使用手册增加db_driver模块的超时时间补充说明
LDP1.0.V202501.02.000	20260429	补充介绍partkeeper的运维指令SendAdminMsg，在什么场景下使用，有什么限制，各种结果的含义

10. 密级

内部